CoE shell model vs. Migkas cluster anisotropy — every step (v3)

This notebook reproduces the v3 term-by-term sheet (`thinshell_termmatch_equations_table_v3.pdf`, Sections 1–4) and the Migkas $\langle H \rangle$ anchor, one step at a time. Each step has a short explanation and then the code that does it.

Rules used throughout

- Gravity only, $\Lambda = 0$, no dark matter. All parameters are free, with no age, boundedness or ΛCDM limits.
- Chris's A: $a_f = |\Delta\vec{V}|/\mathrm{separation}$, used linearly. It is never squared and never turned into D_L^{-2}.
- Symbols: D = CoE–Earth distance; R = the 14.3 Gpc vision sphere; shell edges $R_{\rm in}$, $R_{\rm out}$. Lengths are in Gpc.
- $H_0 = 70$ **is not an input.** It appears in only two labelled places: (i) Section 3's Table 1 masses, which were originally computed with Earth's rate set to 70, and are shown unrescaled with a label; (ii) the unit convention of the LTB integration, which is then rescaled to the Migkas-anchored $\langle H \rangle$.

Sources. Taylor, G. I. 1950, Proc. R. Soc. Lond. A 201, 159; Sedov, L. I. 1959, *Similarity and Dimensional Methods in Mechanics*; Migkas, K. et al. 2020, A&A 636, A15 (arXiv:2004.03305); Migkas, K. et al. 2021, A&A 649, A151 (arXiv:2103.13904); Planck Collaboration 2020, A&A 641, A6 and A1 (Planck 2018 VI and I); Riess, A. G. et al. 2022, ApJL 934, L7 (SH0ES); Watkins, R. et al. 2023, MNRAS 524, 1885 (CosmicFlows-4 bulk flow).

The heavy integrators are Chris's code, used unchanged: `ptshell.py` / `prof_core.py` (Newtonian) and `relfree.py` / `rm_core.py` (LTB). Everything else is written out in the cells.

```
# Step 0 - paths, Chris's integrators (unchanged), plotting
import sys, os, json, csv, numpy as np
import matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as plt
%matplotlib inline
BUB = "/workspace/cosmology/papers/spreadsheet_model/bubble"
THS = "/workspace/cosmology/papers/A_equals_fourier/thinshell"
sys.path.insert(0, BUB); sys.path.insert(0, BUB + "/gr_ltb/refit_mass"); os.chdir(BUB)
import ptshell as S, prof_core as P                # Newtonian shell + core integrator (Chris's code)
np.set_printoptions(precision=4, suppress=True)
TH1 = np.arange(0, 360, 1.0)                       # Galactic longitude grid, 1 deg (the code's 360 directions)
RV = 14.30048287260589                             # R, vision sphere [Gpc]
G = 6.674e-11; c = 299792.458; MPC = 3.0857e19; GPCm = 3.0857e25; MSUN = 1.989e30
def rhoc(H_kms_Mpc):                               # critical density 3H^2/(8 pi G) [kg/m^3]
    h = H_kms_Mpc / MPC; return 3 * h * h / (8 * np.pi * G)
print("R =", round(RV, 2), "Gpc;", len(TH1), "directions")
```

```
R = 14.3 Gpc; 360 directions
```

Step 1 — Model setup (Section 1)

Flow. Sedov–Taylor self-similar expansion (Taylor 1950; Sedov 1959). Every point moves away from the Centre of Expansion (CoE) as

$$\vec{V} = k\,\vec{r} \qquad (\vec{r} = \text{position from the CoE}).$$

Vector subtraction. An observer O (Earth) sees a source P move at

$$\vec{V}_P - \vec{V}_O = k(\vec{r}_P - \vec{r}_O) = k\,\vec{s},$$

so with no mass present, $|\Delta\vec{V}|/s = k$ in every direction and the sky is isotropic. The anisotropy comes only from gravity.

Mass. A point-like core M_c at the CoE, plus a **uniform spherical shell** of ordinary matter with density $\rho_s = \Omega_s \rho_c$ between $R_{\rm in}$ and $R_{\rm out}$ (both centred on the CoE and fixed). Earth sits at distance D from the CoE, and all sources are on the $R = 14.3$ Gpc sphere around Earth, inside the shell. Each fit keeps its own shell rule: Newtonian Ω_s free uses $R_{\rm in} = D - R$, $R_{\rm out} = D + R$; Newtonian $\Omega_s = 0.049$ uses $D - 1.028R$, $D + 1.475R$; LTB uses $D - 1.040R$, $D + 1.171R$.

```
# Step 1a - with no mass, V = k r gives |dV|/s = k in every direction (vector subtraction)
k = 1.0; D = 3.0; rO = np.array([D, 0.0])                    # Earth at distance D from the CoE (units arbitrary)
th = np.radians(TH1); rP = rO + RV / 14.3 * np.c_[np.cos(th), np.sin(th)]
dV = k * rP - k * rO                                         # V_P - V_O
af = np.linalg.norm(dV, axis=1) / np.linalg.norm(rP - rO, axis=1)
print("pure Sedov-Taylor: af/k min, max =", af.min(), af.max())
```

```
pure Sedov-Taylor: af/k min, max = 1.0 1.0
```

```
# Step 1b - the three fitted models (parameters from the fit files; all free, nothing imposed)
T1 = json.load(open(BUB + "/universe_thin.json"))            # Table 1 of universe_bestfit_thinshell.pdf
hug = json.load(open(BUB + "/thin_b_free_hug.json"))         # Newtonian, Omega_s free
omb = json.load(open(BUB + "/universe_newt_omb.json"))["Rs14.3|sine"]   # Newtonian, Omega_s = 0.049
REL = json.load(open(BUB + "/universe_rel_mcfree.json"))     # relativistic LTB, all free
NEWT = {
 "free": dict(DR=hug["DR"], Os=hug["Os"], mcc=hug["mcc"], rin=hug["rin"], rout=hug["rout"], Lc=0.005, mc=0.0, Ob=0.0, l0=hug["l0"], amp=hug["amp"],
phi=hug["phi"]),
 "omb":  dict(DR=omb["D"] / RV, Os=0.049, mcc=omb["mcc"], rin=omb["R_in"] / omb["D"], rout=omb["R_out"] / omb["D"], Lc=0.005, mc=0.0, Ob=0.0,
l0=omb["l0"], amp=omb["amp"], phi=omb["phi"]),
}
for kk, a in NEWT.items():
    D = a["DR"] * RV
    print(f"{kk:5s} D = {D:6.2f} Gpc  D/R = {a['DR']:.3f}  R_in = {a['rin']*D:6.1f}  R_out = {a['rout']*D:6.1f} Gpc  "
          f"Omega_s = {a['Os']:.3f}  Mc/Mcrit(D) = {a['mcc']:.5f}")
print(f"LTB   D = {REL['D']:6.2f} Gpc  D/R = {REL['D']/RV:.3f}  R_in = {REL['R_in']:6.1f}  R_out = {REL['R_out']:6.1f} Gpc  "
      f"Omega_s = 0.049  Mc/Mcrit(D) = {REL['mcc']:.4f}  (k_in {REL['kin']:.3f}, k_out {REL['kout']:.3f})")
```

```
free  D =  93.72 Gpc  D/R = 6.554  R_in =   79.4  R_out =  108.0 Gpc  Omega_s = 1.012  Mc/Mcrit(D) = 0.00265
omb   D =  49.17 Gpc  D/R = 3.439  R_in =   34.5  R_out =   70.3 Gpc  Omega_s = 0.049  Mc/Mcrit(D) = 0.00171
LTB   D =  32.94 Gpc  D/R = 2.303  R_in =   18.1  R_out =   49.7 Gpc  Omega_s = 0.049  Mc/Mcrit(D) = 0.0835  (k_in 1.040, k_out 1.171)
```

Step 2 — Chris's A, exactly as his code computes it

The code integrates the flow from $\tau_i = 0.01$ to today (time unit t_0, length unit D), with Earth and 360 sources on the R sphere (one per degree of Galactic longitude, in the Galactic plane). For each source,

$$a_f(\ell) = \frac{|\vec{V}_P - \vec{V}_O|}{s_{\rm nominal}}, \qquad A(\ell) = \frac{a_f(\ell)}{\langle a_f \rangle_{\rm sky}}.$$

This is compared **linearly** with Migkas's $A/A_{\rm all}$: it is not squared, and not converted to D_L^{-2} or H^2. In code units $1/t_0 = k$, so $\langle a_f \rangle / k$ is dimensionless. The code also returns Earth's radial rate H_E/k (`Ht`) and speed v_E (units D/t_0).

```
# Step 2 - run Chris's integrator (inlined from prof_core.run) and form af = |dV| / separation
def run_newt(a, taui=0.01, nsteps=200, iters=3):
    _, _, m1 = S.build(a)                                  # core + uniform shell enclosed-mass table (ptshell.build)
    fac = max(1.5, 1.02 * a["rout"] / (1 + 1 / a["DR"]))
    q, rr = P.geometry(TH1, a["DR"]); Rb = P.rb_of(a["DR"], fac); Ht = 0.95
    _, MT = P.tables("ptshell", a, Rb)
    for _ in range(iters):                                  # self-consistent Earth rate H_E t0
        mu = m1 * Ht**2 / 2
        X, V, VR = P.integrate(q, mu, P.RG, MT, 0.0, Rb, taui, nsteps)
        Ht = np.hypot(*V[0]) / np.hypot(*X[0])
    dV = V[1:] - V[0]                                       # V_P - V_O
    af = np.hypot(dV[:, 0], dV[:, 1]) / rr                  # / nominal separation (R in units of D)
    return dict(af=af, Ht=Ht, vE=np.hypot(*V[0]), rE=np.hypot(*X[0]), mu=mu, MT=MT, Rb=Rb)
RUN = {kk: run_newt(a) for kk, a in NEWT.items()}
for kk, r in RUN.items():
    A = r["af"] / r["af"].mean()
    print(f"{kk:5s} <af>/k = {r['af'].mean():.4f}  A min/max = {A.min():.3f}/{A.max():.3f}  H_E/k = {r['Ht']:.4f}  v_E = {r['vE']:.4f} D/t0")
```

```
free  <af>/k = 0.9757  A min/max = 0.921/1.200  H_E/k = 0.9902  v_E = 0.8990 D/t0
omb   <af>/k = 1.0119  A min/max = 0.924/1.124  H_E/k = 0.9942  v_E = 0.9353 D/t0
```

Step 3 — Migkas data: $A/A_{\rm all}$ vs. ℓ and the Fourier form A_F

The points are the digitised Migkas et al. 2020 Fig. 6 profile, $A/A_{\rm all}$ against Galactic longitude (71 points). A_F is its 3-harmonic Fourier fit:

$$A_F(\ell) = 0.9965 + 0.0296\cos\ell + 0.0434\sin\ell + 0.0636\cos 2\ell + 0.0366\sin 2\ell - 0.0299\cos 3\ell - 0.0114\sin 3\ell.$$

```
# Step 3 - load the Migkas profile and define A_F
mig = list(csv.DictReader(open("/workspace/cosmology/migkas/fig6_digitized.csv")))
ML = np.array([float(r["l_deg"]) for r in mig]); MA = np.array([float(r["A_over_Aall"]) for r in mig]); IMIG = ML.astype(int)
AFc = {0: (0.9965, 0.0), 1: (0.0296, 0.0434), 2: (0.0636, 0.0366), 3: (-0.0299, -0.0114)}      # (cos, sin) coefficients
def AFfour(l):
    l = np.radians(l); return sum(cs * np.cos(m * l) + sn * np.sin(m * l) for m, (cs, sn) in AFc.items())
def r2(y, m): return 1 - np.sum((y - m) ** 2) / np.sum((y - y.mean()) ** 2)
print(len(ML), "points; l range", ML.min(), "-", ML.max(), "; R2(A_F vs points) =", round(r2(MA, AFfour(ML)), 3))
```

```
71 points; l range 5.0 - 355.0 ; R2(A_F vs points) = 0.915
```

Step 4 — The diagnostic sine term and the headline R^2

The geometry alone gives only $\cos m\psi$ terms about the CoE axis ($\psi = \ell - \ell_0$). A single rotation/tilt factor absorbs the sine parts:

$$A(\ell) = a_f(\ell - \ell_0)\,\big[1 - \mathrm{Amp}\,\sin(\psi + \varphi)\big].$$

It is a diagnostic factor, not a velocity. The headline number is R^2 of this full curve against the 71 Migkas points. Each fit's ℓ_0, Amp and φ are its Table 1 values. The LTB curve comes from the relativistic ray-tracing code (heavy cell below, about 1 s).

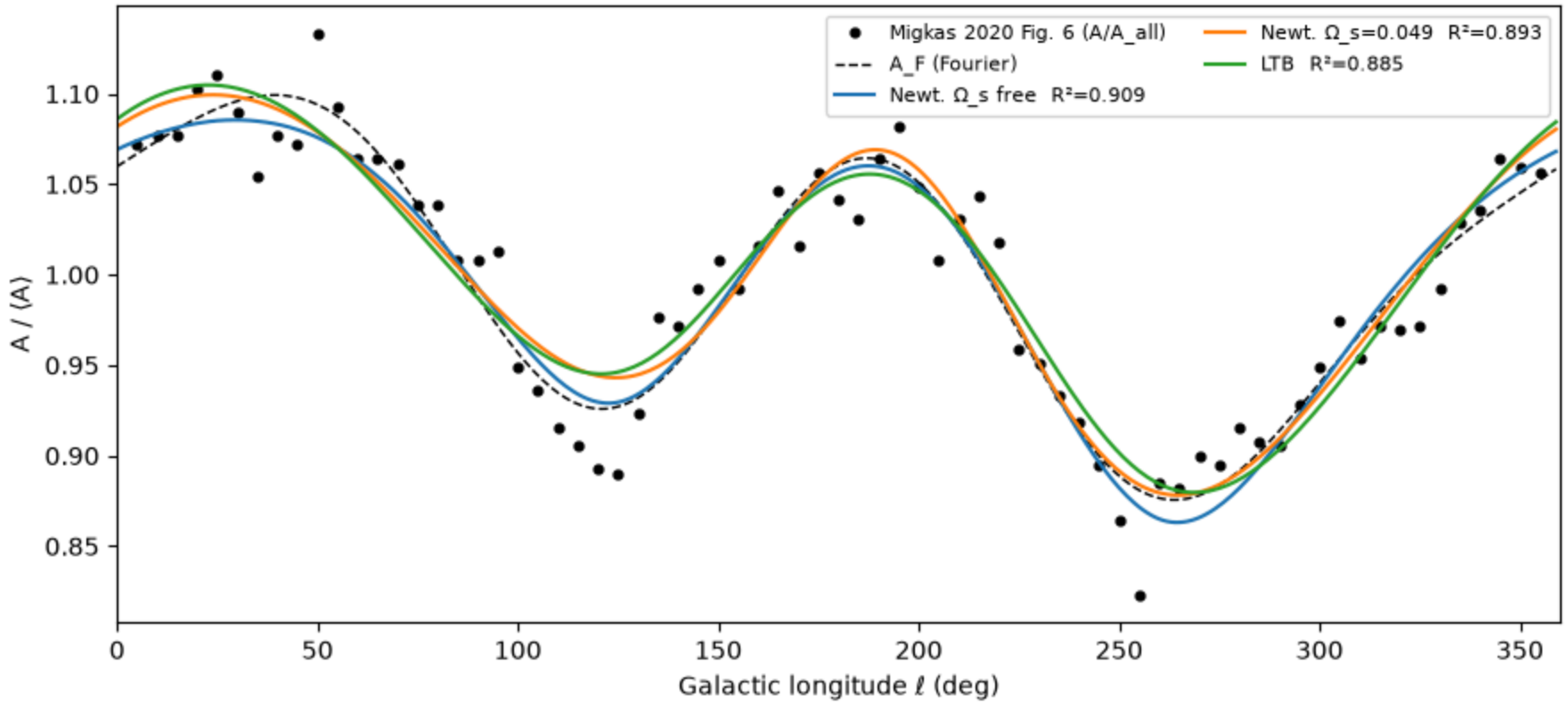
```
# Step 4a - Newtonian: shift to l0, apply the sine factor, R^2 vs Migkas
def shifted(af, l0): return np.interp((TH1 - l0) % 360, np.append(TH1, 360), np.append(af, af[0]))
def A_from(af, th, amp, phi): return (1 - amp * np.sin(np.radians(th) + phi)) * af
CURVE, GEO = {}, {}
for kk, a in NEWT.items():
    g = RUN[kk]["af"] / RUN[kk]["af"].mean(); GEO[kk] = shifted(g, a["l0"])
    CURVE[kk] = A_from(GEO[kk], TH1 - a["l0"], a["amp"], a["phi"])
    print(f"{kk:5s} l0 = {a['l0']:.1f}  Amp = {a['amp']:.4f}  phi = {a['phi']:.3f}   R2 vs Migkas = {r2(MA, CURVE[kk][IMIG]):.3f} "
          f"(geometry only {r2(MA, GEO[kk][IMIG]):.3f})   R2 vs A_F = {r2(AFfour(TH1), CURVE[kk]):.3f}")
```

```
free  l0 = 12.8  Amp = -0.1240  phi = 1.264   R2 vs Migkas = 0.909 (geometry only -0.734)   R2 vs A_F = 0.987
omb   l0 = 13.0  Amp = -0.0622  phi = 0.929   R2 vs Migkas = 0.893 (geometry only 0.497)   R2 vs A_F = 0.977
```

```
# Step 4b - HEAVY CELL (relativistic LTB): Chris's LTB solver + ray tracing at the Table-1 parameters (~1 s)
# Unit note: the LTB code works in units where the 3D-mean local expansion rate is 70 km/s/Mpc (its internal convention);
# only dimensionless ratios are used from it here, and Section 4 rescales to the Migkas-anchored <H>.
import relfree as RF, rm_core as RC
ltb = RF.ev(REL["Mc_kg"], REL["D"], REL["kin"], REL["kout"], full=True)
Ad = np.array(ltb["Ad"])
def ltb_curve(Ad, l0, amp=0.0, phi=0.0):                    # inlined rm_core.curve: 0.5-deg ray table -> 1-deg grid, normalise, sine
    idx = (((TH1 - l0 - 180.0) % 360) / 0.5).astype(int); A = Ad[idx]; A = A / A.mean()
    return A_from(A, TH1 - l0, amp, phi) if amp else A
GEO["rel"] = ltb_curve(Ad, ltb["l0_s"]); CURVE["rel"] = ltb_curve(Ad, ltb["l0_s"], ltb["amp"], ltb["phi"])
json.dump(dict(ltb={k: v for k, v in ltb.items() if k != "Ad"}, A=CURVE["rel"].tolist()),
open("/workspace/cosmology/papers/A_equals_fourier/steps_v3/ltb_cache.json", "w"), default=float)
print(f"LTB   l0 = {ltb['l0_s']:.1f}  Amp = {ltb['amp']:.4f}  phi = {ltb['phi']:.3f}   R2 vs Migkas = {r2(MA, CURVE['rel'][IMIG]):.3f} "
      f"(geometry only {r2(MA, GEO['rel'][IMIG]):.3f})   R2 vs A_F = {r2(AFfour(TH1), CURVE['rel']):.3f}")
print(f"      H_perp = {ltb['H_perp']:.2f}, H_r = {ltb['H_r']:.2f} (fit units), age = {ltb['age_Gyr']:.2f} Gyr (fit units)")
```

```
LTB   l0 = 13.5  Amp = -0.0558  phi = 0.848   R2 vs Migkas = 0.885 (geometry only 0.569)   R2 vs A_F = 0.963
      H_perp = 65.92, H_r = 78.15 (fit units), age = 12.98 Gyr (fit units)
```

```python
# Step 4c - figure: the three full curves vs the Migkas points
fig, ax = plt.subplots(figsize=(9, 4.2))
ax.plot(ML, MA, "ko", ms=3.5, label="Migkas 2020 Fig. 6 (A/A_all)")
ax.plot(TH1, AFfour(TH1), "k--", lw=1, label="A_F (Fourier)")
for kk, col, lab in (("free", "C0", "Newt. Ω_s free"), ("omb", "C1", "Newt. Ω_s=0.049"), ("rel", "C2", "LTB")):
    ax.plot(TH1, CURVE[kk], col, label=f"{lab}  R²={r2(MA, CURVE[kk][IMIG]):.3f}")
ax.set_xlabel("Galactic longitude ℓ (deg)"); ax.set_ylabel("A / ⟨A⟩"); ax.set_xlim(0, 360); ax.legend(fontsize=8, ncol=2); plt.tight_layout();
plt.show()
```



Step 5 — Section 2: term-by-term solution against A_F

To first order in $\varepsilon = R/D$, with $h(q) = V/(kq)$ the radial rate profile, $\beta = h'/h$ and $\gamma = h''/h$ at Earth:

$$a_f^2 \propto 1 + B\mu^2 + \varepsilon(P\mu + Q\mu^3), \quad B = 2\beta + \beta^2,\ P = \beta(3+\beta),\ Q = (1+\beta)(\gamma+\beta) - 2\beta.$$

Taking the square root and projecting onto harmonics gives

$$\frac{a_f}{\langle a_f\rangle} = 1 + a_1\cos\psi + a_2\cos 2\psi + a_3\cos 3\psi, \quad a_2 = \frac{B/4 - B^2/16}{m_0},\ m_0 = 1 + \tfrac{B}{4} - \tfrac{3B^2}{64},$$

$$a_1 = \frac{\varepsilon}{2m_0}\Big[P + \tfrac{3}{4}\big(Q - \tfrac{PB}{2}\big) - \tfrac{5QB}{16}\Big], \qquad a_3 = \frac{\varepsilon}{2m_0}\Big[\tfrac{1}{4}\big(Q - \tfrac{PB}{2}\big) - \tfrac{5QB}{32}\Big].$$

The sine factor adds $C_1 = a_1 - \mathrm{Amp}\sin\varphi(1 + a_2/2)$, $S_1 = -\mathrm{Amp}\cos\varphi(1 - a_2/2)$, $C_3 = a_3 - \frac{\mathrm{Amp}\,a_2}{2}\sin\varphi$, $S_3 = -\frac{\mathrm{Amp}\,a_2}{2}\cos\varphi$. Solve order: 2ℓ phase → ℓ_0; 2ℓ amplitude → β; C_1, S_1, C_3 → Amp, φ, ε (so $D = R/\varepsilon$); S_3 is then a check. Point-core shape: $\gamma = -4\beta$. Mass: $\beta \simeq 3K/(1-K)$, $K = \frac{1}{2}\frac{M_c}{M_{\mathrm{crit}}(D)}(\frac{1}{\tau_i} - 1)$.

```
# Step 5 - sequential (hand) solve with the series, then exact least squares on C_m, S_m
from scipy.optimize import brentq, fsolve, least_squares
TH = np.radians(TH1)
def harm(y, m): return (y.mean(), 0.0) if m == 0 else (2 * np.mean(y * np.cos(m * TH)), 2 * np.mean(y * np.sin(m * TH)))
def rot(cs, sn, m, l0):                 # coefficients about l -> about psi = l - l0
    t = np.radians(m * l0); return cs * np.cos(t) + sn * np.sin(t), -cs * np.sin(t) + sn * np.cos(t)
def series(beta, gam, eps):
    B = 2 * beta + beta**2; Pp = beta * (3 + beta); Q = (1 + beta) * (gam + beta) - 2 * beta
    m0 = 1 + B / 4 - 3 * B**2 / 64; m2 = B / 4 - B**2 / 16
    m1 = eps / 2 * (Pp + 0.75 * (Q - Pp * B / 2) - Q * B / 2 * 10 / 16); m3 = eps / 2 * (0.25 * (Q - Pp * B / 2) - Q * B / 2 * 5 / 16)
    return dict(a1=m1 / m0, a2=m2 / m0, a3=m3 / m0, Q=Q)
C2, S2 = AFc[2]; l0 = (np.degrees(np.arctan2(S2, C2)) / 2) % 180                    # 2-l phase -> axis
Dt = {m: rot(*AFc[m], m, l0) for m in range(4)}; C0 = Dt[0][0]; (C1, S1), (C2r, _), (C3, S3) = Dt[1], Dt[2], Dt[3]
gb = -4.0                                                                           # point core: gamma = -4 beta
beta = brentq(lambda b: series(b, gb * b, 0)["a2"] - C2r / C0, 1e-4, 1.0)
s1 = series(beta, gb * beta, 1.0); a2, g1, g3 = s1["a2"], s1["a1"], s1["a3"]
def F(x):
    amp, phi, eps = x
    return [C0 * (eps * g1 - amp * np.sin(phi) * (1 + a2 / 2)) - C1, -C0 * amp * np.cos(phi) * (1 - a2 / 2) - S1, C0 * (eps * g3 - amp * a2 / 2 *
np.sin(phi)) - C3]
amp, phi, eps = fsolve(F, [-0.06, 0.9, 0.3])
S3_pred = -C0 * amp * a2 / 2 * np.cos(phi); N = C0 / (1 - amp * eps * g1 * np.sin(phi) / 2)
K = beta / (3 + beta); mcc_lin = 2 * K / (1 / 0.01 - 1)
print(f"2l phase  : l0 = {l0:.2f} deg      2l amplitude: C2 = {np.hypot(*AFc[2]):.4f} -> beta = {beta:.3f}")
print(f"1l, 3l cos: Amp = {amp:.3f}, phi = {phi % (2*np.pi):.2f}, eps = {eps:.3f} -> D = R/eps = {RV/eps:.0f} Gpc, eps*Q = {eps*s1['Q']:.3f}")
print(f"3l sin    : predicted S3 = {S3_pred:.4f} vs data {S3:.4f} (not produced)     m=0: N = {N:.4f}")
print(f"mass      : K = {K:.4f} -> Mc/Mcrit(D) = {mcc_lin:.4f} (linear)")
# exact least squares: exact radial field with h(q) = 1 + beta (q-1) + (gamma/2)(q-1)^2, gamma = -4 beta
def A_radial(hfun, eps):
    x = 1 + eps * np.cos(TH); y = eps * np.sin(TH); qs = np.hypot(x, y); hs = hfun(qs); h1 = hfun(np.array([1.0]))[0]
    af = np.hypot(hs * x - h1, hs * y) / eps; return af / af.mean()
def model(p):
    l0_, b, e, am, ph = p; g = A_radial(lambda q: 1 + b * (q - 1) + 0.5 * gb * b * (q - 1)**2, e); return A_from(shifted(g, l0_), TH1 - l0_, am, ph)
def res(p):
    A = model(p); out = []
    for m in range(4):
        a_, b_ = harm(A, m); cs, sn = AFc[m]; out += [a_ - cs, b_ - sn] if m else [a_ - cs]
```

```
    return np.array(out)
ls = least_squares(res, [15, 0.16, 0.3, -0.05, 0.9], bounds=([-30, 0, 0.02, -0.3, -10], [60, 1, 0.95, 0.3, 10])).x
print(f"exact LSQ : l0 = {ls[0]:.1f}, beta = {ls[1]:.3f}, Amp = {ls[3]:.3f}, phi = {ls[4]:.2f}, D = {RV/ls[2]:.0f} Gpc")
```

```
2l phase  : l0 = 14.96 deg      2l amplitude: C2 = 0.0734 -> beta = 0.161
1l, 3l cos: Amp = -0.068, phi = 1.02, eps = 0.348 -> D = R/eps = 41 Gpc, eps*Q = -0.308
3l sin    : predicted S3 = 0.0013 vs data 0.0130 (not produced)     m=0: N = 0.9971
mass      : K = 0.0511 -> Mc/Mcrit(D) = 0.0010 (linear)
exact LSQ : l0 = 12.8, beta = 0.210, Amp = -0.072, phi = 1.03, D = 47 Gpc
```

Step 6 — Section 3: input parameters, term-by-term solve vs. Table 1

For each Newtonian fit, the unknowns $M_c/M_{\rm crit}(D)$, D, ℓ_0, Amp and φ are solved from C_m, S_m ($m = 1$–3) of A_F by least squares. Each solve uses the code's **exact** $h(q)$ from the integrator and keeps that fit's shell rule. The Table 1 columns are the full point fits; for LTB only the Table 1 column is shown.

Label: the masses in this section were computed with Earth's rate set to $H_E = 70$ km s^{-1} Mpc^{-1}, as in the original sheet ($\rho_s = \Omega_s \rho_c(70)$, $M_{\rm crit} = \frac{4\pi}{3}\rho_c(70) D^3$). They are shown unrescaled; Section 4 (Step 8) derives the masses from the anchored $\langle H \rangle$.

```
# Step 6a - term-by-term solve (exact h(q)) for the two Newtonian cases  [masses at H_E = 70: LABELLED, original-sheet convention]
RHOC70 = rhoc(70.0)                                          # <-- only for the Section 3 mass columns (original convention)
def af_code(D, mcc, Os, kin, kout):
    a = dict(DR=D / RV, Os=Os, mcc=mcc, rin=1 - kin * RV / D, rout=1 + kout * RV / D, Lc=0.005, mc=0.0, Ob=0.0)
    if not (0.01 < a["rin"] < 1): return None
    af = run_newt(a)["af"]; return af / af.mean() if np.all(np.isfinite(af)) else None
def hres(A):
    out = []
    for m in (1, 2, 3):
        a_, b_ = harm(A, m); cs, sn = AFc[m]; out += [a_ - cs, b_ - sn]
    return np.array(out)
def masses(D, mcc, Os, kin, kout, rho_c):
    Dm = D * GPCm; Mc = mcc * 4 * np.pi / 3 * rho_c * Dm**3; rhos = Os * rho_c; Rin, Rout = D - kin * RV, D + kout * RV
    Msh = lambda Rr: 4 * np.pi / 3 * rhos * ((min(max(Rr, Rin), Rout) * GPCm)**3 - (Rin * GPCm)**3)
    return dict(Mc=Mc, Mtot=Mc + Msh(D), Mshtot=Msh(Rout), Mvis=4 * np.pi / 3 * rhos * (RV * GPCm)**3, R_in=Rin, R_out=Rout)
def tbt(Os, kin, kout, x0):
    def res6(x):
        lm, lD, l0_, am, ph = x; A = af_code(10**lD, 10**lm, Os, kin, kout)
        return np.ones(6) if A is None else hres(A_from(shifted(A, l0_), TH1 - l0_, am, ph))
    x = least_squares(res6, x0, diff_step=1e-3, x_scale=[0.3, 0.1, 3, 0.03, 0.2]).x
    lm, lD, l0_, am, ph = x; D = 10**lD; A = af_code(D, 10**lm, Os, kin, kout); A = A_from(shifted(A, l0_), TH1 - l0_, am, ph)
    if am > 0: am, ph = -am, ph + np.pi
    return dict(D=D, mcc=10**lm, Os=Os, l0=l0_ % 360, amp=am, phi=ph % (2 * np.pi), R2=r2(MA, A[IMIG]), R2F=r2(AFfour(TH1), A), **masses(D, 10**lm,
Os, kin, kout, RHOC70))
TBT = {"free": tbt(T1["free"]["Os"], 1.0, 1.0, [np.log10(T1["free"]["mcc"]), np.log10(T1["free"]["D"]), 13, -0.12, 1.26]),
       "omb":  tbt(0.049, omb["kin"], omb["kout"], [np.log10(omb["mcc"]), np.log10(omb["D"]), 13, -0.06, 0.93])}
for kk, v in TBT.items(): print(kk, {q: float("%.4g" % v[q]) for q in ("D", "mcc", "l0", "amp", "phi", "R2", "R2F", "Mc", "Mtot")})
```

```
free {'D': 92.58, 'mcc': 0.00265, 'l0': 13.19, 'amp': -0.1246, 'phi': 1.267, 'R2': 0.9086, 'R2F': 0.9868, 'Mc': 2.382e+54, 'Mtot': 3.623e+56}
omb {'D': 37.03, 'mcc': 0.001637, 'l0': 12.75, 'amp': -0.06372, 'phi': 0.9413, 'R2': 0.8808, 'R2F': 0.9698, 'Mc': 9.415e+52, 'Mtot': 2.294e+54}
```

```
# Step 6b - Section 3 table (Table-1 columns from the fit files; masses at H_E = 70 as originally computed - NOT rescaled)
def col_t1(t):
    return dict(D=t["D"], R_in=t["Rin"], R_out=t["Rout"], Mc=t["Mc"], mcc=t["mcc"], Mtot=t["Mtot"], Mshtot=t["Mshtot"], Mvis=t["Mvis"],
                Os=t["Os"], l0=t["l0"], amp=t["amp"], phi=t["phi"], R2=t["R2"], R2F=t["R2F"])
COLS = [("Newt Ω_s free, term-by-term", TBT["free"]), ("Newt Ω_s free, Table 1", col_t1(T1["free"])),
        ("Newt Ω_s=0.049, term-by-term", TBT["omb"]), ("Newt Ω_s=0.049, Table 1", col_t1(T1["newt_omb"])), ("LTB, Table 1", col_t1(T1["rel"]))]
rows = [("D/R", lambda v: f"{v['D']/RV:.3f}"), ("D [Gpc]", lambda v: f"{v['D']:.1f}"), ("R_in [Gpc]", lambda v: f"{v['R_in']:.1f}"),
        ("R_out [Gpc]", lambda v: f"{v['R_out']:.1f}"), ("Mc [kg] (H_E=70)", lambda v: f"{v['Mc']:.2e}"), ("Mc [M_sun] (H_E=70)", lambda v: f"
{v['Mc']/MSUN:.2e}"),
        ("Mc/Mcrit(D)", lambda v: f"{v['mcc']:.3g}"), ("M(<D) total [kg] (H_E=70)", lambda v: f"{v['Mtot']:.2e}"),
        ("M_shell total [kg] (H_E=70)", lambda v: f"{v['Mshtot']:.2e}"), ("M within R [kg] (H_E=70)", lambda v: f"{v['Mvis']:.2e}"),
        ("Ω_s", lambda v: f"{v['Os']:.3f}"), ("ℓ0 [deg]", lambda v: f"{v['l0']:.1f}"), ("Amp", lambda v: f"{v['amp']:.4f}"),
        ("φ [rad]", lambda v: f"{v['phi']:.3f}"), ("R² vs Migkas", lambda v: f"{v['R2']:.3f}"), ("R² vs A_F", lambda v: f"{v['R2F']:.3f}")]
from IPython.display import Markdown, display
tab = "| Quantity | " + " | ".join(n for n, _ in COLS) + " |\n|" + "---|" * (len(COLS) + 1) + "\n"
tab += "\n".join("| " + lab + " | " + " | ".join(f(v) for _, v in COLS) + " |" for lab, f in rows)
display(Markdown(tab))
print("\nThe relativistic term-by-term column (D not fixed; R2 0.824/0.899) is left out, as in the v3 sheet; it can be restored.")
```

Quantity	Newt Ω_s free, term-by-term	Newt Ω_s free, Table 1	Newt Ω_s=0.049, term-by-term	Newt Ω_s=0.049, Table 1	LTB, Table 1
D/R	6.474	6.554	2.589	3.439	2.303
D [Gpc]	92.6	93.7	37.0	49.2	32.9
R_in [Gpc]	78.3	79.4	22.3	34.5	18.1
R_out [Gpc]	106.9	108.0	58.1	70.3	49.7
Mc [kg] (H_E=70)	2.38e+54	2.47e+54	9.41e+52	2.31e+53	3.38e+54
Mc [M_sun] (H_E=70)	1.20e+24	1.24e+24	4.73e+22	1.16e+23	1.70e+24
Mc/Mcrit(D)	0.00265	0.00265	0.00164	0.00171	0.0835
M(<D) total [kg] (H_E=70)	3.62e+56	3.72e+56	2.29e+54	4.56e+54	5.04e+54
M_shell total [kg] (H_E=70)	8.50e+56	8.71e+56	1.03e+55	1.70e+55	6.48e+54
M within R [kg] (H_E=70)	3.35e+54	3.35e+54	1.62e+53	1.62e+53	1.62e+53
Ω_s	1.012	1.012	0.049	0.049	0.049
ℓ0 [deg]	13.2	12.8	12.8	13.0	13.5
Amp	-0.1246	-0.1240	-0.0637	-0.0622	-0.0558
ϕ [rad]	1.267	1.264	0.941	0.929	0.848
R^2 vs Migkas	0.909	0.909	0.881	0.893	0.885
R^2 vs A_F	0.987	0.987	0.970	0.977	0.963

```
The relativistic term-by-term column (D not fixed; R2 0.824/0.899) is left out, as in the v3 sheet; it can be restored.
```

Step 7 — Anchoring $\langle H \rangle$ on Migkas's own H_0 values

The fits fix only the **shape** $A(\ell)/\langle A \rangle$. The absolute sky-mean rate comes from Migkas et al.'s own H_0 numbers by fitting

$$H(\ell) = \langle H \rangle \, \frac{A(\ell)}{\langle A \rangle}$$

by weighted least squares, $\langle H \rangle = \sum_i w_i H_i m_i / \sum_i w_i m_i^2$ with $w_i = 1/\sigma_i^2$. Here m_i is the model at the region's cone centre, or the model mean outside the cone for "rest of sky" (75° cones for LX–T and the joint result, 60° for YSZ–T, 90° for "opposite hemisphere"). Only ℓ is used because the model is planar. The values below were checked against the arXiv full texts (2004.03305 §7, §8.1–8.2; 2103.13904 §6.1–6.4, App. B). The **representative value** is the 2021 all-combined joint result.

Caveat: Migkas's absolute H_0 values rest on their fiducial calibration ($H_0 = 70$ assumed when calibrating the scaling relations; M21 §3), so the anchor inherits that calibration. The H0 maps are published only as figures, not tables, so they are not used.

```
# Step 7 - all 11 Migkas pairs -> <H> for each fit
CH = [  # (label, [(H0, sigma, kind, l, cone)])   kind: 'pt' = cone centre, 'out' = outside cone / opposite hemisphere
 ("M20 our sample LX-T, max/min",           [(75.17, 1.81, "pt", 17, 0), (66.20, 1.72, "pt", 274, 0)]),
 ("M20 ACC LX-T, max/min",                  [(78.76, 4.15, "pt", 77, 0), (58.12, 2.68, "pt", 317, 0)]),
 ("M20 XCS-DR1 LX-T, max/min",              [(77.91, 2.20, "pt", 31, 0), (63.56, 2.32, "pt", 281, 0)]),
 ("M20 joint (3 samples), max/min",         [(76.64, 1.41, "pt", 34, 0), (65.20, 1.48, "pt", 303, 0)]),
 ("M21 our LX-T, region/rest",              [(66.0, 1.7, "pt", 274, 0), (72.1, 1.4, "out", 274, 75)]),
 ("M21 ours+ACC LX-T, region/rest",         [(66.2, 1.6, "pt", 284, 0), (72.7, 1.5, "out", 284, 75)]),
 ("M21 our YSZ-T, region/rest",             [(62.3, 2.2, "pt", 268, 0), (72.1, 0.9, "out", 268, 60)]),
 ("M21 LBCG-T, region/opp. hemisphere",     [(66.2, 2.5, "pt", 257, 0), (72.6, 2.3, "out", 257, 90)]),
 ("M21 ACC LX-T, region/rest",              [(61.3, 4.2, "pt", 318, 0), (72.2, 2.4, "out", 318, 75)]),
 ("M21 ACC YSZ-T, region/opp. hemisphere",  [(60.6, 3.6, "pt", 311, 0), (73.4, 1.9, "out", 311, 90)]),
 ("M21 ALL COMBINED (joint), region/rest",  [(66.5, 1.0, "pt", 273, 0), (72.8, 0.6, "out", 273, 75)]),
]
SHAPE = {kk: CURVE[kk] / CURVE[kk].mean() for kk in ("free", "omb", "rel")}
def m_of(s, kind, l, th):
    if kind == "pt": return s[int(round(l)) % 360]
    dl = np.abs((TH1 - l + 180) % 360 - 180); return s[dl > th].mean()
HM = {kk: [] for kk in SHAPE}
print("choice".ljust(42) + "Newt free    Newt 0.049   LTB")
for lab, pts in CH:
    line = lab.ljust(42)
    for kk, s in SHAPE.items():
        m = np.array([m_of(s, kd, l, th) for _, _, kd, l, th in pts]); H = np.array([p[0] for p in pts]); wt = 1 / np.array([p[1] for p in pts])**2
        Hm = (wt * H * m).sum() / (wt * m * m).sum(); err = 1 / np.sqrt((wt * m * m).sum()); HM[kk].append((Hm, err)); line += f"{Hm:6.2f}±{err:.2f}
"
    print(line)
ANCH = {kk: dict(H=v[-1][0], err=v[-1][1], mean=np.mean([x[0] for x in v]), sd=np.std([x[0] for x in v], ddof=1)) for kk, v in HM.items()}
```

```
print("\nrepresentative (M21 joint):", {kk: f"{a['H']:.1f}±{a['err']:.1f}" for kk, a in ANCH.items()})
print("mean ± s.d. of all 11     :", {kk: f"{a['mean']:.1f}±{a['sd']:.1f}" for kk, a in ANCH.items()})
```

```
choice                                  Newt free    Newt 0.049   LTB
M20 our sample LX-T, max/min             71.84±1.27   71.16±1.26   71.01±1.26
M20 ACC LX-T, max/min                    64.03±2.24   64.73±2.26   65.11±2.28
M20 XCS-DR1 LX-T, max/min                71.46±1.60   71.05±1.59   71.00±1.59
M20 joint (3 samples), max/min           69.55±0.99   69.61±0.99   69.68±1.00
M21 our LX-T, region/rest                71.78±1.11   71.53±1.11   71.58±1.11
M21 ours+ACC LX-T, region/rest           71.94±1.13   71.86±1.13   72.04±1.13
M21 our YSZ-T, region/rest               70.07±0.82   69.94±0.82   69.96±0.82
M21 LBCG-T, region/opp. hemisphere       72.72±1.77   72.40±1.76   72.13±1.75
M21 ACC LX-T, region/rest                69.06±2.07   69.32±2.08   69.44±2.08
M21 ACC YSZ-T, region/opp. hemisphere    70.24±1.67   70.49±1.68   70.70±1.68
M21 ALL COMBINED (joint), region/rest    71.79±0.52   71.61±0.52   71.63±0.52

representative (M21 joint): {'free': '71.8±0.5', 'omb': '71.6±0.5', 'rel': '71.6±0.5'}
mean ± s.d. of all 11     : {'free': '70.4±2.4', 'omb': '70.3±2.1', 'rel': '70.4±2.0'}
```

Step 8 — Section 4: everything that follows from $\langle H \rangle$

Newtonian. $\langle H \rangle$ is the sky mean of the full curve $a_f[1 - \mathrm{Amp}\sin(\psi + \varphi)]$ in code units ($1/t_0 = k$), so $\langle H \rangle / k$ is a pure number and

$$k = \frac{\langle H \rangle}{\langle H \rangle / k}, \quad t_0 = k^{-1}, \quad H_E = (H_E/k)\,k, \quad \rho_s = \Omega_s \rho_c(H_E), \quad M_{\mathrm{crit}} = \tfrac{4\pi}{3}\rho_c(H_E) D^3,$$

$$V_O = v_E\, k\, D, \qquad \text{bulk flow} = \left|\tfrac{\epsilon_b^2}{10}(h'' + 4h')\right| V_O,\ \epsilon_b = r_b/D,\ r_b = 150\,h^{-1}\,\mathrm{Mpc} \text{ with } h = \langle H \rangle / 100.$$

LTB. $\langle H \rangle$ is the in-plane mean of the local rate, $(H_\perp + H_r)/2$. From the LTB run, $\langle H \rangle t_0 = 0.956$ (dimensionless). Masses are rescaled from the fit's unit convention (3D-mean local rate 70) by $(\langle H \rangle / 72.04)^2$. This rescaling is approximate in GR, and the bulk flow is not computed.

Accepted values: Planck 2018 VI ($H_0 = 67.36 \pm 0.54$, $\Omega_b h^2 = 0.02237$, $\Omega_m = 0.3153$, age 13.797 Gyr); SH0ES (73.04 ± 1.04); Planck 2018 I (CMB dipole 369.82 ± 0.11 km/s; Local Group 620 ± 15 km/s); Watkins 2023 (bulk flow 395 ± 29 km/s at $150\,h^{-1}$ Mpc).

```
# Step 8a - accepted values (derived masses use Planck's own H0 = 67.36, not 70)
Pl = dict(H0=67.36, Obh2=0.02237, Om=0.3153, age=13.797); VOL = 4 * np.pi / 3 * (RV * GPCm)**3
Ob = Pl["Obh2"] / (Pl["H0"] / 100)**2; rcP = rhoc(Pl["H0"])
ACC = dict(Ob=Ob, rho_b=Ob * rcP, rhoc=rcP, Mb=Ob * rcP * VOL, Mm=Pl["Om"] * rcP * VOL, vcmb=369.82, vlg=620.0, bulk=395.0)
print({k: float("%.4g" % v) for k, v in ACC.items()})
```

```
{'Ob': 0.0493, 'rho_b': 4.202e-28, 'rhoc': 8.523e-27, 'Mb': 1.512e+53, 'Mm': 9.672e+53, 'vcmb': 369.8, 'vlg': 620.0, 'bulk': 395.0}
```

```
# Step 8b - model columns from the anchored <H> (no H0 = 70 input for the Newtonian columns)
def radial_h(a, r, qs):                                  # h(q) = V(q)/q along the CoE-Earth axis, code units
    q = np.vstack([[1.0, 0.0], np.c_[qs, np.zeros_like(qs)]])
    X, V, VR = P.integrate(q, r["mu"], P.RG, r["MT"], 0.0, r["Rb"], 0.01, 200); return V[1:, 0] / qs
```

```
SEC4 = {}
for kk, a in NEWT.items():
    r = RUN[kk]
    Hk = A_from(shifted(r["af"], a["l0"]), TH1 - a["l0"], a["amp"], a["phi"]).mean()   # <H>/k: sky mean of af x sine factor, code units (1/t0 = k)
    H = ANCH[kk]["H"]; k = H / Hk; HE = r["Ht"] * k; D = a["DR"] * RV
    kin, kout = (1 - a["rin"]) * a["DR"], (a["rout"] - 1) * a["DR"]           # shell edges in units of R: R_in = D - kin R, R_out = D + kout R
    ms = masses(D, a["mcc"], a["Os"], kin, kout, rhoc(HE))                    # rho_c at Earth's rate H_E (the code's Omega reference)
    qs = np.linspace(0.95, 1.05, 41); h = radial_h(a, r, qs); cf = np.polyfit(qs - 1, h, 6); h1 = np.polyval(cf, 0)
    hp = np.polyval(np.polyder(cf), 0) / h1; hpp = np.polyval(np.polyder(cf, 2), 0) / h1
    VO = r["vE"] * k * D * 1000; eb = 0.150 / (H / 100) / D
    SEC4[kk] = dict(H=H, Hk=Hk, k=k, t0=977.8 / k, rho_s=a["Os"] * rhoc(HE), Mvis=ms["Mvis"], Mc=ms["Mc"], Mtot=ms["Mtot"], Os=a["Os"], Os_H=a["Os"] *
(HE / H)**2,
                    rhoc=rhoc(H), VO=VO, bulk=abs(eb**2 / 10 * (hpp + 4 * hp)) * VO)
# LTB: dimensionless <H> t0 from the run; masses from the fit's unit convention (3D local mean = 70) rescaled by (<H>/<H>_fit)^2
Hloc_fit = (ltb["H_perp"] + ltb["H_r"]) / 2; Ht0 = Hloc_fit * ltb["age_Gyr"] / 977.8; H = ANCH["rel"]["H"]; g = H / Hloc_fit; t3 = T1["rel"]
SEC4["rel"] = dict(H=H, Hk=Ht0, k=H / Ht0, t0=977.8 * Ht0 / H, rho_s=0.049 * rhoc(70.0) * g**2, Mvis=t3["Mvis"] * g**2, Mc=t3["Mc"] * g**2,
Mtot=t3["Mtot"] * g**2,
                   Os=0.049, Os_H=0.049 * (70.0 / Hloc_fit)**2,   # 70 = LTB fit's unit convention (3D local mean), labelled
                   rhoc=rhoc(H), VO=t3["vE"] * c * g, bulk=None)
print(f"LTB: <H>_fit = {Hloc_fit:.2f} (fit units), <H> t0 = {Ht0:.3f}, rescale g = {g:.4f}")
```

```
LTB: <H>_fit = 72.04 (fit units), <H> t0 = 0.956, rescale g = 0.9943
```

```
# Step 8c - Section 4 table
F4 = [("(H) [km/s/Mpc]", "67.36±0.54 (CMB); 73.04±1.04 (local)", lambda v: f"{v['H']:.1f}  (= {v['Hk']:.3f} k)" if v is not SEC4['rel'] else f"
{v['H']:.1f}  (= {v['Hk']:.3f}/t0)"),
      ("k (LTB: 1/t0) [km/s/Mpc]", "-", lambda v: f"{v['k']:.1f}"),
      ("Age / k^-1 = t0 [Gyr]", "13.797±0.023", lambda v: f"{v['t0']:.2f}"),
      ("Ordinary mass within R [kg]", f"{ACC['Mb']:.2e}", lambda v: f"{v['Mvis']:.2e}"),
      ("Total matter within R [kg]", f"{ACC['Mm']:.2e}", lambda v: "same (no DM)"),
      ("rho_b [kg/m^3]", f"{ACC['rho_b']:.2e}", lambda v: f"{v['rho_s']:.2e}"),
      ("Omega_b (= Omega_m in model)", f"{ACC['Ob']:.4f}; Om 0.3153", lambda v: f"{v['Os']:.3f} ({v['Os_H']:.3f} vs ⟨H⟩)"),
      ("rho_c at ⟨H⟩ [kg/m^3]", f"{ACC['rhoc']:.2e} (67.36)", lambda v: f"{v['rhoc']:.2e}"),
      ("Earth speed vs CMB V_O", "369.82±0.11 km/s", lambda v: f"{v['VO']:.2e} km/s = {v['VO']/c:.1f}c"),
      ("V_O / CMB dipole", "1", lambda v: f"{v['VO']/369.82:,.0f}"),
      ("Local Group vs CMB", "620±15 km/s", lambda v: "not modelled"),
      ("Bulk flow, 150/h Mpc [km/s]", "395±29", lambda v: "not computed" if v["bulk"] is None else f"{v['bulk']:.2g}"),
      ("Core mass Mc [kg] (x M_b)", "-", lambda v: f"{v['Mc']:.2e} ({v['Mc']/ACC['Mb']:.1f}x)"),
      ("Mass inside D [kg] (x M_b)", "-", lambda v: f"{v['Mtot']:.2e} ({v['Mtot']/ACC['Mb']:,.0f}x)")]
from IPython.display import Markdown, display
tab = "| Quantity | Accepted | Newt Ω_s free (0.909) | Newt Ω_s=0.049 (0.893) | LTB (0.885) |\n|---|---|---|---|---|\n"
tab += "\n".join("| " + lab + " | " + accv + " | " + " | ".join(f(SEC4[kk]) for kk in ("free", "omb", "rel")) + " |" for lab, accv, f in F4)
display(Markdown(tab))
json.dump(dict(anchor=ANCH, sec4=SEC4, tbt=TBT, R2={kk: r2(MA, CURVE[kk][IMIG]) for kk in CURVE}),
open("/workspace/cosmology/papers/A_equals_fourier/steps_v3/notebook_results.json", "w"), indent=1, default=float)
```

Quantity	Accepted	Newt Ω_s free (0.909)	Newt Ω_s=0.049 (0.893)	LTB (0.885)
⟨H⟩ [km/s/Mpc]	67.36±0.54 (CMB); 73.04±1.04 (local)	71.8 (= 0.971 k)	71.6 (= 1.012 k)	71.6 (= 0.956/t0)
k (LTB: 1/t0) [km/s/Mpc]	-	74.0	70.8	74.9
Age / k^-1 = t0 [Gyr]	13.797±0.023	13.22	13.81	13.05
Ordinary mass within R [kg]	1.51e+53	3.67e+54	1.64e+53	1.60e+53
Total matter within R [kg]	9.67e+53	same (no DM)	same (no DM)	same (no DM)
rho_b [kg/m^3]	4.20e-28	1.02e-26	4.56e-28	4.46e-28
Omega_b (= Omega_m in model)	0.0493; Om 0.3153	1.012 (1.053 vs ⟨H⟩)	0.049 (0.047 vs ⟨H⟩)	0.049 (0.046 vs ⟨H⟩)
rho_c at ⟨H⟩ [kg/m^3]	8.52e-27 (67.36)	9.68e-27	9.63e-27	9.64e-27
Earth speed vs CMB V_O	369.82±0.11 km/s	6.23e+06 km/s = 20.8c	3.26e+06 km/s = 10.9c	2.29e+06 km/s = 7.6c
V_O / CMB dipole	1	16,848	8,804	6,199
Local Group vs CMB	620±15 km/s	not modelled	not modelled	not modelled
Bulk flow, 150/h Mpc [km/s]	395±29	5.7	0.32	not computed
Core mass Mc [kg] (x M_b)	-	2.71e+54 (17.9x)	2.33e+53 (1.5x)	3.34e+54 (22.1x)
Mass inside D [kg] (x M_b)	-	4.07e+56 (2,692x)	4.61e+54 (30x)	4.98e+54 (33x)

Summary

- Headline fits (full curve with sine, vs 71 Migkas points): R^2 = 0.909 (Newtonian, Ω_s free), 0.893 (Newtonian, $\Omega_s = 0.049$), 0.885 (LTB).
- Migkas-anchored sky-mean rate (2021 joint result): $\langle H \rangle \approx 71.8/71.6/71.6$ km s^{-1} Mpc^{-1}; the mean over all 11 Migkas pairs is $\approx 70.4 \pm 2$.
- With that anchor, $t_0 = k^{-1} \approx 13.2/13.8/13.05$ Gyr. Earth's speed about the CoE is $V_O \approx 21c/11c/7.6c$, thousands of times the CMB dipole speed. This is the model's main tension.
- The absolute anchor inherits Migkas's fiducial calibration.

Part B — Loops and maps

Part B redraws every loop and map figure made earlier in the project, using **the latest script for each figure, run verbatim**. The only edit to each script is where it writes its output: the images go to `steps_v3/figs/` so the approved originals on disk are not overwritten. That edit is

listed in each step. After each script runs, the new image is shown next to the approved reference image and compared pixel by pixel.

Shared geometry. With a pure CoE flow, A is axisymmetric about the CoE axis, so any value read at angle θ from the anti-CoE direction holds on the whole small circle (cone) of half-angle θ about that direction. The model $A(\ell)$ after the 2ℓ step has two maxima ($\theta \approx 30^\circ$, 185°) and two minima ($\theta \approx 115^\circ$, 265°). Each one becomes a loop: a transparent band of $\pm 9^\circ$ (18° end to end, 5% of 360°) about the centre θ, plus one solid mid-line. The axis can be tilted to latitude b_0 and rolled about itself. Points on a loop are $\hat{n} = \cos\theta\,\hat{a} + \sin\theta(\cos\psi\,\hat{e}_1 + \sin\psi\,\hat{e}_2)$, with $\hat{a}$ the anti-CoE unit vector and $\hat{e}_{1,2}$ perpendicular to it. They are converted to Galactic (ℓ, b) and drawn on a Mollweide projection (equal-area, Snyder 1987, *Map Projections — A Working Manual*, USGS PP 1395) with ℓ increasing to the left, as in Migkas Fig. 8.

Data. Migkas et al. 2020 (A&A 636, A15; arXiv:2004.03305): Table 2 directions of the A extrema, and the Fig. 8 $\theta = 75^\circ$ $A/A_{\rm all}$ map (background image loaded from `/workspace/cosmology/map_compare/FULL_migkas_theta75.png`). Markers: MAX at $(17^\circ, -9^\circ)$ and $(185^\circ, 0^\circ)$; MIN at $(115^\circ, 0^\circ)$ and $(274^\circ, -22^\circ)$.

```
# Part B setup - run each figure script verbatim in its own namespace; outputs go to steps_v3/figs
import os, sys, shlex, matplotlib, numpy as np
from pathlib import Path
from PIL import Image
from IPython.core.magic import register_cell_magic
from IPython.display import display
FIGS = Path("/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs"); FIGS.mkdir(exist_ok=True)
for d in ("journal_rows", "stack_rows"): (FIGS / d).mkdir(exist_ok=True)
FIGDIR = "/workspace/cosmology/figures"
if FIGDIR not in sys.path: sys.path.insert(0, FIGDIR)

@register_cell_magic
def runscript(line, cell):
    """%%runscript <script name> [--cwd DIR] [--argv ARGS...]: exec the cell as that script (__name__ == '__main__'),
    __file__ placed in steps_v3/figs (so OUT_DIR = figs), matplotlib defaults, then restore cwd/argv/rc/modules."""
    tok = shlex.split(line); name = tok[0]; cwd = FIGS; argv = []
    if "--cwd" in tok: cwd = Path(tok[tok.index("--cwd") + 1])
    if "--argv" in tok: argv = tok[tok.index("--argv") + 1:]
    old = (os.getcwd(), sys.argv[:], matplotlib.rcParams.copy(), dict(sys.modules), sys.path[:])
    import matplotlib.pyplot as plt
    try:
        os.chdir(cwd); sys.argv = [name] + argv; matplotlib.rcdefaults(); matplotlib.use("Agg")
        g = {"__name__": "__main__", "__file__": str(FIGS / name)}
        exec(compile(cell, str(FIGS / name), "exec"), g)
    finally:
        plt.close("all"); os.chdir(old[0]); sys.argv = old[1]; matplotlib.rcParams.update(old[2])
        for k in list(sys.modules):
            if k not in old[3] and (k.startswith("coe_") or k.startswith("make_")): del sys.modules[k]
        for k, v in old[3].items():
            if k.startswith("coe_") or k.startswith("make_"): sys.modules[k] = v
        sys.path[:] = old[4]
        if FIGDIR not in sys.path: sys.path.insert(0, FIGDIR)

MATCH = []
```

```
def compare(new, ref, label=None):
    """approved reference (left) | notebook re-run (right), side by side; prints sizes and pixel differences."""
    from IPython.display import Image as IImage
    a = Image.open(ref).convert("RGB"); b = Image.open(new).convert("RGB")
    same = a.size == b.size; bb = b if same else b.resize(a.size, Image.LANCZOS)
    d = np.abs(np.asarray(a, float) - np.asarray(bb, float)); mad = d.mean(); frac = (d.max(axis=2) > 40).mean()
    verdict = "identical" if d.max() == 0 else ("match (tiny rendering differences)" if frac < 0.005 else "DIFFERS")
    MATCH.append(dict(figure=label or Path(new).name, reference=str(ref), size_ref=a.size, size_new=b.size, mean_abs_diff=round(mad, 3),
frac_px_diff=round(frac, 5), verdict=verdict))
    print(f"{Path(new).name}: new {b.size} vs ref {a.size} | mean |diff| = {mad:.3f}/255, pixels differing >40: {100*frac:.3f}% -> {verdict}")
    print("left: approved reference " + str(ref).replace("/workspace/cosmology/", "") + "   |   right: notebook re-run steps_v3/figs/" +
Path(new).name)
    W = 1100; h = round(a.size[1] * W / a.size[0]); pair = Image.new("RGB", (2 * W + 24, h), "white")
    pair.paste(a.resize((W, h), Image.LANCZOS), (0, 0)); pair.paste(b.resize((W, h), Image.LANCZOS), (W + 24, 0))
    out = FIGS / ("cmp_" + Path(new).stem + ".jpg"); pair.save(out, quality=85); display(IImage(filename=str(out)))
```

Step 9 — Mollweide maps of Migkas Table 2 A extrema, and Fig. 8 with the published markers

What it shows. (i) A clean 2D Galactic Mollweide map of the Migkas et al. 2020 Table 2 A maximum and minimum directions for every sample and cone size, with no CoE loops and no model curves. (ii) The same published MAX/MIN markers placed on the real Migkas Fig. 8 ($\theta = 75^\circ$) $A/A_{\rm all}$ map. **Geometry.** Mollweide: $x = \frac{2\sqrt{2}}{\pi}\lambda\cos\vartheta$, $y = \sqrt{2}\sin\vartheta$, with $2\vartheta + \sin 2\vartheta = \pi \sin b$ and $\lambda = -\mathrm{wrap}(\ell)$, so $\ell = 0$ is at the centre and ℓ increases to the left. The Fig. 8 image is registered by its oval: centre and semi-axes $(c_x, c_y, R_x, R_y) = (863.0, 763.25, 693.99, 392.54)$ px.

Source script: `/workspace/cosmology/figures/make_migkas_A_extrema_maps.py` (last modified 2026-09-25 20:22 CT), run verbatim. *Output-path edit:* none. The script's `OUT_DIR` follows `__file__`, which is set to `steps_v3/figs/`. *Reference image:* `figures/migkas_A_extrema_table2_mollweide.png`, `figures/migkas_fig8_theta75_A_extrema_only.png`.

```
%%runscript make_migkas_A_extrema_maps.py
#!/usr/bin/env python3
"""
Clean 2D Galactic Mollweide maps of Migkas et al. 2020 Table 2 A extrema.

Deliverables (this directory):
  migkas_A_extrema_table2_mollweide.png / .pdf
  migkas_fig8_theta75_A_extrema_only.png
  migkas_A_extrema_table2.json

Convention (match Migkas Fig. 8 / existing overlay):
  lon_disp = -wrap(ℓ)  →  ℓ=0 at center, ℓ increases LEFT
  (ℓ=90 left of center, ℓ=270 right of center)

No CoE rings, no model curves — published Table 2 directions only.
"""
```

```
from __future__ import annotations

import json
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
from matplotlib.lines import Line2D
from matplotlib.patches import Ellipse, FancyBboxPatch
from PIL import Image

OUT_DIR = Path(__file__).resolve().parent
DPI = 220

# ---------------------------------------------------------------------------
# Migkas Table 2 — Galactic (ℓ, b) degrees. Authoritative from user brief.
# ---------------------------------------------------------------------------

PANEL_A = [
    # Our sample All θ cones (primary paper maps)
    {
        "id": "all_90",
        "label": "90°",
        "sample": "Our sample All 90°",
        "panel": "A",
        "low": {"ell": 272.0, "b": -8.0},
        "high": {"ell": 47.0, "b": 22.0},
        "dipole": {"ell": 230.0, "b": -20.0},
    },
    {
        "id": "all_75",
        "label": "75°",
        "sample": "Our sample All 75° (Fig. 8 primary)",
        "panel": "A",
        "low": {"ell": 274.0, "b": -22.0, "A": 0.999, "A_err": 0.050},
        "high": {"ell": 17.0, "b": -9.0, "A": 1.288, "A_err": 0.061},
        "dipole": {"ell": 263.0, "b": -21.0},
        "primary": True,
    },
    {
        "id": "all_60",
        "label": "60°",
        "sample": "Our sample All 60°",
        "panel": "A",
        "low": {"ell": 281.0, "b": -16.0},
        "high": {"ell": 34.0, "b": 4.0},
        "dipole": {"ell": 260.0, "b": -36.0},
    },
    {
        "id": "all_45",
        "label": "45°",
        "sample": "Our sample All 45°",
        "panel": "A",
```

```
        "low": {"ell": 280.0, "b": 1.0},
        "high": {"ell": 32.0, "b": 14.0},
        "dipole": {"ell": 255.0, "b": -53.0},
    },
]

PANEL_B = [
    {
        "id": "all_75_freeB_c1",
        "label": "75° free B C1",
        "sample": "All 75° free B Case 1",
        "panel": "B",
        "low": {"ell": 272.0, "b": -21.0},
        "high": {"ell": 26.0, "b": -13.0},
    },
    {
        "id": "all_75_freeB_c2",
        "label": "75° free B C2",
        "sample": "All 75° free B Case 2",
        "panel": "B",
        "low": {"ell": 269.0, "b": -17.0},
        "high": {"ell": 23.0, "b": -10.0},
    },
    {
        "id": "T2p5_z003_75",
        "label": "T>2.5 z>0.03",
        "sample": "T>2.5 keV z>0.03 (75°)",
        "panel": "B",
        "low": {"ell": 288.0, "b": -35.0},
        "high": {"ell": 10.0, "b": 16.0},
    },
    {
        "id": "T3_z005_75",
        "label": "T>3 z>0.05",
        "sample": "T>3 keV z>0.05 (75°)",
        "panel": "B",
        "low": {"ell": 286.0, "b": -36.0},
        "high": {"ell": 9.0, "b": 15.0},
    },
    {
        "id": "Zcore_75",
        "label": "Zcore≤0.59",
        "sample": "Zcore≤0.59 (75°)",
        "panel": "B",
        "low": {"ell": 264.0, "b": -18.0},
        "high": {"ell": 30.0, "b": 23.0},
    },
    {
        "id": "Zout_75",
        "label": "Zout≤0.426",
        "sample": "Zout≤0.426 (75°)",
        "panel": "B",
        "low": {"ell": 270.0, "b": -14.0},
        "high": {"ell": 24.0, "b": 15.0},
```

```
    },
    {
        "id": "NH_75",
        "label": "NH≤5.16e20",
        "sample": "NH≤5.16e20 (75°)",
        "panel": "B",
        "low": {"ell": 242.0, "b": -27.0},
        "high": {"ell": 35.0, "b": -15.0},
    },
    {
        "id": "ACC_75",
        "label": "ACC 75°",
        "sample": "ACC 75°",
        "panel": "B",
        "low": {"ell": 314.0, "b": -17.0},
        "high": {"ell": 77.0, "b": 15.0},
    },
    {
        "id": "XCS_75",
        "label": "XCS-DR1 75°",
        "sample": "XCS-DR1 75°",
        "panel": "B",
        "low": {"ell": 281.0, "b": 24.0},
        "high": {"ell": 31.0, "b": 25.0},
    },
    {
        "id": "joint_ACC_XCS_75",
        "label": "Joint+ACC+XCS",
        "sample": "Joint+ACC+XCS 75°",
        "panel": "B",
        "low": {"ell": 303.0, "b": -27.0},
        "high": {"ell": 34.0, "b": 26.0},
        "dipole": {"ell": 265.0, "b": -20.0},
    },
    {
        "id": "joint_freeB_c1",
        "label": "Joint free B C1",
        "sample": "Joint free B Case 1",
        "panel": "B",
        "low": {"ell": 312.0, "b": -21.0},
        "high": {"ell": 45.0, "b": 21.0},
    },
]

ALL_ROWS = PANEL_A + PANEL_B

# Colors
C_HIGH = "#c0392b"   # warm red — highest A
C_LOW = "#2471a3"    # cool blue — lowest A
C_DIP = "#b7950b"    # gold — dipole
C_ARC = "#7f8c8d"    # gray arcs

# Fig. 8 registration (from make_migkas_fig8_coe_rings_overlay.py)
GOOD_REG = {"cx": 863.0, "cy": 763.25, "Rx": 693.9927, "Ry": 392.542425}
```

```
SOURCE_CANDIDATES = [
    Path("/workspace/cosmology/map_compare/FULL_migkas_theta75.png"),
    Path("/workspace/cosmology/map_compare/migkas_fig8_theta75_crop.png"),
    Path("/workspace/cosmology/migkas/pub_maps/Fig8_full_hires.png"),
]


# ---------------------------------------------------------------------------
# Coordinate helpers (same as Fig. 8 overlay)
# ---------------------------------------------------------------------------

def wrap_lon_deg(lon):
    return ((np.asarray(lon, dtype=float) + 180.0) % 360.0) - 180.0


def lon_disp_from_ell(ell_deg):
    """Display longitude: ℓ=0 center, ℓ increases LEFT (Migkas Fig. 8)."""
    return -wrap_lon_deg(ell_deg)


def mollweide_xy(lon_rad, lat_rad, n_iter=64):
    lon = np.asarray(lon_rad, dtype=float)
    lat = np.asarray(lat_rad, dtype=float)
    target = np.pi * np.sin(np.clip(lat, -np.pi / 2, np.pi / 2))
    lo = np.full_like(lat, -np.pi / 2, dtype=float)
    hi = np.full_like(lat, np.pi / 2, dtype=float)
    for _ in range(n_iter):
        mid = 0.5 * (lo + hi)
        g = 2.0 * mid + np.sin(2.0 * mid)
        lo = np.where(g < target, mid, lo)
        hi = np.where(g >= target, mid, hi)
    theta = 0.5 * (lo + hi)
    theta = np.where(lat >= np.pi / 2 - 1e-15, np.pi / 2, theta)
    theta = np.where(lat <= -np.pi / 2 + 1e-15, -np.pi / 2, theta)
    x = (2.0 * np.sqrt(2.0) / np.pi) * lon * np.cos(theta)
    y = np.sqrt(2.0) * np.sin(theta)
    return x, y


def lb_to_px(ell_deg, b_deg, cx, cy, Rx, Ry):
    lon = np.deg2rad(lon_disp_from_ell(np.asarray(ell_deg, dtype=float)))
    lat = np.deg2rad(np.asarray(b_deg, dtype=float))
    x, y = mollweide_xy(lon, lat)
    Xn = x / (2.0 * np.sqrt(2.0))
    Yn = y / np.sqrt(2.0)
    return cx + Xn * Rx, cy - Yn * Ry


def lb_to_n(l_deg, b_deg):
    l = np.deg2rad(l_deg)
    b = np.deg2rad(b_deg)
    return np.array(
        [np.cos(b) * np.cos(l), np.cos(b) * np.sin(l), np.sin(b)], dtype=float
    )
```

```
def n_to_lb(n):
    n = np.asarray(n, dtype=float)
    n = n / np.linalg.norm(n)
    b = float(np.rad2deg(np.arcsin(np.clip(n[2], -1.0, 1.0))))
    l = float(np.rad2deg(np.arctan2(n[1], n[0])) % 360.0)
    return l, b


def great_circle_lb(ell1, b1, ell2, b2, n=181):
    """Short great-circle arc from (ℓ1,b1) to (ℓ2,b2)."""
    n1 = lb_to_n(ell1, b1)
    n2 = lb_to_n(ell2, b2)
    n1 /= np.linalg.norm(n1)
    n2 /= np.linalg.norm(n2)
    cos_ang = float(np.clip(np.dot(n1, n2), -1.0, 1.0))
    ang = np.arccos(cos_ang)
    if ang < 1e-8:
        return np.array([ell1]), np.array([b1])
    t = np.linspace(0.0, 1.0, n)
    # slerp
    pts = (
        (np.sin((1 - t) * ang)[:, None] * n1 + np.sin(t * ang)[:, None] * n2)
        / np.sin(ang)
    )
    ls = np.array([n_to_lb(p)[0] for p in pts])
    bs = np.array([n_to_lb(p)[1] for p in pts])
    return ls, bs


# ---------------------------------------------------------------------------
# Matplotlib Mollweide helpers
# ---------------------------------------------------------------------------

def to_moll_rad(ell_deg, b_deg):
    """Return (lon_disp_rad, lat_rad) for ax.plot on projection='mollweide'."""
    lon = np.deg2rad(lon_disp_from_ell(ell_deg))
    lat = np.deg2rad(b_deg)
    return lon, lat


def plot_gc_arc(ax, ell1, b1, ell2, b2, **kw):
    ls, bs = great_circle_lb(ell1, b1, ell2, b2, n=241)
    lon, lat = to_moll_rad(ls, bs)
    # Split on antimeridian jumps in display longitude
    dlon = np.diff(lon)
    cuts = np.where(np.abs(dlon) > np.pi)[0] + 1
    idx = np.concatenate([[0], cuts, [len(lon)]])
    for i0, i1 in zip(idx[:-1], idx[1:]):
        if i1 - i0 >= 2:
            ax.plot(lon[i0:i1], lat[i0:i1], **kw)
```

```
def setup_moll_ax(ax, title=None):
    ax.set_longitude_grid(45)
    ax.set_latitude_grid(30)
    ax.set_longitude_grid_ends(90)
    ax.grid(True, linestyle=":", color="#bbbbbb", linewidth=0.6, alpha=0.9)
    # Tick labels: show Galactic ℓ at display positions
    # matplotlib mollweide shows display lon; we remapped so labels need care.
    # Set custom tick labels for longitude every 45°.
    tick_ells = [180, 135, 90, 45, 0, 315, 270, 225]  # left → right on map
    tick_disp = [lon_disp_from_ell(e) for e in tick_ells]
    ax.set_xticks(np.deg2rad(tick_disp))
    ax.set_xticklabels([rf"${e}^\circ$" for e in tick_ells], fontsize=8)
    ax.set_yticks(np.deg2rad([-60, -30, 0, 30, 60]))
    ax.set_yticklabels(
        [r"$-60^\circ$", r"$-30^\circ$", r"$0^\circ$", r"$+30^\circ$", r"$+60^\circ$"],
        fontsize=8,
    )
    if title:
        ax.set_title(title, fontsize=11, pad=10)


def annotate_point(ax, ell, b, text, color, offset=(6, 6), fontsize=7.5, weight="normal"):
    lon, lat = to_moll_rad(ell, b)
    ax.annotate(
        text,
        xy=(lon, lat),
        xytext=offset,
        textcoords="offset points",
        fontsize=fontsize,
        color=color,
        fontweight=weight,
        ha="left",
        va="bottom",
        bbox=dict(
            boxstyle="round,pad=0.15",
            facecolor="white",
            edgecolor="none",
            alpha=0.75,
        ),
        zorder=10,
    )


# ---------------------------------------------------------------------------
# Figure 1: two-panel Table 2 Mollweide
# ---------------------------------------------------------------------------

def draw_panel_a(ax):
    setup_moll_ax(
        ax,
        title=(
            r"Panel A — Our sample All $\theta$ cones "
            r"($90^\circ$, $75^\circ$, $60^\circ$, $45^\circ$)"
        ),
```

```
    )
    # Distinct sizes / slight offset labels per cone
    offsets = {
        "90°": {"high": (8, 8), "low": (8, -12)},
        "75°": {"high": (8, -14), "low": (-55, -14)},
        "60°": {"high": (8, 10), "low": (8, 8)},
        "45°": {"high": (-50, 8), "low": (8, 10)},
    }
    for row in PANEL_A:
        lo, hi = row["low"], row["high"]
        lab = row["label"]
        # pair arc
        plot_gc_arc(
            ax, lo["ell"], lo["b"], hi["ell"], hi["b"],
            color=C_ARC, lw=0.9, ls="--", alpha=0.65, zorder=2,
        )
        # low
        lon_l, lat_l = to_moll_rad(lo["ell"], lo["b"])
        ax.plot(
            lon_l, lat_l, marker="v", ms=9, color=C_LOW,
            markeredgecolor="k", markeredgewidth=0.6,
            linestyle="None", zorder=5,
        )
        # high
        lon_h, lat_h = to_moll_rad(hi["ell"], hi["b"])
        ax.plot(
            lon_h, lat_h, marker="*", ms=13, color=C_HIGH,
            markeredgecolor="k", markeredgewidth=0.55,
            linestyle="None", zorder=6,
        )
        # dipole (smaller)
        if "dipole" in row:
            d = row["dipole"]
            lon_d, lat_d = to_moll_rad(d["ell"], d["b"])
            ax.plot(
                lon_d, lat_d, marker="o", ms=4.5, color=C_DIP,
                markeredgecolor="k", markeredgewidth=0.4,
                linestyle="None", zorder=4, alpha=0.9,
            )
        off = offsets.get(lab, {"high": (6, 6), "low": (6, -10)})
        weight = "bold" if row.get("primary") else "normal"
        annotate_point(
            ax, hi["ell"], hi["b"], f"{lab} max", C_HIGH,
            offset=off["high"], fontsize=7.5 if not row.get("primary") else 8.5,
            weight=weight,
        )
        annotate_point(
            ax, lo["ell"], lo["b"], f"{lab} min", C_LOW,
            offset=off["low"], fontsize=7.5 if not row.get("primary") else 8.5,
            weight=weight,
        )


def draw_panel_b(ax):
```

```
    setup_moll_ax(
        ax,
        title=(
            "Panel B — Other Table 2 rows "
            "(subsamples + ACC / XCS / Joint)"
        ),
    )
    # Compact markers; legend identifies samples rather than crowding labels
    for i, row in enumerate(PANEL_B):
        lo, hi = row["low"], row["high"]
        lon_l, lat_l = to_moll_rad(lo["ell"], lo["b"])
        lon_h, lat_h = to_moll_rad(hi["ell"], hi["b"])
        ax.plot(
            lon_l, lat_l, marker="v", ms=7, color=C_LOW,
            markeredgecolor="k", markeredgewidth=0.45,
            linestyle="None", zorder=5, alpha=0.95,
        )
        ax.plot(
            lon_h, lat_h, marker="*", ms=10, color=C_HIGH,
            markeredgecolor="k", markeredgewidth=0.45,
            linestyle="None", zorder=6, alpha=0.95,
        )
        if "dipole" in row:
            d = row["dipole"]
            lon_d, lat_d = to_moll_rad(d["ell"], d["b"])
            ax.plot(
                lon_d, lat_d, marker="o", ms=3.5, color=C_DIP,
                markeredgecolor="k", markeredgewidth=0.35,
                linestyle="None", zorder=4, alpha=0.85,
            )
        # Tiny numeric index near high-A for legend cross-ref
        ax.annotate(
            str(i + 1),
            xy=(lon_h, lat_h),
            xytext=(5, 4),
            textcoords="offset points",
            fontsize=6.5,
            color="#333333",
            ha="left",
            va="bottom",
            zorder=10,
            bbox=dict(boxstyle="round,pad=0.1", facecolor="white", edgecolor="none", alpha=0.7),
        )


def make_table2_mollweide():
    fig = plt.figure(figsize=(11.5, 12.0), dpi=DPI)
    ax_a = fig.add_subplot(2, 1, 1, projection="mollweide")
    ax_b = fig.add_subplot(2, 1, 2, projection="mollweide")
    draw_panel_a(ax_a)
    draw_panel_b(ax_b)

    # Shared legend (below panel B)
    handles = [
```

```
        Line2D(
            [0], [0], marker="*", color="w", markerfacecolor=C_HIGH,
            markeredgecolor="k", linestyle="None", ms=12,
            label="Highest A (max)",
        ),
        Line2D(
            [0], [0], marker="v", color="w", markerfacecolor=C_LOW,
            markeredgecolor="k", linestyle="None", ms=9,
            label="Lowest A (min)",
        ),
        Line2D(
            [0], [0], marker="o", color="w", markerfacecolor=C_DIP,
            markeredgecolor="k", linestyle="None", ms=6,
            label="Dipole direction (where listed)",
        ),
        Line2D(
            [0], [0], color=C_ARC, lw=1.0, ls="--",
            label=r"Great-circle arc linking All-$\theta$ min–max pair (Panel A)",
        ),
    ]
    leg1 = fig.legend(
        handles=handles, loc="lower center", bbox_to_anchor=(0.5, 0.055),
        ncol=4, fontsize=8.5, frameon=True, fancybox=False,
        edgecolor="#333333", framealpha=1.0, facecolor="white",
    )
    leg1.get_frame().set_linewidth(0.8)

    # Panel B numbered legend
    b_lines = []
    for i, row in enumerate(PANEL_B):
        b_lines.append(f"{i + 1}. {row['sample']}")
    # Two-column text block
    col1 = "\n".join(b_lines[:6])
    col2 = "\n".join(b_lines[6:])
    fig.text(
        0.08, 0.012, col1, fontsize=7.0, family="sans-serif",
        va="bottom", ha="left", color="#222222",
        linespacing=1.35,
    )
    fig.text(
        0.52, 0.012, col2, fontsize=7.0, family="sans-serif",
        va="bottom", ha="left", color="#222222",
        linespacing=1.35,
    )
    fig.text(
        0.08, 0.048,
        "Panel B index (number next to each high-A marker):",
        fontsize=7.5, fontweight="bold", color="#222222",
    )

    fig.suptitle(
        "Migkas et al. 2020 — published Table 2 $A$ extrema on the Galactic sky\n"
        r"(Galactic Mollweide; $\ell\!=\!0$ center, $\ell$ increases LEFT — Fig. 8 convention; "
        "not a CoE / model map)",
```

```
        fontsize=12.5,
        y=0.985,
    )
    fig.subplots_adjust(left=0.04, right=0.98, top=0.91, bottom=0.12, hspace=0.28)

    png = OUT_DIR / "migkas_A_extrema_table2_mollweide.png"
    pdf = OUT_DIR / "migkas_A_extrema_table2_mollweide.pdf"
    fig.savefig(png, dpi=DPI, facecolor="white", bbox_inches="tight")
    fig.savefig(pdf, facecolor="white", bbox_inches="tight")
    plt.close(fig)
    print(f"wrote {png}")
    print(f"wrote {pdf}")
    return png, pdf


# ---------------------------------------------------------------------------
# Figure 2: Fig. 8 θ=75° overlay — published max/min/dipole only
# ---------------------------------------------------------------------------

def choose_source():
    for p in SOURCE_CANDIDATES:
        if p.is_file():
            return p
    raise FileNotFoundError("No Migkas Fig.8 theta=75 source image found")


def make_fig8_overlay():
    source = choose_source()
    rgb = np.asarray(Image.open(source).convert("RGB"))
    h, w, _ = rgb.shape
    cx, cy, Rx, Ry = GOOD_REG["cx"], GOOD_REG["cy"], GOOD_REG["Rx"], GOOD_REG["Ry"]

    # Spot-check: 75° max should land in warmer pixels than 75° min
    def sample(ell, b, r=5):
        px, py = lb_to_px(ell, b, cx, cy, Rx, Ry)
        xi, yi = int(round(float(px))), int(round(float(py)))
        patch = rgb[max(0, yi - r): yi + r + 1, max(0, xi - r): xi + r + 1]
        return patch.mean(axis=(0, 1)), (xi, yi)

    cmax, pmax = sample(17.0, -9.0)
    cmin, pmin = sample(274.0, -22.0)
    cdip, pdip = sample(263.0, -21.0)
    warm_ok = (cmax[0] + cmax[1]) > (cmin[0] + cmin[1]) + 40
    print(
        f"Fig8 source={source.name} size={w}x{h}; "
        f"max px={pmax} RGB={cmax.round(1)}; "
        f"min px={pmin} RGB={cmin.round(1)}; "
        f"dipole px={pdip}; warm_check={'OK' if warm_ok else 'WARN'}"
    )

    dpi = 140
    fig, ax = plt.subplots(figsize=(w / dpi, (h / dpi) + 0.95), dpi=dpi)
    ax.imshow(rgb, origin="upper", extent=(0, w, h, 0), interpolation="nearest")
    ax.set_xlim(0, w)
```

```
    ax.set_ylim(h, 0)
    ax.set_aspect("equal")
    ax.axis("off")

    clip_oval = Ellipse(
        (cx, cy), width=2.0 * Rx, height=2.0 * Ry,
        facecolor="none", edgecolor="none", transform=ax.transData,
    )
    ax.add_patch(clip_oval)

    def mark(ell, b, marker, color, ms, label, offset):
        px, py = lb_to_px(ell, b, cx, cy, Rx, Ry)
        px, py = float(px), float(py)
        ax.plot(
            px, py, marker=marker, ms=ms + 2.2, color="white",
            linestyle="None", zorder=6, clip_path=clip_oval, clip_on=True,
        )
        ax.plot(
            px, py, marker=marker, ms=ms, color=color,
            markeredgecolor="k", markeredgewidth=0.7,
            linestyle="None", zorder=7, clip_path=clip_oval, clip_on=True,
        )
        ax.annotate(
            label,
            xy=(px, py),
            xytext=offset,
            textcoords="offset points",
            fontsize=10,
            fontweight="bold",
            color=color,
            ha="left",
            va="center",
            bbox=dict(
                boxstyle="round,pad=0.25",
                facecolor="white",
                edgecolor=color,
                alpha=0.92,
                linewidth=1.0,
            ),
            zorder=8,
            clip_on=False,
        )

    # Optional thin dashed great-circle between min and max (in image pixels)
    ls, bs = great_circle_lb(274.0, -22.0, 17.0, -9.0, n=301)
    px_arc, py_arc = lb_to_px(ls, bs, cx, cy, Rx, Ry)
    px_arc = np.asarray(px_arc, dtype=float)
    py_arc = np.asarray(py_arc, dtype=float)
    # Split on large pixel jumps (antimeridian)
    segs_x, segs_y = [], []
    cx_list, cy_list = [], []
    for x, y in zip(px_arc, py_arc):
        # keep inside oval
        Xn = (x - cx) / Rx
```

```
        Yn = (y - cy) / Ry
        if Xn * Xn + Yn * Yn > 0.98:
            if len(cx_list) >= 2:
                segs_x.append(cx_list)
                segs_y.append(cy_list)
            cx_list, cy_list = [], []
            continue
        if cx_list and np.hypot(x - cx_list[-1], y - cy_list[-1]) > 40:
            if len(cx_list) >= 2:
                segs_x.append(cx_list)
                segs_y.append(cy_list)
            cx_list, cy_list = [x], [y]
        else:
            cx_list.append(x)
            cy_list.append(y)
    if len(cx_list) >= 2:
        segs_x.append(cx_list)
        segs_y.append(cy_list)
    for sx, sy in zip(segs_x, segs_y):
        ax.plot(
            sx, sy, color="white", lw=2.0, ls="--", alpha=0.85, zorder=4,
            clip_path=clip_oval, clip_on=True,
        )
        ax.plot(
            sx, sy, color="#555555", lw=1.1, ls="--", alpha=0.9, zorder=5,
            clip_path=clip_oval, clip_on=True,
        )

    mark(17.0, -9.0, "*", C_HIGH, 16, r"$75^\circ$ max $(17^\circ,-9^\circ)$", (14, 10))
    mark(274.0, -22.0, "v", C_LOW, 12, r"$75^\circ$ min $(274^\circ,-22^\circ)$", (14, -8))
    mark(263.0, -21.0, "o", C_DIP, 8, r"dipole $(263^\circ,-21^\circ)$", (14, 18))

    handles = [
        Line2D(
            [0], [0], marker="*", color="w", markerfacecolor=C_HIGH,
            markeredgecolor="k", linestyle="None", ms=14,
            label=r"Highest $A$: $(17^\circ,-9^\circ)$, $A=1.288\pm0.061$",
        ),
        Line2D(
            [0], [0], marker="v", color="w", markerfacecolor=C_LOW,
            markeredgecolor="k", linestyle="None", ms=10,
            label=r"Lowest $A$: $(274^\circ,-22^\circ)$, $A=0.999\pm0.050$",
        ),
        Line2D(
            [0], [0], marker="o", color="w", markerfacecolor=C_DIP,
            markeredgecolor="k", linestyle="None", ms=7,
            label=r"Dipole: $(263^\circ,-21^\circ)$",
        ),
    ]
    leg = ax.legend(
        handles=handles, loc="upper center", bbox_to_anchor=(0.5, -0.01),
        ncol=3, fontsize=9, frameon=True, fancybox=False,
        edgecolor="#222222", framealpha=1.0, facecolor="white",
    )
```

```
    leg.get_frame().set_linewidth(1.0)

    fig.suptitle(
        r"Migkas et al. 2020 Fig. 8 — $\theta=75^\circ$ $A/A_{\rm all}$ map "
        r"with published Table 2 extrema only"
        "\n"
        r"(no CoE rings / model; registration: lon$_{\rm disp}=-\mathrm{wrap}(\ell)$, "
        r"GOOD_REG from working Fig. 8 overlay)",
        fontsize=11,
        y=0.99,
    )
    fig.subplots_adjust(left=0.01, right=0.99, top=0.91, bottom=0.10)
    out = OUT_DIR / "migkas_fig8_theta75_A_extrema_only.png"
    fig.savefig(out, dpi=dpi, facecolor="white")
    plt.close(fig)
    print(f"wrote {out}")
    return out, {
        "source_image": str(source),
        "registration": dict(GOOD_REG),
        "pixel_checks": {
            "max_75": {"ell": 17.0, "b": -9.0, "px": list(pmax), "rgb_mean": [float(x) for x in cmax]},
            "min_75": {"ell": 274.0, "b": -22.0, "px": list(pmin), "rgb_mean": [float(x) for x in cmin]},
            "dipole_75": {"ell": 263.0, "b": -21.0, "px": list(pdip)},
            "max_warmer_than_min": bool(warm_ok),
        },
    }


# ---------------------------------------------------------------------------
# JSON
# ---------------------------------------------------------------------------

def write_json(fig8_meta):
    points = []
    for row in ALL_ROWS:
        entry = {
            "id": row["id"],
            "sample": row["sample"],
            "panel": row["panel"],
            "label": row["label"],
            "low_A": {
                "ell_deg": row["low"]["ell"],
                "b_deg": row["low"]["b"],
            },
            "high_A": {
                "ell_deg": row["high"]["ell"],
                "b_deg": row["high"]["b"],
            },
        }
        if "A" in row["low"]:
            entry["low_A"]["A"] = row["low"]["A"]
            entry["low_A"]["A_err"] = row["low"]["A_err"]
        if "A" in row["high"]:
            entry["high_A"]["A"] = row["high"]["A"]
```

```
            entry["high_A"]["A_err"] = row["high"]["A_err"]
        if "dipole" in row:
            entry["dipole"] = {
                "ell_deg": row["dipole"]["ell"],
                "b_deg": row["dipole"]["b"],
            }
        if row.get("primary"):
            entry["primary"] = True
        points.append(entry)

    meta = {
        "source": "Migkas et al. 2020, Table 2 (published A extrema directions)",
        "coordinate_system": "Galactic (ℓ, b) degrees",
        "mollweide_convention": (
            "lon_disp = -wrap(ℓ); ℓ=0 at center; ℓ increases LEFT "
            "(matches Migkas Fig. 8)"
        ),
        "note": (
            "These are published Migkas 2D map extrema from Table 2 — "
            "not CoE model rings or Amp×sin curves."
        ),
        "style": {
            "high_A": "red star",
            "low_A": "blue inverted triangle",
            "dipole": "gold circle (where listed)",
        },
        "points": points,
        "outputs": {
            "mollweide_png": str(OUT_DIR / "migkas_A_extrema_table2_mollweide.png"),
            "mollweide_pdf": str(OUT_DIR / "migkas_A_extrema_table2_mollweide.pdf"),
            "fig8_overlay_png": str(OUT_DIR / "migkas_fig8_theta75_A_extrema_only.png"),
        },
        "fig8_overlay": fig8_meta,
        "script": str(Path(__file__).resolve()),
    }
    path = OUT_DIR / "migkas_A_extrema_table2.json"
    path.write_text(json.dumps(meta, indent=2))
    print(f"wrote {path}")
    return path

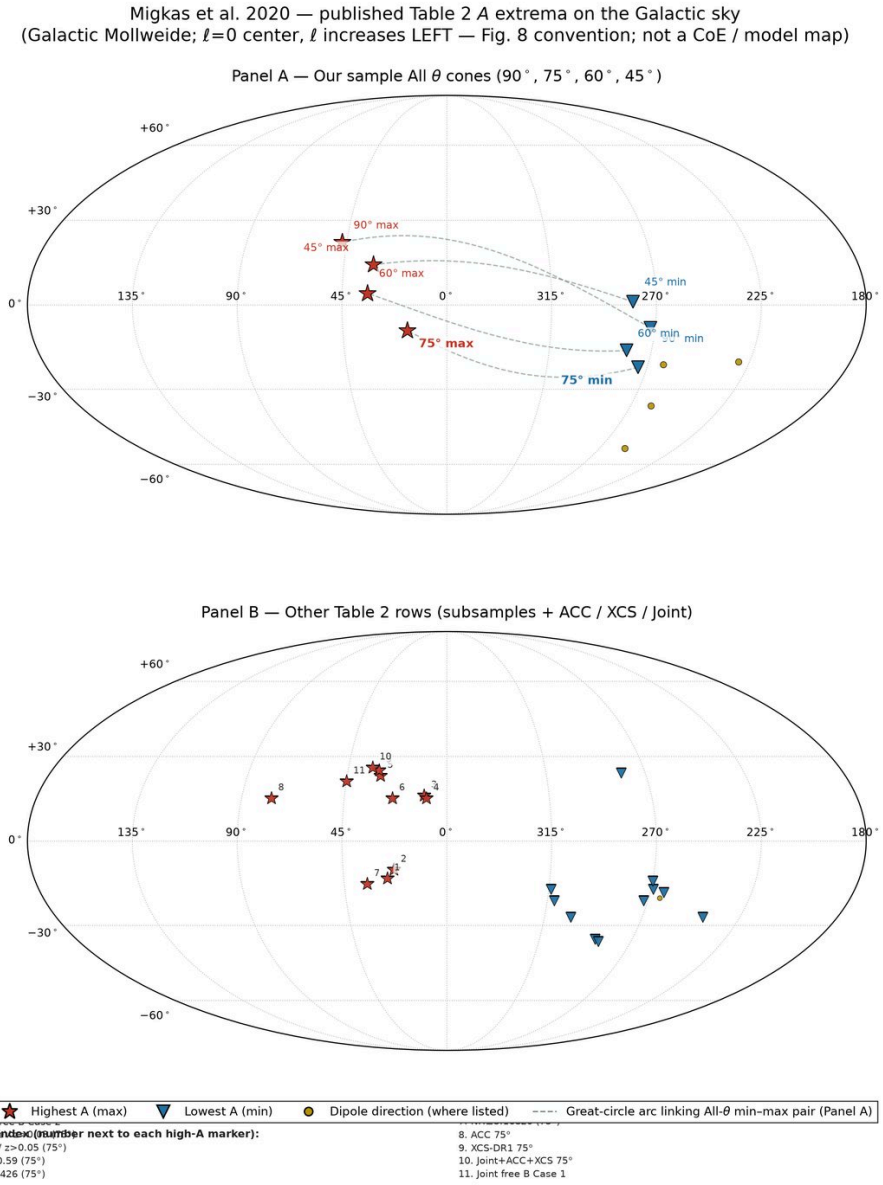

def main():
    png, pdf = make_table2_mollweide()
    fig8_png, fig8_meta = make_fig8_overlay()
    jpath = write_json(fig8_meta)
    print("--- done ---")
    for p in (png, pdf, fig8_png, jpath):
        print(p)

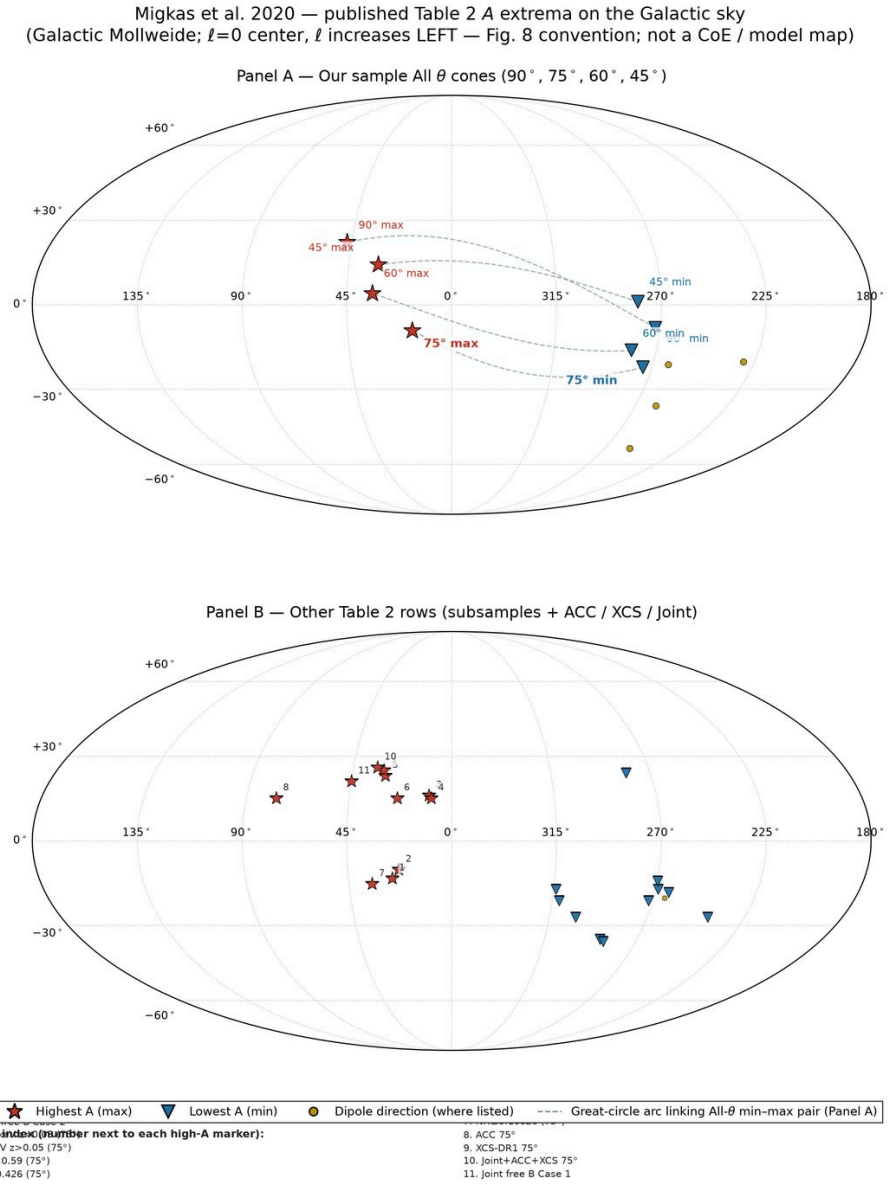

if __name__ == "__main__":
    main()
```

```
wrote /workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/migkas_A_extrema_table2_mollweide.png
wrote /workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/migkas_A_extrema_table2_mollweide.pdf
Fig8 source=FULL_migkas_theta75.png size=1975x1353; max px=(798, 812) RGB=[237.  166.3   0. ]; min px=(1179, 881) RGB=[ 96.6   0.  205.4]; dipole px=
(1221, 875); warm_check=OK
wrote /workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/migkas_fig8_theta75_A_extrema_only.png
wrote /workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/migkas_A_extrema_table2.json
--- done ---
/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/migkas_A_extrema_table2_mollweide.png
/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/migkas_A_extrema_table2_mollweide.pdf
/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/migkas_fig8_theta75_A_extrema_only.png
/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/migkas_A_extrema_table2.json
```

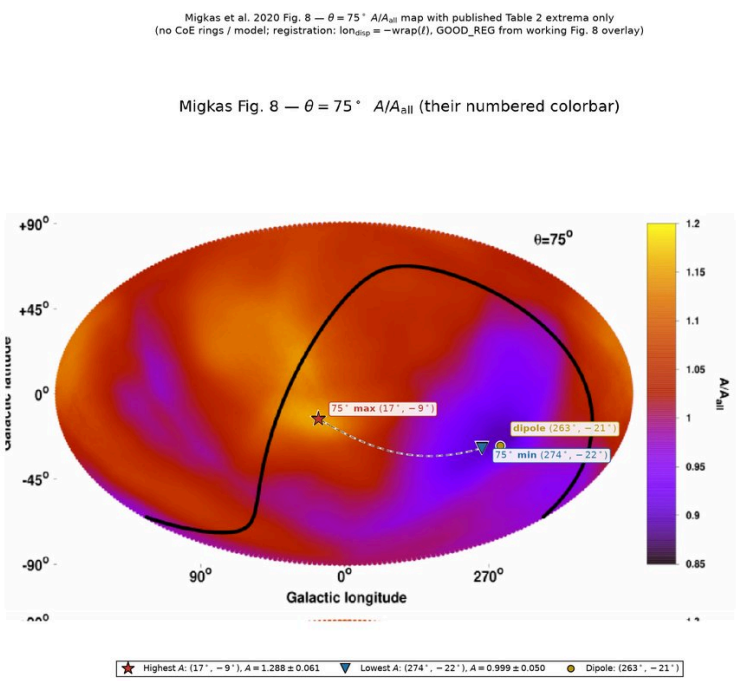
```
compare(FIGS / 'migkas_A_extrema_table2_mollweide.png', '/workspace/cosmology/figures/migkas_A_extrema_table2_mollweide.png')
compare(FIGS / 'migkas_fig8_theta75_A_extrema_only.png', '/workspace/cosmology/figures/migkas_fig8_theta75_A_extrema_only.png')
```

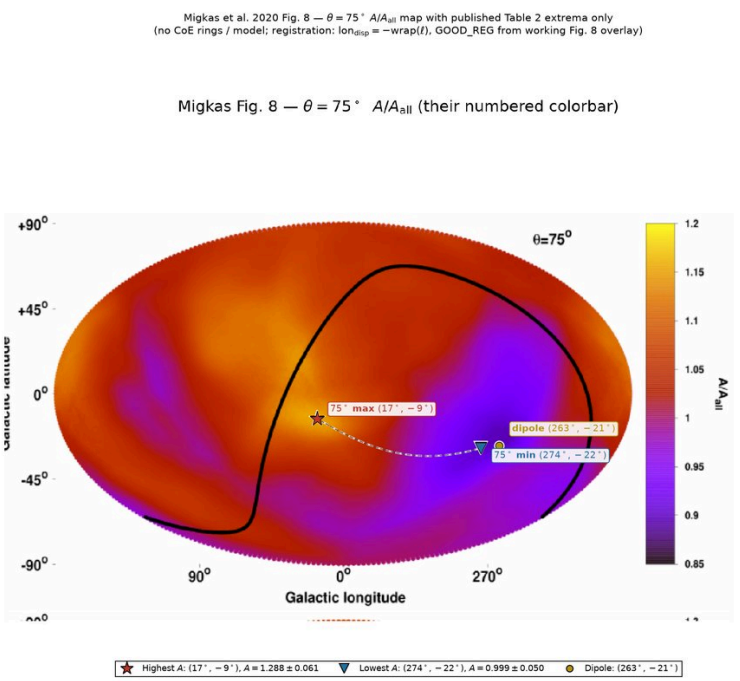
```
migkas_A_extrema_table2_mollweide.png: new (2079, 2612) vs ref (2079, 2612) | mean |diff| = 0.000/255, pixels differing >40: 0.000% -> identical
left: approved reference figures/migkas_A_extrema_table2_mollweide.png   |   right: notebook re-run steps_v3/figs/migkas_A_extrema_table2_mollweide.png
```





migkas_fig8_theta75_A_extrema_only.png: new (1975, 1486) vs ref (1975, 1486) | mean |diff| = 0.000/255, pixels differing >40: 0.000% -> identical
left: approved reference figures/migkas_fig8_theta75_A_extrema_only.png | right: notebook re-run steps_v3/figs/migkas_fig8_theta75_A_extrema_only.png





Step 10 — First three-panel loop figure (axis tilted $+30^\circ$)

What it shows. Panel A: the flat model-frame angle circle with the four extrema bands (two reds for MAX, two blues for MIN) and Galactic ℓ ticks. Panel B: the four loops on a Galactic Mollweide map with the Migkas markers. Panel C: the same loops on a 3D sphere. In this first version the CoE axis is tilted $+30^\circ$: anti-CoE at $(0^\circ, +30^\circ)$, CoE at $(180^\circ, -30^\circ)$. **Maths.** Loop centres $\theta = 30^\circ, 185^\circ$ (MAX) and $115^\circ, 265^\circ$ (MIN) from the anti-CoE direction. Bands $[\theta - 9^\circ, \theta + 9^\circ]$. A small circle of radius θ about $\hat{a}$ is traced by $\psi \in [0, 2\pi)$.

Source script: `/workspace/cosmology/figures/make_coe_rings_migkas_pin_axis.py` (last modified 2026-09-25 20:22 CT), run verbatim. *Output-path edit:* none. The script's `OUT_DIR` follows `__file__`, which is set to `steps_v3/figs/`. *Reference image:* `figures/coe_rings_migkas_pin_axis.png`.

```
%%runscript make_coe_rings_migkas_pin_axis.py
#!/usr/bin/env python3
"""
CoE rings + Migkas A(ell) extrema loops in the Galactic frame.

Concept:
  Pure CoE axisymmetry → constant A on small-circle rings about x = CoE↔anti-CoE.
  Scan A vs angle from +x (anti-CoE) around 360°. After-2ℓ A(ℓ) has TWO maxima
```

```
  and TWO minima (from best_fit_2ell_multiplier.npz A_after):
    MAX at θ = 30° and 185° from +x,
    MIN at θ = 115° and 265° from +x.

  Exactly FOUR loops — each ONE transparent highlight band, 18° end-to-end
  (= ±9° = 5% of 360°) about the center θ:
    MAX: 30±9 → [21°, 39°] and 185±9 → [176°, 194°]
    MIN: 115±9 → [106°, 124°] and 265±9 → [256°, 274°]
  Drawing: ONE smooth transparent filled ribbon between θ−9° and θ+9°
  + ONE solid mid-stroke at the exact center θ. No multi-stroke wash,
  no dense linspace hairball, no dual strong boundaries.
  Four distinct colors (two reds + two blues). No orange, no 283° companion.

  Galactic orientation: anti-CoE (+x) at ℓ=0°, b=+30°; CoE (−x) at ℓ=180°, b=−30°
  (axis tilted +30° so features near ℓ=0 move north on Mollweide; Migkas markers fixed).
  Sky markers (ℓ, b):
    Maxes: map extrema (17°, −9°) for ~30° max; strip (185°, 0°) for second max.
    Mins:  strip (115°, 0°) for ~115° min; map extrema (274°, −22°) for ~265° min.

Outputs:
  coe_rings_migkas_pin_axis.png
  coe_rings_migkas_pin_axis.json
"""

from __future__ import annotations

import json
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
from matplotlib.lines import Line2D
from matplotlib.collections import PolyCollection
from matplotlib.patches import Circle, Patch, Polygon
from mpl_toolkits.mplot3d import Axes3D  # noqa: F401
from mpl_toolkits.mplot3d.art3d import Poly3DCollection

OUT_DIR = Path(__file__).resolve().parent
PNG_PATH = OUT_DIR / "coe_rings_migkas_pin_axis.png"
JSON_PATH = OUT_DIR / "coe_rings_migkas_pin_axis.json"

# 5% of 360° = 18° end-to-end → half-width ±9° about each center
BAND_HALF = 9.0
BAND_FULL = 2.0 * BAND_HALF  # 18° = 5% of 360°
BAND_TOTAL = BAND_FULL  # alias
STRIP_THETA_MAX = [30.0, 185.0]
STRIP_THETA_MIN = [115.0, 265.0]

MIGKAS_MAXES = [(17.0, -9.0), (185.0, 0.0)]
MIGKAS_MINS = [(115.0, 0.0), (274.0, -22.0)]

C_COE = "#6b3fa0"
C_ANTI = "#8e1b1b"  # dark red marker; distinct from loop reds
# Four distinct loop colors: two reds (MAX) + two blues (MIN); no orange/yellow
```

```
C_MAX_30 = "#c0392b"    # deep red — MAX at 30°
C_MAX_185 = "#ec7063"   # lighter coral-red — MAX at 185° (not orange)
C_MIN_115 = "#1a5276"   # dark navy blue — MIN at 115°
C_MIN_265 = "#5dade2"   # light sky blue — MIN at 265°
C_AXIS = "#5b2c6f"
C_EQ = "#1e8449"

# Per-loop color maps (order matches STRIP_THETA_MAX / STRIP_THETA_MIN)
COLORS_MAX = [C_MAX_30, C_MAX_185]
COLORS_MIN = [C_MIN_115, C_MIN_265]


def wrap_lon_deg(lon: np.ndarray) -> np.ndarray:
    return ((np.asarray(lon, dtype=float) + 180.0) % 360.0) - 180.0


def lon_disp_from_ell(ell_deg: np.ndarray) -> np.ndarray:
    """Migkas Galactic: ℓ increases LEFT → lon_disp = −ℓ_wrapped."""
    return -wrap_lon_deg(ell_deg)


def lb_to_n(l_deg: float, b_deg: float) -> np.ndarray:
    l = np.deg2rad(l_deg)
    b = np.deg2rad(b_deg)
    return np.array(
        [np.cos(b) * np.cos(l), np.cos(b) * np.sin(l), np.sin(b)], dtype=float
    )


def n_to_lb(n: np.ndarray) -> tuple[float, float]:
    n = np.asarray(n, dtype=float)
    n = n / np.linalg.norm(n)
    b = float(np.rad2deg(np.arcsin(np.clip(n[2], -1.0, 1.0))))
    l = float(np.rad2deg(np.arctan2(n[1], n[0])) % 360.0)
    return l, b


def celestial_equator_lb(n: int = 720) -> tuple[np.ndarray, np.ndarray]:
    """Dec=0 (ICRS) as Galactic (ℓ, b). IAU J2000 NGP convention, no astropy."""
    ra_ngp = np.deg2rad(192.85948)
    dec_ngp = np.deg2rad(27.12825)
    l_ncp = np.deg2rad(122.93192)
    ra = np.linspace(0.0, 2.0 * np.pi, n, endpoint=False)
    dec = 0.0
    sin_b = np.sin(dec) * np.sin(dec_ngp) + np.cos(dec) * np.cos(dec_ngp) * np.cos(ra - ra_ngp)
    sin_b = np.clip(sin_b, -1.0, 1.0)
    b = np.arcsin(sin_b)
    y = np.cos(dec) * np.sin(ra - ra_ngp)
    x = np.sin(dec) * np.cos(dec_ngp) - np.cos(dec) * np.sin(dec_ngp) * np.cos(ra - ra_ngp)
    l = (l_ncp - np.arctan2(y, x)) % (2.0 * np.pi)
    return np.rad2deg(l), np.rad2deg(b)


def great_circle_through_poles(D: np.ndarray, n: int = 361) -> tuple[np.ndarray, np.ndarray]:
```

```
    """Great circle through ±D (any meridian of the CoE axis)."""
    a = D / np.linalg.norm(D)
    tmp = np.array([0.0, 0.0, 1.0])
    if abs(np.dot(a, tmp)) > 0.9:
        tmp = np.array([1.0, 0.0, 0.0])
    e2 = np.cross(a, tmp)
    e2 /= np.linalg.norm(e2)
    t = np.linspace(0.0, 2.0 * np.pi, n)
    pts = np.outer(np.cos(t), a) + np.outer(np.sin(t), e2)
    ls = np.array([n_to_lb(p)[0] for p in pts])
    bs = np.array([n_to_lb(p)[1] for p in pts])
    return ls, bs


def small_circle_about_axis(axis: np.ndarray, mu: float, n: int = 361) -> np.ndarray:
    """Unit vectors on the small circle n·axis = mu (|mu|<1)."""
    a = axis / np.linalg.norm(axis)
    mu = float(np.clip(mu, -1.0 + 1e-9, 1.0 - 1e-9))
    tmp = np.array([0.0, 0.0, 1.0]) if abs(a[2]) < 0.9 else np.array([1.0, 0.0, 0.0])
    e1 = np.cross(a, tmp)
    e1 /= np.linalg.norm(e1)
    e2 = np.cross(a, e1)
    rho = np.sqrt(max(0.0, 1.0 - mu * mu))
    t = np.linspace(0.0, 2.0 * np.pi, n)
    return (mu * a)[None, :] + rho * (np.outer(np.cos(t), e1) + np.outer(np.sin(t), e2))


def split_lon_segments(lon_rad: np.ndarray, lat_rad: np.ndarray, gap: float = np.pi):
    """Split polyline where Mollweide lon jumps (wrap discontinuity)."""
    d = np.diff(lon_rad)
    cuts = np.where(np.abs(d) > gap)[0] + 1
    idx = np.concatenate([[0], cuts, [len(lon_rad)]])
    segs = []
    for i0, i1 in zip(idx[:-1], idx[1:]):
        if i1 - i0 >= 2:
            segs.append((lon_rad[i0:i1], lat_rad[i0:i1]))
    return segs


def plot_lb_curve_mollweide(ax, l_deg, b_deg, **kwargs):
    lon = np.deg2rad(lon_disp_from_ell(l_deg))
    lat = np.deg2rad(b_deg)
    for xr, yr in split_lon_segments(lon, lat):
        ax.plot(xr, yr, **kwargs)


def band_edges(centers: list[float], half: float = BAND_HALF) -> list[tuple[float, float]]:
    """Return (theta_lo, theta_hi) for each band center (±half about center)."""
    return [(c - half, c + half) for c in centers]


def mu_pair_for_band(lo: float, hi: float) -> tuple[float, float]:
    """Edge μ=cos(θ) for a scan-angle band (two boundary cones only)."""
    return float(np.cos(np.deg2rad(lo))), float(np.cos(np.deg2rad(hi)))
```

```
def migkas_scan_stops() -> dict:
    th_max = [float(t) for t in STRIP_THETA_MAX]
    th_min = [float(t) for t in STRIP_THETA_MIN]
    max_bands = band_edges(th_max)
    min_bands = band_edges(th_min)
    return {
        "theta_max_from_plus_x_deg": th_max,
        "theta_min_from_plus_x_deg": th_min,
        "max_bands_deg": max_bands,
        "min_bands_deg": min_bands,
        "band_half_width_deg": BAND_HALF,
        "band_full_width_deg": BAND_FULL,
        "band_total_width_deg": BAND_TOTAL,
        "band_width_note": "5% of 360° = 18° end-to-end = ±9° about each MAX/MIN center",
        "mu_max_centers": [float(np.cos(np.deg2rad(t))) for t in th_max],
        "mu_min_centers": [float(np.cos(np.deg2rad(t))) for t in th_min],
        "mu_max_bands": [mu_pair_for_band(lo, hi) for lo, hi in max_bands],
        "mu_min_bands": [mu_pair_for_band(lo, hi) for lo, hi in min_bands],
        "source": (
            "best_fit_2ell_multiplier.npz A_after extrema (two max, two min); "
            f"each drawn as one transparent {BAND_FULL:.0f}° end-to-end highlight "
            f"(±{BAND_HALF:.0f}° = 5% of 360°) + one solid mid-stroke"
        ),
    }


def compute_geometry(st: dict) -> dict:
    anti_l, anti_b = 0.0, 30.0
    coe_l, coe_b = 180.0, -30.0

    n_coe = lb_to_n(coe_l, coe_b)
    n_anti = lb_to_n(anti_l, anti_b)
    D = n_anti  # +D = anti-CoE = +x

    th_max = [float(t) for t in st["theta_max_from_plus_x_deg"]]
    th_min = [float(t) for t in st["theta_min_from_plus_x_deg"]]
    max_bands = list(st["max_bands_deg"])
    min_bands = list(st["min_bands_deg"])

    n_maxes = [lb_to_n(*lb) for lb in MIGKAS_MAXES]
    n_mins = [lb_to_n(*lb) for lb in MIGKAS_MINS]

    sep = float(
        np.rad2deg(
            np.arccos(np.clip(np.dot(n_maxes[0], n_mins[1]), -1.0, 1.0))
        )
    )

    return {
        "D": D,
        "n_coe": n_coe,
        "n_anti": n_anti,
```

```
        "n_maxes": n_maxes,
        "n_mins": n_mins,
        "anti_lb": (anti_l, anti_b),
        "coe_lb": (coe_l, coe_b),
        "theta_max_from_plus_x_deg": th_max,
        "theta_min_from_plus_x_deg": th_min,
        "max_bands_deg": max_bands,
        "min_bands_deg": min_bands,
        "mu_max_bands": list(st["mu_max_bands"]),
        "mu_min_bands": list(st["mu_min_bands"]),
        "sep_deg": sep,
        "band_half_width_deg": st["band_half_width_deg"],
    }


def draw_wire_sphere(ax, n_mer=12, n_par=6, **kwargs):
    u = np.linspace(0, 2 * np.pi, 60)
    v = np.linspace(-np.pi / 2, np.pi / 2, 40)
    xs = np.outer(np.cos(u), np.cos(v))
    ys = np.outer(np.sin(u), np.cos(v))
    zs = np.outer(np.ones_like(u), np.sin(v))
    ax.plot_surface(xs, ys, zs, color="#dfe6e9", alpha=0.15, linewidth=0, shade=False)
    kw = dict(color="#95a5a6", lw=0.4, alpha=0.45)
    kw.update(kwargs)
    for i in range(n_mer):
        phi = 2 * np.pi * i / n_mer
        t = np.linspace(-np.pi / 2, np.pi / 2, 80)
        ax.plot(np.cos(t) * np.cos(phi), np.cos(t) * np.sin(phi), np.sin(t), **kw)
    for j in range(1, n_par):
        b = -np.pi / 2 + np.pi * j / n_par
        t = np.linspace(0, 2 * np.pi, 80)
        r = np.cos(b)
        ax.plot(r * np.cos(t), r * np.sin(t), np.full_like(t, np.sin(b)), **kw)


def foreshortened_ellipse_xy(mu: float, n: int = 200):
    """Side-view foreshortened ellipse of a 3D ring at n·x = mu."""
    rho = np.sqrt(max(0.0, 1.0 - mu * mu))
    phi = np.linspace(0, 2 * np.pi, n)
    ex = mu + 0.045 * rho * np.cos(phi)
    ey = rho * np.sin(phi)
    return ex, ey, rho


def draw_panelA_ribbon(ax, theta_lo: float, theta_hi: float, theta_mid: float, color: str):
    """
    One Panel-A highlight: clean vertical strip between x=cos(θ_hi)..cos(θ_lo)
    clipped to the unit circle (uniform alpha), plus ONE solid foreshortened
    mid-ellipse at the exact center θ. No multi-stroke wash / hairball.
    """
    mu_lo = float(np.cos(np.deg2rad(theta_lo)))
    mu_hi = float(np.cos(np.deg2rad(theta_hi)))
    x0, x1 = (min(mu_lo, mu_hi), max(mu_lo, mu_hi))
    # Vertical strip polygon (tall enough to cover the circle; clipped below)
```

```
    strip = Polygon(
        np.array([[x0, -1.15], [x1, -1.15], [x1, 1.15], [x0, 1.15]]),
        closed=True,
        facecolor=color,
        edgecolor="none",
        alpha=0.28,
        zorder=2,
        linewidth=0,
    )
    ax.add_patch(strip)
    clip_circ = Circle((0.0, 0.0), 1.0, transform=ax.transData)
    strip.set_clip_path(clip_circ)

    mu_mid = float(np.cos(np.deg2rad(theta_mid)))
    ex_m, ey_m, _ = foreshortened_ellipse_xy(mu_mid, n=256)
    ax.plot(ex_m, ey_m, color=color, lw=2.8, alpha=0.95, zorder=4, solid_capstyle="round")


def draw_panelA_theta_rays(ax, geo):
    """
    Illustrate θ: from the observer (origin), one ray per loop out to the
    ring at that loop's center θ. θ is measured from +x (anti-CoE, right)
    up to the ring. Emphasize MAX θ=30° (thick semi-transparent red to the
    top of that foreshortened ring), matching Chris's markup.
    """
    items = list(zip(geo["theta_max_from_plus_x_deg"], COLORS_MAX)) + list(
        zip(geo["theta_min_from_plus_x_deg"], COLORS_MIN)
    )
    for mid, color in items:
        t = float(np.deg2rad(mid))
        # Endpoint = top/rim point of the ring in side view = (cos θ, sin θ)
        x1, y1 = float(np.cos(t)), float(np.sin(t))
        is_max30 = abs(mid - 30.0) < 0.5
        lw = 3.2 if is_max30 else 1.8
        alpha = 0.70 if is_max30 else 0.45
        ax.plot(
            [0.0, x1], [0.0, y1],
            color=color, lw=lw, alpha=alpha, zorder=3,
            solid_capstyle="round",
        )
        # Small tick at the ring hit
        ax.plot([x1], [y1], "o", color=color, ms=4.5 if is_max30 else 3.5,
                alpha=0.9, zorder=5, markeredgecolor="none")

    # Angle arc at θ=30° from +x axis up to the ray (makes the measure obvious)
    arc_r = 0.38
    phi = np.linspace(0.0, np.deg2rad(30.0), 60)
    ax.plot(
        arc_r * np.cos(phi), arc_r * np.sin(phi),
        color=C_MAX_30, lw=1.6, alpha=0.85, zorder=3,
    )
    ax.text(
        arc_r * np.cos(np.deg2rad(15.0)) + 0.06,
        arc_r * np.sin(np.deg2rad(15.0)) + 0.04,
```

```
        r"$\theta$",
        color=C_MAX_30, fontsize=10, fontweight="bold",
        ha="left", va="bottom", zorder=6,
    )


def panel_A_model_frame(ax, geo):
    """Side view: exactly four thick foreshortened loop ribbons about x."""
    ax.set_title(
        "A. Model frame (no Galactic yet)\n"
        r"axisymmetric about $x$ = CoE$\leftrightarrow$anti-CoE"
        "\n"
        r"two MAX + two MIN, each $18^\circ$ end-to-end ($\pm9^\circ$)",
        fontsize=10,
        pad=8,
    )
    th = np.linspace(0, 2 * np.pi, 400)
    ax.plot(np.cos(th), np.sin(th), color="#7f8c8d", lw=1.2, zorder=1)
    ax.plot(0, 0, "ko", ms=6, zorder=5)
    ax.text(0.08, -0.12, "observer", fontsize=8, color="k", ha="left")

    ax.annotate(
        "",
        xy=(1.15, 0),
        xytext=(-1.15, 0),
        arrowprops=dict(arrowstyle="<->", color=C_AXIS, lw=2.5),
        zorder=4,
    )
    ax.text(1.18, 0.08, "anti-CoE\n(+x)", color=C_ANTI, fontsize=8, ha="left", va="bottom")
    ax.text(-1.18, 0.08, "CoE\n(−x)", color=C_COE, fontsize=8, ha="right", va="bottom")

    for (lo, hi), mid, color in zip(
        geo["max_bands_deg"], geo["theta_max_from_plus_x_deg"], COLORS_MAX
    ):
        draw_panelA_ribbon(ax, lo, hi, mid, color)
    for (lo, hi), mid, color in zip(
        geo["min_bands_deg"], geo["theta_min_from_plus_x_deg"], COLORS_MIN
    ):
        draw_panelA_ribbon(ax, lo, hi, mid, color)

    # Illustrate θ from +x (anti-CoE) to each loop (Chris markup)
    draw_panelA_theta_rays(ax, geo)

    label_r = 1.30
    for mid, color in list(zip(geo["theta_max_from_plus_x_deg"], COLORS_MAX)) + list(
        zip(geo["theta_min_from_plus_x_deg"], COLORS_MIN)
    ):
        t = np.deg2rad(mid)
        ax.text(
            label_r * np.cos(t),
            label_r * np.sin(t),
            rf"$\theta\!=\!{mid:.0f}^\circ$",
            color=color, fontsize=7.5, ha="center", va="center", fontweight="bold",
        )
```

```
    ax.text(
        0.0,
        -1.38,
        (
            r"four transparent highlight bands about $x$  ·  each $18^\circ$ end-to-end ($\pm9^\circ$ = 5% of $360^\circ$)"
            + "\n"
            + r"(after-2$\ell$ $A(\ell)$: two MAX + two MIN; one mid-stroke each)"
            + "\n"
            + r"$y,z$ free (roll) — model does not pick celestial equator"
        ),
        fontsize=7.5,
        ha="center",
        va="top",
        style="italic",
        color="#34495e",
    )
    ax.set_xlim(-1.55, 1.55)
    ax.set_ylim(-1.55, 1.45)
    ax.set_aspect("equal")
    ax.axis("off")


def _ring_lb(D, mu, n_pts=721):
    pts = small_circle_about_axis(D, mu, n=n_pts)
    rl = np.array([n_to_lb(p)[0] for p in pts])
    rb = np.array([n_to_lb(p)[1] for p in pts])
    return rl, rb


def _ring_lonlat_rad(D, mu, n_pts=361):
    """Small-circle → Mollweide display (lon_disp, lat) in radians."""
    pts = small_circle_about_axis(D, float(mu), n=n_pts)
    ls = np.array([n_to_lb(p)[0] for p in pts])
    bs = np.array([n_to_lb(p)[1] for p in pts])
    lon = np.deg2rad(lon_disp_from_ell(ls))
    lat = np.deg2rad(bs)
    return lon, lat


def fill_mollweide_ribbon(ax, D, theta_lo, theta_hi, theta_mid, color, n_pts=721):
    """
    One Mollweide highlight band:
      continuous transparent polygon strips between ±9° edges
      (broken only at Mollweide antimeridian seams — NOT a dense quad wash)
      + ONE solid mid-stroke at θ_mid.
    """
    mu_a, mu_b = mu_pair_for_band(theta_lo, theta_hi)
    mu_mid = float(np.cos(np.deg2rad(theta_mid)))
    lon_a, lat_a = _ring_lonlat_rad(D, mu_a, n_pts=n_pts)
    lon_b, lat_b = _ring_lonlat_rad(D, mu_b, n_pts=n_pts)
    gap = 0.9 * np.pi
    # Mark steps that cross a seam on either edge (or between edges)
    bad = np.zeros(n_pts - 1, dtype=bool)
```

```
    for i in range(n_pts - 1):
        if abs(lon_a[i + 1] - lon_a[i]) > gap:
            bad[i] = True
        elif abs(lon_b[i + 1] - lon_b[i]) > gap:
            bad[i] = True
        elif abs(lon_a[i] - lon_b[i]) > gap:
            bad[i] = True
        elif abs(lon_a[i + 1] - lon_b[i + 1]) > gap:
            bad[i] = True
    # Group into continuous good runs → one polygon per run
    polys = []
    i = 0
    while i < n_pts - 1:
        while i < n_pts - 1 and bad[i]:
            i += 1
        if i >= n_pts - 1:
            break
        j = i
        while j < n_pts - 1 and not bad[j]:
            j += 1
        # indices i .. j inclusive are a continuous ribbon segment
        i0, i1 = i, j  # i1 is last good vertex index
        if i1 - i0 >= 1:
            xs = np.concatenate([lon_a[i0:i1 + 1], lon_b[i0:i1 + 1][::-1]])
            ys = np.concatenate([lat_a[i0:i1 + 1], lat_b[i0:i1 + 1][::-1]])
            polys.append(np.column_stack([xs, ys]))
        i = j + 1
    if polys:
        ax.add_collection(
            PolyCollection(
                polys, facecolors=color, edgecolors="none",
                alpha=0.26, zorder=2, linewidths=0,
            )
        )
    rl, rb = _ring_lb(D, mu_mid, n_pts=max(n_pts, 721))
    plot_lb_curve_mollweide(
        ax, rl, rb, color=color, lw=2.6, alpha=0.95, zorder=4
    )


def panel_B_mollweide(ax, geo):
    ax.set_title(
        r"B. Migkas $A(\ell)$ loops on sky (anti-CoE at $(0^\circ,+30^\circ)$)"
        "\n"
        r"(Galactic Mollweide; CoE axis tilted $+30^\circ$; $\ell$ increases LEFT; "
        r"two MAX + two MIN, each $18^\circ$ end-to-end ($\pm9^\circ$))",
        fontsize=10,
        pad=8,
    )
    for b in [-60, -30, 0, 30, 60]:
        ell = np.linspace(0, 360, 361)
        plot_lb_curve_mollweide(ax, ell, np.full_like(ell, b), color="#bdc3c7", lw=0.5, zorder=0)
    for ell0 in range(0, 360, 30):
        b = np.linspace(-90, 90, 181)
```

```
    plot_lb_curve_mollweide(ax, np.full_like(b, ell0), b, color="#bdc3c7", lw=0.5, zorder=0)

D = geo["D"]
ls, bs = great_circle_through_poles(D, n=721)
plot_lb_curve_mollweide(ax, ls, bs, color=C_AXIS, lw=2.0, zorder=2)

for (lo, hi), mid, color in zip(
    geo["max_bands_deg"], geo["theta_max_from_plus_x_deg"], COLORS_MAX
):
    fill_mollweide_ribbon(ax, D, lo, hi, mid, color)
for (lo, hi), mid, color in zip(
    geo["min_bands_deg"], geo["theta_min_from_plus_x_deg"], COLORS_MIN
):
    fill_mollweide_ribbon(ax, D, lo, hi, mid, color)

ell_eq, b_eq = celestial_equator_lb(721)
plot_lb_curve_mollweide(ax, ell_eq, b_eq, color=C_EQ, lw=1.8, ls="--", zorder=2)

ax.plot(
    np.deg2rad(lon_disp_from_ell(geo["anti_lb"][0])),
    np.deg2rad(geo["anti_lb"][1]),
    marker="o", ms=8, color=C_ANTI,
    markerfacecolor=C_ANTI, markeredgecolor="k", markeredgewidth=0.6,
    zorder=6, linestyle="None",
)
ax.plot(
    np.deg2rad(lon_disp_from_ell(geo["coe_lb"][0])),
    np.deg2rad(geo["coe_lb"][1]),
    marker="*", ms=14, color=C_COE,
    markerfacecolor=C_COE, markeredgecolor="k", markeredgewidth=0.6,
    zorder=6, linestyle="None",
)
for (l, b), color in zip(MIGKAS_MAXES, COLORS_MAX):
    ax.plot(
        np.deg2rad(lon_disp_from_ell(l)),
        np.deg2rad(b),
        marker="^",
        ms=10,
        color=color,
        markeredgecolor="k",
        markeredgewidth=0.7,
        zorder=7,
        linestyle="None",
    )
for (l, b), color in zip(MIGKAS_MINS, COLORS_MIN):
    ax.plot(
        np.deg2rad(lon_disp_from_ell(l)),
        np.deg2rad(b),
        marker="s",
        ms=10,
        color=color,
        markeredgecolor="k",
        markeredgewidth=0.7,
        zorder=7,
```

```
            linestyle="None",
        )

    xticks = np.deg2rad(lon_disp_from_ell(np.array([180, 90, 0, 270])))
    ax.set_xticks(xticks)
    ax.set_xticklabels([r"$180^\circ$", r"$90^\circ$", r"$0^\circ$", r"$270^\circ$"], fontsize=8)
    ax.set_yticks(np.deg2rad([-60, -30, 0, 30, 60]))
    ax.set_yticklabels([r"$-60^\circ$", r"$-30^\circ$", r"$0^\circ$", r"$+30^\circ$", r"$+60^\circ$"], fontsize=8)
    ax.grid(False)
    ax.set_xlabel(r"Galactic $\ell$ (increases LEFT)", fontsize=8)
    ax.set_ylabel(r"Galactic $b$", fontsize=8)


def fill_sphere_ribbon(ax, D, theta_lo, theta_hi, theta_mid, color, n_pts=160):
    """
    One 3D highlight belt: SINGLE transparent strip between ±9° edges
    + ONE solid mid-stroke. No linspace wash / stacked alpha hairball.
    """
    mu_a, mu_b = mu_pair_for_band(theta_lo, theta_hi)
    mu_mid = float(np.cos(np.deg2rad(theta_mid)))
    p0 = small_circle_about_axis(D, float(mu_a), n=n_pts)
    p1 = small_circle_about_axis(D, float(mu_b), n=n_pts)
    # Single belt between the two edge rings (one collection, no stacked wash)
    verts = [[p0[j], p0[j + 1], p1[j + 1], p1[j]] for j in range(n_pts - 1)]
    coll = Poly3DCollection(
        verts, facecolors=color, edgecolors="none", linewidths=0.0,
        alpha=0.25, zorder=2,
    )
    ax.add_collection3d(coll)
    pts = small_circle_about_axis(D, mu_mid, n=n_pts)
    ax.plot(
        pts[:, 0], pts[:, 1], pts[:, 2],
        color=color, lw=2.6, alpha=0.95, zorder=5,
    )


def panel_C_sphere(ax, geo):
    th_max = geo["theta_max_from_plus_x_deg"]
    th_min = geo["theta_min_from_plus_x_deg"]
    ax.set_title(
        r"C. Sphere 3D — two MAX + two MIN, each $18^\circ$ end-to-end ($\pm9^\circ$)"
        "\n"
        r"(axis line emphasized; celestial equator dashed)",
        fontsize=10,
        pad=6,
    )
    draw_wire_sphere(ax)
    D = geo["D"]
    ell_eq, b_eq = celestial_equator_lb(400)
    eq_xyz = np.array([lb_to_n(l, b) for l, b in zip(ell_eq, b_eq)])
    ax.plot(eq_xyz[:, 0], eq_xyz[:, 1], eq_xyz[:, 2], color=C_EQ, lw=1.8, ls="--", zorder=3)
    ax.plot(
        [-D[0], D[0]],
        [-D[1], D[1]],
```

```
        [-D[2], D[2]],
        color=C_AXIS,
        lw=3.0,
        zorder=5,
    )
    ax.scatter(*(-D), color=C_COE, s=120, marker="*", edgecolors="k", linewidths=0.6,
               depthshade=False, zorder=8)
    ax.scatter(*(D), color=C_ANTI, s=80, edgecolors="k", linewidths=0.6, depthshade=False, zorder=8)
    ax.text(*(-D * 1.22), "CoE", color=C_COE, fontsize=8, fontweight="bold")
    ax.text(*(D * 1.18), "anti-CoE", color=C_ANTI, fontsize=8, fontweight="bold")
    ax.scatter([0], [0], [0], color="k", s=25, depthshade=False, zorder=9)

    for (lo, hi), mid, color in zip(
        geo["max_bands_deg"], geo["theta_max_from_plus_x_deg"], COLORS_MAX
    ):
        fill_sphere_ribbon(ax, D, lo, hi, mid, color)
    for (lo, hi), mid, color in zip(
        geo["min_bands_deg"], geo["theta_min_from_plus_x_deg"], COLORS_MIN
    ):
        fill_sphere_ribbon(ax, D, lo, hi, mid, color)

    for n, color in zip(geo["n_maxes"], COLORS_MAX):
        ax.scatter(
            [n[0]], [n[1]], [n[2]],
            c=[color], marker="^", s=70,
            edgecolors="k", linewidths=0.7, depthshade=False, zorder=10,
        )
        ax.plot([0, n[0]], [0, n[1]], [0, n[2]], color=color, lw=0.8, ls=":", alpha=0.7)
    for n, color in zip(geo["n_mins"], COLORS_MIN):
        ax.scatter(
            [n[0]], [n[1]], [n[2]],
            c=[color], marker="s", s=60,
            edgecolors="k", linewidths=0.7, depthshade=False, zorder=10,
        )
        ax.plot([0, n[0]], [0, n[1]], [0, n[2]], color=color, lw=0.8, ls=":", alpha=0.7)

    ax.set_xlim(-1.25, 1.25)
    ax.set_ylim(-1.25, 1.25)
    ax.set_zlim(-1.25, 1.25)
    ax.set_box_aspect((1, 1, 1))
    ax.view_init(elev=18, azim=-55)
    ax.set_axis_off()
    ax.text2D(
        0.5,
        -0.02,
        rf"MAX $\theta\!=\!{th_max[0]:.0f}^\circ,{th_max[1]:.0f}^\circ$; "
        rf"MIN $\theta\!=\!{th_min[0]:.0f}^\circ,{th_min[1]:.0f}^\circ$  "
        rf"($18^\circ$ end-to-end = $\pm9^\circ$ = 5% of $360^\circ$)",
        transform=ax.transAxes,
        ha="center",
        fontsize=7.5,
        color="#34495e",
    )
```

```
def main():
    st = migkas_scan_stops()
    geo = compute_geometry(st)

    th_max = geo["theta_max_from_plus_x_deg"]
    th_min = geo["theta_min_from_plus_x_deg"]
    max_bands = geo["max_bands_deg"]
    min_bands = geo["min_bands_deg"]

    fig = plt.figure(figsize=(14.5, 5.6), dpi=160)
    fig.suptitle(
        r"Pure CoE axisymmetry: two MAX + two MIN highlight bands"
        "\n"
        rf"(each $18^\circ$ end-to-end = $\pm9^\circ$ = 5% of $360^\circ$; "
        rf"MAX $\theta\!=\!{th_max[0]:.0f}^\circ/{th_max[1]:.0f}^\circ$; "
        rf"MIN $\theta\!=\!{th_min[0]:.0f}^\circ/{th_min[1]:.0f}^\circ$; "
        r"anti-CoE $(0^\circ,+30^\circ)$, CoE axis tilted $+30^\circ$)",
        fontsize=11,
        fontweight="bold",
        y=1.02,
    )

    axA = fig.add_subplot(1, 3, 1)
    panel_A_model_frame(axA, geo)

    axB = fig.add_subplot(1, 3, 2, projection="mollweide")
    panel_B_mollweide(axB, geo)

    axC = fig.add_subplot(1, 3, 3, projection="3d")
    panel_C_sphere(axC, geo)

    handles = [
        Line2D([0], [0], color=C_AXIS, lw=2.5, label=r"CoE$\leftrightarrow$anti-CoE axis"),
        Patch(
            facecolor=C_MAX_30, alpha=0.45, edgecolor=C_MAX_30,
            label=(
                rf"MAX $\theta\!=\!{th_max[0]:.0f}^\circ$ band "
                rf"[{max_bands[0][0]:.0f},{max_bands[0][1]:.0f}]$^\circ$ "
                rf"($18^\circ$=±$9^\circ$=5% of $360^\circ$)"
            ),
        ),
        Patch(
            facecolor=C_MAX_185, alpha=0.45, edgecolor=C_MAX_185,
            label=(
                rf"MAX $\theta\!=\!{th_max[1]:.0f}^\circ$ band "
                rf"[{max_bands[1][0]:.0f},{max_bands[1][1]:.0f}]$^\circ$ "
                rf"($18^\circ$=±$9^\circ$)"
            ),
        ),
        Patch(
            facecolor=C_MIN_115, alpha=0.45, edgecolor=C_MIN_115,
            label=(
                rf"MIN $\theta\!=\!{th_min[0]:.0f}^\circ$ band "
```

```
            rf"[{min_bands[0][0]:.0f},{min_bands[0][1]:.0f}]$^\circ$ "
            rf"($18^\circ$=±$9^\circ$)"
        ),
    ),
    Patch(
        facecolor=C_MIN_265, alpha=0.45, edgecolor=C_MIN_265,
        label=(
            rf"MIN $\theta\!=\!{th_min[1]:.0f}^\circ$ band "
            rf"[{min_bands[1][0]:.0f},{min_bands[1][1]:.0f}]$^\circ$ "
            rf"($18^\circ$=±$9^\circ$)"
        ),
    ),
    Line2D([0], [0], color=C_EQ, lw=1.8, ls="--", label="celestial equator (Dec$=0$)"),
    Line2D(
        [0], [0], marker="^", color="w", markerfacecolor=C_MAX_30, markeredgecolor="k",
        ms=9,
        label=rf"MAX $({MIGKAS_MAXES[0][0]:.0f}^\circ,{MIGKAS_MAXES[0][1]:+.0f}^\circ)$",
    ),
    Line2D(
        [0], [0], marker="^", color="w", markerfacecolor=C_MAX_185, markeredgecolor="k",
        ms=9,
        label=rf"MAX $({MIGKAS_MAXES[1][0]:.0f}^\circ,{MIGKAS_MAXES[1][1]:+.0f}^\circ)$",
    ),
    Line2D(
        [0], [0], marker="s", color="w", markerfacecolor=C_MIN_115, markeredgecolor="k",
        ms=8,
        label=rf"MIN $({MIGKAS_MINS[0][0]:.0f}^\circ,{MIGKAS_MINS[0][1]:+.0f}^\circ)$",
    ),
    Line2D(
        [0], [0], marker="s", color="w", markerfacecolor=C_MIN_265, markeredgecolor="k",
        ms=8,
        label=rf"MIN $({MIGKAS_MINS[1][0]:.0f}^\circ,{MIGKAS_MINS[1][1]:+.0f}^\circ)$",
    ),
    Line2D(
        [0], [0], marker="o", color="w", markerfacecolor=C_ANTI, markeredgecolor="k",
        ms=8, label=rf"anti-CoE $({geo['anti_lb'][0]:.1f}^\circ,{geo['anti_lb'][1]:+.1f}^\circ)$",
    ),
    Line2D(
        [0], [0], marker="*", color="w", markerfacecolor=C_COE, markeredgecolor="k",
        ms=11, label=rf"CoE $({geo['coe_lb'][0]:.1f}^\circ,{geo['coe_lb'][1]:+.1f}^\circ)$",
    ),
]
fig.legend(
    handles=handles,
    loc="lower center",
    ncol=6,
    fontsize=7.0,
    frameon=True,
    fancybox=True,
    bbox_to_anchor=(0.5, -0.10),
)

fig.tight_layout()
fig.savefig(PNG_PATH, bbox_inches="tight", facecolor="white")
```

```
    plt.close(fig)

    payload = {
        "png": str(PNG_PATH),
        "script": str(OUT_DIR / "make_coe_rings_migkas_pin_axis.py"),
        "migkas_max_lb_deg": [list(lb) for lb in MIGKAS_MAXES],
        "migkas_min_lb_deg": [list(lb) for lb in MIGKAS_MINS],
        "strip_theta_max_deg": list(STRIP_THETA_MAX),
        "strip_theta_min_deg": list(STRIP_THETA_MIN),
        "band_half_width_deg": BAND_HALF,
        "band_full_width_deg": BAND_FULL,
        "band_total_width_deg": BAND_TOTAL,
        "band_width_note": "5% of 360° = 18° end-to-end = ±9° about each MAX/MIN center",
        "max_bands_deg": [list(b) for b in max_bands],
        "min_bands_deg": [list(b) for b in min_bands],
        "band_intervals": {
            "MAX_30": [float(max_bands[0][0]), float(max_bands[0][1])],
            "MAX_185": [float(max_bands[1][0]), float(max_bands[1][1])],
            "MIN_115": [float(min_bands[0][0]), float(min_bands[0][1])],
            "MIN_265": [float(min_bands[1][0]), float(min_bands[1][1])],
        },
        "scan_stop_source": st["source"],
        "theta_max_from_plus_x_deg": [round(t, 4) for t in th_max],
        "theta_min_from_plus_x_deg": [round(t, 4) for t in th_min],
        "mu_max_bands": [[round(a, 8), round(b, 8)] for a, b in geo["mu_max_bands"]],
        "mu_min_bands": [[round(a, 8), round(b, 8)] for a, b in geo["mu_min_bands"]],
        "CoE_lb_deg": [round(geo["coe_lb"][0], 4), round(geo["coe_lb"][1], 4)],
        "anti_CoE_lb_deg": [round(geo["anti_lb"][0], 4), round(geo["anti_lb"][1], 4)],
        "D_hat_galactic_xyz": geo["D"].tolist(),
        "angular_sep_migkas_max0_min1_deg": round(geo["sep_deg"], 6),
        "axis_tilt_b_deg": 30,
        "axis_tilt_reason": (
            "Chris 2026-09-22 — markers fixed, rings rotate up 30°"
        ),
        "orientation": (
            "Galactic orientation: anti-CoE at (ell=0°, b=+30°), CoE at "
            "(ell=180°, b=−30°); +D = anti-CoE = +x; axis tilted +30° "
            "(Chris 2026-09-22 — markers fixed, rings rotate up 30°)"
        ),
        "drawing": (
            "Exactly 4 features. Each = ONE smooth transparent highlight band "
            "between θ−9° and θ+9° (18° end-to-end = 5% of 360°) + ONE solid "
            "mid-stroke at the exact center θ. No multi-stroke wash / hairball. "
            "Four distinct colors (two reds + two blues); no orange; no 283° "
            "companion. Panel A: clean vertical strip clipped to unit circle; theta rays from observer to each loop (emphasized at MAX 30°). Mollweide
"
            "seam-split honestly. Sphere: single Poly3D strip (no n_wash stack)."
        ),
        "note": (
            "Stops are after-2ℓ A(ℓ) extrema from best_fit_2ell_multiplier.npz "
            f"A_after: two MAX at θ={th_max[0]:.0f}°/{th_max[1]:.0f}° and "
            f"two MIN at θ={th_min[0]:.0f}°/{th_min[1]:.0f}°, each drawn as "
            f"one {BAND_FULL:.0f}° end-to-end highlight (±{BAND_HALF:.0f}° = 5% of 360°). "
            "Rings use μ=cos(θ); sky markers are map extrema (17°,−9°) and "
```

```
            "(274°,−22°) plus strip stand-ins (185°,0°) and (115°,0°)."
        ),
        "plot_convention": "Migkas Galactic Mollweide: lon_disp=-ell_wrapped (ell increases LEFT)",
        "legend_summary": (
            "two MAX + two MIN highlight bands, each 18° end-to-end "
            "(±9° = 5% of 360°); four distinct colors"
        ),
        "colors": {
            "MAX_30": C_MAX_30,
            "MAX_185": C_MAX_185,
            "MIN_115": C_MIN_115,
            "MIN_265": C_MIN_265,
            "no_orange": True,
        },
    }
    JSON_PATH.write_text(json.dumps(payload, indent=2) + "\n")
    print(json.dumps(payload, indent=2))
    print(f"\nWrote {PNG_PATH}")
    print(f"Wrote {JSON_PATH}")


if __name__ == "__main__":
    main()
```

```
{
  "png": "/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/coe_rings_migkas_pin_axis.png",
  "script": "/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/make_coe_rings_migkas_pin_axis.py",
  "migkas_max_lb_deg": [
    [
      17.0,
      -9.0
    ],
    [
      185.0,
      0.0
    ]
  ],
  "migkas_min_lb_deg": [
    [
      115.0,
      0.0
    ],
    [
      274.0,
      -22.0
    ]
  ],
  "strip_theta_max_deg": [
    30.0,
    185.0
  ],
  "strip_theta_min_deg": [
    115.0,
    265.0
  ],
  "band_half_width_deg": 9.0,
  "band_full_width_deg": 18.0,
  "band_total_width_deg": 18.0,
  "band_width_note": "5% of 360\u00b0 = 18\u00b0 end-to-end = \u00b19\u00b0 about each MAX/MIN center",
  "max_bands_deg": [
    [
      21.0,
      39.0
    ],
    [
      176.0,
      194.0
    ]
  ],
  "min_bands_deg": [
    [
      106.0,
      124.0
    ],
    [
      256.0,
      274.0
    ]
  ],
  "band_intervals": {
    "MAX_30": [
```

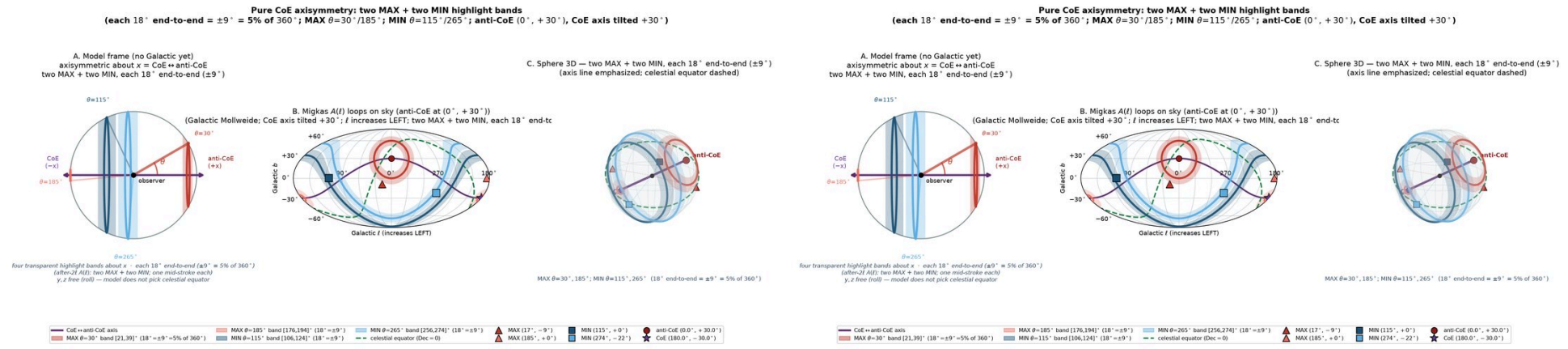
```
      21.0,
      39.0
    ],
    "MAX_185": [
      176.0,
      194.0
    ],
    "MIN_115": [
      106.0,
      124.0
    ],
    "MIN_265": [
      256.0,
      274.0
    ]
  },
  "scan_stop_source": "best_fit_2ell_multiplier.npz A_after extrema (two max, two min); each drawn as one transparent 18\u00b0 end-to-end highlight
(\u00b19\u00b0 = 5% of 360\u00b0) + one solid mid-stroke",
  "theta_max_from_plus_x_deg": [
    30.0,
    185.0
  ],
  "theta_min_from_plus_x_deg": [
    115.0,
    265.0
  ],
  "mu_max_bands": [
    [
      0.93358043,
      0.77714596
    ],
    [
      -0.99756405,
      -0.97029573
    ]
  ],
  "mu_min_bands": [
    [
      -0.27563736,
      -0.5591929
    ],
    [
      -0.2419219,
      0.06975647
    ]
  ],
  "CoE_lb_deg": [
    180.0,
    -30.0
  ],
  "anti_CoE_lb_deg": [
    0.0,
    30.0
  ],
  "D_hat_galactic_xyz": [
    0.8660254037844387,
    0.0,
```

```
    0.49999999999999994
  ],
  "angular_sep_migkas_max0_min1_deg": 98.476384,
  "axis_tilt_b_deg": 30,
  "axis_tilt_reason": "Chris 2026-09-22 \u2014 markers fixed, rings rotate up 30\u00b0",
  "orientation": "Galactic orientation: anti-CoE at (ell=0\u00b0, b=+30\u00b0), CoE at (ell=180\u00b0, b=\u221230\u00b0); +D = anti-CoE = +x; axis
tilted +30\u00b0 (Chris 2026-09-22 \u2014 markers fixed, rings rotate up 30\u00b0)",
  "drawing": "Exactly 4 features. Each = ONE smooth transparent highlight band between \u03b8\u22129\u00b0 and \u03b8+9\u00b0 (18\u00b0 end-to-end = 5%
of 360\u00b0) + ONE solid mid-stroke at the exact center \u03b8. No multi-stroke wash / hairball. Four distinct colors (two reds + two blues); no
orange; no 283\u00b0 companion. Panel A: clean vertical strip clipped to unit circle; theta rays from observer to each loop (emphasized at MAX
30\u00b0). Mollweide seam-split honestly. Sphere: single Poly3D strip (no n_wash stack).",
  "note": "Stops are after-2\u2113 A(\u2113) extrema from best_fit_2ell_multiplier.npz A_after: two MAX at \u03b8=30\u00b0/185\u00b0 and two MIN at
\u03b8=115\u00b0/265\u00b0, each drawn as one 18\u00b0 end-to-end highlight (\u00b19\u00b0 = 5% of 360\u00b0). Rings use \u03bc=cos(\u03b8); sky
markers are map extrema (17\u00b0,\u22129\u00b0) and (274\u00b0,\u221222\u00b0) plus strip stand-ins (185\u00b0,0\u00b0) and (115\u00b0,0\u00b0).",
  "plot_convention": "Migkas Galactic Mollweide: lon_disp=-ell_wrapped (ell increases LEFT)",
  "legend_summary": "two MAX + two MIN highlight bands, each 18\u00b0 end-to-end (\u00b19\u00b0 = 5% of 360\u00b0); four distinct colors",
  "colors": {
    "MAX_30": "#c0392b",
    "MAX_185": "#ec7063",
    "MIN_115": "#1a5276",
    "MIN_265": "#5dade2",
    "no_orange": true
  }
}

Wrote /workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/coe_rings_migkas_pin_axis.png
Wrote /workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/coe_rings_migkas_pin_axis.json
```


```
compare(FIGS / 'coe_rings_migkas_pin_axis.png', '/workspace/cosmology/figures/coe_rings_migkas_pin_axis.png')
```

```
coe_rings_migkas_pin_axis.png: new (2283, 1027) vs ref (2283, 1027) | mean |diff| = 0.000/255, pixels differing >40: 0.000% -> identical
left: approved reference figures/coe_rings_migkas_pin_axis.png   |   right: notebook re-run steps_v3/figs/coe_rings_migkas_pin_axis.png
```



Step 11 — The Step 10 loops overlaid on Migkas Fig. 8

What it shows. The four MAX/MIN bands of Step 10 (axis $+30^\circ$) drawn on the real Migkas Fig. 8 $\theta = 75^\circ$ map, in high-contrast colours against magma (cyan, hot pink, lime, sky blue), with the published MAX triangles and MIN squares. Drawing is clipped to the coloured oval.

Source script: `/workspace/cosmology/figures/make_migkas_fig8_coe_rings_overlay.py` (last modified 2026-09-25 20:22 CT), run verbatim. *Output-path edit:* none. The script's `OUT_DIR` follows `__file__`, which is set to `steps_v3/figs/`. *Reference image:* `figures/migkas_fig8_theta75_with_coe_rings.png`.

```
%%runscript make_migkas_fig8_coe_rings_overlay.py
#!/usr/bin/env python3
"""
Superimpose CoE MAX/MIN highlight bands + Migkas markers onto the REAL Migkas
Fig. 8 theta=75 A/A_all Mollweide map.

Matches make_coe_rings_migkas_pin_axis.py (three-panel Panel B) geometry:
  Exactly FOUR features — each ONE transparent highlight band, 18° end-to-end
  (= ±9° = 5% of 360°) about the center θ + ONE solid mid-stroke:
    MAX 30°  → [21°, 39°]   color #00e5ff (bright cyan)
    MAX 185° → [176°, 194°] color #ff2d95 (magenta/hot pink)
    MIN 115° → [106°, 124°] color #b8f200 (lime)
    MIN 265° → [256°, 274°] color #4fc3f7 (sky blue)
  Drawing: clean PolyCollection fill + one mid-stroke (optional thin white
  underlay on mid only). NO black rim, NO thick white halo stacks, NO dual
  boundary-edge strokes. No 283° companion. No Dec=0 overlay.
  MAX triangles: (17°, −9°) and (185°, 0°)
  MIN squares:   (115°, 0°) and (274°, −22°)
  CoE axis tilted +30°: anti-CoE (0°,+30°), CoE (180°,−30°) (purple).

Registration (unchanged from working Fig. 8):
  GOOD_REG = (cx, cy, Rx, Ry) = (863.0, 763.25, 693.9927, 392.542425)
  Source: FULL_migkas_theta75.png
  CoE axis tilted +30°; clip/inset so overlays stay inside colored oval
  lon_disp = -wrap(ℓ)

Outputs:
  migkas_fig8_theta75_with_coe_rings.png
  migkas_fig8_theta75_with_coe_rings.json
"""

from __future__ import annotations

import json
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
from matplotlib.collections import PolyCollection
from matplotlib.lines import Line2D
from matplotlib.patches import Ellipse, Patch
from PIL import Image
```

```
OUT_DIR = Path(__file__).resolve().parent
PNG_PATH = OUT_DIR / "migkas_fig8_theta75_with_coe_rings.png"
JSON_PATH = OUT_DIR / "migkas_fig8_theta75_with_coe_rings.json"

SOURCE_CANDIDATES = [
    Path("/workspace/cosmology/map_compare/FULL_migkas_theta75.png"),
    Path("/workspace/cosmology/map_compare/migkas_fig8_theta75_crop.png"),
    Path("/workspace/cosmology/migkas/pub_maps/Fig8_full_hires.png"),
]

# Strip A(ℓ) scan stops from +x (after-2ℓ A_after extrema) — four band centers.
# 5% of 360° = 18° end-to-end → half-width ±9° about each center (match three-panel).
BAND_HALF = 9.0
BAND_FULL = 2.0 * BAND_HALF  # 18°
THETA_MAX = [30.0, 185.0]
THETA_MIN = [115.0, 265.0]
MAX_BANDS = [(t - BAND_HALF, t + BAND_HALF) for t in THETA_MAX]  # [21,39], [176,194]
MIN_BANDS = [(t - BAND_HALF, t + BAND_HALF) for t in THETA_MIN]  # [106,124], [256,274]


def mu_pair_for_band(lo, hi):
    """Edge μ=cos(θ) for a scan-angle band (μ-space, not θ-space)."""
    return float(np.cos(np.deg2rad(lo))), float(np.cos(np.deg2rad(hi)))

# Visually-good registration (pre medial-axis retune). Hard-set as working default.
GOOD_REG = {
    "cx": 863.0,
    "cy": 763.25,
    "Rx": 693.9927,
    "Ry": 392.542425,
}

# Sky markers (ℓ, b): map extrema + strip stand-ins (same as 3-panel Panel B).
MIGKAS_MAXES = [(17.0, -9.0), (185.0, 0.0)]
MIGKAS_MINS = [(115.0, 0.0), (274.0, -22.0)]

# Galactic pose (Chris 2026-09-22): anti-CoE (+D/+x) at (0°, +30°); CoE at (180°, −30°)
ANTI_COE_LB = (0.0, 30.0)
COE_LB = (180.0, -30.0)
AXIS_TILT_B_DEG = 30.0


def lb_to_n(l_deg, b_deg):
    l = np.deg2rad(l_deg)
    b = np.deg2rad(b_deg)
    return np.array(
        [np.cos(b) * np.cos(l), np.cos(b) * np.sin(l), np.sin(b)], dtype=float
    )


D_HAT = lb_to_n(*ANTI_COE_LB)  # +D = anti-CoE = +x


def great_circle_through_poles(D, n=721):
```

```
    """Great circle through ±D (CoE axis meridian on the sky)."""
    a = D / np.linalg.norm(D)
    tmp = np.array([0.0, 0.0, 1.0])
    if abs(np.dot(a, tmp)) > 0.9:
        tmp = np.array([1.0, 0.0, 0.0])
    e2 = np.cross(a, tmp)
    e2 /= np.linalg.norm(e2)
    t = np.linspace(0.0, 2.0 * np.pi, n)
    pts = np.outer(np.cos(t), a) + np.outer(np.sin(t), e2)
    ls = np.array([n_to_lb(p)[0] for p in pts])
    bs = np.array([n_to_lb(p)[1] for p in pts])
    return ls, bs

# High-contrast loop colors that pop on magma (avoid reds/oranges/corals)
C_MAX_30 = "#00e5ff"    # bright cyan — MAX at 30°
C_MAX_185 = "#ff2d95"   # magenta/hot pink — MAX at 185°
C_MIN_115 = "#b8f200"   # lime — MIN at 115°
C_MIN_265 = "#4fc3f7"   # sky blue — MIN at 265°
COLORS_MAX = [C_MAX_30, C_MAX_185]
COLORS_MIN = [C_MIN_115, C_MIN_265]
C_COE = "#6b3fa0"
C_ANTI = "#8e1b1b"  # dark red marker; distinct from loop reds (match three-panel)
C_AXIS = "#5b2c6f"
C_EQ = "#1e8449"


def wrap_lon_deg(lon):
    return ((np.asarray(lon, dtype=float) + 180.0) % 360.0) - 180.0


def lon_disp_from_ell(ell_deg):
    return -wrap_lon_deg(ell_deg)


def n_to_lb(n):
    n = np.asarray(n, dtype=float)
    n = n / np.linalg.norm(n)
    b = float(np.rad2deg(np.arcsin(np.clip(n[2], -1.0, 1.0))))
    l = float(np.rad2deg(np.arctan2(n[1], n[0])) % 360.0)
    return l, b


def celestial_equator_lb(n=2500):
    ra_ngp = np.deg2rad(192.85948)
    dec_ngp = np.deg2rad(27.12825)
    l_ncp = np.deg2rad(122.93192)
    ra = np.linspace(0.0, 2.0 * np.pi, n, endpoint=False)
    dec = 0.0
    sin_b = np.clip(
        np.sin(dec) * np.sin(dec_ngp)
        + np.cos(dec) * np.cos(dec_ngp) * np.cos(ra - ra_ngp),
        -1.0,
        1.0,
    )
```

```
    b = np.arcsin(sin_b)
    y = np.cos(dec) * np.sin(ra - ra_ngp)
    x = (
        np.sin(dec) * np.cos(dec_ngp)
        - np.cos(dec) * np.sin(dec_ngp) * np.cos(ra - ra_ngp)
    )
    l = (l_ncp - np.arctan2(y, x)) % (2.0 * np.pi)
    return np.rad2deg(l), np.rad2deg(b)


def small_circle_about_axis(axis, mu, n=721):
    a = axis / np.linalg.norm(axis)
    mu = float(np.clip(mu, -1.0 + 1e-9, 1.0 - 1e-9))
    tmp = np.array([0.0, 0.0, 1.0]) if abs(a[2]) < 0.9 else np.array([1.0, 0.0, 0.0])
    e1 = np.cross(a, tmp)
    e1 /= np.linalg.norm(e1)
    e2 = np.cross(a, e1)
    rho = np.sqrt(max(0.0, 1.0 - mu * mu))
    t = np.linspace(0.0, 2.0 * np.pi, n)
    return (mu * a)[None, :] + rho * (
        np.outer(np.cos(t), e1) + np.outer(np.sin(t), e2)
    )


def mollweide_xy(lon_rad, lat_rad, n_iter=64):
    """Standard Mollweide; stable bracketed solve of 2θ+sin(2θ)=π sin(lat)."""
    lon = np.asarray(lon_rad, dtype=float)
    lat = np.asarray(lat_rad, dtype=float)
    target = np.pi * np.sin(np.clip(lat, -np.pi / 2, np.pi / 2))
    lo = np.full_like(lat, -np.pi / 2, dtype=float)
    hi = np.full_like(lat, np.pi / 2, dtype=float)
    for _ in range(n_iter):
        mid = 0.5 * (lo + hi)
        g = 2.0 * mid + np.sin(2.0 * mid)
        lo = np.where(g < target, mid, lo)
        hi = np.where(g >= target, mid, hi)
    theta = 0.5 * (lo + hi)
    theta = np.where(lat >= np.pi / 2 - 1e-15, np.pi / 2, theta)
    theta = np.where(lat <= -np.pi / 2 + 1e-15, -np.pi / 2, theta)
    x = (2.0 * np.sqrt(2.0) / np.pi) * lon * np.cos(theta)
    y = np.sqrt(2.0) * np.sin(theta)
    return x, y


def lb_to_px(ell_deg, b_deg, cx, cy, Rx, Ry):
    lon = np.deg2rad(lon_disp_from_ell(np.asarray(ell_deg, dtype=float)))
    lat = np.deg2rad(np.asarray(b_deg, dtype=float))
    x, y = mollweide_xy(lon, lat)
    Xn = x / (2.0 * np.sqrt(2.0))
    Yn = y / np.sqrt(2.0)
    return cx + Xn * Rx, cy - Yn * Ry


def as_xy(px, py):
```

```
    return float(np.asarray(px).reshape(-1)[0]), float(np.asarray(py).reshape(-1)[0])


def chamfer_distance_to_dark(dark):
    h, w = dark.shape
    INF = 1e6
    D = np.where(dark, 0.0, INF)
    for y in range(h):
        for x in range(w):
            v = D[y, x]
            if x > 0:
                v = min(v, D[y, x - 1] + 1.0)
            if y > 0:
                v = min(v, D[y - 1, x] + 1.0)
            if x > 0 and y > 0:
                v = min(v, D[y - 1, x - 1] + 1.414)
            if x + 1 < w and y > 0:
                v = min(v, D[y - 1, x + 1] + 1.414)
            D[y, x] = v
    for y in range(h - 1, -1, -1):
        for x in range(w - 1, -1, -1):
            v = D[y, x]
            if x + 1 < w:
                v = min(v, D[y, x + 1] + 1.0)
            if y + 1 < h:
                v = min(v, D[y + 1, x] + 1.0)
            if x + 1 < w and y + 1 < h:
                v = min(v, D[y + 1, x + 1] + 1.414)
            if x > 0 and y + 1 < h:
                v = min(v, D[y + 1, x - 1] + 1.414)
            D[y, x] = v
    return D


def detect_oval_bbox(rgb):
    h, w, _ = rgb.shape
    white = (rgb[:, :, 0] > 250) & (rgb[:, :, 1] > 250) & (rgb[:, :, 2] > 250)
    oval_left = np.full(h, np.nan)
    oval_right = np.full(h, np.nan)
    for y in range(h):
        row = ~white[y]
        d = np.diff(np.r_[0, row.astype(int), 0])
        starts = np.where(d == 1)[0]
        ends = np.where(d == -1)[0] - 1
        if len(starts) == 0:
            continue
        lengths = ends - starts + 1
        candidates = [
            (int(lengths[i]), int(starts[i]), int(ends[i]))
            for i in range(len(starts))
            if starts[i] < 0.72 * w and lengths[i] > 40
        ]
        if not candidates:
            continue
```

```
        candidates.sort(reverse=True)
        _, L, R = candidates[0]
        oval_left[y] = L
        oval_right[y] = R
    widths = np.where(np.isfinite(oval_left), oval_right - oval_left, 0.0)
    yeq = int(np.argmax(widths))
    thresh = 0.08 * widths[yeq]
    y0 = yeq
    while y0 > 0 and widths[y0 - 1] >= thresh:
        y0 -= 1
    y1 = yeq
    while y1 < h - 1 and widths[y1 + 1] >= thresh:
        y1 += 1
    x0 = float(oval_left[yeq])
    x1 = float(oval_right[yeq])
    return {
        "x0": int(x0),
        "x1": int(x1),
        "y0": int(y0),
        "y1": int(y1),
        "yeq": int(yeq),
        "cx": 0.5 * (x0 + x1),
        "cy": 0.5 * (y0 + y1),
        "Rx": 0.5 * (x1 - x0),
        "Ry": 0.5 * (y1 - y0 + 1),
    }


def refine_registration(rgb, seed):
    """Gray/dark Dec=0 lock (pre medial-axis). Prefer GOOD_REG hard-set in main."""
    h, w, _ = rgb.shape
    dark = rgb.mean(axis=2) < 55
    dist = chamfer_distance_to_dark(dark)

    ell_eq, b_eq = celestial_equator_lb(2500)
    lon = np.deg2rad(lon_disp_from_ell(ell_eq))
    lat = np.deg2rad(b_eq)
    mx, my = mollweide_xy(lon, lat)
    Xn = mx / (2.0 * np.sqrt(2.0))
    Yn = my / np.sqrt(2.0)

    def median_dist(cx, cy, Rx, Ry):
        px = cx + Xn * Rx
        py = cy - Yn * Ry
        m = (px >= 1) & (px < w - 1) & (py >= 1) & (py < h - 1)
        xi = np.clip(np.round(px[m]).astype(int), 0, w - 1)
        yi = np.clip(np.round(py[m]).astype(int), 0, h - 1)
        return float(np.median(dist[yi, xi]))

    base = {k: float(seed[k]) for k in ("cx", "cy", "Rx", "Ry")}
    best_sc = median_dist(**base)
    best = dict(base)
    for dcx in np.linspace(-10, 10, 21):
        for dcy in np.linspace(-10, 10, 21):
```

```
            for sRx in (0.98, 0.99, 1.0, 1.01, 1.02):
                for sRy in (0.98, 0.99, 1.0, 1.01, 1.02):
                    p = dict(
                        cx=base["cx"] + dcx,
                        cy=base["cy"] + dcy,
                        Rx=base["Rx"] * sRx,
                        Ry=base["Ry"] * sRy,
                    )
                    sc = median_dist(**p)
                    if sc < best_sc:
                        best_sc = sc
                        best = p
    b = best
    for dcx in np.linspace(-2, 2, 17):
        for dcy in np.linspace(-2, 2, 17):
            for sRx in np.linspace(0.995, 1.005, 11):
                for sRy in np.linspace(0.995, 1.005, 11):
                    p = dict(
                        cx=b["cx"] + dcx,
                        cy=b["cy"] + dcy,
                        Rx=b["Rx"] * sRx,
                        Ry=b["Ry"] * sRy,
                    )
                    sc = median_dist(**p)
                    if sc < best_sc:
                        best_sc = sc
                        best = p
    best["ceq_median_px"] = float(best_sc)
    return best


def choose_source():
    for p in SOURCE_CANDIDATES:
        if p.is_file():
            return p
    raise FileNotFoundError("No Migkas Fig.8 theta=75 source image found")


def sample_rgb(rgb, ell, b, cx, cy, Rx, Ry, r=4):
    px, py = lb_to_px(ell, b, cx, cy, Rx, Ry)
    xi, yi = int(round(as_xy(px, py)[0])), int(round(as_xy(px, py)[1]))
    patch = rgb[max(0, yi - r) : yi + r + 1, max(0, xi - r) : xi + r + 1]
    return patch.mean(axis=(0, 1))


def validate_landmarks(rgb, reg):
    cx, cy, Rx, Ry = reg["cx"], reg["cy"], reg["Rx"], reg["Ry"]
    pts = {}
    landmarks = [
        ("center", 0.0, 0.0),
        ("ell90", 90.0, 0.0),
        ("ell270", 270.0, 0.0),
        ("north", 0.0, 90.0),
        ("south", 0.0, -90.0),
```

```
    ]
    for i, (ell, b) in enumerate(MIGKAS_MAXES):
        landmarks.append((f"migkas_max{i}", ell, b))
    for i, (ell, b) in enumerate(MIGKAS_MINS):
        landmarks.append((f"migkas_min{i}", ell, b))
    for name, ell, b in landmarks:
        x, y = as_xy(*lb_to_px(ell, b, cx, cy, Rx, Ry))
        pts[name] = {"ell": ell, "b": b, "px": x, "py": y}

    assert pts["ell90"]["px"] < pts["center"]["px"], "l=90 must be LEFT of center"
    assert pts["ell270"]["px"] > pts["center"]["px"], "l=270 must be RIGHT of center"
    assert abs(pts["center"]["px"] - cx) < 1.5
    assert abs(pts["center"]["py"] - cy) < 1.5
    assert pts["north"]["py"] < pts["center"]["py"]
    assert pts["south"]["py"] > pts["center"]["py"]

    # Map-extrema pair: (17,-9) should sit in a warmer patch than (274,-22).
    cmax = sample_rgb(rgb, *MIGKAS_MAXES[0], cx, cy, Rx, Ry)
    cmin = sample_rgb(rgb, *MIGKAS_MINS[1], cx, cy, Rx, Ry)
    assert (cmax[0] + cmax[1]) > (cmin[0] + cmin[1]) + 50
    return {
        "landmarks_px": pts,
        "migkas_max0_rgb_mean": [float(x) for x in cmax],
        "migkas_min1_rgb_mean": [float(x) for x in cmin],
    }


def inside_ellipse(px, py, ecx, ecy, eRx, eRy, r_lim=1.0):
    """Boolean mask: points inside axis-aligned ellipse (image y down)."""
    px = np.asarray(px, dtype=float)
    py = np.asarray(py, dtype=float)
    Xn = (px - ecx) / eRx
    Yn = (py - ecy) / eRy
    return Xn * Xn + Yn * Yn <= float(r_lim) * float(r_lim)


def plot_lb_curve(
    ax, ell, b, cx, cy, Rx, Ry, inset=1.0, jump_px=45.0,
    clip_r2=None, clip_path=None, color_oval=None, **kw
):
    """Draw curve in oval; inset<1 keeps strokes inside the published Mollweide edge."""
    px, py = lb_to_px(ell, b, cx, cy, Rx, Ry)
    px = np.asarray(px, dtype=float).reshape(-1)
    py = np.asarray(py, dtype=float).reshape(-1)
    # Pull points slightly toward oval center so thick strokes do not spill past the rim.
    px = cx + (px - cx) * inset
    py = cy + (py - cy) * inset
    Xn = (px - cx) / Rx
    Yn = (cy - py) / Ry
    r2_lim = (inset * inset) if clip_r2 is None else float(clip_r2)
    inside = Xn * Xn + Yn * Yn <= r2_lim
    # Also require published colored oval (seed) so registration oversize cannot bleed.
    if color_oval is not None:
        ocx, ocy, oRx, oRy, o_r = color_oval
```

```
        inside = inside & inside_ellipse(px, py, ocx, ocy, oRx, oRy, o_r)
    px = np.where(inside, px, np.nan)
    py = np.where(inside, py, np.nan)

    segs = []
    cur_x, cur_y = [], []
    for x, y in zip(px, py):
        if not (np.isfinite(x) and np.isfinite(y)):
            if len(cur_x) >= 2:
                segs.append((cur_x, cur_y))
            cur_x, cur_y = [], []
            continue
        if cur_x and np.hypot(x - cur_x[-1], y - cur_y[-1]) > jump_px:
            if len(cur_x) >= 2:
                segs.append((cur_x, cur_y))
            cur_x, cur_y = [float(x)], [float(y)]
        else:
            cur_x.append(float(x))
            cur_y.append(float(y))
    if len(cur_x) >= 2:
        segs.append((cur_x, cur_y))
    if clip_path is not None:
        kw["clip_path"] = clip_path
        kw["clip_on"] = True
    for sx, sy in segs:
        (line,) = ax.plot(sx, sy, **kw)
        if clip_path is not None:
            line.set_clip_path(clip_path)
            line.set_clip_on(True)


def plot_closed_ring(
    ax, mu, cx, cy, Rx, Ry, color, lw, inset=1.0, jump_px=80.0,
    clip_path=None, color_oval=None, ls="-", wash_alpha=None,
):
    """
    Exact projected small-circle about D_HAT=+x: n·x̂ = μ.

    Parameterize on the sphere, convert to Galactic (ℓ,b), apply
    lon_disp = -wrap(ℓ), Mollweide-project, and stroke continuous polylines.
    Split wherever display-longitude jumps across the antimeridian (or pixel
    distance jumps). Never invent interior chords, ellipse fits, or decorative
    oval-rim bridges.

    Notes on θ labels used elsewhere:
      μ=cos(30°)  → r=30° about +x (closed oval near map center)
      μ=cos(185°)=cos(175°) → r=175° about +x ≡ 5° about CoE
        → small arcs on BOTH left and right map edges (not a rim loop)
      μ=cos(115°) → r=115° about +x (large seam-crossing curve)
      μ=cos(265°)=cos(95°) → r=95° about +x (near-meridian, seam-crossing)
    """
    pts = small_circle_about_axis(D_HAT, mu, n=4001)
    # Keep sampling order around the axis (azimuth) — stable for splitting.
    phi = np.arctan2(pts[:, 2], pts[:, 1])
```

```
    order = np.argsort(phi)
    pts = pts[order]

    ell = np.empty(len(pts))
    b = np.empty(len(pts))
    for i, p in enumerate(pts):
        ell[i], b[i] = n_to_lb(p)

    lon = np.deg2rad(lon_disp_from_ell(ell))  # λ ∈ (-π, π]
    lat = np.deg2rad(b)

    # Split on antimeridian jumps in display longitude BEFORE pixel projection.
    dlon = np.diff(lon)
    cuts = np.where(np.abs(dlon) > np.pi)[0] + 1
    # Also close the periodic sample: check wrap between last and first.
    # Build open segments from azimuth-ordered points.
    idx = np.concatenate([[0], cuts, [len(lon)]])
    lon_segs = []
    for i0, i1 in zip(idx[:-1], idx[1:]):
        if i1 - i0 >= 2:
            lon_segs.append((lon[i0:i1].copy(), lat[i0:i1].copy()))

    # Circle sampling is cyclic. Merge the artificial last→first break when
    # that periodic join is not itself an antimeridian crossing. Otherwise a
    # seam-crossing circle falsely appears as three pieces with a tiny hole.
    if len(lon_segs) > 1 and abs(lon[0] - lon[-1]) <= np.pi:
        first_lo, first_la = lon_segs[0]
        last_lo, last_la = lon_segs[-1]
        lon_segs = [(
            np.concatenate([last_lo, first_lo]),
            np.concatenate([last_la, first_la]),
        )] + lon_segs[1:-1]

    # Only now pin TRUE seam endpoints to ±π. Never snap the arbitrary
    # parameterization start/end to the boundary.
    snapped = []
    for lo, la in lon_segs:
        if abs(abs(lo[0]) - np.pi) < np.deg2rad(10):
            lo[0] = np.sign(lo[0] if lo[0] != 0 else 1.0) * np.pi
        if abs(abs(lo[-1]) - np.pi) < np.deg2rad(10):
            lo[-1] = np.sign(lo[-1] if lo[-1] != 0 else 1.0) * np.pi
        snapped.append((lo, la))
    lon_segs = snapped

    def stroke_xy(xs, ys):
        if len(xs) < 2:
            return
        if wash_alpha is not None:
            # Soft μ-wash stroke: no white underlay; translucent color only.
            layers = ((color, lw, 2, float(wash_alpha)),)
        else:
            # Clean mid-stroke: optional thin white underlay only (lw+1.5), then color.
            # No black outer rim, no thick white halo stack.
            layers = (
```

```
            ("white", lw + 1.5, 3, 0.95),
            (color, lw, 4, 1.0),
        )
    for col, width, z, a in layers:
        (line,) = ax.plot(
            xs, ys, color=col, lw=width, linestyle=ls,
            solid_capstyle="round", solid_joinstyle="round",
            zorder=z, alpha=a,
            clip_path=clip_path, clip_on=True,
        )
        if clip_path is not None:
            line.set_clip_path(clip_path)
            line.set_clip_on(True)

r2_lim = inset * inset + 1e-10
drawn = 0
for lo, la in lon_segs:
    # Mollweide in normalized oval coords, then pixels.
    x_m, y_m = mollweide_xy(lo, la)
    Xn = x_m / (2.0 * np.sqrt(2.0))
    Yn = y_m / np.sqrt(2.0)
    px = cx + Xn * Rx
    py = cy - Yn * Ry
    # Mild inset so thick strokes stay inside published oval (no rim-snap).
    px = cx + (px - cx) * inset
    py = cy + (py - cy) * inset
    Xn2 = (px - cx) / Rx
    Yn2 = (cy - py) / Ry
    inside = (Xn2 * Xn2 + Yn2 * Yn2) <= r2_lim
    if color_oval is not None:
        ocx, ocy, oRx, oRy, o_r = color_oval
        inside = inside & inside_ellipse(px, py, ocx, ocy, oRx, oRy, o_r)

    # Pixel-space split on NaN / large jumps / outside oval.
    cur_x, cur_y = [], []
    for x, y, ok in zip(px, py, inside):
        if (not ok) or (not np.isfinite(x)) or (not np.isfinite(y)):
            if len(cur_x) >= 2:
                stroke_xy(cur_x, cur_y)
                drawn += 1
            cur_x, cur_y = [], []
            continue
        if cur_x and np.hypot(x - cur_x[-1], y - cur_y[-1]) > jump_px:
            if len(cur_x) >= 2:
                stroke_xy(cur_x, cur_y)
                drawn += 1
            cur_x, cur_y = [float(x)], [float(y)]
        else:
            cur_x.append(float(x))
            cur_y.append(float(y))
    if len(cur_x) >= 2:
        # Close only if this single segment is already nearly a loop
        # (non-wrapping ring such as MAX 30°).
        if (
```

```
                drawn == 0
                and len(lon_segs) == 1
                and np.hypot(cur_x[0] - cur_x[-1], cur_y[0] - cur_y[-1]) < jump_px
            ):
                cur_x.append(cur_x[0])
                cur_y.append(cur_y[0])
            stroke_xy(cur_x, cur_y)
            drawn += 1



def _ring_px_arrays(mu, cx, cy, Rx, Ry, inset, n_pts=721):
    """Small-circle n·D=μ → pixel (px, py) arrays, azimuth-ordered, with inset."""
    pts = small_circle_about_axis(D_HAT, float(mu), n=n_pts)
    phi = np.arctan2(pts[:, 2], pts[:, 1])
    order = np.argsort(phi)
    pts = pts[order]
    ell = np.empty(n_pts)
    b = np.empty(n_pts)
    for i, p in enumerate(pts):
        ell[i], b[i] = n_to_lb(p)
    lon = np.deg2rad(lon_disp_from_ell(ell))
    lat = np.deg2rad(b)
    x_m, y_m = mollweide_xy(lon, lat)
    Xn = x_m / (2.0 * np.sqrt(2.0))
    Yn = y_m / np.sqrt(2.0)
    px = cx + Xn * Rx
    py = cy - Yn * Ry
    px = cx + (px - cx) * inset
    py = cy + (py - cy) * inset
    return lon, px, py


def fill_highlight_band(
    ax, theta_lo, theta_hi, theta_mid, cx, cy, Rx, Ry, color,
    inset=1.0, clip_path=None, color_oval=None, n_pts=721, jump_px=80.0,
):
    """
    One highlight band matching three-panel Panel B:
      continuous transparent polygon strips between ±BAND_HALF edges
      (broken only at Mollweide antimeridian / large pixel jumps — NOT a dense
      multi-stroke wash) + ONE solid mid-stroke at θ_mid.
    """
    mu_a, mu_b = mu_pair_for_band(theta_lo, theta_hi)
    mu_mid = float(np.cos(np.deg2rad(theta_mid)))
    lon_a, px_a, py_a = _ring_px_arrays(mu_a, cx, cy, Rx, Ry, inset, n_pts=n_pts)
    lon_b, px_b, py_b = _ring_px_arrays(mu_b, cx, cy, Rx, Ry, inset, n_pts=n_pts)

    r2_lim = inset * inset + 1e-10

    def _inside(px, py):
        Xn2 = (px - cx) / Rx
        Yn2 = (cy - py) / Ry
        ok = (Xn2 * Xn2 + Yn2 * Yn2) <= r2_lim
```

```
        if color_oval is not None:
            ocx, ocy, oRx, oRy, o_r = color_oval
            ok = ok & inside_ellipse(px, py, ocx, ocy, oRx, oRy, o_r)
        return ok

    gap = 0.9 * np.pi
    bad = np.zeros(n_pts - 1, dtype=bool)
    for i in range(n_pts - 1):
        if abs(lon_a[i + 1] - lon_a[i]) > gap:
            bad[i] = True
        elif abs(lon_b[i + 1] - lon_b[i]) > gap:
            bad[i] = True
        elif abs(lon_a[i] - lon_b[i]) > gap:
            bad[i] = True
        elif abs(lon_a[i + 1] - lon_b[i + 1]) > gap:
            bad[i] = True
        elif np.hypot(px_a[i + 1] - px_a[i], py_a[i + 1] - py_a[i]) > jump_px:
            bad[i] = True
        elif np.hypot(px_b[i + 1] - px_b[i], py_b[i + 1] - py_b[i]) > jump_px:
            bad[i] = True
        elif (not _inside(px_a[i], py_a[i])) or (not _inside(px_b[i], py_b[i])):
            bad[i] = True
        elif (not _inside(px_a[i + 1], py_a[i + 1])) or (not _inside(px_b[i + 1], py_b[i + 1])):
            bad[i] = True

    polys = []
    i = 0
    while i < n_pts - 1:
        while i < n_pts - 1 and bad[i]:
            i += 1
        if i >= n_pts - 1:
            break
        j = i
        while j < n_pts - 1 and not bad[j]:
            j += 1
        i0, i1 = i, j
        if i1 - i0 >= 1:
            xs = np.concatenate([px_a[i0:i1 + 1], px_b[i0:i1 + 1][::-1]])
            ys = np.concatenate([py_a[i0:i1 + 1], py_b[i0:i1 + 1][::-1]])
            polys.append(np.column_stack([xs, ys]))
        i = j + 1

    if polys:
        # Soft transparent highlight (clean pre-halo look).
        coll = PolyCollection(
            polys, facecolors=color, edgecolors="none",
            alpha=0.30, zorder=2, linewidths=0,
        )
        ax.add_collection(coll)
        if clip_path is not None:
            coll.set_clip_path(clip_path)
            coll.set_clip_on(True)

    # ONE solid mid-stroke only (thin white underlay lw+1.5). No boundary-edge
```

```
    # strokes — dual edges with halos multiply visual lines into "extra loops".
    plot_closed_ring(
        ax, mu_mid, cx, cy, Rx, Ry, color, lw=2.8,
        inset=inset, clip_path=clip_path, color_oval=color_oval,
    )



def draw_overlay(rgb, reg, seed, source):

    h, w, _ = rgb.shape
    cx, cy, Rx, Ry = reg["cx"], reg["cy"], reg["Rx"], reg["Ry"]
    # Published colored Mollweide (seed detect) — used to keep overlays inside
    # the Migkas map even when registration ellipse is slightly larger.
    scx = float(seed["cx"])
    scy = float(seed["cy"])
    sRx = float(seed["Rx"])
    sRy = float(seed["Ry"])

    dpi = 120
    fig, ax = plt.subplots(figsize=(w / dpi, (h / dpi) + 1.35), dpi=dpi)
    ax.imshow(rgb, origin="upper", extent=(0, w, h, 0), interpolation="nearest")
    ax.set_xlim(0, w)
    ax.set_ylim(h, 0)
    ax.set_aspect("equal")
    ax.axis("off")

    # clip_path = registered oval (cx,cy,Rx,Ry). Registration itself is unchanged.
    # Because the refined ellipse can sit a bit outside the published colored
    # map, we ALSO inset strokes and cull/inset markers against the seed oval.
    clip_oval = Ellipse(
        (cx, cy), width=2.0 * Rx, height=2.0 * Ry,
        facecolor="none", edgecolor="none", transform=ax.transData,
    )
    ax.add_patch(clip_oval)

    # Stroke inset so thick white+color underlays do not AA-bleed past the rim.
    STROKE_INSET = 0.97
    # Cull points outside this fraction of the published colored seed oval.
    COLOR_R = 0.978
    color_oval = (scx, scy, sRx, sRy, COLOR_R)

    def stroke_curve(ell, b, color, lw, ls="-"):
        # White underlay then color — stands out on magma without looking oversized.
        plot_lb_curve(
            ax, ell, b, cx, cy, Rx, Ry, inset=STROKE_INSET, clip_path=clip_oval,
            color_oval=color_oval,
            color="white", lw=lw + 1.6, ls=ls, alpha=0.95,
            solid_capstyle="round", zorder=3,
        )
        plot_lb_curve(
            ax, ell, b, cx, cy, Rx, Ry, inset=STROKE_INSET, clip_path=clip_oval,
            color_oval=color_oval,
            color=color, lw=lw, ls=ls, alpha=1.0,
```

```
            solid_capstyle="round", zorder=4,
        )

    def mark_point(ell, b, marker, color, ms):
        """Place marker; pull limb symbols inward so halo stays in colored oval."""
        px, py = as_xy(*lb_to_px(ell, b, cx, cy, Rx, Ry))
        # Approx marker+halo radius in pixels (ms is points).
        halo_ms = ms + 1.8
        ms_px = 0.5 * halo_ms * (dpi / 72.0)
        # Normalized margin inside the published colored oval.
        margin = ms_px / min(sRx, sRy) + 0.012
        r_lim = max(0.55, COLOR_R - margin)

        Xn = (px - scx) / sRx
        Yn = (py - scy) / sRy
        r_col = float(np.hypot(Xn, Yn))

        if r_col > 1e-9:
            # Scale toward colored-oval center so the whole symbol fits.
            if r_col > r_lim:
                scale = r_lim / r_col
                px = scx + (px - scx) * scale
                py = scy + (py - scy) * scale
                Xn = (px - scx) / sRx
                Yn = (py - scy) / sRy
                r_col = float(np.hypot(Xn, Yn))
            # Mild extra inset for everyone (matches stroke inset spirit).
            mild = min(STROKE_INSET, 0.995)
            px = scx + (px - scx) * mild
            py = scy + (py - scy) * mild
            Xn = (px - scx) / sRx
            Yn = (py - scy) / sRy
            r_col = float(np.hypot(Xn, Yn))

        # Skip if still too close to the rim for this marker size.
        if r_col + margin > COLOR_R + 1e-6:
            return

        def _mark(m_size, m_color, m_edge, m_ew, z):
            (line,) = ax.plot(
                px, py, marker=marker, ms=m_size, color=m_color,
                markeredgecolor=m_edge, markeredgewidth=m_ew, linestyle="None",
                zorder=z, clip_path=clip_oval, clip_on=True,
            )
            line.set_clip_path(clip_oval)
            line.set_clip_on(True)

        _mark(ms + 1.8, "white", "white", 0, 6)
        _mark(ms, color, "k", 0.55, 7)

    # CoE axis at geometric inset=1.0. Do NOT overlay Dec=0 — Migkas already
    # printed the celestial equator on Fig. 8; a second green dashed stroke fights it.
    def stroke_curve_geom(ell, b, color, lw, ls="-"):
        plot_lb_curve(
```

```
        ax, ell, b, cx, cy, Rx, Ry, inset=1.0, clip_path=clip_oval,
        color_oval=color_oval,
        color="white", lw=lw + 1.6, ls=ls, alpha=0.95,
        solid_capstyle="round", zorder=3,
    )
    plot_lb_curve(
        ax, ell, b, cx, cy, Rx, Ry, inset=1.0, clip_path=clip_oval,
        color_oval=color_oval,
        color=color, lw=lw, ls=ls, alpha=1.0,
        solid_capstyle="round", zorder=4,
    )

# CoE↔anti-CoE axis = great circle through ±D (tilted +30°; not Galactic equator)
_ax_ell, _ax_b = great_circle_through_poles(D_HAT, n=721)
stroke_curve_geom(_ax_ell, _ax_b, C_AXIS, lw=1.15)

# Exactly four highlight bands (match three-panel Panel B): each ±9°
# (= 18° end-to-end = 5% of 360°) + one solid mid-stroke. No companion.
for (lo, hi), mid, color in zip(MAX_BANDS, THETA_MAX, COLORS_MAX):
    fill_highlight_band(
        ax, lo, hi, mid, cx, cy, Rx, Ry, color,
        inset=STROKE_INSET, clip_path=clip_oval, color_oval=color_oval,
    )
for (lo, hi), mid, color in zip(MIN_BANDS, THETA_MIN, COLORS_MIN):
    fill_highlight_band(
        ax, lo, hi, mid, cx, cy, Rx, Ry, color,
        inset=STROKE_INSET, clip_path=clip_oval, color_oval=color_oval,
    )

# Markers (limb ones get stronger inset inside mark_point); per-loop colors
mark_point(ANTI_COE_LB[0], ANTI_COE_LB[1], "o", C_ANTI, ms=5.5)
mark_point(COE_LB[0], COE_LB[1], "*", C_COE, ms=7.5)
for (ell, b), color in zip(MIGKAS_MAXES, COLORS_MAX):
    mark_point(ell, b, "^", color, ms=6.5)
for (ell, b), color in zip(MIGKAS_MINS, COLORS_MIN):
    mark_point(ell, b, "s", color, ms=6.0)

handles = [
    Patch(
        facecolor=C_MAX_30, alpha=0.55, edgecolor=C_MAX_30,
        label=(
            rf"MAX $\theta\!=\!{THETA_MAX[0]:.0f}^\circ$ band "
            rf"[{MAX_BANDS[0][0]:.0f},{MAX_BANDS[0][1]:.0f}]$^\circ$ "
            rf"($18^\circ$=$\pm9^\circ$=5% of $360^\circ$)"
        ),
    ),
    Patch(
        facecolor=C_MAX_185, alpha=0.55, edgecolor=C_MAX_185,
        label=(
            rf"MAX $\theta\!=\!{THETA_MAX[1]:.0f}^\circ$ band "
            rf"[{MAX_BANDS[1][0]:.0f},{MAX_BANDS[1][1]:.0f}]$^\circ$ "
            rf"($18^\circ$=$\pm9^\circ$)"
        ),
    ),
```

```
        Patch(
            facecolor=C_MIN_115, alpha=0.55, edgecolor=C_MIN_115,
            label=(
                rf"MIN $\theta\!=\!{THETA_MIN[0]:.0f}^\circ$ band "
                rf"[{MIN_BANDS[0][0]:.0f},{MIN_BANDS[0][1]:.0f}]$^\circ$ "
                rf"($18^\circ$=$\pm9^\circ$)"
            ),
        ),
        Patch(
            facecolor=C_MIN_265, alpha=0.55, edgecolor=C_MIN_265,
            label=(
                rf"MIN $\theta\!=\!{THETA_MIN[1]:.0f}^\circ$ band "
                rf"[{MIN_BANDS[1][0]:.0f},{MIN_BANDS[1][1]:.0f}]$^\circ$ "
                rf"($18^\circ$=$\pm9^\circ$)"
            ),
        ),
        Line2D([0], [0], color=C_AXIS, lw=2.0, label=r"CoE axis tilted $+30^\circ$"),
        Line2D(
            [0], [0], marker="^", color="w", markerfacecolor=C_MAX_30,
            markeredgecolor="k", linestyle="None", ms=8,
            label=rf"MAX $({MIGKAS_MAXES[0][0]:.0f}^\circ,{MIGKAS_MAXES[0][1]:+.0f}^\circ)$",
        ),
        Line2D(
            [0], [0], marker="^", color="w", markerfacecolor=C_MAX_185,
            markeredgecolor="k", linestyle="None", ms=8,
            label=rf"MAX $({MIGKAS_MAXES[1][0]:.0f}^\circ,{MIGKAS_MAXES[1][1]:+.0f}^\circ)$",
        ),
        Line2D(
            [0], [0], marker="s", color="w", markerfacecolor=C_MIN_115,
            markeredgecolor="k", linestyle="None", ms=7,
            label=rf"MIN $({MIGKAS_MINS[0][0]:.0f}^\circ,{MIGKAS_MINS[0][1]:+.0f}^\circ)$",
        ),
        Line2D(
            [0], [0], marker="s", color="w", markerfacecolor=C_MIN_265,
            markeredgecolor="k", linestyle="None", ms=7,
            label=rf"MIN $({MIGKAS_MINS[1][0]:.0f}^\circ,{MIGKAS_MINS[1][1]:+.0f}^\circ)$",
        ),
        Line2D([0], [0], marker="o", color=C_ANTI, markeredgecolor="k",
               linestyle="None", ms=6,
               label=r"anti-CoE $+x$ $(0^\circ,+30^\circ)$"),
        Line2D([0], [0], marker="*", color=C_COE, markeredgecolor="k",
               linestyle="None", ms=9,
               label=r"CoE $(180^\circ,-30^\circ)$"),
    ]
    leg = ax.legend(
        handles=handles, loc="upper center", bbox_to_anchor=(0.5, -0.012),
        ncol=4, fontsize=7.5, frameon=True, fancybox=False,
        edgecolor="#222222", framealpha=1.0, facecolor="white",
        labelcolor="#111111", borderpad=0.6, handlelength=2.0,
    )
    leg.get_frame().set_linewidth(1.2)
    fig.suptitle(
        r"Migkas Fig. 8 $\theta=75^\circ$ $A/A_{\rm all}$"
        r" + four CoE highlight bands, each $18^\circ$ end-to-end ($\pm9^\circ$ = 5% of $360^\circ$)"
```

```
        "\n"
        r"(two MAX + two MIN; cyan/magenta/lime/sky; anti-CoE at $(0^\circ,+30^\circ)$; "
        r"CoE axis tilted $+30^\circ$; $\ell$ increases LEFT)",
        fontsize=10.5, y=0.985,
    )
    fig.subplots_adjust(left=0.01, right=0.99, top=0.92, bottom=0.13)
    fig.savefig(PNG_PATH, dpi=dpi, facecolor="white")
    plt.close(fig)
    print(f"wrote {PNG_PATH}")


def main():
    source = choose_source()
    print(f"source: {source}")
    rgb = np.asarray(Image.open(source).convert("RGB"))
    seed = detect_oval_bbox(rgb)
    print("oval seed:", {k: seed[k] for k in ("cx", "cy", "Rx", "Ry", "x0", "x1", "y0", "y1")})
    # Restore previous visually-good oval (Chris preferred). No medial-axis Rx stretch.
    reg = dict(GOOD_REG)
    h, w, _ = rgb.shape
    dark = rgb.mean(axis=2) < 55
    dist = chamfer_distance_to_dark(dark)
    ell_eq, b_eq = celestial_equator_lb(2500)
    lon = np.deg2rad(lon_disp_from_ell(ell_eq))
    lat = np.deg2rad(b_eq)
    mx, my = mollweide_xy(lon, lat)
    Xn = mx / (2.0 * np.sqrt(2.0))
    Yn = my / np.sqrt(2.0)
    px = reg["cx"] + Xn * reg["Rx"]
    py = reg["cy"] - Yn * reg["Ry"]
    m = (px >= 1) & (px < w - 1) & (py >= 1) & (py < h - 1)
    xi = np.clip(np.round(px[m]).astype(int), 0, w - 1)
    yi = np.clip(np.round(py[m]).astype(int), 0, h - 1)
    reg["ceq_median_px"] = float(np.median(dist[yi, xi]))
    reg["registration_source"] = "hard-set GOOD_REG (pre medial-axis retune)"
    print("restored reg:", {k: reg[k] for k in ("cx", "cy", "Rx", "Ry", "ceq_median_px")})
    checks = validate_landmarks(rgb, reg)
    print(
        "validation OK — max0 RGB", checks["migkas_max0_rgb_mean"],
        "min1 RGB", checks["migkas_min1_rgb_mean"],
    )
    draw_overlay(rgb, reg, seed, source)

    meta = {
        "png": str(PNG_PATH),
        "script": str(Path(__file__).resolve()),
        "source_image": str(source),
        "convention": (
            "Galactic Mollweide; lon_disp=-wrap(ell); ell increases LEFT "
            "(Migkas Fig.8)"
        ),
        "axis_tilt_b_deg": AXIS_TILT_B_DEG,
        "anti_CoE_lb_deg": [float(ANTI_COE_LB[0]), float(ANTI_COE_LB[1])],
        "CoE_lb_deg": [float(COE_LB[0]), float(COE_LB[1])],
```

```
        "D_hat_galactic_xyz": D_HAT.tolist(),
        "axis_tilt_reason": (
            "Chris 2026-09-22 — markers fixed, rings rotate up 30°"
        ),
        "migkas_max_lb_deg": [list(p) for p in MIGKAS_MAXES],
        "migkas_min_lb_deg": [list(p) for p in MIGKAS_MINS],
        "theta_max_from_plus_x_deg": list(THETA_MAX),
        "theta_min_from_plus_x_deg": list(THETA_MIN),
        "band_half_width_deg": BAND_HALF,
        "band_full_width_deg": BAND_FULL,
        "band_width_note": "5% of 360° = 18° end-to-end = ±9° about each MAX/MIN center",
        "max_bands_deg": [list(b) for b in MAX_BANDS],
        "min_bands_deg": [list(b) for b in MIN_BANDS],
        "band_intervals": {
            "MAX_30": [float(MAX_BANDS[0][0]), float(MAX_BANDS[0][1])],
            "MAX_185": [float(MAX_BANDS[1][0]), float(MAX_BANDS[1][1])],
            "MIN_115": [float(MIN_BANDS[0][0]), float(MIN_BANDS[0][1])],
            "MIN_265": [float(MIN_BANDS[1][0]), float(MIN_BANDS[1][1])],
        },
        "colors": {
            "MAX_30": C_MAX_30,
            "MAX_185": C_MAX_185,
            "MIN_115": C_MIN_115,
            "MIN_265": C_MIN_265,
            "palette": "magma-popping cyan/magenta/lime/sky (no reds/oranges)",
        },
        "mu_max_bands": [list(mu_pair_for_band(lo, hi)) for lo, hi in MAX_BANDS],
        "mu_min_bands": [list(mu_pair_for_band(lo, hi)) for lo, hi in MIN_BANDS],
        "scan_stop_source": (
            "best_fit_2ell_multiplier.npz A_after extrema (two max, two min); "
            f"each drawn as one transparent {BAND_FULL:.0f}° end-to-end highlight "
            f"(±{BAND_HALF:.0f}° = 5% of 360°) + one solid mid-stroke; "
            "matches three-panel Panel B; no companion"
        ),
        "matches_three_panel": str(OUT_DIR / "make_coe_rings_migkas_pin_axis.py"),
        "oval_seed_bbox": {
            "x0": seed["x0"], "x1": seed["x1"], "y0": seed["y0"], "y1": seed["y1"],
            "cx": seed["cx"], "cy": seed["cy"], "Rx": seed["Rx"], "Ry": seed["Ry"],
        },
        "registration": {
            "cx": reg["cx"], "cy": reg["cy"], "Rx": reg["Rx"], "Ry": reg["Ry"],
            "ceq_median_px": reg["ceq_median_px"],
            "registration_source": reg.get("registration_source"),
            "ceq_residual_before_registration": dict(GOOD_REG),
            "rejected_medial_axis_reg": {
                "cx": 865.6, "cy": 766.4, "Rx": 706.19, "Ry": 393.70,
                "note": "medial-axis / achromatic DT-ridge retune; blew up Rx; reverted",
            },
            "dofs": ["cx", "cy", "Rx", "Ry"],
            "drawing": {
                "dec0_drawn": False,
                "dec0_note": "not drawn; use Migkas published celestial equator",
                "coe_axis_inset": 1.0,
                "rings_inset": 0.97,
```

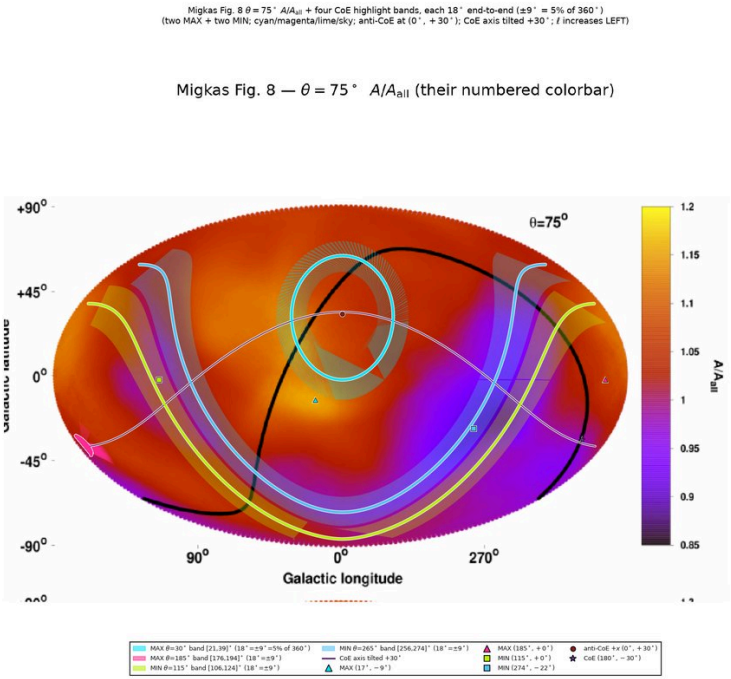
```
                "note": "CoE axis geometric inset=1; rings mild inset; no Dec=0 overlay",
            },
            "method": (
                "oval seed detect + hard-set to pre-retune visually-good (cx,cy,Rx,Ry); "
                "medial-axis Rx stretch rejected; Dec=0 NOT overlaid (published CEQ kept); "
                "CoE-axis inset=1.0; rings inset~0.97"
            ),
        },
        "landmark_checks": checks,
        "drawing": (
            "Exactly 4 features. Each = ONE smooth transparent PolyCollection "
            "band (θ±9°, 18° end-to-end) + ONE solid mid-stroke at center θ. "
            "Fill alpha≈0.30. Mid-stroke: thin white underlay (lw+1.5) then color "
            "(lw=2.8). NO black outer rim; NO thick white halo stacks; NO dual "
            "boundary-edge strokes. Colors: cyan/magenta/lime/sky for magma. "
            "No 283° companion. Dec=0 NOT overlaid."
        ),
        "legend_summary": (
            "four highlight bands, each 18° end-to-end (±9° = 5% of 360%); "
            "MAX cyan+magenta, MIN lime+sky; intervals [21,39]/[176,194]/[106,124]/[256,274]"
        ),
        "notes": [
            "Exactly 4 features matching three-panel Panel B geometry/width",
            "Each = ONE smooth transparent PolyCollection ribbon (±9°) + ONE solid mid-stroke only",
            "Clean draw: fill alpha=0.30; mid-stroke thin white underlay (lw+1.5) + color lw=2.8; no black rim; no boundary-edge halos",
            "No multi-stroke scribble wash; no 283° companion; no Dec=0 overlay",
            "MAX bands [21,39] & [176,194]; MIN bands [106,124] & [256,274]",
            "Colors: MAX_30=#00e5ff, MAX_185=#ff2d95, MIN_115=#b8f200, MIN_265=#4fc3f7",
            "band_half_width_deg=9, band_full_width_deg=18 (5% of 360°)",
            "Exact n·x̂=μ small-circles; seam-split polygon fill; mid via plot_closed_ring",
            "No ellipse-fit, no interior chords, no decorative oval-rim loops",
            "No blocking text box on the map; legend below",
            "Colorbar preserved from source (overlays stay on oval)",
            "Overlays clip_path = registered Ellipse(cx,cy,Rx,Ry); rings stroke inset 0.97",
            "Dec=0 NOT drawn (Migkas Fig.8 already has celestial equator); CoE axis = great circle through ±D (tilted +30°) at inset=1.0",
            "axis_tilt_b_deg=30; anti-CoE (0°,+30°), CoE (180°,−30°); Migkas markers fixed at true sky positions",
            "Registration hard-set to pre medial-axis params (863,763.25,693.9927,392.542425); DT-ridge fit rejected",
            "Limb markers inset toward colored-oval center so halo stays inside",
            "Points culled at COLOR_R=0.978 of seed oval; limb markers inset to fit halo",
        ],
    }
    JSON_PATH.write_text(json.dumps(meta, indent=2))
    print(f"wrote {JSON_PATH}")

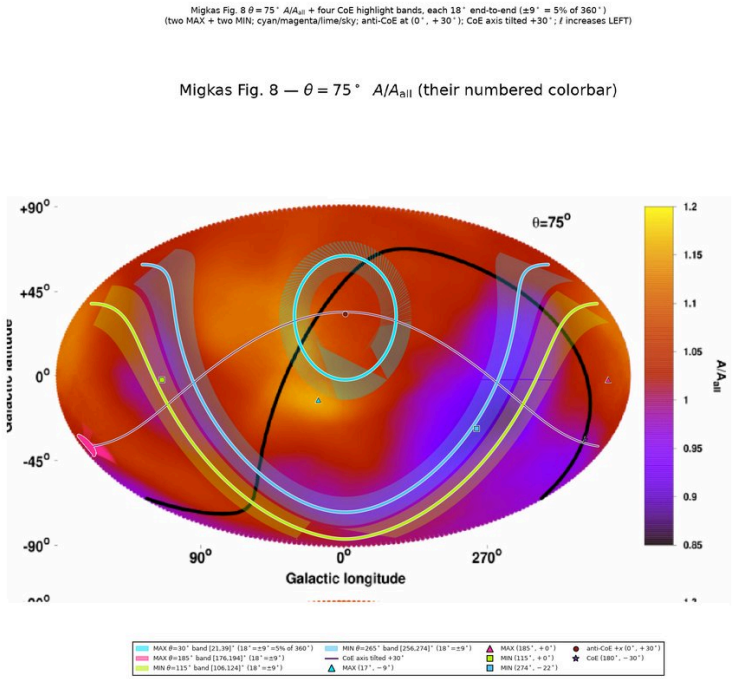

if __name__ == "__main__":
    main()
```

```
source: /workspace/cosmology/map_compare/FULL_migkas_theta75.png
oval seed: {'cx': 856.0, 'cy': 754.0, 'Rx': 677.0, 'Ry': 398.5, 'x0': 179, 'x1': 1533, 'y0': 356, 'y1': 1152}
restored reg: {'cx': 863.0, 'cy': 763.25, 'Rx': 693.9927, 'Ry': 392.542425, 'ceq_median_px': 1.0}
validation OK — max0 RGB [237.0, 166.33333333333334, 0.0] min1 RGB [96.61728395061728, 0.0, 205.39506172839506]
wrote /workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/migkas_fig8_theta75_with_coe_rings.png
wrote /workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/migkas_fig8_theta75_with_coe_rings.json
```

```
compare(FIGS / 'migkas_fig8_theta75_with_coe_rings.png', '/workspace/cosmology/figures/migkas_fig8_theta75_with_coe_rings.png')
```

```
migkas_fig8_theta75_with_coe_rings.png: new (1975, 1515) vs ref (1975, 1515) | mean |diff| = 0.000/255, pixels differing >40: 0.000% -> identical
left: approved reference figures/migkas_fig8_theta75_with_coe_rings.png   |   right: notebook re-run
steps_v3/figs/migkas_fig8_theta75_with_coe_rings.png
```





Step 12 — Half-loops: CoE axis $+15^\circ$, and the $+45^\circ$ CCW roll pair

What it shows. The same three panels, but with the axis tilted $+15^\circ$ (anti-CoE $(0^\circ, +15^\circ)$) and each loop kept only on its θ-ray side, an open 180° arc. First run: roll 0. Second run: roll 45° counter-clockwise about the CoE axis (looking along $+D$). **Maths.** Half-cut: keep ring points with $(\hat{n} \cdot \hat{Y})\ \sin\theta_{\rm mid} \geq 0$ (top half when $\sin\theta > 0$, bottom when $\sin\theta < 0$). A roll by ρ rotates $\hat{e}_{1,2}$ about $\hat{a}$: $\hat{e}_1' = \cos\rho\,\hat{e}_1 + \sin\rho\,\hat{e}_2$.

Source script: `/workspace/cosmology/figures/make_coe_rings_half_loops.py` (last modified 2026-09-25 20:22 CT), run verbatim. *Output-path edit:* none. The script's `OUT_DIR` follows `__file__`, which is set to `steps_v3/figs/`. *Reference image:* `figures/coe_rings_half_loops_tilt15.png`, `figures/coe_rings_half_loops_tilt15_rot45ccw.png`.

```
%%runscript make_coe_rings_half_loops.py --argv --roll 0
#!/usr/bin/env python3
"""
CoE rings — theta-ray-side half-loops only (Chris 2026-09-23 whiteout, CORRECTED).
```

```
Flags (CLI or module-level defaults):
  AXIS_TILT_B_DEG = 15   -> anti-CoE (0 deg, +15 deg), CoE (180 deg, -15 deg)
  ROLL_ABOUT_COE_DEG = 0 or 45  -> CCW about +D looking along +D
  HALF_LOOPS = True       -> open 180 deg arcs only (never full loops)

Half-cut (theta-ray side — NOT outer limb):
  foreshortened ex=mu+0.045*rho*cos(phi), ey=rho*sin(phi);
  keep ey*sin(theta_mid) >= 0  (TOP if sin theta>0, BOTTOM if sin theta<0).
  3D: keep (n·Y_hat)*sin(theta_mid) >= 0. Discard sign(mu)*cos(phi)>=0.

Outputs depend on roll:
  roll=0  -> coe_rings_half_loops_tilt15.{png,pdf,json}
  roll=45 -> coe_rings_half_loops_tilt15_rot45ccw.{png,pdf,json}
"""

from __future__ import annotations

import argparse
import json
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
from matplotlib.lines import Line2D
from matplotlib.collections import PolyCollection
from matplotlib.patches import Circle, Patch, Polygon
from mpl_toolkits.mplot3d import Axes3D  # noqa: F401
from mpl_toolkits.mplot3d.art3d import Poly3DCollection

from coe_half_loop_geometry import (
    BAND_FULL,
    BAND_HALF,
    HALF_LOOP_RULE,
    MIGKAS_MAXES,
    MIGKAS_MINS,
    STRIP_THETA_MAX,
    STRIP_THETA_MIN,
    axis_pose,
    band_edges,
    celestial_equator_lb,
    foreshortened_half_band_polygon,
    foreshortened_half_ellipse_xy,
    great_circle_through_poles,
    lb_to_n,
    lon_disp_from_ell,
    mu_pair_for_band,
    n_to_lb,
    small_circle_theta_ray_half,
)

OUT_DIR = Path(__file__).resolve().parent

# Defaults (overridden by CLI)
```

```
AXIS_TILT_B_DEG = 15.0
ROLL_ABOUT_COE_DEG = 0.0
HALF_LOOPS = True

C_COE = "#6b3fa0"
C_ANTI = "#8e1b1b"
C_MAX_30 = "#c0392b"
C_MAX_185 = "#ec7063"
C_MIN_115 = "#1a5276"
C_MIN_265 = "#5dade2"
C_AXIS = "#5b2c6f"
C_EQ = "#1e8449"
COLORS_MAX = [C_MAX_30, C_MAX_185]
COLORS_MIN = [C_MIN_115, C_MIN_265]


def output_stem(tilt_b: float, roll: float) -> str:
    base = f"coe_rings_half_loops_tilt{int(round(tilt_b))}"
    if abs(roll) > 0.5:
        base += f"_rot{int(round(roll))}ccw"
    return base


def split_lon_segments(lon_rad, lat_rad, gap=np.pi):
    d = np.diff(lon_rad)
    cuts = np.where(np.abs(d) > gap)[0] + 1
    idx = np.concatenate([[0], cuts, [len(lon_rad)]])
    segs = []
    for i0, i1 in zip(idx[:-1], idx[1:]):
        if i1 - i0 >= 2:
            segs.append((lon_rad[i0:i1], lat_rad[i0:i1]))
    return segs


def plot_lb_curve_mollweide(ax, l_deg, b_deg, **kwargs):
    lon = np.deg2rad(lon_disp_from_ell(l_deg))
    lat = np.deg2rad(b_deg)
    for xr, yr in split_lon_segments(lon, lat):
        ax.plot(xr, yr, **kwargs)


def _half_ring_lb(D, mu, theta_mid, roll_deg, n_pts=721):
    pts, _ = small_circle_theta_ray_half(
        D, mu, theta_mid, roll_deg=roll_deg, n=n_pts
    )
    rl = np.array([n_to_lb(p)[0] for p in pts])
    rb = np.array([n_to_lb(p)[1] for p in pts])
    return rl, rb


def _half_ring_lonlat_rad(D, mu, theta_mid, roll_deg, n_pts=361):
    pts, _ = small_circle_theta_ray_half(
        D, float(mu), theta_mid, roll_deg=roll_deg, n=n_pts
    )
```

```
    ls = np.array([n_to_lb(p)[0] for p in pts])
    bs = np.array([n_to_lb(p)[1] for p in pts])
    lon = np.deg2rad(lon_disp_from_ell(ls))
    lat = np.deg2rad(bs)
    return lon, lat


def fill_mollweide_half_ribbon(ax, D, theta_lo, theta_hi, theta_mid, color, roll_deg, n_pts=721):
    """Theta-ray-side Mollweide ribbon between +/-9 deg edges + solid mid half-arc."""
    mu_a, mu_b = mu_pair_for_band(theta_lo, theta_hi)
    mu_mid = float(np.cos(np.deg2rad(theta_mid)))
    lon_a, lat_a = _half_ring_lonlat_rad(D, mu_a, theta_mid, roll_deg, n_pts=n_pts)
    lon_b, lat_b = _half_ring_lonlat_rad(D, mu_b, theta_mid, roll_deg, n_pts=n_pts)
    gap = 0.9 * np.pi
    bad = np.zeros(n_pts - 1, dtype=bool)
    for i in range(n_pts - 1):
        if abs(lon_a[i + 1] - lon_a[i]) > gap:
            bad[i] = True
        elif abs(lon_b[i + 1] - lon_b[i]) > gap:
            bad[i] = True
        elif abs(lon_a[i] - lon_b[i]) > gap:
            bad[i] = True
        elif abs(lon_a[i + 1] - lon_b[i + 1]) > gap:
            bad[i] = True
    polys = []
    i = 0
    while i < n_pts - 1:
        while i < n_pts - 1 and bad[i]:
            i += 1
        if i >= n_pts - 1:
            break
        j = i
        while j < n_pts - 1 and not bad[j]:
            j += 1
        i0, i1 = i, j
        if i1 - i0 >= 1:
            xs = np.concatenate([lon_a[i0 : i1 + 1], lon_b[i0 : i1 + 1][::-1]])
            ys = np.concatenate([lat_a[i0 : i1 + 1], lat_b[i0 : i1 + 1][::-1]])
            polys.append(np.column_stack([xs, ys]))
        i = j + 1
    if polys:
        ax.add_collection(
            PolyCollection(
                polys, facecolors=color, edgecolors="none",
                alpha=0.26, zorder=2, linewidths=0,
            )
        )
    rl, rb = _half_ring_lb(D, mu_mid, theta_mid, roll_deg, n_pts=max(n_pts, 721))
    plot_lb_curve_mollweide(ax, rl, rb, color=color, lw=2.6, alpha=0.95, zorder=4)


def draw_panelA_half_ribbon(ax, theta_lo, theta_hi, theta_mid, color, roll_deg):
    mu_lo = float(np.cos(np.deg2rad(theta_lo)))
    mu_hi = float(np.cos(np.deg2rad(theta_hi)))
```

```
    poly = foreshortened_half_band_polygon(
        mu_lo, mu_hi, theta_mid, roll_deg=roll_deg, n=160
    )
    patch = Polygon(
        poly, closed=True, facecolor=color, edgecolor="none",
        alpha=0.28, zorder=2, linewidth=0,
    )
    ax.add_patch(patch)
    clip_circ = Circle((0.0, 0.0), 1.0, transform=ax.transData)
    patch.set_clip_path(clip_circ)

    mu_mid = float(np.cos(np.deg2rad(theta_mid)))
    ex_m, ey_m, _ = foreshortened_half_ellipse_xy(
        mu_mid, theta_mid, roll_deg=roll_deg, n=256
    )
    ax.plot(ex_m, ey_m, color=color, lw=2.8, alpha=0.95, zorder=4, solid_capstyle="round")


def draw_panelA_theta_rays(ax, geo):
    items = list(zip(geo["theta_max_from_plus_x_deg"], COLORS_MAX)) + list(
        zip(geo["theta_min_from_plus_x_deg"], COLORS_MIN)
    )
    for mid, color in items:
        t = float(np.deg2rad(mid))
        x1, y1 = float(np.cos(t)), float(np.sin(t))
        is_max30 = abs(mid - 30.0) < 0.5
        lw = 3.2 if is_max30 else 1.8
        alpha = 0.70 if is_max30 else 0.45
        ax.plot(
            [0.0, x1], [0.0, y1],
            color=color, lw=lw, alpha=alpha, zorder=3, solid_capstyle="round",
        )
        ax.plot(
            [x1], [y1], "o", color=color, ms=4.5 if is_max30 else 3.5,
            alpha=0.9, zorder=5, markeredgecolor="none",
        )
    arc_r = 0.38
    phi = np.linspace(0.0, np.deg2rad(30.0), 60)
    ax.plot(
        arc_r * np.cos(phi), arc_r * np.sin(phi),
        color=C_MAX_30, lw=1.6, alpha=0.85, zorder=3,
    )
    ax.text(
        arc_r * np.cos(np.deg2rad(15.0)) + 0.06,
        arc_r * np.sin(np.deg2rad(15.0)) + 0.04,
        r"$\theta$",
        color=C_MAX_30, fontsize=10, fontweight="bold",
        ha="left", va="bottom", zorder=6,
    )


def panel_A_model_frame(ax, geo, roll_deg):
    roll_note = (
        rf"; $+{roll_deg:.0f}^\circ$ CCW roll about CoE"
```

```
        if abs(roll_deg) > 0.5
        else ""
    )
    ax.set_title(
        r"A. Model frame — $\theta$-ray-side half-loops only"
        "\n"
        r"axisymmetric about $x$ = CoE$\leftrightarrow$anti-CoE"
        f"{roll_note}\n"
        r"keep $(n\cdot\hat y)\sin\theta\geq0$ (same side as $\theta$-ray); "
        r"each $18^\circ$ end-to-end ($\pm9^\circ$)",
        fontsize=9.5,
        pad=8,
    )
    th = np.linspace(0, 2 * np.pi, 400)
    ax.plot(np.cos(th), np.sin(th), color="#7f8c8d", lw=1.2, zorder=1)
    ax.plot(0, 0, "ko", ms=6, zorder=5)
    ax.text(0.08, -0.12, "observer", fontsize=8, color="k", ha="left")
    ax.annotate(
        "",
        xy=(1.15, 0),
        xytext=(-1.15, 0),
        arrowprops=dict(arrowstyle="<->", color=C_AXIS, lw=2.5),
        zorder=4,
    )
    ax.text(1.18, 0.08, "anti-CoE\n(+x)", color=C_ANTI, fontsize=8, ha="left", va="bottom")
    ax.text(-1.18, 0.08, "CoE\n(−x)", color=C_COE, fontsize=8, ha="right", va="bottom")

    for (lo, hi), mid, color in zip(
        geo["max_bands_deg"], geo["theta_max_from_plus_x_deg"], COLORS_MAX
    ):
        draw_panelA_half_ribbon(ax, lo, hi, mid, color, roll_deg)
    for (lo, hi), mid, color in zip(
        geo["min_bands_deg"], geo["theta_min_from_plus_x_deg"], COLORS_MIN
    ):
        draw_panelA_half_ribbon(ax, lo, hi, mid, color, roll_deg)

    draw_panelA_theta_rays(ax, geo)

    label_r = 1.30
    for mid, color in list(zip(geo["theta_max_from_plus_x_deg"], COLORS_MAX)) + list(
        zip(geo["theta_min_from_plus_x_deg"], COLORS_MIN)
    ):
        t = np.deg2rad(mid)
        ax.text(
            label_r * np.cos(t),
            label_r * np.sin(t),
            rf"$\theta\!=\!{mid:.0f}^\circ$",
            color=color, fontsize=7.5, ha="center", va="center", fontweight="bold",
        )

    ax.text(
        0.0,
        -1.38,
        (
```

```
            r"$\theta$-ray-side half-loops  ·  keep $y\,\mathrm{sgn}(\sin\theta)\geq0$  ·  "
            r"each $18^\circ$ end-to-end ($\pm9^\circ$)"
            + "\n"
            + r"(after-2$\ell$ $A(\ell)$: two MAX + two MIN; open $180^\circ$ arcs — no full loops)"
        ),
        fontsize=7.5,
        ha="center",
        va="top",
        style="italic",
        color="#34495e",
    )
    ax.set_xlim(-1.55, 1.55)
    ax.set_ylim(-1.55, 1.45)
    ax.set_aspect("equal")
    ax.axis("off")


def panel_B_mollweide(ax, geo, roll_deg):
    tilt = geo["tilt_b_deg"]
    roll_note = (
        "\n" + rf"$+{roll_deg:.0f}^\circ$ CCW about CoE"
        if abs(roll_deg) > 0.5
        else ""
    )
    ax.set_title(
        rf"B. Migkas $A(\ell)$ half-loops on sky (anti-CoE at $(0^\circ,+{tilt:.0f}^\circ)$)"
        "\n"
        rf"(Galactic Mollweide; CoE axis tilted $+{tilt:.0f}^\circ$; "
        rf"$\theta$-ray-side half-loops only; $\ell$ increases LEFT){roll_note}",
        fontsize=9.5,
        pad=8,
    )
    for b in [-60, -30, 0, 30, 60]:
        ell = np.linspace(0, 360, 361)
        plot_lb_curve_mollweide(ax, ell, np.full_like(ell, b), color="#bdc3c7", lw=0.5, zorder=0)
    for ell0 in range(0, 360, 30):
        b = np.linspace(-90, 90, 181)
        plot_lb_curve_mollweide(ax, np.full_like(b, ell0), b, color="#bdc3c7", lw=0.5, zorder=0)

    D = geo["D"]
    ls, bs = great_circle_through_poles(D, n=721)
    plot_lb_curve_mollweide(ax, ls, bs, color=C_AXIS, lw=2.0, zorder=2)

    for (lo, hi), mid, color in zip(
        geo["max_bands_deg"], geo["theta_max_from_plus_x_deg"], COLORS_MAX
    ):
        fill_mollweide_half_ribbon(ax, D, lo, hi, mid, color, roll_deg)
    for (lo, hi), mid, color in zip(
        geo["min_bands_deg"], geo["theta_min_from_plus_x_deg"], COLORS_MIN
    ):
        fill_mollweide_half_ribbon(ax, D, lo, hi, mid, color, roll_deg)

    ell_eq, b_eq = celestial_equator_lb(721)
    plot_lb_curve_mollweide(ax, ell_eq, b_eq, color=C_EQ, lw=1.8, ls="--", zorder=2)
```

```
    ax.plot(
        np.deg2rad(lon_disp_from_ell(geo["anti_lb"][0])),
        np.deg2rad(geo["anti_lb"][1]),
        marker="o", ms=8, color=C_ANTI,
        markerfacecolor=C_ANTI, markeredgecolor="k", markeredgewidth=0.6,
        zorder=6, linestyle="None",
    )
    ax.plot(
        np.deg2rad(lon_disp_from_ell(geo["coe_lb"][0])),
        np.deg2rad(geo["coe_lb"][1]),
        marker="*", ms=14, color=C_COE,
        markerfacecolor=C_COE, markeredgecolor="k", markeredgewidth=0.6,
        zorder=6, linestyle="None",
    )
    for (l, b), color in zip(MIGKAS_MAXES, COLORS_MAX):
        ax.plot(
            np.deg2rad(lon_disp_from_ell(l)), np.deg2rad(b),
            marker="^", ms=10, color=color,
            markeredgecolor="k", markeredgewidth=0.7, zorder=7, linestyle="None",
        )
    for (l, b), color in zip(MIGKAS_MINS, COLORS_MIN):
        ax.plot(
            np.deg2rad(lon_disp_from_ell(l)), np.deg2rad(b),
            marker="s", ms=10, color=color,
            markeredgecolor="k", markeredgewidth=0.7, zorder=7, linestyle="None",
        )

    xticks = np.deg2rad(lon_disp_from_ell(np.array([180, 90, 0, 270])))
    ax.set_xticks(xticks)
    ax.set_xticklabels([r"$180^\circ$", r"$90^\circ$", r"$0^\circ$", r"$270^\circ$"], fontsize=8)
    ax.set_yticks(np.deg2rad([-60, -30, 0, 30, 60]))
    ax.set_yticklabels(
        [r"$-60^\circ$", r"$-30^\circ$", r"$0^\circ$", r"$+30^\circ$", r"$+60^\circ$"], fontsize=8
    )
    ax.grid(False)
    ax.set_xlabel(r"Galactic $\ell$ (increases LEFT)", fontsize=8)
    ax.set_ylabel(r"Galactic $b$", fontsize=8)


def fill_sphere_half_ribbon(ax, D, theta_lo, theta_hi, theta_mid, color, roll_deg, n_pts=160):
    mu_a, mu_b = mu_pair_for_band(theta_lo, theta_hi)
    mu_mid = float(np.cos(np.deg2rad(theta_mid)))
    p0, _ = small_circle_theta_ray_half(
        D, float(mu_a), theta_mid, roll_deg=roll_deg, n=n_pts
    )
    p1, _ = small_circle_theta_ray_half(
        D, float(mu_b), theta_mid, roll_deg=roll_deg, n=n_pts
    )
    verts = [[p0[j], p0[j + 1], p1[j + 1], p1[j]] for j in range(n_pts - 1)]
    coll = Poly3DCollection(
        verts, facecolors=color, edgecolors="none", linewidths=0.0,
        alpha=0.25, zorder=2,
    )
```

```
    ax.add_collection3d(coll)
    pts, _ = small_circle_theta_ray_half(
        D, mu_mid, theta_mid, roll_deg=roll_deg, n=n_pts
    )
    ax.plot(
        pts[:, 0], pts[:, 1], pts[:, 2],
        color=color, lw=2.6, alpha=0.95, zorder=5, solid_capstyle="round",
    )


def draw_wire_sphere(ax, n_mer=12, n_par=6, **kwargs):
    u = np.linspace(0, 2 * np.pi, 60)
    v = np.linspace(-np.pi / 2, np.pi / 2, 40)
    xs = np.outer(np.cos(u), np.cos(v))
    ys = np.outer(np.sin(u), np.cos(v))
    zs = np.outer(np.ones_like(u), np.sin(v))
    ax.plot_surface(xs, ys, zs, color="#dfe6e9", alpha=0.15, linewidth=0, shade=False)
    kw = dict(color="#95a5a6", lw=0.4, alpha=0.45)
    kw.update(kwargs)
    for i in range(n_mer):
        phi = 2 * np.pi * i / n_mer
        t = np.linspace(-np.pi / 2, np.pi / 2, 80)
        ax.plot(np.cos(t) * np.cos(phi), np.cos(t) * np.sin(phi), np.sin(t), **kw)
    for j in range(1, n_par):
        b = -np.pi / 2 + np.pi * j / n_par
        t = np.linspace(0, 2 * np.pi, 80)
        r = np.cos(b)
        ax.plot(r * np.cos(t), r * np.sin(t), np.full_like(t, np.sin(b)), **kw)


def panel_C_sphere(ax, geo, roll_deg):
    th_max = geo["theta_max_from_plus_x_deg"]
    th_min = geo["theta_min_from_plus_x_deg"]
    roll_note = (
        rf"; $+{roll_deg:.0f}^\circ$ CCW about CoE"
        if abs(roll_deg) > 0.5
        else ""
    )
    ax.set_title(
        rf"C. Sphere 3D — $\theta$-ray-side half-loops{roll_note}"
        "\n"
        r"(open $180^\circ$ arcs; celestial equator dashed)",
        fontsize=9.5,
        pad=6,
    )
    draw_wire_sphere(ax)
    D = geo["D"]
    ell_eq, b_eq = celestial_equator_lb(400)
    eq_xyz = np.array([lb_to_n(l, b) for l, b in zip(ell_eq, b_eq)])
    ax.plot(eq_xyz[:, 0], eq_xyz[:, 1], eq_xyz[:, 2], color=C_EQ, lw=1.8, ls="--", zorder=3)
    ax.plot(
        [-D[0], D[0]], [-D[1], D[1]], [-D[2], D[2]],
        color=C_AXIS, lw=3.0, zorder=5,
    )
```

```
    ax.scatter(*(-D), color=C_COE, s=120, marker="*", edgecolors="k", linewidths=0.6,
               depthshade=False, zorder=8)
    ax.scatter(*(D), color=C_ANTI, s=80, edgecolors="k", linewidths=0.6, depthshade=False, zorder=8)
    ax.text(*(-D * 1.22), "CoE", color=C_COE, fontsize=8, fontweight="bold")
    ax.text(*(D * 1.18), "anti-CoE", color=C_ANTI, fontsize=8, fontweight="bold")
    ax.scatter([0], [0], [0], color="k", s=25, depthshade=False, zorder=9)

    for (lo, hi), mid, color in zip(
        geo["max_bands_deg"], geo["theta_max_from_plus_x_deg"], COLORS_MAX
    ):
        fill_sphere_half_ribbon(ax, D, lo, hi, mid, color, roll_deg)
    for (lo, hi), mid, color in zip(
        geo["min_bands_deg"], geo["theta_min_from_plus_x_deg"], COLORS_MIN
    ):
        fill_sphere_half_ribbon(ax, D, lo, hi, mid, color, roll_deg)

    for n, color in zip(geo["n_maxes"], COLORS_MAX):
        ax.scatter(
            [n[0]], [n[1]], [n[2]],
            c=[color], marker="^", s=70,
            edgecolors="k", linewidths=0.7, depthshade=False, zorder=10,
        )
        ax.plot([0, n[0]], [0, n[1]], [0, n[2]], color=color, lw=0.8, ls=":", alpha=0.7)
    for n, color in zip(geo["n_mins"], COLORS_MIN):
        ax.scatter(
            [n[0]], [n[1]], [n[2]],
            c=[color], marker="s", s=60,
            edgecolors="k", linewidths=0.7, depthshade=False, zorder=10,
        )
        ax.plot([0, n[0]], [0, n[1]], [0, n[2]], color=color, lw=0.8, ls=":", alpha=0.7)

    ax.set_xlim(-1.25, 1.25)
    ax.set_ylim(-1.25, 1.25)
    ax.set_zlim(-1.25, 1.25)
    ax.set_box_aspect((1, 1, 1))
    ax.view_init(elev=18, azim=-55)
    ax.set_axis_off()
    ax.text2D(
        0.5, -0.02,
        rf"MAX $\theta\!=\!{th_max[0]:.0f}^\circ,{th_max[1]:.0f}^\circ$; "
        rf"MIN $\theta\!=\!{th_min[0]:.0f}^\circ,{th_min[1]:.0f}^\circ$  "
        rf"($\theta$-ray-side half-loops; $18^\circ$ end-to-end = $\pm9^\circ$)",
        transform=ax.transAxes, ha="center", fontsize=7.5, color="#34495e",
    )


def compute_geometry(tilt_b, roll_deg):
    pose = axis_pose(tilt_b)
    th_max = list(STRIP_THETA_MAX)
    th_min = list(STRIP_THETA_MIN)
    max_bands = band_edges(th_max)
    min_bands = band_edges(th_min)
    n_maxes = [lb_to_n(*lb) for lb in MIGKAS_MAXES]
    n_mins = [lb_to_n(*lb) for lb in MIGKAS_MINS]
```

```
    sep = float(
        np.rad2deg(np.arccos(np.clip(np.dot(n_maxes[0], n_mins[1]), -1.0, 1.0)))
    )
    return {
        **pose,
        "n_maxes": n_maxes,
        "n_mins": n_mins,
        "theta_max_from_plus_x_deg": th_max,
        "theta_min_from_plus_x_deg": th_min,
        "max_bands_deg": max_bands,
        "min_bands_deg": min_bands,
        "mu_max_bands": [mu_pair_for_band(lo, hi) for lo, hi in max_bands],
        "mu_min_bands": [mu_pair_for_band(lo, hi) for lo, hi in min_bands],
        "sep_deg": sep,
        "band_half_width_deg": BAND_HALF,
        "roll_about_coe_deg": float(roll_deg),
        "half_loops": True,
        "half_loop_rule": HALF_LOOP_RULE,
    }


def main(tilt_b=None, roll_deg=None):
    tilt_b = float(AXIS_TILT_B_DEG if tilt_b is None else tilt_b)
    roll_deg = float(ROLL_ABOUT_COE_DEG if roll_deg is None else roll_deg)
    assert HALF_LOOPS

    geo = compute_geometry(tilt_b, roll_deg)
    th_max = geo["theta_max_from_plus_x_deg"]
    th_min = geo["theta_min_from_plus_x_deg"]
    max_bands = geo["max_bands_deg"]
    min_bands = geo["min_bands_deg"]

    stem = output_stem(tilt_b, roll_deg)
    png_path = OUT_DIR / f"{stem}.png"
    pdf_path = OUT_DIR / f"{stem}.pdf"
    json_path = OUT_DIR / f"{stem}.json"

    roll_sup = (
        rf"; $+{roll_deg:.0f}^\circ$ CCW about CoE axis"
        if abs(roll_deg) > 0.5
        else ""
    )
    fig = plt.figure(figsize=(14.5, 5.6), dpi=160)
    fig.suptitle(
        r"Pure CoE axisymmetry: $\theta$-ray-side half-loops "
        r"(keep $(n\cdot\hat y)\sin\theta\geq0$)"
        "\n"
        rf"(each $18^\circ$ end-to-end = $\pm9^\circ$; "
        rf"MAX $\theta\!=\!{th_max[0]:.0f}^\circ/{th_max[1]:.0f}^\circ$; "
        rf"MIN $\theta\!=\!{th_min[0]:.0f}^\circ/{th_min[1]:.0f}^\circ$; "
        rf"anti-CoE $(0^\circ,+{tilt_b:.0f}^\circ)$, CoE axis tilted $+{tilt_b:.0f}^\circ$"
        f"{roll_sup})",
        fontsize=11,
        fontweight="bold",
```

```
        y=1.02,
    )

    axA = fig.add_subplot(1, 3, 1)
    panel_A_model_frame(axA, geo, 0.0)  # model frame never rolls
    axB = fig.add_subplot(1, 3, 2, projection="mollweide")
    panel_B_mollweide(axB, geo, roll_deg)
    axC = fig.add_subplot(1, 3, 3, projection="3d")
    panel_C_sphere(axC, geo, roll_deg)

    handles = [
        Line2D([0], [0], color=C_AXIS, lw=2.5, label=r"CoE$\leftrightarrow$anti-CoE axis"),
        Patch(
            facecolor=C_MAX_30, alpha=0.45, edgecolor=C_MAX_30,
            label=rf"MAX $\theta\!=\!{th_max[0]:.0f}^\circ$ $\theta$-ray half "
            rf"[{max_bands[0][0]:.0f},{max_bands[0][1]:.0f}]$^\circ$",
        ),
        Patch(
            facecolor=C_MAX_185, alpha=0.45, edgecolor=C_MAX_185,
            label=rf"MAX $\theta\!=\!{th_max[1]:.0f}^\circ$ $\theta$-ray half "
            rf"[{max_bands[1][0]:.0f},{max_bands[1][1]:.0f}]$^\circ$",
        ),
        Patch(
            facecolor=C_MIN_115, alpha=0.45, edgecolor=C_MIN_115,
            label=rf"MIN $\theta\!=\!{th_min[0]:.0f}^\circ$ $\theta$-ray half "
            rf"[{min_bands[0][0]:.0f},{min_bands[0][1]:.0f}]$^\circ$",
        ),
        Patch(
            facecolor=C_MIN_265, alpha=0.45, edgecolor=C_MIN_265,
            label=rf"MIN $\theta\!=\!{th_min[1]:.0f}^\circ$ $\theta$-ray half "
            rf"[{min_bands[1][0]:.0f},{min_bands[1][1]:.0f}]$^\circ$",
        ),
        Line2D([0], [0], color=C_EQ, lw=1.8, ls="--", label="celestial equator (Dec$=0$)"),
        Line2D(
            [0], [0], marker="^", color="w", markerfacecolor=C_MAX_30, markeredgecolor="k",
            ms=9, label=rf"MAX $({MIGKAS_MAXES[0][0]:.0f}^\circ,{MIGKAS_MAXES[0][1]:+.0f}^\circ)$",
        ),
        Line2D(
            [0], [0], marker="^", color="w", markerfacecolor=C_MAX_185, markeredgecolor="k",
            ms=9, label=rf"MAX $({MIGKAS_MAXES[1][0]:.0f}^\circ,{MIGKAS_MAXES[1][1]:+.0f}^\circ)$",
        ),
        Line2D(
            [0], [0], marker="s", color="w", markerfacecolor=C_MIN_115, markeredgecolor="k",
            ms=8, label=rf"MIN $({MIGKAS_MINS[0][0]:.0f}^\circ,{MIGKAS_MINS[0][1]:+.0f}^\circ)$",
        ),
        Line2D(
            [0], [0], marker="s", color="w", markerfacecolor=C_MIN_265, markeredgecolor="k",
            ms=8, label=rf"MIN $({MIGKAS_MINS[1][0]:.0f}^\circ,{MIGKAS_MINS[1][1]:+.0f}^\circ)$",
        ),
        Line2D(
            [0], [0], marker="o", color="w", markerfacecolor=C_ANTI, markeredgecolor="k",
            ms=8, label=rf"anti-CoE $({geo['anti_lb'][0]:.0f}^\circ,{geo['anti_lb'][1]:+.0f}^\circ)$",
        ),
        Line2D(
```

```
        [0], [0], marker="*", color="w", markerfacecolor=C_COE, markeredgecolor="k",
        ms=11, label=rf"CoE $({geo['coe_lb'][0]:.0f}^\circ,{geo['coe_lb'][1]:+.0f}^\circ)$",
    ),
]
fig.legend(
    handles=handles, loc="lower center", ncol=6, fontsize=7.0,
    frameon=True, fancybox=True, bbox_to_anchor=(0.5, -0.10),
)
fig.tight_layout()
fig.savefig(png_path, bbox_inches="tight", facecolor="white")
fig.savefig(pdf_path, bbox_inches="tight", facecolor="white")
plt.close(fig)

payload = {
    "png": str(png_path),
    "pdf": str(pdf_path),
    "script": str(OUT_DIR / "make_coe_rings_half_loops.py"),
    "geometry_module": str(OUT_DIR / "coe_half_loop_geometry.py"),
    "HALF_LOOPS": True,
    "half_loop_rule": HALF_LOOP_RULE,
    "axis_tilt_b_deg": tilt_b,
    "roll_about_coe_deg": roll_deg,
    "migkas_max_lb_deg": [list(lb) for lb in MIGKAS_MAXES],
    "migkas_min_lb_deg": [list(lb) for lb in MIGKAS_MINS],
    "strip_theta_max_deg": list(STRIP_THETA_MAX),
    "strip_theta_min_deg": list(STRIP_THETA_MIN),
    "band_half_width_deg": BAND_HALF,
    "band_full_width_deg": BAND_FULL,
    "max_bands_deg": [list(b) for b in max_bands],
    "min_bands_deg": [list(b) for b in min_bands],
    "CoE_lb_deg": [round(geo["coe_lb"][0], 4), round(geo["coe_lb"][1], 4)],
    "anti_CoE_lb_deg": [round(geo["anti_lb"][0], 4), round(geo["anti_lb"][1], 4)],
    "D_hat_galactic_xyz": geo["D"].tolist(),
    "angular_sep_migkas_max0_min1_deg": round(geo["sep_deg"], 6),
    "orientation": (
        f"anti-CoE at (0°, +{tilt_b:.0f}°), CoE at (180°, −{tilt_b:.0f}°); "
        f"theta-ray-side half-loops (n·Y)*sin(theta)>=0; roll about CoE = {roll_deg:.0f}° CCW"
    ),
    "plot_convention": "Migkas Galactic Mollweide: lon_disp=-ell_wrapped (ell increases LEFT)",
    "colors": {
        "MAX_30": C_MAX_30,
        "MAX_185": C_MAX_185,
        "MIN_115": C_MIN_115,
        "MIN_265": C_MIN_265,
    },
}
json_path.write_text(json.dumps(payload, indent=2) + "\n")
print(json.dumps(payload, indent=2))
print(f"\nWrote {png_path}")
print(f"Wrote {pdf_path}")
print(f"Wrote {json_path}")
return png_path, pdf_path, json_path
```

```
if __name__ == "__main__":
    p = argparse.ArgumentParser(description=__doc__)
    p.add_argument("--tilt", type=float, default=AXIS_TILT_B_DEG)
    p.add_argument("--roll", type=float, default=ROLL_ABOUT_COE_DEG,
                   help="CCW degrees about +D (0 or 45)")
    args = p.parse_args()
    main(tilt_b=args.tilt, roll_deg=args.roll)
```

```
{
  "png": "/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/coe_rings_half_loops_tilt15.png",
  "pdf": "/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/coe_rings_half_loops_tilt15.pdf",
  "script": "/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/make_coe_rings_half_loops.py",
  "geometry_module": "/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/coe_half_loop_geometry.py",
  "HALF_LOOPS": true,
  "half_loop_rule": "\u03b8-ray side: keep (n\u00b7\u0176)*sin(\u03b8_mid)>=0 (Panel A: ey*sin(\u03b8_mid)>=0; TOP if sin\u03b8>0, BOTTOM if
sin\u03b8<0)",
  "axis_tilt_b_deg": 15.0,
  "roll_about_coe_deg": 0.0,
  "migkas_max_lb_deg": [
    [
      17.0,
      -9.0
    ],
    [
      185.0,
      0.0
    ]
  ],
  "migkas_min_lb_deg": [
    [
      115.0,
      0.0
    ],
    [
      274.0,
      -22.0
    ]
  ],
  "strip_theta_max_deg": [
    30.0,
    185.0
  ],
  "strip_theta_min_deg": [
    115.0,
    265.0
  ],
  "band_half_width_deg": 9.0,
  "band_full_width_deg": 18.0,
  "max_bands_deg": [
    [
      21.0,
      39.0
    ],
    [
      176.0,
      194.0
    ]
  ],
  "min_bands_deg": [
    [
      106.0,
      124.0
    ],
    [
      256.0,
```

```
      274.0
    ]
  ],
  "CoE_lb_deg": [
    180.0,
    -15.0
  ],
  "anti_CoE_lb_deg": [
    0.0,
    15.0
  ],
  "D_hat_galactic_xyz": [
    0.9659258262890683,
    0.0,
    0.25881904510252074
  ],
  "angular_sep_migkas_max0_min1_deg": 98.476384,
  "orientation": "anti-CoE at (0\u00b0, +15\u00b0), CoE at (180\u00b0, \u221215\u00b0); theta-ray-side half-loops (n\u00b7Y)*sin(theta)>=0; roll about
CoE = 0\u00b0 CCW",
  "plot_convention": "Migkas Galactic Mollweide: lon_disp=-ell_wrapped (ell increases LEFT)",
  "colors": {
    "MAX_30": "#c0392b",
    "MAX_185": "#ec7063",
    "MIN_115": "#1a5276",
    "MIN_265": "#5dade2"
  }
}
```


```
Wrote /workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/coe_rings_half_loops_tilt15.png
Wrote /workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/coe_rings_half_loops_tilt15.pdf
Wrote /workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/coe_rings_half_loops_tilt15.json
```

```
%%runscript make_coe_rings_half_loops.py_roll45 --argv --roll 45
# second run of the same script with the +45 deg CCW roll: load its definitions (everything above its __main__ block), then call main()
exec(open('/workspace/cosmology/figures/make_coe_rings_half_loops.py').read().split('if __name__ == "__main__":')[0])
main(tilt_b=15.0, roll_deg=45.0)
```

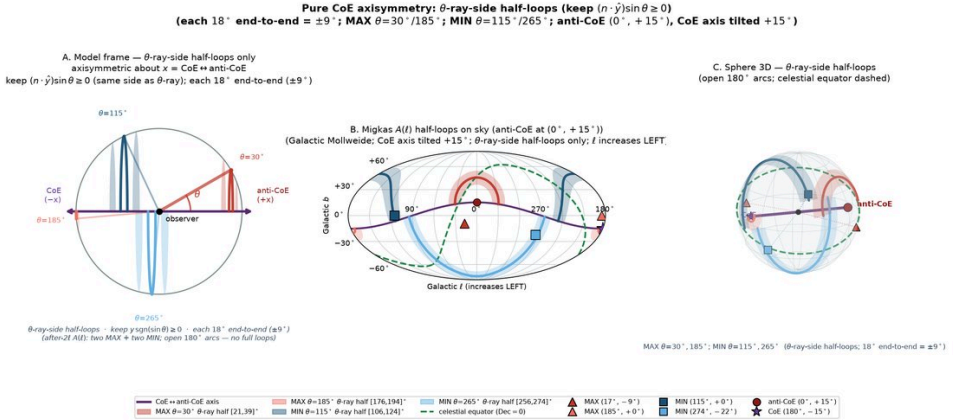
```
{
  "png": "/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/coe_rings_half_loops_tilt15_rot45ccw.png",
  "pdf": "/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/coe_rings_half_loops_tilt15_rot45ccw.pdf",
  "script": "/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/make_coe_rings_half_loops.py",
  "geometry_module": "/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/coe_half_loop_geometry.py",
  "HALF_LOOPS": true,
  "half_loop_rule": "\u03b8-ray side: keep (n\u00b7\u0176)*sin(\u03b8_mid)>=0 (Panel A: ey*sin(\u03b8_mid)>=0; TOP if sin\u03b8>0, BOTTOM if
sin\u03b8<0)",
  "axis_tilt_b_deg": 15.0,
  "roll_about_coe_deg": 45.0,
  "migkas_max_lb_deg": [
    [
      17.0,
      -9.0
    ],
    [
      185.0,
      0.0
    ]
  ],
  "migkas_min_lb_deg": [
    [
      115.0,
      0.0
    ],
    [
      274.0,
      -22.0
    ]
  ],
  "strip_theta_max_deg": [
    30.0,
    185.0
  ],
  "strip_theta_min_deg": [
    115.0,
    265.0
  ],
  "band_half_width_deg": 9.0,
  "band_full_width_deg": 18.0,
  "max_bands_deg": [
    [
      21.0,
      39.0
    ],
    [
      176.0,
      194.0
    ]
  ],
  "min_bands_deg": [
    [
      106.0,
      124.0
    ],
    [
      256.0,
```

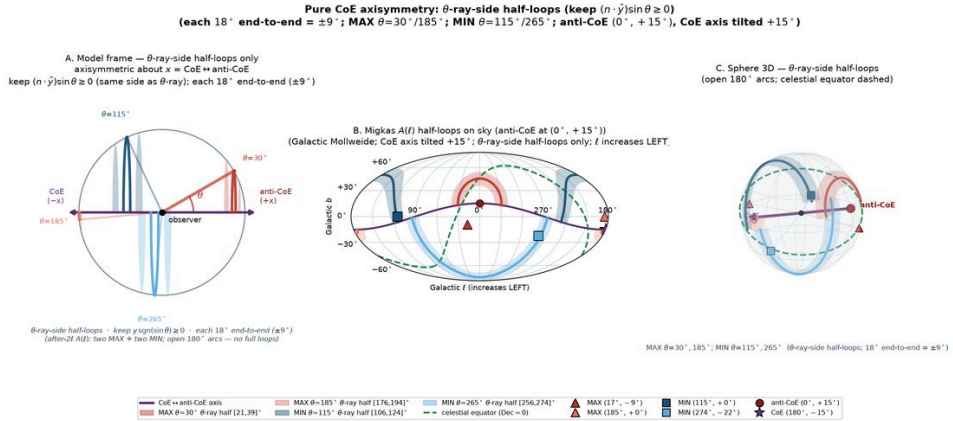
```
      274.0
    ]
  ],
  "CoE_lb_deg": [
    180.0,
    -15.0
  ],
  "anti_CoE_lb_deg": [
    0.0,
    15.0
  ],
  "D_hat_galactic_xyz": [
    0.9659258262890683,
    0.0,
    0.25881904510252074
  ],
  "angular_sep_migkas_max0_min1_deg": 98.476384,
  "orientation": "anti-CoE at (0\u00b0, +15\u00b0), CoE at (180\u00b0, \u221215\u00b0); theta-ray-side half-loops (n\u00b7Y)*sin(theta)>=0; roll about
CoE = 45\u00b0 CCW",
  "plot_convention": "Migkas Galactic Mollweide: lon_disp=-ell_wrapped (ell increases LEFT)",
  "colors": {
    "MAX_30": "#c0392b",
    "MAX_185": "#ec7063",
    "MIN_115": "#1a5276",
    "MIN_265": "#5dade2"
  }
}

Wrote /workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/coe_rings_half_loops_tilt15_rot45ccw.png
Wrote /workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/coe_rings_half_loops_tilt15_rot45ccw.pdf
Wrote /workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/coe_rings_half_loops_tilt15_rot45ccw.json
```

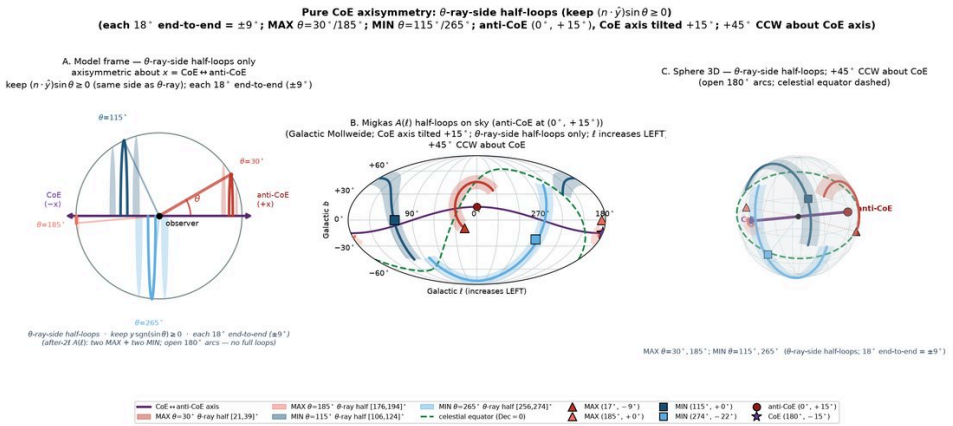

```
compare(FIGS / 'coe_rings_half_loops_tilt15.png', '/workspace/cosmology/figures/coe_rings_half_loops_tilt15.png')
compare(FIGS / 'coe_rings_half_loops_tilt15_rot45ccw.png', '/workspace/cosmology/figures/coe_rings_half_loops_tilt15_rot45ccw.png')
```

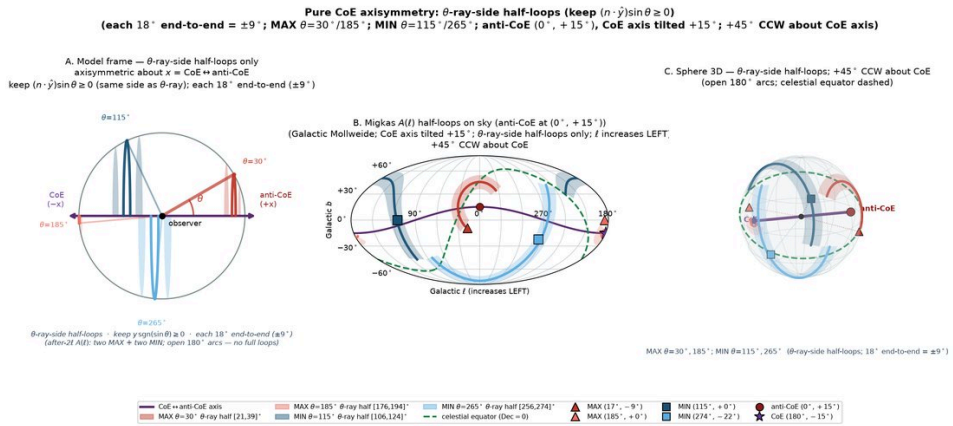
```
coe_rings_half_loops_tilt15.png: new (2272, 1027) vs ref (2272, 1027) | mean |diff| = 0.000/255, pixels differing >40: 0.000% -> identical
left: approved reference figures/coe_rings_half_loops_tilt15.png   |   right: notebook re-run steps_v3/figs/coe_rings_half_loops_tilt15.png
```






```
coe_rings_half_loops_tilt15_rot45ccw.png: new (2272, 1027) vs ref (2272, 1027) | mean |diff| = 0.000/255, pixels differing >40: 0.000% -> identical
left: approved reference figures/coe_rings_half_loops_tilt15_rot45ccw.png   |   right: notebook re-run
steps_v3/figs/coe_rings_half_loops_tilt15_rot45ccw.png
```





Step 13 — Half-loops on Migkas Fig. 8 (roll 0 and $+45^\circ$ CCW)

What it shows. The Step 12 half-arcs drawn on the Migkas Fig. 8 $\theta = 75^\circ$ map, with the same registration and marker positions as Step 11. The Migkas markers stay fixed; no Dec $= 0$ line is drawn.

Source script: `/workspace/cosmology/figures/make_migkas_fig8_half_loops_overlay.py` (last modified 2026-09-25 20:22 CT), run verbatim. *Output-path edit:* none. The script's `OUT_DIR` follows `__file__`, which is set to `steps_v3/figs/`. *Reference image:* `figures/migkas_fig8_half_loops_tilt15.png`, `figures/migkas_fig8_half_loops_tilt15_rot45ccw.png`.

```
%%runscript make_migkas_fig8_half_loops_overlay.py --argv --roll 0
#!/usr/bin/env python3
"""
Migkas Fig. 8 overlay — OUTER CoE half-loops (Chris 2026-09-23 whiteout).

Matches make_coe_rings_half_loops.py geometry:
  AXIS_TILT_B_DEG = 15 → anti-CoE (0°, +15°), CoE (180°, −15°)
  ROLL_ABOUT_COE_DEG = 0 or 45
  OUTER half-arcs only (sign(mu)*cos(phi-roll) >= 0)

GOOD_REG and lon_disp = -wrap(ℓ) unchanged from working Fig. 8 overlay.
Migkas markers FIXED. No Dec=0 overlay (published CEQ kept).

Outputs:
  roll=0  → migkas_fig8_half_loops_tilt15.{png,json}
  roll=45 → migkas_fig8_half_loops_tilt15_rot45ccw.{png,json}
"""

from __future__ import annotations

import argparse
import json
```

```
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
from matplotlib.collections import PolyCollection
from matplotlib.lines import Line2D
from matplotlib.patches import Ellipse, Patch
from PIL import Image

from coe_half_loop_geometry import (
    BAND_FULL,
    BAND_HALF,
    MIGKAS_MAXES,
    MIGKAS_MINS,
    STRIP_THETA_MAX,
    STRIP_THETA_MIN,
    axis_pose,
    band_edges,
    celestial_equator_lb,
    great_circle_through_poles,
    lon_disp_from_ell,
    mu_pair_for_band,
    n_to_lb,
    small_circle_outer_half,
)

OUT_DIR = Path(__file__).resolve().parent

AXIS_TILT_B_DEG = 15.0
ROLL_ABOUT_COE_DEG = 0.0
HALF_LOOPS = True

SOURCE_CANDIDATES = [
    Path("/workspace/cosmology/map_compare/FULL_migkas_theta75.png"),
    Path("/workspace/cosmology/map_compare/migkas_fig8_theta75_crop.png"),
    Path("/workspace/cosmology/migkas/pub_maps/Fig8_full_hires.png"),
]

THETA_MAX = list(STRIP_THETA_MAX)
THETA_MIN = list(STRIP_THETA_MIN)
MAX_BANDS = band_edges(THETA_MAX)
MIN_BANDS = band_edges(THETA_MIN)

GOOD_REG = {
    "cx": 863.0,
    "cy": 763.25,
    "Rx": 693.9927,
    "Ry": 392.542425,
}

C_MAX_30 = "#00e5ff"
C_MAX_185 = "#ff2d95"
C_MIN_115 = "#b8f200"
C_MIN_265 = "#4fc3f7"
```

```
COLORS_MAX = [C_MAX_30, C_MAX_185]
COLORS_MIN = [C_MIN_115, C_MIN_265]
C_COE = "#6b3fa0"
C_ANTI = "#8e1b1b"
C_AXIS = "#5b2c6f"


def output_stem(tilt_b, roll):
    base = f"migkas_fig8_half_loops_tilt{int(round(tilt_b))}"
    if abs(roll) > 0.5:
        base += f"_rot{int(round(roll))}ccw"
    return base


def mollweide_xy(lon_rad, lat_rad, n_iter=64):
    lon = np.asarray(lon_rad, dtype=float)
    lat = np.asarray(lat_rad, dtype=float)
    target = np.pi * np.sin(np.clip(lat, -np.pi / 2, np.pi / 2))
    lo = np.full_like(lat, -np.pi / 2, dtype=float)
    hi = np.full_like(lat, np.pi / 2, dtype=float)
    for _ in range(n_iter):
        mid = 0.5 * (lo + hi)
        g = 2.0 * mid + np.sin(2.0 * mid)
        lo = np.where(g < target, mid, lo)
        hi = np.where(g >= target, mid, hi)
    theta = 0.5 * (lo + hi)
    theta = np.where(lat >= np.pi / 2 - 1e-15, np.pi / 2, theta)
    theta = np.where(lat <= -np.pi / 2 + 1e-15, -np.pi / 2, theta)
    x = (2.0 * np.sqrt(2.0) / np.pi) * lon * np.cos(theta)
    y = np.sqrt(2.0) * np.sin(theta)
    return x, y


def lb_to_px(ell_deg, b_deg, cx, cy, Rx, Ry):
    lon = np.deg2rad(lon_disp_from_ell(np.asarray(ell_deg, dtype=float)))
    lat = np.deg2rad(np.asarray(b_deg, dtype=float))
    x, y = mollweide_xy(lon, lat)
    Xn = x / (2.0 * np.sqrt(2.0))
    Yn = y / np.sqrt(2.0)
    return cx + Xn * Rx, cy - Yn * Ry


def as_xy(px, py):
    return float(np.asarray(px).reshape(-1)[0]), float(np.asarray(py).reshape(-1)[0])


def chamfer_distance_to_dark(dark):
    h, w = dark.shape
    INF = 1e6
    D = np.where(dark, 0.0, INF)
    for y in range(h):
        for x in range(w):
            v = D[y, x]
            if x > 0:
```

```
                v = min(v, D[y, x - 1] + 1.0)
            if y > 0:
                v = min(v, D[y - 1, x] + 1.0)
            if x > 0 and y > 0:
                v = min(v, D[y - 1, x - 1] + 1.414)
            if x + 1 < w and y > 0:
                v = min(v, D[y - 1, x + 1] + 1.414)
            D[y, x] = v
    for y in range(h - 1, -1, -1):
        for x in range(w - 1, -1, -1):
            v = D[y, x]
            if x + 1 < w:
                v = min(v, D[y, x + 1] + 1.0)
            if y + 1 < h:
                v = min(v, D[y + 1, x] + 1.0)
            if x + 1 < w and y + 1 < h:
                v = min(v, D[y + 1, x + 1] + 1.414)
            if x > 0 and y + 1 < h:
                v = min(v, D[y + 1, x - 1] + 1.414)
            D[y, x] = v
    return D


def detect_oval_bbox(rgb):
    h, w, _ = rgb.shape
    white = (rgb[:, :, 0] > 250) & (rgb[:, :, 1] > 250) & (rgb[:, :, 2] > 250)
    oval_left = np.full(h, np.nan)
    oval_right = np.full(h, np.nan)
    for y in range(h):
        row = ~white[y]
        d = np.diff(np.r_[0, row.astype(int), 0])
        starts = np.where(d == 1)[0]
        ends = np.where(d == -1)[0] - 1
        if len(starts) == 0:
            continue
        lengths = ends - starts + 1
        candidates = [
            (int(lengths[i]), int(starts[i]), int(ends[i]))
            for i in range(len(starts))
            if starts[i] < 0.72 * w and lengths[i] > 40
        ]
        if not candidates:
            continue
        candidates.sort(reverse=True)
        _, L, R = candidates[0]
        oval_left[y] = L
        oval_right[y] = R
    widths = np.where(np.isfinite(oval_left), oval_right - oval_left, 0.0)
    yeq = int(np.argmax(widths))
    thresh = 0.08 * widths[yeq]
    y0 = yeq
    while y0 > 0 and widths[y0 - 1] >= thresh:
        y0 -= 1
    y1 = yeq
```

```
    while y1 < h - 1 and widths[y1 + 1] >= thresh:
        y1 += 1
    x0 = float(oval_left[yeq])
    x1 = float(oval_right[yeq])
    return {
        "x0": int(x0), "x1": int(x1), "y0": int(y0), "y1": int(y1),
        "yeq": int(yeq),
        "cx": 0.5 * (x0 + x1), "cy": 0.5 * (y0 + y1),
        "Rx": 0.5 * (x1 - x0), "Ry": 0.5 * (y1 - y0 + 1),
    }


def choose_source():
    for p in SOURCE_CANDIDATES:
        if p.is_file():
            return p
    raise FileNotFoundError("No Migkas Fig.8 theta=75 source image found")


def sample_rgb(rgb, ell, b, cx, cy, Rx, Ry, r=4):
    px, py = lb_to_px(ell, b, cx, cy, Rx, Ry)
    xi, yi = int(round(as_xy(px, py)[0])), int(round(as_xy(px, py)[1]))
    patch = rgb[max(0, yi - r) : yi + r + 1, max(0, xi - r) : xi + r + 1]
    return patch.mean(axis=(0, 1))


def validate_landmarks(rgb, reg):
    cx, cy, Rx, Ry = reg["cx"], reg["cy"], reg["Rx"], reg["Ry"]
    pts = {}
    landmarks = [
        ("center", 0.0, 0.0),
        ("ell90", 90.0, 0.0),
        ("ell270", 270.0, 0.0),
        ("north", 0.0, 90.0),
        ("south", 0.0, -90.0),
    ]
    for i, (ell, b) in enumerate(MIGKAS_MAXES):
        landmarks.append((f"migkas_max{i}", ell, b))
    for i, (ell, b) in enumerate(MIGKAS_MINS):
        landmarks.append((f"migkas_min{i}", ell, b))
    for name, ell, b in landmarks:
        x, y = as_xy(*lb_to_px(ell, b, cx, cy, Rx, Ry))
        pts[name] = {"ell": ell, "b": b, "px": x, "py": y}
    assert pts["ell90"]["px"] < pts["center"]["px"], "l=90 must be LEFT of center"
    assert pts["ell270"]["px"] > pts["center"]["px"], "l=270 must be RIGHT of center"
    assert abs(pts["center"]["px"] - cx) < 1.5
    assert abs(pts["center"]["py"] - cy) < 1.5
    assert pts["north"]["py"] < pts["center"]["py"]
    assert pts["south"]["py"] > pts["center"]["py"]
    cmax = sample_rgb(rgb, *MIGKAS_MAXES[0], cx, cy, Rx, Ry)
    cmin = sample_rgb(rgb, *MIGKAS_MINS[1], cx, cy, Rx, Ry)
    assert (cmax[0] + cmax[1]) > (cmin[0] + cmin[1]) + 50
    return {
        "landmarks_px": pts,
```

```
        "migkas_max0_rgb_mean": [float(x) for x in cmax],
        "migkas_min1_rgb_mean": [float(x) for x in cmin],
    }


def inside_ellipse(px, py, ecx, ecy, eRx, eRy, r_lim=1.0):
    px = np.asarray(px, dtype=float)
    py = np.asarray(py, dtype=float)
    Xn = (px - ecx) / eRx
    Yn = (py - ecy) / eRy
    return Xn * Xn + Yn * Yn <= float(r_lim) * float(r_lim)


def plot_lb_curve(
    ax, ell, b, cx, cy, Rx, Ry, inset=1.0, jump_px=45.0,
    clip_r2=None, clip_path=None, color_oval=None, **kw
):
    px, py = lb_to_px(ell, b, cx, cy, Rx, Ry)
    px = np.asarray(px, dtype=float).reshape(-1)
    py = np.asarray(py, dtype=float).reshape(-1)
    px = cx + (px - cx) * inset
    py = cy + (py - cy) * inset
    Xn = (px - cx) / Rx
    Yn = (cy - py) / Ry
    r2_lim = (inset * inset) if clip_r2 is None else float(clip_r2)
    inside = Xn * Xn + Yn * Yn <= r2_lim
    if color_oval is not None:
        ocx, ocy, oRx, oRy, o_r = color_oval
        inside = inside & inside_ellipse(px, py, ocx, ocy, oRx, oRy, o_r)
    px = np.where(inside, px, np.nan)
    py = np.where(inside, py, np.nan)
    segs = []
    cur_x, cur_y = [], []
    for x, y in zip(px, py):
        if not (np.isfinite(x) and np.isfinite(y)):
            if len(cur_x) >= 2:
                segs.append((cur_x, cur_y))
            cur_x, cur_y = [], []
            continue
        if cur_x and np.hypot(x - cur_x[-1], y - cur_y[-1]) > jump_px:
            if len(cur_x) >= 2:
                segs.append((cur_x, cur_y))
            cur_x, cur_y = [float(x)], [float(y)]
        else:
            cur_x.append(float(x))
            cur_y.append(float(y))
    if len(cur_x) >= 2:
        segs.append((cur_x, cur_y))
    if clip_path is not None:
        kw["clip_path"] = clip_path
        kw["clip_on"] = True
    for sx, sy in segs:
        (line,) = ax.plot(sx, sy, **kw)
        if clip_path is not None:
```

```
            line.set_clip_path(clip_path)
            line.set_clip_on(True)


def _half_ring_px_arrays(D, mu, roll_deg, cx, cy, Rx, Ry, inset, n_pts=721, theta_mid=None):
    """OUTER half-arc → pixel arrays (open, azimuth-ordered along keep arc)."""
    pts, _ = small_circle_outer_half(D, float(mu), roll_deg=roll_deg, n=n_pts, theta_mid_deg=theta_mid)
    ell = np.empty(n_pts)
    b = np.empty(n_pts)
    for i, p in enumerate(pts):
        ell[i], b[i] = n_to_lb(p)
    lon = np.deg2rad(lon_disp_from_ell(ell))
    lat = np.deg2rad(b)
    x_m, y_m = mollweide_xy(lon, lat)
    Xn = x_m / (2.0 * np.sqrt(2.0))
    Yn = y_m / np.sqrt(2.0)
    px = cx + Xn * Rx
    py = cy - Yn * Ry
    px = cx + (px - cx) * inset
    py = cy + (py - cy) * inset
    return lon, px, py


def plot_half_ring(
    ax, D, mu, roll_deg, cx, cy, Rx, Ry, color, lw, inset=1.0, jump_px=80.0,
    clip_path=None, color_oval=None, theta_mid=None,
):
    """Stroke OUTER half-arc only (open 180°; never close into a full loop)."""
    lon, px, py = _half_ring_px_arrays(
        D, mu, roll_deg, cx, cy, Rx, Ry, inset, n_pts=2001, theta_mid=theta_mid
    )
    r2_lim = inset * inset + 1e-10

    def stroke_xy(xs, ys):
        if len(xs) < 2:
            return
        layers = (
            ("white", lw + 1.5, 3, 0.95),
            (color, lw, 4, 1.0),
        )
        for col, width, z, a in layers:
            (line,) = ax.plot(
                xs, ys, color=col, lw=width, linestyle="-",
                solid_capstyle="round", solid_joinstyle="round",
                zorder=z, alpha=a, clip_path=clip_path, clip_on=True,
            )
            if clip_path is not None:
                line.set_clip_path(clip_path)
                line.set_clip_on(True)

    # Split on antimeridian / pixel jumps / outside oval — NEVER close the arc
    gap = 0.9 * np.pi
    n = len(lon)
    bad = np.zeros(n - 1, dtype=bool)
```

```
    for i in range(n - 1):
        if abs(lon[i + 1] - lon[i]) > gap:
            bad[i] = True
        elif np.hypot(px[i + 1] - px[i], py[i + 1] - py[i]) > jump_px:
            bad[i] = True
        else:
            Xn2 = (px[i] - cx) / Rx
            Yn2 = (cy - py[i]) / Ry
            ok = (Xn2 * Xn2 + Yn2 * Yn2) <= r2_lim
            Xn3 = (px[i + 1] - cx) / Rx
            Yn3 = (cy - py[i + 1]) / Ry
            ok2 = (Xn3 * Xn3 + Yn3 * Yn3) <= r2_lim
            if color_oval is not None:
                ocx, ocy, oRx, oRy, o_r = color_oval
                ok = ok & inside_ellipse(px[i], py[i], ocx, ocy, oRx, oRy, o_r)
                ok2 = ok2 & inside_ellipse(px[i + 1], py[i + 1], ocx, ocy, oRx, oRy, o_r)
            if (not ok) or (not ok2):
                bad[i] = True

    i = 0
    while i < n - 1:
        while i < n - 1 and bad[i]:
            i += 1
        if i >= n - 1:
            break
        j = i
        while j < n - 1 and not bad[j]:
            j += 1
        if j - i >= 1:
            stroke_xy(list(px[i : j + 1]), list(py[i : j + 1]))
        i = j + 1


def fill_half_highlight_band(
    ax, D, theta_lo, theta_hi, theta_mid, roll_deg, cx, cy, Rx, Ry, color,
    inset=1.0, clip_path=None, color_oval=None, n_pts=721, jump_px=80.0,
):
    mu_a, mu_b = mu_pair_for_band(theta_lo, theta_hi)
    mu_mid = float(np.cos(np.deg2rad(theta_mid)))
    lon_a, px_a, py_a = _half_ring_px_arrays(
        D, mu_a, roll_deg, cx, cy, Rx, Ry, inset, n_pts=n_pts, theta_mid=theta_mid
    )
    lon_b, px_b, py_b = _half_ring_px_arrays(
        D, mu_b, roll_deg, cx, cy, Rx, Ry, inset, n_pts=n_pts, theta_mid=theta_mid
    )
    r2_lim = inset * inset + 1e-10

    def _inside(px, py):
        Xn2 = (px - cx) / Rx
        Yn2 = (cy - py) / Ry
        ok = (Xn2 * Xn2 + Yn2 * Yn2) <= r2_lim
        if color_oval is not None:
            ocx, ocy, oRx, oRy, o_r = color_oval
            ok = ok & inside_ellipse(px, py, ocx, ocy, oRx, oRy, o_r)
```

```
        return ok

    gap = 0.9 * np.pi
    bad = np.zeros(n_pts - 1, dtype=bool)
    for i in range(n_pts - 1):
        if abs(lon_a[i + 1] - lon_a[i]) > gap:
            bad[i] = True
        elif abs(lon_b[i + 1] - lon_b[i]) > gap:
            bad[i] = True
        elif abs(lon_a[i] - lon_b[i]) > gap:
            bad[i] = True
        elif abs(lon_a[i + 1] - lon_b[i + 1]) > gap:
            bad[i] = True
        elif np.hypot(px_a[i + 1] - px_a[i], py_a[i + 1] - py_a[i]) > jump_px:
            bad[i] = True
        elif np.hypot(px_b[i + 1] - px_b[i], py_b[i + 1] - py_b[i]) > jump_px:
            bad[i] = True
        elif (not _inside(px_a[i], py_a[i])) or (not _inside(px_b[i], py_b[i])):
            bad[i] = True
        elif (not _inside(px_a[i + 1], py_a[i + 1])) or (not _inside(px_b[i + 1], py_b[i + 1])):
            bad[i] = True

    polys = []
    i = 0
    while i < n_pts - 1:
        while i < n_pts - 1 and bad[i]:
            i += 1
        if i >= n_pts - 1:
            break
        j = i
        while j < n_pts - 1 and not bad[j]:
            j += 1
        i0, i1 = i, j
        if i1 - i0 >= 1:
            xs = np.concatenate([px_a[i0 : i1 + 1], px_b[i0 : i1 + 1][::-1]])
            ys = np.concatenate([py_a[i0 : i1 + 1], py_b[i0 : i1 + 1][::-1]])
            polys.append(np.column_stack([xs, ys]))
        i = j + 1

    if polys:
        coll = PolyCollection(
            polys, facecolors=color, edgecolors="none",
            alpha=0.30, zorder=2, linewidths=0,
        )
        ax.add_collection(coll)
        if clip_path is not None:
            coll.set_clip_path(clip_path)
            coll.set_clip_on(True)

    plot_half_ring(
        ax, D, mu_mid, roll_deg, cx, cy, Rx, Ry, color, lw=2.8,
        inset=inset, clip_path=clip_path, color_oval=color_oval, theta_mid=theta_mid,
    )
```

```
def draw_overlay(rgb, reg, seed, source, pose, roll_deg, png_path):
    h, w, _ = rgb.shape
    cx, cy, Rx, Ry = reg["cx"], reg["cy"], reg["Rx"], reg["Ry"]
    scx = float(seed["cx"])
    scy = float(seed["cy"])
    sRx = float(seed["Rx"])
    sRy = float(seed["Ry"])
    D = pose["D"]
    anti_lb = pose["anti_lb"]
    coe_lb = pose["coe_lb"]
    tilt = pose["tilt_b_deg"]

    dpi = 120
    fig, ax = plt.subplots(figsize=(w / dpi, (h / dpi) + 1.35), dpi=dpi)
    ax.imshow(rgb, origin="upper", extent=(0, w, h, 0), interpolation="nearest")
    ax.set_xlim(0, w)
    ax.set_ylim(h, 0)
    ax.set_aspect("equal")
    ax.axis("off")

    clip_oval = Ellipse(
        (cx, cy), width=2.0 * Rx, height=2.0 * Ry,
        facecolor="none", edgecolor="none", transform=ax.transData,
    )
    ax.add_patch(clip_oval)
    STROKE_INSET = 0.97
    COLOR_R = 0.978
    color_oval = (scx, scy, sRx, sRy, COLOR_R)

    def stroke_curve_geom(ell, b, color, lw, ls="-"):
        plot_lb_curve(
            ax, ell, b, cx, cy, Rx, Ry, inset=1.0, clip_path=clip_oval,
            color_oval=color_oval,
            color="white", lw=lw + 1.6, ls=ls, alpha=0.95,
            solid_capstyle="round", zorder=3,
        )
        plot_lb_curve(
            ax, ell, b, cx, cy, Rx, Ry, inset=1.0, clip_path=clip_oval,
            color_oval=color_oval,
            color=color, lw=lw, ls=ls, alpha=1.0,
            solid_capstyle="round", zorder=4,
        )

    def mark_point(ell, b, marker, color, ms):
        px, py = as_xy(*lb_to_px(ell, b, cx, cy, Rx, Ry))
        halo_ms = ms + 1.8
        ms_px = 0.5 * halo_ms * (dpi / 72.0)
        margin = ms_px / min(sRx, sRy) + 0.012
        r_lim = max(0.55, COLOR_R - margin)
        Xn = (px - scx) / sRx
        Yn = (py - scy) / sRy
        r_col = float(np.hypot(Xn, Yn))
        if r_col > 1e-9:
```

```
            if r_col > r_lim:
                scale = r_lim / r_col
                px = scx + (px - scx) * scale
                py = scy + (py - scy) * scale
                Xn = (px - scx) / sRx
                Yn = (py - scy) / sRy
                r_col = float(np.hypot(Xn, Yn))
            mild = min(STROKE_INSET, 0.995)
            px = scx + (px - scx) * mild
            py = scy + (py - scy) * mild
            Xn = (px - scx) / sRx
            Yn = (py - scy) / sRy
            r_col = float(np.hypot(Xn, Yn))
        if r_col + margin > COLOR_R + 1e-6:
            return

        def _mark(m_size, m_color, m_edge, m_ew, z):
            (line,) = ax.plot(
                px, py, marker=marker, ms=m_size, color=m_color,
                markeredgecolor=m_edge, markeredgewidth=m_ew, linestyle="None",
                zorder=z, clip_path=clip_oval, clip_on=True,
            )
            line.set_clip_path(clip_oval)
            line.set_clip_on(True)

        _mark(ms + 1.8, "white", "white", 0, 6)
        _mark(ms, color, "k", 0.55, 7)

    _ax_ell, _ax_b = great_circle_through_poles(D, n=721)
    stroke_curve_geom(_ax_ell, _ax_b, C_AXIS, lw=1.15)

    for (lo, hi), mid, color in zip(MAX_BANDS, THETA_MAX, COLORS_MAX):
        fill_half_highlight_band(
            ax, D, lo, hi, mid, roll_deg, cx, cy, Rx, Ry, color,
            inset=STROKE_INSET, clip_path=clip_oval, color_oval=color_oval,
        )
    for (lo, hi), mid, color in zip(MIN_BANDS, THETA_MIN, COLORS_MIN):
        fill_half_highlight_band(
            ax, D, lo, hi, mid, roll_deg, cx, cy, Rx, Ry, color,
            inset=STROKE_INSET, clip_path=clip_oval, color_oval=color_oval,
        )

    mark_point(anti_lb[0], anti_lb[1], "o", C_ANTI, ms=5.5)
    mark_point(coe_lb[0], coe_lb[1], "*", C_COE, ms=7.5)
    for (ell, b), color in zip(MIGKAS_MAXES, COLORS_MAX):
        mark_point(ell, b, "^", color, ms=6.5)
    for (ell, b), color in zip(MIGKAS_MINS, COLORS_MIN):
        mark_point(ell, b, "s", color, ms=6.0)

    roll_lab = (
        rf"; $+{roll_deg:.0f}^\circ$ CCW about CoE"
        if abs(roll_deg) > 0.5
        else ""
    )
```

```
    handles = [
        Patch(
            facecolor=C_MAX_30, alpha=0.55, edgecolor=C_MAX_30,
            label=rf"MAX $\theta\!=\!{THETA_MAX[0]:.0f}^\circ$ $\theta$-ray half "
            rf"[{MAX_BANDS[0][0]:.0f},{MAX_BANDS[0][1]:.0f}]$^\circ$",
        ),
        Patch(
            facecolor=C_MAX_185, alpha=0.55, edgecolor=C_MAX_185,
            label=rf"MAX $\theta\!=\!{THETA_MAX[1]:.0f}^\circ$ $\theta$-ray half "
            rf"[{MAX_BANDS[1][0]:.0f},{MAX_BANDS[1][1]:.0f}]$^\circ$",
        ),
        Patch(
            facecolor=C_MIN_115, alpha=0.55, edgecolor=C_MIN_115,
            label=rf"MIN $\theta\!=\!{THETA_MIN[0]:.0f}^\circ$ $\theta$-ray half "
            rf"[{MIN_BANDS[0][0]:.0f},{MIN_BANDS[0][1]:.0f}]$^\circ$",
        ),
        Patch(
            facecolor=C_MIN_265, alpha=0.55, edgecolor=C_MIN_265,
            label=rf"MIN $\theta\!=\!{THETA_MIN[1]:.0f}^\circ$ $\theta$-ray half "
            rf"[{MIN_BANDS[1][0]:.0f},{MIN_BANDS[1][1]:.0f}]$^\circ$",
        ),
        Line2D(
            [0], [0], color=C_AXIS, lw=2.0,
            label=rf"CoE axis tilted $+{tilt:.0f}^\circ$",
        ),
        Line2D(
            [0], [0], marker="^", color="w", markerfacecolor=C_MAX_30,
            markeredgecolor="k", linestyle="None", ms=8,
            label=rf"MAX $({MIGKAS_MAXES[0][0]:.0f}^\circ,{MIGKAS_MAXES[0][1]:+.0f}^\circ)$",
        ),
        Line2D(
            [0], [0], marker="^", color="w", markerfacecolor=C_MAX_185,
            markeredgecolor="k", linestyle="None", ms=8,
            label=rf"MAX $({MIGKAS_MAXES[1][0]:.0f}^\circ,{MIGKAS_MAXES[1][1]:+.0f}^\circ)$",
        ),
        Line2D(
            [0], [0], marker="s", color="w", markerfacecolor=C_MIN_115,
            markeredgecolor="k", linestyle="None", ms=7,
            label=rf"MIN $({MIGKAS_MINS[0][0]:.0f}^\circ,{MIGKAS_MINS[0][1]:+.0f}^\circ)$",
        ),
        Line2D(
            [0], [0], marker="s", color="w", markerfacecolor=C_MIN_265,
            markeredgecolor="k", linestyle="None", ms=7,
            label=rf"MIN $({MIGKAS_MINS[1][0]:.0f}^\circ,{MIGKAS_MINS[1][1]:+.0f}^\circ)$",
        ),
        Line2D(
            [0], [0], marker="o", color=C_ANTI, markeredgecolor="k",
            linestyle="None", ms=6,
            label=rf"anti-CoE $+x$ $(0^\circ,+{tilt:.0f}^\circ)$",
        ),
        Line2D(
            [0], [0], marker="*", color=C_COE, markeredgecolor="k",
            linestyle="None", ms=9,
            label=rf"CoE $(180^\circ,-{tilt:.0f}^\circ)$",
```

```
        ),
    ]
    leg = ax.legend(
        handles=handles, loc="upper center", bbox_to_anchor=(0.5, -0.012),
        ncol=4, fontsize=7.5, frameon=True, fancybox=False,
        edgecolor="#222222", framealpha=1.0, facecolor="white",
        labelcolor="#111111", borderpad=0.6, handlelength=2.0,
    )
    leg.get_frame().set_linewidth(1.2)
    fig.suptitle(
        r"Migkas Fig. 8 $\theta=75^\circ$ $A/A_{\rm all}$"
        r" + $\theta$-ray-side CoE half-loops (each $18^\circ$ end-to-end = $\pm9^\circ$)"
        "\n"
        rf"(anti-CoE at $(0^\circ,+{tilt:.0f}^\circ)$; CoE axis tilted $+{tilt:.0f}^\circ$; "
        rf"$\theta$-ray-side half-loops{roll_lab}; $\ell$ increases LEFT)",
        fontsize=10.5, y=0.985,
    )
    fig.subplots_adjust(left=0.01, right=0.99, top=0.92, bottom=0.13)
    fig.savefig(png_path, dpi=dpi, facecolor="white")
    plt.close(fig)
    print(f"wrote {png_path}")


def main(tilt_b=None, roll_deg=None):
    tilt_b = float(AXIS_TILT_B_DEG if tilt_b is None else tilt_b)
    roll_deg = float(ROLL_ABOUT_COE_DEG if roll_deg is None else roll_deg)
    assert HALF_LOOPS

    pose = axis_pose(tilt_b)
    stem = output_stem(tilt_b, roll_deg)
    png_path = OUT_DIR / f"{stem}.png"
    json_path = OUT_DIR / f"{stem}.json"

    source = choose_source()
    print(f"source: {source}")
    rgb = np.asarray(Image.open(source).convert("RGB"))
    seed = detect_oval_bbox(rgb)
    reg = dict(GOOD_REG)
    h, w, _ = rgb.shape
    dark = rgb.mean(axis=2) < 55
    dist = chamfer_distance_to_dark(dark)
    ell_eq, b_eq = celestial_equator_lb(2500)
    lon = np.deg2rad(lon_disp_from_ell(ell_eq))
    lat = np.deg2rad(b_eq)
    mx, my = mollweide_xy(lon, lat)
    Xn = mx / (2.0 * np.sqrt(2.0))
    Yn = my / np.sqrt(2.0)
    px = reg["cx"] + Xn * reg["Rx"]
    py = reg["cy"] - Yn * reg["Ry"]
    m = (px >= 1) & (px < w - 1) & (py >= 1) & (py < h - 1)
    xi = np.clip(np.round(px[m]).astype(int), 0, w - 1)
    yi = np.clip(np.round(py[m]).astype(int), 0, h - 1)
    reg["ceq_median_px"] = float(np.median(dist[yi, xi]))
    reg["registration_source"] = "hard-set GOOD_REG (pre medial-axis retune)"
```

```
    checks = validate_landmarks(rgb, reg)
    print("validation OK")

    draw_overlay(rgb, reg, seed, source, pose, roll_deg, png_path)

    meta = {
        "png": str(png_path),
        "script": str(Path(__file__).resolve()),
        "geometry_module": str(OUT_DIR / "coe_half_loop_geometry.py"),
        "source_image": str(source),
        "HALF_LOOPS": True,
        "half_loop_rule": (
            "theta-ray side: keep (n.Y)*sin(theta_mid)>=0; "
            "same 3D cut via ring_basis Z_hat"
        ),
        "axis_tilt_b_deg": tilt_b,
        "roll_about_coe_deg": roll_deg,
        "anti_CoE_lb_deg": [float(pose["anti_lb"][0]), float(pose["anti_lb"][1])],
        "CoE_lb_deg": [float(pose["coe_lb"][0]), float(pose["coe_lb"][1])],
        "D_hat_galactic_xyz": pose["D"].tolist(),
        "migkas_max_lb_deg": [list(p) for p in MIGKAS_MAXES],
        "migkas_min_lb_deg": [list(p) for p in MIGKAS_MINS],
        "theta_max_from_plus_x_deg": list(THETA_MAX),
        "theta_min_from_plus_x_deg": list(THETA_MIN),
        "band_half_width_deg": BAND_HALF,
        "band_full_width_deg": BAND_FULL,
        "max_bands_deg": [list(b) for b in MAX_BANDS],
        "min_bands_deg": [list(b) for b in MIN_BANDS],
        "colors": {
            "MAX_30": C_MAX_30,
            "MAX_185": C_MAX_185,
            "MIN_115": C_MIN_115,
            "MIN_265": C_MIN_265,
        },
        "convention": (
            "Galactic Mollweide; lon_disp=-wrap(ell); ell increases LEFT "
            "(Migkas Fig.8)"
        ),
        "registration": {
            "cx": reg["cx"], "cy": reg["cy"], "Rx": reg["Rx"], "Ry": reg["Ry"],
            "ceq_median_px": reg["ceq_median_px"],
            "registration_source": reg.get("registration_source"),
        },
        "landmark_checks": checks,
        "matches_three_panel": str(OUT_DIR / "make_coe_rings_half_loops.py"),
        "drawing": (
            "OUTER half-loops only (open 180° arcs). Each = transparent "
            "PolyCollection between ±9° outer-half edges + one solid mid "
            "half-stroke. No full 360° loops. Dec=0 NOT overlaid."
        ),
    }
    json_path.write_text(json.dumps(meta, indent=2) + "\n")
    print(f"wrote {json_path}")
    return png_path, json_path
```

```
if __name__ == "__main__":
    p = argparse.ArgumentParser(description=__doc__)
    p.add_argument("--tilt", type=float, default=AXIS_TILT_B_DEG)
    p.add_argument("--roll", type=float, default=ROLL_ABOUT_COE_DEG)
    args = p.parse_args()
    main(tilt_b=args.tilt, roll_deg=args.roll)
```

```
source: /workspace/cosmology/map_compare/FULL_migkas_theta75.png
validation OK
wrote /workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/migkas_fig8_half_loops_tilt15.png
wrote /workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/migkas_fig8_half_loops_tilt15.json
```

```
%%runscript make_migkas_fig8_half_loops_overlay.py_roll45 --argv --roll 45
# second run of the same script with the +45 deg CCW roll: load its definitions (everything above its __main__ block), then call main()
exec(open('/workspace/cosmology/figures/make_migkas_fig8_half_loops_overlay.py').read().split('if __name__ == "__main__":')[0])
main(tilt_b=15.0, roll_deg=45.0)
```

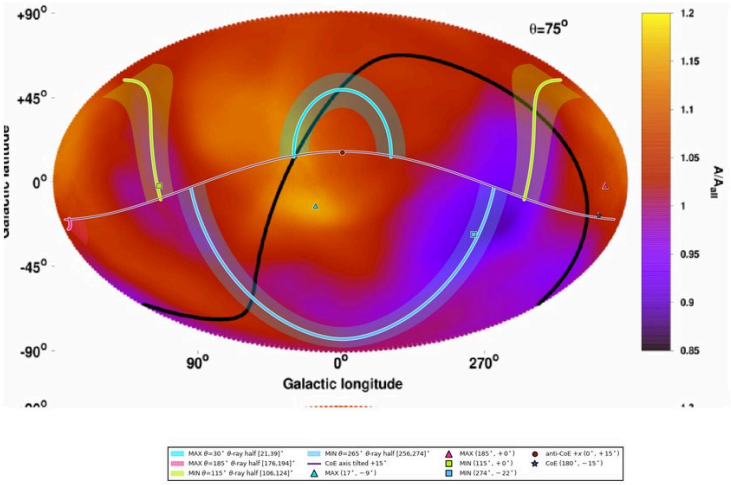
```
source: /workspace/cosmology/map_compare/FULL_migkas_theta75.png
validation OK
wrote /workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/migkas_fig8_half_loops_tilt15_rot45ccw.png
wrote /workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/migkas_fig8_half_loops_tilt15_rot45ccw.json
```

```
compare(FIGS / 'migkas_fig8_half_loops_tilt15.png', '/workspace/cosmology/figures/migkas_fig8_half_loops_tilt15.png')
compare(FIGS / 'migkas_fig8_half_loops_tilt15_rot45ccw.png', '/workspace/cosmology/figures/migkas_fig8_half_loops_tilt15_rot45ccw.png')
```

```
migkas_fig8_half_loops_tilt15.png: new (1975, 1515) vs ref (1975, 1515) | mean |diff| = 0.000/255, pixels differing >40: 0.000% -> identical
left: approved reference figures/migkas_fig8_half_loops_tilt15.png   |   right: notebook re-run steps_v3/figs/migkas_fig8_half_loops_tilt15.png
```

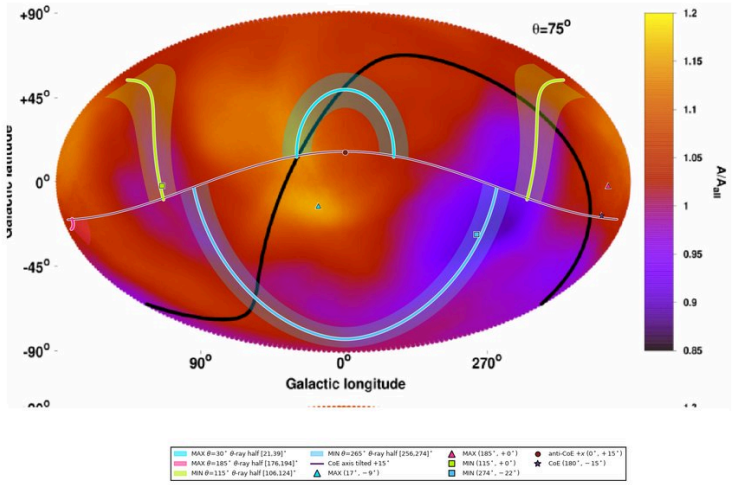
Migkas Fig. 8 $\theta = 75^\circ$ $A/A_{\rm all}$ + θ-ray-side CoE half-loops (each 18° end-to-end = ±9°)
(anti-CoE at (0°, +15°); CoE axis tilted +15°; θ-ray-side half-loops; ℓ increases LEFT)

Migkas Fig. 8 — $\theta = 75^\circ$ $A/A_{\rm all}$ (their numbered colorbar)

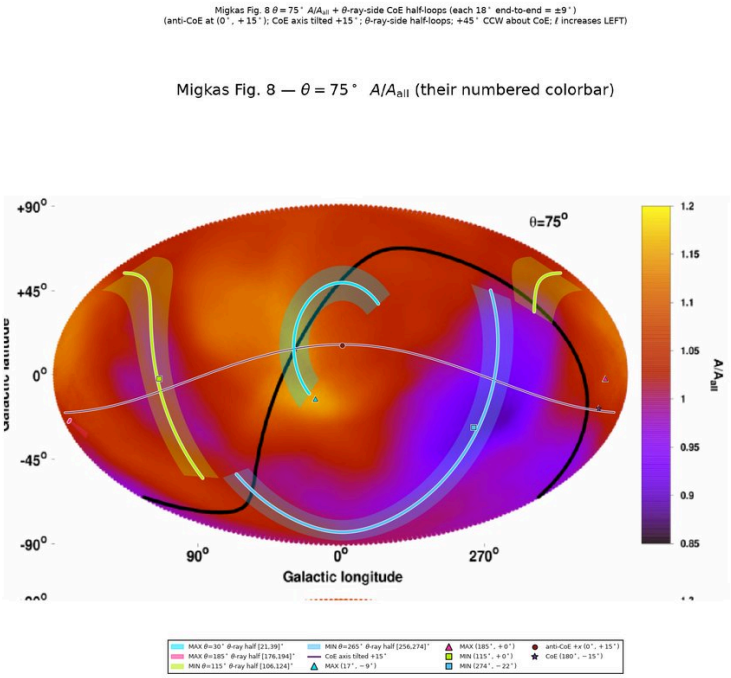


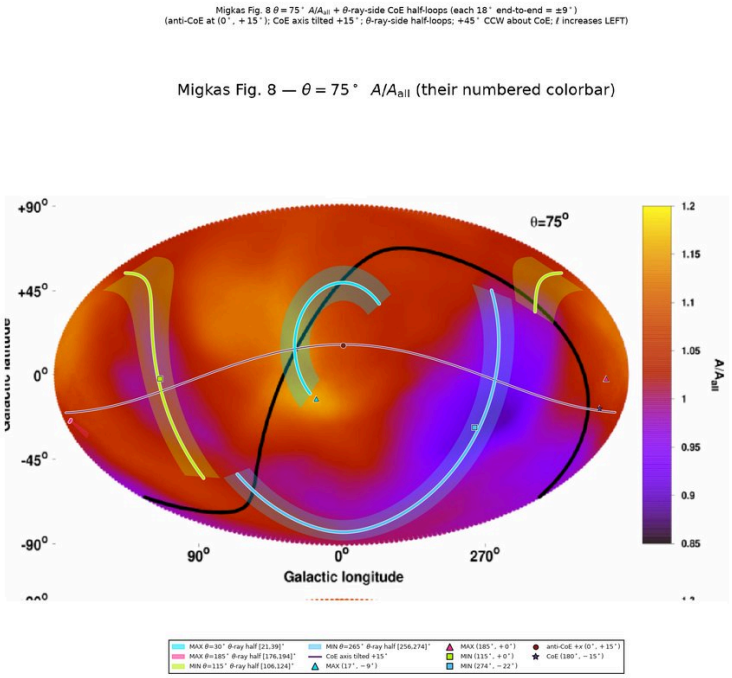
Migkas Fig. 8 $\theta = 75^\circ$ $A/A_{\rm all}$ + θ-ray-side CoE half-loops (each 18° end-to-end = ±9°)
(anti-CoE at (0°, +15°); CoE axis tilted +15°; θ-ray-side half-loops; ℓ increases LEFT)

Migkas Fig. 8 — $\theta = 75^\circ$ $A/A_{\rm all}$ (their numbered colorbar)



migkas_fig8_half_loops_tilt15_rot45ccw.png: new (1975, 1515) vs ref (1975, 1515) | mean |diff| = 0.000/255, pixels differing >40: 0.000% -> identical
left: approved reference figures/migkas_fig8_half_loops_tilt15_rot45ccw.png | right: notebook re-run steps_v3/figs/migkas_fig8_half_loops_tilt15_rot45ccw.png





Step 14 — Full loops with an opacity gradient (roll 0 and $+45^\circ$ CCW)

What it shows. Full loops instead of half-loops. Each loop is drawn as short segments whose opacity fades with ring azimuth ψ from the θ-ray point: darkest there, fully transparent opposite. Panel A is always drawn at roll 0; the roll applies to Panels B and C only. **Maths.** $\alpha(\psi) = \alpha_0 \, \frac{1}{2}(1 + \cos\psi)$, where ψ is measured from the ring point furthest along $\mathrm{sign}(\sin\theta)\,\hat{Y}$ ($\hat{Y}$ rolled).

Source script: `/workspace/cosmology/figures/make_coe_rings_gradient_loops.py` (last modified 2026-09-25 20:22 CT), run verbatim. *Output-path edit:* none. The script's `OUT_DIR` follows `__file__`, which is set to `steps_v3/figs/`. *Reference image:* `figures/coe_rings_gradient_loops_tilt15.png`, `figures/coe_rings_gradient_loops_tilt15_rot45ccw.png`.

```
%%runscript make_coe_rings_gradient_loops.py --argv --roll 0
#!/usr/bin/env python3
"""
CoE rings — FULL loops with opacity gradient (Chris 2026-09-25).

Derived from make_coe_rings_half_loops.py (left unchanged). Instead of erasing
the opposite half of each ring, the full loop (±9° band fill + solid mid line)
is drawn as many short elements (LineCollection segments / PolyCollection quads)
with per-element alpha = base_alpha * (1 + cos psi)/2, psi = ring azimuth from
```

```
the θ-ray point (ring point most along sign(sinθ)·Ŷ; Ŷ rolled for B/C only).
Darkest at the θ-ray point, fully transparent at the diametrically opposite point.

Panel A is ALWAYS drawn with roll 0. Roll applies to Panels B and C only.

Outputs:
  roll=0  -> coe_rings_gradient_loops_tilt15.{png,pdf,json}
  roll=45 -> coe_rings_gradient_loops_tilt15_rot45ccw.{png,pdf,json}
"""

from __future__ import annotations

import argparse
import json
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
from matplotlib.lines import Line2D
from matplotlib.collections import LineCollection, PolyCollection
from matplotlib.patches import Circle, Patch, Polygon
from mpl_toolkits.mplot3d import Axes3D  # noqa: F401
from mpl_toolkits.mplot3d.art3d import Line3DCollection, Poly3DCollection

from coe_half_loop_geometry import (
    BAND_FULL,
    BAND_HALF,
    HALF_LOOP_RULE,
    MIGKAS_MAXES,
    MIGKAS_MINS,
    STRIP_THETA_MAX,
    STRIP_THETA_MIN,
    axis_pose,
    band_edges,
    celestial_equator_lb,
    great_circle_through_poles,
    lb_to_n,
    lon_disp_from_ell,
    mu_pair_for_band,
    n_to_lb,
)

from coe_gradient_loop_geometry import (
    GRADIENT_RULE,
    foreshortened_full_ellipse_gradient,
    rgba_array,
    quantized_runs,
    segment_scale,
    small_circle_full_gradient,
)

OUT_DIR = Path(__file__).resolve().parent

# Defaults (overridden by CLI)
```

```
AXIS_TILT_B_DEG = 15.0
ROLL_ABOUT_COE_DEG = 0.0
FULL_GRADIENT_LOOPS = True
GRAD_TXT = "full loops, opacity gradient darkest at θ ray, fading to 12% opacity opposite"

C_COE = "#6b3fa0"
C_ANTI = "#8e1b1b"
C_MAX_30 = "#c0392b"
C_MAX_185 = "#ec7063"
C_MIN_115 = "#1a5276"
C_MIN_265 = "#5dade2"
C_AXIS = "#5b2c6f"
C_EQ = "#1e8449"
COLORS_MAX = [C_MAX_30, C_MAX_185]
COLORS_MIN = [C_MIN_115, C_MIN_265]


def output_stem(tilt_b: float, roll: float) -> str:
    base = f"coe_rings_gradient_loops_tilt{int(round(tilt_b))}"
    if abs(roll) > 0.5:
        base += f"_rot{int(round(roll))}ccw"
    return base


def split_lon_segments(lon_rad, lat_rad, gap=np.pi):
    d = np.diff(lon_rad)
    cuts = np.where(np.abs(d) > gap)[0] + 1
    idx = np.concatenate([[0], cuts, [len(lon_rad)]])
    segs = []
    for i0, i1 in zip(idx[:-1], idx[1:]):
        if i1 - i0 >= 2:
            segs.append((lon_rad[i0:i1], lat_rad[i0:i1]))
    return segs


def plot_lb_curve_mollweide(ax, l_deg, b_deg, **kwargs):
    lon = np.deg2rad(lon_disp_from_ell(l_deg))
    lat = np.deg2rad(b_deg)
    for xr, yr in split_lon_segments(lon, lat):
        ax.plot(xr, yr, **kwargs)




def _full_ring_lonlat_rad(D, mu, theta_mid, roll_deg, n_pts=721):
    pts, _, scale = small_circle_full_gradient(
        D, float(mu), theta_mid, roll_deg=roll_deg, n=n_pts
    )
    ls = np.array([n_to_lb(p)[0] for p in pts])
    bs = np.array([n_to_lb(p)[1] for p in pts])
    lon = np.deg2rad(lon_disp_from_ell(ls))
    lat = np.deg2rad(bs)
    return lon, lat, scale
```

```
FILL_ALPHA_B = 0.26
LINE_ALPHA = 0.95


def _strip_polys_2d(xa, ya, xb, yb, runs):
    return [
        np.column_stack([
            np.concatenate([xa[i0:i1 + 1], xb[i0:i1 + 1][::-1]]),
            np.concatenate([ya[i0:i1 + 1], yb[i0:i1 + 1][::-1]]),
        ])
        for i0, i1, _ in runs
    ]


def _polylines_2d(x, y, runs):
    return [np.column_stack([x[i0:i1 + 1], y[i0:i1 + 1]]) for i0, i1, _ in runs]


def fill_mollweide_gradient_ribbon(ax, D, theta_lo, theta_hi, theta_mid, color, roll_deg, n_pts=721):
    """Full Mollweide ribbon between ±9° edges + solid mid loop, opacity gradient.
    Elements crossing the antimeridian seam are dropped (never drawn)."""
    mu_a, mu_b = mu_pair_for_band(theta_lo, theta_hi)
    mu_mid = float(np.cos(np.deg2rad(theta_mid)))
    lon_a, lat_a, sc = _full_ring_lonlat_rad(D, mu_a, theta_mid, roll_deg, n_pts=n_pts)
    lon_b, lat_b, _ = _full_ring_lonlat_rad(D, mu_b, theta_mid, roll_deg, n_pts=n_pts)
    gap = 0.9 * np.pi
    quad_lons = np.stack([lon_a[:-1], lon_a[1:], lon_b[1:], lon_b[:-1]])
    valid = (quad_lons.max(axis=0) - quad_lons.min(axis=0)) <= gap
    runs = quantized_runs(segment_scale(sc), valid)
    if runs:
        ax.add_collection(
            PolyCollection(
                _strip_polys_2d(lon_a, lat_a, lon_b, lat_b, runs),
                facecolors=rgba_array(color, [FILL_ALPHA_B * r[2] for r in runs]),
                edgecolors="none", zorder=2, linewidths=0,
            )
        )
    n_line = max(n_pts, 721)
    lon_m, lat_m, sc_m = _full_ring_lonlat_rad(D, mu_mid, theta_mid, roll_deg, n_pts=n_line)
    valid = np.abs(np.diff(lon_m)) <= gap
    runs = quantized_runs(segment_scale(sc_m), valid)
    if runs:
        ax.add_collection(
            LineCollection(
                _polylines_2d(lon_m, lat_m, runs),
                colors=rgba_array(color, [LINE_ALPHA * r[2] for r in runs]),
                linewidths=2.6, zorder=4, capstyle="butt", joinstyle="round",
            )
        )


FILL_ALPHA_A = 0.28
```

```
def draw_panelA_gradient_ribbon(ax, theta_lo, theta_hi, theta_mid, color, n=721):
    """Panel A full foreshortened band + mid loop with opacity gradient (roll 0)."""
    mu_lo = float(np.cos(np.deg2rad(theta_lo)))
    mu_hi = float(np.cos(np.deg2rad(theta_hi)))
    xa, ya, sc = foreshortened_full_ellipse_gradient(mu_lo, theta_mid, n=n)
    xb, yb, _ = foreshortened_full_ellipse_gradient(mu_hi, theta_mid, n=n)
    runs = quantized_runs(segment_scale(sc))
    coll = PolyCollection(
        _strip_polys_2d(xa, ya, xb, yb, runs),
        facecolors=rgba_array(color, [FILL_ALPHA_A * r[2] for r in runs]),
        edgecolors="none", linewidths=0, zorder=2,
    )
    ax.add_collection(coll)
    coll.set_clip_path(Circle((0.0, 0.0), 1.0, transform=ax.transData))

    mu_mid = float(np.cos(np.deg2rad(theta_mid)))
    xm, ym, scm = foreshortened_full_ellipse_gradient(mu_mid, theta_mid, n=n)
    runs = quantized_runs(segment_scale(scm))
    ax.add_collection(
        LineCollection(
            _polylines_2d(xm, ym, runs),
            colors=rgba_array(color, [LINE_ALPHA * r[2] for r in runs]),
            linewidths=2.8, zorder=4, capstyle="butt", joinstyle="round",
        )
    )


def draw_panelA_theta_rays(ax, geo):
    items = list(zip(geo["theta_max_from_plus_x_deg"], COLORS_MAX)) + list(
        zip(geo["theta_min_from_plus_x_deg"], COLORS_MIN)
    )
    for mid, color in items:
        t = float(np.deg2rad(mid))
        x1, y1 = float(np.cos(t)), float(np.sin(t))
        is_max30 = abs(mid - 30.0) < 0.5
        lw = 3.2 if is_max30 else 1.8
        alpha = 0.70 if is_max30 else 0.45
        ax.plot(
            [0.0, x1], [0.0, y1],
            color=color, lw=lw, alpha=alpha, zorder=3, solid_capstyle="round",
        )
        ax.plot(
            [x1], [y1], "o", color=color, ms=4.5 if is_max30 else 3.5,
            alpha=0.9, zorder=5, markeredgecolor="none",
        )
    arc_r = 0.38
    phi = np.linspace(0.0, np.deg2rad(30.0), 60)
    ax.plot(
        arc_r * np.cos(phi), arc_r * np.sin(phi),
        color=C_MAX_30, lw=1.6, alpha=0.85, zorder=3,
    )
    ax.text(
```

```
        arc_r * np.cos(np.deg2rad(15.0)) + 0.06,
        arc_r * np.sin(np.deg2rad(15.0)) + 0.04,
        r"$\theta$",
        color=C_MAX_30, fontsize=10, fontweight="bold",
        ha="left", va="bottom", zorder=6,
    )


def panel_A_model_frame(ax, geo, roll_deg):
    roll_note = (
        rf"; $+{roll_deg:.0f}^\circ$ CCW roll about CoE"
        if abs(roll_deg) > 0.5
        else ""
    )
    ax.set_title(
        r"A. Model frame — full loops, opacity gradient"
        "\n"
        r"axisymmetric about $x$ = CoE$\leftrightarrow$anti-CoE"
        f"{roll_note}\n"
        r"darkest at $\theta$ ray, fading to 12% opacity opposite; "
        r"each $18^\circ$ end-to-end ($\pm9^\circ$)",
        fontsize=9.5,
        pad=8,
    )
    th = np.linspace(0, 2 * np.pi, 400)
    ax.plot(np.cos(th), np.sin(th), color="#7f8c8d", lw=1.2, zorder=1)
    ax.plot(0, 0, "ko", ms=6, zorder=5)
    ax.text(0.08, -0.12, "observer", fontsize=8, color="k", ha="left")
    ax.annotate(
        "",
        xy=(1.15, 0),
        xytext=(-1.15, 0),
        arrowprops=dict(arrowstyle="<->", color=C_AXIS, lw=2.5),
        zorder=4,
    )
    ax.text(1.18, 0.08, "anti-CoE\n(+x)", color=C_ANTI, fontsize=8, ha="left", va="bottom")
    ax.text(-1.18, 0.08, "CoE\n(−x)", color=C_COE, fontsize=8, ha="right", va="bottom")

    for (lo, hi), mid, color in zip(
        geo["max_bands_deg"], geo["theta_max_from_plus_x_deg"], COLORS_MAX
    ):
        draw_panelA_gradient_ribbon(ax, lo, hi, mid, color)
    for (lo, hi), mid, color in zip(
        geo["min_bands_deg"], geo["theta_min_from_plus_x_deg"], COLORS_MIN
    ):
        draw_panelA_gradient_ribbon(ax, lo, hi, mid, color)

    draw_panelA_theta_rays(ax, geo)

    label_r = 1.30
    for mid, color in list(zip(geo["theta_max_from_plus_x_deg"], COLORS_MAX)) + list(
        zip(geo["theta_min_from_plus_x_deg"], COLORS_MIN)
    ):
        t = np.deg2rad(mid)
```

```
        ax.text(
            label_r * np.cos(t),
            label_r * np.sin(t),
            rf"$\theta\!=\!{mid:.0f}^\circ$",
            color=color, fontsize=7.5, ha="center", va="center", fontweight="bold",
        )

    ax.text(
        0.0,
        -1.38,
        (
            r"full loops, opacity gradient darkest at $\theta$ ray, fading to 12% opacity opposite"
            + "\n"
            + r"$\alpha\propto1-0.88|\psi|/\pi$, $\psi$ = ring azimuth from $\theta$-ray point  ·  "
            r"each $18^\circ$ end-to-end ($\pm9^\circ$)"
            + "\n"
            + r"(after-2$\ell$ $A(\ell)$: two MAX + two MIN)"
        ),
        fontsize=7.5,
        ha="center",
        va="top",
        style="italic",
        color="#34495e",
    )
    ax.set_xlim(-1.55, 1.55)
    ax.set_ylim(-1.55, 1.45)
    ax.set_aspect("equal")
    ax.axis("off")


def panel_B_mollweide(ax, geo, roll_deg):
    tilt = geo["tilt_b_deg"]
    # Always 4 title lines so the figure layout (and hence Panel A) is identical
    # between roll=0 and roll!=0 outputs.
    roll_note = "\n" + (
        rf"$+{roll_deg:.0f}^\circ$ CCW about CoE (B/C only)"
        if abs(roll_deg) > 0.5
        else r"no roll about CoE ($0^\circ$)"
    )
    ax.set_title(
        rf"B. Migkas $A(\ell)$ loops on sky (anti-CoE at $(0^\circ,+{tilt:.0f}^\circ)$)"
        "\n"
        rf"(Galactic Mollweide; CoE axis tilted $+{tilt:.0f}^\circ$; $\ell$ increases LEFT)"
        "\n"
        r"full loops, opacity gradient darkest at $\theta$ ray, fading to 12% opacity opposite"
        f"{roll_note}",
        fontsize=9.5,
        pad=8,
    )
    for b in [-60, -30, 0, 30, 60]:
        ell = np.linspace(0, 360, 361)
        plot_lb_curve_mollweide(ax, ell, np.full_like(ell, b), color="#bdc3c7", lw=0.5, zorder=0)
    for ell0 in range(0, 360, 30):
        b = np.linspace(-90, 90, 181)
```

```
    plot_lb_curve_mollweide(ax, np.full_like(b, ell0), b, color="#bdc3c7", lw=0.5, zorder=0)

D = geo["D"]
ls, bs = great_circle_through_poles(D, n=721)
plot_lb_curve_mollweide(ax, ls, bs, color=C_AXIS, lw=2.0, zorder=2)

for (lo, hi), mid, color in zip(
    geo["max_bands_deg"], geo["theta_max_from_plus_x_deg"], COLORS_MAX
):
    fill_mollweide_gradient_ribbon(ax, D, lo, hi, mid, color, roll_deg)
for (lo, hi), mid, color in zip(
    geo["min_bands_deg"], geo["theta_min_from_plus_x_deg"], COLORS_MIN
):
    fill_mollweide_gradient_ribbon(ax, D, lo, hi, mid, color, roll_deg)

ell_eq, b_eq = celestial_equator_lb(721)
plot_lb_curve_mollweide(ax, ell_eq, b_eq, color=C_EQ, lw=1.8, ls="--", zorder=2)

ax.plot(
    np.deg2rad(lon_disp_from_ell(geo["anti_lb"][0])),
    np.deg2rad(geo["anti_lb"][1]),
    marker="o", ms=8, color=C_ANTI,
    markerfacecolor=C_ANTI, markeredgecolor="k", markeredgewidth=0.6,
    zorder=6, linestyle="None",
)
ax.plot(
    np.deg2rad(lon_disp_from_ell(geo["coe_lb"][0])),
    np.deg2rad(geo["coe_lb"][1]),
    marker="*", ms=14, color=C_COE,
    markerfacecolor=C_COE, markeredgecolor="k", markeredgewidth=0.6,
    zorder=6, linestyle="None",
)
for (l, b), color in zip(MIGKAS_MAXES, COLORS_MAX):
    ax.plot(
        np.deg2rad(lon_disp_from_ell(l)), np.deg2rad(b),
        marker="^", ms=10, color=color,
        markeredgecolor="k", markeredgewidth=0.7, zorder=7, linestyle="None",
    )
for (l, b), color in zip(MIGKAS_MINS, COLORS_MIN):
    ax.plot(
        np.deg2rad(lon_disp_from_ell(l)), np.deg2rad(b),
        marker="s", ms=10, color=color,
        markeredgecolor="k", markeredgewidth=0.7, zorder=7, linestyle="None",
    )

tick_ell = np.array([135, 90, 45, 0, 315, 270, 225])
ax.set_xticks(np.deg2rad(lon_disp_from_ell(tick_ell)))
ax.set_xticklabels([rf"${e:d}^\circ$" for e in tick_ell], fontsize=7)
ax.set_yticks(np.deg2rad([-60, -30, 0, 30, 60]))
ax.set_yticklabels(
    [r"$-60^\circ$", r"$-30^\circ$", r"$0^\circ$", r"$+30^\circ$", r"$+60^\circ$"], fontsize=8
)
ax.grid(False)
ax.set_xlabel(r"Galactic $\ell$ (increases LEFT)", fontsize=8)
```

```
    ax.set_ylabel(r"Galactic $b$", fontsize=8)


FILL_ALPHA_C = 0.25


def fill_sphere_gradient_ribbon(ax, D, theta_lo, theta_hi, theta_mid, color, roll_deg, n_pts=361):
    mu_a, mu_b = mu_pair_for_band(theta_lo, theta_hi)
    mu_mid = float(np.cos(np.deg2rad(theta_mid)))
    p0, _, sc = small_circle_full_gradient(D, float(mu_a), theta_mid, roll_deg=roll_deg, n=n_pts)
    p1, _, _ = small_circle_full_gradient(D, float(mu_b), theta_mid, roll_deg=roll_deg, n=n_pts)
    seg_sc = segment_scale(sc)
    idx = np.where(seg_sc > 1e-4)[0]
    verts = [[p0[j], p0[j + 1], p1[j + 1], p1[j]] for j in idx]
    coll = Poly3DCollection(
        verts, facecolors=rgba_array(color, FILL_ALPHA_C * seg_sc[idx]),
        edgecolors="none", linewidths=0.0, zorder=2,
    )
    ax.add_collection3d(coll)
    pts, _, scm = small_circle_full_gradient(D, mu_mid, theta_mid, roll_deg=roll_deg, n=n_pts)
    runs = quantized_runs(segment_scale(scm))
    ax.add_collection3d(
        Line3DCollection(
            [pts[i0:i1 + 1] for i0, i1, _ in runs],
            colors=rgba_array(color, [LINE_ALPHA * r[2] for r in runs]),
            linewidths=2.6, zorder=5, capstyle="butt", joinstyle="round",
        )
    )


def draw_wire_sphere(ax, n_mer=12, n_par=6, **kwargs):
    u = np.linspace(0, 2 * np.pi, 60)
    v = np.linspace(-np.pi / 2, np.pi / 2, 40)
    xs = np.outer(np.cos(u), np.cos(v))
    ys = np.outer(np.sin(u), np.cos(v))
    zs = np.outer(np.ones_like(u), np.sin(v))
    ax.plot_surface(xs, ys, zs, color="#dfe6e9", alpha=0.15, linewidth=0, shade=False)
    kw = dict(color="#95a5a6", lw=0.4, alpha=0.45)
    kw.update(kwargs)
    for i in range(n_mer):
        phi = 2 * np.pi * i / n_mer
        t = np.linspace(-np.pi / 2, np.pi / 2, 80)
        ax.plot(np.cos(t) * np.cos(phi), np.cos(t) * np.sin(phi), np.sin(t), **kw)
    for j in range(1, n_par):
        b = -np.pi / 2 + np.pi * j / n_par
        t = np.linspace(0, 2 * np.pi, 80)
        r = np.cos(b)
        ax.plot(r * np.cos(t), r * np.sin(t), np.full_like(t, np.sin(b)), **kw)


def panel_C_sphere(ax, geo, roll_deg):
    th_max = geo["theta_max_from_plus_x_deg"]
    th_min = geo["theta_min_from_plus_x_deg"]
    roll_note = (
```

```
        rf"; $+{roll_deg:.0f}^\circ$ CCW about CoE"
        if abs(roll_deg) > 0.5
        else ""
    )
    ax.set_title(
        rf"C. Sphere 3D — full loops{roll_note}"
        "\n"
        r"(opacity gradient darkest at $\theta$ ray, fading to 12% opacity opposite;"
        "\n"
        r"celestial equator dashed)",
        fontsize=9.5,
        pad=6,
    )
    draw_wire_sphere(ax)
    D = geo["D"]
    ell_eq, b_eq = celestial_equator_lb(400)
    eq_xyz = np.array([lb_to_n(l, b) for l, b in zip(ell_eq, b_eq)])
    ax.plot(eq_xyz[:, 0], eq_xyz[:, 1], eq_xyz[:, 2], color=C_EQ, lw=1.8, ls="--", zorder=3)
    ax.plot(
        [-D[0], D[0]], [-D[1], D[1]], [-D[2], D[2]],
        color=C_AXIS, lw=3.0, zorder=5,
    )
    ax.scatter(*(-D), color=C_COE, s=120, marker="*", edgecolors="k", linewidths=0.6,
               depthshade=False, zorder=8)
    ax.scatter(*(D), color=C_ANTI, s=80, edgecolors="k", linewidths=0.6, depthshade=False, zorder=8)
    ax.text(*(-D * 1.22), "CoE", color=C_COE, fontsize=8, fontweight="bold")
    ax.text(*(D * 1.18), "anti-CoE", color=C_ANTI, fontsize=8, fontweight="bold")
    ax.scatter([0], [0], [0], color="k", s=25, depthshade=False, zorder=9)

    for (lo, hi), mid, color in zip(
        geo["max_bands_deg"], geo["theta_max_from_plus_x_deg"], COLORS_MAX
    ):
        fill_sphere_gradient_ribbon(ax, D, lo, hi, mid, color, roll_deg)
    for (lo, hi), mid, color in zip(
        geo["min_bands_deg"], geo["theta_min_from_plus_x_deg"], COLORS_MIN
    ):
        fill_sphere_gradient_ribbon(ax, D, lo, hi, mid, color, roll_deg)

    for n, color in zip(geo["n_maxes"], COLORS_MAX):
        ax.scatter(
            [n[0]], [n[1]], [n[2]],
            c=[color], marker="^", s=70,
            edgecolors="k", linewidths=0.7, depthshade=False, zorder=10,
        )
        ax.plot([0, n[0]], [0, n[1]], [0, n[2]], color=color, lw=0.8, ls=":", alpha=0.7)
    for n, color in zip(geo["n_mins"], COLORS_MIN):
        ax.scatter(
            [n[0]], [n[1]], [n[2]],
            c=[color], marker="s", s=60,
            edgecolors="k", linewidths=0.7, depthshade=False, zorder=10,
        )
        ax.plot([0, n[0]], [0, n[1]], [0, n[2]], color=color, lw=0.8, ls=":", alpha=0.7)

    ax.set_xlim(-1.25, 1.25)
```

```
    ax.set_ylim(-1.25, 1.25)
    ax.set_zlim(-1.25, 1.25)
    ax.set_box_aspect((1, 1, 1))
    ax.view_init(elev=18, azim=-55)
    ax.set_axis_off()
    ax.text2D(
        0.5, -0.02,
        rf"MAX $\theta\!=\!{th_max[0]:.0f}^\circ,{th_max[1]:.0f}^\circ$; "
        rf"MIN $\theta\!=\!{th_min[0]:.0f}^\circ,{th_min[1]:.0f}^\circ$  "
        rf"(full loops, gradient; $18^\circ$ end-to-end = $\pm9^\circ$)",
        transform=ax.transAxes, ha="center", fontsize=7.5, color="#34495e",
    )


def compute_geometry(tilt_b, roll_deg):
    pose = axis_pose(tilt_b)
    th_max = list(STRIP_THETA_MAX)
    th_min = list(STRIP_THETA_MIN)
    max_bands = band_edges(th_max)
    min_bands = band_edges(th_min)
    n_maxes = [lb_to_n(*lb) for lb in MIGKAS_MAXES]
    n_mins = [lb_to_n(*lb) for lb in MIGKAS_MINS]
    sep = float(
        np.rad2deg(np.arccos(np.clip(np.dot(n_maxes[0], n_mins[1]), -1.0, 1.0)))
    )
    return {
        **pose,
        "n_maxes": n_maxes,
        "n_mins": n_mins,
        "theta_max_from_plus_x_deg": th_max,
        "theta_min_from_plus_x_deg": th_min,
        "max_bands_deg": max_bands,
        "min_bands_deg": min_bands,
        "mu_max_bands": [mu_pair_for_band(lo, hi) for lo, hi in max_bands],
        "mu_min_bands": [mu_pair_for_band(lo, hi) for lo, hi in min_bands],
        "sep_deg": sep,
        "band_half_width_deg": BAND_HALF,
        "roll_about_coe_deg": float(roll_deg),
        "full_gradient_loops": True,
        "gradient_rule": GRADIENT_RULE,
    }


def main(tilt_b=None, roll_deg=None):
    tilt_b = float(AXIS_TILT_B_DEG if tilt_b is None else tilt_b)
    roll_deg = float(ROLL_ABOUT_COE_DEG if roll_deg is None else roll_deg)
    assert FULL_GRADIENT_LOOPS

    geo = compute_geometry(tilt_b, roll_deg)
    th_max = geo["theta_max_from_plus_x_deg"]
    th_min = geo["theta_min_from_plus_x_deg"]
    max_bands = geo["max_bands_deg"]
    min_bands = geo["min_bands_deg"]
```

```
    stem = output_stem(tilt_b, roll_deg)
    png_path = OUT_DIR / f"{stem}.png"
    pdf_path = OUT_DIR / f"{stem}.pdf"
    json_path = OUT_DIR / f"{stem}.json"

    roll_sup = (
        rf"; $+{roll_deg:.0f}^\circ$ CCW about CoE axis"
        if abs(roll_deg) > 0.5
        else ""
    )
    fig = plt.figure(figsize=(14.5, 5.6), dpi=160)
    fig.suptitle(
        r"Pure CoE axisymmetry: full loops, opacity gradient darkest at $\theta$ ray, "
        r"fading to 12% opacity opposite ($\alpha\propto1-0.88|\psi|/\pi$)"
        "\n"
        rf"(each $18^\circ$ end-to-end = $\pm9^\circ$; "
        rf"MAX $\theta\!=\!{th_max[0]:.0f}^\circ/{th_max[1]:.0f}^\circ$; "
        rf"MIN $\theta\!=\!{th_min[0]:.0f}^\circ/{th_min[1]:.0f}^\circ$; "
        rf"anti-CoE $(0^\circ,+{tilt_b:.0f}^\circ)$, CoE axis tilted $+{tilt_b:.0f}^\circ$"
        f"{roll_sup})",
        fontsize=11,
        fontweight="bold",
        y=1.02,
    )

    axA = fig.add_subplot(1, 3, 1)
    panel_A_model_frame(axA, geo, 0.0)  # model frame never rolls
    axB = fig.add_subplot(1, 3, 2, projection="mollweide")
    panel_B_mollweide(axB, geo, roll_deg)
    axC = fig.add_subplot(1, 3, 3, projection="3d")
    panel_C_sphere(axC, geo, roll_deg)

    handles = [
        Line2D([0], [0], color=C_AXIS, lw=2.5, label=r"CoE$\leftrightarrow$anti-CoE axis"),
        Patch(
            facecolor=C_MAX_30, alpha=0.45, edgecolor=C_MAX_30,
            label=rf"MAX $\theta\!=\!{th_max[0]:.0f}^\circ$ full loop, gradient "
            rf"[{max_bands[0][0]:.0f},{max_bands[0][1]:.0f}]$^\circ$",
        ),
        Patch(
            facecolor=C_MAX_185, alpha=0.45, edgecolor=C_MAX_185,
            label=rf"MAX $\theta\!=\!{th_max[1]:.0f}^\circ$ full loop, gradient "
            rf"[{max_bands[1][0]:.0f},{max_bands[1][1]:.0f}]$^\circ$",
        ),
        Patch(
            facecolor=C_MIN_115, alpha=0.45, edgecolor=C_MIN_115,
            label=rf"MIN $\theta\!=\!{th_min[0]:.0f}^\circ$ full loop, gradient "
            rf"[{min_bands[0][0]:.0f},{min_bands[0][1]:.0f}]$^\circ$",
        ),
        Patch(
            facecolor=C_MIN_265, alpha=0.45, edgecolor=C_MIN_265,
            label=rf"MIN $\theta\!=\!{th_min[1]:.0f}^\circ$ full loop, gradient "
            rf"[{min_bands[1][0]:.0f},{min_bands[1][1]:.0f}]$^\circ$",
        ),
```

```
        Line2D([0], [0], color=C_EQ, lw=1.8, ls="--", label="celestial equator (Dec$=0$)"),
        Line2D(
            [0], [0], marker="^", color="w", markerfacecolor=C_MAX_30, markeredgecolor="k",
            ms=9, label=rf"MAX $({MIGKAS_MAXES[0][0]:.0f}^\circ,{MIGKAS_MAXES[0][1]:+.0f}^\circ)$",
        ),
        Line2D(
            [0], [0], marker="^", color="w", markerfacecolor=C_MAX_185, markeredgecolor="k",
            ms=9, label=rf"MAX $({MIGKAS_MAXES[1][0]:.0f}^\circ,{MIGKAS_MAXES[1][1]:+.0f}^\circ)$",
        ),
        Line2D(
            [0], [0], marker="s", color="w", markerfacecolor=C_MIN_115, markeredgecolor="k",
            ms=8, label=rf"MIN $({MIGKAS_MINS[0][0]:.0f}^\circ,{MIGKAS_MINS[0][1]:+.0f}^\circ)$",
        ),
        Line2D(
            [0], [0], marker="s", color="w", markerfacecolor=C_MIN_265, markeredgecolor="k",
            ms=8, label=rf"MIN $({MIGKAS_MINS[1][0]:.0f}^\circ,{MIGKAS_MINS[1][1]:+.0f}^\circ)$",
        ),
        Line2D(
            [0], [0], marker="o", color="w", markerfacecolor=C_ANTI, markeredgecolor="k",
            ms=8, label=rf"anti-CoE $({geo['anti_lb'][0]:.0f}^\circ,{geo['anti_lb'][1]:+.0f}^\circ)$",
        ),
        Line2D(
            [0], [0], marker="*", color="w", markerfacecolor=C_COE, markeredgecolor="k",
            ms=11, label=rf"CoE $({geo['coe_lb'][0]:.0f}^\circ,{geo['coe_lb'][1]:+.0f}^\circ)$",
        ),
    ]
    fig.legend(
        handles=handles, loc="lower center", ncol=6, fontsize=7.0,
        frameon=True, fancybox=True, bbox_to_anchor=(0.5, -0.10),
    )
    # Fixed layout (no tight_layout) so Panel A geometry never depends on roll.
    fig.subplots_adjust(left=0.03, right=0.99, bottom=0.10, top=0.80, wspace=0.12)
    fig.savefig(png_path, bbox_inches="tight", facecolor="white")
    fig.savefig(pdf_path, bbox_inches="tight", facecolor="white")
    plt.close(fig)

    payload = {
        "png": str(png_path),
        "pdf": str(pdf_path),
        "script": str(OUT_DIR / "make_coe_rings_gradient_loops.py"),
        "geometry_module": str(OUT_DIR / "coe_half_loop_geometry.py"),
        "gradient_module": str(OUT_DIR / "coe_gradient_loop_geometry.py"),
        "FULL_GRADIENT_LOOPS": True,
        "gradient_rule": GRADIENT_RULE,
        "legacy_half_loop_rule_equiv_alpha_scale_ge_half": HALF_LOOP_RULE,
        "panel_A_roll_deg": 0.0,
        "roll_applies_to_panels": ["B", "C"],
        "base_alphas": {"fill_A": FILL_ALPHA_A, "fill_B": FILL_ALPHA_B,
                        "fill_C": FILL_ALPHA_C, "mid_line": LINE_ALPHA},
        "axis_tilt_b_deg": tilt_b,
        "roll_about_coe_deg": roll_deg,
        "migkas_max_lb_deg": [list(lb) for lb in MIGKAS_MAXES],
        "migkas_min_lb_deg": [list(lb) for lb in MIGKAS_MINS],
        "strip_theta_max_deg": list(STRIP_THETA_MAX),
```

```
        "strip_theta_min_deg": list(STRIP_THETA_MIN),
        "band_half_width_deg": BAND_HALF,
        "band_full_width_deg": BAND_FULL,
        "max_bands_deg": [list(b) for b in max_bands],
        "min_bands_deg": [list(b) for b in min_bands],
        "CoE_lb_deg": [round(geo["coe_lb"][0], 4), round(geo["coe_lb"][1], 4)],
        "anti_CoE_lb_deg": [round(geo["anti_lb"][0], 4), round(geo["anti_lb"][1], 4)],
        "D_hat_galactic_xyz": geo["D"].tolist(),
        "angular_sep_migkas_max0_min1_deg": round(geo["sep_deg"], 6),
        "orientation": (
            f"anti-CoE at (0°, +{tilt_b:.0f}°), CoE at (180°, −{tilt_b:.0f}°); "
            f"full loops, opacity gradient darkest at 0 ray, fading to 12% opacity opposite; "
            f"roll about CoE = {roll_deg:.0f}° CCW (panels B/C only; panel A unrolled)"
        ),
        "plot_convention": "Migkas Galactic Mollweide: lon_disp=-ell_wrapped (ell increases LEFT)",
        "colors": {
            "MAX_30": C_MAX_30,
            "MAX_185": C_MAX_185,
            "MIN_115": C_MIN_115,
            "MIN_265": C_MIN_265,
        },
    }
    json_path.write_text(json.dumps(payload, indent=2) + "\n")
    print(json.dumps(payload, indent=2))
    print(f"\nWrote {png_path}")
    print(f"Wrote {pdf_path}")
    print(f"Wrote {json_path}")
    return png_path, pdf_path, json_path


if __name__ == "__main__":
    p = argparse.ArgumentParser(description=__doc__)
    p.add_argument("--tilt", type=float, default=AXIS_TILT_B_DEG)
    p.add_argument("--roll", type=float, default=ROLL_ABOUT_COE_DEG,
                   help="CCW degrees about +D (0 or 45)")
    args = p.parse_args()
    main(tilt_b=args.tilt, roll_deg=args.roll)
```

```
{
  "png": "/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/coe_rings_gradient_loops_tilt15.png",
  "pdf": "/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/coe_rings_gradient_loops_tilt15.pdf",
  "script": "/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/make_coe_rings_gradient_loops.py",
  "geometry_module": "/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/coe_half_loop_geometry.py",
  "gradient_module": "/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/coe_gradient_loop_geometry.py",
  "FULL_GRADIENT_LOOPS": true,
  "gradient_rule": "full loops; alpha *= (1+cos psi)/2, psi = ring azimuth from the \u03b8-ray point (ring point most along
sign(sin\u03b8_mid)\u00b7\u0176, \u0176 rolled for B/C; A unrolled); equivalently alpha_scale = (1 + sign(sin\u03b8_mid)\u00b7sin(phi))/2",
  "legacy_half_loop_rule_equiv_alpha_scale_ge_half": "\u03b8-ray side: keep (n\u00b7\u0176)*sin(\u03b8_mid)>=0 (Panel A: ey*sin(\u03b8_mid)>=0; TOP if
sin\u03b8>0, BOTTOM if sin\u03b8<0)",
  "panel_A_roll_deg": 0.0,
  "roll_applies_to_panels": [
    "B",
    "C"
  ],
  "base_alphas": {
    "fill_A": 0.28,
    "fill_B": 0.26,
    "fill_C": 0.25,
    "mid_line": 0.95
  },
  "axis_tilt_b_deg": 15.0,
  "roll_about_coe_deg": 0.0,
  "migkas_max_lb_deg": [
    [
      17.0,
      -9.0
    ],
    [
      185.0,
      0.0
    ]
  ],
  "migkas_min_lb_deg": [
    [
      115.0,
      0.0
    ],
    [
      274.0,
      -22.0
    ]
  ],
  "strip_theta_max_deg": [
    30.0,
    185.0
  ],
  "strip_theta_min_deg": [
    115.0,
    265.0
  ],
  "band_half_width_deg": 9.0,
  "band_full_width_deg": 18.0,
  "max_bands_deg": [
    [
      21.0,
```

```
      39.0
    ],
    [
      176.0,
      194.0
    ]
  ],
  "min_bands_deg": [
    [
      106.0,
      124.0
    ],
    [
      256.0,
      274.0
    ]
  ],
  "CoE_lb_deg": [
    180.0,
    -15.0
  ],
  "anti_CoE_lb_deg": [
    0.0,
    15.0
  ],
  "D_hat_galactic_xyz": [
    0.9659258262890683,
    0.0,
    0.25881904510252074
  ],
  "angular_sep_migkas_max0_min1_deg": 98.476384,
  "orientation": "anti-CoE at (0\u00b0, +15\u00b0), CoE at (180\u00b0, \u221215\u00b0); full loops, opacity gradient darkest at \u03b8 ray, fading to
12% opacity opposite; roll about CoE = 0\u00b0 CCW (panels B/C only; panel A unrolled)",
  "plot_convention": "Migkas Galactic Mollweide: lon_disp=-ell_wrapped (ell increases LEFT)",
  "colors": {
    "MAX_30": "#c0392b",
    "MAX_185": "#ec7063",
    "MIN_115": "#1a5276",
    "MIN_265": "#5dade2"
  }
}

Wrote /workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/coe_rings_gradient_loops_tilt15.png
Wrote /workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/coe_rings_gradient_loops_tilt15.pdf
Wrote /workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/coe_rings_gradient_loops_tilt15.json
```


```
%%runscript make_coe_rings_gradient_loops.py_roll45 --argv --roll 45
# second run of the same script with the +45 deg CCW roll: load its definitions (everything above its __main__ block), then call main()
exec(open('/workspace/cosmology/figures/make_coe_rings_gradient_loops.py').read().split('if __name__ == "__main__":')[0])
main(tilt_b=15.0, roll_deg=45.0)
```

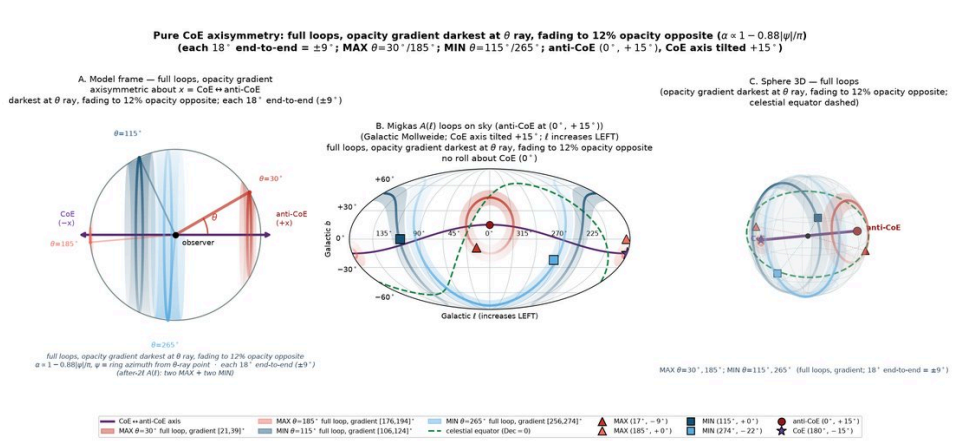
```
{
  "png": "/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/coe_rings_gradient_loops_tilt15_rot45ccw.png",
  "pdf": "/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/coe_rings_gradient_loops_tilt15_rot45ccw.pdf",
  "script": "/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/make_coe_rings_gradient_loops.py",
  "geometry_module": "/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/coe_half_loop_geometry.py",
  "gradient_module": "/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/coe_gradient_loop_geometry.py",
  "FULL_GRADIENT_LOOPS": true,
  "gradient_rule": "full loops; alpha *= (1+cos psi)/2, psi = ring azimuth from the \u03b8-ray point (ring point most along
sign(sin\u03b8_mid)\u00b7\u0176, \u0176 rolled for B/C; A unrolled); equivalently alpha_scale = (1 + sign(sin\u03b8_mid)\u00b7sin(phi))/2",
  "legacy_half_loop_rule_equiv_alpha_scale_ge_half": "\u03b8-ray side: keep (n\u00b7\u0176)*sin(\u03b8_mid)>=0 (Panel A: ey*sin(\u03b8_mid)>=0; TOP if
sin\u03b8>0, BOTTOM if sin\u03b8<0)",
  "panel_A_roll_deg": 0.0,
  "roll_applies_to_panels": [
    "B",
    "C"
  ],
  "base_alphas": {
    "fill_A": 0.28,
    "fill_B": 0.26,
    "fill_C": 0.25,
    "mid_line": 0.95
  },
  "axis_tilt_b_deg": 15.0,
  "roll_about_coe_deg": 45.0,
  "migkas_max_lb_deg": [
    [
      17.0,
      -9.0
    ],
    [
      185.0,
      0.0
    ]
  ],
  "migkas_min_lb_deg": [
    [
      115.0,
      0.0
    ],
    [
      274.0,
      -22.0
    ]
  ],
  "strip_theta_max_deg": [
    30.0,
    185.0
  ],
  "strip_theta_min_deg": [
    115.0,
    265.0
  ],
  "band_half_width_deg": 9.0,
  "band_full_width_deg": 18.0,
  "max_bands_deg": [
    [
      21.0,
```

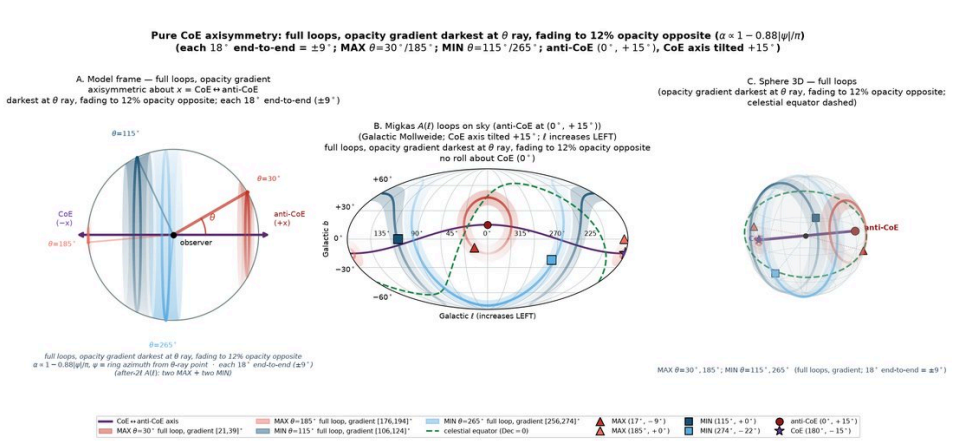
```
      39.0
    ],
    [
      176.0,
      194.0
    ]
  ],
  "min_bands_deg": [
    [
      106.0,
      124.0
    ],
    [
      256.0,
      274.0
    ]
  ],
  "CoE_lb_deg": [
    180.0,
    -15.0
  ],
  "anti_CoE_lb_deg": [
    0.0,
    15.0
  ],
  "D_hat_galactic_xyz": [
    0.9659258262890683,
    0.0,
    0.25881904510252074
  ],
  "angular_sep_migkas_max0_min1_deg": 98.476384,
  "orientation": "anti-CoE at (0\u00b0, +15\u00b0), CoE at (180\u00b0, \u221215\u00b0); full loops, opacity gradient darkest at \u03b8 ray, fading to
12% opacity opposite; roll about CoE = 45\u00b0 CCW (panels B/C only; panel A unrolled)",
  "plot_convention": "Migkas Galactic Mollweide: lon_disp=-ell_wrapped (ell increases LEFT)",
  "colors": {
    "MAX_30": "#c0392b",
    "MAX_185": "#ec7063",
    "MIN_115": "#1a5276",
    "MIN_265": "#5dade2"
  }
}
```


```
Wrote /workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/coe_rings_gradient_loops_tilt15_rot45ccw.png
Wrote /workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/coe_rings_gradient_loops_tilt15_rot45ccw.pdf
Wrote /workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/coe_rings_gradient_loops_tilt15_rot45ccw.json
```

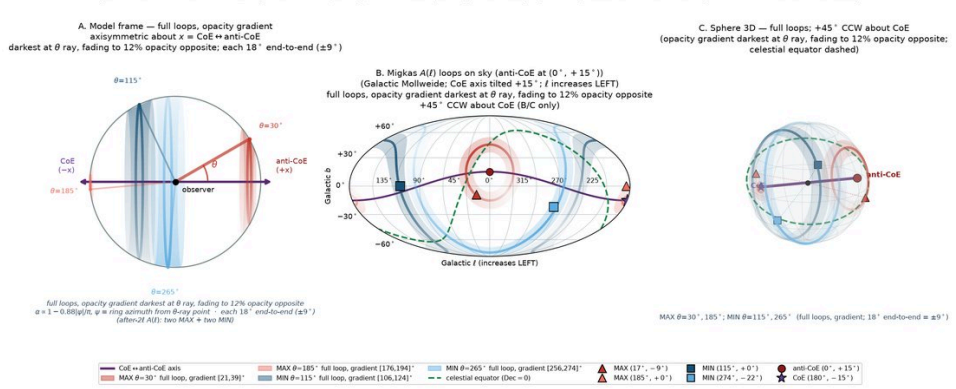
```
compare(FIGS / 'coe_rings_gradient_loops_tilt15.png', '/workspace/cosmology/figures/coe_rings_gradient_loops_tilt15.png')
compare(FIGS / 'coe_rings_gradient_loops_tilt15_rot45ccw.png', '/workspace/cosmology/figures/coe_rings_gradient_loops_tilt15_rot45ccw.png')
```

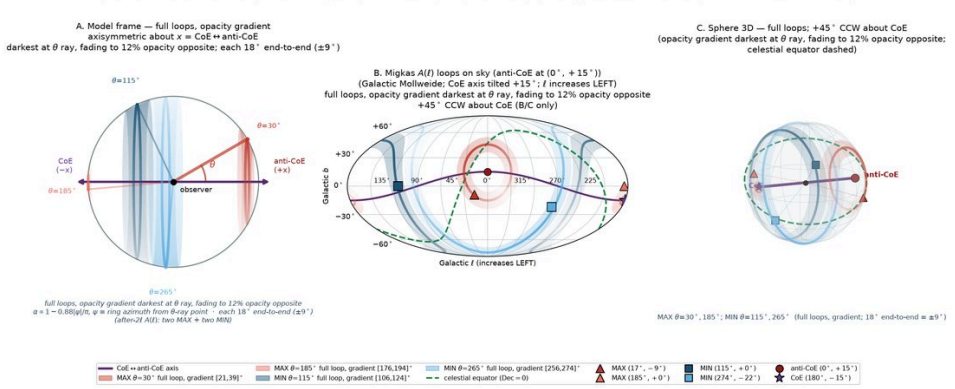
```
coe_rings_gradient_loops_tilt15.png: new (2334, 1027) vs ref (2334, 1027) | mean |diff| = 0.000/255, pixels differing >40: 0.000% -> identical
left: approved reference figures/coe_rings_gradient_loops_tilt15.png   |   right: notebook re-run steps_v3/figs/coe_rings_gradient_loops_tilt15.png
```






```
coe_rings_gradient_loops_tilt15_rot45ccw.png: new (2334, 1027) vs ref (2334, 1027) | mean |diff| = 0.000/255, pixels differing >40: 0.000% -> identical
left: approved reference figures/coe_rings_gradient_loops_tilt15_rot45ccw.png   |   right: notebook re-run
steps_v3/figs/coe_rings_gradient_loops_tilt15_rot45ccw.png
```





Step 15 — Gradient loops on Migkas Fig. 8 (roll 0 and $+45^\circ$ CCW)

What it shows. The Step 14 gradient loops on the Migkas Fig. 8 map, with the same registration, markers and gradient rule.

Source script: `/workspace/cosmology/figures/make_migkas_fig8_gradient_loops_overlay.py` (last modified 2026-09-25 20:22 CT), run verbatim. *Output-path edit:* none. The script's `OUT_DIR` follows `__file__`, which is set to `steps_v3/figs/`. *Reference image:* `figures/migkas_fig8_gradient_loops_tilt15.png`, `figures/migkas_fig8_gradient_loops_tilt15_rot45ccw.png`.

```
%%runscript make_migkas_fig8_gradient_loops_overlay.py --argv --roll 0
#!/usr/bin/env python3
"""
Migkas Fig. 8 overlay — FULL CoE loops with opacity gradient (Chris 2026-09-25).

Derived from make_migkas_fig8_half_loops_overlay.py (left unchanged).
```

```
Each FULL ring (±9° band fill + solid mid line with its white underlay) is drawn
with alpha multiplied by (1+cos psi)/2, psi = ring azimuth from the θ-ray point
(ring point most along sign(sin0)·Ŷ, Ŷ from ring_basis rolled by roll_deg) —
same helper as make_coe_rings_gradient_loops.py (coe_gradient_loop_geometry).
Contiguous elements sharing a quantized alpha level (64 levels) are merged into
one strip polygon / polyline (no beading, few AA seams).

GOOD_REG, lon_disp = -wrap(ℓ), oval clipping, antimeridian splitting, colours,
markers all unchanged. Published map untouched (drawn underneath only).

Outputs:
  roll=0  → migkas_fig8_gradient_loops_tilt15.{png,json}
  roll=45 → migkas_fig8_gradient_loops_tilt15_rot45ccw.{png,json}
"""

from __future__ import annotations

import argparse
import json
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
from matplotlib.collections import LineCollection, PolyCollection
from matplotlib.lines import Line2D
from matplotlib.patches import Ellipse, Patch
from PIL import Image

from coe_half_loop_geometry import (
    BAND_FULL,
    BAND_HALF,
    MIGKAS_MAXES,
    MIGKAS_MINS,
    STRIP_THETA_MAX,
    STRIP_THETA_MIN,
    axis_pose,
    band_edges,
    celestial_equator_lb,
    great_circle_through_poles,
    lon_disp_from_ell,
    mu_pair_for_band,
    n_to_lb,
)

from coe_gradient_loop_geometry import (
    GRADIENT_RULE,
    quantized_runs,
    rgba_array,
    segment_scale,
    small_circle_full_gradient,
)

OUT_DIR = Path(__file__).resolve().parent
```

```
AXIS_TILT_B_DEG = 15.0
ROLL_ABOUT_COE_DEG = 0.0
FULL_GRADIENT_LOOPS = True
FILL_ALPHA = 0.30
N_ALPHA_LEVELS = 64

SOURCE_CANDIDATES = [
    Path("/workspace/cosmology/map_compare/FULL_migkas_theta75.png"),
    Path("/workspace/cosmology/map_compare/migkas_fig8_theta75_crop.png"),
    Path("/workspace/cosmology/migkas/pub_maps/Fig8_full_hires.png"),
]

THETA_MAX = list(STRIP_THETA_MAX)
THETA_MIN = list(STRIP_THETA_MIN)
MAX_BANDS = band_edges(THETA_MAX)
MIN_BANDS = band_edges(THETA_MIN)

GOOD_REG = {
    "cx": 863.0,
    "cy": 763.25,
    "Rx": 693.9927,
    "Ry": 392.542425,
}

C_MAX_30 = "#00e5ff"
C_MAX_185 = "#ff2d95"
C_MIN_115 = "#b8f200"
C_MIN_265 = "#4fc3f7"
COLORS_MAX = [C_MAX_30, C_MAX_185]
COLORS_MIN = [C_MIN_115, C_MIN_265]
C_COE = "#6b3fa0"
C_ANTI = "#8e1b1b"
C_AXIS = "#5b2c6f"


def output_stem(tilt_b, roll):
    base = f"migkas_fig8_gradient_loops_tilt{int(round(tilt_b))}"
    if abs(roll) > 0.5:
        base += f"_rot{int(round(roll))}ccw"
    return base


def mollweide_xy(lon_rad, lat_rad, n_iter=64):
    lon = np.asarray(lon_rad, dtype=float)
    lat = np.asarray(lat_rad, dtype=float)
    target = np.pi * np.sin(np.clip(lat, -np.pi / 2, np.pi / 2))
    lo = np.full_like(lat, -np.pi / 2, dtype=float)
    hi = np.full_like(lat, np.pi / 2, dtype=float)
    for _ in range(n_iter):
        mid = 0.5 * (lo + hi)
        g = 2.0 * mid + np.sin(2.0 * mid)
        lo = np.where(g < target, mid, lo)
        hi = np.where(g >= target, mid, hi)
    theta = 0.5 * (lo + hi)
```

```
    theta = np.where(lat >= np.pi / 2 - 1e-15, np.pi / 2, theta)
    theta = np.where(lat <= -np.pi / 2 + 1e-15, -np.pi / 2, theta)
    x = (2.0 * np.sqrt(2.0) / np.pi) * lon * np.cos(theta)
    y = np.sqrt(2.0) * np.sin(theta)
    return x, y


def lb_to_px(ell_deg, b_deg, cx, cy, Rx, Ry):
    lon = np.deg2rad(lon_disp_from_ell(np.asarray(ell_deg, dtype=float)))
    lat = np.deg2rad(np.asarray(b_deg, dtype=float))
    x, y = mollweide_xy(lon, lat)
    Xn = x / (2.0 * np.sqrt(2.0))
    Yn = y / np.sqrt(2.0)
    return cx + Xn * Rx, cy - Yn * Ry


def as_xy(px, py):
    return float(np.asarray(px).reshape(-1)[0]), float(np.asarray(py).reshape(-1)[0])


def chamfer_distance_to_dark(dark):
    h, w = dark.shape
    INF = 1e6
    D = np.where(dark, 0.0, INF)
    for y in range(h):
        for x in range(w):
            v = D[y, x]
            if x > 0:
                v = min(v, D[y, x - 1] + 1.0)
            if y > 0:
                v = min(v, D[y - 1, x] + 1.0)
            if x > 0 and y > 0:
                v = min(v, D[y - 1, x - 1] + 1.414)
            if x + 1 < w and y > 0:
                v = min(v, D[y - 1, x + 1] + 1.414)
            D[y, x] = v
    for y in range(h - 1, -1, -1):
        for x in range(w - 1, -1, -1):
            v = D[y, x]
            if x + 1 < w:
                v = min(v, D[y, x + 1] + 1.0)
            if y + 1 < h:
                v = min(v, D[y + 1, x] + 1.0)
            if x + 1 < w and y + 1 < h:
                v = min(v, D[y + 1, x + 1] + 1.414)
            if x > 0 and y + 1 < h:
                v = min(v, D[y + 1, x - 1] + 1.414)
            D[y, x] = v
    return D


def detect_oval_bbox(rgb):
    h, w, _ = rgb.shape
    white = (rgb[:, :, 0] > 250) & (rgb[:, :, 1] > 250) & (rgb[:, :, 2] > 250)
```

```
    oval_left = np.full(h, np.nan)
    oval_right = np.full(h, np.nan)
    for y in range(h):
        row = ~white[y]
        d = np.diff(np.r_[0, row.astype(int), 0])
        starts = np.where(d == 1)[0]
        ends = np.where(d == -1)[0] - 1
        if len(starts) == 0:
            continue
        lengths = ends - starts + 1
        candidates = [
            (int(lengths[i]), int(starts[i]), int(ends[i]))
            for i in range(len(starts))
            if starts[i] < 0.72 * w and lengths[i] > 40
        ]
        if not candidates:
            continue
        candidates.sort(reverse=True)
        _, L, R = candidates[0]
        oval_left[y] = L
        oval_right[y] = R
    widths = np.where(np.isfinite(oval_left), oval_right - oval_left, 0.0)
    yeq = int(np.argmax(widths))
    thresh = 0.08 * widths[yeq]
    y0 = yeq
    while y0 > 0 and widths[y0 - 1] >= thresh:
        y0 -= 1
    y1 = yeq
    while y1 < h - 1 and widths[y1 + 1] >= thresh:
        y1 += 1
    x0 = float(oval_left[yeq])
    x1 = float(oval_right[yeq])
    return {
        "x0": int(x0), "x1": int(x1), "y0": int(y0), "y1": int(y1),
        "yeq": int(yeq),
        "cx": 0.5 * (x0 + x1), "cy": 0.5 * (y0 + y1),
        "Rx": 0.5 * (x1 - x0), "Ry": 0.5 * (y1 - y0 + 1),
    }


def choose_source():
    for p in SOURCE_CANDIDATES:
        if p.is_file():
            return p
    raise FileNotFoundError("No Migkas Fig.8 theta=75 source image found")


def sample_rgb(rgb, ell, b, cx, cy, Rx, Ry, r=4):
    px, py = lb_to_px(ell, b, cx, cy, Rx, Ry)
    xi, yi = int(round(as_xy(px, py)[0])), int(round(as_xy(px, py)[1]))
    patch = rgb[max(0, yi - r) : yi + r + 1, max(0, xi - r) : xi + r + 1]
    return patch.mean(axis=(0, 1))
```

```
def validate_landmarks(rgb, reg):
    cx, cy, Rx, Ry = reg["cx"], reg["cy"], reg["Rx"], reg["Ry"]
    pts = {}
    landmarks = [
        ("center", 0.0, 0.0),
        ("ell90", 90.0, 0.0),
        ("ell270", 270.0, 0.0),
        ("north", 0.0, 90.0),
        ("south", 0.0, -90.0),
    ]
    for i, (ell, b) in enumerate(MIGKAS_MAXES):
        landmarks.append((f"migkas_max{i}", ell, b))
    for i, (ell, b) in enumerate(MIGKAS_MINS):
        landmarks.append((f"migkas_min{i}", ell, b))
    for name, ell, b in landmarks:
        x, y = as_xy(*lb_to_px(ell, b, cx, cy, Rx, Ry))
        pts[name] = {"ell": ell, "b": b, "px": x, "py": y}
    assert pts["ell90"]["px"] < pts["center"]["px"], "l=90 must be LEFT of center"
    assert pts["ell270"]["px"] > pts["center"]["px"], "l=270 must be RIGHT of center"
    assert abs(pts["center"]["px"] - cx) < 1.5
    assert abs(pts["center"]["py"] - cy) < 1.5
    assert pts["north"]["py"] < pts["center"]["py"]
    assert pts["south"]["py"] > pts["center"]["py"]
    cmax = sample_rgb(rgb, *MIGKAS_MAXES[0], cx, cy, Rx, Ry)
    cmin = sample_rgb(rgb, *MIGKAS_MINS[1], cx, cy, Rx, Ry)
    assert (cmax[0] + cmax[1]) > (cmin[0] + cmin[1]) + 50
    return {
        "landmarks_px": pts,
        "migkas_max0_rgb_mean": [float(x) for x in cmax],
        "migkas_min1_rgb_mean": [float(x) for x in cmin],
    }


def inside_ellipse(px, py, ecx, ecy, eRx, eRy, r_lim=1.0):
    px = np.asarray(px, dtype=float)
    py = np.asarray(py, dtype=float)
    Xn = (px - ecx) / eRx
    Yn = (py - ecy) / eRy
    return Xn * Xn + Yn * Yn <= float(r_lim) * float(r_lim)


def plot_lb_curve(
    ax, ell, b, cx, cy, Rx, Ry, inset=1.0, jump_px=45.0,
    clip_r2=None, clip_path=None, color_oval=None, **kw
):
    px, py = lb_to_px(ell, b, cx, cy, Rx, Ry)
    px = np.asarray(px, dtype=float).reshape(-1)
    py = np.asarray(py, dtype=float).reshape(-1)
    px = cx + (px - cx) * inset
    py = cy + (py - cy) * inset
    Xn = (px - cx) / Rx
    Yn = (cy - py) / Ry
    r2_lim = (inset * inset) if clip_r2 is None else float(clip_r2)
    inside = Xn * Xn + Yn * Yn <= r2_lim
```

```
    if color_oval is not None:
        ocx, ocy, oRx, oRy, o_r = color_oval
        inside = inside & inside_ellipse(px, py, ocx, ocy, oRx, oRy, o_r)
    px = np.where(inside, px, np.nan)
    py = np.where(inside, py, np.nan)
    segs = []
    cur_x, cur_y = [], []
    for x, y in zip(px, py):
        if not (np.isfinite(x) and np.isfinite(y)):
            if len(cur_x) >= 2:
                segs.append((cur_x, cur_y))
            cur_x, cur_y = [], []
            continue
        if cur_x and np.hypot(x - cur_x[-1], y - cur_y[-1]) > jump_px:
            if len(cur_x) >= 2:
                segs.append((cur_x, cur_y))
            cur_x, cur_y = [float(x)], [float(y)]
        else:
            cur_x.append(float(x))
            cur_y.append(float(y))
    if len(cur_x) >= 2:
        segs.append((cur_x, cur_y))
    if clip_path is not None:
        kw["clip_path"] = clip_path
        kw["clip_on"] = True
    for sx, sy in segs:
        (line,) = ax.plot(sx, sy, **kw)
        if clip_path is not None:
            line.set_clip_path(clip_path)
            line.set_clip_on(True)


def _full_ring_px_arrays(D, mu, theta_mid, roll_deg, cx, cy, Rx, Ry, inset, n_pts=721):
    """FULL ring → pixel arrays + per-point alpha scale (1+cos psi)/2.
    Sampled psi = -180°..+180° so the only closure point is the invisible one."""
    pts, _, scale = small_circle_full_gradient(
        D, float(mu), theta_mid, roll_deg=roll_deg, n=n_pts
    )
    ell = np.empty(n_pts)
    b = np.empty(n_pts)
    for i, p in enumerate(pts):
        ell[i], b[i] = n_to_lb(p)
    lon = np.deg2rad(lon_disp_from_ell(ell))
    lat = np.deg2rad(b)
    x_m, y_m = mollweide_xy(lon, lat)
    Xn = x_m / (2.0 * np.sqrt(2.0))
    Yn = y_m / np.sqrt(2.0)
    px = cx + Xn * Rx
    py = cy - Yn * Ry
    px = cx + (px - cx) * inset
    py = cy + (py - cy) * inset
    return lon, px, py, scale
```

```
def plot_full_ring_gradient(
    ax, D, mu, theta_mid, roll_deg, cx, cy, Rx, Ry, color, lw, inset=1.0, jump_px=80.0,
    clip_path=None, color_oval=None,
):
    """Stroke the FULL mid ring (white underlay + colour) with opacity gradient.
    Segments crossing the antimeridian / jumping / leaving the oval are dropped."""
    lon, px, py, sc = _full_ring_px_arrays(
        D, mu, theta_mid, roll_deg, cx, cy, Rx, Ry, inset, n_pts=2001
    )
    r2_lim = inset * inset + 1e-10
    gap = 0.9 * np.pi
    n = len(lon)
    ok_pt = ((px - cx) / Rx) ** 2 + ((cy - py) / Ry) ** 2 <= r2_lim
    if color_oval is not None:
        ocx, ocy, oRx, oRy, o_r = color_oval
        ok_pt = ok_pt & inside_ellipse(px, py, ocx, ocy, oRx, oRy, o_r)
    valid = (
        (np.abs(np.diff(lon)) <= gap)
        & (np.hypot(np.diff(px), np.diff(py)) <= jump_px)
        & ok_pt[:-1] & ok_pt[1:]
    )
    runs = quantized_runs(segment_scale(sc), valid, n_levels=N_ALPHA_LEVELS)
    if not runs:
        return
    lines = [np.column_stack([px[i0:i1 + 1], py[i0:i1 + 1]]) for i0, i1, _ in runs]
    levels = [r[2] for r in runs]
    for col, width, z, a in (("white", lw + 1.5, 3, 0.95), (color, lw, 4, 1.0)):
        lc = LineCollection(
            lines, colors=rgba_array(col, [a * L for L in levels]), linewidths=width,
            capstyle="butt", joinstyle="round", zorder=z,
        )
        ax.add_collection(lc)
        if clip_path is not None:
            lc.set_clip_path(clip_path)
            lc.set_clip_on(True)


def fill_full_gradient_band(
    ax, D, theta_lo, theta_hi, theta_mid, roll_deg, cx, cy, Rx, Ry, color,
    inset=1.0, clip_path=None, color_oval=None, n_pts=721, jump_px=80.0,
):
    mu_a, mu_b = mu_pair_for_band(theta_lo, theta_hi)
    mu_mid = float(np.cos(np.deg2rad(theta_mid)))
    lon_a, px_a, py_a, sc = _full_ring_px_arrays(
        D, mu_a, theta_mid, roll_deg, cx, cy, Rx, Ry, inset, n_pts=n_pts
    )
    lon_b, px_b, py_b, _ = _full_ring_px_arrays(
        D, mu_b, theta_mid, roll_deg, cx, cy, Rx, Ry, inset, n_pts=n_pts
    )
    r2_lim = inset * inset + 1e-10

    def _inside(px, py):
        ok = ((px - cx) / Rx) ** 2 + ((cy - py) / Ry) ** 2 <= r2_lim
        if color_oval is not None:
```

```
            ocx, ocy, oRx, oRy, o_r = color_oval
            ok = ok & inside_ellipse(px, py, ocx, ocy, oRx, oRy, o_r)
        return ok

    gap = 0.9 * np.pi
    ia = _inside(px_a, py_a)
    ib = _inside(px_b, py_b)
    valid = (
        (np.abs(np.diff(lon_a)) <= gap)
        & (np.abs(np.diff(lon_b)) <= gap)
        & (np.abs(lon_a - lon_b)[:-1] <= gap)
        & (np.abs(lon_a - lon_b)[1:] <= gap)
        & (np.hypot(np.diff(px_a), np.diff(py_a)) <= jump_px)
        & (np.hypot(np.diff(px_b), np.diff(py_b)) <= jump_px)
        & ia[:-1] & ib[:-1] & ia[1:] & ib[1:]
    )
    runs = quantized_runs(segment_scale(sc), valid, n_levels=N_ALPHA_LEVELS)
    polys = [
        np.column_stack([
            np.concatenate([px_a[i0:i1 + 1], px_b[i0:i1 + 1][::-1]]),
            np.concatenate([py_a[i0:i1 + 1], py_b[i0:i1 + 1][::-1]]),
        ])
        for i0, i1, _ in runs
    ]
    if polys:
        coll = PolyCollection(
            polys, facecolors=rgba_array(color, [FILL_ALPHA * r[2] for r in runs]),
            edgecolors="none", zorder=2, linewidths=0,
        )
        ax.add_collection(coll)
        if clip_path is not None:
            coll.set_clip_path(clip_path)
            coll.set_clip_on(True)

    plot_full_ring_gradient(
        ax, D, mu_mid, theta_mid, roll_deg, cx, cy, Rx, Ry, color, lw=2.8,
        inset=inset, clip_path=clip_path, color_oval=color_oval,
    )


def draw_overlay(rgb, reg, seed, source, pose, roll_deg, png_path):
    h, w, _ = rgb.shape
    cx, cy, Rx, Ry = reg["cx"], reg["cy"], reg["Rx"], reg["Ry"]
    scx = float(seed["cx"])
    scy = float(seed["cy"])
    sRx = float(seed["Rx"])
    sRy = float(seed["Ry"])
    D = pose["D"]
    anti_lb = pose["anti_lb"]
    coe_lb = pose["coe_lb"]
    tilt = pose["tilt_b_deg"]

    dpi = 120
    fig, ax = plt.subplots(figsize=(w / dpi, (h / dpi) + 1.35), dpi=dpi)
```

```
    ax.imshow(rgb, origin="upper", extent=(0, w, h, 0), interpolation="nearest")
    ax.set_xlim(0, w)
    ax.set_ylim(h, 0)
    ax.set_aspect("equal")
    ax.axis("off")

    clip_oval = Ellipse(
        (cx, cy), width=2.0 * Rx, height=2.0 * Ry,
        facecolor="none", edgecolor="none", transform=ax.transData,
    )
    ax.add_patch(clip_oval)
    STROKE_INSET = 0.97
    COLOR_R = 0.978
    color_oval = (scx, scy, sRx, sRy, COLOR_R)

    def stroke_curve_geom(ell, b, color, lw, ls="-"):
        plot_lb_curve(
            ax, ell, b, cx, cy, Rx, Ry, inset=1.0, clip_path=clip_oval,
            color_oval=color_oval,
            color="white", lw=lw + 1.6, ls=ls, alpha=0.95,
            solid_capstyle="round", zorder=3,
        )
        plot_lb_curve(
            ax, ell, b, cx, cy, Rx, Ry, inset=1.0, clip_path=clip_oval,
            color_oval=color_oval,
            color=color, lw=lw, ls=ls, alpha=1.0,
            solid_capstyle="round", zorder=4,
        )

    def mark_point(ell, b, marker, color, ms):
        px, py = as_xy(*lb_to_px(ell, b, cx, cy, Rx, Ry))
        halo_ms = ms + 1.8
        ms_px = 0.5 * halo_ms * (dpi / 72.0)
        margin = ms_px / min(sRx, sRy) + 0.012
        r_lim = max(0.55, COLOR_R - margin)
        Xn = (px - scx) / sRx
        Yn = (py - scy) / sRy
        r_col = float(np.hypot(Xn, Yn))
        if r_col > 1e-9:
            if r_col > r_lim:
                scale = r_lim / r_col
                px = scx + (px - scx) * scale
                py = scy + (py - scy) * scale
                Xn = (px - scx) / sRx
                Yn = (py - scy) / sRy
                r_col = float(np.hypot(Xn, Yn))
            mild = min(STROKE_INSET, 0.995)
            px = scx + (px - scx) * mild
            py = scy + (py - scy) * mild
            Xn = (px - scx) / sRx
            Yn = (py - scy) / sRy
            r_col = float(np.hypot(Xn, Yn))
        if r_col + margin > COLOR_R + 1e-6:
            return
```

```
    def _mark(m_size, m_color, m_edge, m_ew, z):
        (line,) = ax.plot(
            px, py, marker=marker, ms=m_size, color=m_color,
            markeredgecolor=m_edge, markeredgewidth=m_ew, linestyle="None",
            zorder=z, clip_path=clip_oval, clip_on=True,
        )
        line.set_clip_path(clip_oval)
        line.set_clip_on(True)

    _mark(ms + 1.8, "white", "white", 0, 6)
    _mark(ms, color, "k", 0.55, 7)

_ax_ell, _ax_b = great_circle_through_poles(D, n=721)
stroke_curve_geom(_ax_ell, _ax_b, C_AXIS, lw=1.15)

for (lo, hi), mid, color in zip(MAX_BANDS, THETA_MAX, COLORS_MAX):
    fill_full_gradient_band(
        ax, D, lo, hi, mid, roll_deg, cx, cy, Rx, Ry, color,
        inset=STROKE_INSET, clip_path=clip_oval, color_oval=color_oval,
    )
for (lo, hi), mid, color in zip(MIN_BANDS, THETA_MIN, COLORS_MIN):
    fill_full_gradient_band(
        ax, D, lo, hi, mid, roll_deg, cx, cy, Rx, Ry, color,
        inset=STROKE_INSET, clip_path=clip_oval, color_oval=color_oval,
    )

mark_point(anti_lb[0], anti_lb[1], "o", C_ANTI, ms=5.5)
mark_point(coe_lb[0], coe_lb[1], "*", C_COE, ms=7.5)
for (ell, b), color in zip(MIGKAS_MAXES, COLORS_MAX):
    mark_point(ell, b, "^", color, ms=6.5)
for (ell, b), color in zip(MIGKAS_MINS, COLORS_MIN):
    mark_point(ell, b, "s", color, ms=6.0)

roll_lab = (
    rf"; $+{roll_deg:.0f}^\circ$ CCW about CoE"
    if abs(roll_deg) > 0.5
    else ""
)
handles = [
    Patch(
        facecolor=C_MAX_30, alpha=0.55, edgecolor=C_MAX_30,
        label=rf"MAX $\theta\!=\!{THETA_MAX[0]:.0f}^\circ$ full loop, gradient "
        rf"[{MAX_BANDS[0][0]:.0f},{MAX_BANDS[0][1]:.0f}]$^\circ$",
    ),
    Patch(
        facecolor=C_MAX_185, alpha=0.55, edgecolor=C_MAX_185,
        label=rf"MAX $\theta\!=\!{THETA_MAX[1]:.0f}^\circ$ full loop, gradient "
        rf"[{MAX_BANDS[1][0]:.0f},{MAX_BANDS[1][1]:.0f}]$^\circ$",
    ),
    Patch(
        facecolor=C_MIN_115, alpha=0.55, edgecolor=C_MIN_115,
        label=rf"MIN $\theta\!=\!{THETA_MIN[0]:.0f}^\circ$ full loop, gradient "
        rf"[{MIN_BANDS[0][0]:.0f},{MIN_BANDS[0][1]:.0f}]$^\circ$",
```

```
        ),
        Patch(
            facecolor=C_MIN_265, alpha=0.55, edgecolor=C_MIN_265,
            label=rf"MIN $\theta\!=\!{THETA_MIN[1]:.0f}^\circ$ full loop, gradient "
            rf"[{MIN_BANDS[1][0]:.0f},{MIN_BANDS[1][1]:.0f}]$^\circ$",
        ),
        Line2D(
            [0], [0], color=C_AXIS, lw=2.0,
            label=rf"CoE axis tilted $+{tilt:.0f}^\circ$",
        ),
        Line2D(
            [0], [0], marker="^", color="w", markerfacecolor=C_MAX_30,
            markeredgecolor="k", linestyle="None", ms=8,
            label=rf"MAX $({MIGKAS_MAXES[0][0]:.0f}^\circ,{MIGKAS_MAXES[0][1]:+.0f}^\circ)$",
        ),
        Line2D(
            [0], [0], marker="^", color="w", markerfacecolor=C_MAX_185,
            markeredgecolor="k", linestyle="None", ms=8,
            label=rf"MAX $({MIGKAS_MAXES[1][0]:.0f}^\circ,{MIGKAS_MAXES[1][1]:+.0f}^\circ)$",
        ),
        Line2D(
            [0], [0], marker="s", color="w", markerfacecolor=C_MIN_115,
            markeredgecolor="k", linestyle="None", ms=7,
            label=rf"MIN $({MIGKAS_MINS[0][0]:.0f}^\circ,{MIGKAS_MINS[0][1]:+.0f}^\circ)$",
        ),
        Line2D(
            [0], [0], marker="s", color="w", markerfacecolor=C_MIN_265,
            markeredgecolor="k", linestyle="None", ms=7,
            label=rf"MIN $({MIGKAS_MINS[1][0]:.0f}^\circ,{MIGKAS_MINS[1][1]:+.0f}^\circ)$",
        ),
        Line2D(
            [0], [0], marker="o", color=C_ANTI, markeredgecolor="k",
            linestyle="None", ms=6,
            label=rf"anti-CoE $+x$ $(0^\circ,+{tilt:.0f}^\circ)$",
        ),
        Line2D(
            [0], [0], marker="*", color=C_COE, markeredgecolor="k",
            linestyle="None", ms=9,
            label=rf"CoE $(180^\circ,-{tilt:.0f}^\circ)$",
        ),
    ]
    leg = ax.legend(
        handles=handles, loc="upper center", bbox_to_anchor=(0.5, -0.012),
        ncol=4, fontsize=7.5, frameon=True, fancybox=False,
        edgecolor="#222222", framealpha=1.0, facecolor="white",
        labelcolor="#111111", borderpad=0.6, handlelength=2.0,
    )
    leg.get_frame().set_linewidth(1.2)
    fig.suptitle(
        r"Migkas Fig. 8 $\theta=75^\circ$ $A/A_{\rm all}$"
        r" + CoE full loops, opacity gradient darkest at $\theta$ ray, fading to 12% opacity opposite"
        "\n"
        rf"(anti-CoE at $(0^\circ,+{tilt:.0f}^\circ)$; CoE axis tilted $+{tilt:.0f}^\circ$; "
        rf"each $18^\circ$ end-to-end = $\pm9^\circ$; $\alpha\propto1-0.88|\psi|/\pi${roll_lab}; $\ell$ increases LEFT)",
```

```
        fontsize=10.5, y=0.985,
    )
    fig.subplots_adjust(left=0.01, right=0.99, top=0.92, bottom=0.13)
    fig.savefig(png_path, dpi=dpi, facecolor="white")
    plt.close(fig)
    print(f"wrote {png_path}")


def main(tilt_b=None, roll_deg=None):
    tilt_b = float(AXIS_TILT_B_DEG if tilt_b is None else tilt_b)
    roll_deg = float(ROLL_ABOUT_COE_DEG if roll_deg is None else roll_deg)
    assert FULL_GRADIENT_LOOPS

    pose = axis_pose(tilt_b)
    stem = output_stem(tilt_b, roll_deg)
    png_path = OUT_DIR / f"{stem}.png"
    json_path = OUT_DIR / f"{stem}.json"

    source = choose_source()
    print(f"source: {source}")
    rgb = np.asarray(Image.open(source).convert("RGB"))
    seed = detect_oval_bbox(rgb)
    reg = dict(GOOD_REG)
    h, w, _ = rgb.shape
    dark = rgb.mean(axis=2) < 55
    dist = chamfer_distance_to_dark(dark)
    ell_eq, b_eq = celestial_equator_lb(2500)
    lon = np.deg2rad(lon_disp_from_ell(ell_eq))
    lat = np.deg2rad(b_eq)
    mx, my = mollweide_xy(lon, lat)
    Xn = mx / (2.0 * np.sqrt(2.0))
    Yn = my / np.sqrt(2.0)
    px = reg["cx"] + Xn * reg["Rx"]
    py = reg["cy"] - Yn * reg["Ry"]
    m = (px >= 1) & (px < w - 1) & (py >= 1) & (py < h - 1)
    xi = np.clip(np.round(px[m]).astype(int), 0, w - 1)
    yi = np.clip(np.round(py[m]).astype(int), 0, h - 1)
    reg["ceq_median_px"] = float(np.median(dist[yi, xi]))
    reg["registration_source"] = "hard-set GOOD_REG (pre medial-axis retune)"
    checks = validate_landmarks(rgb, reg)
    print("validation OK")

    draw_overlay(rgb, reg, seed, source, pose, roll_deg, png_path)

    meta = {
        "png": str(png_path),
        "script": str(Path(__file__).resolve()),
        "geometry_module": str(OUT_DIR / "coe_half_loop_geometry.py"),
        "source_image": str(source),
        "gradient_module": str(OUT_DIR / "coe_gradient_loop_geometry.py"),
        "FULL_GRADIENT_LOOPS": True,
        "gradient_rule": GRADIENT_RULE,
        "alpha_levels": N_ALPHA_LEVELS,
        "base_alphas": {"fill": FILL_ALPHA, "white_underlay": 0.95, "mid_line": 1.0},
```

```
        "axis_tilt_b_deg": tilt_b,
        "roll_about_coe_deg": roll_deg,
        "anti_CoE_lb_deg": [float(pose["anti_lb"][0]), float(pose["anti_lb"][1])],
        "CoE_lb_deg": [float(pose["coe_lb"][0]), float(pose["coe_lb"][1])],
        "D_hat_galactic_xyz": pose["D"].tolist(),
        "migkas_max_lb_deg": [list(p) for p in MIGKAS_MAXES],
        "migkas_min_lb_deg": [list(p) for p in MIGKAS_MINS],
        "theta_max_from_plus_x_deg": list(THETA_MAX),
        "theta_min_from_plus_x_deg": list(THETA_MIN),
        "band_half_width_deg": BAND_HALF,
        "band_full_width_deg": BAND_FULL,
        "max_bands_deg": [list(b) for b in MAX_BANDS],
        "min_bands_deg": [list(b) for b in MIN_BANDS],
        "colors": {
            "MAX_30": C_MAX_30,
            "MAX_185": C_MAX_185,
            "MIN_115": C_MIN_115,
            "MIN_265": C_MIN_265,
        },
        "convention": (
            "Galactic Mollweide; lon_disp=-wrap(ell); ell increases LEFT "
            "(Migkas Fig.8)"
        ),
        "registration": {
            "cx": reg["cx"], "cy": reg["cy"], "Rx": reg["Rx"], "Ry": reg["Ry"],
            "ceq_median_px": reg["ceq_median_px"],
            "registration_source": reg.get("registration_source"),
        },
        "landmark_checks": checks,
        "matches_three_panel": str(OUT_DIR / "make_coe_rings_gradient_loops.py"),
        "drawing": (
            "FULL loops with opacity gradient. Each = PolyCollection strips between "
            "±9° edges + solid mid line (white underlay + colour LineCollections), "
            "alpha * (1+cos psi)/2 grouped into quantized levels; darkest at θ-ray "
            "point, alpha 0 at the opposite point. Antimeridian crossings dropped. "
            "Dec=0 NOT overlaid; published map untouched."
        ),
    }
    json_path.write_text(json.dumps(meta, indent=2) + "\n")
    print(f"wrote {json_path}")
    return png_path, json_path


if __name__ == "__main__":
    p = argparse.ArgumentParser(description=__doc__)
    p.add_argument("--tilt", type=float, default=AXIS_TILT_B_DEG)
    p.add_argument("--roll", type=float, default=ROLL_ABOUT_COE_DEG)
    args = p.parse_args()
    main(tilt_b=args.tilt, roll_deg=args.roll)
```

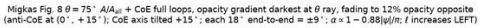
```
source: /workspace/cosmology/map_compare/FULL_migkas_theta75.png
validation OK
wrote /workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/migkas_fig8_gradient_loops_tilt15.png
wrote /workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/migkas_fig8_gradient_loops_tilt15.json
```

```
%%runscript make_migkas_fig8_gradient_loops_overlay.py_roll45 --argv --roll 45
# second run of the same script with the +45 deg CCW roll: load its definitions (everything above its __main__ block), then call main()
exec(open('/workspace/cosmology/figures/make_migkas_fig8_gradient_loops_overlay.py').read().split('if __name__ == "__main__":')[0])
main(tilt_b=15.0, roll_deg=45.0)
```

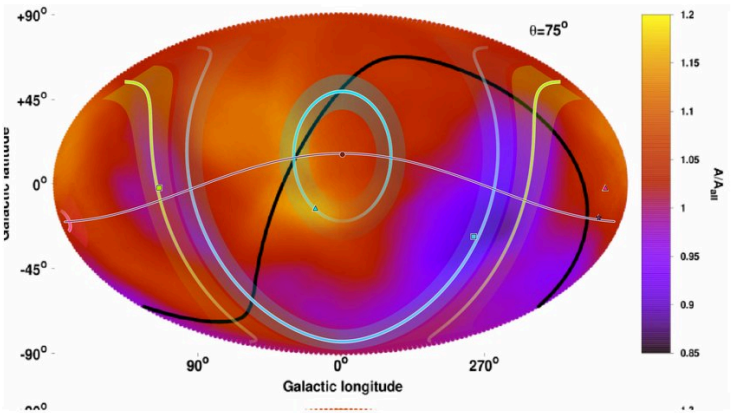
```
source: /workspace/cosmology/map_compare/FULL_migkas_theta75.png
validation OK
wrote /workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/migkas_fig8_gradient_loops_tilt15_rot45ccw.png
wrote /workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/migkas_fig8_gradient_loops_tilt15_rot45ccw.json
```

```
compare(FIGS / 'migkas_fig8_gradient_loops_tilt15.png', '/workspace/cosmology/figures/migkas_fig8_gradient_loops_tilt15.png')
compare(FIGS / 'migkas_fig8_gradient_loops_tilt15_rot45ccw.png', '/workspace/cosmology/figures/migkas_fig8_gradient_loops_tilt15_rot45ccw.png')
```

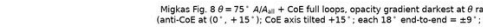
```
migkas_fig8_gradient_loops_tilt15.png: new (1975, 1515) vs ref (1975, 1515) | mean |diff| = 0.000/255, pixels differing >40: 0.000% -> identical
left: approved reference figures/migkas_fig8_gradient_loops_tilt15.png   |   right: notebook re-run steps_v3/figs/migkas_fig8_gradient_loops_tilt15.png
```



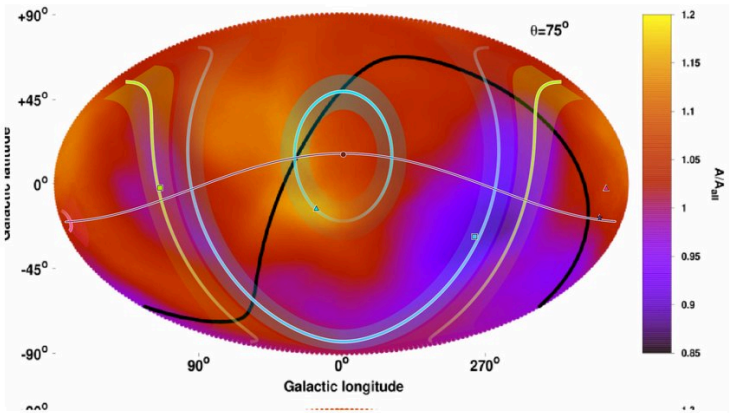
Migkas Fig. 8 — $\theta = 75°$ $A/A_{\rm all}$ (their numbered colorbar)

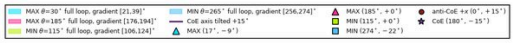




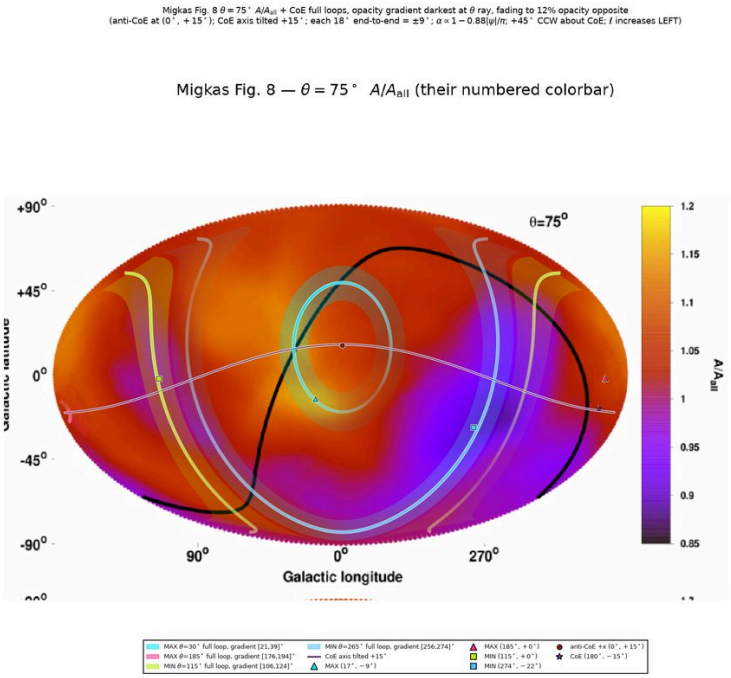


Migkas Fig. 8 — $\theta = 75°$ $A/A_{\rm all}$ (their numbered colorbar)

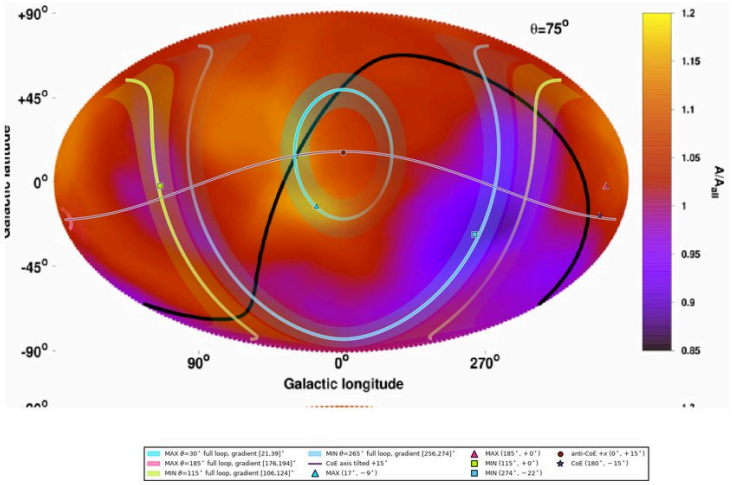





```
migkas_fig8_gradient_loops_tilt15_rot45ccw.png: new (1975, 1515) vs ref (1975, 1515) | mean |diff| = 0.000/255, pixels differing >40: 0.000% ->
identical
left: approved reference figures/migkas_fig8_gradient_loops_tilt15_rot45ccw.png   |   right: notebook re-run
steps_v3/figs/migkas_fig8_gradient_loops_tilt15_rot45ccw.png
```



Migkas Fig. 8 θ = 75° A/A_all + CoE full loops, opacity gradient darkest at θ ray, fading to 12% opacity opposite
(anti-CoE at (0°, +15°); CoE axis tilted +15°; each 18° end-to-end = ±9°; α ∝ 1 − 0.88|ψ|/π; +45° CCW about CoE; ℓ increases LEFT)
Migkas Fig. 8 — θ = 75° A/A_all (their numbered colorbar)



Step 16a — Matched three-panel: 2 MAX + 2 MIN loops, $\pm 9^\circ$ bands, axis $+15^\circ$

What it shows. The final three-panel layout: two MAX loops (two reds) and two MIN loops (two blues), each a clean transparent $\pm 9^\circ$ band with a mid-line, and no orange loop. Axis tilted $+15^\circ$. Panel A is the flat model-frame angle circle with Galactic ℓ ticks; Panel B the Mollweide map; Panel C the 3D sphere, drawn **at the same diameter as Panel A** (the canvas was widened rather than anything shrunk).

Source script: `/workspace/cosmology/figures/make_coe_rings_migkas_pin_axis_matched.py` (last modified 2026-09-26 22:54 CT), run verbatim. *Output-path edit:* `/workspace/cosmology/papers/spreadsheet_model/loop_threepanel_matched` → `/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/loop_threep`. *Reference image:* `papers/spreadsheet_model/loop_threepanel_matched.png`.

```
%%runscript make_coe_rings_migkas_pin_axis_matched.py
#!/usr/bin/env python3
"""
CoE rings + Migkas A(ell) extrema loops in the Galactic frame.

Concept:
  Pure CoE axisymmetry → constant A on small-circle rings about x = CoE↔anti-CoE.
  Scan A vs angle from +x (anti-CoE) around 360°. After-2ℓ A(ℓ) has TWO maxima
```

```
  and TWO minima (from best_fit_2ell_multiplier.npz A_after):
    MAX at θ = 30° and 185° from +x,
    MIN at θ = 115° and 265° from +x.

  Exactly FOUR loops — each ONE transparent highlight band, 18° end-to-end
  (= ±9° = 5% of 360°) about the center θ:
    MAX: 30±9 → [21°, 39°] and 185±9 → [176°, 194°]
    MIN: 115±9 → [106°, 124°] and 265±9 → [256°, 274°]
  Drawing: ONE smooth transparent filled ribbon between θ−9° and θ+9°
  + ONE solid mid-stroke at the exact center θ. No multi-stroke wash,
  no dense linspace hairball, no dual strong boundaries.
  Four distinct colors (two reds + two blues). No orange, no 283° companion.

  Galactic orientation: anti-CoE (+x) at ℓ=0°, b=+15°; CoE (−x) at ℓ=180°, b=−15°
  (axis tilted +15° so features near ℓ=0 move north on Mollweide; Migkas markers fixed).
  Sky markers (ℓ, b):
    Maxes: map extrema (17°, −9°) for ~30° max; strip (185°, 0°) for second max.
    Mins:  strip (115°, 0°) for ~115° min; map extrema (274°, −22°) for ~265° min.

Outputs:
  coe_rings_migkas_pin_axis.png
  coe_rings_migkas_pin_axis.json
"""

from __future__ import annotations

import json
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
from matplotlib.lines import Line2D
from matplotlib.collections import PolyCollection
from matplotlib.patches import Circle, Patch, Polygon
from mpl_toolkits.mplot3d import Axes3D  # noqa: F401
from mpl_toolkits.mplot3d.art3d import Poly3DCollection

OUT_DIR = Path(__file__).resolve().parent
PNG_PATH = Path("/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/loop_threepanel_matched.png")
JSON_PATH = Path("/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/loop_threepanel_matched.json")

# 5% of 360° = 18° end-to-end → half-width ±9° about each center
BAND_HALF = 9.0
BAND_FULL = 2.0 * BAND_HALF  # 18° = 5% of 360°
BAND_TOTAL = BAND_FULL  # alias
STRIP_THETA_MAX = [29.5, 187.5]
STRIP_THETA_MIN = [122.6, 264.4]

MIGKAS_MAXES = [(17.0, -9.0), (185.0, 0.0)]
MIGKAS_MINS = [(115.0, 0.0), (274.0, -22.0)]

C_COE = "#6b3fa0"
C_ANTI = "#8e1b1b"  # dark red marker; distinct from loop reds
# Four distinct loop colors: two reds (MAX) + two blues (MIN); no orange/yellow
```

```
C_MAX_30 = "#0097b2"    # cyan — MAX at 29.5°
C_MAX_185 = "#e0107a"   # magenta — MAX at 187.5°
C_MIN_115 = "#6fa800"   # green — MIN at 122.6°
C_MIN_265 = "#2166d4"   # blue — MIN at 264.4°
C_AXIS = "#5b2c6f"
C_EQ = "#1e8449"

# Per-loop color maps (order matches STRIP_THETA_MAX / STRIP_THETA_MIN)
COLORS_MAX = [C_MAX_30, C_MAX_185]
COLORS_MIN = [C_MIN_115, C_MIN_265]


def wrap_lon_deg(lon: np.ndarray) -> np.ndarray:
    return ((np.asarray(lon, dtype=float) + 180.0) % 360.0) - 180.0


def lon_disp_from_ell(ell_deg: np.ndarray) -> np.ndarray:
    """Migkas Galactic: ℓ increases LEFT → lon_disp = −ℓ_wrapped."""
    return -wrap_lon_deg(ell_deg)


def lb_to_n(l_deg: float, b_deg: float) -> np.ndarray:
    l = np.deg2rad(l_deg)
    b = np.deg2rad(b_deg)
    return np.array(
        [np.cos(b) * np.cos(l), np.cos(b) * np.sin(l), np.sin(b)], dtype=float
    )


def n_to_lb(n: np.ndarray) -> tuple[float, float]:
    n = np.asarray(n, dtype=float)
    n = n / np.linalg.norm(n)
    b = float(np.rad2deg(np.arcsin(np.clip(n[2], -1.0, 1.0))))
    l = float(np.rad2deg(np.arctan2(n[1], n[0])) % 360.0)
    return l, b


def celestial_equator_lb(n: int = 720) -> tuple[np.ndarray, np.ndarray]:
    """Dec=0 (ICRS) as Galactic (ℓ, b). IAU J2000 NGP convention, no astropy."""
    ra_ngp = np.deg2rad(192.85948)
    dec_ngp = np.deg2rad(27.12825)
    l_ncp = np.deg2rad(122.93192)
    ra = np.linspace(0.0, 2.0 * np.pi, n, endpoint=False)
    dec = 0.0
    sin_b = np.sin(dec) * np.sin(dec_ngp) + np.cos(dec) * np.cos(dec_ngp) * np.cos(ra - ra_ngp)
    sin_b = np.clip(sin_b, -1.0, 1.0)
    b = np.arcsin(sin_b)
    y = np.cos(dec) * np.sin(ra - ra_ngp)
    x = np.sin(dec) * np.cos(dec_ngp) - np.cos(dec) * np.sin(dec_ngp) * np.cos(ra - ra_ngp)
    l = (l_ncp - np.arctan2(y, x)) % (2.0 * np.pi)
    return np.rad2deg(l), np.rad2deg(b)


def great_circle_through_poles(D: np.ndarray, n: int = 361) -> tuple[np.ndarray, np.ndarray]:
```

```
    """Great circle through ±D (any meridian of the CoE axis)."""
    a = D / np.linalg.norm(D)
    tmp = np.array([0.0, 0.0, 1.0])
    if abs(np.dot(a, tmp)) > 0.9:
        tmp = np.array([1.0, 0.0, 0.0])
    e2 = np.cross(a, tmp)
    e2 /= np.linalg.norm(e2)
    t = np.linspace(0.0, 2.0 * np.pi, n)
    pts = np.outer(np.cos(t), a) + np.outer(np.sin(t), e2)
    ls = np.array([n_to_lb(p)[0] for p in pts])
    bs = np.array([n_to_lb(p)[1] for p in pts])
    return ls, bs


def small_circle_about_axis(axis: np.ndarray, mu: float, n: int = 361) -> np.ndarray:
    """Unit vectors on the small circle n·axis = mu (|mu|<1)."""
    a = axis / np.linalg.norm(axis)
    mu = float(np.clip(mu, -1.0 + 1e-9, 1.0 - 1e-9))
    tmp = np.array([0.0, 0.0, 1.0]) if abs(a[2]) < 0.9 else np.array([1.0, 0.0, 0.0])
    e1 = np.cross(a, tmp)
    e1 /= np.linalg.norm(e1)
    e2 = np.cross(a, e1)
    rho = np.sqrt(max(0.0, 1.0 - mu * mu))
    t = np.linspace(0.0, 2.0 * np.pi, n)
    return (mu * a)[None, :] + rho * (np.outer(np.cos(t), e1) + np.outer(np.sin(t), e2))


def split_lon_segments(lon_rad: np.ndarray, lat_rad: np.ndarray, gap: float = np.pi):
    """Split polyline where Mollweide lon jumps (wrap discontinuity)."""
    d = np.diff(lon_rad)
    cuts = np.where(np.abs(d) > gap)[0] + 1
    idx = np.concatenate([[0], cuts, [len(lon_rad)]])
    segs = []
    for i0, i1 in zip(idx[:-1], idx[1:]):
        if i1 - i0 >= 2:
            segs.append((lon_rad[i0:i1], lat_rad[i0:i1]))
    return segs


def plot_lb_curve_mollweide(ax, l_deg, b_deg, **kwargs):
    lon = np.deg2rad(lon_disp_from_ell(l_deg))
    lat = np.deg2rad(b_deg)
    for xr, yr in split_lon_segments(lon, lat):
        ax.plot(xr, yr, **kwargs)


def band_edges(centers: list[float], half: float = BAND_HALF) -> list[tuple[float, float]]:
    """Return (theta_lo, theta_hi) for each band center (±half about center)."""
    return [(c - half, c + half) for c in centers]


def mu_pair_for_band(lo: float, hi: float) -> tuple[float, float]:
    """Edge μ=cos(θ) for a scan-angle band (two boundary cones only)."""
    return float(np.cos(np.deg2rad(lo))), float(np.cos(np.deg2rad(hi)))
```

```
def migkas_scan_stops() -> dict:
    th_max = [float(t) for t in STRIP_THETA_MAX]
    th_min = [float(t) for t in STRIP_THETA_MIN]
    max_bands = band_edges(th_max)
    min_bands = band_edges(th_min)
    return {
        "theta_max_from_plus_x_deg": th_max,
        "theta_min_from_plus_x_deg": th_min,
        "max_bands_deg": max_bands,
        "min_bands_deg": min_bands,
        "band_half_width_deg": BAND_HALF,
        "band_full_width_deg": BAND_FULL,
        "band_total_width_deg": BAND_TOTAL,
        "band_width_note": "5% of 360° = 18° end-to-end = ±9° about each MAX/MIN center",
        "mu_max_centers": [float(np.cos(np.deg2rad(t))) for t in th_max],
        "mu_min_centers": [float(np.cos(np.deg2rad(t))) for t in th_min],
        "mu_max_bands": [mu_pair_for_band(lo, hi) for lo, hi in max_bands],
        "mu_min_bands": [mu_pair_for_band(lo, hi) for lo, hi in min_bands],
        "source": (
            "best_fit_2ell_multiplier.npz A_after extrema (two max, two min); "
            f"each drawn as one transparent {BAND_FULL:.0f}° end-to-end highlight "
            f"(±{BAND_HALF:.0f}° = 5% of 360°) + one solid mid-stroke"
        ),
    }


def compute_geometry(st: dict) -> dict:
    anti_l, anti_b = 0.0, 15.0
    coe_l, coe_b = 180.0, -15.0

    n_coe = lb_to_n(coe_l, coe_b)
    n_anti = lb_to_n(anti_l, anti_b)
    D = n_anti  # +D = anti-CoE = +x

    th_max = [float(t) for t in st["theta_max_from_plus_x_deg"]]
    th_min = [float(t) for t in st["theta_min_from_plus_x_deg"]]
    max_bands = list(st["max_bands_deg"])
    min_bands = list(st["min_bands_deg"])

    n_maxes = [lb_to_n(*lb) for lb in MIGKAS_MAXES]
    n_mins = [lb_to_n(*lb) for lb in MIGKAS_MINS]

    sep = float(
        np.rad2deg(
            np.arccos(np.clip(np.dot(n_maxes[0], n_mins[1]), -1.0, 1.0))
        )
    )

    return {
        "D": D,
        "n_coe": n_coe,
        "n_anti": n_anti,
```

```
        "n_maxes": n_maxes,
        "n_mins": n_mins,
        "anti_lb": (anti_l, anti_b),
        "coe_lb": (coe_l, coe_b),
        "theta_max_from_plus_x_deg": th_max,
        "theta_min_from_plus_x_deg": th_min,
        "max_bands_deg": max_bands,
        "min_bands_deg": min_bands,
        "mu_max_bands": list(st["mu_max_bands"]),
        "mu_min_bands": list(st["mu_min_bands"]),
        "sep_deg": sep,
        "band_half_width_deg": st["band_half_width_deg"],
    }


def draw_wire_sphere(ax, n_mer=12, n_par=6, **kwargs):
    u = np.linspace(0, 2 * np.pi, 60)
    v = np.linspace(-np.pi / 2, np.pi / 2, 40)
    xs = np.outer(np.cos(u), np.cos(v))
    ys = np.outer(np.sin(u), np.cos(v))
    zs = np.outer(np.ones_like(u), np.sin(v))
    ax.plot_surface(xs, ys, zs, color="#dfe6e9", alpha=0.15, linewidth=0, shade=False)
    kw = dict(color="#95a5a6", lw=0.4, alpha=0.45)
    kw.update(kwargs)
    for i in range(n_mer):
        phi = 2 * np.pi * i / n_mer
        t = np.linspace(-np.pi / 2, np.pi / 2, 80)
        ax.plot(np.cos(t) * np.cos(phi), np.cos(t) * np.sin(phi), np.sin(t), **kw)
    for j in range(1, n_par):
        b = -np.pi / 2 + np.pi * j / n_par
        t = np.linspace(0, 2 * np.pi, 80)
        r = np.cos(b)
        ax.plot(r * np.cos(t), r * np.sin(t), np.full_like(t, np.sin(b)), **kw)


def foreshortened_ellipse_xy(mu: float, n: int = 200):
    """Side-view foreshortened ellipse of a 3D ring at n·x = mu."""
    rho = np.sqrt(max(0.0, 1.0 - mu * mu))
    phi = np.linspace(0, 2 * np.pi, n)
    ex = mu + 0.045 * rho * np.cos(phi)
    ey = rho * np.sin(phi)
    return ex, ey, rho


def draw_panelA_ribbon(ax, theta_lo: float, theta_hi: float, theta_mid: float, color: str):
    """
    One Panel-A highlight: clean vertical strip between x=cos(θ_hi)..cos(θ_lo)
    clipped to the unit circle (uniform alpha), plus ONE solid foreshortened
    mid-ellipse at the exact center θ. No multi-stroke wash / hairball.
    """
    mu_lo = float(np.cos(np.deg2rad(theta_lo)))
    mu_hi = float(np.cos(np.deg2rad(theta_hi)))
    x0, x1 = (min(mu_lo, mu_hi), max(mu_lo, mu_hi))
    # Vertical strip polygon (tall enough to cover the circle; clipped below)
```

```
    strip = Polygon(
        np.array([[x0, -1.15], [x1, -1.15], [x1, 1.15], [x0, 1.15]]),
        closed=True,
        facecolor=color,
        edgecolor="none",
        alpha=0.28,
        zorder=2,
        linewidth=0,
    )
    ax.add_patch(strip)
    clip_circ = Circle((0.0, 0.0), 1.0, transform=ax.transData)
    strip.set_clip_path(clip_circ)

    mu_mid = float(np.cos(np.deg2rad(theta_mid)))
    ex_m, ey_m, _ = foreshortened_ellipse_xy(mu_mid, n=256)
    ax.plot(ex_m, ey_m, color=color, lw=2.8, alpha=0.95, zorder=4, solid_capstyle="round")


def draw_panelA_theta_rays(ax, geo):
    """
    Illustrate θ: from the observer (origin), one ray per loop out to the
    ring at that loop's center θ. θ is measured from +x (anti-CoE, right)
    up to the ring. Emphasize MAX θ=30° (thick semi-transparent red to the
    top of that foreshortened ring), matching Chris's markup.
    """
    items = list(zip(geo["theta_max_from_plus_x_deg"], COLORS_MAX)) + list(
        zip(geo["theta_min_from_plus_x_deg"], COLORS_MIN)
    )
    for mid, color in items:
        t = float(np.deg2rad(mid))
        # Endpoint = top/rim point of the ring in side view = (cos θ, sin θ)
        x1, y1 = float(np.cos(t)), float(np.sin(t))
        is_max30 = abs(mid - 29.5) < 0.5
        lw = 3.2 if is_max30 else 1.8
        alpha = 0.70 if is_max30 else 0.45
        ax.plot(
            [0.0, x1], [0.0, y1],
            color=color, lw=lw, alpha=alpha, zorder=3,
            solid_capstyle="round",
        )
        # Small tick at the ring hit
        ax.plot([x1], [y1], "o", color=color, ms=4.5 if is_max30 else 3.5,
                alpha=0.9, zorder=5, markeredgecolor="none")

    # Angle arc at θ=30° from +x axis up to the ray (makes the measure obvious)
    arc_r = 0.38
    phi = np.linspace(0.0, np.deg2rad(29.5), 60)
    ax.plot(
        arc_r * np.cos(phi), arc_r * np.sin(phi),
        color=C_MAX_30, lw=1.6, alpha=0.85, zorder=3,
    )
    ax.text(
        arc_r * np.cos(np.deg2rad(15.0)) + 0.06,
        arc_r * np.sin(np.deg2rad(15.0)) + 0.04,
```

```
        r"$\theta$",
        color=C_MAX_30, fontsize=10, fontweight="bold",
        ha="left", va="bottom", zorder=6,
    )


def panel_A_model_frame(ax, geo):
    """Side view: exactly four thick foreshortened loop ribbons about x."""
    ax.set_title(
        "A. Model frame (no Galactic yet)\n"
        r"axisymmetric about $x$ = CoE$\leftrightarrow$anti-CoE"
        "\n"
        r"two MAX + two MIN, each $18^\circ$ end-to-end ($\pm9^\circ$)",
        fontsize=10,
        pad=8,
    )
    th = np.linspace(0, 2 * np.pi, 400)
    ax.plot(np.cos(th), np.sin(th), color="#7f8c8d", lw=1.2, zorder=1)
    ax.plot(0, 0, "ko", ms=6, zorder=5)
    ax.text(0.08, -0.12, "observer", fontsize=8, color="k", ha="left")

    ax.annotate(
        "",
        xy=(1.15, 0),
        xytext=(-1.15, 0),
        arrowprops=dict(arrowstyle="<->", color=C_AXIS, lw=2.5),
        zorder=4,
    )
    ax.text(1.18, 0.08, "anti-CoE\n(+x)", color=C_ANTI, fontsize=8, ha="left", va="bottom")
    ax.text(-1.18, 0.08, "CoE\n(−x)", color=C_COE, fontsize=8, ha="right", va="bottom")

    for (lo, hi), mid, color in zip(
        geo["max_bands_deg"], geo["theta_max_from_plus_x_deg"], COLORS_MAX
    ):
        draw_panelA_ribbon(ax, lo, hi, mid, color)
    for (lo, hi), mid, color in zip(
        geo["min_bands_deg"], geo["theta_min_from_plus_x_deg"], COLORS_MIN
    ):
        draw_panelA_ribbon(ax, lo, hi, mid, color)

    # Illustrate θ from +x (anti-CoE) to each loop (Chris markup)
    draw_panelA_theta_rays(ax, geo)

    label_r = 1.30
    for mid, color in list(zip(geo["theta_max_from_plus_x_deg"], COLORS_MAX)) + list(
        zip(geo["theta_min_from_plus_x_deg"], COLORS_MIN)
    ):
        t = np.deg2rad(mid)
        ax.text(
            label_r * np.cos(t),
            label_r * np.sin(t),
            rf"$\theta\!=\!{mid:g}^\circ$",
            color=color, fontsize=7.5, ha="center", va="center", fontweight="bold",
        )
```

```
    ax.text(
        0.0,
        -1.38,
        (
            r"four transparent highlight bands about $x$  ·  each $18^\circ$ end-to-end ($\pm9^\circ$ = 5% of $360^\circ$)"
            + "\n"
            + r"(after-2$\ell$ $A(\ell)$: two MAX + two MIN; one mid-stroke each)"
            + "\n"
            + r"$y,z$ free (roll) — model does not pick celestial equator"
        ),
        fontsize=7.5,
        ha="center",
        va="top",
        style="italic",
        color="#34495e",
    )
    ax.set_xlim(-1.55, 1.55)
    ax.set_ylim(-1.55, 1.45)
    ax.set_aspect("equal")
    ax.axis("off")


def _ring_lb(D, mu, n_pts=721):
    pts = small_circle_about_axis(D, mu, n=n_pts)
    rl = np.array([n_to_lb(p)[0] for p in pts])
    rb = np.array([n_to_lb(p)[1] for p in pts])
    return rl, rb


def _ring_lonlat_rad(D, mu, n_pts=361):
    """Small-circle → Mollweide display (lon_disp, lat) in radians."""
    pts = small_circle_about_axis(D, float(mu), n=n_pts)
    ls = np.array([n_to_lb(p)[0] for p in pts])
    bs = np.array([n_to_lb(p)[1] for p in pts])
    lon = np.deg2rad(lon_disp_from_ell(ls))
    lat = np.deg2rad(bs)
    return lon, lat


def fill_mollweide_ribbon(ax, D, theta_lo, theta_hi, theta_mid, color, n_pts=721):
    """
    One Mollweide highlight band:
      continuous transparent polygon strips between ±9° edges
      (broken only at Mollweide antimeridian seams — NOT a dense quad wash)
      + ONE solid mid-stroke at θ_mid.
    """
    mu_a, mu_b = mu_pair_for_band(theta_lo, theta_hi)
    mu_mid = float(np.cos(np.deg2rad(theta_mid)))
    lon_a, lat_a = _ring_lonlat_rad(D, mu_a, n_pts=n_pts)
    lon_b, lat_b = _ring_lonlat_rad(D, mu_b, n_pts=n_pts)
    gap = 0.9 * np.pi
    # Mark steps that cross a seam on either edge (or between edges)
    bad = np.zeros(n_pts - 1, dtype=bool)
```

```
    for i in range(n_pts - 1):
        if abs(lon_a[i + 1] - lon_a[i]) > gap:
            bad[i] = True
        elif abs(lon_b[i + 1] - lon_b[i]) > gap:
            bad[i] = True
        elif abs(lon_a[i] - lon_b[i]) > gap:
            bad[i] = True
        elif abs(lon_a[i + 1] - lon_b[i + 1]) > gap:
            bad[i] = True
    # Group into continuous good runs → one polygon per run
    polys = []
    i = 0
    while i < n_pts - 1:
        while i < n_pts - 1 and bad[i]:
            i += 1
        if i >= n_pts - 1:
            break
        j = i
        while j < n_pts - 1 and not bad[j]:
            j += 1
        # indices i .. j inclusive are a continuous ribbon segment
        i0, i1 = i, j  # i1 is last good vertex index
        if i1 - i0 >= 1:
            xs = np.concatenate([lon_a[i0:i1 + 1], lon_b[i0:i1 + 1][::-1]])
            ys = np.concatenate([lat_a[i0:i1 + 1], lat_b[i0:i1 + 1][::-1]])
            polys.append(np.column_stack([xs, ys]))
        i = j + 1
    if polys:
        ax.add_collection(
            PolyCollection(
                polys, facecolors=color, edgecolors="none",
                alpha=0.26, zorder=2, linewidths=0,
            )
        )
    rl, rb = _ring_lb(D, mu_mid, n_pts=max(n_pts, 721))
    plot_lb_curve_mollweide(
        ax, rl, rb, color=color, lw=2.6, alpha=0.95, zorder=4
    )


def panel_B_mollweide(ax, geo):
    ax.set_title(
        r"B. Migkas $A(\ell)$ loops on sky (anti-CoE at $(0^\circ,+15^\circ)$)"
        "\n"
        r"(Galactic Mollweide; CoE axis tilted $+15^\circ$; $\ell$ increases LEFT; "
        r"two MAX + two MIN, each $18^\circ$ end-to-end ($\pm9^\circ$))",
        fontsize=10,
        pad=8,
    )
    for b in [-60, -30, 0, 30, 60]:
        ell = np.linspace(0, 360, 361)
        plot_lb_curve_mollweide(ax, ell, np.full_like(ell, b), color="#bdc3c7", lw=0.5, zorder=0)
    for ell0 in range(0, 360, 30):
        b = np.linspace(-90, 90, 181)
```

```
    plot_lb_curve_mollweide(ax, np.full_like(b, ell0), b, color="#bdc3c7", lw=0.5, zorder=0)

D = geo["D"]
ls, bs = great_circle_through_poles(D, n=721)
plot_lb_curve_mollweide(ax, ls, bs, color=C_AXIS, lw=2.0, zorder=2)

for (lo, hi), mid, color in zip(
    geo["max_bands_deg"], geo["theta_max_from_plus_x_deg"], COLORS_MAX
):
    fill_mollweide_ribbon(ax, D, lo, hi, mid, color)
for (lo, hi), mid, color in zip(
    geo["min_bands_deg"], geo["theta_min_from_plus_x_deg"], COLORS_MIN
):
    fill_mollweide_ribbon(ax, D, lo, hi, mid, color)

ell_eq, b_eq = celestial_equator_lb(721)
plot_lb_curve_mollweide(ax, ell_eq, b_eq, color=C_EQ, lw=1.8, ls="--", zorder=2)

ax.plot(
    np.deg2rad(lon_disp_from_ell(geo["anti_lb"][0])),
    np.deg2rad(geo["anti_lb"][1]),
    marker="o", ms=8, color=C_ANTI,
    markerfacecolor=C_ANTI, markeredgecolor="k", markeredgewidth=0.6,
    zorder=6, linestyle="None",
)
ax.plot(
    np.deg2rad(lon_disp_from_ell(geo["coe_lb"][0])),
    np.deg2rad(geo["coe_lb"][1]),
    marker="*", ms=14, color=C_COE,
    markerfacecolor=C_COE, markeredgecolor="k", markeredgewidth=0.6,
    zorder=6, linestyle="None",
)
for (l, b), color in zip(MIGKAS_MAXES, COLORS_MAX):
    ax.plot(
        np.deg2rad(lon_disp_from_ell(l)),
        np.deg2rad(b),
        marker="^",
        ms=10,
        color=color,
        markeredgecolor="k",
        markeredgewidth=0.7,
        zorder=7,
        linestyle="None",
    )
for (l, b), color in zip(MIGKAS_MINS, COLORS_MIN):
    ax.plot(
        np.deg2rad(lon_disp_from_ell(l)),
        np.deg2rad(b),
        marker="s",
        ms=10,
        color=color,
        markeredgecolor="k",
        markeredgewidth=0.7,
        zorder=7,
```

```
            linestyle="None",
        )

    xticks = np.deg2rad(lon_disp_from_ell(np.array([180, 90, 0, 270])))
    ax.set_xticks(xticks)
    ax.set_xticklabels([r"$180^\circ$", r"$90^\circ$", r"$0^\circ$", r"$270^\circ$"], fontsize=8)
    ax.set_yticks(np.deg2rad([-60, -30, 0, 30, 60]))
    ax.set_yticklabels([r"$-60^\circ$", r"$-30^\circ$", r"$0^\circ$", r"$+30^\circ$", r"$+60^\circ$"], fontsize=8)
    ax.grid(False)
    ax.set_xlabel(r"Galactic $\ell$ (increases LEFT)", fontsize=8)
    ax.set_ylabel(r"Galactic $b$", fontsize=8)


def fill_sphere_ribbon(ax, D, theta_lo, theta_hi, theta_mid, color, n_pts=160):
    """
    One 3D highlight belt: SINGLE transparent strip between ±9° edges
    + ONE solid mid-stroke. No linspace wash / stacked alpha hairball.
    """
    mu_a, mu_b = mu_pair_for_band(theta_lo, theta_hi)
    mu_mid = float(np.cos(np.deg2rad(theta_mid)))
    p0 = small_circle_about_axis(D, float(mu_a), n=n_pts)
    p1 = small_circle_about_axis(D, float(mu_b), n=n_pts)
    # Single belt between the two edge rings (one collection, no stacked wash)
    verts = [[p0[j], p0[j + 1], p1[j + 1], p1[j]] for j in range(n_pts - 1)]
    coll = Poly3DCollection(
        verts, facecolors=color, edgecolors="none", linewidths=0.0,
        alpha=0.25, zorder=2,
    )
    ax.add_collection3d(coll)
    pts = small_circle_about_axis(D, mu_mid, n=n_pts)
    ax.plot(
        pts[:, 0], pts[:, 1], pts[:, 2],
        color=color, lw=2.6, alpha=0.95, zorder=5,
    )


def panel_C_sphere(ax, geo):
    th_max = geo["theta_max_from_plus_x_deg"]
    th_min = geo["theta_min_from_plus_x_deg"]
    ax.set_title(
        r"C. Sphere 3D — two MAX + two MIN, each $18^\circ$ end-to-end ($\pm9^\circ$)"
        "\n"
        r"(axis line emphasized; celestial equator dashed)",
        fontsize=10,
        pad=6,
    )
    draw_wire_sphere(ax)
    D = geo["D"]
    ell_eq, b_eq = celestial_equator_lb(400)
    eq_xyz = np.array([lb_to_n(l, b) for l, b in zip(ell_eq, b_eq)])
    ax.plot(eq_xyz[:, 0], eq_xyz[:, 1], eq_xyz[:, 2], color=C_EQ, lw=1.8, ls="--", zorder=3)
    ax.plot(
        [-D[0], D[0]],
        [-D[1], D[1]],
```

```
        [-D[2], D[2]],
        color=C_AXIS,
        lw=3.0,
        zorder=5,
    )
    ax.scatter(*(-D), color=C_COE, s=120, marker="*", edgecolors="k", linewidths=0.6,
               depthshade=False, zorder=8)
    ax.scatter(*(D), color=C_ANTI, s=80, edgecolors="k", linewidths=0.6, depthshade=False, zorder=8)
    ax.text(*(-D * 1.22), "CoE", color=C_COE, fontsize=8, fontweight="bold")
    ax.text(*(D * 1.18), "anti-CoE", color=C_ANTI, fontsize=8, fontweight="bold")
    ax.scatter([0], [0], [0], color="k", s=25, depthshade=False, zorder=9)

    for (lo, hi), mid, color in zip(
        geo["max_bands_deg"], geo["theta_max_from_plus_x_deg"], COLORS_MAX
    ):
        fill_sphere_ribbon(ax, D, lo, hi, mid, color)
    for (lo, hi), mid, color in zip(
        geo["min_bands_deg"], geo["theta_min_from_plus_x_deg"], COLORS_MIN
    ):
        fill_sphere_ribbon(ax, D, lo, hi, mid, color)

    for n, color in zip(geo["n_maxes"], COLORS_MAX):
        ax.scatter(
            [n[0]], [n[1]], [n[2]],
            c=[color], marker="^", s=70,
            edgecolors="k", linewidths=0.7, depthshade=False, zorder=10,
        )
        ax.plot([0, n[0]], [0, n[1]], [0, n[2]], color=color, lw=0.8, ls=":", alpha=0.7)
    for n, color in zip(geo["n_mins"], COLORS_MIN):
        ax.scatter(
            [n[0]], [n[1]], [n[2]],
            c=[color], marker="s", s=60,
            edgecolors="k", linewidths=0.7, depthshade=False, zorder=10,
        )
        ax.plot([0, n[0]], [0, n[1]], [0, n[2]], color=color, lw=0.8, ls=":", alpha=0.7)

    ax.set_xlim(-1.25, 1.25)
    ax.set_ylim(-1.25, 1.25)
    ax.set_zlim(-1.25, 1.25)
    ax.set_box_aspect((1, 1, 1))
    ax.view_init(elev=18, azim=-55)
    ax.set_axis_off()
    ax.text2D(
        0.5,
        -0.02,
        rf"MAX $\theta\!=\!{th_max[0]:g}^\circ,{th_max[1]:g}^\circ$; "
        rf"MIN $\theta\!=\!{th_min[0]:g}^\circ,{th_min[1]:g}^\circ$  "
        rf"($18^\circ$ end-to-end = $\pm9^\circ$ = 5% of $360^\circ$)",
        transform=ax.transAxes,
        ha="center",
        fontsize=7.5,
        color="#34495e",
    )
```

```
def main():
    st = migkas_scan_stops()
    geo = compute_geometry(st)

    th_max = geo["theta_max_from_plus_x_deg"]
    th_min = geo["theta_min_from_plus_x_deg"]
    max_bands = geo["max_bands_deg"]
    min_bands = geo["min_bands_deg"]

    fig = plt.figure(figsize=(14.5, 5.6), dpi=160)
    fig.suptitle(
        r"Pure CoE axisymmetry: two MAX + two MIN highlight bands"
        "\n"
        rf"(each $18^\circ$ end-to-end = $\pm9^\circ$ = 5% of $360^\circ$; "
        rf"MAX $\theta\!=\!{th_max[0]:g}^\circ/{th_max[1]:g}^\circ$; "
        rf"MIN $\theta\!=\!{th_min[0]:g}^\circ/{th_min[1]:g}^\circ$; "
        r"anti-CoE $(0^\circ,+15^\circ)$, CoE axis tilted $+15^\circ$)",
        fontsize=11,
        fontweight="bold",
        y=1.02,
    )

    axA = fig.add_subplot(1, 3, 1)
    panel_A_model_frame(axA, geo)

    axB = fig.add_subplot(1, 3, 2, projection="mollweide")
    panel_B_mollweide(axB, geo)

    axC = fig.add_subplot(1, 3, 3, projection="3d")
    panel_C_sphere(axC, geo)

    handles = [
        Line2D([0], [0], color=C_AXIS, lw=2.5, label=r"CoE$\leftrightarrow$anti-CoE axis"),
        Patch(
            facecolor=C_MAX_30, alpha=0.45, edgecolor=C_MAX_30,
            label=(
                rf"MAX $\theta\!=\!{th_max[0]:g}^\circ$ band "
                rf"[{max_bands[0][0]:g},{max_bands[0][1]:g}]$^\circ$ "
                rf"($18^\circ$=±$9^\circ$=5% of $360^\circ$)"
            ),
        ),
        Patch(
            facecolor=C_MAX_185, alpha=0.45, edgecolor=C_MAX_185,
            label=(
                rf"MAX $\theta\!=\!{th_max[1]:g}^\circ$ band "
                rf"[{max_bands[1][0]:g},{max_bands[1][1]:g}]$^\circ$ "
                rf"($18^\circ$=±$9^\circ$)"
            ),
        ),
        Patch(
            facecolor=C_MIN_115, alpha=0.45, edgecolor=C_MIN_115,
            label=(
                rf"MIN $\theta\!=\!{th_min[0]:g}^\circ$ band "
```

```
            rf"[{min_bands[0][0]:g},{min_bands[0][1]:g}]$^\circ$ "
            rf"($18^\circ$=±$9^\circ$)"
        ),
    ),
    Patch(
        facecolor=C_MIN_265, alpha=0.45, edgecolor=C_MIN_265,
        label=(
            rf"MIN $\theta\!=\!{th_min[1]:g}^\circ$ band "
            rf"[{min_bands[1][0]:g},{min_bands[1][1]:g}]$^\circ$ "
            rf"($18^\circ$=±$9^\circ$)"
        ),
    ),
    Line2D([0], [0], color=C_EQ, lw=1.8, ls="--", label="celestial equator (Dec$=0$)"),
    Line2D(
        [0], [0], marker="^", color="w", markerfacecolor=C_MAX_30, markeredgecolor="k",
        ms=9,
        label=rf"MAX $({MIGKAS_MAXES[0][0]:.0f}^\circ,{MIGKAS_MAXES[0][1]:+.0f}^\circ)$",
    ),
    Line2D(
        [0], [0], marker="^", color="w", markerfacecolor=C_MAX_185, markeredgecolor="k",
        ms=9,
        label=rf"MAX $({MIGKAS_MAXES[1][0]:.0f}^\circ,{MIGKAS_MAXES[1][1]:+.0f}^\circ)$",
    ),
    Line2D(
        [0], [0], marker="s", color="w", markerfacecolor=C_MIN_115, markeredgecolor="k",
        ms=8,
        label=rf"MIN $({MIGKAS_MINS[0][0]:.0f}^\circ,{MIGKAS_MINS[0][1]:+.0f}^\circ)$",
    ),
    Line2D(
        [0], [0], marker="s", color="w", markerfacecolor=C_MIN_265, markeredgecolor="k",
        ms=8,
        label=rf"MIN $({MIGKAS_MINS[1][0]:.0f}^\circ,{MIGKAS_MINS[1][1]:+.0f}^\circ)$",
    ),
    Line2D(
        [0], [0], marker="o", color="w", markerfacecolor=C_ANTI, markeredgecolor="k",
        ms=8, label=rf"anti-CoE $({geo['anti_lb'][0]:.1f}^\circ,{geo['anti_lb'][1]:+.1f}^\circ)$",
    ),
    Line2D(
        [0], [0], marker="*", color="w", markerfacecolor=C_COE, markeredgecolor="k",
        ms=11, label=rf"CoE $({geo['coe_lb'][0]:.1f}^\circ,{geo['coe_lb'][1]:+.1f}^\circ)$",
    ),
]
fig.legend(
    handles=handles,
    loc="lower center",
    ncol=6,
    fontsize=7.0,
    frameon=True,
    fancybox=True,
    bbox_to_anchor=(0.5, -0.10),
)

fig.tight_layout()
# --- Chris 2026-09-26: panel C sphere drawn at the same on-page diameter as
```

```
# panel A's model-frame circle; panel C axes matched to A's vertical
# position/height. Nothing else changes.
from mpl_toolkits.mplot3d import proj3d
pa, pc = axA.get_position(), axC.get_position()
axC.set_position([pc.x0, pa.y0, pc.width, pa.height])

def _diam_A():
    fig.canvas.draw()
    p0 = axA.transData.transform((-1.0, 0.0)); p1 = axA.transData.transform((1.0, 0.0))
    return float(p1[0] - p0[0])

def _diam_C():
    fig.canvas.draw()
    u = np.random.default_rng(0).normal(size=(20000, 3))
    u /= np.linalg.norm(u, axis=1)[:, None]
    xs, ys, _ = proj3d.proj_transform(u[:, 0], u[:, 1], u[:, 2], axC.get_proj())
    d = axC.transData.transform(np.column_stack([xs, ys]))
    return float(d[:, 0].max() - d[:, 0].min()), float(d[:, 1].max() - d[:, 1].min())

zoom = 1.0
for _ in range(8):
    dA = _diam_A(); dCx, dCy = _diam_C()
    zoom *= dA / dCx
    axC.set_box_aspect((1, 1, 1), zoom=zoom)
dA = _diam_A(); dCx, dCy = _diam_C()
print(f"PANEL_DIAM A={dA:.1f}px C={dCx:.1f}x{dCy:.1f}px zoom={zoom:.3f}")
fig.savefig(PNG_PATH, bbox_inches="tight", facecolor="white")
plt.close(fig)

payload = {
    "png": str(PNG_PATH),
    "script": str(OUT_DIR / "make_coe_rings_migkas_pin_axis_matched.py"),
    "migkas_max_lb_deg": [list(lb) for lb in MIGKAS_MAXES],
    "migkas_min_lb_deg": [list(lb) for lb in MIGKAS_MINS],
    "strip_theta_max_deg": list(STRIP_THETA_MAX),
    "strip_theta_min_deg": list(STRIP_THETA_MIN),
    "band_half_width_deg": BAND_HALF,
    "band_full_width_deg": BAND_FULL,
    "band_total_width_deg": BAND_TOTAL,
    "band_width_note": "5% of 360° = 18° end-to-end = ±9° about each MAX/MIN center",
    "max_bands_deg": [list(b) for b in max_bands],
    "min_bands_deg": [list(b) for b in min_bands],
    "band_intervals": {
        "MAX_30": [float(max_bands[0][0]), float(max_bands[0][1])],
        "MAX_185": [float(max_bands[1][0]), float(max_bands[1][1])],
        "MIN_115": [float(min_bands[0][0]), float(min_bands[0][1])],
        "MIN_265": [float(min_bands[1][0]), float(min_bands[1][1])],
    },
    "scan_stop_source": st["source"],
    "theta_max_from_plus_x_deg": [round(t, 4) for t in th_max],
    "theta_min_from_plus_x_deg": [round(t, 4) for t in th_min],
    "mu_max_bands": [[round(a, 8), round(b, 8)] for a, b in geo["mu_max_bands"]],
    "mu_min_bands": [[round(a, 8), round(b, 8)] for a, b in geo["mu_min_bands"]],
    "CoE_lb_deg": [round(geo["coe_lb"][0], 4), round(geo["coe_lb"][1], 4)],
```

```
        "anti_CoE_lb_deg": [round(geo["anti_lb"][0], 4), round(geo["anti_lb"][1], 4)],
        "D_hat_galactic_xyz": geo["D"].tolist(),
        "angular_sep_migkas_max0_min1_deg": round(geo["sep_deg"], 6),
        "axis_tilt_b_deg": 15,
        "axis_tilt_reason": (
            "Chris 2026-09-22 — markers fixed, rings rotate up 30°"
        ),
        "orientation": (
            "Galactic orientation: anti-CoE at (ell=0°, b=+15°), CoE at "
            "(ell=180°, b=−15°); +D = anti-CoE = +x; axis tilted +15° "
            "(Chris 2026-09-22 — markers fixed, rings rotate up 30°)"
        ),
        "drawing": (
            "Exactly 4 features. Each = ONE smooth transparent highlight band "
            "between θ−9° and θ+9° (18° end-to-end = 5% of 360°) + ONE solid "
            "mid-stroke at the exact center θ. No multi-stroke wash / hairball. "
            "Four distinct colors (two reds + two blues); no orange; no 283° "
            "companion. Panel A: clean vertical strip clipped to unit circle; theta rays from observer to each loop (emphasized at MAX 30°). Mollweide
"
            "seam-split honestly. Sphere: single Poly3D strip (no n_wash stack)."
        ),
        "note": (
            "Stops are after-2ℓ A(ℓ) extrema from best_fit_2ell_multiplier.npz "
            f"A_after: two MAX at θ={th_max[0]:g}°/{th_max[1]:g}° and "
            f"two MIN at θ={th_min[0]:g}°/{th_min[1]:g}°, each drawn as "
            f"one {BAND_FULL:.0f}° end-to-end highlight (±{BAND_HALF:.0f}° = 5% of 360°). "
            "Rings use μ=cos(θ); sky markers are map extrema (17°,−9°) and "
            "(274°,−22°) plus strip stand-ins (185°,0°) and (115°,0°)."
        ),
        "plot_convention": "Migkas Galactic Mollweide: lon_disp=-ell_wrapped (ell increases LEFT)",
        "legend_summary": (
            "two MAX + two MIN highlight bands, each 18° end-to-end "
            "(±9° = 5% of 360°); four distinct colors"
        ),
        "colors": {
            "MAX_30": C_MAX_30,
            "MAX_185": C_MAX_185,
            "MIN_115": C_MIN_115,
            "MIN_265": C_MIN_265,
            "no_orange": True,
        },
    }
    JSON_PATH.write_text(json.dumps(payload, indent=2) + "\n")
    print(json.dumps(payload, indent=2))
    print(f"\nWrote {PNG_PATH}")
    print(f"Wrote {JSON_PATH}")


if __name__ == "__main__":
    main()
```

PANEL_DIAM A=367.9px C=367.9x368.0px zoom=1.399

```
{
  "png": "/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/loop_threepanel_matched.png",
  "script": "/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/make_coe_rings_migkas_pin_axis_matched.py",
  "migkas_max_lb_deg": [
    [
      17.0,
      -9.0
    ],
    [
      185.0,
      0.0
    ]
  ],
  "migkas_min_lb_deg": [
    [
      115.0,
      0.0
    ],
    [
      274.0,
      -22.0
    ]
  ],
  "strip_theta_max_deg": [
    29.5,
    187.5
  ],
  "strip_theta_min_deg": [
    122.6,
    264.4
  ],
  "band_half_width_deg": 9.0,
  "band_full_width_deg": 18.0,
  "band_total_width_deg": 18.0,
  "band_width_note": "5% of 360\u00b0 = 18\u00b0 end-to-end = \u00b19\u00b0 about each MAX/MIN center",
  "max_bands_deg": [
    [
      20.5,
      38.5
    ],
    [
      178.5,
      196.5
    ]
  ],
  "min_bands_deg": [
    [
      113.6,
      131.6
    ],
    [
      255.39999999999998,
      273.4
    ]
  ],
  "band_intervals": {
    "MAX_30": [
```

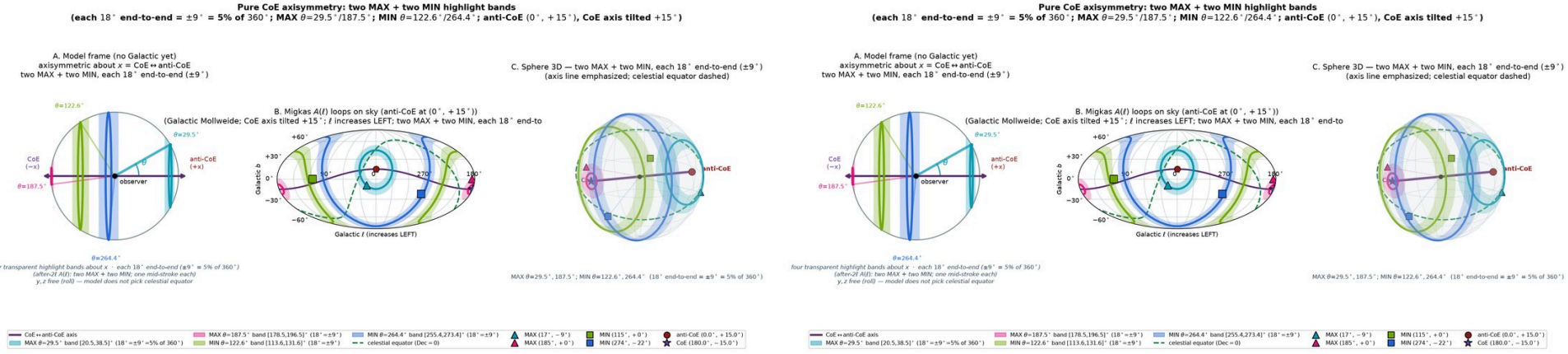
```
      20.5,
      38.5
    ],
    "MAX_185": [
      178.5,
      196.5
    ],
    "MIN_115": [
      113.6,
      131.6
    ],
    "MIN_265": [
      255.39999999999998,
      273.4
    ]
  },
  "scan_stop_source": "best_fit_2ell_multiplier.npz A_after extrema (two max, two min); each drawn as one transparent 18\u00b0 end-to-end highlight
(\u00b19\u00b0 = 5% of 360\u00b0) + one solid mid-stroke",
  "theta_max_from_plus_x_deg": [
    29.5,
    187.5
  ],
  "theta_min_from_plus_x_deg": [
    122.6,
    264.4
  ],
  "mu_max_bands": [
    [
      0.93667219,
      0.78260816
    ],
    [
      -0.99965732,
      -0.95881973
    ]
  ],
  "mu_min_bands": [
    [
      -0.40034903,
      -0.66392621
    ],
    [
      -0.25206936,
      0.05930637
    ]
  ],
  "CoE_lb_deg": [
    180.0,
    -15.0
  ],
  "anti_CoE_lb_deg": [
    0.0,
    15.0
  ],
  "D_hat_galactic_xyz": [
    0.9659258262890683,
    0.0,
```

```
    0.25881904510252074
  ],
  "angular_sep_migkas_max0_min1_deg": 98.476384,
  "axis_tilt_b_deg": 15,
  "axis_tilt_reason": "Chris 2026-09-22 \u2014 markers fixed, rings rotate up 30\u00b0",
  "orientation": "Galactic orientation: anti-CoE at (ell=0\u00b0, b=+15\u00b0), CoE at (ell=180\u00b0, b=\u221215\u00b0); +D = anti-CoE = +x; axis
tilted +15\u00b0 (Chris 2026-09-22 \u2014 markers fixed, rings rotate up 30\u00b0)",
  "drawing": "Exactly 4 features. Each = ONE smooth transparent highlight band between \u03b8\u22129\u00b0 and \u03b8+9\u00b0 (18\u00b0 end-to-end = 5%
of 360\u00b0) + ONE solid mid-stroke at the exact center \u03b8. No multi-stroke wash / hairball. Four distinct colors (two reds + two blues); no
orange; no 283\u00b0 companion. Panel A: clean vertical strip clipped to unit circle; theta rays from observer to each loop (emphasized at MAX
30\u00b0). Mollweide seam-split honestly. Sphere: single Poly3D strip (no n_wash stack).",
  "note": "Stops are after-2\u2113 A(\u2113) extrema from best_fit_2ell_multiplier.npz A_after: two MAX at \u03b8=29.5\u00b0/187.5\u00b0 and two MIN at
\u03b8=122.6\u00b0/264.4\u00b0, each drawn as one 18\u00b0 end-to-end highlight (\u00b19\u00b0 = 5% of 360\u00b0). Rings use \u03bc=cos(\u03b8); sky
markers are map extrema (17\u00b0,\u22129\u00b0) and (274\u00b0,\u221222\u00b0) plus strip stand-ins (185\u00b0,0\u00b0) and (115\u00b0,0\u00b0).",
  "plot_convention": "Migkas Galactic Mollweide: lon_disp=-ell_wrapped (ell increases LEFT)",
  "legend_summary": "two MAX + two MIN highlight bands, each 18\u00b0 end-to-end (\u00b19\u00b0 = 5% of 360\u00b0); four distinct colors",
  "colors": {
    "MAX_30": "#0097b2",
    "MAX_185": "#e0107a",
    "MIN_115": "#6fa800",
    "MIN_265": "#2166d4",
    "no_orange": true
  }
}

Wrote /workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/loop_threepanel_matched.png
Wrote /workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/loop_threepanel_matched.json
```


```
compare(FIGS / 'loop_threepanel_matched.png', '/workspace/cosmology/papers/spreadsheet_model/loop_threepanel_matched.png')
```

```
loop_threepanel_matched.png: new (2262, 1027) vs ref (2262, 1027) | mean |diff| = 0.000/255, pixels differing >40: 0.000% -> identical
left: approved reference papers/spreadsheet_model/loop_threepanel_matched.png   |   right: notebook re-run steps_v3/figs/loop_threepanel_matched.png
```



Step 16b — Matched three-panel with the opacity gradient

What it shows. Step 16a with each loop fading from its θ-ray point, $\alpha \propto \frac{1}{2}(1+\cos\psi)$, in Panels B and C.

Source script: `/workspace/cosmology/figures/make_coe_rings_migkas_pin_axis_matched_grad.py` (last modified 2026-09-26 22:54 CT), run verbatim. *Output-path edit:* `/workspace/cosmology/papers/spreadsheet_model/loop_threepanel_matched_` → `/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/loop_threep`. *Reference image:* `papers/spreadsheet_model/loop_threepanel_matched_grad.png`.

```
%%runscript make_coe_rings_migkas_pin_axis_matched_grad.py
#!/usr/bin/env python3
"""
CoE rings + Migkas A(ell) extrema loops in the Galactic frame.

Concept:
  Pure CoE axisymmetry → constant A on small-circle rings about x = CoE↔anti-CoE.
  Scan A vs angle from +x (anti-CoE) around 360°. After-2ℓ A(ℓ) has TWO maxima
  and TWO minima (from best_fit_2ell_multiplier.npz A_after):
    MAX at θ = 30° and 185° from +x,
    MIN at θ = 115° and 265° from +x.

  Exactly FOUR loops — each ONE transparent highlight band, 18° end-to-end
  (= ±9° = 5% of 360°) about the center θ:
    MAX: 30±9 → [21°, 39°] and 185±9 → [176°, 194°]
    MIN: 115±9 → [106°, 124°] and 265±9 → [256°, 274°]
  Drawing: ONE smooth transparent filled ribbon between θ−9° and θ+9°
  + ONE solid mid-stroke at the exact center θ. No multi-stroke wash,
  no dense linspace hairball, no dual strong boundaries.
  Four distinct colors (two reds + two blues). No orange, no 283° companion.

  Galactic orientation: anti-CoE (+x) at ℓ=0°, b=+15°; CoE (−x) at ℓ=180°, b=−15°
  (axis tilted +15° so features near ℓ=0 move north on Mollweide; Migkas markers fixed).
  Sky markers (ℓ, b):
    Maxes: map extrema (17°, −9°) for ~30° max; strip (185°, 0°) for second max.
    Mins:  strip (115°, 0°) for ~115° min; map extrema (274°, −22°) for ~265° min.

Outputs:
  coe_rings_migkas_pin_axis.png
  coe_rings_migkas_pin_axis.json
"""

from __future__ import annotations

import json
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
from matplotlib.lines import Line2D
from matplotlib.collections import PolyCollection
from matplotlib.patches import Circle, Patch, Polygon
from mpl_toolkits.mplot3d import Axes3D  # noqa: F401
from mpl_toolkits.mplot3d.art3d import Poly3DCollection

OUT_DIR = Path(__file__).resolve().parent
```

```
PNG_PATH = Path("/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/loop_threepanel_matched_grad.png")
JSON_PATH = Path("/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/loop_threepanel_matched_grad.json")

# 5% of 360° = 18° end-to-end → half-width ±9° about each center
BAND_HALF = 9.0
BAND_FULL = 2.0 * BAND_HALF  # 18° = 5% of 360°
BAND_TOTAL = BAND_FULL  # alias
STRIP_THETA_MAX = [29.5, 187.5]
STRIP_THETA_MIN = [122.6, 264.4]

MIGKAS_MAXES = [(17.0, -9.0), (185.0, 0.0)]
MIGKAS_MINS = [(115.0, 0.0), (274.0, -22.0)]

C_COE = "#6b3fa0"
C_ANTI = "#8e1b1b"  # dark red marker; distinct from loop reds
# Four distinct loop colors: two reds (MAX) + two blues (MIN); no orange/yellow
C_MAX_30 = "#0097b2"    # cyan — MAX at 29.5°
C_MAX_185 = "#e0107a"   # magenta — MAX at 187.5°
C_MIN_115 = "#6fa800"   # green — MIN at 122.6°
C_MIN_265 = "#2166d4"   # blue — MIN at 264.4°
C_AXIS = "#5b2c6f"
C_EQ = "#1e8449"

# Per-loop color maps (order matches STRIP_THETA_MAX / STRIP_THETA_MIN)
COLORS_MAX = [C_MAX_30, C_MAX_185]
COLORS_MIN = [C_MIN_115, C_MIN_265]


def wrap_lon_deg(lon: np.ndarray) -> np.ndarray:
    return ((np.asarray(lon, dtype=float) + 180.0) % 360.0) - 180.0


def lon_disp_from_ell(ell_deg: np.ndarray) -> np.ndarray:
    """Migkas Galactic: ℓ increases LEFT → lon_disp = −ℓ_wrapped."""
    return -wrap_lon_deg(ell_deg)


def lb_to_n(l_deg: float, b_deg: float) -> np.ndarray:
    l = np.deg2rad(l_deg)
    b = np.deg2rad(b_deg)
    return np.array(
        [np.cos(b) * np.cos(l), np.cos(b) * np.sin(l), np.sin(b)], dtype=float
    )


def n_to_lb(n: np.ndarray) -> tuple[float, float]:
    n = np.asarray(n, dtype=float)
    n = n / np.linalg.norm(n)
    b = float(np.rad2deg(np.arcsin(np.clip(n[2], -1.0, 1.0))))
    l = float(np.rad2deg(np.arctan2(n[1], n[0])) % 360.0)
    return l, b


def celestial_equator_lb(n: int = 720) -> tuple[np.ndarray, np.ndarray]:
```

```
    """Dec=0 (ICRS) as Galactic (ℓ, b). IAU J2000 NGP convention, no astropy."""
    ra_ngp = np.deg2rad(192.85948)
    dec_ngp = np.deg2rad(27.12825)
    l_ncp = np.deg2rad(122.93192)
    ra = np.linspace(0.0, 2.0 * np.pi, n, endpoint=False)
    dec = 0.0
    sin_b = np.sin(dec) * np.sin(dec_ngp) + np.cos(dec) * np.cos(dec_ngp) * np.cos(ra - ra_ngp)
    sin_b = np.clip(sin_b, -1.0, 1.0)
    b = np.arcsin(sin_b)
    y = np.cos(dec) * np.sin(ra - ra_ngp)
    x = np.sin(dec) * np.cos(dec_ngp) - np.cos(dec) * np.sin(dec_ngp) * np.cos(ra - ra_ngp)
    l = (l_ncp - np.arctan2(y, x)) % (2.0 * np.pi)
    return np.rad2deg(l), np.rad2deg(b)


def great_circle_through_poles(D: np.ndarray, n: int = 361) -> tuple[np.ndarray, np.ndarray]:
    """Great circle through ±D (any meridian of the CoE axis)."""
    a = D / np.linalg.norm(D)
    tmp = np.array([0.0, 0.0, 1.0])
    if abs(np.dot(a, tmp)) > 0.9:
        tmp = np.array([1.0, 0.0, 0.0])
    e2 = np.cross(a, tmp)
    e2 /= np.linalg.norm(e2)
    t = np.linspace(0.0, 2.0 * np.pi, n)
    pts = np.outer(np.cos(t), a) + np.outer(np.sin(t), e2)
    ls = np.array([n_to_lb(p)[0] for p in pts])
    bs = np.array([n_to_lb(p)[1] for p in pts])
    return ls, bs


def small_circle_about_axis(axis: np.ndarray, mu: float, n: int = 361) -> np.ndarray:
    """Unit vectors on the small circle n·axis = mu (|mu|<1)."""
    a = axis / np.linalg.norm(axis)
    mu = float(np.clip(mu, -1.0 + 1e-9, 1.0 - 1e-9))
    tmp = np.array([0.0, 0.0, 1.0]) if abs(a[2]) < 0.9 else np.array([1.0, 0.0, 0.0])
    e1 = np.cross(a, tmp)
    e1 /= np.linalg.norm(e1)
    e2 = np.cross(a, e1)
    rho = np.sqrt(max(0.0, 1.0 - mu * mu))
    t = np.linspace(0.0, 2.0 * np.pi, n)
    return (mu * a)[None, :] + rho * (np.outer(np.cos(t), e1) + np.outer(np.sin(t), e2))


def split_lon_segments(lon_rad: np.ndarray, lat_rad: np.ndarray, gap: float = np.pi):
    """Split polyline where Mollweide lon jumps (wrap discontinuity)."""
    d = np.diff(lon_rad)
    cuts = np.where(np.abs(d) > gap)[0] + 1
    idx = np.concatenate([[0], cuts, [len(lon_rad)]])
    segs = []
    for i0, i1 in zip(idx[:-1], idx[1:]):
        if i1 - i0 >= 2:
            segs.append((lon_rad[i0:i1], lat_rad[i0:i1]))
    return segs
```

```
def plot_lb_curve_mollweide(ax, l_deg, b_deg, **kwargs):
    lon = np.deg2rad(lon_disp_from_ell(l_deg))
    lat = np.deg2rad(b_deg)
    for xr, yr in split_lon_segments(lon, lat):
        ax.plot(xr, yr, **kwargs)


def band_edges(centers: list[float], half: float = BAND_HALF) -> list[tuple[float, float]]:
    """Return (theta_lo, theta_hi) for each band center (±half about center)."""
    return [(c - half, c + half) for c in centers]


def mu_pair_for_band(lo: float, hi: float) -> tuple[float, float]:
    """Edge μ=cos(θ) for a scan-angle band (two boundary cones only)."""
    return float(np.cos(np.deg2rad(lo))), float(np.cos(np.deg2rad(hi)))


def migkas_scan_stops() -> dict:
    th_max = [float(t) for t in STRIP_THETA_MAX]
    th_min = [float(t) for t in STRIP_THETA_MIN]
    max_bands = band_edges(th_max)
    min_bands = band_edges(th_min)
    return {
        "theta_max_from_plus_x_deg": th_max,
        "theta_min_from_plus_x_deg": th_min,
        "max_bands_deg": max_bands,
        "min_bands_deg": min_bands,
        "band_half_width_deg": BAND_HALF,
        "band_full_width_deg": BAND_FULL,
        "band_total_width_deg": BAND_TOTAL,
        "band_width_note": "5% of 360° = 18° end-to-end = ±9° about each MAX/MIN center",
        "mu_max_centers": [float(np.cos(np.deg2rad(t))) for t in th_max],
        "mu_min_centers": [float(np.cos(np.deg2rad(t))) for t in th_min],
        "mu_max_bands": [mu_pair_for_band(lo, hi) for lo, hi in max_bands],
        "mu_min_bands": [mu_pair_for_band(lo, hi) for lo, hi in min_bands],
        "source": (
            "best_fit_2ell_multiplier.npz A_after extrema (two max, two min); "
            f"each drawn as one transparent {BAND_FULL:.0f}° end-to-end highlight "
            f"(±{BAND_HALF:.0f}° = 5% of 360°) + one solid mid-stroke"
        ),
    }


def compute_geometry(st: dict) -> dict:
    anti_l, anti_b = 0.0, 15.0
    coe_l, coe_b = 180.0, -15.0

    n_coe = lb_to_n(coe_l, coe_b)
    n_anti = lb_to_n(anti_l, anti_b)
    D = n_anti  # +D = anti-CoE = +x

    th_max = [float(t) for t in st["theta_max_from_plus_x_deg"]]
    th_min = [float(t) for t in st["theta_min_from_plus_x_deg"]]
```

```
    max_bands = list(st["max_bands_deg"])
    min_bands = list(st["min_bands_deg"])

    n_maxes = [lb_to_n(*lb) for lb in MIGKAS_MAXES]
    n_mins = [lb_to_n(*lb) for lb in MIGKAS_MINS]

    sep = float(
        np.rad2deg(
            np.arccos(np.clip(np.dot(n_maxes[0], n_mins[1]), -1.0, 1.0))
        )
    )

    return {
        "D": D,
        "n_coe": n_coe,
        "n_anti": n_anti,
        "n_maxes": n_maxes,
        "n_mins": n_mins,
        "anti_lb": (anti_l, anti_b),
        "coe_lb": (coe_l, coe_b),
        "theta_max_from_plus_x_deg": th_max,
        "theta_min_from_plus_x_deg": th_min,
        "max_bands_deg": max_bands,
        "min_bands_deg": min_bands,
        "mu_max_bands": list(st["mu_max_bands"]),
        "mu_min_bands": list(st["mu_min_bands"]),
        "sep_deg": sep,
        "band_half_width_deg": st["band_half_width_deg"],
    }


def draw_wire_sphere(ax, n_mer=12, n_par=6, **kwargs):
    u = np.linspace(0, 2 * np.pi, 60)
    v = np.linspace(-np.pi / 2, np.pi / 2, 40)
    xs = np.outer(np.cos(u), np.cos(v))
    ys = np.outer(np.sin(u), np.cos(v))
    zs = np.outer(np.ones_like(u), np.sin(v))
    ax.plot_surface(xs, ys, zs, color="#dfe6e9", alpha=0.15, linewidth=0, shade=False)
    kw = dict(color="#95a5a6", lw=0.4, alpha=0.45)
    kw.update(kwargs)
    for i in range(n_mer):
        phi = 2 * np.pi * i / n_mer
        t = np.linspace(-np.pi / 2, np.pi / 2, 80)
        ax.plot(np.cos(t) * np.cos(phi), np.cos(t) * np.sin(phi), np.sin(t), **kw)
    for j in range(1, n_par):
        b = -np.pi / 2 + np.pi * j / n_par
        t = np.linspace(0, 2 * np.pi, 80)
        r = np.cos(b)
        ax.plot(r * np.cos(t), r * np.sin(t), np.full_like(t, np.sin(b)), **kw)


def foreshortened_ellipse_xy(mu: float, n: int = 200):
    """Side-view foreshortened ellipse of a 3D ring at n·x = mu."""
    rho = np.sqrt(max(0.0, 1.0 - mu * mu))
```

```
    phi = np.linspace(0, 2 * np.pi, n)
    ex = mu + 0.045 * rho * np.cos(phi)
    ey = rho * np.sin(phi)
    return ex, ey, rho



# ---------------------------------------------------------------------------
# Opacity gradient along each loop (Chris 2026-09-26), same method as
# make_coe_rings_gradient_loops.py: loop drawn as runs of short elements with
# per-element alpha = base_alpha * (1 + cos psi)/2, psi = ring azimuth from the
# θ-ray point (ring point most along sign(sin θ_mid)·Ŷ; Ŷ = Galactic +z
# projected ⊥ D, i.e. model-frame "up"; no roll). Darkest at the θ-ray point,
# fully transparent at the diametrically opposite point.
# ---------------------------------------------------------------------------
from matplotlib.collections import LineCollection
from matplotlib.colors import to_rgba
from mpl_toolkits.mplot3d.art3d import Line3DCollection


def theta_ray_sign(theta_mid_deg):
    return 1.0 if np.sin(np.deg2rad(float(theta_mid_deg))) >= 0 else -1.0


def ring_basis(D):
    a = np.asarray(D, dtype=float)
    a = a / np.linalg.norm(a)
    tmp = np.array([0.0, 0.0, 1.0])
    if abs(np.dot(a, tmp)) > 0.9:
        tmp = np.array([0.0, 1.0, 0.0])
    y = tmp - np.dot(tmp, a) * a
    y /= np.linalg.norm(y)
    z = np.cross(a, y)
    z /= np.linalg.norm(z)
    return y, z


def grad_phi(theta_mid_deg, n):
    """phi from antipode of θ-ray point round to antipode (psi = -180..+180)."""
    phi0 = 0.5 * np.pi if theta_ray_sign(theta_mid_deg) > 0 else 1.5 * np.pi
    return phi0, phi0 + np.linspace(-np.pi, np.pi, n)


def grad_scale(phi, phi0):
    return 0.5 * (1.0 + np.cos(np.asarray(phi) - phi0))


def small_circle_grad(D, mu, theta_mid_deg, n=721):
    a = np.asarray(D, dtype=float)
    a = a / np.linalg.norm(a)
    mu = float(np.clip(mu, -1.0 + 1e-9, 1.0 - 1e-9))
    rho = np.sqrt(max(0.0, 1.0 - mu * mu))
    y, z = ring_basis(a)
    phi0, phi = grad_phi(theta_mid_deg, n)
```

```
    pts = (mu * a)[None, :] + rho * (np.outer(np.sin(phi), y) + np.outer(np.cos(phi), z))
    return pts, grad_scale(phi, phi0)


def segment_scale(sc):
    sc = np.asarray(sc, dtype=float)
    return 0.5 * (sc[:-1] + sc[1:])


def rgba_array(color, alphas):
    r, g, b, _ = to_rgba(color)
    al = np.clip(np.asarray(alphas, dtype=float), 0.0, 1.0)
    out = np.empty((al.size, 4))
    out[:, 0], out[:, 1], out[:, 2], out[:, 3] = r, g, b, al
    return out


def quantized_runs(seg_sc, valid=None, n_levels=64):
    s = np.asarray(seg_sc, dtype=float)
    lev = np.round(np.clip(s, 0.0, 1.0) * n_levels) / n_levels
    ok = lev > 0 if valid is None else (lev > 0) & np.asarray(valid, dtype=bool)
    runs, i, n = [], 0, s.size
    while i < n:
        if not ok[i]:
            i += 1
            continue
        j = i + 1
        while j < n and ok[j] and lev[j] == lev[i]:
            j += 1
        runs.append((i, j, float(lev[i])))
        i = j
    return runs


def _strip_polys_2d(xa, ya, xb, yb, runs):
    return [
        np.column_stack([
            np.concatenate([xa[i0:i1 + 1], xb[i0:i1 + 1][::-1]]),
            np.concatenate([ya[i0:i1 + 1], yb[i0:i1 + 1][::-1]]),
        ])
        for i0, i1, _ in runs
    ]


def _polylines_2d(x, y, runs):
    return [np.column_stack([x[i0:i1 + 1], y[i0:i1 + 1]]) for i0, i1, _ in runs]


def draw_panelA_ribbon(ax, theta_lo: float, theta_hi: float, theta_mid: float, color: str):
    """
    Panel-A highlight (edge-on chord): same vertical strip between
    x=cos(θ_hi)..cos(θ_lo) clipped to the unit circle, base alpha 0.28, and the
    same foreshortened mid-ellipse (lw 2.8, alpha 0.95) — now with the opacity
    gradient along the chord: alpha *= (1 + sgn·y/ρ(x))/2 = (1+cos psi)/2,
```

```
    darkest at the θ-ray end, transparent at the opposite end.
    """
    mu_lo = float(np.cos(np.deg2rad(theta_lo)))
    mu_hi = float(np.cos(np.deg2rad(theta_hi)))
    x0, x1 = (min(mu_lo, mu_hi), max(mu_lo, mu_hi))
    sgn = theta_ray_sign(theta_mid)
    nx, ny = 64, 1200
    xs = np.linspace(x0, x1, nx)
    ys = np.linspace(-1.0, 1.0, ny)
    X, Y = np.meshgrid(xs, ys)
    rho = np.sqrt(np.clip(1.0 - X * X, 1e-6, None))
    sc = np.clip(0.5 * (1.0 + sgn * Y / rho), 0.0, 1.0)
    img = np.zeros((ny, nx, 4))
    img[..., :3] = to_rgba(color)[:3]
    img[..., 3] = 0.28 * sc
    im = ax.imshow(img, extent=(x0, x1, -1.0, 1.0), origin="lower",
                   interpolation="bilinear", zorder=2, aspect="auto")
    im.set_clip_path(Circle((0.0, 0.0), 1.0, transform=ax.transData))

    mu_mid = float(np.cos(np.deg2rad(theta_mid)))
    rho_m = np.sqrt(max(0.0, 1.0 - mu_mid * mu_mid))
    phi0, phi = grad_phi(theta_mid, 721)
    ex_m = mu_mid + 0.045 * rho_m * np.cos(phi)
    ey_m = rho_m * np.sin(phi)
    runs = quantized_runs(segment_scale(grad_scale(phi, phi0)))
    ax.add_collection(
        LineCollection(
            _polylines_2d(ex_m, ey_m, runs),
            colors=rgba_array(color, [0.95 * r[2] for r in runs]),
            linewidths=2.8, zorder=4, capstyle="butt", joinstyle="round",
        )
    )


def draw_panelA_theta_rays(ax, geo):
    """
    Illustrate θ: from the observer (origin), one ray per loop out to the
    ring at that loop's center θ. θ is measured from +x (anti-CoE, right)
    up to the ring. Emphasize MAX θ=30° (thick semi-transparent red to the
    top of that foreshortened ring), matching Chris's markup.
    """
    items = list(zip(geo["theta_max_from_plus_x_deg"], COLORS_MAX)) + list(
        zip(geo["theta_min_from_plus_x_deg"], COLORS_MIN)
    )
    for mid, color in items:
        t = float(np.deg2rad(mid))
        # Endpoint = top/rim point of the ring in side view = (cos θ, sin θ)
        x1, y1 = float(np.cos(t)), float(np.sin(t))
        is_max30 = abs(mid - 29.5) < 0.5
        lw = 3.2 if is_max30 else 1.8
        alpha = 0.70 if is_max30 else 0.45
        ax.plot(
            [0.0, x1], [0.0, y1],
            color=color, lw=lw, alpha=alpha, zorder=3,
```

```
            solid_capstyle="round",
        )
        # Small tick at the ring hit
        ax.plot([x1], [y1], "o", color=color, ms=4.5 if is_max30 else 3.5,
                alpha=0.9, zorder=5, markeredgecolor="none")

    # Angle arc at θ=30° from +x axis up to the ray (makes the measure obvious)
    arc_r = 0.38
    phi = np.linspace(0.0, np.deg2rad(29.5), 60)
    ax.plot(
        arc_r * np.cos(phi), arc_r * np.sin(phi),
        color=C_MAX_30, lw=1.6, alpha=0.85, zorder=3,
    )
    ax.text(
        arc_r * np.cos(np.deg2rad(15.0)) + 0.06,
        arc_r * np.sin(np.deg2rad(15.0)) + 0.04,
        r"$\theta$",
        color=C_MAX_30, fontsize=10, fontweight="bold",
        ha="left", va="bottom", zorder=6,
    )


def panel_A_model_frame(ax, geo):
    """Side view: exactly four thick foreshortened loop ribbons about x."""
    ax.set_title(
        "A. Model frame (no Galactic yet)\n"
        r"axisymmetric about $x$ = CoE$\leftrightarrow$anti-CoE"
        "\n"
        r"two MAX + two MIN, each $18^\circ$ end-to-end ($\pm9^\circ$)",
        fontsize=10,
        pad=8,
    )
    th = np.linspace(0, 2 * np.pi, 400)
    ax.plot(np.cos(th), np.sin(th), color="#7f8c8d", lw=1.2, zorder=1)
    ax.plot(0, 0, "ko", ms=6, zorder=5)
    ax.text(0.08, -0.12, "observer", fontsize=8, color="k", ha="left")

    ax.annotate(
        "",
        xy=(1.15, 0),
        xytext=(-1.15, 0),
        arrowprops=dict(arrowstyle="<->", color=C_AXIS, lw=2.5),
        zorder=4,
    )
    ax.text(1.18, 0.08, "anti-CoE\n(+x)", color=C_ANTI, fontsize=8, ha="left", va="bottom")
    ax.text(-1.18, 0.08, "CoE\n(−x)", color=C_COE, fontsize=8, ha="right", va="bottom")

    for (lo, hi), mid, color in zip(
        geo["max_bands_deg"], geo["theta_max_from_plus_x_deg"], COLORS_MAX
    ):
        draw_panelA_ribbon(ax, lo, hi, mid, color)
    for (lo, hi), mid, color in zip(
        geo["min_bands_deg"], geo["theta_min_from_plus_x_deg"], COLORS_MIN
    ):
```

```
        draw_panelA_ribbon(ax, lo, hi, mid, color)

    # Illustrate θ from +x (anti-CoE) to each loop (Chris markup)
    draw_panelA_theta_rays(ax, geo)

    label_r = 1.30
    for mid, color in list(zip(geo["theta_max_from_plus_x_deg"], COLORS_MAX)) + list(
        zip(geo["theta_min_from_plus_x_deg"], COLORS_MIN)
    ):
        t = np.deg2rad(mid)
        ax.text(
            label_r * np.cos(t),
            label_r * np.sin(t),
            rf"$\theta\!=\!{mid:g}^\circ$",
            color=color, fontsize=7.5, ha="center", va="center", fontweight="bold",
        )

    ax.text(
        0.0,
        -1.38,
        (
            r"four transparent highlight bands about $x$  ·  each $18^\circ$ end-to-end ($\pm9^\circ$ = 5% of $360^\circ$)"
            + "\n"
            + r"(after-2$\ell$ $A(\ell)$: two MAX + two MIN; one mid-stroke each)"
            + "\n"
            + r"$y,z$ free (roll) — model does not pick celestial equator"
        ),
        fontsize=7.5,
        ha="center",
        va="top",
        style="italic",
        color="#34495e",
    )
    ax.set_xlim(-1.55, 1.55)
    ax.set_ylim(-1.55, 1.45)
    ax.set_aspect("equal")
    ax.axis("off")


def _ring_lb(D, mu, n_pts=721):
    pts = small_circle_about_axis(D, mu, n=n_pts)
    rl = np.array([n_to_lb(p)[0] for p in pts])
    rb = np.array([n_to_lb(p)[1] for p in pts])
    return rl, rb


def _ring_lonlat_rad(D, mu, n_pts=361):
    """Small-circle → Mollweide display (lon_disp, lat) in radians."""
    pts = small_circle_about_axis(D, float(mu), n=n_pts)
    ls = np.array([n_to_lb(p)[0] for p in pts])
    bs = np.array([n_to_lb(p)[1] for p in pts])
    lon = np.deg2rad(lon_disp_from_ell(ls))
    lat = np.deg2rad(bs)
    return lon, lat
```

```
def fill_mollweide_ribbon(ax, D, theta_lo, theta_hi, theta_mid, color, n_pts=721):
    """
    Mollweide highlight band (±9° edges, base alpha 0.26) + mid-stroke
    (lw 2.6, alpha 0.95), both with the (1+cos psi)/2 opacity gradient.
    Elements crossing the antimeridian seam are dropped.
    """
    mu_a, mu_b = mu_pair_for_band(theta_lo, theta_hi)
    mu_mid = float(np.cos(np.deg2rad(theta_mid)))

    def lonlat(mu, n):
        pts, sc = small_circle_grad(D, mu, theta_mid, n=n)
        ls = np.array([n_to_lb(p)[0] for p in pts])
        bs = np.array([n_to_lb(p)[1] for p in pts])
        return np.deg2rad(lon_disp_from_ell(ls)), np.deg2rad(bs), sc

    lon_a, lat_a, sc = lonlat(mu_a, n_pts)
    lon_b, lat_b, _ = lonlat(mu_b, n_pts)
    gap = 0.9 * np.pi
    quad = np.stack([lon_a[:-1], lon_a[1:], lon_b[1:], lon_b[:-1]])
    valid = (quad.max(axis=0) - quad.min(axis=0)) <= gap
    runs = quantized_runs(segment_scale(sc), valid)
    if runs:
        ax.add_collection(
            PolyCollection(
                _strip_polys_2d(lon_a, lat_a, lon_b, lat_b, runs),
                facecolors=rgba_array(color, [0.26 * r[2] for r in runs]),
                edgecolors="none", zorder=2, linewidths=0,
            )
        )
    lon_m, lat_m, sc_m = lonlat(mu_mid, max(n_pts, 721))
    valid = np.abs(np.diff(lon_m)) <= gap
    runs = quantized_runs(segment_scale(sc_m), valid)
    if runs:
        ax.add_collection(
            LineCollection(
                _polylines_2d(lon_m, lat_m, runs),
                colors=rgba_array(color, [0.95 * r[2] for r in runs]),
                linewidths=2.6, zorder=4, capstyle="butt", joinstyle="round",
            )
        )


def panel_B_mollweide(ax, geo):
    ax.set_title(
        r"B. Migkas $A(\ell)$ loops on sky (anti-CoE at $(0^\circ,+15^\circ)$)"
        "\n"
        r"(Galactic Mollweide; CoE axis tilted $+15^\circ$; $\ell$ increases LEFT; "
        r"two MAX + two MIN, each $18^\circ$ end-to-end ($\pm9^\circ$))",
        fontsize=10,
        pad=8,
    )
    for b in [-60, -30, 0, 30, 60]:
```

```
        ell = np.linspace(0, 360, 361)
        plot_lb_curve_mollweide(ax, ell, np.full_like(ell, b), color="#bdc3c7", lw=0.5, zorder=0)
    for ell0 in range(0, 360, 30):
        b = np.linspace(-90, 90, 181)
        plot_lb_curve_mollweide(ax, np.full_like(b, ell0), b, color="#bdc3c7", lw=0.5, zorder=0)

    D = geo["D"]
    ls, bs = great_circle_through_poles(D, n=721)
    plot_lb_curve_mollweide(ax, ls, bs, color=C_AXIS, lw=2.0, zorder=2)

    for (lo, hi), mid, color in zip(
        geo["max_bands_deg"], geo["theta_max_from_plus_x_deg"], COLORS_MAX
    ):
        fill_mollweide_ribbon(ax, D, lo, hi, mid, color)
    for (lo, hi), mid, color in zip(
        geo["min_bands_deg"], geo["theta_min_from_plus_x_deg"], COLORS_MIN
    ):
        fill_mollweide_ribbon(ax, D, lo, hi, mid, color)

    ell_eq, b_eq = celestial_equator_lb(721)
    plot_lb_curve_mollweide(ax, ell_eq, b_eq, color=C_EQ, lw=1.8, ls="--", zorder=2)

    ax.plot(
        np.deg2rad(lon_disp_from_ell(geo["anti_lb"][0])),
        np.deg2rad(geo["anti_lb"][1]),
        marker="o", ms=8, color=C_ANTI,
        markerfacecolor=C_ANTI, markeredgecolor="k", markeredgewidth=0.6,
        zorder=6, linestyle="None",
    )
    ax.plot(
        np.deg2rad(lon_disp_from_ell(geo["coe_lb"][0])),
        np.deg2rad(geo["coe_lb"][1]),
        marker="*", ms=14, color=C_COE,
        markerfacecolor=C_COE, markeredgecolor="k", markeredgewidth=0.6,
        zorder=6, linestyle="None",
    )
    for (l, b), color in zip(MIGKAS_MAXES, COLORS_MAX):
        ax.plot(
            np.deg2rad(lon_disp_from_ell(l)),
            np.deg2rad(b),
            marker="^",
            ms=10,
            color=color,
            markeredgecolor="k",
            markeredgewidth=0.7,
            zorder=7,
            linestyle="None",
        )
    for (l, b), color in zip(MIGKAS_MINS, COLORS_MIN):
        ax.plot(
            np.deg2rad(lon_disp_from_ell(l)),
            np.deg2rad(b),
            marker="s",
            ms=10,
```

```
            color=color,
            markeredgecolor="k",
            markeredgewidth=0.7,
            zorder=7,
            linestyle="None",
        )

    xticks = np.deg2rad(lon_disp_from_ell(np.array([180, 90, 0, 270])))
    ax.set_xticks(xticks)
    ax.set_xticklabels([r"$180^\circ$", r"$90^\circ$", r"$0^\circ$", r"$270^\circ$"], fontsize=8)
    ax.set_yticks(np.deg2rad([-60, -30, 0, 30, 60]))
    ax.set_yticklabels([r"$-60^\circ$", r"$-30^\circ$", r"$0^\circ$", r"$+30^\circ$", r"$+60^\circ$"], fontsize=8)
    ax.grid(False)
    ax.set_xlabel(r"Galactic $\ell$ (increases LEFT)", fontsize=8)
    ax.set_ylabel(r"Galactic $b$", fontsize=8)


def fill_sphere_ribbon(ax, D, theta_lo, theta_hi, theta_mid, color, n_pts=160):
    """
    3D highlight belt (±9° edges, base alpha 0.25) + mid-stroke (lw 2.6,
    alpha 0.95), both with the (1+cos psi)/2 opacity gradient.
    """
    mu_a, mu_b = mu_pair_for_band(theta_lo, theta_hi)
    mu_mid = float(np.cos(np.deg2rad(theta_mid)))
    n_g = 361
    p0, sc = small_circle_grad(D, float(mu_a), theta_mid, n=n_g)
    p1, _ = small_circle_grad(D, float(mu_b), theta_mid, n=n_g)
    seg = segment_scale(sc)
    idx = np.where(seg > 1e-4)[0]
    verts = [[p0[j], p0[j + 1], p1[j + 1], p1[j]] for j in idx]
    coll = Poly3DCollection(
        verts, facecolors=rgba_array(color, 0.25 * seg[idx]),
        edgecolors="none", linewidths=0.0, zorder=2,
    )
    ax.add_collection3d(coll)
    pts, scm = small_circle_grad(D, mu_mid, theta_mid, n=n_g)
    runs = quantized_runs(segment_scale(scm))
    ax.add_collection3d(
        Line3DCollection(
            [pts[i0:i1 + 1] for i0, i1, _ in runs],
            colors=rgba_array(color, [0.95 * r[2] for r in runs]),
            linewidths=2.6, zorder=5, capstyle="butt", joinstyle="round",
        )
    )


def panel_C_sphere(ax, geo):
    th_max = geo["theta_max_from_plus_x_deg"]
    th_min = geo["theta_min_from_plus_x_deg"]
    ax.set_title(
        r"C. Sphere 3D — two MAX + two MIN, each $18^\circ$ end-to-end ($\pm9^\circ$)"
        "\n"
        r"(axis line emphasized; celestial equator dashed)",
        fontsize=10,
```

```
    pad=6,
)
draw_wire_sphere(ax)
D = geo["D"]
ell_eq, b_eq = celestial_equator_lb(400)
eq_xyz = np.array([lb_to_n(l, b) for l, b in zip(ell_eq, b_eq)])
ax.plot(eq_xyz[:, 0], eq_xyz[:, 1], eq_xyz[:, 2], color=C_EQ, lw=1.8, ls="--", zorder=3)
ax.plot(
    [-D[0], D[0]],
    [-D[1], D[1]],
    [-D[2], D[2]],
    color=C_AXIS,
    lw=3.0,
    zorder=5,
)
ax.scatter(*(-D), color=C_COE, s=120, marker="*", edgecolors="k", linewidths=0.6,
           depthshade=False, zorder=8)
ax.scatter(*(D), color=C_ANTI, s=80, edgecolors="k", linewidths=0.6, depthshade=False, zorder=8)
ax.text(*(-D * 1.22), "CoE", color=C_COE, fontsize=8, fontweight="bold")
ax.text(*(D * 1.18), "anti-CoE", color=C_ANTI, fontsize=8, fontweight="bold")
ax.scatter([0], [0], [0], color="k", s=25, depthshade=False, zorder=9)

for (lo, hi), mid, color in zip(
    geo["max_bands_deg"], geo["theta_max_from_plus_x_deg"], COLORS_MAX
):
    fill_sphere_ribbon(ax, D, lo, hi, mid, color)
for (lo, hi), mid, color in zip(
    geo["min_bands_deg"], geo["theta_min_from_plus_x_deg"], COLORS_MIN
):
    fill_sphere_ribbon(ax, D, lo, hi, mid, color)

for n, color in zip(geo["n_maxes"], COLORS_MAX):
    ax.scatter(
        [n[0]], [n[1]], [n[2]],
        c=[color], marker="^", s=70,
        edgecolors="k", linewidths=0.7, depthshade=False, zorder=10,
    )
    ax.plot([0, n[0]], [0, n[1]], [0, n[2]], color=color, lw=0.8, ls=":", alpha=0.7)
for n, color in zip(geo["n_mins"], COLORS_MIN):
    ax.scatter(
        [n[0]], [n[1]], [n[2]],
        c=[color], marker="s", s=60,
        edgecolors="k", linewidths=0.7, depthshade=False, zorder=10,
    )
    ax.plot([0, n[0]], [0, n[1]], [0, n[2]], color=color, lw=0.8, ls=":", alpha=0.7)

ax.set_xlim(-1.25, 1.25)
ax.set_ylim(-1.25, 1.25)
ax.set_zlim(-1.25, 1.25)
ax.set_box_aspect((1, 1, 1))
ax.view_init(elev=18, azim=-55)
ax.set_axis_off()
ax.text2D(
    0.5,
```

```
        -0.02,
        rf"MAX $\theta\!=\!{th_max[0]:g}^\circ,{th_max[1]:g}^\circ$; "
        rf"MIN $\theta\!=\!{th_min[0]:g}^\circ,{th_min[1]:g}^\circ$  "
        rf"($18^\circ$ end-to-end = $\pm9^\circ$ = 5% of $360^\circ$)",
        transform=ax.transAxes,
        ha="center",
        fontsize=7.5,
        color="#34495e",
    )


def main():
    st = migkas_scan_stops()
    geo = compute_geometry(st)

    th_max = geo["theta_max_from_plus_x_deg"]
    th_min = geo["theta_min_from_plus_x_deg"]
    max_bands = geo["max_bands_deg"]
    min_bands = geo["min_bands_deg"]

    fig = plt.figure(figsize=(14.5, 5.6), dpi=160)
    fig.suptitle(
        r"Pure CoE axisymmetry: two MAX + two MIN highlight bands  (gradient: darkest at $\theta$ ray, transparent opposite)"
        "\n"
        rf"(each $18^\circ$ end-to-end = $\pm9^\circ$ = 5% of $360^\circ$; "
        rf"MAX $\theta\!=\!{th_max[0]:g}^\circ/{th_max[1]:g}^\circ$; "
        rf"MIN $\theta\!=\!{th_min[0]:g}^\circ/{th_min[1]:g}^\circ$; "
        r"anti-CoE $(0^\circ,+15^\circ)$, CoE axis tilted $+15^\circ$)",
        fontsize=11,
        fontweight="bold",
        y=1.02,
    )

    axA = fig.add_subplot(1, 3, 1)
    panel_A_model_frame(axA, geo)

    axB = fig.add_subplot(1, 3, 2, projection="mollweide")
    panel_B_mollweide(axB, geo)

    axC = fig.add_subplot(1, 3, 3, projection="3d")
    panel_C_sphere(axC, geo)

    handles = [
        Line2D([0], [0], color=C_AXIS, lw=2.5, label=r"CoE$\leftrightarrow$anti-CoE axis"),
        Patch(
            facecolor=C_MAX_30, alpha=0.45, edgecolor=C_MAX_30,
            label=(
                rf"MAX $\theta\!=\!{th_max[0]:g}^\circ$ band "
                rf"[{max_bands[0][0]:g},{max_bands[0][1]:g}]$^\circ$ "
                rf"($18^\circ$=±$9^\circ$=5% of $360^\circ$)"
            ),
        ),
        Patch(
            facecolor=C_MAX_185, alpha=0.45, edgecolor=C_MAX_185,
```

```
            label=(
                rf"MAX $\theta\!=\!{th_max[1]:g}^\circ$ band "
                rf"[{max_bands[1][0]:g},{max_bands[1][1]:g}]$^\circ$ "
                rf"($18^\circ$=±$9^\circ$)"
            ),
        ),
        Patch(
            facecolor=C_MIN_115, alpha=0.45, edgecolor=C_MIN_115,
            label=(
                rf"MIN $\theta\!=\!{th_min[0]:g}^\circ$ band "
                rf"[{min_bands[0][0]:g},{min_bands[0][1]:g}]$^\circ$ "
                rf"($18^\circ$=±$9^\circ$)"
            ),
        ),
        Patch(
            facecolor=C_MIN_265, alpha=0.45, edgecolor=C_MIN_265,
            label=(
                rf"MIN $\theta\!=\!{th_min[1]:g}^\circ$ band "
                rf"[{min_bands[1][0]:g},{min_bands[1][1]:g}]$^\circ$ "
                rf"($18^\circ$=±$9^\circ$)"
            ),
        ),
        Line2D([0], [0], color=C_EQ, lw=1.8, ls="--", label="celestial equator (Dec$=0$)"),
        Line2D(
            [0], [0], marker="^", color="w", markerfacecolor=C_MAX_30, markeredgecolor="k",
            ms=9,
            label=rf"MAX $({MIGKAS_MAXES[0][0]:.0f}^\circ,{MIGKAS_MAXES[0][1]:+.0f}^\circ)$",
        ),
        Line2D(
            [0], [0], marker="^", color="w", markerfacecolor=C_MAX_185, markeredgecolor="k",
            ms=9,
            label=rf"MAX $({MIGKAS_MAXES[1][0]:.0f}^\circ,{MIGKAS_MAXES[1][1]:+.0f}^\circ)$",
        ),
        Line2D(
            [0], [0], marker="s", color="w", markerfacecolor=C_MIN_115, markeredgecolor="k",
            ms=8,
            label=rf"MIN $({MIGKAS_MINS[0][0]:.0f}^\circ,{MIGKAS_MINS[0][1]:+.0f}^\circ)$",
        ),
        Line2D(
            [0], [0], marker="s", color="w", markerfacecolor=C_MIN_265, markeredgecolor="k",
            ms=8,
            label=rf"MIN $({MIGKAS_MINS[1][0]:.0f}^\circ,{MIGKAS_MINS[1][1]:+.0f}^\circ)$",
        ),
        Line2D(
            [0], [0], marker="o", color="w", markerfacecolor=C_ANTI, markeredgecolor="k",
            ms=8, label=rf"anti-CoE $({geo['anti_lb'][0]:.1f}^\circ,{geo['anti_lb'][1]:+.1f}^\circ)$",
        ),
        Line2D(
            [0], [0], marker="*", color="w", markerfacecolor=C_COE, markeredgecolor="k",
            ms=11, label=rf"CoE $({geo['coe_lb'][0]:.1f}^\circ,{geo['coe_lb'][1]:+.1f}^\circ)$",
        ),
    ]
    fig.legend(
        handles=handles,
```

```
        loc="lower center",
        ncol=6,
        fontsize=7.0,
        frameon=True,
        fancybox=True,
        bbox_to_anchor=(0.5, -0.10),
    )

    fig.tight_layout()
    # --- Chris 2026-09-26: panel C sphere drawn at the same on-page diameter as
    # panel A's model-frame circle; panel C axes matched to A's vertical
    # position/height. Nothing else changes.
    from mpl_toolkits.mplot3d import proj3d
    pa, pc = axA.get_position(), axC.get_position()
    axC.set_position([pc.x0, pa.y0, pc.width, pa.height])

    def _diam_A():
        fig.canvas.draw()
        p0 = axA.transData.transform((-1.0, 0.0)); p1 = axA.transData.transform((1.0, 0.0))
        return float(p1[0] - p0[0])

    def _diam_C():
        fig.canvas.draw()
        u = np.random.default_rng(0).normal(size=(20000, 3))
        u /= np.linalg.norm(u, axis=1)[:, None]
        xs, ys, _ = proj3d.proj_transform(u[:, 0], u[:, 1], u[:, 2], axC.get_proj())
        d = axC.transData.transform(np.column_stack([xs, ys]))
        return float(d[:, 0].max() - d[:, 0].min()), float(d[:, 1].max() - d[:, 1].min())

    zoom = 1.0
    for _ in range(8):
        dA = _diam_A(); dCx, dCy = _diam_C()
        zoom *= dA / dCx
        axC.set_box_aspect((1, 1, 1), zoom=zoom)
    dA = _diam_A(); dCx, dCy = _diam_C()
    print(f"PANEL_DIAM A={dA:.1f}px C={dCx:.1f}x{dCy:.1f}px zoom={zoom:.3f}")
    fig.savefig(PNG_PATH, bbox_inches="tight", facecolor="white")
    plt.close(fig)

    payload = {
        "png": str(PNG_PATH),
        "script": str(OUT_DIR / "make_coe_rings_migkas_pin_axis_matched_grad.py"),
        "migkas_max_lb_deg": [list(lb) for lb in MIGKAS_MAXES],
        "migkas_min_lb_deg": [list(lb) for lb in MIGKAS_MINS],
        "strip_theta_max_deg": list(STRIP_THETA_MAX),
        "strip_theta_min_deg": list(STRIP_THETA_MIN),
        "band_half_width_deg": BAND_HALF,
        "band_full_width_deg": BAND_FULL,
        "band_total_width_deg": BAND_TOTAL,
        "band_width_note": "5% of 360° = 18° end-to-end = ±9° about each MAX/MIN center",
        "max_bands_deg": [list(b) for b in max_bands],
        "min_bands_deg": [list(b) for b in min_bands],
        "band_intervals": {
            "MAX_30": [float(max_bands[0][0]), float(max_bands[0][1])],
```

```
            "MAX_185": [float(max_bands[1][0]), float(max_bands[1][1])],
            "MIN_115": [float(min_bands[0][0]), float(min_bands[0][1])],
            "MIN_265": [float(min_bands[1][0]), float(min_bands[1][1])],
        },
        "scan_stop_source": st["source"],
        "theta_max_from_plus_x_deg": [round(t, 4) for t in th_max],
        "theta_min_from_plus_x_deg": [round(t, 4) for t in th_min],
        "mu_max_bands": [[round(a, 8), round(b, 8)] for a, b in geo["mu_max_bands"]],
        "mu_min_bands": [[round(a, 8), round(b, 8)] for a, b in geo["mu_min_bands"]],
        "CoE_lb_deg": [round(geo["coe_lb"][0], 4), round(geo["coe_lb"][1], 4)],
        "anti_CoE_lb_deg": [round(geo["anti_lb"][0], 4), round(geo["anti_lb"][1], 4)],
        "D_hat_galactic_xyz": geo["D"].tolist(),
        "angular_sep_migkas_max0_min1_deg": round(geo["sep_deg"], 6),
        "axis_tilt_b_deg": 15,
        "axis_tilt_reason": (
            "Chris 2026-09-22 — markers fixed, rings rotate up 30°"
        ),
        "orientation": (
            "Galactic orientation: anti-CoE at (ell=0°, b=+15°), CoE at "
            "(ell=180°, b=−15°); +D = anti-CoE = +x; axis tilted +15° "
            "(Chris 2026-09-22 — markers fixed, rings rotate up 30°)"
        ),
        "drawing": (
            "Exactly 4 features. Each = ONE smooth transparent highlight band "
            "between θ−9° and θ+9° (18° end-to-end = 5% of 360°) + ONE solid "
            "mid-stroke at the exact center θ. No multi-stroke wash / hairball. "
            "Four distinct colors (two reds + two blues); no orange; no 283° "
            "companion. Panel A: clean vertical strip clipped to unit circle; theta rays from observer to each loop (emphasized at MAX 30°). Mollweide
"
            "seam-split honestly. Sphere: single Poly3D strip (no n_wash stack)."
        ),
        "note": (
            "Stops are after-2ℓ A(ℓ) extrema from best_fit_2ell_multiplier.npz "
            f"A_after: two MAX at θ={th_max[0]:g}°/{th_max[1]:g}° and "
            f"two MIN at θ={th_min[0]:g}°/{th_min[1]:g}°, each drawn as "
            f"one {BAND_FULL:.0f}° end-to-end highlight (±{BAND_HALF:.0f}° = 5% of 360°). "
            "Rings use μ=cos(θ); sky markers are map extrema (17°,−9°) and "
            "(274°,−22°) plus strip stand-ins (185°,0°) and (115°,0°)."
        ),
        "plot_convention": "Migkas Galactic Mollweide: lon_disp=-ell_wrapped (ell increases LEFT)",
        "legend_summary": (
            "two MAX + two MIN highlight bands, each 18° end-to-end "
            "(±9° = 5% of 360°); four distinct colors"
        ),
        "colors": {
            "MAX_30": C_MAX_30,
            "MAX_185": C_MAX_185,
            "MIN_115": C_MIN_115,
            "MIN_265": C_MIN_265,
            "no_orange": True,
        },
    }
    JSON_PATH.write_text(json.dumps(payload, indent=2) + "\n")
    print(json.dumps(payload, indent=2))
```

```
    print(f"\nWrote {PNG_PATH}")
    print(f"Wrote {JSON_PATH}")


if __name__ == "__main__":
    main()
```

```
PANEL_DIAM A=367.9px C=367.9x368.0px zoom=1.399
```

```
{
  "png": "/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/loop_threepanel_matched_grad.png",
  "script": "/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/make_coe_rings_migkas_pin_axis_matched_grad.py",
  "migkas_max_lb_deg": [
    [
      17.0,
      -9.0
    ],
    [
      185.0,
      0.0
    ]
  ],
  "migkas_min_lb_deg": [
    [
      115.0,
      0.0
    ],
    [
      274.0,
      -22.0
    ]
  ],
  "strip_theta_max_deg": [
    29.5,
    187.5
  ],
  "strip_theta_min_deg": [
    122.6,
    264.4
  ],
  "band_half_width_deg": 9.0,
  "band_full_width_deg": 18.0,
  "band_total_width_deg": 18.0,
  "band_width_note": "5% of 360\u00b0 = 18\u00b0 end-to-end = \u00b19\u00b0 about each MAX/MIN center",
  "max_bands_deg": [
    [
      20.5,
      38.5
    ],
    [
      178.5,
      196.5
    ]
  ],
  "min_bands_deg": [
    [
      113.6,
      131.6
    ],
    [
      255.39999999999998,
      273.4
    ]
  ],
  "band_intervals": {
    "MAX_30": [
```

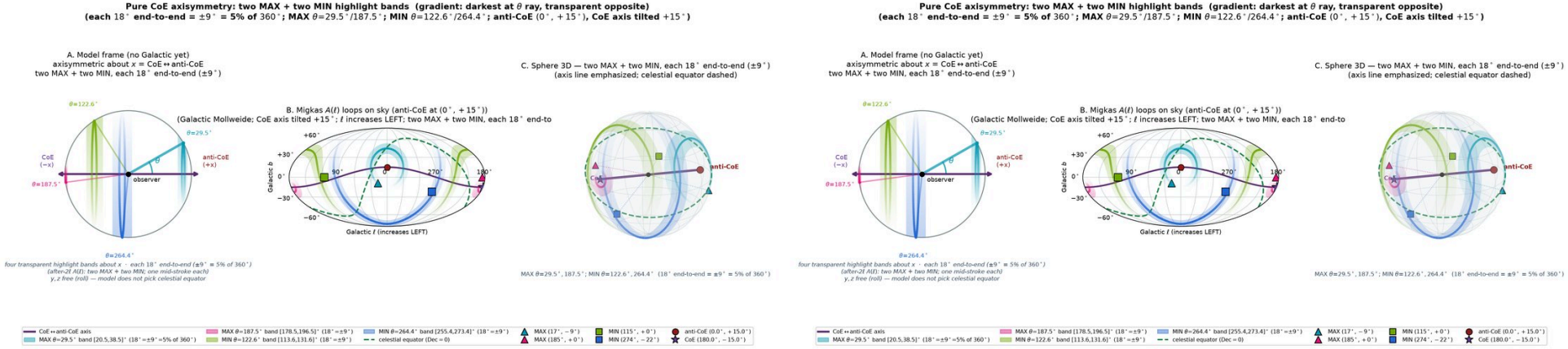
```
      20.5,
      38.5
    ],
    "MAX_185": [
      178.5,
      196.5
    ],
    "MIN_115": [
      113.6,
      131.6
    ],
    "MIN_265": [
      255.39999999999998,
      273.4
    ]
  },
  "scan_stop_source": "best_fit_2ell_multiplier.npz A_after extrema (two max, two min); each drawn as one transparent 18\u00b0 end-to-end highlight
(\u00b19\u00b0 = 5% of 360\u00b0) + one solid mid-stroke",
  "theta_max_from_plus_x_deg": [
    29.5,
    187.5
  ],
  "theta_min_from_plus_x_deg": [
    122.6,
    264.4
  ],
  "mu_max_bands": [
    [
      0.93667219,
      0.78260816
    ],
    [
      -0.99965732,
      -0.95881973
    ]
  ],
  "mu_min_bands": [
    [
      -0.40034903,
      -0.66392621
    ],
    [
      -0.25206936,
      0.05930637
    ]
  ],
  "CoE_lb_deg": [
    180.0,
    -15.0
  ],
  "anti_CoE_lb_deg": [
    0.0,
    15.0
  ],
  "D_hat_galactic_xyz": [
    0.9659258262890683,
    0.0,
```

```
    0.25881904510252074
  ],
  "angular_sep_migkas_max0_min1_deg": 98.476384,
  "axis_tilt_b_deg": 15,
  "axis_tilt_reason": "Chris 2026-09-22 \u2014 markers fixed, rings rotate up 30\u00b0",
  "orientation": "Galactic orientation: anti-CoE at (ell=0\u00b0, b=+15\u00b0), CoE at (ell=180\u00b0, b=\u221215\u00b0); +D = anti-CoE = +x; axis
tilted +15\u00b0 (Chris 2026-09-22 \u2014 markers fixed, rings rotate up 30\u00b0)",
  "drawing": "Exactly 4 features. Each = ONE smooth transparent highlight band between \u03b8\u22129\u00b0 and \u03b8+9\u00b0 (18\u00b0 end-to-end = 5%
of 360\u00b0) + ONE solid mid-stroke at the exact center \u03b8. No multi-stroke wash / hairball. Four distinct colors (two reds + two blues); no
orange; no 283\u00b0 companion. Panel A: clean vertical strip clipped to unit circle; theta rays from observer to each loop (emphasized at MAX
30\u00b0). Mollweide seam-split honestly. Sphere: single Poly3D strip (no n_wash stack).",
  "note": "Stops are after-2\u2113 A(\u2113) extrema from best_fit_2ell_multiplier.npz A_after: two MAX at \u03b8=29.5\u00b0/187.5\u00b0 and two MIN at
\u03b8=122.6\u00b0/264.4\u00b0, each drawn as one 18\u00b0 end-to-end highlight (\u00b19\u00b0 = 5% of 360\u00b0). Rings use \u03bc=cos(\u03b8); sky
markers are map extrema (17\u00b0,\u22129\u00b0) and (274\u00b0,\u221222\u00b0) plus strip stand-ins (185\u00b0,0\u00b0) and (115\u00b0,0\u00b0).",
  "plot_convention": "Migkas Galactic Mollweide: lon_disp=-ell_wrapped (ell increases LEFT)",
  "legend_summary": "two MAX + two MIN highlight bands, each 18\u00b0 end-to-end (\u00b19\u00b0 = 5% of 360\u00b0); four distinct colors",
  "colors": {
    "MAX_30": "#0097b2",
    "MAX_185": "#e0107a",
    "MIN_115": "#6fa800",
    "MIN_265": "#2166d4",
    "no_orange": true
  }
}

Wrote /workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/loop_threepanel_matched_grad.png
Wrote /workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/loop_threepanel_matched_grad.json
```


```
compare(FIGS / 'loop_threepanel_matched_grad.png', '/workspace/cosmology/papers/spreadsheet_model/loop_threepanel_matched_grad.png')
```

```
loop_threepanel_matched_grad.png: new (2262, 1027) vs ref (2262, 1027) | mean |diff| = 0.000/255, pixels differing >40: 0.000% -> identical
left: approved reference papers/spreadsheet_model/loop_threepanel_matched_grad.png   |   right: notebook re-run
steps_v3/figs/loop_threepanel_matched_grad.png
```



Step 16c — Matched three-panel, gradient, 45° roll

What it shows. Step 16b with the loops rolled 45° about the CoE axis. This is the 45° roll version of the three-panel figure.

Source script: `/workspace/cosmology/figures/make_coe_rings_migkas_pin_axis_matched_grad_rot45.py` (last modified 2026-09-26 22:54 CT), run verbatim. *Output-path edit:* `/workspace/cosmology/papers/spreadsheet_model/loop_threepanel_matched_` → `/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/loop_threep`. *Reference image:* `papers/spreadsheet_model/loop_threepanel_matched_grad_rot45.png`.

```
%%runscript make_coe_rings_migkas_pin_axis_matched_grad_rot45.py
#!/usr/bin/env python3
"""
CoE rings + Migkas A(ell) extrema loops in the Galactic frame.

Concept:
  Pure CoE axisymmetry → constant A on small-circle rings about x = CoE↔anti-CoE.
  Scan A vs angle from +x (anti-CoE) around 360°. After-2ℓ A(ℓ) has TWO maxima
  and TWO minima (from best_fit_2ell_multiplier.npz A_after):
    MAX at θ = 30° and 185° from +x,
    MIN at θ = 115° and 265° from +x.

  Exactly FOUR loops — each ONE transparent highlight band, 18° end-to-end
  (= ±9° = 5% of 360°) about the center θ:
    MAX: 30±9 → [21°, 39°] and 185±9 → [176°, 194°]
    MIN: 115±9 → [106°, 124°] and 265±9 → [256°, 274°]
  Drawing: ONE smooth transparent filled ribbon between θ−9° and θ+9°
  + ONE solid mid-stroke at the exact center θ. No multi-stroke wash,
  no dense linspace hairball, no dual strong boundaries.
  Four distinct colors (two reds + two blues). No orange, no 283° companion.

  Galactic orientation: anti-CoE (+x) at ℓ=0°, b=+15°; CoE (−x) at ℓ=180°, b=−15°
  (axis tilted +15° so features near ℓ=0 move north on Mollweide; Migkas markers fixed).
  Sky markers (ℓ, b):
    Maxes: map extrema (17°, −9°) for ~30° max; strip (185°, 0°) for second max.
    Mins:  strip (115°, 0°) for ~115° min; map extrema (274°, −22°) for ~265° min.

Outputs:
  coe_rings_migkas_pin_axis.png
  coe_rings_migkas_pin_axis.json
"""

from __future__ import annotations

import json
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
from matplotlib.lines import Line2D
from matplotlib.collections import PolyCollection
from matplotlib.patches import Circle, Patch, Polygon
from mpl_toolkits.mplot3d import Axes3D  # noqa: F401
from mpl_toolkits.mplot3d.art3d import Poly3DCollection
```

```
OUT_DIR = Path(__file__).resolve().parent
PNG_PATH = Path("/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/loop_threepanel_matched_grad_rot45.png")
JSON_PATH = Path("/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/loop_threepanel_matched_grad_rot45.json")

# 5% of 360° = 18° end-to-end → half-width ±9° about each center
BAND_HALF = 9.0
BAND_FULL = 2.0 * BAND_HALF  # 18° = 5% of 360°
BAND_TOTAL = BAND_FULL  # alias
STRIP_THETA_MAX = [29.5, 187.5]
STRIP_THETA_MIN = [122.6, 264.4]

MIGKAS_MAXES = [(17.0, -9.0), (185.0, 0.0)]
MIGKAS_MINS = [(115.0, 0.0), (274.0, -22.0)]

C_COE = "#6b3fa0"
C_ANTI = "#8e1b1b"  # dark red marker; distinct from loop reds
# Four distinct loop colors: two reds (MAX) + two blues (MIN); no orange/yellow
C_MAX_30 = "#0097b2"    # cyan — MAX at 29.5°
C_MAX_185 = "#e0107a"   # magenta — MAX at 187.5°
C_MIN_115 = "#6fa800"   # green — MIN at 122.6°
C_MIN_265 = "#2166d4"   # blue — MIN at 264.4°
C_AXIS = "#5b2c6f"
C_EQ = "#1e8449"

# Per-loop color maps (order matches STRIP_THETA_MAX / STRIP_THETA_MIN)
COLORS_MAX = [C_MAX_30, C_MAX_185]
COLORS_MIN = [C_MIN_115, C_MIN_265]


def wrap_lon_deg(lon: np.ndarray) -> np.ndarray:
    return ((np.asarray(lon, dtype=float) + 180.0) % 360.0) - 180.0


def lon_disp_from_ell(ell_deg: np.ndarray) -> np.ndarray:
    """Migkas Galactic: ℓ increases LEFT → lon_disp = −ℓ_wrapped."""
    return -wrap_lon_deg(ell_deg)


def lb_to_n(l_deg: float, b_deg: float) -> np.ndarray:
    l = np.deg2rad(l_deg)
    b = np.deg2rad(b_deg)
    return np.array(
        [np.cos(b) * np.cos(l), np.cos(b) * np.sin(l), np.sin(b)], dtype=float
    )


def n_to_lb(n: np.ndarray) -> tuple[float, float]:
    n = np.asarray(n, dtype=float)
    n = n / np.linalg.norm(n)
    b = float(np.rad2deg(np.arcsin(np.clip(n[2], -1.0, 1.0))))
    l = float(np.rad2deg(np.arctan2(n[1], n[0])) % 360.0)
    return l, b
```

```
def celestial_equator_lb(n: int = 720) -> tuple[np.ndarray, np.ndarray]:
    """Dec=0 (ICRS) as Galactic (ℓ, b). IAU J2000 NGP convention, no astropy."""
    ra_ngp = np.deg2rad(192.85948)
    dec_ngp = np.deg2rad(27.12825)
    l_ncp = np.deg2rad(122.93192)
    ra = np.linspace(0.0, 2.0 * np.pi, n, endpoint=False)
    dec = 0.0
    sin_b = np.sin(dec) * np.sin(dec_ngp) + np.cos(dec) * np.cos(dec_ngp) * np.cos(ra - ra_ngp)
    sin_b = np.clip(sin_b, -1.0, 1.0)
    b = np.arcsin(sin_b)
    y = np.cos(dec) * np.sin(ra - ra_ngp)
    x = np.sin(dec) * np.cos(dec_ngp) - np.cos(dec) * np.sin(dec_ngp) * np.cos(ra - ra_ngp)
    l = (l_ncp - np.arctan2(y, x)) % (2.0 * np.pi)
    return np.rad2deg(l), np.rad2deg(b)


def great_circle_through_poles(D: np.ndarray, n: int = 361) -> tuple[np.ndarray, np.ndarray]:
    """Great circle through ±D (any meridian of the CoE axis)."""
    a = D / np.linalg.norm(D)
    tmp = np.array([0.0, 0.0, 1.0])
    if abs(np.dot(a, tmp)) > 0.9:
        tmp = np.array([1.0, 0.0, 0.0])
    e2 = np.cross(a, tmp)
    e2 /= np.linalg.norm(e2)
    t = np.linspace(0.0, 2.0 * np.pi, n)
    pts = np.outer(np.cos(t), a) + np.outer(np.sin(t), e2)
    ls = np.array([n_to_lb(p)[0] for p in pts])
    bs = np.array([n_to_lb(p)[1] for p in pts])
    return ls, bs


def small_circle_about_axis(axis: np.ndarray, mu: float, n: int = 361) -> np.ndarray:
    """Unit vectors on the small circle n·axis = mu (|mu|<1)."""
    a = axis / np.linalg.norm(axis)
    mu = float(np.clip(mu, -1.0 + 1e-9, 1.0 - 1e-9))
    tmp = np.array([0.0, 0.0, 1.0]) if abs(a[2]) < 0.9 else np.array([1.0, 0.0, 0.0])
    e1 = np.cross(a, tmp)
    e1 /= np.linalg.norm(e1)
    e2 = np.cross(a, e1)
    rho = np.sqrt(max(0.0, 1.0 - mu * mu))
    t = np.linspace(0.0, 2.0 * np.pi, n)
    return (mu * a)[None, :] + rho * (np.outer(np.cos(t), e1) + np.outer(np.sin(t), e2))


def split_lon_segments(lon_rad: np.ndarray, lat_rad: np.ndarray, gap: float = np.pi):
    """Split polyline where Mollweide lon jumps (wrap discontinuity)."""
    d = np.diff(lon_rad)
    cuts = np.where(np.abs(d) > gap)[0] + 1
    idx = np.concatenate([[0], cuts, [len(lon_rad)]])
    segs = []
    for i0, i1 in zip(idx[:-1], idx[1:]):
        if i1 - i0 >= 2:
            segs.append((lon_rad[i0:i1], lat_rad[i0:i1]))
    return segs
```

```
def plot_lb_curve_mollweide(ax, l_deg, b_deg, **kwargs):
    lon = np.deg2rad(lon_disp_from_ell(l_deg))
    lat = np.deg2rad(b_deg)
    for xr, yr in split_lon_segments(lon, lat):
        ax.plot(xr, yr, **kwargs)


def band_edges(centers: list[float], half: float = BAND_HALF) -> list[tuple[float, float]]:
    """Return (theta_lo, theta_hi) for each band center (±half about center)."""
    return [(c - half, c + half) for c in centers]


def mu_pair_for_band(lo: float, hi: float) -> tuple[float, float]:
    """Edge μ=cos(θ) for a scan-angle band (two boundary cones only)."""
    return float(np.cos(np.deg2rad(lo))), float(np.cos(np.deg2rad(hi)))


def migkas_scan_stops() -> dict:
    th_max = [float(t) for t in STRIP_THETA_MAX]
    th_min = [float(t) for t in STRIP_THETA_MIN]
    max_bands = band_edges(th_max)
    min_bands = band_edges(th_min)
    return {
        "theta_max_from_plus_x_deg": th_max,
        "theta_min_from_plus_x_deg": th_min,
        "max_bands_deg": max_bands,
        "min_bands_deg": min_bands,
        "band_half_width_deg": BAND_HALF,
        "band_full_width_deg": BAND_FULL,
        "band_total_width_deg": BAND_TOTAL,
        "band_width_note": "5% of 360° = 18° end-to-end = ±9° about each MAX/MIN center",
        "mu_max_centers": [float(np.cos(np.deg2rad(t))) for t in th_max],
        "mu_min_centers": [float(np.cos(np.deg2rad(t))) for t in th_min],
        "mu_max_bands": [mu_pair_for_band(lo, hi) for lo, hi in max_bands],
        "mu_min_bands": [mu_pair_for_band(lo, hi) for lo, hi in min_bands],
        "source": (
            "best_fit_2ell_multiplier.npz A_after extrema (two max, two min); "
            f"each drawn as one transparent {BAND_FULL:.0f}° end-to-end highlight "
            f"(±{BAND_HALF:.0f}° = 5% of 360°) + one solid mid-stroke"
        ),
    }


def compute_geometry(st: dict) -> dict:
    anti_l, anti_b = 0.0, 15.0
    coe_l, coe_b = 180.0, -15.0

    n_coe = lb_to_n(coe_l, coe_b)
    n_anti = lb_to_n(anti_l, anti_b)
    D = n_anti  # +D = anti-CoE = +x

    th_max = [float(t) for t in st["theta_max_from_plus_x_deg"]]
```

```
    th_min = [float(t) for t in st["theta_min_from_plus_x_deg"]]
    max_bands = list(st["max_bands_deg"])
    min_bands = list(st["min_bands_deg"])

    n_maxes = [lb_to_n(*lb) for lb in MIGKAS_MAXES]
    n_mins = [lb_to_n(*lb) for lb in MIGKAS_MINS]

    sep = float(
        np.rad2deg(
            np.arccos(np.clip(np.dot(n_maxes[0], n_mins[1]), -1.0, 1.0))
        )
    )

    return {
        "D": D,
        "n_coe": n_coe,
        "n_anti": n_anti,
        "n_maxes": n_maxes,
        "n_mins": n_mins,
        "anti_lb": (anti_l, anti_b),
        "coe_lb": (coe_l, coe_b),
        "theta_max_from_plus_x_deg": th_max,
        "theta_min_from_plus_x_deg": th_min,
        "max_bands_deg": max_bands,
        "min_bands_deg": min_bands,
        "mu_max_bands": list(st["mu_max_bands"]),
        "mu_min_bands": list(st["mu_min_bands"]),
        "sep_deg": sep,
        "band_half_width_deg": st["band_half_width_deg"],
    }


def draw_wire_sphere(ax, n_mer=12, n_par=6, **kwargs):
    u = np.linspace(0, 2 * np.pi, 60)
    v = np.linspace(-np.pi / 2, np.pi / 2, 40)
    xs = np.outer(np.cos(u), np.cos(v))
    ys = np.outer(np.sin(u), np.cos(v))
    zs = np.outer(np.ones_like(u), np.sin(v))
    ax.plot_surface(xs, ys, zs, color="#dfe6e9", alpha=0.15, linewidth=0, shade=False)
    kw = dict(color="#95a5a6", lw=0.4, alpha=0.45)
    kw.update(kwargs)
    for i in range(n_mer):
        phi = 2 * np.pi * i / n_mer
        t = np.linspace(-np.pi / 2, np.pi / 2, 80)
        ax.plot(np.cos(t) * np.cos(phi), np.cos(t) * np.sin(phi), np.sin(t), **kw)
    for j in range(1, n_par):
        b = -np.pi / 2 + np.pi * j / n_par
        t = np.linspace(0, 2 * np.pi, 80)
        r = np.cos(b)
        ax.plot(r * np.cos(t), r * np.sin(t), np.full_like(t, np.sin(b)), **kw)


def foreshortened_ellipse_xy(mu: float, n: int = 200):
    """Side-view foreshortened ellipse of a 3D ring at n·x = mu."""
```

```
    rho = np.sqrt(max(0.0, 1.0 - mu * mu))
    phi = np.linspace(0, 2 * np.pi, n)
    ex = mu + 0.045 * rho * np.cos(phi)
    ey = rho * np.sin(phi)
    return ex, ey, rho



# ---------------------------------------------------------------------------
# Opacity gradient along each loop (Chris 2026-09-26), same method as
# make_coe_rings_gradient_loops.py: loop drawn as runs of short elements with
# per-element alpha = base_alpha * (1 + cos psi)/2, psi = ring azimuth from the
# θ-ray point (ring point most along sign(sin θ_mid)·Ŷ; Ŷ = Galactic +z
# projected ⊥ D, i.e. model-frame "up"; no roll). Darkest at the θ-ray point,
# fully transparent at the diametrically opposite point.
# ---------------------------------------------------------------------------
from matplotlib.collections import LineCollection
from matplotlib.colors import to_rgba
from mpl_toolkits.mplot3d.art3d import Line3DCollection


def theta_ray_sign(theta_mid_deg):
    return 1.0 if np.sin(np.deg2rad(float(theta_mid_deg))) >= 0 else -1.0


ROLL_BC_DEG = 45.0  # roll about CoE axis, applied only via small_circle_grad (Panels B/C)


def ring_basis(D):
    a = np.asarray(D, dtype=float)
    a = a / np.linalg.norm(a)
    tmp = np.array([0.0, 0.0, 1.0])
    if abs(np.dot(a, tmp)) > 0.9:
        tmp = np.array([0.0, 1.0, 0.0])
    y = tmp - np.dot(tmp, a) * a
    y /= np.linalg.norm(y)
    z = np.cross(a, y)
    z /= np.linalg.norm(z)
    # +45° roll (Panels B/C only), same convention as coe_half_loop_geometry.ring_basis
    # / earlier "+45° CCW about CoE" figures: Y' = c Y - s Z, Z' = s Y + c Z
    r = np.deg2rad(float(ROLL_BC_DEG))
    c, s_ = np.cos(r), np.sin(r)
    return c * y - s_ * z, s_ * y + c * z


def grad_phi(theta_mid_deg, n):
    """phi from antipode of θ-ray point round to antipode (psi = -180..+180)."""
    phi0 = 0.5 * np.pi if theta_ray_sign(theta_mid_deg) > 0 else 1.5 * np.pi
    return phi0, phi0 + np.linspace(-np.pi, np.pi, n)


def grad_scale(phi, phi0):
    return 0.5 * (1.0 + np.cos(np.asarray(phi) - phi0))
```

```
def small_circle_grad(D, mu, theta_mid_deg, n=721):
    a = np.asarray(D, dtype=float)
    a = a / np.linalg.norm(a)
    mu = float(np.clip(mu, -1.0 + 1e-9, 1.0 - 1e-9))
    rho = np.sqrt(max(0.0, 1.0 - mu * mu))
    y, z = ring_basis(a)
    phi0, phi = grad_phi(theta_mid_deg, n)
    pts = (mu * a)[None, :] + rho * (np.outer(np.sin(phi), y) + np.outer(np.cos(phi), z))
    return pts, grad_scale(phi, phi0)


def segment_scale(sc):
    sc = np.asarray(sc, dtype=float)
    return 0.5 * (sc[:-1] + sc[1:])


def rgba_array(color, alphas):
    r, g, b, _ = to_rgba(color)
    al = np.clip(np.asarray(alphas, dtype=float), 0.0, 1.0)
    out = np.empty((al.size, 4))
    out[:, 0], out[:, 1], out[:, 2], out[:, 3] = r, g, b, al
    return out


def quantized_runs(seg_sc, valid=None, n_levels=64):
    s = np.asarray(seg_sc, dtype=float)
    lev = np.round(np.clip(s, 0.0, 1.0) * n_levels) / n_levels
    ok = lev > 0 if valid is None else (lev > 0) & np.asarray(valid, dtype=bool)
    runs, i, n = [], 0, s.size
    while i < n:
        if not ok[i]:
            i += 1
            continue
        j = i + 1
        while j < n and ok[j] and lev[j] == lev[i]:
            j += 1
        runs.append((i, j, float(lev[i])))
        i = j
    return runs


def _strip_polys_2d(xa, ya, xb, yb, runs):
    return [
        np.column_stack([
            np.concatenate([xa[i0:i1 + 1], xb[i0:i1 + 1][::-1]]),
            np.concatenate([ya[i0:i1 + 1], yb[i0:i1 + 1][::-1]]),
        ])
        for i0, i1, _ in runs
    ]


def _polylines_2d(x, y, runs):
    return [np.column_stack([x[i0:i1 + 1], y[i0:i1 + 1]]) for i0, i1, _ in runs]
```

```
def draw_panelA_ribbon(ax, theta_lo: float, theta_hi: float, theta_mid: float, color: str):
    """
    Panel-A highlight (edge-on chord): same vertical strip between
    x=cos(θ_hi)..cos(θ_lo) clipped to the unit circle, base alpha 0.28, and the
    same foreshortened mid-ellipse (lw 2.8, alpha 0.95) — now with the opacity
    gradient along the chord: alpha *= (1 + sgn·y/ρ(x))/2 = (1+cos psi)/2,
    darkest at the θ-ray end, transparent at the opposite end.
    """
    mu_lo = float(np.cos(np.deg2rad(theta_lo)))
    mu_hi = float(np.cos(np.deg2rad(theta_hi)))
    x0, x1 = (min(mu_lo, mu_hi), max(mu_lo, mu_hi))
    sgn = theta_ray_sign(theta_mid)
    nx, ny = 64, 1200
    xs = np.linspace(x0, x1, nx)
    ys = np.linspace(-1.0, 1.0, ny)
    X, Y = np.meshgrid(xs, ys)
    rho = np.sqrt(np.clip(1.0 - X * X, 1e-6, None))
    sc = np.clip(0.5 * (1.0 + sgn * Y / rho), 0.0, 1.0)
    img = np.zeros((ny, nx, 4))
    img[..., :3] = to_rgba(color)[:3]
    img[..., 3] = 0.28 * sc
    im = ax.imshow(img, extent=(x0, x1, -1.0, 1.0), origin="lower",
                   interpolation="bilinear", zorder=2, aspect="auto")
    im.set_clip_path(Circle((0.0, 0.0), 1.0, transform=ax.transData))

    mu_mid = float(np.cos(np.deg2rad(theta_mid)))
    rho_m = np.sqrt(max(0.0, 1.0 - mu_mid * mu_mid))
    phi0, phi = grad_phi(theta_mid, 721)
    ex_m = mu_mid + 0.045 * rho_m * np.cos(phi)
    ey_m = rho_m * np.sin(phi)
    runs = quantized_runs(segment_scale(grad_scale(phi, phi0)))
    ax.add_collection(
        LineCollection(
            _polylines_2d(ex_m, ey_m, runs),
            colors=rgba_array(color, [0.95 * r[2] for r in runs]),
            linewidths=2.8, zorder=4, capstyle="butt", joinstyle="round",
        )
    )


def draw_panelA_theta_rays(ax, geo):
    """
    Illustrate θ: from the observer (origin), one ray per loop out to the
    ring at that loop's center θ. θ is measured from +x (anti-CoE, right)
    up to the ring. Emphasize MAX θ=30° (thick semi-transparent red to the
    top of that foreshortened ring), matching Chris's markup.
    """
    items = list(zip(geo["theta_max_from_plus_x_deg"], COLORS_MAX)) + list(
        zip(geo["theta_min_from_plus_x_deg"], COLORS_MIN)
    )
    for mid, color in items:
        t = float(np.deg2rad(mid))
```

```
        # Endpoint = top/rim point of the ring in side view = (cos θ, sin θ)
        x1, y1 = float(np.cos(t)), float(np.sin(t))
        is_max30 = abs(mid - 29.5) < 0.5
        lw = 3.2 if is_max30 else 1.8
        alpha = 0.70 if is_max30 else 0.45
        ax.plot(
            [0.0, x1], [0.0, y1],
            color=color, lw=lw, alpha=alpha, zorder=3,
            solid_capstyle="round",
        )
        # Small tick at the ring hit
        ax.plot([x1], [y1], "o", color=color, ms=4.5 if is_max30 else 3.5,
                alpha=0.9, zorder=5, markeredgecolor="none")

    # Angle arc at θ=30° from +x axis up to the ray (makes the measure obvious)
    arc_r = 0.38
    phi = np.linspace(0.0, np.deg2rad(29.5), 60)
    ax.plot(
        arc_r * np.cos(phi), arc_r * np.sin(phi),
        color=C_MAX_30, lw=1.6, alpha=0.85, zorder=3,
    )
    ax.text(
        arc_r * np.cos(np.deg2rad(15.0)) + 0.06,
        arc_r * np.sin(np.deg2rad(15.0)) + 0.04,
        r"$\theta$",
        color=C_MAX_30, fontsize=10, fontweight="bold",
        ha="left", va="bottom", zorder=6,
    )


def panel_A_model_frame(ax, geo):
    """Side view: exactly four thick foreshortened loop ribbons about x."""
    ax.set_title(
        "A. Model frame (no Galactic yet)\n"
        r"axisymmetric about $x$ = CoE$\leftrightarrow$anti-CoE"
        "\n"
        r"two MAX + two MIN, each $18^\circ$ end-to-end ($\pm9^\circ$)",
        fontsize=10,
        pad=8,
    )
    th = np.linspace(0, 2 * np.pi, 400)
    ax.plot(np.cos(th), np.sin(th), color="#7f8c8d", lw=1.2, zorder=1)
    ax.plot(0, 0, "ko", ms=6, zorder=5)
    ax.text(0.08, -0.12, "observer", fontsize=8, color="k", ha="left")

    ax.annotate(
        "",
        xy=(1.15, 0),
        xytext=(-1.15, 0),
        arrowprops=dict(arrowstyle="<->", color=C_AXIS, lw=2.5),
        zorder=4,
    )
    ax.text(1.18, 0.08, "anti-CoE\n(+x)", color=C_ANTI, fontsize=8, ha="left", va="bottom")
    ax.text(-1.18, 0.08, "CoE\n(−x)", color=C_COE, fontsize=8, ha="right", va="bottom")
```

```
    for (lo, hi), mid, color in zip(
        geo["max_bands_deg"], geo["theta_max_from_plus_x_deg"], COLORS_MAX
    ):
        draw_panelA_ribbon(ax, lo, hi, mid, color)
    for (lo, hi), mid, color in zip(
        geo["min_bands_deg"], geo["theta_min_from_plus_x_deg"], COLORS_MIN
    ):
        draw_panelA_ribbon(ax, lo, hi, mid, color)

    # Illustrate θ from +x (anti-CoE) to each loop (Chris markup)
    draw_panelA_theta_rays(ax, geo)

    label_r = 1.30
    for mid, color in list(zip(geo["theta_max_from_plus_x_deg"], COLORS_MAX)) + list(
        zip(geo["theta_min_from_plus_x_deg"], COLORS_MIN)
    ):
        t = np.deg2rad(mid)
        ax.text(
            label_r * np.cos(t),
            label_r * np.sin(t),
            rf"$\theta\!=\!{mid:g}^\circ$",
            color=color, fontsize=7.5, ha="center", va="center", fontweight="bold",
        )

    ax.text(
        0.0,
        -1.38,
        (
            r"four transparent highlight bands about $x$  ·  each $18^\circ$ end-to-end ($\pm9^\circ$ = 5% of $360^\circ$)"
            + "\n"
            + r"(after-2$\ell$ $A(\ell)$: two MAX + two MIN; one mid-stroke each)"
            + "\n"
            + r"$y,z$ free (roll) — model does not pick celestial equator"
        ),
        fontsize=7.5,
        ha="center",
        va="top",
        style="italic",
        color="#34495e",
    )
    ax.set_xlim(-1.55, 1.55)
    ax.set_ylim(-1.55, 1.45)
    ax.set_aspect("equal")
    ax.axis("off")


def _ring_lb(D, mu, n_pts=721):
    pts = small_circle_about_axis(D, mu, n=n_pts)
    rl = np.array([n_to_lb(p)[0] for p in pts])
    rb = np.array([n_to_lb(p)[1] for p in pts])
    return rl, rb
```

```
def _ring_lonlat_rad(D, mu, n_pts=361):
    """Small-circle → Mollweide display (lon_disp, lat) in radians."""
    pts = small_circle_about_axis(D, float(mu), n=n_pts)
    ls = np.array([n_to_lb(p)[0] for p in pts])
    bs = np.array([n_to_lb(p)[1] for p in pts])
    lon = np.deg2rad(lon_disp_from_ell(ls))
    lat = np.deg2rad(bs)
    return lon, lat


def fill_mollweide_ribbon(ax, D, theta_lo, theta_hi, theta_mid, color, n_pts=721):
    """
    Mollweide highlight band (±9° edges, base alpha 0.26) + mid-stroke
    (lw 2.6, alpha 0.95), both with the (1+cos psi)/2 opacity gradient.
    Elements crossing the antimeridian seam are dropped.
    """
    mu_a, mu_b = mu_pair_for_band(theta_lo, theta_hi)
    mu_mid = float(np.cos(np.deg2rad(theta_mid)))

    def lonlat(mu, n):
        pts, sc = small_circle_grad(D, mu, theta_mid, n=n)
        ls = np.array([n_to_lb(p)[0] for p in pts])
        bs = np.array([n_to_lb(p)[1] for p in pts])
        return np.deg2rad(lon_disp_from_ell(ls)), np.deg2rad(bs), sc

    lon_a, lat_a, sc = lonlat(mu_a, n_pts)
    lon_b, lat_b, _ = lonlat(mu_b, n_pts)
    gap = 0.9 * np.pi
    quad = np.stack([lon_a[:-1], lon_a[1:], lon_b[1:], lon_b[:-1]])
    valid = (quad.max(axis=0) - quad.min(axis=0)) <= gap
    runs = quantized_runs(segment_scale(sc), valid)
    if runs:
        ax.add_collection(
            PolyCollection(
                _strip_polys_2d(lon_a, lat_a, lon_b, lat_b, runs),
                facecolors=rgba_array(color, [0.26 * r[2] for r in runs]),
                edgecolors="none", zorder=2, linewidths=0,
            )
        )
    lon_m, lat_m, sc_m = lonlat(mu_mid, max(n_pts, 721))
    valid = np.abs(np.diff(lon_m)) <= gap
    runs = quantized_runs(segment_scale(sc_m), valid)
    if runs:
        ax.add_collection(
            LineCollection(
                _polylines_2d(lon_m, lat_m, runs),
                colors=rgba_array(color, [0.95 * r[2] for r in runs]),
                linewidths=2.6, zorder=4, capstyle="butt", joinstyle="round",
            )
        )


def panel_B_mollweide(ax, geo):
    ax.set_title(
```

```
        r"B. Migkas $A(\ell)$ loops on sky (anti-CoE at $(0^\circ,+15^\circ)$)"
        "\n"
        r"(Galactic Mollweide; CoE axis tilted $+15^\circ$; $\ell$ increases LEFT; "
        r"two MAX + two MIN, each $18^\circ$ end-to-end ($\pm9^\circ$))",
        fontsize=10,
        pad=8,
    )
    for b in [-60, -30, 0, 30, 60]:
        ell = np.linspace(0, 360, 361)
        plot_lb_curve_mollweide(ax, ell, np.full_like(ell, b), color="#bdc3c7", lw=0.5, zorder=0)
    for ell0 in range(0, 360, 30):
        b = np.linspace(-90, 90, 181)
        plot_lb_curve_mollweide(ax, np.full_like(b, ell0), b, color="#bdc3c7", lw=0.5, zorder=0)

    D = geo["D"]
    ls, bs = great_circle_through_poles(D, n=721)
    plot_lb_curve_mollweide(ax, ls, bs, color=C_AXIS, lw=2.0, zorder=2)

    for (lo, hi), mid, color in zip(
        geo["max_bands_deg"], geo["theta_max_from_plus_x_deg"], COLORS_MAX
    ):
        fill_mollweide_ribbon(ax, D, lo, hi, mid, color)
    for (lo, hi), mid, color in zip(
        geo["min_bands_deg"], geo["theta_min_from_plus_x_deg"], COLORS_MIN
    ):
        fill_mollweide_ribbon(ax, D, lo, hi, mid, color)

    ell_eq, b_eq = celestial_equator_lb(721)
    plot_lb_curve_mollweide(ax, ell_eq, b_eq, color=C_EQ, lw=1.8, ls="--", zorder=2)

    ax.plot(
        np.deg2rad(lon_disp_from_ell(geo["anti_lb"][0])),
        np.deg2rad(geo["anti_lb"][1]),
        marker="o", ms=8, color=C_ANTI,
        markerfacecolor=C_ANTI, markeredgecolor="k", markeredgewidth=0.6,
        zorder=6, linestyle="None",
    )
    ax.plot(
        np.deg2rad(lon_disp_from_ell(geo["coe_lb"][0])),
        np.deg2rad(geo["coe_lb"][1]),
        marker="*", ms=14, color=C_COE,
        markerfacecolor=C_COE, markeredgecolor="k", markeredgewidth=0.6,
        zorder=6, linestyle="None",
    )
    for (l, b), color in zip(MIGKAS_MAXES, COLORS_MAX):
        ax.plot(
            np.deg2rad(lon_disp_from_ell(l)),
            np.deg2rad(b),
            marker="^",
            ms=10,
            color=color,
            markeredgecolor="k",
            markeredgewidth=0.7,
            zorder=7,
```

```
            linestyle="None",
        )
    for (l, b), color in zip(MIGKAS_MINS, COLORS_MIN):
        ax.plot(
            np.deg2rad(lon_disp_from_ell(l)),
            np.deg2rad(b),
            marker="s",
            ms=10,
            color=color,
            markeredgecolor="k",
            markeredgewidth=0.7,
            zorder=7,
            linestyle="None",
        )

    xticks = np.deg2rad(lon_disp_from_ell(np.array([180, 90, 0, 270])))
    ax.set_xticks(xticks)
    ax.set_xticklabels([r"$180^\circ$", r"$90^\circ$", r"$0^\circ$", r"$270^\circ$"], fontsize=8)
    ax.set_yticks(np.deg2rad([-60, -30, 0, 30, 60]))
    ax.set_yticklabels([r"$-60^\circ$", r"$-30^\circ$", r"$0^\circ$", r"$+30^\circ$", r"$+60^\circ$"], fontsize=8)
    ax.grid(False)
    ax.set_xlabel(r"Galactic $\ell$ (increases LEFT)", fontsize=8)
    ax.set_ylabel(r"Galactic $b$", fontsize=8)


def fill_sphere_ribbon(ax, D, theta_lo, theta_hi, theta_mid, color, n_pts=160):
    """
    3D highlight belt (±9° edges, base alpha 0.25) + mid-stroke (lw 2.6,
    alpha 0.95), both with the (1+cos psi)/2 opacity gradient.
    """
    mu_a, mu_b = mu_pair_for_band(theta_lo, theta_hi)
    mu_mid = float(np.cos(np.deg2rad(theta_mid)))
    n_g = 361
    p0, sc = small_circle_grad(D, float(mu_a), theta_mid, n=n_g)
    p1, _ = small_circle_grad(D, float(mu_b), theta_mid, n=n_g)
    seg = segment_scale(sc)
    idx = np.where(seg > 1e-4)[0]
    verts = [[p0[j], p0[j + 1], p1[j + 1], p1[j]] for j in idx]
    coll = Poly3DCollection(
        verts, facecolors=rgba_array(color, 0.25 * seg[idx]),
        edgecolors="none", linewidths=0.0, zorder=2,
    )
    ax.add_collection3d(coll)
    pts, scm = small_circle_grad(D, mu_mid, theta_mid, n=n_g)
    runs = quantized_runs(segment_scale(scm))
    ax.add_collection3d(
        Line3DCollection(
            [pts[i0:i1 + 1] for i0, i1, _ in runs],
            colors=rgba_array(color, [0.95 * r[2] for r in runs]),
            linewidths=2.6, zorder=5, capstyle="butt", joinstyle="round",
        )
    )
```

```
def panel_C_sphere(ax, geo):
    th_max = geo["theta_max_from_plus_x_deg"]
    th_min = geo["theta_min_from_plus_x_deg"]
    ax.set_title(
        r"C. Sphere 3D — two MAX + two MIN, each $18^\circ$ end-to-end ($\pm9^\circ$)"
        "\n"
        r"(axis line emphasized; celestial equator dashed)",
        fontsize=10,
        pad=6,
    )
    draw_wire_sphere(ax)
    D = geo["D"]
    ell_eq, b_eq = celestial_equator_lb(400)
    eq_xyz = np.array([lb_to_n(l, b) for l, b in zip(ell_eq, b_eq)])
    ax.plot(eq_xyz[:, 0], eq_xyz[:, 1], eq_xyz[:, 2], color=C_EQ, lw=1.8, ls="--", zorder=3)
    ax.plot(
        [-D[0], D[0]],
        [-D[1], D[1]],
        [-D[2], D[2]],
        color=C_AXIS,
        lw=3.0,
        zorder=5,
    )
    ax.scatter(*(-D), color=C_COE, s=120, marker="*", edgecolors="k", linewidths=0.6,
               depthshade=False, zorder=8)
    ax.scatter(*(D), color=C_ANTI, s=80, edgecolors="k", linewidths=0.6, depthshade=False, zorder=8)
    ax.text(*(-D * 1.22), "CoE", color=C_COE, fontsize=8, fontweight="bold")
    ax.text(*(D * 1.18), "anti-CoE", color=C_ANTI, fontsize=8, fontweight="bold")
    ax.scatter([0], [0], [0], color="k", s=25, depthshade=False, zorder=9)

    for (lo, hi), mid, color in zip(
        geo["max_bands_deg"], geo["theta_max_from_plus_x_deg"], COLORS_MAX
    ):
        fill_sphere_ribbon(ax, D, lo, hi, mid, color)
    for (lo, hi), mid, color in zip(
        geo["min_bands_deg"], geo["theta_min_from_plus_x_deg"], COLORS_MIN
    ):
        fill_sphere_ribbon(ax, D, lo, hi, mid, color)

    for n, color in zip(geo["n_maxes"], COLORS_MAX):
        ax.scatter(
            [n[0]], [n[1]], [n[2]],
            c=[color], marker="^", s=70,
            edgecolors="k", linewidths=0.7, depthshade=False, zorder=10,
        )
        ax.plot([0, n[0]], [0, n[1]], [0, n[2]], color=color, lw=0.8, ls=":", alpha=0.7)
    for n, color in zip(geo["n_mins"], COLORS_MIN):
        ax.scatter(
            [n[0]], [n[1]], [n[2]],
            c=[color], marker="s", s=60,
            edgecolors="k", linewidths=0.7, depthshade=False, zorder=10,
        )
        ax.plot([0, n[0]], [0, n[1]], [0, n[2]], color=color, lw=0.8, ls=":", alpha=0.7)
```

```
    ax.set_xlim(-1.25, 1.25)
    ax.set_ylim(-1.25, 1.25)
    ax.set_zlim(-1.25, 1.25)
    ax.set_box_aspect((1, 1, 1))
    ax.view_init(elev=18, azim=-55)
    ax.set_axis_off()
    ax.text2D(
        0.5,
        -0.02,
        rf"MAX $\theta\!=\!{th_max[0]:g}^\circ,{th_max[1]:g}^\circ$; "
        rf"MIN $\theta\!=\!{th_min[0]:g}^\circ,{th_min[1]:g}^\circ$  "
        rf"($18^\circ$ end-to-end = $\pm9^\circ$ = 5% of $360^\circ$)",
        transform=ax.transAxes,
        ha="center",
        fontsize=7.5,
        color="#34495e",
    )


def main():
    st = migkas_scan_stops()
    geo = compute_geometry(st)

    th_max = geo["theta_max_from_plus_x_deg"]
    th_min = geo["theta_min_from_plus_x_deg"]
    max_bands = geo["max_bands_deg"]
    min_bands = geo["min_bands_deg"]

    fig = plt.figure(figsize=(14.5, 5.6), dpi=160)
    fig.suptitle(
        r"Pure CoE axisymmetry: two MAX + two MIN highlight bands  (gradient: darkest at $\theta$ ray, transparent opposite; +45$^\circ$ CCW about CoE
axis (B/C))"
        "\n"
        rf"(each $18^\circ$ end-to-end = $\pm9^\circ$ = 5% of $360^\circ$; "
        rf"MAX $\theta\!=\!{th_max[0]:g}^\circ/{th_max[1]:g}^\circ$; "
        rf"MIN $\theta\!=\!{th_min[0]:g}^\circ/{th_min[1]:g}^\circ$; "
        r"anti-CoE $(0^\circ,+15^\circ)$, CoE axis tilted $+15^\circ$)",
        fontsize=11,
        fontweight="bold",
        y=1.02,
    )

    axA = fig.add_subplot(1, 3, 1)
    panel_A_model_frame(axA, geo)

    axB = fig.add_subplot(1, 3, 2, projection="mollweide")
    panel_B_mollweide(axB, geo)

    axC = fig.add_subplot(1, 3, 3, projection="3d")
    panel_C_sphere(axC, geo)

    handles = [
        Line2D([0], [0], color=C_AXIS, lw=2.5, label=r"CoE$\leftrightarrow$anti-CoE axis"),
        Patch(
```

```
            facecolor=C_MAX_30, alpha=0.45, edgecolor=C_MAX_30,
            label=(
                rf"MAX $\theta\!=\!{th_max[0]:g}^\circ$ band "
                rf"[{max_bands[0][0]:g},{max_bands[0][1]:g}]$^\circ$ "
                rf"($18^\circ$=±$9^\circ$=5% of $360^\circ$)"
            ),
        ),
        Patch(
            facecolor=C_MAX_185, alpha=0.45, edgecolor=C_MAX_185,
            label=(
                rf"MAX $\theta\!=\!{th_max[1]:g}^\circ$ band "
                rf"[{max_bands[1][0]:g},{max_bands[1][1]:g}]$^\circ$ "
                rf"($18^\circ$=±$9^\circ$)"
            ),
        ),
        Patch(
            facecolor=C_MIN_115, alpha=0.45, edgecolor=C_MIN_115,
            label=(
                rf"MIN $\theta\!=\!{th_min[0]:g}^\circ$ band "
                rf"[{min_bands[0][0]:g},{min_bands[0][1]:g}]$^\circ$ "
                rf"($18^\circ$=±$9^\circ$)"
            ),
        ),
        Patch(
            facecolor=C_MIN_265, alpha=0.45, edgecolor=C_MIN_265,
            label=(
                rf"MIN $\theta\!=\!{th_min[1]:g}^\circ$ band "
                rf"[{min_bands[1][0]:g},{min_bands[1][1]:g}]$^\circ$ "
                rf"($18^\circ$=±$9^\circ$)"
            ),
        ),
        Line2D([0], [0], color=C_EQ, lw=1.8, ls="--", label="celestial equator (Dec$=0$)"),
        Line2D(
            [0], [0], marker="^", color="w", markerfacecolor=C_MAX_30, markeredgecolor="k",
            ms=9,
            label=rf"MAX $({MIGKAS_MAXES[0][0]:.0f}^\circ,{MIGKAS_MAXES[0][1]:+.0f}^\circ)$",
        ),
        Line2D(
            [0], [0], marker="^", color="w", markerfacecolor=C_MAX_185, markeredgecolor="k",
            ms=9,
            label=rf"MAX $({MIGKAS_MAXES[1][0]:.0f}^\circ,{MIGKAS_MAXES[1][1]:+.0f}^\circ)$",
        ),
        Line2D(
            [0], [0], marker="s", color="w", markerfacecolor=C_MIN_115, markeredgecolor="k",
            ms=8,
            label=rf"MIN $({MIGKAS_MINS[0][0]:.0f}^\circ,{MIGKAS_MINS[0][1]:+.0f}^\circ)$",
        ),
        Line2D(
            [0], [0], marker="s", color="w", markerfacecolor=C_MIN_265, markeredgecolor="k",
            ms=8,
            label=rf"MIN $({MIGKAS_MINS[1][0]:.0f}^\circ,{MIGKAS_MINS[1][1]:+.0f}^\circ)$",
        ),
        Line2D(
            [0], [0], marker="o", color="w", markerfacecolor=C_ANTI, markeredgecolor="k",
```

```
        ms=8, label=rf"anti-CoE $({geo['anti_lb'][0]:.1f}^\circ,{geo['anti_lb'][1]:+.1f}^\circ)$",
    ),
    Line2D(
        [0], [0], marker="*", color="w", markerfacecolor=C_COE, markeredgecolor="k",
        ms=11, label=rf"CoE $({geo['coe_lb'][0]:.1f}^\circ,{geo['coe_lb'][1]:+.1f}^\circ)$",
    ),
]
fig.legend(
    handles=handles,
    loc="lower center",
    ncol=6,
    fontsize=7.0,
    frameon=True,
    fancybox=True,
    bbox_to_anchor=(0.5, -0.10),
)

fig.tight_layout()
# --- Chris 2026-09-26: panel C sphere drawn at the same on-page diameter as
# panel A's model-frame circle; panel C axes matched to A's vertical
# position/height. Nothing else changes.
from mpl_toolkits.mplot3d import proj3d
pa, pc = axA.get_position(), axC.get_position()
axC.set_position([pc.x0, pa.y0, pc.width, pa.height])

def _diam_A():
    fig.canvas.draw()
    p0 = axA.transData.transform((-1.0, 0.0)); p1 = axA.transData.transform((1.0, 0.0))
    return float(p1[0] - p0[0])

def _diam_C():
    fig.canvas.draw()
    u = np.random.default_rng(0).normal(size=(20000, 3))
    u /= np.linalg.norm(u, axis=1)[:, None]
    xs, ys, _ = proj3d.proj_transform(u[:, 0], u[:, 1], u[:, 2], axC.get_proj())
    d = axC.transData.transform(np.column_stack([xs, ys]))
    return float(d[:, 0].max() - d[:, 0].min()), float(d[:, 1].max() - d[:, 1].min())

zoom = 1.0
for _ in range(8):
    dA = _diam_A(); dCx, dCy = _diam_C()
    zoom *= dA / dCx
    axC.set_box_aspect((1, 1, 1), zoom=zoom)
dA = _diam_A(); dCx, dCy = _diam_C()
print(f"PANEL_DIAM A={dA:.1f}px C={dCx:.1f}x{dCy:.1f}px zoom={zoom:.3f}")
fig.savefig(PNG_PATH, bbox_inches="tight", facecolor="white")
plt.close(fig)

payload = {
    "png": str(PNG_PATH),
    "script": str(OUT_DIR / "make_coe_rings_migkas_pin_axis_matched_grad_rot45.py"),
    "migkas_max_lb_deg": [list(lb) for lb in MIGKAS_MAXES],
    "migkas_min_lb_deg": [list(lb) for lb in MIGKAS_MINS],
    "strip_theta_max_deg": list(STRIP_THETA_MAX),
```

```
        "strip_theta_min_deg": list(STRIP_THETA_MIN),
        "band_half_width_deg": BAND_HALF,
        "band_full_width_deg": BAND_FULL,
        "band_total_width_deg": BAND_TOTAL,
        "band_width_note": "5% of 360° = 18° end-to-end = ±9° about each MAX/MIN center",
        "max_bands_deg": [list(b) for b in max_bands],
        "min_bands_deg": [list(b) for b in min_bands],
        "band_intervals": {
            "MAX_30": [float(max_bands[0][0]), float(max_bands[0][1])],
            "MAX_185": [float(max_bands[1][0]), float(max_bands[1][1])],
            "MIN_115": [float(min_bands[0][0]), float(min_bands[0][1])],
            "MIN_265": [float(min_bands[1][0]), float(min_bands[1][1])],
        },
        "scan_stop_source": st["source"],
        "theta_max_from_plus_x_deg": [round(t, 4) for t in th_max],
        "theta_min_from_plus_x_deg": [round(t, 4) for t in th_min],
        "mu_max_bands": [[round(a, 8), round(b, 8)] for a, b in geo["mu_max_bands"]],
        "mu_min_bands": [[round(a, 8), round(b, 8)] for a, b in geo["mu_min_bands"]],
        "CoE_lb_deg": [round(geo["coe_lb"][0], 4), round(geo["coe_lb"][1], 4)],
        "anti_CoE_lb_deg": [round(geo["anti_lb"][0], 4), round(geo["anti_lb"][1], 4)],
        "D_hat_galactic_xyz": geo["D"].tolist(),
        "angular_sep_migkas_max0_min1_deg": round(geo["sep_deg"], 6),
        "axis_tilt_b_deg": 15,
        "axis_tilt_reason": (
            "Chris 2026-09-22 — markers fixed, rings rotate up 30°"
        ),
        "orientation": (
            "Galactic orientation: anti-CoE at (ell=0°, b=+15°), CoE at "
            "(ell=180°, b=−15°); +D = anti-CoE = +x; axis tilted +15° "
            "(Chris 2026-09-22 — markers fixed, rings rotate up 30°)"
        ),
        "drawing": (
            "Exactly 4 features. Each = ONE smooth transparent highlight band "
            "between θ−9° and θ+9° (18° end-to-end = 5% of 360°) + ONE solid "
            "mid-stroke at the exact center θ. No multi-stroke wash / hairball. "
            "Four distinct colors (two reds + two blues); no orange; no 283° "
            "companion. Panel A: clean vertical strip clipped to unit circle; theta rays from observer to each loop (emphasized at MAX 30°). Mollweide
"
            "seam-split honestly. Sphere: single Poly3D strip (no n_wash stack)."
        ),
        "note": (
            "Stops are after-2ℓ A(ℓ) extrema from best_fit_2ell_multiplier.npz "
            f"A_after: two MAX at θ={th_max[0]:g}°/{th_max[1]:g}° and "
            f"two MIN at θ={th_min[0]:g}°/{th_min[1]:g}°, each drawn as "
            f"one {BAND_FULL:.0f}° end-to-end highlight (±{BAND_HALF:.0f}° = 5% of 360°). "
            "Rings use μ=cos(θ); sky markers are map extrema (17°,−9°) and "
            "(274°,−22°) plus strip stand-ins (185°,0°) and (115°,0°)."
        ),
        "plot_convention": "Migkas Galactic Mollweide: lon_disp=-ell_wrapped (ell increases LEFT)",
        "legend_summary": (
            "two MAX + two MIN highlight bands, each 18° end-to-end "
            "(±9° = 5% of 360°); four distinct colors"
        ),
        "colors": {
```

```
            "MAX_30": C_MAX_30,
            "MAX_185": C_MAX_185,
            "MIN_115": C_MIN_115,
            "MIN_265": C_MIN_265,
            "no_orange": True,
        },
    }
    JSON_PATH.write_text(json.dumps(payload, indent=2) + "\n")
    print(json.dumps(payload, indent=2))
    print(f"\nWrote {PNG_PATH}")
    print(f"Wrote {JSON_PATH}")


if __name__ == "__main__":
    main()
```

```
PANEL_DIAM A=367.9px C=367.9x368.0px zoom=1.399
```

```
{
  "png": "/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/loop_threepanel_matched_grad_rot45.png",
  "script": "/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/make_coe_rings_migkas_pin_axis_matched_grad_rot45.py",
  "migkas_max_lb_deg": [
    [
      17.0,
      -9.0
    ],
    [
      185.0,
      0.0
    ]
  ],
  "migkas_min_lb_deg": [
    [
      115.0,
      0.0
    ],
    [
      274.0,
      -22.0
    ]
  ],
  "strip_theta_max_deg": [
    29.5,
    187.5
  ],
  "strip_theta_min_deg": [
    122.6,
    264.4
  ],
  "band_half_width_deg": 9.0,
  "band_full_width_deg": 18.0,
  "band_total_width_deg": 18.0,
  "band_width_note": "5% of 360\u00b0 = 18\u00b0 end-to-end = \u00b19\u00b0 about each MAX/MIN center",
  "max_bands_deg": [
    [
      20.5,
      38.5
    ],
    [
      178.5,
      196.5
    ]
  ],
  "min_bands_deg": [
    [
      113.6,
      131.6
    ],
    [
      255.39999999999998,
      273.4
    ]
  ],
  "band_intervals": {
    "MAX_30": [
```

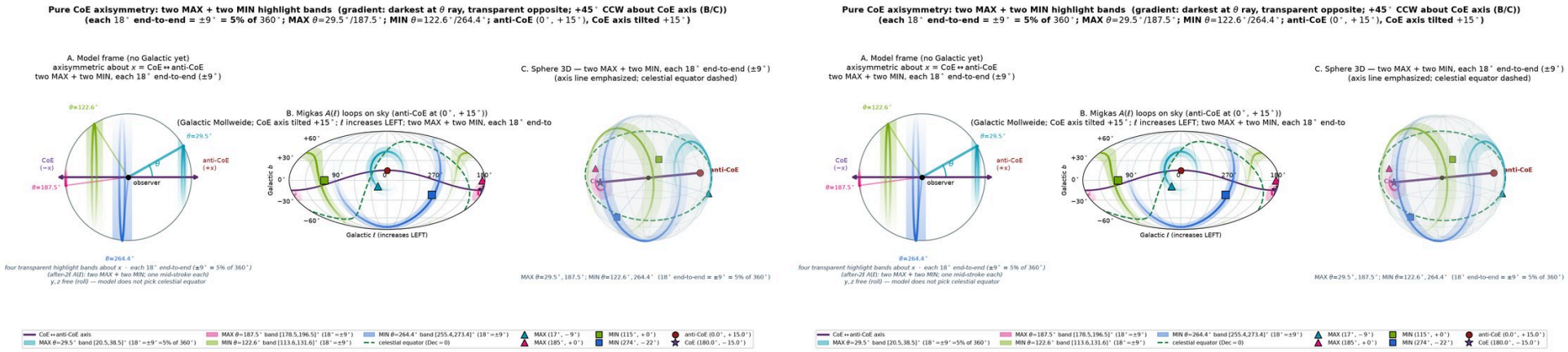
```
      20.5,
      38.5
    ],
    "MAX_185": [
      178.5,
      196.5
    ],
    "MIN_115": [
      113.6,
      131.6
    ],
    "MIN_265": [
      255.39999999999998,
      273.4
    ]
  },
  "scan_stop_source": "best_fit_2ell_multiplier.npz A_after extrema (two max, two min); each drawn as one transparent 18\u00b0 end-to-end highlight
(\u00b19\u00b0 = 5% of 360\u00b0) + one solid mid-stroke",
  "theta_max_from_plus_x_deg": [
    29.5,
    187.5
  ],
  "theta_min_from_plus_x_deg": [
    122.6,
    264.4
  ],
  "mu_max_bands": [
    [
      0.93667219,
      0.78260816
    ],
    [
      -0.99965732,
      -0.95881973
    ]
  ],
  "mu_min_bands": [
    [
      -0.40034903,
      -0.66392621
    ],
    [
      -0.25206936,
      0.05930637
    ]
  ],
  "CoE_lb_deg": [
    180.0,
    -15.0
  ],
  "anti_CoE_lb_deg": [
    0.0,
    15.0
  ],
  "D_hat_galactic_xyz": [
    0.9659258262890683,
    0.0,
```

```
    0.25881904510252074
  ],
  "angular_sep_migkas_max0_min1_deg": 98.476384,
  "axis_tilt_b_deg": 15,
  "axis_tilt_reason": "Chris 2026-09-22 \u2014 markers fixed, rings rotate up 30\u00b0",
  "orientation": "Galactic orientation: anti-CoE at (ell=0\u00b0, b=+15\u00b0), CoE at (ell=180\u00b0, b=\u221215\u00b0); +D = anti-CoE = +x; axis
tilted +15\u00b0 (Chris 2026-09-22 \u2014 markers fixed, rings rotate up 30\u00b0)",
  "drawing": "Exactly 4 features. Each = ONE smooth transparent highlight band between \u03b8\u22129\u00b0 and \u03b8+9\u00b0 (18\u00b0 end-to-end = 5%
of 360\u00b0) + ONE solid mid-stroke at the exact center \u03b8. No multi-stroke wash / hairball. Four distinct colors (two reds + two blues); no
orange; no 283\u00b0 companion. Panel A: clean vertical strip clipped to unit circle; theta rays from observer to each loop (emphasized at MAX
30\u00b0). Mollweide seam-split honestly. Sphere: single Poly3D strip (no n_wash stack).",
  "note": "Stops are after-2\u2113 A(\u2113) extrema from best_fit_2ell_multiplier.npz A_after: two MAX at \u03b8=29.5\u00b0/187.5\u00b0 and two MIN at
\u03b8=122.6\u00b0/264.4\u00b0, each drawn as one 18\u00b0 end-to-end highlight (\u00b19\u00b0 = 5% of 360\u00b0). Rings use \u03bc=cos(\u03b8); sky
markers are map extrema (17\u00b0,\u22129\u00b0) and (274\u00b0,\u221222\u00b0) plus strip stand-ins (185\u00b0,0\u00b0) and (115\u00b0,0\u00b0).",
  "plot_convention": "Migkas Galactic Mollweide: lon_disp=-ell_wrapped (ell increases LEFT)",
  "legend_summary": "two MAX + two MIN highlight bands, each 18\u00b0 end-to-end (\u00b19\u00b0 = 5% of 360\u00b0); four distinct colors",
  "colors": {
    "MAX_30": "#0097b2",
    "MAX_185": "#e0107a",
    "MIN_115": "#6fa800",
    "MIN_265": "#2166d4",
    "no_orange": true
  }
}

Wrote /workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/loop_threepanel_matched_grad_rot45.png
Wrote /workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/loop_threepanel_matched_grad_rot45.json
```


```
compare(FIGS / 'loop_threepanel_matched_grad_rot45.png', '/workspace/cosmology/papers/spreadsheet_model/loop_threepanel_matched_grad_rot45.png')
```

```
loop_threepanel_matched_grad_rot45.png: new (2262, 1027) vs ref (2262, 1027) | mean |diff| = 0.000/255, pixels differing >40: 0.000% -> identical
left: approved reference papers/spreadsheet_model/loop_threepanel_matched_grad_rot45.png   |   right: notebook re-run
steps_v3/figs/loop_threepanel_matched_grad_rot45.png
```



Step 16d — Matched three-panel, gradient, fitted 80.8° roll

What it shows. Step 16b with the roll set to the fitted 80.8°: the roll that brings the loops' θ-ray points closest to the Migkas markers.

Source script: `/workspace/cosmology/figures/make_coe_rings_migkas_pin_axis_matched_grad_rot_fit.py` (last modified 2026-09-26 22:54 CT), run verbatim. *Output-path edit:* `/workspace/cosmology/papers/spreadsheet_model/loop_threepanel_matched_` → `/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/loop_threep`. *Reference image:* `papers/spreadsheet_model/loop_threepanel_matched_grad_rotfit_rot81.png`.

```
%%runscript make_coe_rings_migkas_pin_axis_matched_grad_rot_fit.py
#!/usr/bin/env python3
"""
CoE rings + Migkas A(ell) extrema loops in the Galactic frame.

Concept:
  Pure CoE axisymmetry → constant A on small-circle rings about x = CoE↔anti-CoE.
  Scan A vs angle from +x (anti-CoE) around 360°. After-2ℓ A(ℓ) has TWO maxima
  and TWO minima (from best_fit_2ell_multiplier.npz A_after):
    MAX at θ = 30° and 185° from +x,
    MIN at θ = 115° and 265° from +x.

  Exactly FOUR loops — each ONE transparent highlight band, 18° end-to-end
  (= ±9° = 5% of 360°) about the center θ:
    MAX: 30±9 → [21°, 39°] and 185±9 → [176°, 194°]
    MIN: 115±9 → [106°, 124°] and 265±9 → [256°, 274°]
  Drawing: ONE smooth transparent filled ribbon between θ−9° and θ+9°
  + ONE solid mid-stroke at the exact center θ. No multi-stroke wash,
  no dense linspace hairball, no dual strong boundaries.
  Four distinct colors (two reds + two blues). No orange, no 283° companion.

  Galactic orientation: anti-CoE (+x) at ℓ=0°, b=+15°; CoE (−x) at ℓ=180°, b=−15°
  (axis tilted +15° so features near ℓ=0 move north on Mollweide; Migkas markers fixed).
  Sky markers (ℓ, b):
    Maxes: map extrema (17°, −9°) for ~30° max; strip (185°, 0°) for second max.
    Mins:  strip (115°, 0°) for ~115° min; map extrema (274°, −22°) for ~265° min.

Outputs:
  coe_rings_migkas_pin_axis.png
  coe_rings_migkas_pin_axis.json
"""

from __future__ import annotations

import json
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
from matplotlib.lines import Line2D
from matplotlib.collections import PolyCollection
from matplotlib.patches import Circle, Patch, Polygon
from mpl_toolkits.mplot3d import Axes3D  # noqa: F401
from mpl_toolkits.mplot3d.art3d import Poly3DCollection
```

```
OUT_DIR = Path(__file__).resolve().parent
PNG_PATH = Path("/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/loop_threepanel_matched_grad_rotfit_rot81.png")
JSON_PATH = Path("/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/loop_threepanel_matched_grad_rotfit_rot81.json")

# 5% of 360° = 18° end-to-end → half-width ±9° about each center
BAND_HALF = 9.0
BAND_FULL = 2.0 * BAND_HALF  # 18° = 5% of 360°
BAND_TOTAL = BAND_FULL  # alias
STRIP_THETA_MAX = [29.5, 187.5]
STRIP_THETA_MIN = [122.6, 264.4]

MIGKAS_MAXES = [(17.0, -9.0), (185.0, 0.0)]
MIGKAS_MINS = [(115.0, 0.0), (274.0, -22.0)]

C_COE = "#6b3fa0"
C_ANTI = "#8e1b1b"  # dark red marker; distinct from loop reds
# Four distinct loop colors: two reds (MAX) + two blues (MIN); no orange/yellow
C_MAX_30 = "#0097b2"    # cyan — MAX at 29.5°
C_MAX_185 = "#e0107a"   # magenta — MAX at 187.5°
C_MIN_115 = "#6fa800"   # green — MIN at 122.6°
C_MIN_265 = "#2166d4"   # blue — MIN at 264.4°
C_AXIS = "#5b2c6f"
C_EQ = "#1e8449"

# Per-loop color maps (order matches STRIP_THETA_MAX / STRIP_THETA_MIN)
COLORS_MAX = [C_MAX_30, C_MAX_185]
COLORS_MIN = [C_MIN_115, C_MIN_265]


def wrap_lon_deg(lon: np.ndarray) -> np.ndarray:
    return ((np.asarray(lon, dtype=float) + 180.0) % 360.0) - 180.0


def lon_disp_from_ell(ell_deg: np.ndarray) -> np.ndarray:
    """Migkas Galactic: ℓ increases LEFT → lon_disp = −ℓ_wrapped."""
    return -wrap_lon_deg(ell_deg)


def lb_to_n(l_deg: float, b_deg: float) -> np.ndarray:
    l = np.deg2rad(l_deg)
    b = np.deg2rad(b_deg)
    return np.array(
        [np.cos(b) * np.cos(l), np.cos(b) * np.sin(l), np.sin(b)], dtype=float
    )


def n_to_lb(n: np.ndarray) -> tuple[float, float]:
    n = np.asarray(n, dtype=float)
    n = n / np.linalg.norm(n)
    b = float(np.rad2deg(np.arcsin(np.clip(n[2], -1.0, 1.0))))
    l = float(np.rad2deg(np.arctan2(n[1], n[0])) % 360.0)
    return l, b
```

```
def celestial_equator_lb(n: int = 720) -> tuple[np.ndarray, np.ndarray]:
    """Dec=0 (ICRS) as Galactic (ℓ, b). IAU J2000 NGP convention, no astropy."""
    ra_ngp = np.deg2rad(192.85948)
    dec_ngp = np.deg2rad(27.12825)
    l_ncp = np.deg2rad(122.93192)
    ra = np.linspace(0.0, 2.0 * np.pi, n, endpoint=False)
    dec = 0.0
    sin_b = np.sin(dec) * np.sin(dec_ngp) + np.cos(dec) * np.cos(dec_ngp) * np.cos(ra - ra_ngp)
    sin_b = np.clip(sin_b, -1.0, 1.0)
    b = np.arcsin(sin_b)
    y = np.cos(dec) * np.sin(ra - ra_ngp)
    x = np.sin(dec) * np.cos(dec_ngp) - np.cos(dec) * np.sin(dec_ngp) * np.cos(ra - ra_ngp)
    l = (l_ncp - np.arctan2(y, x)) % (2.0 * np.pi)
    return np.rad2deg(l), np.rad2deg(b)


def great_circle_through_poles(D: np.ndarray, n: int = 361) -> tuple[np.ndarray, np.ndarray]:
    """Great circle through ±D (any meridian of the CoE axis)."""
    a = D / np.linalg.norm(D)
    tmp = np.array([0.0, 0.0, 1.0])
    if abs(np.dot(a, tmp)) > 0.9:
        tmp = np.array([1.0, 0.0, 0.0])
    e2 = np.cross(a, tmp)
    e2 /= np.linalg.norm(e2)
    t = np.linspace(0.0, 2.0 * np.pi, n)
    pts = np.outer(np.cos(t), a) + np.outer(np.sin(t), e2)
    ls = np.array([n_to_lb(p)[0] for p in pts])
    bs = np.array([n_to_lb(p)[1] for p in pts])
    return ls, bs


def small_circle_about_axis(axis: np.ndarray, mu: float, n: int = 361) -> np.ndarray:
    """Unit vectors on the small circle n·axis = mu (|mu|<1)."""
    a = axis / np.linalg.norm(axis)
    mu = float(np.clip(mu, -1.0 + 1e-9, 1.0 - 1e-9))
    tmp = np.array([0.0, 0.0, 1.0]) if abs(a[2]) < 0.9 else np.array([1.0, 0.0, 0.0])
    e1 = np.cross(a, tmp)
    e1 /= np.linalg.norm(e1)
    e2 = np.cross(a, e1)
    rho = np.sqrt(max(0.0, 1.0 - mu * mu))
    t = np.linspace(0.0, 2.0 * np.pi, n)
    return (mu * a)[None, :] + rho * (np.outer(np.cos(t), e1) + np.outer(np.sin(t), e2))


def split_lon_segments(lon_rad: np.ndarray, lat_rad: np.ndarray, gap: float = np.pi):
    """Split polyline where Mollweide lon jumps (wrap discontinuity)."""
    d = np.diff(lon_rad)
    cuts = np.where(np.abs(d) > gap)[0] + 1
    idx = np.concatenate([[0], cuts, [len(lon_rad)]])
    segs = []
    for i0, i1 in zip(idx[:-1], idx[1:]):
        if i1 - i0 >= 2:
            segs.append((lon_rad[i0:i1], lat_rad[i0:i1]))
    return segs
```

```
def plot_lb_curve_mollweide(ax, l_deg, b_deg, **kwargs):
    lon = np.deg2rad(lon_disp_from_ell(l_deg))
    lat = np.deg2rad(b_deg)
    for xr, yr in split_lon_segments(lon, lat):
        ax.plot(xr, yr, **kwargs)


def band_edges(centers: list[float], half: float = BAND_HALF) -> list[tuple[float, float]]:
    """Return (theta_lo, theta_hi) for each band center (±half about center)."""
    return [(c - half, c + half) for c in centers]


def mu_pair_for_band(lo: float, hi: float) -> tuple[float, float]:
    """Edge μ=cos(θ) for a scan-angle band (two boundary cones only)."""
    return float(np.cos(np.deg2rad(lo))), float(np.cos(np.deg2rad(hi)))


def migkas_scan_stops() -> dict:
    th_max = [float(t) for t in STRIP_THETA_MAX]
    th_min = [float(t) for t in STRIP_THETA_MIN]
    max_bands = band_edges(th_max)
    min_bands = band_edges(th_min)
    return {
        "theta_max_from_plus_x_deg": th_max,
        "theta_min_from_plus_x_deg": th_min,
        "max_bands_deg": max_bands,
        "min_bands_deg": min_bands,
        "band_half_width_deg": BAND_HALF,
        "band_full_width_deg": BAND_FULL,
        "band_total_width_deg": BAND_TOTAL,
        "band_width_note": "5% of 360° = 18° end-to-end = ±9° about each MAX/MIN center",
        "mu_max_centers": [float(np.cos(np.deg2rad(t))) for t in th_max],
        "mu_min_centers": [float(np.cos(np.deg2rad(t))) for t in th_min],
        "mu_max_bands": [mu_pair_for_band(lo, hi) for lo, hi in max_bands],
        "mu_min_bands": [mu_pair_for_band(lo, hi) for lo, hi in min_bands],
        "source": (
            "best_fit_2ell_multiplier.npz A_after extrema (two max, two min); "
            f"each drawn as one transparent {BAND_FULL:.0f}° end-to-end highlight "
            f"(±{BAND_HALF:.0f}° = 5% of 360°) + one solid mid-stroke"
        ),
    }


def compute_geometry(st: dict) -> dict:
    anti_l, anti_b = 0.0, 15.0
    coe_l, coe_b = 180.0, -15.0

    n_coe = lb_to_n(coe_l, coe_b)
    n_anti = lb_to_n(anti_l, anti_b)
    D = n_anti  # +D = anti-CoE = +x

    th_max = [float(t) for t in st["theta_max_from_plus_x_deg"]]
```

```
    th_min = [float(t) for t in st["theta_min_from_plus_x_deg"]]
    max_bands = list(st["max_bands_deg"])
    min_bands = list(st["min_bands_deg"])

    n_maxes = [lb_to_n(*lb) for lb in MIGKAS_MAXES]
    n_mins = [lb_to_n(*lb) for lb in MIGKAS_MINS]

    sep = float(
        np.rad2deg(
            np.arccos(np.clip(np.dot(n_maxes[0], n_mins[1]), -1.0, 1.0))
        )
    )

    return {
        "D": D,
        "n_coe": n_coe,
        "n_anti": n_anti,
        "n_maxes": n_maxes,
        "n_mins": n_mins,
        "anti_lb": (anti_l, anti_b),
        "coe_lb": (coe_l, coe_b),
        "theta_max_from_plus_x_deg": th_max,
        "theta_min_from_plus_x_deg": th_min,
        "max_bands_deg": max_bands,
        "min_bands_deg": min_bands,
        "mu_max_bands": list(st["mu_max_bands"]),
        "mu_min_bands": list(st["mu_min_bands"]),
        "sep_deg": sep,
        "band_half_width_deg": st["band_half_width_deg"],
    }


def draw_wire_sphere(ax, n_mer=12, n_par=6, **kwargs):
    u = np.linspace(0, 2 * np.pi, 60)
    v = np.linspace(-np.pi / 2, np.pi / 2, 40)
    xs = np.outer(np.cos(u), np.cos(v))
    ys = np.outer(np.sin(u), np.cos(v))
    zs = np.outer(np.ones_like(u), np.sin(v))
    ax.plot_surface(xs, ys, zs, color="#dfe6e9", alpha=0.15, linewidth=0, shade=False)
    kw = dict(color="#95a5a6", lw=0.4, alpha=0.45)
    kw.update(kwargs)
    for i in range(n_mer):
        phi = 2 * np.pi * i / n_mer
        t = np.linspace(-np.pi / 2, np.pi / 2, 80)
        ax.plot(np.cos(t) * np.cos(phi), np.cos(t) * np.sin(phi), np.sin(t), **kw)
    for j in range(1, n_par):
        b = -np.pi / 2 + np.pi * j / n_par
        t = np.linspace(0, 2 * np.pi, 80)
        r = np.cos(b)
        ax.plot(r * np.cos(t), r * np.sin(t), np.full_like(t, np.sin(b)), **kw)


def foreshortened_ellipse_xy(mu: float, n: int = 200):
    """Side-view foreshortened ellipse of a 3D ring at n·x = mu."""
```

```
    rho = np.sqrt(max(0.0, 1.0 - mu * mu))
    phi = np.linspace(0, 2 * np.pi, n)
    ex = mu + 0.045 * rho * np.cos(phi)
    ey = rho * np.sin(phi)
    return ex, ey, rho



# ---------------------------------------------------------------------------
# Opacity gradient along each loop (Chris 2026-09-26), same method as
# make_coe_rings_gradient_loops.py: loop drawn as runs of short elements with
# per-element alpha = base_alpha * (1 + cos psi)/2, psi = ring azimuth from the
# θ-ray point (ring point most along sign(sin θ_mid)·Ŷ; Ŷ = Galactic +z
# projected ⊥ D, i.e. model-frame "up"; no roll). Darkest at the θ-ray point,
# fully transparent at the diametrically opposite point.
# ---------------------------------------------------------------------------
from matplotlib.collections import LineCollection
from matplotlib.colors import to_rgba
from mpl_toolkits.mplot3d.art3d import Line3DCollection


def theta_ray_sign(theta_mid_deg):
    return 1.0 if np.sin(np.deg2rad(float(theta_mid_deg))) >= 0 else -1.0


ROLL_BC_DEG = 80.8  # roll about CoE axis, applied only via small_circle_grad (Panels B/C)


def ring_basis(D):
    a = np.asarray(D, dtype=float)
    a = a / np.linalg.norm(a)
    tmp = np.array([0.0, 0.0, 1.0])
    if abs(np.dot(a, tmp)) > 0.9:
        tmp = np.array([0.0, 1.0, 0.0])
    y = tmp - np.dot(tmp, a) * a
    y /= np.linalg.norm(y)
    z = np.cross(a, y)
    z /= np.linalg.norm(z)
    # +45° roll (Panels B/C only), same convention as coe_half_loop_geometry.ring_basis
    # / earlier "+45° CCW about CoE" figures: Y' = c Y - s Z, Z' = s Y + c Z
    r = np.deg2rad(float(ROLL_BC_DEG))
    c, s_ = np.cos(r), np.sin(r)
    return c * y - s_ * z, s_ * y + c * z


def grad_phi(theta_mid_deg, n):
    """phi from antipode of θ-ray point round to antipode (psi = -180..+180)."""
    phi0 = 0.5 * np.pi if theta_ray_sign(theta_mid_deg) > 0 else 1.5 * np.pi
    return phi0, phi0 + np.linspace(-np.pi, np.pi, n)


def grad_scale(phi, phi0):
    return 0.5 * (1.0 + np.cos(np.asarray(phi) - phi0))
```

```
def small_circle_grad(D, mu, theta_mid_deg, n=721):
    a = np.asarray(D, dtype=float)
    a = a / np.linalg.norm(a)
    mu = float(np.clip(mu, -1.0 + 1e-9, 1.0 - 1e-9))
    rho = np.sqrt(max(0.0, 1.0 - mu * mu))
    y, z = ring_basis(a)
    phi0, phi = grad_phi(theta_mid_deg, n)
    pts = (mu * a)[None, :] + rho * (np.outer(np.sin(phi), y) + np.outer(np.cos(phi), z))
    return pts, grad_scale(phi, phi0)


def segment_scale(sc):
    sc = np.asarray(sc, dtype=float)
    return 0.5 * (sc[:-1] + sc[1:])


def rgba_array(color, alphas):
    r, g, b, _ = to_rgba(color)
    al = np.clip(np.asarray(alphas, dtype=float), 0.0, 1.0)
    out = np.empty((al.size, 4))
    out[:, 0], out[:, 1], out[:, 2], out[:, 3] = r, g, b, al
    return out


def quantized_runs(seg_sc, valid=None, n_levels=64):
    s = np.asarray(seg_sc, dtype=float)
    lev = np.round(np.clip(s, 0.0, 1.0) * n_levels) / n_levels
    ok = lev > 0 if valid is None else (lev > 0) & np.asarray(valid, dtype=bool)
    runs, i, n = [], 0, s.size
    while i < n:
        if not ok[i]:
            i += 1
            continue
        j = i + 1
        while j < n and ok[j] and lev[j] == lev[i]:
            j += 1
        runs.append((i, j, float(lev[i])))
        i = j
    return runs


def _strip_polys_2d(xa, ya, xb, yb, runs):
    return [
        np.column_stack([
            np.concatenate([xa[i0:i1 + 1], xb[i0:i1 + 1][::-1]]),
            np.concatenate([ya[i0:i1 + 1], yb[i0:i1 + 1][::-1]]),
        ])
        for i0, i1, _ in runs
    ]


def _polylines_2d(x, y, runs):
    return [np.column_stack([x[i0:i1 + 1], y[i0:i1 + 1]]) for i0, i1, _ in runs]
```

```
def draw_panelA_ribbon(ax, theta_lo: float, theta_hi: float, theta_mid: float, color: str):
    """
    Panel-A highlight (edge-on chord): same vertical strip between
    x=cos(θ_hi)..cos(θ_lo) clipped to the unit circle, base alpha 0.28, and the
    same foreshortened mid-ellipse (lw 2.8, alpha 0.95) — now with the opacity
    gradient along the chord: alpha *= (1 + sgn·y/ρ(x))/2 = (1+cos psi)/2,
    darkest at the θ-ray end, transparent at the opposite end.
    """
    mu_lo = float(np.cos(np.deg2rad(theta_lo)))
    mu_hi = float(np.cos(np.deg2rad(theta_hi)))
    x0, x1 = (min(mu_lo, mu_hi), max(mu_lo, mu_hi))
    sgn = theta_ray_sign(theta_mid)
    nx, ny = 64, 1200
    xs = np.linspace(x0, x1, nx)
    ys = np.linspace(-1.0, 1.0, ny)
    X, Y = np.meshgrid(xs, ys)
    rho = np.sqrt(np.clip(1.0 - X * X, 1e-6, None))
    sc = np.clip(0.5 * (1.0 + sgn * Y / rho), 0.0, 1.0)
    img = np.zeros((ny, nx, 4))
    img[..., :3] = to_rgba(color)[:3]
    img[..., 3] = 0.28 * sc
    im = ax.imshow(img, extent=(x0, x1, -1.0, 1.0), origin="lower",
                   interpolation="bilinear", zorder=2, aspect="auto")
    im.set_clip_path(Circle((0.0, 0.0), 1.0, transform=ax.transData))

    mu_mid = float(np.cos(np.deg2rad(theta_mid)))
    rho_m = np.sqrt(max(0.0, 1.0 - mu_mid * mu_mid))
    phi0, phi = grad_phi(theta_mid, 721)
    ex_m = mu_mid + 0.045 * rho_m * np.cos(phi)
    ey_m = rho_m * np.sin(phi)
    runs = quantized_runs(segment_scale(grad_scale(phi, phi0)))
    ax.add_collection(
        LineCollection(
            _polylines_2d(ex_m, ey_m, runs),
            colors=rgba_array(color, [0.95 * r[2] for r in runs]),
            linewidths=2.8, zorder=4, capstyle="butt", joinstyle="round",
        )
    )


def draw_panelA_theta_rays(ax, geo):
    """
    Illustrate θ: from the observer (origin), one ray per loop out to the
    ring at that loop's center θ. θ is measured from +x (anti-CoE, right)
    up to the ring. Emphasize MAX θ=30° (thick semi-transparent red to the
    top of that foreshortened ring), matching Chris's markup.
    """
    items = list(zip(geo["theta_max_from_plus_x_deg"], COLORS_MAX)) + list(
        zip(geo["theta_min_from_plus_x_deg"], COLORS_MIN)
    )
    for mid, color in items:
        t = float(np.deg2rad(mid))
```

```
        # Endpoint = top/rim point of the ring in side view = (cos θ, sin θ)
        x1, y1 = float(np.cos(t)), float(np.sin(t))
        is_max30 = abs(mid - 29.5) < 0.5
        lw = 3.2 if is_max30 else 1.8
        alpha = 0.70 if is_max30 else 0.45
        ax.plot(
            [0.0, x1], [0.0, y1],
            color=color, lw=lw, alpha=alpha, zorder=3,
            solid_capstyle="round",
        )
        # Small tick at the ring hit
        ax.plot([x1], [y1], "o", color=color, ms=4.5 if is_max30 else 3.5,
                alpha=0.9, zorder=5, markeredgecolor="none")

    # Angle arc at θ=30° from +x axis up to the ray (makes the measure obvious)
    arc_r = 0.38
    phi = np.linspace(0.0, np.deg2rad(29.5), 60)
    ax.plot(
        arc_r * np.cos(phi), arc_r * np.sin(phi),
        color=C_MAX_30, lw=1.6, alpha=0.85, zorder=3,
    )
    ax.text(
        arc_r * np.cos(np.deg2rad(15.0)) + 0.06,
        arc_r * np.sin(np.deg2rad(15.0)) + 0.04,
        r"$\theta$",
        color=C_MAX_30, fontsize=10, fontweight="bold",
        ha="left", va="bottom", zorder=6,
    )


def panel_A_model_frame(ax, geo):
    """Side view: exactly four thick foreshortened loop ribbons about x."""
    ax.set_title(
        "A. Model frame (no Galactic yet)\n"
        r"axisymmetric about $x$ = CoE$\leftrightarrow$anti-CoE"
        "\n"
        r"two MAX + two MIN, each $18^\circ$ end-to-end ($\pm9^\circ$)",
        fontsize=10,
        pad=8,
    )
    th = np.linspace(0, 2 * np.pi, 400)
    ax.plot(np.cos(th), np.sin(th), color="#7f8c8d", lw=1.2, zorder=1)
    ax.plot(0, 0, "ko", ms=6, zorder=5)
    ax.text(0.08, -0.12, "observer", fontsize=8, color="k", ha="left")

    ax.annotate(
        "",
        xy=(1.15, 0),
        xytext=(-1.15, 0),
        arrowprops=dict(arrowstyle="<->", color=C_AXIS, lw=2.5),
        zorder=4,
    )
    ax.text(1.18, 0.08, "anti-CoE\n(+x)", color=C_ANTI, fontsize=8, ha="left", va="bottom")
    ax.text(-1.18, 0.08, "CoE\n(−x)", color=C_COE, fontsize=8, ha="right", va="bottom")
```

```
    for (lo, hi), mid, color in zip(
        geo["max_bands_deg"], geo["theta_max_from_plus_x_deg"], COLORS_MAX
    ):
        draw_panelA_ribbon(ax, lo, hi, mid, color)
    for (lo, hi), mid, color in zip(
        geo["min_bands_deg"], geo["theta_min_from_plus_x_deg"], COLORS_MIN
    ):
        draw_panelA_ribbon(ax, lo, hi, mid, color)

    # Illustrate θ from +x (anti-CoE) to each loop (Chris markup)
    draw_panelA_theta_rays(ax, geo)

    label_r = 1.30
    for mid, color in list(zip(geo["theta_max_from_plus_x_deg"], COLORS_MAX)) + list(
        zip(geo["theta_min_from_plus_x_deg"], COLORS_MIN)
    ):
        t = np.deg2rad(mid)
        ax.text(
            label_r * np.cos(t),
            label_r * np.sin(t),
            rf"$\theta\!=\!{mid:g}^\circ$",
            color=color, fontsize=7.5, ha="center", va="center", fontweight="bold",
        )

    ax.text(
        0.0,
        -1.38,
        (
            r"four transparent highlight bands about $x$  ·  each $18^\circ$ end-to-end ($\pm9^\circ$ = 5% of $360^\circ$)"
            + "\n"
            + r"(after-2$\ell$ $A(\ell)$: two MAX + two MIN; one mid-stroke each)"
            + "\n"
            + r"$y,z$ free (roll) — model does not pick celestial equator"
        ),
        fontsize=7.5,
        ha="center",
        va="top",
        style="italic",
        color="#34495e",
    )
    ax.set_xlim(-1.55, 1.55)
    ax.set_ylim(-1.55, 1.45)
    ax.set_aspect("equal")
    ax.axis("off")


def _ring_lb(D, mu, n_pts=721):
    pts = small_circle_about_axis(D, mu, n=n_pts)
    rl = np.array([n_to_lb(p)[0] for p in pts])
    rb = np.array([n_to_lb(p)[1] for p in pts])
    return rl, rb
```

```
def _ring_lonlat_rad(D, mu, n_pts=361):
    """Small-circle → Mollweide display (lon_disp, lat) in radians."""
    pts = small_circle_about_axis(D, float(mu), n=n_pts)
    ls = np.array([n_to_lb(p)[0] for p in pts])
    bs = np.array([n_to_lb(p)[1] for p in pts])
    lon = np.deg2rad(lon_disp_from_ell(ls))
    lat = np.deg2rad(bs)
    return lon, lat


def fill_mollweide_ribbon(ax, D, theta_lo, theta_hi, theta_mid, color, n_pts=721):
    """
    Mollweide highlight band (±9° edges, base alpha 0.26) + mid-stroke
    (lw 2.6, alpha 0.95), both with the (1+cos psi)/2 opacity gradient.
    Elements crossing the antimeridian seam are dropped.
    """
    mu_a, mu_b = mu_pair_for_band(theta_lo, theta_hi)
    mu_mid = float(np.cos(np.deg2rad(theta_mid)))

    def lonlat(mu, n):
        pts, sc = small_circle_grad(D, mu, theta_mid, n=n)
        ls = np.array([n_to_lb(p)[0] for p in pts])
        bs = np.array([n_to_lb(p)[1] for p in pts])
        return np.deg2rad(lon_disp_from_ell(ls)), np.deg2rad(bs), sc

    lon_a, lat_a, sc = lonlat(mu_a, n_pts)
    lon_b, lat_b, _ = lonlat(mu_b, n_pts)
    gap = 0.9 * np.pi
    quad = np.stack([lon_a[:-1], lon_a[1:], lon_b[1:], lon_b[:-1]])
    valid = (quad.max(axis=0) - quad.min(axis=0)) <= gap
    runs = quantized_runs(segment_scale(sc), valid)
    if runs:
        ax.add_collection(
            PolyCollection(
                _strip_polys_2d(lon_a, lat_a, lon_b, lat_b, runs),
                facecolors=rgba_array(color, [0.26 * r[2] for r in runs]),
                edgecolors="none", zorder=2, linewidths=0,
            )
        )
    lon_m, lat_m, sc_m = lonlat(mu_mid, max(n_pts, 721))
    valid = np.abs(np.diff(lon_m)) <= gap
    runs = quantized_runs(segment_scale(sc_m), valid)
    if runs:
        ax.add_collection(
            LineCollection(
                _polylines_2d(lon_m, lat_m, runs),
                colors=rgba_array(color, [0.95 * r[2] for r in runs]),
                linewidths=2.6, zorder=4, capstyle="butt", joinstyle="round",
            )
        )


def panel_B_mollweide(ax, geo):
    ax.set_title(
```

```
        r"B. Migkas $A(\ell)$ loops on sky (anti-CoE at $(0^\circ,+15^\circ)$)"
        "\n"
        r"(Galactic Mollweide; CoE axis tilted $+15^\circ$; $\ell$ increases LEFT; "
        r"two MAX + two MIN, each $18^\circ$ end-to-end ($\pm9^\circ$))",
        fontsize=10,
        pad=8,
    )
    for b in [-60, -30, 0, 30, 60]:
        ell = np.linspace(0, 360, 361)
        plot_lb_curve_mollweide(ax, ell, np.full_like(ell, b), color="#bdc3c7", lw=0.5, zorder=0)
    for ell0 in range(0, 360, 30):
        b = np.linspace(-90, 90, 181)
        plot_lb_curve_mollweide(ax, np.full_like(b, ell0), b, color="#bdc3c7", lw=0.5, zorder=0)

    D = geo["D"]
    ls, bs = great_circle_through_poles(D, n=721)
    plot_lb_curve_mollweide(ax, ls, bs, color=C_AXIS, lw=2.0, zorder=2)

    for (lo, hi), mid, color in zip(
        geo["max_bands_deg"], geo["theta_max_from_plus_x_deg"], COLORS_MAX
    ):
        fill_mollweide_ribbon(ax, D, lo, hi, mid, color)
    for (lo, hi), mid, color in zip(
        geo["min_bands_deg"], geo["theta_min_from_plus_x_deg"], COLORS_MIN
    ):
        fill_mollweide_ribbon(ax, D, lo, hi, mid, color)

    ell_eq, b_eq = celestial_equator_lb(721)
    plot_lb_curve_mollweide(ax, ell_eq, b_eq, color=C_EQ, lw=1.8, ls="--", zorder=2)

    ax.plot(
        np.deg2rad(lon_disp_from_ell(geo["anti_lb"][0])),
        np.deg2rad(geo["anti_lb"][1]),
        marker="o", ms=8, color=C_ANTI,
        markerfacecolor=C_ANTI, markeredgecolor="k", markeredgewidth=0.6,
        zorder=6, linestyle="None",
    )
    ax.plot(
        np.deg2rad(lon_disp_from_ell(geo["coe_lb"][0])),
        np.deg2rad(geo["coe_lb"][1]),
        marker="*", ms=14, color=C_COE,
        markerfacecolor=C_COE, markeredgecolor="k", markeredgewidth=0.6,
        zorder=6, linestyle="None",
    )
    for (l, b), color in zip(MIGKAS_MAXES, COLORS_MAX):
        ax.plot(
            np.deg2rad(lon_disp_from_ell(l)),
            np.deg2rad(b),
            marker="^",
            ms=10,
            color=color,
            markeredgecolor="k",
            markeredgewidth=0.7,
            zorder=7,
```

```
            linestyle="None",
        )
    for (l, b), color in zip(MIGKAS_MINS, COLORS_MIN):
        ax.plot(
            np.deg2rad(lon_disp_from_ell(l)),
            np.deg2rad(b),
            marker="s",
            ms=10,
            color=color,
            markeredgecolor="k",
            markeredgewidth=0.7,
            zorder=7,
            linestyle="None",
        )

    xticks = np.deg2rad(lon_disp_from_ell(np.array([180, 90, 0, 270])))
    ax.set_xticks(xticks)
    ax.set_xticklabels([r"$180^\circ$", r"$90^\circ$", r"$0^\circ$", r"$270^\circ$"], fontsize=8)
    ax.set_yticks(np.deg2rad([-60, -30, 0, 30, 60]))
    ax.set_yticklabels([r"$-60^\circ$", r"$-30^\circ$", r"$0^\circ$", r"$+30^\circ$", r"$+60^\circ$"], fontsize=8)
    ax.grid(False)
    ax.set_xlabel(r"Galactic $\ell$ (increases LEFT)", fontsize=8)
    ax.set_ylabel(r"Galactic $b$", fontsize=8)


def fill_sphere_ribbon(ax, D, theta_lo, theta_hi, theta_mid, color, n_pts=160):
    """
    3D highlight belt (±9° edges, base alpha 0.25) + mid-stroke (lw 2.6,
    alpha 0.95), both with the (1+cos psi)/2 opacity gradient.
    """
    mu_a, mu_b = mu_pair_for_band(theta_lo, theta_hi)
    mu_mid = float(np.cos(np.deg2rad(theta_mid)))
    n_g = 361
    p0, sc = small_circle_grad(D, float(mu_a), theta_mid, n=n_g)
    p1, _ = small_circle_grad(D, float(mu_b), theta_mid, n=n_g)
    seg = segment_scale(sc)
    idx = np.where(seg > 1e-4)[0]
    verts = [[p0[j], p0[j + 1], p1[j + 1], p1[j]] for j in idx]
    coll = Poly3DCollection(
        verts, facecolors=rgba_array(color, 0.25 * seg[idx]),
        edgecolors="none", linewidths=0.0, zorder=2,
    )
    ax.add_collection3d(coll)
    pts, scm = small_circle_grad(D, mu_mid, theta_mid, n=n_g)
    runs = quantized_runs(segment_scale(scm))
    ax.add_collection3d(
        Line3DCollection(
            [pts[i0:i1 + 1] for i0, i1, _ in runs],
            colors=rgba_array(color, [0.95 * r[2] for r in runs]),
            linewidths=2.6, zorder=5, capstyle="butt", joinstyle="round",
        )
    )
```

```
def panel_C_sphere(ax, geo):
    th_max = geo["theta_max_from_plus_x_deg"]
    th_min = geo["theta_min_from_plus_x_deg"]
    ax.set_title(
        r"C. Sphere 3D — two MAX + two MIN, each $18^\circ$ end-to-end ($\pm9^\circ$)"
        "\n"
        r"(axis line emphasized; celestial equator dashed)",
        fontsize=10,
        pad=6,
    )
    draw_wire_sphere(ax)
    D = geo["D"]
    ell_eq, b_eq = celestial_equator_lb(400)
    eq_xyz = np.array([lb_to_n(l, b) for l, b in zip(ell_eq, b_eq)])
    ax.plot(eq_xyz[:, 0], eq_xyz[:, 1], eq_xyz[:, 2], color=C_EQ, lw=1.8, ls="--", zorder=3)
    ax.plot(
        [-D[0], D[0]],
        [-D[1], D[1]],
        [-D[2], D[2]],
        color=C_AXIS,
        lw=3.0,
        zorder=5,
    )
    ax.scatter(*(-D), color=C_COE, s=120, marker="*", edgecolors="k", linewidths=0.6,
               depthshade=False, zorder=8)
    ax.scatter(*(D), color=C_ANTI, s=80, edgecolors="k", linewidths=0.6, depthshade=False, zorder=8)
    ax.text(*(-D * 1.22), "CoE", color=C_COE, fontsize=8, fontweight="bold")
    ax.text(*(D * 1.18), "anti-CoE", color=C_ANTI, fontsize=8, fontweight="bold")
    ax.scatter([0], [0], [0], color="k", s=25, depthshade=False, zorder=9)

    for (lo, hi), mid, color in zip(
        geo["max_bands_deg"], geo["theta_max_from_plus_x_deg"], COLORS_MAX
    ):
        fill_sphere_ribbon(ax, D, lo, hi, mid, color)
    for (lo, hi), mid, color in zip(
        geo["min_bands_deg"], geo["theta_min_from_plus_x_deg"], COLORS_MIN
    ):
        fill_sphere_ribbon(ax, D, lo, hi, mid, color)

    for n, color in zip(geo["n_maxes"], COLORS_MAX):
        ax.scatter(
            [n[0]], [n[1]], [n[2]],
            c=[color], marker="^", s=70,
            edgecolors="k", linewidths=0.7, depthshade=False, zorder=10,
        )
        ax.plot([0, n[0]], [0, n[1]], [0, n[2]], color=color, lw=0.8, ls=":", alpha=0.7)
    for n, color in zip(geo["n_mins"], COLORS_MIN):
        ax.scatter(
            [n[0]], [n[1]], [n[2]],
            c=[color], marker="s", s=60,
            edgecolors="k", linewidths=0.7, depthshade=False, zorder=10,
        )
        ax.plot([0, n[0]], [0, n[1]], [0, n[2]], color=color, lw=0.8, ls=":", alpha=0.7)
```

```
    ax.set_xlim(-1.25, 1.25)
    ax.set_ylim(-1.25, 1.25)
    ax.set_zlim(-1.25, 1.25)
    ax.set_box_aspect((1, 1, 1))
    ax.view_init(elev=18, azim=-55)
    ax.set_axis_off()
    ax.text2D(
        0.5,
        -0.02,
        rf"MAX $\theta\!=\!{th_max[0]:g}^\circ,{th_max[1]:g}^\circ$; "
        rf"MIN $\theta\!=\!{th_min[0]:g}^\circ,{th_min[1]:g}^\circ$  "
        rf"($18^\circ$ end-to-end = $\pm9^\circ$ = 5% of $360^\circ$)",
        transform=ax.transAxes,
        ha="center",
        fontsize=7.5,
        color="#34495e",
    )


def main():
    st = migkas_scan_stops()
    geo = compute_geometry(st)

    th_max = geo["theta_max_from_plus_x_deg"]
    th_min = geo["theta_min_from_plus_x_deg"]
    max_bands = geo["max_bands_deg"]
    min_bands = geo["min_bands_deg"]

    fig = plt.figure(figsize=(14.5, 5.6), dpi=160)
    fig.suptitle(
        r"Pure CoE axisymmetry: two MAX + two MIN highlight bands  (gradient: darkest at $\theta$ ray, transparent opposite; +80.8$^\circ$ CCW about
CoE axis (B/C))"
        "\n"
        rf"(each $18^\circ$ end-to-end = $\pm9^\circ$ = 5% of $360^\circ$; "
        rf"MAX $\theta\!=\!{th_max[0]:g}^\circ/{th_max[1]:g}^\circ$; "
        rf"MIN $\theta\!=\!{th_min[0]:g}^\circ/{th_min[1]:g}^\circ$; "
        r"anti-CoE $(0^\circ,+15^\circ)$, CoE axis tilted $+15^\circ$)",
        fontsize=11,
        fontweight="bold",
        y=1.02,
    )

    axA = fig.add_subplot(1, 3, 1)
    panel_A_model_frame(axA, geo)

    axB = fig.add_subplot(1, 3, 2, projection="mollweide")
    panel_B_mollweide(axB, geo)

    axC = fig.add_subplot(1, 3, 3, projection="3d")
    panel_C_sphere(axC, geo)

    handles = [
        Line2D([0], [0], color=C_AXIS, lw=2.5, label=r"CoE$\leftrightarrow$anti-CoE axis"),
        Patch(
```

```
            facecolor=C_MAX_30, alpha=0.45, edgecolor=C_MAX_30,
            label=(
                rf"MAX $\theta\!=\!{th_max[0]:g}^\circ$ band "
                rf"[{max_bands[0][0]:g},{max_bands[0][1]:g}]$^\circ$ "
                rf"($18^\circ$=±$9^\circ$=5% of $360^\circ$)"
            ),
        ),
        Patch(
            facecolor=C_MAX_185, alpha=0.45, edgecolor=C_MAX_185,
            label=(
                rf"MAX $\theta\!=\!{th_max[1]:g}^\circ$ band "
                rf"[{max_bands[1][0]:g},{max_bands[1][1]:g}]$^\circ$ "
                rf"($18^\circ$=±$9^\circ$)"
            ),
        ),
        Patch(
            facecolor=C_MIN_115, alpha=0.45, edgecolor=C_MIN_115,
            label=(
                rf"MIN $\theta\!=\!{th_min[0]:g}^\circ$ band "
                rf"[{min_bands[0][0]:g},{min_bands[0][1]:g}]$^\circ$ "
                rf"($18^\circ$=±$9^\circ$)"
            ),
        ),
        Patch(
            facecolor=C_MIN_265, alpha=0.45, edgecolor=C_MIN_265,
            label=(
                rf"MIN $\theta\!=\!{th_min[1]:g}^\circ$ band "
                rf"[{min_bands[1][0]:g},{min_bands[1][1]:g}]$^\circ$ "
                rf"($18^\circ$=±$9^\circ$)"
            ),
        ),
        Line2D([0], [0], color=C_EQ, lw=1.8, ls="--", label="celestial equator (Dec$=0$)"),
        Line2D(
            [0], [0], marker="^", color="w", markerfacecolor=C_MAX_30, markeredgecolor="k",
            ms=9,
            label=rf"MAX $({MIGKAS_MAXES[0][0]:.0f}^\circ,{MIGKAS_MAXES[0][1]:+.0f}^\circ)$",
        ),
        Line2D(
            [0], [0], marker="^", color="w", markerfacecolor=C_MAX_185, markeredgecolor="k",
            ms=9,
            label=rf"MAX $({MIGKAS_MAXES[1][0]:.0f}^\circ,{MIGKAS_MAXES[1][1]:+.0f}^\circ)$",
        ),
        Line2D(
            [0], [0], marker="s", color="w", markerfacecolor=C_MIN_115, markeredgecolor="k",
            ms=8,
            label=rf"MIN $({MIGKAS_MINS[0][0]:.0f}^\circ,{MIGKAS_MINS[0][1]:+.0f}^\circ)$",
        ),
        Line2D(
            [0], [0], marker="s", color="w", markerfacecolor=C_MIN_265, markeredgecolor="k",
            ms=8,
            label=rf"MIN $({MIGKAS_MINS[1][0]:.0f}^\circ,{MIGKAS_MINS[1][1]:+.0f}^\circ)$",
        ),
        Line2D(
            [0], [0], marker="o", color="w", markerfacecolor=C_ANTI, markeredgecolor="k",
```

```
        ms=8, label=rf"anti-CoE ${geo['anti_lb'][0]:.1f}^\circ,{geo['anti_lb'][1]:+.1f}^\circ)$",
    ),
    Line2D(
        [0], [0], marker="*", color="w", markerfacecolor=C_COE, markeredgecolor="k",
        ms=11, label=rf"CoE ${geo['coe_lb'][0]:.1f}^\circ,{geo['coe_lb'][1]:+.1f}^\circ)$",
    ),
]
fig.legend(
    handles=handles,
    loc="lower center",
    ncol=6,
    fontsize=7.0,
    frameon=True,
    fancybox=True,
    bbox_to_anchor=(0.5, -0.10),
)

fig.tight_layout()
# --- Chris 2026-09-26: panel C sphere drawn at the same on-page diameter as
# panel A's model-frame circle; panel C axes matched to A's vertical
# position/height. Nothing else changes.
from mpl_toolkits.mplot3d import proj3d
pa, pc = axA.get_position(), axC.get_position()
axC.set_position([pc.x0, pa.y0, pc.width, pa.height])

def _diam_A():
    fig.canvas.draw()
    p0 = axA.transData.transform((-1.0, 0.0)); p1 = axA.transData.transform((1.0, 0.0))
    return float(p1[0] - p0[0])

def _diam_C():
    fig.canvas.draw()
    u = np.random.default_rng(0).normal(size=(20000, 3))
    u /= np.linalg.norm(u, axis=1)[:, None]
    xs, ys, _ = proj3d.proj_transform(u[:, 0], u[:, 1], u[:, 2], axC.get_proj())
    d = axC.transData.transform(np.column_stack([xs, ys]))
    return float(d[:, 0].max() - d[:, 0].min()), float(d[:, 1].max() - d[:, 1].min())

zoom = 1.0
for _ in range(8):
    dA = _diam_A(); dCx, dCy = _diam_C()
    zoom *= dA / dCx
    axC.set_box_aspect((1, 1, 1), zoom=zoom)
dA = _diam_A(); dCx, dCy = _diam_C()
print(f"PANEL_DIAM A={dA:.1f}px C={dCx:.1f}x{dCy:.1f}px zoom={zoom:.3f}")
fig.savefig(PNG_PATH, bbox_inches="tight", facecolor="white")
plt.close(fig)

payload = {
    "png": str(PNG_PATH),
    "script": str(OUT_DIR / "make_coe_rings_migkas_pin_axis_matched_grad_rot_fit.py"),
    "migkas_max_lb_deg": [list(lb) for lb in MIGKAS_MAXES],
    "migkas_min_lb_deg": [list(lb) for lb in MIGKAS_MINS],
    "strip_theta_max_deg": list(STRIP_THETA_MAX),
```

```
        "strip_theta_min_deg": list(STRIP_THETA_MIN),
        "band_half_width_deg": BAND_HALF,
        "band_full_width_deg": BAND_FULL,
        "band_total_width_deg": BAND_TOTAL,
        "band_width_note": "5% of 360° = 18° end-to-end = ±9° about each MAX/MIN center",
        "max_bands_deg": [list(b) for b in max_bands],
        "min_bands_deg": [list(b) for b in min_bands],
        "band_intervals": {
            "MAX_30": [float(max_bands[0][0]), float(max_bands[0][1])],
            "MAX_185": [float(max_bands[1][0]), float(max_bands[1][1])],
            "MIN_115": [float(min_bands[0][0]), float(min_bands[0][1])],
            "MIN_265": [float(min_bands[1][0]), float(min_bands[1][1])],
        },
        "scan_stop_source": st["source"],
        "theta_max_from_plus_x_deg": [round(t, 4) for t in th_max],
        "theta_min_from_plus_x_deg": [round(t, 4) for t in th_min],
        "mu_max_bands": [[round(a, 8), round(b, 8)] for a, b in geo["mu_max_bands"]],
        "mu_min_bands": [[round(a, 8), round(b, 8)] for a, b in geo["mu_min_bands"]],
        "CoE_lb_deg": [round(geo["coe_lb"][0], 4), round(geo["coe_lb"][1], 4)],
        "anti_CoE_lb_deg": [round(geo["anti_lb"][0], 4), round(geo["anti_lb"][1], 4)],
        "D_hat_galactic_xyz": geo["D"].tolist(),
        "angular_sep_migkas_max0_min1_deg": round(geo["sep_deg"], 6),
        "axis_tilt_b_deg": 15,
        "axis_tilt_reason": (
            "Chris 2026-09-22 — markers fixed, rings rotate up 30°"
        ),
        "orientation": (
            "Galactic orientation: anti-CoE at (ell=0°, b=+15°), CoE at "
            "(ell=180°, b=−15°); +D = anti-CoE = +x; axis tilted +15° "
            "(Chris 2026-09-22 — markers fixed, rings rotate up 30°)"
        ),
        "drawing": (
            "Exactly 4 features. Each = ONE smooth transparent highlight band "
            "between θ−9° and θ+9° (18° end-to-end = 5% of 360°) + ONE solid "
            "mid-stroke at the exact center θ. No multi-stroke wash / hairball. "
            "Four distinct colors (two reds + two blues); no orange; no 283° "
            "companion. Panel A: clean vertical strip clipped to unit circle; theta rays from observer to each loop (emphasized at MAX 30°). Mollweide
"
            "seam-split honestly. Sphere: single Poly3D strip (no n_wash stack)."
        ),
        "note": (
            "Stops are after-2ℓ A(ℓ) extrema from best_fit_2ell_multiplier.npz "
            f"A_after: two MAX at θ={th_max[0]:g}°/{th_max[1]:g}° and "
            f"two MIN at θ={th_min[0]:g}°/{th_min[1]:g}°, each drawn as "
            f"one {BAND_FULL:.0f}° end-to-end highlight (±{BAND_HALF:.0f}° = 5% of 360°). "
            "Rings use μ=cos(θ); sky markers are map extrema (17°,−9°) and "
            "(274°,−22°) plus strip stand-ins (185°,0°) and (115°,0°)."
        ),
        "plot_convention": "Migkas Galactic Mollweide: lon_disp=-ell_wrapped (ell increases LEFT)",
        "legend_summary": (
            "two MAX + two MIN highlight bands, each 18° end-to-end "
            "(±9° = 5% of 360°); four distinct colors"
        ),
        "colors": {
```

```
            "MAX_30": C_MAX_30,
            "MAX_185": C_MAX_185,
            "MIN_115": C_MIN_115,
            "MIN_265": C_MIN_265,
            "no_orange": True,
        },
    }
    JSON_PATH.write_text(json.dumps(payload, indent=2) + "\n")
    print(json.dumps(payload, indent=2))
    print(f"\nWrote {PNG_PATH}")
    print(f"Wrote {JSON_PATH}")


if __name__ == "__main__":
    main()
```

```
PANEL_DIAM A=367.9px C=367.9x368.0px zoom=1.399
```

```
{
  "png": "/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/loop_threepanel_matched_grad_rotfit_rot81.png",
  "script": "/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/make_coe_rings_migkas_pin_axis_matched_grad_rot_fit.py",
  "migkas_max_lb_deg": [
    [
      17.0,
      -9.0
    ],
    [
      185.0,
      0.0
    ]
  ],
  "migkas_min_lb_deg": [
    [
      115.0,
      0.0
    ],
    [
      274.0,
      -22.0
    ]
  ],
  "strip_theta_max_deg": [
    29.5,
    187.5
  ],
  "strip_theta_min_deg": [
    122.6,
    264.4
  ],
  "band_half_width_deg": 9.0,
  "band_full_width_deg": 18.0,
  "band_total_width_deg": 18.0,
  "band_width_note": "5% of 360\u00b0 = 18\u00b0 end-to-end = \u00b19\u00b0 about each MAX/MIN center",
  "max_bands_deg": [
    [
      20.5,
      38.5
    ],
    [
      178.5,
      196.5
    ]
  ],
  "min_bands_deg": [
    [
      113.6,
      131.6
    ],
    [
      255.39999999999998,
      273.4
    ]
  ],
  "band_intervals": {
    "MAX_30": [
```

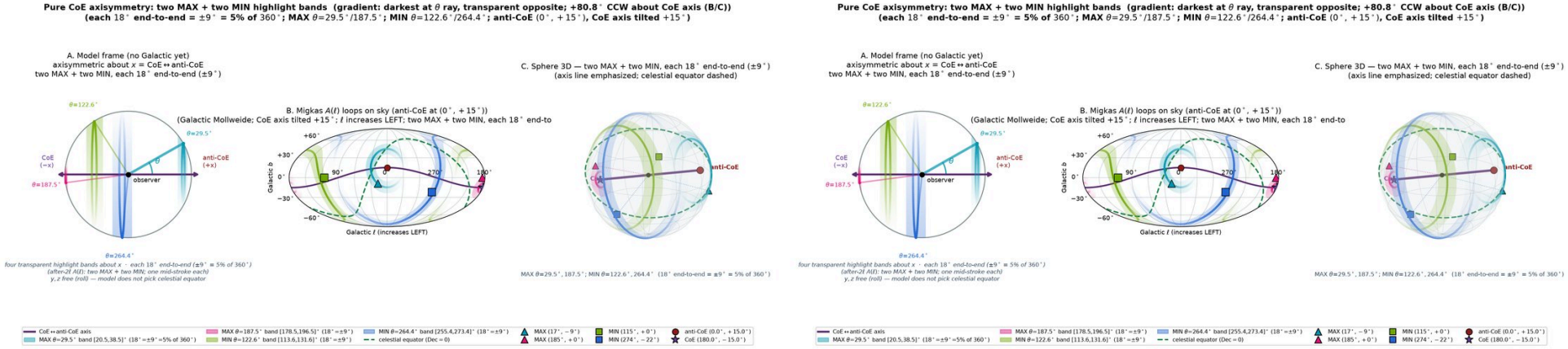
```
      20.5,
      38.5
    ],
    "MAX_185": [
      178.5,
      196.5
    ],
    "MIN_115": [
      113.6,
      131.6
    ],
    "MIN_265": [
      255.39999999999998,
      273.4
    ]
  },
  "scan_stop_source": "best_fit_2ell_multiplier.npz A_after extrema (two max, two min); each drawn as one transparent 18\u00b0 end-to-end highlight
(\u00b19\u00b0 = 5% of 360\u00b0) + one solid mid-stroke",
  "theta_max_from_plus_x_deg": [
    29.5,
    187.5
  ],
  "theta_min_from_plus_x_deg": [
    122.6,
    264.4
  ],
  "mu_max_bands": [
    [
      0.93667219,
      0.78260816
    ],
    [
      -0.99965732,
      -0.95881973
    ]
  ],
  "mu_min_bands": [
    [
      -0.40034903,
      -0.66392621
    ],
    [
      -0.25206936,
      0.05930637
    ]
  ],
  "CoE_lb_deg": [
    180.0,
    -15.0
  ],
  "anti_CoE_lb_deg": [
    0.0,
    15.0
  ],
  "D_hat_galactic_xyz": [
    0.9659258262890683,
    0.0,
```

```
    0.25881904510252074
  ],
  "angular_sep_migkas_max0_min1_deg": 98.476384,
  "axis_tilt_b_deg": 15,
  "axis_tilt_reason": "Chris 2026-09-22 \u2014 markers fixed, rings rotate up 30\u00b0",
  "orientation": "Galactic orientation: anti-CoE at (ell=0\u00b0, b=+15\u00b0), CoE at (ell=180\u00b0, b=\u221215\u00b0); +D = anti-CoE = +x; axis
tilted +15\u00b0 (Chris 2026-09-22 \u2014 markers fixed, rings rotate up 30\u00b0)",
  "drawing": "Exactly 4 features. Each = ONE smooth transparent highlight band between \u03b8\u22129\u00b0 and \u03b8+9\u00b0 (18\u00b0 end-to-end = 5%
of 360\u00b0) + ONE solid mid-stroke at the exact center \u03b8. No multi-stroke wash / hairball. Four distinct colors (two reds + two blues); no
orange; no 283\u00b0 companion. Panel A: clean vertical strip clipped to unit circle; theta rays from observer to each loop (emphasized at MAX
30\u00b0). Mollweide seam-split honestly. Sphere: single Poly3D strip (no n_wash stack).",
  "note": "Stops are after-2\u2113 A(\u2113) extrema from best_fit_2ell_multiplier.npz A_after: two MAX at \u03b8=29.5\u00b0/187.5\u00b0 and two MIN at
\u03b8=122.6\u00b0/264.4\u00b0, each drawn as one 18\u00b0 end-to-end highlight (\u00b19\u00b0 = 5% of 360\u00b0). Rings use \u03bc=cos(\u03b8); sky
markers are map extrema (17\u00b0,\u22129\u00b0) and (274\u00b0,\u221222\u00b0) plus strip stand-ins (185\u00b0,0\u00b0) and (115\u00b0,0\u00b0).",
  "plot_convention": "Migkas Galactic Mollweide: lon_disp=-ell_wrapped (ell increases LEFT)",
  "legend_summary": "two MAX + two MIN highlight bands, each 18\u00b0 end-to-end (\u00b19\u00b0 = 5% of 360\u00b0); four distinct colors",
  "colors": {
    "MAX_30": "#0097b2",
    "MAX_185": "#e0107a",
    "MIN_115": "#6fa800",
    "MIN_265": "#2166d4",
    "no_orange": true
  }
}

Wrote /workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/loop_threepanel_matched_grad_rotfit_rot81.png
Wrote /workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/loop_threepanel_matched_grad_rotfit_rot81.json
```


```
compare(FIGS / 'loop_threepanel_matched_grad_rotfit_rot81.png',
'/workspace/cosmology/papers/spreadsheet_model/loop_threepanel_matched_grad_rotfit_rot81.png')
```

```
loop_threepanel_matched_grad_rotfit_rot81.png: new (2262, 1027) vs ref (2262, 1027) | mean |diff| = 0.000/255, pixels differing >40: 0.000% ->
identical
left: approved reference papers/spreadsheet_model/loop_threepanel_matched_grad_rotfit_rot81.png   |   right: notebook re-run
steps_v3/figs/loop_threepanel_matched_grad_rotfit_rot81.png
```



Step 17 — Untilted, unrolled three-panel (anti-CoE $(0^\circ, 0^\circ)$, CoE $(180^\circ, 0^\circ)$)

What it shows. The matched three-panel with no tilt and no roll. The CoE axis lies in the Galactic plane, with anti-CoE at $(\ell, b) = (0^\circ, 0^\circ)$ and CoE at $(180^\circ, 0^\circ)$, so each loop is a small circle centred on the plane. Band centres (from the strip) are $\theta_{\max} = 29.5^\circ, 187.5^\circ$ and $\theta_{\min} = 122.6^\circ, 264.4^\circ$, each $\pm 9^\circ$. Panel C is at the same diameter as Panel A.

Source script: `/workspace/cosmology/figures/make_coe_rings_notilt.py` (last modified 2026-09-27 09:38 CT), run verbatim. *Output-path edit:* `/workspace/cosmology/papers/spreadsheet_model/loop_threepanel_notilt` → `/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/loop_threep`. *Reference image:* `papers/spreadsheet_model/loop_threepanel_notilt.png`.

```
%%runscript make_coe_rings_notilt.py
#!/usr/bin/env python3
"""
CoE rings + Migkas A(ell) extrema loops in the Galactic frame.

Concept:
  Pure CoE axisymmetry → constant A on small-circle rings about x = CoE↔anti-CoE.
  Scan A vs angle from +x (anti-CoE) around 360°. After-2ℓ A(ℓ) has TWO maxima
  and TWO minima (from best_fit_2ell_multiplier.npz A_after):
    MAX at θ = 30° and 185° from +x,
    MIN at θ = 115° and 265° from +x.

  Exactly FOUR loops — each ONE transparent highlight band, 18° end-to-end
  (= ±9° = 5% of 360°) about the center θ:
    MAX: 30±9 → [21°, 39°] and 185±9 → [176°, 194°]
    MIN: 115±9 → [106°, 124°] and 265±9 → [256°, 274°]
  Drawing: ONE smooth transparent filled ribbon between θ−9° and θ+9°
  + ONE solid mid-stroke at the exact center θ. No multi-stroke wash,
  no dense linspace hairball, no dual strong boundaries.
  Four distinct colors (two reds + two blues). No orange, no 283° companion.

  Galactic orientation: anti-CoE (+x) at ℓ=0°, b=+15°; CoE (−x) at ℓ=180°, b=−15°
  (axis tilted +15° so features near ℓ=0 move north on Mollweide; Migkas markers fixed).
  Sky markers (ℓ, b):
    Maxes: map extrema (17°, −9°) for ~30° max; strip (185°, 0°) for second max.
    Mins:  strip (115°, 0°) for ~115° min; map extrema (274°, −22°) for ~265° min.

Outputs:
  coe_rings_migkas_pin_axis.png
  coe_rings_migkas_pin_axis.json
"""

from __future__ import annotations

import json
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
```

```
from matplotlib.lines import Line2D
from matplotlib.collections import PolyCollection
from matplotlib.patches import Circle, Patch, Polygon
from mpl_toolkits.mplot3d import Axes3D  # noqa: F401
from mpl_toolkits.mplot3d.art3d import Poly3DCollection

OUT_DIR = Path(__file__).resolve().parent
PNG_PATH = Path("/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/loop_threepanel_notilt.png")
JSON_PATH = Path("/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/loop_threepanel_notilt.json")

# 5% of 360° = 18° end-to-end → half-width ±9° about each center
BAND_HALF = 9.0
BAND_FULL = 2.0 * BAND_HALF  # 18° = 5% of 360°
BAND_TOTAL = BAND_FULL  # alias
STRIP_THETA_MAX = [29.5, 187.5]
STRIP_THETA_MIN = [122.6, 264.4]

MIGKAS_MAXES = [(17.0, -9.0), (185.0, 0.0)]
MIGKAS_MINS = [(115.0, 0.0), (274.0, -22.0)]

C_COE = "#6b3fa0"
C_ANTI = "#8e1b1b"  # dark red marker; distinct from loop reds
# Four distinct loop colors: two reds (MAX) + two blues (MIN); no orange/yellow
C_MAX_30 = "#0097b2"    # cyan — MAX at 29.5°
C_MAX_185 = "#e0107a"   # magenta — MAX at 187.5°
C_MIN_115 = "#6fa800"   # green — MIN at 122.6°
C_MIN_265 = "#2166d4"   # blue — MIN at 264.4°
C_AXIS = "#5b2c6f"
C_EQ = "#1e8449"

# Per-loop color maps (order matches STRIP_THETA_MAX / STRIP_THETA_MIN)
COLORS_MAX = [C_MAX_30, C_MAX_185]
COLORS_MIN = [C_MIN_115, C_MIN_265]


def wrap_lon_deg(lon: np.ndarray) -> np.ndarray:
    return ((np.asarray(lon, dtype=float) + 180.0) % 360.0) - 180.0


def lon_disp_from_ell(ell_deg: np.ndarray) -> np.ndarray:
    """Migkas Galactic: ℓ increases LEFT → lon_disp = −ℓ_wrapped."""
    return -wrap_lon_deg(ell_deg)


def lb_to_n(l_deg: float, b_deg: float) -> np.ndarray:
    l = np.deg2rad(l_deg)
    b = np.deg2rad(b_deg)
    return np.array(
        [np.cos(b) * np.cos(l), np.cos(b) * np.sin(l), np.sin(b)], dtype=float
    )


def n_to_lb(n: np.ndarray) -> tuple[float, float]:
    n = np.asarray(n, dtype=float)
```

```
    n = n / np.linalg.norm(n)
    b = float(np.rad2deg(np.arcsin(np.clip(n[2], -1.0, 1.0))))
    l = float(np.rad2deg(np.arctan2(n[1], n[0])) % 360.0)
    return l, b


def celestial_equator_lb(n: int = 720) -> tuple[np.ndarray, np.ndarray]:
    """Dec=0 (ICRS) as Galactic (ℓ, b). IAU J2000 NGP convention, no astropy."""
    ra_ngp = np.deg2rad(192.85948)
    dec_ngp = np.deg2rad(27.12825)
    l_ncp = np.deg2rad(122.93192)
    ra = np.linspace(0.0, 2.0 * np.pi, n, endpoint=False)
    dec = 0.0
    sin_b = np.sin(dec) * np.sin(dec_ngp) + np.cos(dec) * np.cos(dec_ngp) * np.cos(ra - ra_ngp)
    sin_b = np.clip(sin_b, -1.0, 1.0)
    b = np.arcsin(sin_b)
    y = np.cos(dec) * np.sin(ra - ra_ngp)
    x = np.sin(dec) * np.cos(dec_ngp) - np.cos(dec) * np.sin(dec_ngp) * np.cos(ra - ra_ngp)
    l = (l_ncp - np.arctan2(y, x)) % (2.0 * np.pi)
    return np.rad2deg(l), np.rad2deg(b)


def great_circle_through_poles(D: np.ndarray, n: int = 361) -> tuple[np.ndarray, np.ndarray]:
    """Great circle through ±D (any meridian of the CoE axis)."""
    a = D / np.linalg.norm(D)
    tmp = np.array([0.0, 0.0, 1.0])
    if abs(np.dot(a, tmp)) > 0.9:
        tmp = np.array([1.0, 0.0, 0.0])
    e2 = np.cross(a, tmp)
    e2 /= np.linalg.norm(e2)
    t = np.linspace(0.0, 2.0 * np.pi, n)
    pts = np.outer(np.cos(t), a) + np.outer(np.sin(t), e2)
    ls = np.array([n_to_lb(p)[0] for p in pts])
    bs = np.array([n_to_lb(p)[1] for p in pts])
    return ls, bs


def small_circle_about_axis(axis: np.ndarray, mu: float, n: int = 361) -> np.ndarray:
    """Unit vectors on the small circle n·axis = mu (|mu|<1)."""
    a = axis / np.linalg.norm(axis)
    mu = float(np.clip(mu, -1.0 + 1e-9, 1.0 - 1e-9))
    tmp = np.array([0.0, 0.0, 1.0]) if abs(a[2]) < 0.9 else np.array([1.0, 0.0, 0.0])
    e1 = np.cross(a, tmp)
    e1 /= np.linalg.norm(e1)
    e2 = np.cross(a, e1)
    rho = np.sqrt(max(0.0, 1.0 - mu * mu))
    t = np.linspace(0.0, 2.0 * np.pi, n)
    return (mu * a)[None, :] + rho * (np.outer(np.cos(t), e1) + np.outer(np.sin(t), e2))


def split_lon_segments(lon_rad: np.ndarray, lat_rad: np.ndarray, gap: float = np.pi):
    """Split polyline where Mollweide lon jumps (wrap discontinuity)."""
    d = np.diff(lon_rad)
    cuts = np.where(np.abs(d) > gap)[0] + 1
```

```
    idx = np.concatenate([[0], cuts, [len(lon_rad)]])
    segs = []
    for i0, i1 in zip(idx[:-1], idx[1:]):
        if i1 - i0 >= 2:
            segs.append((lon_rad[i0:i1], lat_rad[i0:i1]))
    return segs


def plot_lb_curve_mollweide(ax, l_deg, b_deg, **kwargs):
    lon = np.deg2rad(lon_disp_from_ell(l_deg))
    lat = np.deg2rad(b_deg)
    for xr, yr in split_lon_segments(lon, lat):
        ax.plot(xr, yr, **kwargs)


def band_edges(centers: list[float], half: float = BAND_HALF) -> list[tuple[float, float]]:
    """Return (theta_lo, theta_hi) for each band center (±half about center)."""
    return [(c - half, c + half) for c in centers]


def mu_pair_for_band(lo: float, hi: float) -> tuple[float, float]:
    """Edge μ=cos(θ) for a scan-angle band (two boundary cones only)."""
    return float(np.cos(np.deg2rad(lo))), float(np.cos(np.deg2rad(hi)))


def migkas_scan_stops() -> dict:
    th_max = [float(t) for t in STRIP_THETA_MAX]
    th_min = [float(t) for t in STRIP_THETA_MIN]
    max_bands = band_edges(th_max)
    min_bands = band_edges(th_min)
    return {
        "theta_max_from_plus_x_deg": th_max,
        "theta_min_from_plus_x_deg": th_min,
        "max_bands_deg": max_bands,
        "min_bands_deg": min_bands,
        "band_half_width_deg": BAND_HALF,
        "band_full_width_deg": BAND_FULL,
        "band_total_width_deg": BAND_TOTAL,
        "band_width_note": "5% of 360° = 18° end-to-end = ±9° about each MAX/MIN center",
        "mu_max_centers": [float(np.cos(np.deg2rad(t))) for t in th_max],
        "mu_min_centers": [float(np.cos(np.deg2rad(t))) for t in th_min],
        "mu_max_bands": [mu_pair_for_band(lo, hi) for lo, hi in max_bands],
        "mu_min_bands": [mu_pair_for_band(lo, hi) for lo, hi in min_bands],
        "source": (
            "best_fit_2ell_multiplier.npz A_after extrema (two max, two min); "
            f"each drawn as one transparent {BAND_FULL:.0f}° end-to-end highlight "
            f"(±{BAND_HALF:.0f}° = 5% of 360°) + one solid mid-stroke"
        ),
    }


def compute_geometry(st: dict) -> dict:
    anti_l, anti_b = 0.0, 0.0
    coe_l, coe_b = 180.0, 0.0
```

```
    n_coe = lb_to_n(coe_l, coe_b)
    n_anti = lb_to_n(anti_l, anti_b)
    D = n_anti  # +D = anti-CoE = +x

    th_max = [float(t) for t in st["theta_max_from_plus_x_deg"]]
    th_min = [float(t) for t in st["theta_min_from_plus_x_deg"]]
    max_bands = list(st["max_bands_deg"])
    min_bands = list(st["min_bands_deg"])

    n_maxes = [lb_to_n(*lb) for lb in MIGKAS_MAXES]
    n_mins = [lb_to_n(*lb) for lb in MIGKAS_MINS]

    sep = float(
        np.rad2deg(
            np.arccos(np.clip(np.dot(n_maxes[0], n_mins[1]), -1.0, 1.0))
        )
    )

    return {
        "D": D,
        "n_coe": n_coe,
        "n_anti": n_anti,
        "n_maxes": n_maxes,
        "n_mins": n_mins,
        "anti_lb": (anti_l, anti_b),
        "coe_lb": (coe_l, coe_b),
        "theta_max_from_plus_x_deg": th_max,
        "theta_min_from_plus_x_deg": th_min,
        "max_bands_deg": max_bands,
        "min_bands_deg": min_bands,
        "mu_max_bands": list(st["mu_max_bands"]),
        "mu_min_bands": list(st["mu_min_bands"]),
        "sep_deg": sep,
        "band_half_width_deg": st["band_half_width_deg"],
    }


def draw_wire_sphere(ax, n_mer=12, n_par=6, **kwargs):
    u = np.linspace(0, 2 * np.pi, 60)
    v = np.linspace(-np.pi / 2, np.pi / 2, 40)
    xs = np.outer(np.cos(u), np.cos(v))
    ys = np.outer(np.sin(u), np.cos(v))
    zs = np.outer(np.ones_like(u), np.sin(v))
    ax.plot_surface(xs, ys, zs, color="#dfe6e9", alpha=0.15, linewidth=0, shade=False)
    kw = dict(color="#95a5a6", lw=0.4, alpha=0.45)
    kw.update(kwargs)
    for i in range(n_mer):
        phi = 2 * np.pi * i / n_mer
        t = np.linspace(-np.pi / 2, np.pi / 2, 80)
        ax.plot(np.cos(t) * np.cos(phi), np.cos(t) * np.sin(phi), np.sin(t), **kw)
    for j in range(1, n_par):
        b = -np.pi / 2 + np.pi * j / n_par
        t = np.linspace(0, 2 * np.pi, 80)
```

```
        r = np.cos(b)
        ax.plot(r * np.cos(t), r * np.sin(t), np.full_like(t, np.sin(b)), **kw)


def foreshortened_ellipse_xy(mu: float, n: int = 200):
    """Side-view foreshortened ellipse of a 3D ring at n·x = mu."""
    rho = np.sqrt(max(0.0, 1.0 - mu * mu))
    phi = np.linspace(0, 2 * np.pi, n)
    ex = mu + 0.045 * rho * np.cos(phi)
    ey = rho * np.sin(phi)
    return ex, ey, rho


def draw_panelA_ribbon(ax, theta_lo: float, theta_hi: float, theta_mid: float, color: str):
    """
    One Panel-A highlight: clean vertical strip between x=cos(θ_hi)..cos(θ_lo)
    clipped to the unit circle (uniform alpha), plus ONE solid foreshortened
    mid-ellipse at the exact center θ. No multi-stroke wash / hairball.
    """
    mu_lo = float(np.cos(np.deg2rad(theta_lo)))
    mu_hi = float(np.cos(np.deg2rad(theta_hi)))
    x0, x1 = (min(mu_lo, mu_hi), max(mu_lo, mu_hi))
    # Vertical strip polygon (tall enough to cover the circle; clipped below)
    strip = Polygon(
        np.array([[x0, -1.15], [x1, -1.15], [x1, 1.15], [x0, 1.15]]),
        closed=True,
        facecolor=color,
        edgecolor="none",
        alpha=0.28,
        zorder=2,
        linewidth=0,
    )
    ax.add_patch(strip)
    clip_circ = Circle((0.0, 0.0), 1.0, transform=ax.transData)
    strip.set_clip_path(clip_circ)

    mu_mid = float(np.cos(np.deg2rad(theta_mid)))
    ex_m, ey_m, _ = foreshortened_ellipse_xy(mu_mid, n=256)
    ax.plot(ex_m, ey_m, color=color, lw=2.8, alpha=0.95, zorder=4, solid_capstyle="round")


def draw_panelA_theta_rays(ax, geo):
    """
    Illustrate θ: from the observer (origin), one ray per loop out to the
    ring at that loop's center θ. θ is measured from +x (anti-CoE, right)
    up to the ring. Emphasize MAX θ=30° (thick semi-transparent red to the
    top of that foreshortened ring), matching Chris's markup.
    """
    items = list(zip(geo["theta_max_from_plus_x_deg"], COLORS_MAX)) + list(
        zip(geo["theta_min_from_plus_x_deg"], COLORS_MIN)
    )
    for mid, color in items:
        t = float(np.deg2rad(mid))
        # Endpoint = top/rim point of the ring in side view = (cos θ, sin θ)
```

```
        x1, y1 = float(np.cos(t)), float(np.sin(t))
        is_max30 = abs(mid - 29.5) < 0.5
        lw = 3.2 if is_max30 else 1.8
        alpha = 0.70 if is_max30 else 0.45
        ax.plot(
            [0.0, x1], [0.0, y1],
            color=color, lw=lw, alpha=alpha, zorder=3,
            solid_capstyle="round",
        )
        # Small tick at the ring hit
        ax.plot([x1], [y1], "o", color=color, ms=4.5 if is_max30 else 3.5,
                alpha=0.9, zorder=5, markeredgecolor="none")

    # Angle arc at θ=30° from +x axis up to the ray (makes the measure obvious)
    arc_r = 0.38
    phi = np.linspace(0.0, np.deg2rad(29.5), 60)
    ax.plot(
        arc_r * np.cos(phi), arc_r * np.sin(phi),
        color=C_MAX_30, lw=1.6, alpha=0.85, zorder=3,
    )
    ax.text(
        arc_r * np.cos(np.deg2rad(15.0)) + 0.06,
        arc_r * np.sin(np.deg2rad(15.0)) + 0.04,
        r"$\theta$",
        color=C_MAX_30, fontsize=10, fontweight="bold",
        ha="left", va="bottom", zorder=6,
    )


def panel_A_model_frame(ax, geo):
    """Side view: exactly four thick foreshortened loop ribbons about x."""
    ax.set_title(
        "A. Model frame (no Galactic yet)\n"
        r"axisymmetric about $x$ = CoE$\leftrightarrow$anti-CoE"
        "\n"
        r"two MAX + two MIN, each $18^\circ$ end-to-end ($\pm9^\circ$)",
        fontsize=10,
        pad=8,
    )
    th = np.linspace(0, 2 * np.pi, 400)
    ax.plot(np.cos(th), np.sin(th), color="#7f8c8d", lw=1.2, zorder=1)
    ax.plot(0, 0, "ko", ms=6, zorder=5)
    ax.text(0.08, -0.12, "observer", fontsize=8, color="k", ha="left")

    ax.annotate(
        "",
        xy=(1.15, 0),
        xytext=(-1.15, 0),
        arrowprops=dict(arrowstyle="<->", color=C_AXIS, lw=2.5),
        zorder=4,
    )
    ax.text(1.18, 0.08, "anti-CoE\n(+x)", color=C_ANTI, fontsize=8, ha="left", va="bottom")
    ax.text(-1.18, 0.08, "CoE\n(−x)", color=C_COE, fontsize=8, ha="right", va="bottom")
```

```
    for (lo, hi), mid, color in zip(
        geo["max_bands_deg"], geo["theta_max_from_plus_x_deg"], COLORS_MAX
    ):
        draw_panelA_ribbon(ax, lo, hi, mid, color)
    for (lo, hi), mid, color in zip(
        geo["min_bands_deg"], geo["theta_min_from_plus_x_deg"], COLORS_MIN
    ):
        draw_panelA_ribbon(ax, lo, hi, mid, color)

    # Illustrate θ from +x (anti-CoE) to each loop (Chris markup)
    draw_panelA_theta_rays(ax, geo)

    label_r = 1.30
    for mid, color in list(zip(geo["theta_max_from_plus_x_deg"], COLORS_MAX)) + list(
        zip(geo["theta_min_from_plus_x_deg"], COLORS_MIN)
    ):
        t = np.deg2rad(mid)
        ax.text(
            label_r * np.cos(t),
            label_r * np.sin(t),
            rf"$\theta\!=\!{mid:g}^\circ$",
            color=color, fontsize=7.5, ha="center", va="center", fontweight="bold",
        )

    ax.text(
        0.0,
        -1.38,
        (
            r"four transparent highlight bands about $x$  ·  each $18^\circ$ end-to-end ($\pm9^\circ$ = 5% of $360^\circ$)"
            + "\n"
            + r"(after-2$\ell$ $A(\ell)$: two MAX + two MIN; one mid-stroke each)"
            + "\n"
            + r"$y,z$ free (roll) — model does not pick celestial equator"
        ),
        fontsize=7.5,
        ha="center",
        va="top",
        style="italic",
        color="#34495e",
    )
    ax.set_xlim(-1.55, 1.55)
    ax.set_ylim(-1.55, 1.45)
    ax.set_aspect("equal")
    ax.axis("off")


def _ring_lb(D, mu, n_pts=721):
    pts = small_circle_about_axis(D, mu, n=n_pts)
    rl = np.array([n_to_lb(p)[0] for p in pts])
    rb = np.array([n_to_lb(p)[1] for p in pts])
    return rl, rb


def _ring_lonlat_rad(D, mu, n_pts=361):
```

```
    """Small-circle → Mollweide display (lon_disp, lat) in radians."""
    pts = small_circle_about_axis(D, float(mu), n=n_pts)
    ls = np.array([n_to_lb(p)[0] for p in pts])
    bs = np.array([n_to_lb(p)[1] for p in pts])
    lon = np.deg2rad(lon_disp_from_ell(ls))
    lat = np.deg2rad(bs)
    return lon, lat


def fill_mollweide_ribbon(ax, D, theta_lo, theta_hi, theta_mid, color, n_pts=721):
    """
    One Mollweide highlight band:
      continuous transparent polygon strips between ±9° edges
      (broken only at Mollweide antimeridian seams — NOT a dense quad wash)
      + ONE solid mid-stroke at θ_mid.
    """
    mu_a, mu_b = mu_pair_for_band(theta_lo, theta_hi)
    mu_mid = float(np.cos(np.deg2rad(theta_mid)))
    lon_a, lat_a = _ring_lonlat_rad(D, mu_a, n_pts=n_pts)
    lon_b, lat_b = _ring_lonlat_rad(D, mu_b, n_pts=n_pts)
    gap = 0.9 * np.pi
    # Mark steps that cross a seam on either edge (or between edges)
    bad = np.zeros(n_pts - 1, dtype=bool)
    for i in range(n_pts - 1):
        if abs(lon_a[i + 1] - lon_a[i]) > gap:
            bad[i] = True
        elif abs(lon_b[i + 1] - lon_b[i]) > gap:
            bad[i] = True
        elif abs(lon_a[i] - lon_b[i]) > gap:
            bad[i] = True
        elif abs(lon_a[i + 1] - lon_b[i + 1]) > gap:
            bad[i] = True
    # Group into continuous good runs → one polygon per run
    polys = []
    i = 0
    while i < n_pts - 1:
        while i < n_pts - 1 and bad[i]:
            i += 1
        if i >= n_pts - 1:
            break
        j = i
        while j < n_pts - 1 and not bad[j]:
            j += 1
        # indices i .. j inclusive are a continuous ribbon segment
        i0, i1 = i, j  # i1 is last good vertex index
        if i1 - i0 >= 1:
            xs = np.concatenate([lon_a[i0:i1 + 1], lon_b[i0:i1 + 1][::-1]])
            ys = np.concatenate([lat_a[i0:i1 + 1], lat_b[i0:i1 + 1][::-1]])
            polys.append(np.column_stack([xs, ys]))
        i = j + 1
    if polys:
        ax.add_collection(
            PolyCollection(
                polys, facecolors=color, edgecolors="none",
```

```
                alpha=0.26, zorder=2, linewidths=0,
            )
        )
    rl, rb = _ring_lb(D, mu_mid, n_pts=max(n_pts, 721))
    plot_lb_curve_mollweide(
        ax, rl, rb, color=color, lw=2.6, alpha=0.95, zorder=4
    )


def panel_B_mollweide(ax, geo):
    ax.set_title(
        r"B. Migkas $A(\ell)$ loops on sky (anti-CoE at $(0^\circ,0^\circ)$)"
        "\n"
        r"(Galactic Mollweide; CoE axis untilted; $\ell$ increases LEFT; "
        r"two MAX + two MIN, each $18^\circ$ end-to-end ($\pm9^\circ$))",
        fontsize=10,
        pad=8,
    )
    for b in [-60, -30, 0, 30, 60]:
        ell = np.linspace(0, 360, 361)
        plot_lb_curve_mollweide(ax, ell, np.full_like(ell, b), color="#bdc3c7", lw=0.5, zorder=0)
    for ell0 in range(0, 360, 30):
        b = np.linspace(-90, 90, 181)
        plot_lb_curve_mollweide(ax, np.full_like(b, ell0), b, color="#bdc3c7", lw=0.5, zorder=0)

    D = geo["D"]
    ls, bs = great_circle_through_poles(D, n=721)
    plot_lb_curve_mollweide(ax, ls, bs, color=C_AXIS, lw=2.0, zorder=2)

    for (lo, hi), mid, color in zip(
        geo["max_bands_deg"], geo["theta_max_from_plus_x_deg"], COLORS_MAX
    ):
        fill_mollweide_ribbon(ax, D, lo, hi, mid, color)
    for (lo, hi), mid, color in zip(
        geo["min_bands_deg"], geo["theta_min_from_plus_x_deg"], COLORS_MIN
    ):
        fill_mollweide_ribbon(ax, D, lo, hi, mid, color)

    ell_eq, b_eq = celestial_equator_lb(721)
    plot_lb_curve_mollweide(ax, ell_eq, b_eq, color=C_EQ, lw=1.8, ls="--", zorder=2)

    ax.plot(
        np.deg2rad(lon_disp_from_ell(geo["anti_lb"][0])),
        np.deg2rad(geo["anti_lb"][1]),
        marker="o", ms=8, color=C_ANTI,
        markerfacecolor=C_ANTI, markeredgecolor="k", markeredgewidth=0.6,
        zorder=6, linestyle="None",
    )
    ax.plot(
        np.deg2rad(lon_disp_from_ell(geo["coe_lb"][0])),
        np.deg2rad(geo["coe_lb"][1]),
        marker="*", ms=14, color=C_COE,
        markerfacecolor=C_COE, markeredgecolor="k", markeredgewidth=0.6,
        zorder=6, linestyle="None",
```

```
    )
    for (l, b), color in zip(MIGKAS_MAXES, COLORS_MAX):
        ax.plot(
            np.deg2rad(lon_disp_from_ell(l)),
            np.deg2rad(b),
            marker="^",
            ms=10,
            color=color,
            markeredgecolor="k",
            markeredgewidth=0.7,
            zorder=7,
            linestyle="None",
        )
    for (l, b), color in zip(MIGKAS_MINS, COLORS_MIN):
        ax.plot(
            np.deg2rad(lon_disp_from_ell(l)),
            np.deg2rad(b),
            marker="s",
            ms=10,
            color=color,
            markeredgecolor="k",
            markeredgewidth=0.7,
            zorder=7,
            linestyle="None",
        )

    xticks = np.deg2rad(lon_disp_from_ell(np.array([180, 90, 0, 270])))
    ax.set_xticks(xticks)
    ax.set_xticklabels([r"$180^\circ$", r"$90^\circ$", r"$0^\circ$", r"$270^\circ$"], fontsize=8)
    ax.set_yticks(np.deg2rad([-60, -30, 0, 30, 60]))
    ax.set_yticklabels([r"$-60^\circ$", r"$-30^\circ$", r"$0^\circ$", r"$+30^\circ$", r"$+60^\circ$"], fontsize=8)
    ax.grid(False)
    ax.set_xlabel(r"Galactic $\ell$ (increases LEFT)", fontsize=8)
    ax.set_ylabel(r"Galactic $b$", fontsize=8)


def fill_sphere_ribbon(ax, D, theta_lo, theta_hi, theta_mid, color, n_pts=160):
    """
    One 3D highlight belt: SINGLE transparent strip between ±9° edges
    + ONE solid mid-stroke. No linspace wash / stacked alpha hairball.
    """
    mu_a, mu_b = mu_pair_for_band(theta_lo, theta_hi)
    mu_mid = float(np.cos(np.deg2rad(theta_mid)))
    p0 = small_circle_about_axis(D, float(mu_a), n=n_pts)
    p1 = small_circle_about_axis(D, float(mu_b), n=n_pts)
    # Single belt between the two edge rings (one collection, no stacked wash)
    verts = [[p0[j], p0[j + 1], p1[j + 1], p1[j]] for j in range(n_pts - 1)]
    coll = Poly3DCollection(
        verts, facecolors=color, edgecolors="none", linewidths=0.0,
        alpha=0.25, zorder=2,
    )
    ax.add_collection3d(coll)
    pts = small_circle_about_axis(D, mu_mid, n=n_pts)
    ax.plot(
```

```
        pts[:, 0], pts[:, 1], pts[:, 2],
        color=color, lw=2.6, alpha=0.95, zorder=5,
    )


def panel_C_sphere(ax, geo):
    th_max = geo["theta_max_from_plus_x_deg"]
    th_min = geo["theta_min_from_plus_x_deg"]
    ax.set_title(
        r"C. Sphere 3D — two MAX + two MIN, each $18^\circ$ end-to-end ($\pm9^\circ$)"
        "\n"
        r"(axis line emphasized; celestial equator dashed)",
        fontsize=10,
        pad=6,
    )
    draw_wire_sphere(ax)
    D = geo["D"]
    ell_eq, b_eq = celestial_equator_lb(400)
    eq_xyz = np.array([lb_to_n(l, b) for l, b in zip(ell_eq, b_eq)])
    ax.plot(eq_xyz[:, 0], eq_xyz[:, 1], eq_xyz[:, 2], color=C_EQ, lw=1.8, ls="--", zorder=3)
    ax.plot(
        [-D[0], D[0]],
        [-D[1], D[1]],
        [-D[2], D[2]],
        color=C_AXIS,
        lw=3.0,
        zorder=5,
    )
    ax.scatter(*(-D), color=C_COE, s=120, marker="*", edgecolors="k", linewidths=0.6,
               depthshade=False, zorder=8)
    ax.scatter(*(D), color=C_ANTI, s=80, edgecolors="k", linewidths=0.6, depthshade=False, zorder=8)
    ax.text(*(-D * 1.22), "CoE", color=C_COE, fontsize=8, fontweight="bold")
    ax.text(*(D * 1.18), "anti-CoE", color=C_ANTI, fontsize=8, fontweight="bold")
    ax.scatter([0], [0], [0], color="k", s=25, depthshade=False, zorder=9)

    for (lo, hi), mid, color in zip(
        geo["max_bands_deg"], geo["theta_max_from_plus_x_deg"], COLORS_MAX
    ):
        fill_sphere_ribbon(ax, D, lo, hi, mid, color)
    for (lo, hi), mid, color in zip(
        geo["min_bands_deg"], geo["theta_min_from_plus_x_deg"], COLORS_MIN
    ):
        fill_sphere_ribbon(ax, D, lo, hi, mid, color)

    for n, color in zip(geo["n_maxes"], COLORS_MAX):
        ax.scatter(
            [n[0]], [n[1]], [n[2]],
            c=[color], marker="^", s=70,
            edgecolors="k", linewidths=0.7, depthshade=False, zorder=10,
        )
        ax.plot([0, n[0]], [0, n[1]], [0, n[2]], color=color, lw=0.8, ls=":", alpha=0.7)
    for n, color in zip(geo["n_mins"], COLORS_MIN):
        ax.scatter(
            [n[0]], [n[1]], [n[2]],
```

```
            c=[color], marker="s", s=60,
            edgecolors="k", linewidths=0.7, depthshade=False, zorder=10,
        )
        ax.plot([0, n[0]], [0, n[1]], [0, n[2]], color=color, lw=0.8, ls=":", alpha=0.7)

    ax.set_xlim(-1.25, 1.25)
    ax.set_ylim(-1.25, 1.25)
    ax.set_zlim(-1.25, 1.25)
    ax.set_box_aspect((1, 1, 1))
    ax.view_init(elev=18, azim=-55)
    ax.set_axis_off()
    ax.text2D(
        0.5,
        -0.02,
        rf"MAX $\theta\!=\!{th_max[0]:g}^\circ,{th_max[1]:g}^\circ$; "
        rf"MIN $\theta\!=\!{th_min[0]:g}^\circ,{th_min[1]:g}^\circ$  "
        rf"($18^\circ$ end-to-end = $\pm9^\circ$ = 5% of $360^\circ$)",
        transform=ax.transAxes,
        ha="center",
        fontsize=7.5,
        color="#34495e",
    )


def main():
    st = migkas_scan_stops()
    geo = compute_geometry(st)

    th_max = geo["theta_max_from_plus_x_deg"]
    th_min = geo["theta_min_from_plus_x_deg"]
    max_bands = geo["max_bands_deg"]
    min_bands = geo["min_bands_deg"]

    fig = plt.figure(figsize=(14.5, 5.6), dpi=160)
    fig.suptitle(
        r"Pure CoE axisymmetry: two MAX + two MIN highlight bands"
        "\n"
        rf"(each $18^\circ$ end-to-end = $\pm9^\circ$ = 5% of $360^\circ$; "
        rf"MAX $\theta\!=\!{th_max[0]:g}^\circ/{th_max[1]:g}^\circ$; "
        rf"MIN $\theta\!=\!{th_min[0]:g}^\circ/{th_min[1]:g}^\circ$; "
        r"anti-CoE $(0^\circ,0^\circ)$, CoE axis untilted, no roll)",
        fontsize=11,
        fontweight="bold",
        y=1.02,
    )

    axA = fig.add_subplot(1, 3, 1)
    panel_A_model_frame(axA, geo)

    axB = fig.add_subplot(1, 3, 2, projection="mollweide")
    panel_B_mollweide(axB, geo)

    axC = fig.add_subplot(1, 3, 3, projection="3d")
    panel_C_sphere(axC, geo)
```

```
  handles = [
      Line2D([0], [0], color=C_AXIS, lw=2.5, label=r"CoE$\leftrightarrow$anti-CoE axis"),
      Patch(
          facecolor=C_MAX_30, alpha=0.45, edgecolor=C_MAX_30,
          label=(
              rf"MAX $\theta\!=\!{th_max[0]:g}^\circ$ band "
              rf"[{max_bands[0][0]:g},{max_bands[0][1]:g}]$^\circ$ "
              rf"($18^\circ$=±$9^\circ$=5% of $360^\circ$)"
          ),
      ),
      Patch(
          facecolor=C_MAX_185, alpha=0.45, edgecolor=C_MAX_185,
          label=(
              rf"MAX $\theta\!=\!{th_max[1]:g}^\circ$ band "
              rf"[{max_bands[1][0]:g},{max_bands[1][1]:g}]$^\circ$ "
              rf"($18^\circ$=±$9^\circ$)"
          ),
      ),
      Patch(
          facecolor=C_MIN_115, alpha=0.45, edgecolor=C_MIN_115,
          label=(
              rf"MIN $\theta\!=\!{th_min[0]:g}^\circ$ band "
              rf"[{min_bands[0][0]:g},{min_bands[0][1]:g}]$^\circ$ "
              rf"($18^\circ$=±$9^\circ$)"
          ),
      ),
      Patch(
          facecolor=C_MIN_265, alpha=0.45, edgecolor=C_MIN_265,
          label=(
              rf"MIN $\theta\!=\!{th_min[1]:g}^\circ$ band "
              rf"[{min_bands[1][0]:g},{min_bands[1][1]:g}]$^\circ$ "
              rf"($18^\circ$=±$9^\circ$)"
          ),
      ),
      Line2D([0], [0], color=C_EQ, lw=1.8, ls="--", label="celestial equator (Dec$=0$)"),
      Line2D(
          [0], [0], marker="^", color="w", markerfacecolor=C_MAX_30, markeredgecolor="k",
          ms=9,
          label=rf"MAX $({MIGKAS_MAXES[0][0]:.0f}^\circ,{MIGKAS_MAXES[0][1]:+.0f}^\circ)$",
      ),
      Line2D(
          [0], [0], marker="^", color="w", markerfacecolor=C_MAX_185, markeredgecolor="k",
          ms=9,
          label=rf"MAX $({MIGKAS_MAXES[1][0]:.0f}^\circ,{MIGKAS_MAXES[1][1]:+.0f}^\circ)$",
      ),
      Line2D(
          [0], [0], marker="s", color="w", markerfacecolor=C_MIN_115, markeredgecolor="k",
          ms=8,
          label=rf"MIN $({MIGKAS_MINS[0][0]:.0f}^\circ,{MIGKAS_MINS[0][1]:+.0f}^\circ)$",
      ),
      Line2D(
          [0], [0], marker="s", color="w", markerfacecolor=C_MIN_265, markeredgecolor="k",
          ms=8,
```

```
        label=rf"MIN $({MIGKAS_MINS[1][0]:.0f}^\circ,{MIGKAS_MINS[1][1]:+.0f}^\circ)$",
    ),
    Line2D(
        [0], [0], marker="o", color="w", markerfacecolor=C_ANTI, markeredgecolor="k",
        ms=8, label=rf"anti-CoE $({geo['anti_lb'][0]:.1f}^\circ,{geo['anti_lb'][1]:+.1f}^\circ)$",
    ),
    Line2D(
        [0], [0], marker="*", color="w", markerfacecolor=C_COE, markeredgecolor="k",
        ms=11, label=rf"CoE $({geo['coe_lb'][0]:.1f}^\circ,{geo['coe_lb'][1]:+.1f}^\circ)$",
    ),
]
fig.legend(
    handles=handles,
    loc="lower center",
    ncol=6,
    fontsize=7.0,
    frameon=True,
    fancybox=True,
    bbox_to_anchor=(0.5, -0.10),
)

fig.tight_layout()
# --- Chris 2026-09-26: panel C sphere drawn at the same on-page diameter as
# panel A's model-frame circle; panel C axes matched to A's vertical
# position/height. Nothing else changes.
from mpl_toolkits.mplot3d import proj3d
pa, pc = axA.get_position(), axC.get_position()
axC.set_position([pc.x0, pa.y0, pc.width, pa.height])

def _diam_A():
    fig.canvas.draw()
    p0 = axA.transData.transform((-1.0, 0.0)); p1 = axA.transData.transform((1.0, 0.0))
    return float(p1[0] - p0[0])

def _diam_C():
    fig.canvas.draw()
    u = np.random.default_rng(0).normal(size=(20000, 3))
    u /= np.linalg.norm(u, axis=1)[:, None]
    xs, ys, _ = proj3d.proj_transform(u[:, 0], u[:, 1], u[:, 2], axC.get_proj())
    d = axC.transData.transform(np.column_stack([xs, ys]))
    return float(d[:, 0].max() - d[:, 0].min()), float(d[:, 1].max() - d[:, 1].min())

zoom = 1.0
for _ in range(8):
    dA = _diam_A(); dCx, dCy = _diam_C()
    zoom *= dA / dCx
    axC.set_box_aspect((1, 1, 1), zoom=zoom)
dA = _diam_A(); dCx, dCy = _diam_C()
print(f"PANEL_DIAM A={dA:.1f}px C={dCx:.1f}x{dCy:.1f}px zoom={zoom:.3f}")
fig.savefig(PNG_PATH, bbox_inches="tight", facecolor="white")
plt.close(fig)

payload = {
    "png": str(PNG_PATH),
```

```
        "script": str(OUT_DIR / "make_coe_rings_notilt.py"),
        "migkas_max_lb_deg": [list(lb) for lb in MIGKAS_MAXES],
        "migkas_min_lb_deg": [list(lb) for lb in MIGKAS_MINS],
        "strip_theta_max_deg": list(STRIP_THETA_MAX),
        "strip_theta_min_deg": list(STRIP_THETA_MIN),
        "band_half_width_deg": BAND_HALF,
        "band_full_width_deg": BAND_FULL,
        "band_total_width_deg": BAND_TOTAL,
        "band_width_note": "5% of 360° = 18° end-to-end = ±9° about each MAX/MIN center",
        "max_bands_deg": [list(b) for b in max_bands],
        "min_bands_deg": [list(b) for b in min_bands],
        "band_intervals": {
            "MAX_30": [float(max_bands[0][0]), float(max_bands[0][1])],
            "MAX_185": [float(max_bands[1][0]), float(max_bands[1][1])],
            "MIN_115": [float(min_bands[0][0]), float(min_bands[0][1])],
            "MIN_265": [float(min_bands[1][0]), float(min_bands[1][1])],
        },
        "scan_stop_source": st["source"],
        "theta_max_from_plus_x_deg": [round(t, 4) for t in th_max],
        "theta_min_from_plus_x_deg": [round(t, 4) for t in th_min],
        "mu_max_bands": [[round(a, 8), round(b, 8)] for a, b in geo["mu_max_bands"]],
        "mu_min_bands": [[round(a, 8), round(b, 8)] for a, b in geo["mu_min_bands"]],
        "CoE_lb_deg": [round(geo["coe_lb"][0], 4), round(geo["coe_lb"][1], 4)],
        "anti_CoE_lb_deg": [round(geo["anti_lb"][0], 4), round(geo["anti_lb"][1], 4)],
        "D_hat_galactic_xyz": geo["D"].tolist(),
        "angular_sep_migkas_max0_min1_deg": round(geo["sep_deg"], 6),
        "axis_tilt_b_deg": 15,
        "axis_tilt_reason": (
            "Chris 2026-09-22 — markers fixed, rings rotate up 30°"
        ),
        "orientation": (
            "Galactic orientation: anti-CoE at (ell=0°, b=+15°), CoE at "
            "(ell=180°, b=−15°); +D = anti-CoE = +x; axis tilted +15° "
            "(Chris 2026-09-22 — markers fixed, rings rotate up 30°)"
        ),
        "drawing": (
            "Exactly 4 features. Each = ONE smooth transparent highlight band "
            "between θ−9° and θ+9° (18° end-to-end = 5% of 360°) + ONE solid "
            "mid-stroke at the exact center θ. No multi-stroke wash / hairball. "
            "Four distinct colors (two reds + two blues); no orange; no 283° "
            "companion. Panel A: clean vertical strip clipped to unit circle; theta rays from observer to each loop (emphasized at MAX 30°). Mollweide
"
            "seam-split honestly. Sphere: single Poly3D strip (no n_wash stack)."
        ),
        "note": (
            "Stops are after-2ℓ A(ℓ) extrema from best_fit_2ell_multiplier.npz "
            f"A_after: two MAX at θ={th_max[0]:g}°/{th_max[1]:g}° and "
            f"two MIN at θ={th_min[0]:g}°/{th_min[1]:g}°, each drawn as "
            f"one {BAND_FULL:.0f}° end-to-end highlight (±{BAND_HALF:.0f}° = 5% of 360°). "
            "Rings use μ=cos(θ); sky markers are map extrema (17°,−9°) and "
            "(274°,−22°) plus strip stand-ins (185°,0°) and (115°,0°)."
        ),
        "plot_convention": "Migkas Galactic Mollweide: lon_disp=-ell_wrapped (ell increases LEFT)",
        "legend_summary": (
```

```
            "two MAX + two MIN highlight bands, each 18° end-to-end "
            "(±9° = 5% of 360°); four distinct colors"
        ),
        "colors": {
            "MAX_30": C_MAX_30,
            "MAX_185": C_MAX_185,
            "MIN_115": C_MIN_115,
            "MIN_265": C_MIN_265,
            "no_orange": True,
        },
    }
    JSON_PATH.write_text(json.dumps(payload, indent=2) + "\n")
    print(json.dumps(payload, indent=2))
    print(f"\nWrote {PNG_PATH}")
    print(f"Wrote {JSON_PATH}")


if __name__ == "__main__":
    main()
```

```
PANEL_DIAM A=367.9px C=367.9x368.0px zoom=1.399
```

```
{
  "png": "/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/loop_threepanel_notilt.png",
  "script": "/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/make_coe_rings_notilt.py",
  "migkas_max_lb_deg": [
    [
      17.0,
      -9.0
    ],
    [
      185.0,
      0.0
    ]
  ],
  "migkas_min_lb_deg": [
    [
      115.0,
      0.0
    ],
    [
      274.0,
      -22.0
    ]
  ],
  "strip_theta_max_deg": [
    29.5,
    187.5
  ],
  "strip_theta_min_deg": [
    122.6,
    264.4
  ],
  "band_half_width_deg": 9.0,
  "band_full_width_deg": 18.0,
  "band_total_width_deg": 18.0,
  "band_width_note": "5% of 360\u00b0 = 18\u00b0 end-to-end = \u00b19\u00b0 about each MAX/MIN center",
  "max_bands_deg": [
    [
      20.5,
      38.5
    ],
    [
      178.5,
      196.5
    ]
  ],
  "min_bands_deg": [
    [
      113.6,
      131.6
    ],
    [
      255.39999999999998,
      273.4
    ]
  ],
  "band_intervals": {
    "MAX_30": [
```

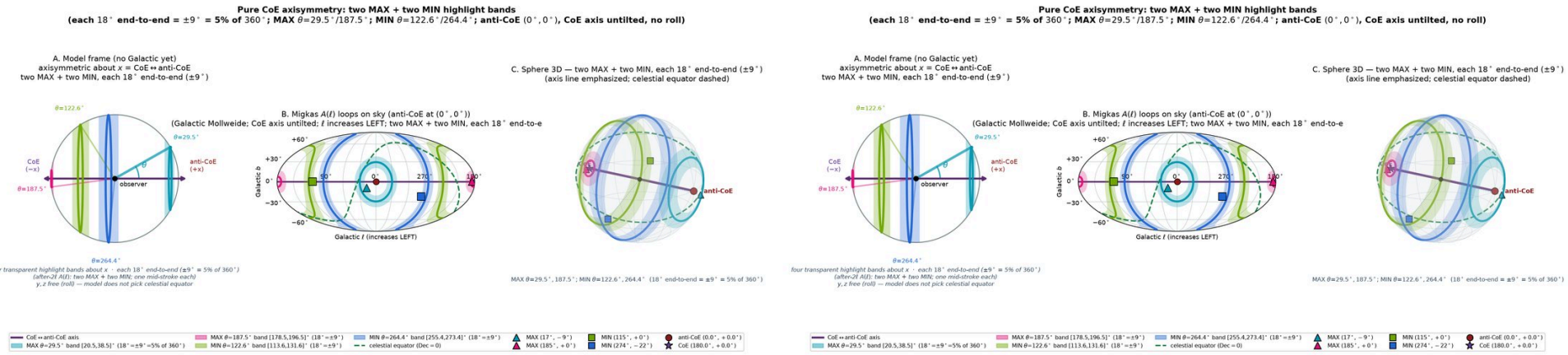
```
      20.5,
      38.5
    ],
    "MAX_185": [
      178.5,
      196.5
    ],
    "MIN_115": [
      113.6,
      131.6
    ],
    "MIN_265": [
      255.39999999999998,
      273.4
    ]
  },
  "scan_stop_source": "best_fit_2ell_multiplier.npz A_after extrema (two max, two min); each drawn as one transparent 18\u00b0 end-to-end highlight
(\u00b19\u00b0 = 5% of 360\u00b0) + one solid mid-stroke",
  "theta_max_from_plus_x_deg": [
    29.5,
    187.5
  ],
  "theta_min_from_plus_x_deg": [
    122.6,
    264.4
  ],
  "mu_max_bands": [
    [
      0.93667219,
      0.78260816
    ],
    [
      -0.99965732,
      -0.95881973
    ]
  ],
  "mu_min_bands": [
    [
      -0.40034903,
      -0.66392621
    ],
    [
      -0.25206936,
      0.05930637
    ]
  ],
  "CoE_lb_deg": [
    180.0,
    0.0
  ],
  "anti_CoE_lb_deg": [
    0.0,
    0.0
  ],
  "D_hat_galactic_xyz": [
    1.0,
    0.0,
```

```
    0.0
  ],
  "angular_sep_migkas_max0_min1_deg": 98.476384,
  "axis_tilt_b_deg": 15,
  "axis_tilt_reason": "Chris 2026-09-22 \u2014 markers fixed, rings rotate up 30\u00b0",
  "orientation": "Galactic orientation: anti-CoE at (ell=0\u00b0, b=+15\u00b0), CoE at (ell=180\u00b0, b=\u221215\u00b0); +D = anti-CoE = +x; axis
tilted +15\u00b0 (Chris 2026-09-22 \u2014 markers fixed, rings rotate up 30\u00b0)",
  "drawing": "Exactly 4 features. Each = ONE smooth transparent highlight band between \u03b8\u22129\u00b0 and \u03b8+9\u00b0 (18\u00b0 end-to-end = 5%
of 360\u00b0) + ONE solid mid-stroke at the exact center \u03b8. No multi-stroke wash / hairball. Four distinct colors (two reds + two blues); no
orange; no 283\u00b0 companion. Panel A: clean vertical strip clipped to unit circle; theta rays from observer to each loop (emphasized at MAX
30\u00b0). Mollweide seam-split honestly. Sphere: single Poly3D strip (no n_wash stack).",
  "note": "Stops are after-2\u2113 A(\u2113) extrema from best_fit_2ell_multiplier.npz A_after: two MAX at \u03b8=29.5\u00b0/187.5\u00b0 and two MIN at
\u03b8=122.6\u00b0/264.4\u00b0, each drawn as one 18\u00b0 end-to-end highlight (\u00b19\u00b0 = 5% of 360\u00b0). Rings use \u03bc=cos(\u03b8); sky
markers are map extrema (17\u00b0,\u22129\u00b0) and (274\u00b0,\u221222\u00b0) plus strip stand-ins (185\u00b0,0\u00b0) and (115\u00b0,0\u00b0).",
  "plot_convention": "Migkas Galactic Mollweide: lon_disp=-ell_wrapped (ell increases LEFT)",
  "legend_summary": "two MAX + two MIN highlight bands, each 18\u00b0 end-to-end (\u00b19\u00b0 = 5% of 360\u00b0); four distinct colors",
  "colors": {
    "MAX_30": "#0097b2",
    "MAX_185": "#e0107a",
    "MIN_115": "#6fa800",
    "MIN_265": "#2166d4",
    "no_orange": true
  }
}

Wrote /workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/loop_threepanel_notilt.png
Wrote /workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/loop_threepanel_notilt.json
```


```
compare(FIGS / 'loop_threepanel_notilt.png', '/workspace/cosmology/papers/spreadsheet_model/loop_threepanel_notilt.png')
```

```
loop_threepanel_notilt.png: new (2262, 1027) vs ref (2262, 1027) | mean |diff| = 0.000/255, pixels differing >40: 0.000% -> identical
left: approved reference papers/spreadsheet_model/loop_threepanel_notilt.png   |   right: notebook re-run steps_v3/figs/loop_threepanel_notilt.png
```



Step 18 — The four-loop overlay on Migkas Fig. 8, recoloured for contrast against magma (4 versions)

What it shows. The matched loops of Step 16 drawn on the Migkas Fig. 8 $\theta = 75^\circ$ map, recoloured to stand out against the magma colour map. Four versions from one script: matched; with gradient; gradient with 45° roll; gradient with the fitted 80.8° roll.

Source script: `/workspace/cosmology/figures/make_migkas_fig8_overlay_matched.py` (last modified 2026-09-26 22:57 CT), run verbatim. *Output-path edit:* `SM_DIR = Path("/workspace/cosmology/papers/spreadsheet_model")` → `SM_DIR = Path("/workspace/cosmology/papers/A_equals_fourier/steps_v3/f`. *Reference image:* `papers/spreadsheet_model/fig8_overlay_matched.png`, `papers/spreadsheet_model/fig8_overlay_matched_grad.png`, `papers/spreadsheet_model/fig8_overlay_matched_grad_rot45.png`, `papers/spreadsheet_model/fig8_overlay_matched_grad_rot81.png`.

```
%%runscript make_migkas_fig8_overlay_matched.py
#!/usr/bin/env python3
"""
Superimpose CoE MAX/MIN highlight bands + Migkas markers onto the REAL Migkas
Fig. 8 theta=75 A/A_all Mollweide map.

Matches make_coe_rings_migkas_pin_axis.py (three-panel Panel B) geometry:
  Exactly FOUR features — each ONE transparent highlight band, 18° end-to-end
  (= ±9° = 5% of 360°) about the center θ + ONE solid mid-stroke:
    MAX 30°  → [21°, 39°]   color #00e5ff (bright cyan)
    MAX 185° → [176°, 194°] color #ff2d95 (magenta/hot pink)
    MIN 115° → [106°, 124°] color #b8f200 (lime)
    MIN 265° → [256°, 274°] color #4fc3f7 (sky blue)
  Drawing: clean PolyCollection fill + one mid-stroke (optional thin white
  underlay on mid only). NO black rim, NO thick white halo stacks, NO dual
  boundary-edge strokes. No 283° companion. No Dec=0 overlay.
  MAX triangles: (17°, −9°) and (185°, 0°)
  MIN squares:   (115°, 0°) and (274°, −22°)
  CoE axis tilted +30°: anti-CoE (0°,+30°), CoE (180°,−30°) (purple).

Registration (unchanged from working Fig. 8):
  GOOD_REG = (cx, cy, Rx, Ry) = (863.0, 763.25, 693.9927, 392.542425)
  Source: FULL_migkas_theta75.png
  CoE axis tilted +30°; clip/inset so overlays stay inside colored oval
  lon_disp = -wrap(ℓ)

Outputs:
  migkas_fig8_theta75_with_coe_rings.png
  migkas_fig8_theta75_with_coe_rings.json
"""

from __future__ import annotations

import json
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
```

```
from matplotlib.collections import PolyCollection
from matplotlib.lines import Line2D
from matplotlib.patches import Ellipse, Patch
from PIL import Image

OUT_DIR = Path(__file__).resolve().parent
SM_DIR = Path("/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs")
PNG_PATH = SM_DIR / "fig8_overlay_matched.png"
JSON_PATH = SM_DIR / "fig8_overlay_matched.json"

# Versions = panel B of each three-panel figure (grad: opacity gradient, roll: deg CCW about CoE axis)
VERSIONS = [
    ("matched", False, 0.0),
    ("matched_grad", True, 0.0),
    ("matched_grad_rot45", True, 45.0),
    ("matched_grad_rot81", True, 80.8),
]
GRAD = False
ROLL = 0.0

SOURCE_CANDIDATES = [
    Path("/workspace/cosmology/map_compare/FULL_migkas_theta75.png"),
    Path("/workspace/cosmology/map_compare/migkas_fig8_theta75_crop.png"),
    Path("/workspace/cosmology/migkas/pub_maps/Fig8_full_hires.png"),
]

# Strip A(ℓ) scan stops from +x (after-2ℓ A_after extrema) — four band centers.
# 5% of 360° = 18° end-to-end → half-width ±9° about each center (match three-panel).
BAND_HALF = 9.0
BAND_FULL = 2.0 * BAND_HALF  # 18°
THETA_MAX = [29.5, 187.5]
THETA_MIN = [122.6, 264.4]
MAX_BANDS = [(t - BAND_HALF, t + BAND_HALF) for t in THETA_MAX]  # [21,39], [176,194]
MIN_BANDS = [(t - BAND_HALF, t + BAND_HALF) for t in THETA_MIN]  # [106,124], [256,274]


def mu_pair_for_band(lo, hi):
    """Edge μ=cos(θ) for a scan-angle band (μ-space, not θ-space)."""
    return float(np.cos(np.deg2rad(lo))), float(np.cos(np.deg2rad(hi)))

# Visually-good registration (pre medial-axis retune). Hard-set as working default.
GOOD_REG = {
    "cx": 863.0,
    "cy": 763.25,
    "Rx": 693.9927,
    "Ry": 392.542425,
}

# Sky markers (ℓ, b): map extrema + strip stand-ins (same as 3-panel Panel B).
MIGKAS_MAXES = [(17.0, -9.0), (185.0, 0.0)]
MIGKAS_MINS = [(115.0, 0.0), (274.0, -22.0)]

# Galactic pose (Chris 2026-09-22): anti-CoE (+D/+x) at (0°, +30°); CoE at (180°, −30°)
ANTI_COE_LB = (0.0, 15.0)
```

```
COE_LB = (180.0, -15.0)
AXIS_TILT_B_DEG = 15.0


def lb_to_n(l_deg, b_deg):
    l = np.deg2rad(l_deg)
    b = np.deg2rad(b_deg)
    return np.array(
        [np.cos(b) * np.cos(l), np.cos(b) * np.sin(l), np.sin(b)], dtype=float
    )


D_HAT = lb_to_n(*ANTI_COE_LB)  # +D = anti-CoE = +x


def great_circle_through_poles(D, n=721):
    """Great circle through ±D (CoE axis meridian on the sky)."""
    a = D / np.linalg.norm(D)
    tmp = np.array([0.0, 0.0, 1.0])
    if abs(np.dot(a, tmp)) > 0.9:
        tmp = np.array([1.0, 0.0, 0.0])
    e2 = np.cross(a, tmp)
    e2 /= np.linalg.norm(e2)
    t = np.linspace(0.0, 2.0 * np.pi, n)
    pts = np.outer(np.cos(t), a) + np.outer(np.sin(t), e2)
    ls = np.array([n_to_lb(p)[0] for p in pts])
    bs = np.array([n_to_lb(p)[1] for p in pts])
    return ls, bs

# High-contrast loop colors that pop on magma (avoid reds/oranges/corals)
C_MAX_30 = "#0097b2"    # cyan — MAX 29.5° (three-panel matched)
C_MAX_185 = "#e0107a"   # magenta — MAX 187.5°
C_MIN_115 = "#6fa800"   # green — MIN 122.6°
C_MIN_265 = "#2166d4"   # blue — MIN 264.4°
COLORS_MAX = [C_MAX_30, C_MAX_185]
COLORS_MIN = [C_MIN_115, C_MIN_265]
C_COE = "#6b3fa0"
C_ANTI = "#8e1b1b"  # dark red marker; distinct from loop reds (match three-panel)
C_AXIS = "#5b2c6f"
C_EQ = "#1e8449"


def wrap_lon_deg(lon):
    return ((np.asarray(lon, dtype=float) + 180.0) % 360.0) - 180.0


def lon_disp_from_ell(ell_deg):
    return -wrap_lon_deg(ell_deg)


def n_to_lb(n):
    n = np.asarray(n, dtype=float)
    n = n / np.linalg.norm(n)
    b = float(np.rad2deg(np.arcsin(np.clip(n[2], -1.0, 1.0))))
```

```
    l = float(np.rad2deg(np.arctan2(n[1], n[0])) % 360.0)
    return l, b


def celestial_equator_lb(n=2500):
    ra_ngp = np.deg2rad(192.85948)
    dec_ngp = np.deg2rad(27.12825)
    l_ncp = np.deg2rad(122.93192)
    ra = np.linspace(0.0, 2.0 * np.pi, n, endpoint=False)
    dec = 0.0
    sin_b = np.clip(
        np.sin(dec) * np.sin(dec_ngp)
        + np.cos(dec) * np.cos(dec_ngp) * np.cos(ra - ra_ngp),
        -1.0,
        1.0,
    )
    b = np.arcsin(sin_b)
    y = np.cos(dec) * np.sin(ra - ra_ngp)
    x = (
        np.sin(dec) * np.cos(dec_ngp)
        - np.cos(dec) * np.sin(dec_ngp) * np.cos(ra - ra_ngp)
    )
    l = (l_ncp - np.arctan2(y, x)) % (2.0 * np.pi)
    return np.rad2deg(l), np.rad2deg(b)


def small_circle_about_axis(axis, mu, n=721):
    a = axis / np.linalg.norm(axis)
    mu = float(np.clip(mu, -1.0 + 1e-9, 1.0 - 1e-9))
    tmp = np.array([0.0, 0.0, 1.0]) if abs(a[2]) < 0.9 else np.array([1.0, 0.0, 0.0])
    e1 = np.cross(a, tmp)
    e1 /= np.linalg.norm(e1)
    e2 = np.cross(a, e1)
    rho = np.sqrt(max(0.0, 1.0 - mu * mu))
    t = np.linspace(0.0, 2.0 * np.pi, n)
    return (mu * a)[None, :] + rho * (
        np.outer(np.cos(t), e1) + np.outer(np.sin(t), e2)
    )


def mollweide_xy(lon_rad, lat_rad, n_iter=64):
    """Standard Mollweide; stable bracketed solve of 2θ+sin(2θ)=π sin(lat)."""
    lon = np.asarray(lon_rad, dtype=float)
    lat = np.asarray(lat_rad, dtype=float)
    target = np.pi * np.sin(np.clip(lat, -np.pi / 2, np.pi / 2))
    lo = np.full_like(lat, -np.pi / 2, dtype=float)
    hi = np.full_like(lat, np.pi / 2, dtype=float)
    for _ in range(n_iter):
        mid = 0.5 * (lo + hi)
        g = 2.0 * mid + np.sin(2.0 * mid)
        lo = np.where(g < target, mid, lo)
        hi = np.where(g >= target, mid, hi)
    theta = 0.5 * (lo + hi)
    theta = np.where(lat >= np.pi / 2 - 1e-15, np.pi / 2, theta)
```

```
    theta = np.where(lat <= -np.pi / 2 + 1e-15, -np.pi / 2, theta)
    x = (2.0 * np.sqrt(2.0) / np.pi) * lon * np.cos(theta)
    y = np.sqrt(2.0) * np.sin(theta)
    return x, y


def lb_to_px(ell_deg, b_deg, cx, cy, Rx, Ry):
    lon = np.deg2rad(lon_disp_from_ell(np.asarray(ell_deg, dtype=float)))
    lat = np.deg2rad(np.asarray(b_deg, dtype=float))
    x, y = mollweide_xy(lon, lat)
    Xn = x / (2.0 * np.sqrt(2.0))
    Yn = y / np.sqrt(2.0)
    return cx + Xn * Rx, cy - Yn * Ry


def as_xy(px, py):
    return float(np.asarray(px).reshape(-1)[0]), float(np.asarray(py).reshape(-1)[0])


def chamfer_distance_to_dark(dark):
    h, w = dark.shape
    INF = 1e6
    D = np.where(dark, 0.0, INF)
    for y in range(h):
        for x in range(w):
            v = D[y, x]
            if x > 0:
                v = min(v, D[y, x - 1] + 1.0)
            if y > 0:
                v = min(v, D[y - 1, x] + 1.0)
            if x > 0 and y > 0:
                v = min(v, D[y - 1, x - 1] + 1.414)
            if x + 1 < w and y > 0:
                v = min(v, D[y - 1, x + 1] + 1.414)
            D[y, x] = v
    for y in range(h - 1, -1, -1):
        for x in range(w - 1, -1, -1):
            v = D[y, x]
            if x + 1 < w:
                v = min(v, D[y, x + 1] + 1.0)
            if y + 1 < h:
                v = min(v, D[y + 1, x] + 1.0)
            if x + 1 < w and y + 1 < h:
                v = min(v, D[y + 1, x + 1] + 1.414)
            if x > 0 and y + 1 < h:
                v = min(v, D[y + 1, x - 1] + 1.414)
            D[y, x] = v
    return D


def detect_oval_bbox(rgb):
    h, w, _ = rgb.shape
    white = (rgb[:, :, 0] > 250) & (rgb[:, :, 1] > 250) & (rgb[:, :, 2] > 250)
    oval_left = np.full(h, np.nan)
```

```
    oval_right = np.full(h, np.nan)
    for y in range(h):
        row = ~white[y]
        d = np.diff(np.r_[0, row.astype(int), 0])
        starts = np.where(d == 1)[0]
        ends = np.where(d == -1)[0] - 1
        if len(starts) == 0:
            continue
        lengths = ends - starts + 1
        candidates = [
            (int(lengths[i]), int(starts[i]), int(ends[i]))
            for i in range(len(starts))
            if starts[i] < 0.72 * w and lengths[i] > 40
        ]
        if not candidates:
            continue
        candidates.sort(reverse=True)
        _, L, R = candidates[0]
        oval_left[y] = L
        oval_right[y] = R
    widths = np.where(np.isfinite(oval_left), oval_right - oval_left, 0.0)
    yeq = int(np.argmax(widths))
    thresh = 0.08 * widths[yeq]
    y0 = yeq
    while y0 > 0 and widths[y0 - 1] >= thresh:
        y0 -= 1
    y1 = yeq
    while y1 < h - 1 and widths[y1 + 1] >= thresh:
        y1 += 1
    x0 = float(oval_left[yeq])
    x1 = float(oval_right[yeq])
    return {
        "x0": int(x0),
        "x1": int(x1),
        "y0": int(y0),
        "y1": int(y1),
        "yeq": int(yeq),
        "cx": 0.5 * (x0 + x1),
        "cy": 0.5 * (y0 + y1),
        "Rx": 0.5 * (x1 - x0),
        "Ry": 0.5 * (y1 - y0 + 1),
    }


def refine_registration(rgb, seed):
    """Gray/dark Dec=0 lock (pre medial-axis). Prefer GOOD_REG hard-set in main."""
    h, w, _ = rgb.shape
    dark = rgb.mean(axis=2) < 55
    dist = chamfer_distance_to_dark(dark)

    ell_eq, b_eq = celestial_equator_lb(2500)
    lon = np.deg2rad(lon_disp_from_ell(ell_eq))
    lat = np.deg2rad(b_eq)
    mx, my = mollweide_xy(lon, lat)
```

```
    Xn = mx / (2.0 * np.sqrt(2.0))
    Yn = my / np.sqrt(2.0)

    def median_dist(cx, cy, Rx, Ry):
        px = cx + Xn * Rx
        py = cy - Yn * Ry
        m = (px >= 1) & (px < w - 1) & (py >= 1) & (py < h - 1)
        xi = np.clip(np.round(px[m]).astype(int), 0, w - 1)
        yi = np.clip(np.round(py[m]).astype(int), 0, h - 1)
        return float(np.median(dist[yi, xi]))

    base = {k: float(seed[k]) for k in ("cx", "cy", "Rx", "Ry")}
    best_sc = median_dist(**base)
    best = dict(base)
    for dcx in np.linspace(-10, 10, 21):
        for dcy in np.linspace(-10, 10, 21):
            for sRx in (0.98, 0.99, 1.0, 1.01, 1.02):
                for sRy in (0.98, 0.99, 1.0, 1.01, 1.02):
                    p = dict(
                        cx=base["cx"] + dcx,
                        cy=base["cy"] + dcy,
                        Rx=base["Rx"] * sRx,
                        Ry=base["Ry"] * sRy,
                    )
                    sc = median_dist(**p)
                    if sc < best_sc:
                        best_sc = sc
                        best = p
    b = best
    for dcx in np.linspace(-2, 2, 17):
        for dcy in np.linspace(-2, 2, 17):
            for sRx in np.linspace(0.995, 1.005, 11):
                for sRy in np.linspace(0.995, 1.005, 11):
                    p = dict(
                        cx=b["cx"] + dcx,
                        cy=b["cy"] + dcy,
                        Rx=b["Rx"] * sRx,
                        Ry=b["Ry"] * sRy,
                    )
                    sc = median_dist(**p)
                    if sc < best_sc:
                        best_sc = sc
                        best = p
    best["ceq_median_px"] = float(best_sc)
    return best


def choose_source():
    for p in SOURCE_CANDIDATES:
        if p.is_file():
            return p
    raise FileNotFoundError("No Migkas Fig.8 theta=75 source image found")
```

```
def sample_rgb(rgb, ell, b, cx, cy, Rx, Ry, r=4):
    px, py = lb_to_px(ell, b, cx, cy, Rx, Ry)
    xi, yi = int(round(as_xy(px, py)[0])), int(round(as_xy(px, py)[1]))
    patch = rgb[max(0, yi - r) : yi + r + 1, max(0, xi - r) : xi + r + 1]
    return patch.mean(axis=(0, 1))


def validate_landmarks(rgb, reg):
    cx, cy, Rx, Ry = reg["cx"], reg["cy"], reg["Rx"], reg["Ry"]
    pts = {}
    landmarks = [
        ("center", 0.0, 0.0),
        ("ell90", 90.0, 0.0),
        ("ell270", 270.0, 0.0),
        ("north", 0.0, 90.0),
        ("south", 0.0, -90.0),
    ]
    for i, (ell, b) in enumerate(MIGKAS_MAXES):
        landmarks.append((f"migkas_max{i}", ell, b))
    for i, (ell, b) in enumerate(MIGKAS_MINS):
        landmarks.append((f"migkas_min{i}", ell, b))
    for name, ell, b in landmarks:
        x, y = as_xy(*lb_to_px(ell, b, cx, cy, Rx, Ry))
        pts[name] = {"ell": ell, "b": b, "px": x, "py": y}

    assert pts["ell90"]["px"] < pts["center"]["px"], "l=90 must be LEFT of center"
    assert pts["ell270"]["px"] > pts["center"]["px"], "l=270 must be RIGHT of center"
    assert abs(pts["center"]["px"] - cx) < 1.5
    assert abs(pts["center"]["py"] - cy) < 1.5
    assert pts["north"]["py"] < pts["center"]["py"]
    assert pts["south"]["py"] > pts["center"]["py"]

    # Map-extrema pair: (17,-9) should sit in a warmer patch than (274,-22).
    cmax = sample_rgb(rgb, *MIGKAS_MAXES[0], cx, cy, Rx, Ry)
    cmin = sample_rgb(rgb, *MIGKAS_MINS[1], cx, cy, Rx, Ry)
    assert (cmax[0] + cmax[1]) > (cmin[0] + cmin[1]) + 50
    return {
        "landmarks_px": pts,
        "migkas_max0_rgb_mean": [float(x) for x in cmax],
        "migkas_min1_rgb_mean": [float(x) for x in cmin],
    }


def inside_ellipse(px, py, ecx, ecy, eRx, eRy, r_lim=1.0):
    """Boolean mask: points inside axis-aligned ellipse (image y down)."""
    px = np.asarray(px, dtype=float)
    py = np.asarray(py, dtype=float)
    Xn = (px - ecx) / eRx
    Yn = (py - ecy) / eRy
    return Xn * Xn + Yn * Yn <= float(r_lim) * float(r_lim)


def plot_lb_curve(
    ax, ell, b, cx, cy, Rx, Ry, inset=1.0, jump_px=45.0,
```

```
    clip_r2=None, clip_path=None, color_oval=None, **kw
):
    """Draw curve in oval; inset<1 keeps strokes inside the published Mollweide edge."""
    px, py = lb_to_px(ell, b, cx, cy, Rx, Ry)
    px = np.asarray(px, dtype=float).reshape(-1)
    py = np.asarray(py, dtype=float).reshape(-1)
    # Pull points slightly toward oval center so thick strokes do not spill past the rim.
    px = cx + (px - cx) * inset
    py = cy + (py - cy) * inset
    Xn = (px - cx) / Rx
    Yn = (cy - py) / Ry
    r2_lim = (inset * inset) if clip_r2 is None else float(clip_r2)
    inside = Xn * Xn + Yn * Yn <= r2_lim
    # Also require published colored oval (seed) so registration oversize cannot bleed.
    if color_oval is not None:
        ocx, ocy, oRx, oRy, o_r = color_oval
        inside = inside & inside_ellipse(px, py, ocx, ocy, oRx, oRy, o_r)
    px = np.where(inside, px, np.nan)
    py = np.where(inside, py, np.nan)

    segs = []
    cur_x, cur_y = [], []
    for x, y in zip(px, py):
        if not (np.isfinite(x) and np.isfinite(y)):
            if len(cur_x) >= 2:
                segs.append((cur_x, cur_y))
            cur_x, cur_y = [], []
            continue
        if cur_x and np.hypot(x - cur_x[-1], y - cur_y[-1]) > jump_px:
            if len(cur_x) >= 2:
                segs.append((cur_x, cur_y))
            cur_x, cur_y = [float(x)], [float(y)]
        else:
            cur_x.append(float(x))
            cur_y.append(float(y))
    if len(cur_x) >= 2:
        segs.append((cur_x, cur_y))
    if clip_path is not None:
        kw["clip_path"] = clip_path
        kw["clip_on"] = True
    for sx, sy in segs:
        (line,) = ax.plot(sx, sy, **kw)
        if clip_path is not None:
            line.set_clip_path(clip_path)
            line.set_clip_on(True)


def plot_closed_ring(
    ax, mu, cx, cy, Rx, Ry, color, lw, inset=1.0, jump_px=80.0,
    clip_path=None, color_oval=None, ls="-", wash_alpha=None,
):
    """
    Exact projected small-circle about D_HAT=+x: n·x̂ = μ.
```

```
    Parameterize on the sphere, convert to Galactic (ℓ,b), apply
    lon_disp = -wrap(ℓ), Mollweide-project, and stroke continuous polylines.
    Split wherever display-longitude jumps across the antimeridian (or pixel
    distance jumps). Never invent interior chords, ellipse fits, or decorative
    oval-rim bridges.

    Notes on θ labels used elsewhere:
      μ=cos(30°)  → r=30° about +x (closed oval near map center)
      μ=cos(185°)=cos(175°) → r=175° about +x ≡ 5° about CoE
        → small arcs on BOTH left and right map edges (not a rim loop)
      μ=cos(115°) → r=115° about +x (large seam-crossing curve)
      μ=cos(265°)=cos(95°) → r=95° about +x (near-meridian, seam-crossing)
    """
    pts = small_circle_about_axis(D_HAT, mu, n=4001)
    # Keep sampling order around the axis (azimuth) — stable for splitting.
    phi = np.arctan2(pts[:, 2], pts[:, 1])
    order = np.argsort(phi)
    pts = pts[order]

    ell = np.empty(len(pts))
    b = np.empty(len(pts))
    for i, p in enumerate(pts):
        ell[i], b[i] = n_to_lb(p)

    lon = np.deg2rad(lon_disp_from_ell(ell))  # λ ∈ (-π, π]
    lat = np.deg2rad(b)

    # Split on antimeridian jumps in display longitude BEFORE pixel projection.
    dlon = np.diff(lon)
    cuts = np.where(np.abs(dlon) > np.pi)[0] + 1
    # Also close the periodic sample: check wrap between last and first.
    # Build open segments from azimuth-ordered points.
    idx = np.concatenate([[0], cuts, [len(lon)]])
    lon_segs = []
    for i0, i1 in zip(idx[:-1], idx[1:]):
        if i1 - i0 >= 2:
            lon_segs.append((lon[i0:i1].copy(), lat[i0:i1].copy()))

    # Circle sampling is cyclic. Merge the artificial last→first break when
    # that periodic join is not itself an antimeridian crossing. Otherwise a
    # seam-crossing circle falsely appears as three pieces with a tiny hole.
    if len(lon_segs) > 1 and abs(lon[0] - lon[-1]) <= np.pi:
        first_lo, first_la = lon_segs[0]
        last_lo, last_la = lon_segs[-1]
        lon_segs = [(
            np.concatenate([last_lo, first_lo]),
            np.concatenate([last_la, first_la]),
        )] + lon_segs[1:-1]

    # Only now pin TRUE seam endpoints to ±π. Never snap the arbitrary
    # parameterization start/end to the boundary.
    snapped = []
    for lo, la in lon_segs:
        if abs(abs(lo[0]) - np.pi) < np.deg2rad(10):
```

```
            lo[0] = np.sign(lo[0] if lo[0] != 0 else 1.0) * np.pi
        if abs(abs(lo[-1]) - np.pi) < np.deg2rad(10):
            lo[-1] = np.sign(lo[-1] if lo[-1] != 0 else 1.0) * np.pi
        snapped.append((lo, la))
    lon_segs = snapped

    def stroke_xy(xs, ys):
        if len(xs) < 2:
            return
        if wash_alpha is not None:
            # Soft μ-wash stroke: no white underlay; translucent color only.
            layers = ((color, lw, 2, float(wash_alpha)),)
        else:
            # Clean mid-stroke: optional thin white underlay only (lw+1.5), then color.
            # No black outer rim, no thick white halo stack.
            layers = (
                ("white", lw + 1.5, 3, 0.95),
                (color, lw, 4, 1.0),
            )
        for col, width, z, a in layers:
            (line,) = ax.plot(
                xs, ys, color=col, lw=width, linestyle=ls,
                solid_capstyle="round", solid_joinstyle="round",
                zorder=z, alpha=a,
                clip_path=clip_path, clip_on=True,
            )
            if clip_path is not None:
                line.set_clip_path(clip_path)
                line.set_clip_on(True)

    r2_lim = inset * inset + 1e-10
    drawn = 0
    for lo, la in lon_segs:
        # Mollweide in normalized oval coords, then pixels.
        x_m, y_m = mollweide_xy(lo, la)
        Xn = x_m / (2.0 * np.sqrt(2.0))
        Yn = y_m / np.sqrt(2.0)
        px = cx + Xn * Rx
        py = cy - Yn * Ry
        # Mild inset so thick strokes stay inside published oval (no rim-snap).
        px = cx + (px - cx) * inset
        py = cy + (py - cy) * inset
        Xn2 = (px - cx) / Rx
        Yn2 = (cy - py) / Ry
        inside = (Xn2 * Xn2 + Yn2 * Yn2) <= r2_lim
        if color_oval is not None:
            ocx, ocy, oRx, oRy, o_r = color_oval
            inside = inside & inside_ellipse(px, py, ocx, ocy, oRx, oRy, o_r)

        # Pixel-space split on NaN / large jumps / outside oval.
        cur_x, cur_y = [], []
        for x, y, ok in zip(px, py, inside):
            if (not ok) or (not np.isfinite(x)) or (not np.isfinite(y)):
                if len(cur_x) >= 2:
```

```
                    stroke_xy(cur_x, cur_y)
                    drawn += 1
                cur_x, cur_y = [], []
                continue
            if cur_x and np.hypot(x - cur_x[-1], y - cur_y[-1]) > jump_px:
                if len(cur_x) >= 2:
                    stroke_xy(cur_x, cur_y)
                    drawn += 1
                cur_x, cur_y = [float(x)], [float(y)]
            else:
                cur_x.append(float(x))
                cur_y.append(float(y))
        if len(cur_x) >= 2:
            # Close only if this single segment is already nearly a loop
            # (non-wrapping ring such as MAX 30°).
            if (
                drawn == 0
                and len(lon_segs) == 1
                and np.hypot(cur_x[0] - cur_x[-1], cur_y[0] - cur_y[-1]) < jump_px
            ):
                cur_x.append(cur_x[0])
                cur_y.append(cur_y[0])
            stroke_xy(cur_x, cur_y)
            drawn += 1



def _ring_px_arrays(mu, cx, cy, Rx, Ry, inset, n_pts=721):
    """Small-circle n·D=μ → pixel (px, py) arrays, azimuth-ordered, with inset."""
    pts = small_circle_about_axis(D_HAT, float(mu), n=n_pts)
    phi = np.arctan2(pts[:, 2], pts[:, 1])
    order = np.argsort(phi)
    pts = pts[order]
    ell = np.empty(n_pts)
    b = np.empty(n_pts)
    for i, p in enumerate(pts):
        ell[i], b[i] = n_to_lb(p)
    lon = np.deg2rad(lon_disp_from_ell(ell))
    lat = np.deg2rad(b)
    x_m, y_m = mollweide_xy(lon, lat)
    Xn = x_m / (2.0 * np.sqrt(2.0))
    Yn = y_m / np.sqrt(2.0)
    px = cx + Xn * Rx
    py = cy - Yn * Ry
    px = cx + (px - cx) * inset
    py = cy + (py - cy) * inset
    return lon, px, py


def fill_highlight_band(
    ax, theta_lo, theta_hi, theta_mid, cx, cy, Rx, Ry, color,
    inset=1.0, clip_path=None, color_oval=None, n_pts=721, jump_px=80.0,
):
    """
```

```
    One highlight band matching three-panel Panel B:
      continuous transparent polygon strips between ±BAND_HALF edges
      (broken only at Mollweide antimeridian / large pixel jumps — NOT a dense
      multi-stroke wash) + ONE solid mid-stroke at θ_mid.
    """
    mu_a, mu_b = mu_pair_for_band(theta_lo, theta_hi)
    mu_mid = float(np.cos(np.deg2rad(theta_mid)))
    lon_a, px_a, py_a = _ring_px_arrays(mu_a, cx, cy, Rx, Ry, inset, n_pts=n_pts)
    lon_b, px_b, py_b = _ring_px_arrays(mu_b, cx, cy, Rx, Ry, inset, n_pts=n_pts)

    r2_lim = inset * inset + 1e-10

    def _inside(px, py):
        Xn2 = (px - cx) / Rx
        Yn2 = (cy - py) / Ry
        ok = (Xn2 * Xn2 + Yn2 * Yn2) <= r2_lim
        if color_oval is not None:
            ocx, ocy, oRx, oRy, o_r = color_oval
            ok = ok & inside_ellipse(px, py, ocx, ocy, oRx, oRy, o_r)
        return ok

    gap = 0.9 * np.pi
    bad = np.zeros(n_pts - 1, dtype=bool)
    for i in range(n_pts - 1):
        if abs(lon_a[i + 1] - lon_a[i]) > gap:
            bad[i] = True
        elif abs(lon_b[i + 1] - lon_b[i]) > gap:
            bad[i] = True
        elif abs(lon_a[i] - lon_b[i]) > gap:
            bad[i] = True
        elif abs(lon_a[i + 1] - lon_b[i + 1]) > gap:
            bad[i] = True
        elif np.hypot(px_a[i + 1] - px_a[i], py_a[i + 1] - py_a[i]) > jump_px:
            bad[i] = True
        elif np.hypot(px_b[i + 1] - px_b[i], py_b[i + 1] - py_b[i]) > jump_px:
            bad[i] = True
        elif (not _inside(px_a[i], py_a[i])) or (not _inside(px_b[i], py_b[i])):
            bad[i] = True
        elif (not _inside(px_a[i + 1], py_a[i + 1])) or (not _inside(px_b[i + 1], py_b[i + 1])):
            bad[i] = True

    polys = []
    i = 0
    while i < n_pts - 1:
        while i < n_pts - 1 and bad[i]:
            i += 1
        if i >= n_pts - 1:
            break
        j = i
        while j < n_pts - 1 and not bad[j]:
            j += 1
        i0, i1 = i, j
        if i1 - i0 >= 1:
            xs = np.concatenate([px_a[i0:i1 + 1], px_b[i0:i1 + 1][::-1]])
```

```
            ys = np.concatenate([py_a[i0:i1 + 1], py_b[i0:i1 + 1][::-1]])
            polys.append(np.column_stack([xs, ys]))
        i = j + 1

    if polys:
        # Soft transparent highlight (clean pre-halo look).
        coll = PolyCollection(
            polys, facecolors=color, edgecolors="none",
            alpha=0.30, zorder=2, linewidths=0,
        )
        ax.add_collection(coll)
        if clip_path is not None:
            coll.set_clip_path(clip_path)
            coll.set_clip_on(True)

    # ONE solid mid-stroke only (thin white underlay lw+1.5). No boundary-edge
    # strokes — dual edges with halos multiply visual lines into "extra loops".
    plot_closed_ring(
        ax, mu_mid, cx, cy, Rx, Ry, color, lw=2.8,
        inset=inset, clip_path=clip_path, color_oval=color_oval,
    )



# --- Opacity gradient (same rule as make_coe_rings_migkas_pin_axis_matched_grad*.py):
# alpha *= (1+cos psi)/2, psi = ring azimuth from the θ-ray point (ring point most
# along sign(sin θ)·Ŷ, Ŷ = Galactic +z projected ⊥ D), rolled by ROLL (Y' = cY − sZ).
from matplotlib.collections import LineCollection
from matplotlib.colors import to_rgba


def _ring_basis_roll(D, roll_deg):
    a = D / np.linalg.norm(D)
    tmp = np.array([0.0, 0.0, 1.0])
    if abs(np.dot(a, tmp)) > 0.9:
        tmp = np.array([0.0, 1.0, 0.0])
    y = tmp - np.dot(tmp, a) * a
    y /= np.linalg.norm(y)
    z = np.cross(a, y)
    z /= np.linalg.norm(z)
    r = np.deg2rad(float(roll_deg))
    c, s_ = np.cos(r), np.sin(r)
    return c * y - s_ * z, s_ * y + c * z


def _grad_ring_px(mu, theta_mid, cx, cy, Rx, Ry, inset, n):
    a = D_HAT / np.linalg.norm(D_HAT)
    mu = float(np.clip(mu, -1.0 + 1e-9, 1.0 - 1e-9))
    rho = np.sqrt(1.0 - mu * mu)
    y, z = _ring_basis_roll(a, ROLL)
    phi0 = 0.5 * np.pi if np.sin(np.deg2rad(theta_mid)) >= 0 else 1.5 * np.pi
    phi = phi0 + np.linspace(-np.pi, np.pi, n)
    pts = (mu * a)[None, :] + rho * (np.outer(np.sin(phi), y) + np.outer(np.cos(phi), z))
    ell = np.array([n_to_lb(p)[0] for p in pts]); b = np.array([n_to_lb(p)[1] for p in pts])
```

```
    lon = np.deg2rad(lon_disp_from_ell(ell)); lat = np.deg2rad(b)
    x_m, y_m = mollweide_xy(lon, lat)
    px = cx + x_m / (2.0 * np.sqrt(2.0)) * Rx
    py = cy - y_m / np.sqrt(2.0) * Ry
    px = cx + (px - cx) * inset; py = cy + (py - cy) * inset
    return lon, px, py, 0.5 * (1.0 + np.cos(phi - phi0))


def _rgba(color, alphas):
    r, g, b_, _ = to_rgba(color)
    al = np.clip(np.asarray(alphas, dtype=float), 0.0, 1.0)
    out = np.empty((al.size, 4)); out[:, 0], out[:, 1], out[:, 2], out[:, 3] = r, g, b_, al
    return out


def _runs(seg_sc, valid, n_levels=64):
    lev = np.round(np.clip(seg_sc, 0.0, 1.0) * n_levels) / n_levels
    ok = (lev > 0) & valid
    runs, i, n = [], 0, lev.size
    while i < n:
        if not ok[i]:
            i += 1; continue
        j = i + 1
        while j < n and ok[j] and lev[j] == lev[i]:
            j += 1
        runs.append((i, j, float(lev[i]))); i = j
    return runs


def fill_highlight_band_grad(
    ax, theta_lo, theta_hi, theta_mid, cx, cy, Rx, Ry, color,
    inset=1.0, clip_path=None, color_oval=None, n_pts=721, jump_px=80.0,
):
    """Same band (alpha 0.30) + mid-stroke (white lw+1.5 @0.95 under colour lw 2.8)
    as fill_highlight_band, but with the (1+cos psi)/2 opacity gradient."""
    mu_a, mu_b = mu_pair_for_band(theta_lo, theta_hi)
    mu_mid = float(np.cos(np.deg2rad(theta_mid)))
    r2_lim = inset * inset + 1e-10

    def _inside(px, py):
        ok = ((px - cx) / Rx) ** 2 + ((cy - py) / Ry) ** 2 <= r2_lim
        if color_oval is not None:
            ocx, ocy, oRx, oRy, o_r = color_oval
            ok = ok & inside_ellipse(px, py, ocx, ocy, oRx, oRy, o_r)
        return ok

    def _clip(coll):
        if clip_path is not None:
            coll.set_clip_path(clip_path); coll.set_clip_on(True)
        ax.add_collection(coll)

    gap = 0.9 * np.pi
    lon_a, pxa, pya, sc = _grad_ring_px(mu_a, theta_mid, cx, cy, Rx, Ry, inset, n_pts)
    lon_b, pxb, pyb, _ = _grad_ring_px(mu_b, theta_mid, cx, cy, Rx, Ry, inset, n_pts)
```

```
    qa = np.stack([lon_a[:-1], lon_a[1:], lon_b[1:], lon_b[:-1]])
    ins_a, ins_b = _inside(pxa, pya), _inside(pxb, pyb)
    valid = ((qa.max(0) - qa.min(0)) <= gap)
    valid &= np.hypot(np.diff(pxa), np.diff(pya)) <= jump_px
    valid &= np.hypot(np.diff(pxb), np.diff(pyb)) <= jump_px
    valid &= ins_a[:-1] & ins_a[1:] & ins_b[:-1] & ins_b[1:]
    runs = _runs(0.5 * (sc[:-1] + sc[1:]), valid)
    if runs:
        polys = [np.column_stack([np.concatenate([pxa[i0:i1 + 1], pxb[i0:i1 + 1][::-1]]),
                                  np.concatenate([pya[i0:i1 + 1], pyb[i0:i1 + 1][::-1]])])
                 for i0, i1, _ in runs]
        _clip(PolyCollection(polys, facecolors=_rgba(color, [0.30 * r[2] for r in runs]),
                             edgecolors="none", linewidths=0, zorder=2))
    lon_m, pxm, pym, scm = _grad_ring_px(mu_mid, theta_mid, cx, cy, Rx, Ry, inset, 4001)
    ins = _inside(pxm, pym)
    valid = (np.abs(np.diff(lon_m)) <= np.pi) & (np.hypot(np.diff(pxm), np.diff(pym)) <= jump_px)
    valid &= ins[:-1] & ins[1:]
    runs = _runs(0.5 * (scm[:-1] + scm[1:]), valid)
    if runs:
        segs = [np.column_stack([pxm[i0:i1 + 1], pym[i0:i1 + 1]]) for i0, i1, _ in runs]
        _clip(LineCollection(segs, colors=_rgba("white", [0.95 * r[2] for r in runs]),
                             linewidths=2.8 + 1.5, zorder=3, capstyle="butt", joinstyle="round"))
        _clip(LineCollection(segs, colors=_rgba(color, [1.0 * r[2] for r in runs]),
                             linewidths=2.8, zorder=4, capstyle="butt", joinstyle="round"))


def draw_overlay(rgb, reg, seed, source):

    h, w, _ = rgb.shape
    cx, cy, Rx, Ry = reg["cx"], reg["cy"], reg["Rx"], reg["Ry"]
    # Published colored Mollweide (seed detect) — used to keep overlays inside
    # the Migkas map even when registration ellipse is slightly larger.
    scx = float(seed["cx"])
    scy = float(seed["cy"])
    sRx = float(seed["Rx"])
    sRy = float(seed["Ry"])

    dpi = 120
    fig, ax = plt.subplots(figsize=(w / dpi, (h / dpi) + 1.35), dpi=dpi)
    ax.imshow(rgb, origin="upper", extent=(0, w, h, 0), interpolation="nearest")
    ax.set_xlim(0, w)
    ax.set_ylim(h, 0)
    ax.set_aspect("equal")
    ax.axis("off")

    # clip_path = registered oval (cx,cy,Rx,Ry). Registration itself is unchanged.
    # Because the refined ellipse can sit a bit outside the published colored
    # map, we ALSO inset strokes and cull/inset markers against the seed oval.
    clip_oval = Ellipse(
        (cx, cy), width=2.0 * Rx, height=2.0 * Ry,
        facecolor="none", edgecolor="none", transform=ax.transData,
    )
    ax.add_patch(clip_oval)
```

```
    # Stroke inset so thick white+color underlays do not AA-bleed past the rim.
    STROKE_INSET = 0.97
    # Cull points outside this fraction of the published colored seed oval.
    COLOR_R = 0.978
    color_oval = (scx, scy, sRx, sRy, COLOR_R)

    def stroke_curve(ell, b, color, lw, ls="-"):
        # White underlay then color — stands out on magma without looking oversized.
        plot_lb_curve(
            ax, ell, b, cx, cy, Rx, Ry, inset=STROKE_INSET, clip_path=clip_oval,
            color_oval=color_oval,
            color="white", lw=lw + 1.6, ls=ls, alpha=0.95,
            solid_capstyle="round", zorder=3,
        )
        plot_lb_curve(
            ax, ell, b, cx, cy, Rx, Ry, inset=STROKE_INSET, clip_path=clip_oval,
            color_oval=color_oval,
            color=color, lw=lw, ls=ls, alpha=1.0,
            solid_capstyle="round", zorder=4,
        )

    def mark_point(ell, b, marker, color, ms):
        """Place marker; pull limb symbols inward so halo stays in colored oval."""
        px, py = as_xy(*lb_to_px(ell, b, cx, cy, Rx, Ry))
        # Approx marker+halo radius in pixels (ms is points).
        halo_ms = ms + 1.8
        ms_px = 0.5 * halo_ms * (dpi / 72.0)
        # Normalized margin inside the published colored oval.
        margin = ms_px / min(sRx, sRy) + 0.012
        r_lim = max(0.55, COLOR_R - margin)

        Xn = (px - scx) / sRx
        Yn = (py - scy) / sRy
        r_col = float(np.hypot(Xn, Yn))

        if r_col > 1e-9:
            # Scale toward colored-oval center so the whole symbol fits.
            if r_col > r_lim:
                scale = r_lim / r_col
                px = scx + (px - scx) * scale
                py = scy + (py - scy) * scale
                Xn = (px - scx) / sRx
                Yn = (py - scy) / sRy
                r_col = float(np.hypot(Xn, Yn))
            # Mild extra inset for everyone (matches stroke inset spirit).
            mild = min(STROKE_INSET, 0.995)
            px = scx + (px - scx) * mild
            py = scy + (py - scy) * mild
            Xn = (px - scx) / sRx
            Yn = (py - scy) / sRy
            r_col = float(np.hypot(Xn, Yn))

        # Skip if still too close to the rim for this marker size.
        if r_col + margin > COLOR_R + 1e-6:
```

```
        return

    def _mark(m_size, m_color, m_edge, m_ew, z):
        (line,) = ax.plot(
            px, py, marker=marker, ms=m_size, color=m_color,
            markeredgecolor=m_edge, markeredgewidth=m_ew, linestyle="None",
            zorder=z, clip_path=clip_oval, clip_on=True,
        )
        line.set_clip_path(clip_oval)
        line.set_clip_on(True)

    _mark(ms + 1.8, "white", "white", 0, 6)
    _mark(ms, color, "k", 0.55, 7)

# CoE axis at geometric inset=1.0. Do NOT overlay Dec=0 — Migkas already
# printed the celestial equator on Fig. 8; a second green dashed stroke fights it.
def stroke_curve_geom(ell, b, color, lw, ls="-"):
    plot_lb_curve(
        ax, ell, b, cx, cy, Rx, Ry, inset=1.0, clip_path=clip_oval,
        color_oval=color_oval,
        color="white", lw=lw + 1.6, ls=ls, alpha=0.95,
        solid_capstyle="round", zorder=3,
    )
    plot_lb_curve(
        ax, ell, b, cx, cy, Rx, Ry, inset=1.0, clip_path=clip_oval,
        color_oval=color_oval,
        color=color, lw=lw, ls=ls, alpha=1.0,
        solid_capstyle="round", zorder=4,
    )

# CoE↔anti-CoE axis = great circle through ±D (tilted +30°; not Galactic equator)
_ax_ell, _ax_b = great_circle_through_poles(D_HAT, n=721)
stroke_curve_geom(_ax_ell, _ax_b, C_AXIS, lw=1.15)

# Exactly four highlight bands (match three-panel Panel B): each ±9°
# (= 18° end-to-end = 5% of 360°) + one solid mid-stroke. No companion.
for (lo, hi), mid, color in zip(MAX_BANDS, THETA_MAX, COLORS_MAX):
    (fill_highlight_band_grad if GRAD else fill_highlight_band)(
        ax, lo, hi, mid, cx, cy, Rx, Ry, color,
        inset=STROKE_INSET, clip_path=clip_oval, color_oval=color_oval,
    )
for (lo, hi), mid, color in zip(MIN_BANDS, THETA_MIN, COLORS_MIN):
    (fill_highlight_band_grad if GRAD else fill_highlight_band)(
        ax, lo, hi, mid, cx, cy, Rx, Ry, color,
        inset=STROKE_INSET, clip_path=clip_oval, color_oval=color_oval,
    )

# Markers (limb ones get stronger inset inside mark_point); per-loop colors
mark_point(ANTI_COE_LB[0], ANTI_COE_LB[1], "o", C_ANTI, ms=5.5)
mark_point(COE_LB[0], COE_LB[1], "*", C_COE, ms=7.5)
for (ell, b), color in zip(MIGKAS_MAXES, COLORS_MAX):
    mark_point(ell, b, "^", color, ms=6.5)
for (ell, b), color in zip(MIGKAS_MINS, COLORS_MIN):
    mark_point(ell, b, "s", color, ms=6.0)
```

```
  handles = [
      Patch(
          facecolor=C_MAX_30, alpha=0.55, edgecolor=C_MAX_30,
          label=(
              rf"MAX $\theta\!=\!{THETA_MAX[0]:g}^\circ$ band "
              rf"[{MAX_BANDS[0][0]:g},{MAX_BANDS[0][1]:g}]$^\circ$ "
              rf"($18^\circ$=$\pm9^\circ$=5% of $360^\circ$)"
          ),
      ),
      Patch(
          facecolor=C_MAX_185, alpha=0.55, edgecolor=C_MAX_185,
          label=(
              rf"MAX $\theta\!=\!{THETA_MAX[1]:g}^\circ$ band "
              rf"[{MAX_BANDS[1][0]:g},{MAX_BANDS[1][1]:g}]$^\circ$ "
              rf"($18^\circ$=$\pm9^\circ$)"
          ),
      ),
      Patch(
          facecolor=C_MIN_115, alpha=0.55, edgecolor=C_MIN_115,
          label=(
              rf"MIN $\theta\!=\!{THETA_MIN[0]:g}^\circ$ band "
              rf"[{MIN_BANDS[0][0]:g},{MIN_BANDS[0][1]:g}]$^\circ$ "
              rf"($18^\circ$=$\pm9^\circ$)"
          ),
      ),
      Patch(
          facecolor=C_MIN_265, alpha=0.55, edgecolor=C_MIN_265,
          label=(
              rf"MIN $\theta\!=\!{THETA_MIN[1]:g}^\circ$ band "
              rf"[{MIN_BANDS[1][0]:g},{MIN_BANDS[1][1]:g}]$^\circ$ "
              rf"($18^\circ$=$\pm9^\circ$)"
          ),
      ),
      Line2D([0], [0], color=C_AXIS, lw=2.0, label=r"CoE axis tilted $+15^\circ$"),
      Line2D(
          [0], [0], marker="^", color="w", markerfacecolor=C_MAX_30,
          markeredgecolor="k", linestyle="None", ms=8,
          label=rf"MAX $({MIGKAS_MAXES[0][0]:.0f}^\circ,{MIGKAS_MAXES[0][1]:+.0f}^\circ)$",
      ),
      Line2D(
          [0], [0], marker="^", color="w", markerfacecolor=C_MAX_185,
          markeredgecolor="k", linestyle="None", ms=8,
          label=rf"MAX $({MIGKAS_MAXES[1][0]:.0f}^\circ,{MIGKAS_MAXES[1][1]:+.0f}^\circ)$",
      ),
      Line2D(
          [0], [0], marker="s", color="w", markerfacecolor=C_MIN_115,
          markeredgecolor="k", linestyle="None", ms=7,
          label=rf"MIN $({MIGKAS_MINS[0][0]:.0f}^\circ,{MIGKAS_MINS[0][1]:+.0f}^\circ)$",
      ),
      Line2D(
          [0], [0], marker="s", color="w", markerfacecolor=C_MIN_265,
          markeredgecolor="k", linestyle="None", ms=7,
          label=rf"MIN $({MIGKAS_MINS[1][0]:.0f}^\circ,{MIGKAS_MINS[1][1]:+.0f}^\circ)$",
```

```
        ),
        Line2D([0], [0], marker="o", color=C_ANTI, markeredgecolor="k",
               linestyle="None", ms=6,
               label=r"anti-CoE $+x$ $(0^\circ,+15^\circ)$"),
        Line2D([0], [0], marker="*", color=C_COE, markeredgecolor="k",
               linestyle="None", ms=9,
               label=r"CoE $(180^\circ,-15^\circ)$"),
    ]
    leg = ax.legend(
        handles=handles, loc="upper center", bbox_to_anchor=(0.5, -0.012),
        ncol=4, fontsize=7.5, frameon=True, fancybox=False,
        edgecolor="#222222", framealpha=1.0, facecolor="white",
        labelcolor="#111111", borderpad=0.6, handlelength=2.0,
    )
    leg.get_frame().set_linewidth(1.2)
    fig.suptitle(
        r"Migkas Fig. 8 $\theta=75^\circ$ $A/A_{\rm all}$"
        r" + four CoE highlight bands, each $18^\circ$ end-to-end ($\pm9^\circ$ = 5% of $360^\circ$)"
        "\n"
        r"(two MAX + two MIN; cyan/green/magenta/blue; anti-CoE at $(0^\circ,+15^\circ)$; "
        r"CoE axis tilted $+15^\circ$; $\ell$ increases LEFT"
        + (r"; gradient: darkest at $\theta$ ray, transparent opposite" if GRAD else "")
        + (rf"; ${ROLL:+g}^\circ$ CCW about CoE axis" if abs(ROLL) > 1e-9 else "")
        + ")",
        fontsize=10.5, y=0.985,
    )
    fig.subplots_adjust(left=0.01, right=0.99, top=0.92, bottom=0.13)
    fig.savefig(PNG_PATH, dpi=dpi, facecolor="white")
    plt.close(fig)
    print(f"wrote {PNG_PATH}")


def main():
    source = choose_source()
    print(f"source: {source}")
    rgb = np.asarray(Image.open(source).convert("RGB"))
    seed = detect_oval_bbox(rgb)
    print("oval seed:", {k: seed[k] for k in ("cx", "cy", "Rx", "Ry", "x0", "x1", "y0", "y1")})
    # Restore previous visually-good oval (Chris preferred). No medial-axis Rx stretch.
    reg = dict(GOOD_REG)
    h, w, _ = rgb.shape
    dark = rgb.mean(axis=2) < 55
    dist = chamfer_distance_to_dark(dark)
    ell_eq, b_eq = celestial_equator_lb(2500)
    lon = np.deg2rad(lon_disp_from_ell(ell_eq))
    lat = np.deg2rad(b_eq)
    mx, my = mollweide_xy(lon, lat)
    Xn = mx / (2.0 * np.sqrt(2.0))
    Yn = my / np.sqrt(2.0)
    px = reg["cx"] + Xn * reg["Rx"]
    py = reg["cy"] - Yn * reg["Ry"]
    m = (px >= 1) & (px < w - 1) & (py >= 1) & (py < h - 1)
    xi = np.clip(np.round(px[m]).astype(int), 0, w - 1)
    yi = np.clip(np.round(py[m]).astype(int), 0, h - 1)
```

```
    reg["ceq_median_px"] = float(np.median(dist[yi, xi]))
    reg["registration_source"] = "hard-set GOOD_REG (pre medial-axis retune)"
    print("restored reg:", {k: reg[k] for k in ("cx", "cy", "Rx", "Ry", "ceq_median_px")})
    checks = validate_landmarks(rgb, reg)
    print(
        "validation OK — max0 RGB", checks["migkas_max0_rgb_mean"],
        "min1 RGB", checks["migkas_min1_rgb_mean"],
    )
    global PNG_PATH, JSON_PATH, GRAD, ROLL
    for name, g, r in VERSIONS:
        PNG_PATH = SM_DIR / f"fig8_overlay_{name}.png"
        JSON_PATH = SM_DIR / f"fig8_overlay_{name}.json"
        GRAD, ROLL = g, r
        draw_overlay(rgb, reg, seed, source)
        img = Image.open(PNG_PATH).convert("RGB")
        img.resize((1700, round(img.height * 1700 / img.width)), Image.LANCZOS).save(
            SM_DIR / f"fig8_overlay_{name}_web.jpg", quality=88)

    meta = {
        "png": str(PNG_PATH),
        "script": str(Path(__file__).resolve()),
        "source_image": str(source),
        "convention": (
            "Galactic Mollweide; lon_disp=-wrap(ell); ell increases LEFT "
            "(Migkas Fig.8)"
        ),
        "axis_tilt_b_deg": AXIS_TILT_B_DEG,
        "anti_CoE_lb_deg": [float(ANTI_COE_LB[0]), float(ANTI_COE_LB[1])],
        "CoE_lb_deg": [float(COE_LB[0]), float(COE_LB[1])],
        "D_hat_galactic_xyz": D_HAT.tolist(),
        "axis_tilt_reason": (
            "Chris 2026-09-22 — markers fixed, rings rotate up 30°"
        ),
        "migkas_max_lb_deg": [list(p) for p in MIGKAS_MAXES],
        "migkas_min_lb_deg": [list(p) for p in MIGKAS_MINS],
        "theta_max_from_plus_x_deg": list(THETA_MAX),
        "theta_min_from_plus_x_deg": list(THETA_MIN),
        "band_half_width_deg": BAND_HALF,
        "band_full_width_deg": BAND_FULL,
        "band_width_note": "5% of 360° = 18° end-to-end = ±9° about each MAX/MIN center",
        "max_bands_deg": [list(b) for b in MAX_BANDS],
        "min_bands_deg": [list(b) for b in MIN_BANDS],
        "band_intervals": {
            "MAX_30": [float(MAX_BANDS[0][0]), float(MAX_BANDS[0][1])],
            "MAX_185": [float(MAX_BANDS[1][0]), float(MAX_BANDS[1][1])],
            "MIN_115": [float(MIN_BANDS[0][0]), float(MIN_BANDS[0][1])],
            "MIN_265": [float(MIN_BANDS[1][0]), float(MIN_BANDS[1][1])],
        },
        "colors": {
            "MAX_30": C_MAX_30,
            "MAX_185": C_MAX_185,
            "MIN_115": C_MIN_115,
            "MIN_265": C_MIN_265,
            "palette": "magma-popping cyan/magenta/lime/sky (no reds/oranges)",
```

```
        },
        "mu_max_bands": [list(mu_pair_for_band(lo, hi)) for lo, hi in MAX_BANDS],
        "mu_min_bands": [list(mu_pair_for_band(lo, hi)) for lo, hi in MIN_BANDS],
        "scan_stop_source": (
            "best_fit_2ell_multiplier.npz A_after extrema (two max, two min); "
            f"each drawn as one transparent {BAND_FULL:.0f}° end-to-end highlight "
            f"(±{BAND_HALF:.0f}° = 5% of 360°) + one solid mid-stroke; "
            "matches three-panel Panel B; no companion"
        ),
        "matches_three_panel": str(OUT_DIR / "make_coe_rings_migkas_pin_axis.py"),
        "oval_seed_bbox": {
            "x0": seed["x0"], "x1": seed["x1"], "y0": seed["y0"], "y1": seed["y1"],
            "cx": seed["cx"], "cy": seed["cy"], "Rx": seed["Rx"], "Ry": seed["Ry"],
        },
        "registration": {
            "cx": reg["cx"], "cy": reg["cy"], "Rx": reg["Rx"], "Ry": reg["Ry"],
            "ceq_median_px": reg["ceq_median_px"],
            "registration_source": reg.get("registration_source"),
            "ceq_residual_before_registration": dict(GOOD_REG),
            "rejected_medial_axis_reg": {
                "cx": 865.6, "cy": 766.4, "Rx": 706.19, "Ry": 393.70,
                "note": "medial-axis / achromatic DT-ridge retune; blew up Rx; reverted",
            },
            "dofs": ["cx", "cy", "Rx", "Ry"],
            "drawing": {
                "dec0_drawn": False,
                "dec0_note": "not drawn; use Migkas published celestial equator",
                "coe_axis_inset": 1.0,
                "rings_inset": 0.97,
                "note": "CoE axis geometric inset=1; rings mild inset; no Dec=0 overlay",
            },
            "method": (
                "oval seed detect + hard-set to pre-retune visually-good (cx,cy,Rx,Ry); "
                "medial-axis Rx stretch rejected; Dec=0 NOT overlaid (published CEQ kept); "
                "CoE-axis inset=1.0; rings inset~0.97"
            ),
        },
        "landmark_checks": checks,
        "drawing": (
            "Exactly 4 features. Each = ONE smooth transparent PolyCollection "
            "band (θ±9°, 18° end-to-end) + ONE solid mid-stroke at center θ. "
            "Fill alpha≈0.30. Mid-stroke: thin white underlay (lw+1.5) then color "
            "(lw=2.8). NO black outer rim; NO thick white halo stacks; NO dual "
            "boundary-edge strokes. Colors: cyan/magenta/lime/sky for magma. "
            "No 283° companion. Dec=0 NOT overlaid."
        ),
        "legend_summary": (
            "four highlight bands, each 18° end-to-end (±9° = 5% of 360%); "
            "MAX cyan+magenta, MIN lime+sky; intervals [21,39]/[176,194]/[106,124]/[256,274]"
        ),
        "notes": [
            "Exactly 4 features matching three-panel Panel B geometry/width",
            "Each = ONE smooth transparent PolyCollection ribbon (±9°) + ONE solid mid-stroke only",
            "Clean draw: fill alpha=0.30; mid-stroke thin white underlay (lw+1.5) + color lw=2.8; no black rim; no boundary-edge halos",
```

```
            "No multi-stroke scribble wash; no 283° companion; no Dec=0 overlay",
            "MAX bands [21,39] & [176,194]; MIN bands [106,124] & [256,274]",
            "Colors: MAX_30=#00e5ff, MAX_185=#ff2d95, MIN_115=#b8f200, MIN_265=#4fc3f7",
            "band_half_width_deg=9, band_full_width_deg=18 (5% of 360°)",
            "Exact n·x̂=μ small-circles; seam-split polygon fill; mid via plot_closed_ring",
            "No ellipse-fit, no interior chords, no decorative oval-rim loops",
            "No blocking text box on the map; legend below",
            "Colorbar preserved from source (overlays stay on oval)",
            "Overlays clip_path = registered Ellipse(cx,cy,Rx,Ry); rings stroke inset 0.97",
            "Dec=0 NOT drawn (Migkas Fig.8 already has celestial equator); CoE axis = great circle through ±D (tilted +30°) at inset=1.0",
            "axis_tilt_b_deg=30; anti-CoE (0°,+30°), CoE (180°,−30°); Migkas markers fixed at true sky positions",
            "Registration hard-set to pre medial-axis params (863,763.25,693.9927,392.542425); DT-ridge fit rejected",
            "Limb markers inset toward colored-oval center so halo stays inside",
            "Points culled at COLOR_R=0.978 of seed oval; limb markers inset to fit halo",
        ],
    }
    pass  # per-version JSON not written (PNG only)
    print(f"wrote {JSON_PATH}")


if __name__ == "__main__":
    main()
```

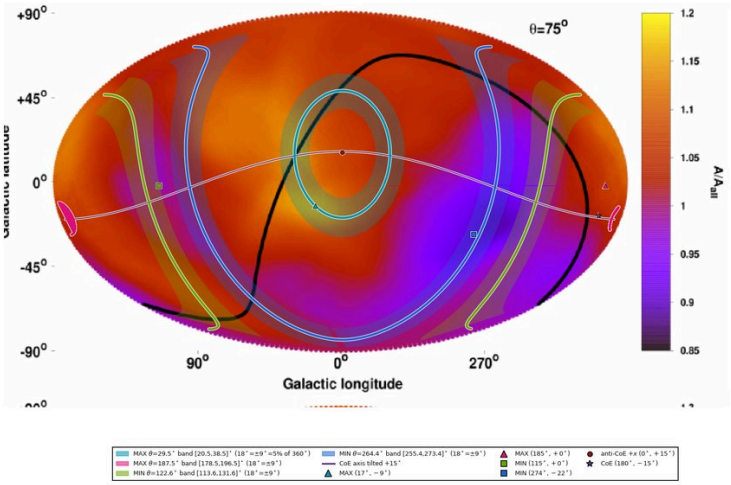
```
source: /workspace/cosmology/map_compare/FULL_migkas_theta75.png
oval seed: {'cx': 856.0, 'cy': 754.0, 'Rx': 677.0, 'Ry': 398.5, 'x0': 179, 'x1': 1533, 'y0': 356, 'y1': 1152}
restored reg: {'cx': 863.0, 'cy': 763.25, 'Rx': 693.9927, 'Ry': 392.542425, 'ceq_median_px': 1.0}
validation OK — max0 RGB [237.0, 166.33333333333334, 0.0] min1 RGB [96.61728395061728, 0.0, 205.39506172839506]
wrote /workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/fig8_overlay_matched.png
wrote /workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/fig8_overlay_matched_grad.png
wrote /workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/fig8_overlay_matched_grad_rot45.png
wrote /workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/fig8_overlay_matched_grad_rot81.png
wrote /workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/fig8_overlay_matched_grad_rot81.json
```

```
compare(FIGS / 'fig8_overlay_matched.png', '/workspace/cosmology/papers/spreadsheet_model/fig8_overlay_matched.png')
compare(FIGS / 'fig8_overlay_matched_grad.png', '/workspace/cosmology/papers/spreadsheet_model/fig8_overlay_matched_grad.png')
compare(FIGS / 'fig8_overlay_matched_grad_rot45.png', '/workspace/cosmology/papers/spreadsheet_model/fig8_overlay_matched_grad_rot45.png')
compare(FIGS / 'fig8_overlay_matched_grad_rot81.png', '/workspace/cosmology/papers/spreadsheet_model/fig8_overlay_matched_grad_rot81.png')
```

```
fig8_overlay_matched.png: new (1975, 1515) vs ref (1975, 1515) | mean |diff| = 0.000/255, pixels differing >40: 0.000% -> identical
left: approved reference papers/spreadsheet_model/fig8_overlay_matched.png   |   right: notebook re-run steps_v3/figs/fig8_overlay_matched.png
```

Migkas Fig. 8 $\theta = 75^\circ$ A/A_{all} + four CoE highlight bands, each 18° end-to-end (±9° = 5% of 360°)
(two MAX + two MIN; cyan/green/magenta/blue; anti-CoE at (0°, +15°); CoE axis tilted +15°; ℓ increases LEFT)

Migkas Fig. 8 — $\theta = 75^\circ$ A/A_{all} (their numbered colorbar)



Migkas Fig. 8 $\theta = 75^\circ$ A/A_{all} + four CoE highlight bands, each 18° end-to-end (±9° = 5% of 360°)
(two MAX + two MIN; cyan/green/magenta/blue; anti-CoE at (0°, +15°); CoE axis tilted +15°; ℓ increases LEFT)

Migkas Fig. 8 — $\theta = 75^\circ$ A/A_{all} (their numbered colorbar)

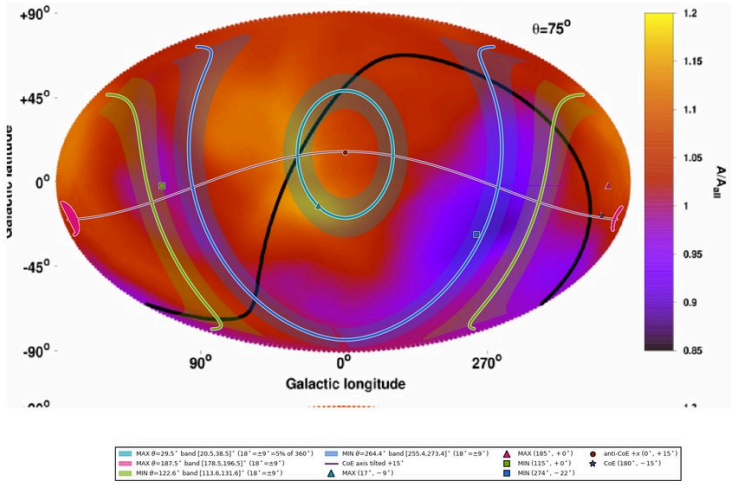
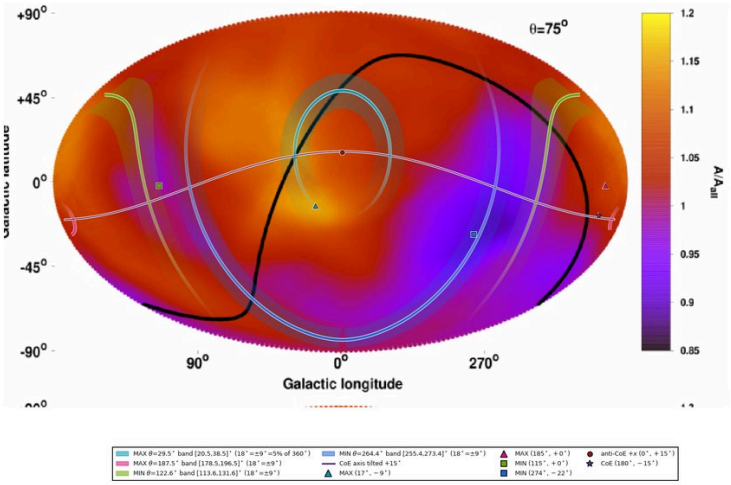


fig8_overlay_matched_grad.png: new (1975, 1515) vs ref (1975, 1515) | mean |diff| = 0.000/255, pixels differing >40: 0.000% -> identical
left: approved reference papers/spreadsheet_model/fig8_overlay_matched_grad.png | right: notebook re-run steps_v3/figs/fig8_overlay_matched_grad.png

Migkas Fig. 8 $\theta = 75^\circ$ A/A_{all} + four CoE highlight bands, each 18° end-to-end (±9° = 5% of 360°)
(two MAX + two MIN; cyan/green/magenta/blue; anti-CoE at (0°, +15°); CoE axis tilted +15°; l increases LEFT; gradient: darkest at θ ray, transparent opposite)

Migkas Fig. 8 — $\theta = 75^\circ$ A/A_{all} (their numbered colorbar)



Migkas Fig. 8 $\theta = 75^\circ$ A/A_{all} + four CoE highlight bands, each 18° end-to-end (±9° = 5% of 360°)
(two MAX + two MIN; cyan/green/magenta/blue; anti-CoE at (0°, +15°); CoE axis tilted +15°; l increases LEFT; gradient: darkest at θ ray, transparent opposite)

Migkas Fig. 8 — $\theta = 75^\circ$ A/A_{all} (their numbered colorbar)

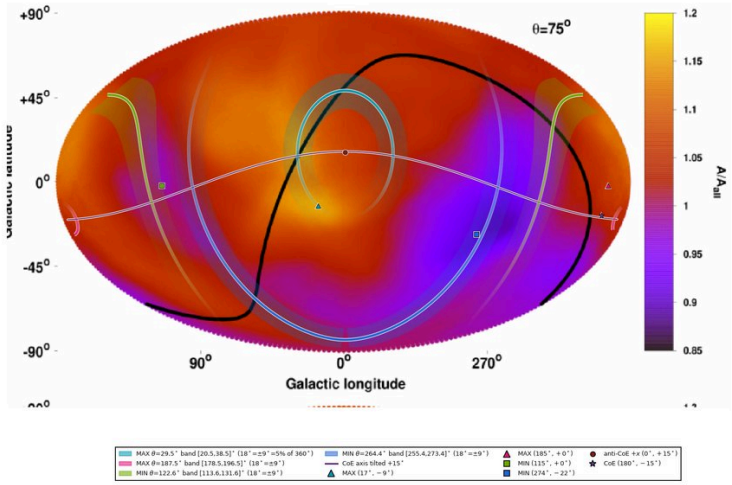
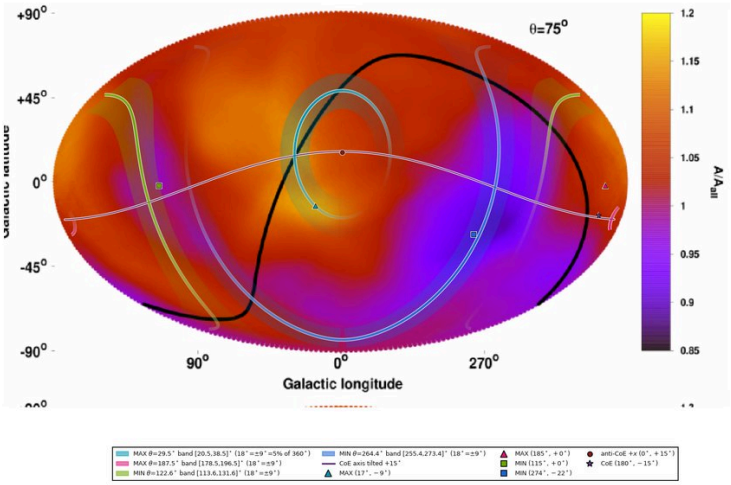


fig8_overlay_matched_grad_rot45.png: new (1975, 1515) vs ref (1975, 1515) | mean |diff| = 0.000/255, pixels differing >40: 0.000% -> identical
left: approved reference papers/spreadsheet_model/fig8_overlay_matched_grad_rot45.png | right: notebook re-run steps_v3/figs/fig8_overlay_matched_grad_rot45.png

Migkas Fig. 8 $\theta = 75^\circ$ A/A_{all} + four CoE highlight bands, each 18° end-to-end (±9° = 5% of 360°)
(two MAX + two MIN; cyan/green/magenta/blue; anti-CoE at (0°, +15°); CoE axis tilted +15°; ℓ increases LEFT; gradient: darkest at θ ray, transparent opposite; +45° CCW about CoE axis)

Migkas Fig. 8 — $\theta = 75^\circ$ A/A_{all} (their numbered colorbar)



Migkas Fig. 8 $\theta = 75^\circ$ A/A_{all} + four CoE highlight bands, each 18° end-to-end (±9° = 5% of 360°)
(two MAX + two MIN; cyan/green/magenta/blue; anti-CoE at (0°, +15°); CoE axis tilted +15°; ℓ increases LEFT; gradient: darkest at θ ray, transparent opposite; +45° CCW about CoE axis)

Migkas Fig. 8 — $\theta = 75^\circ$ A/A_{all} (their numbered colorbar)

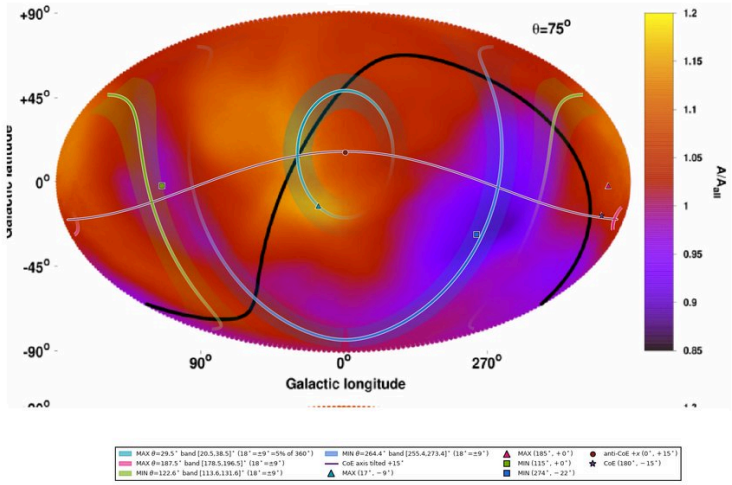
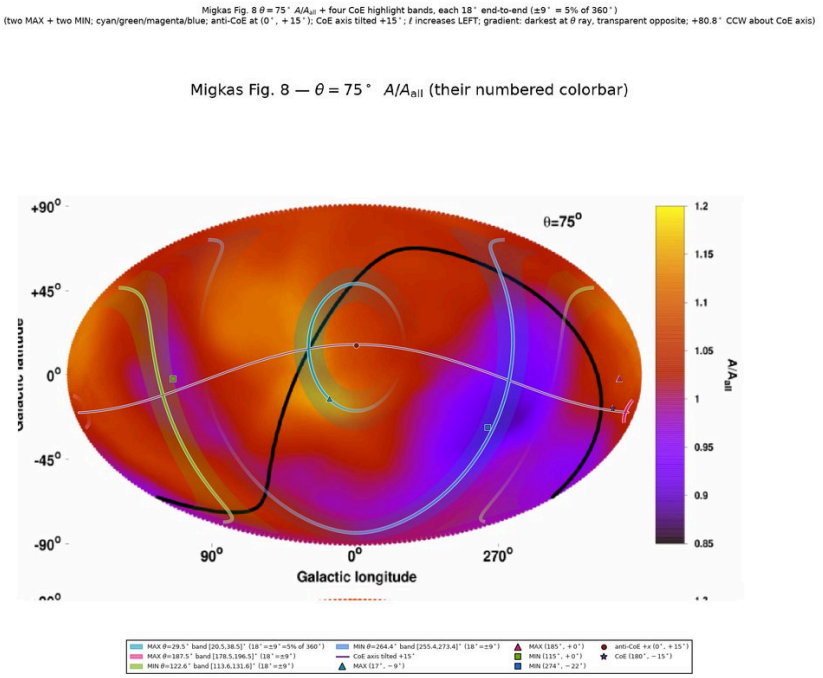
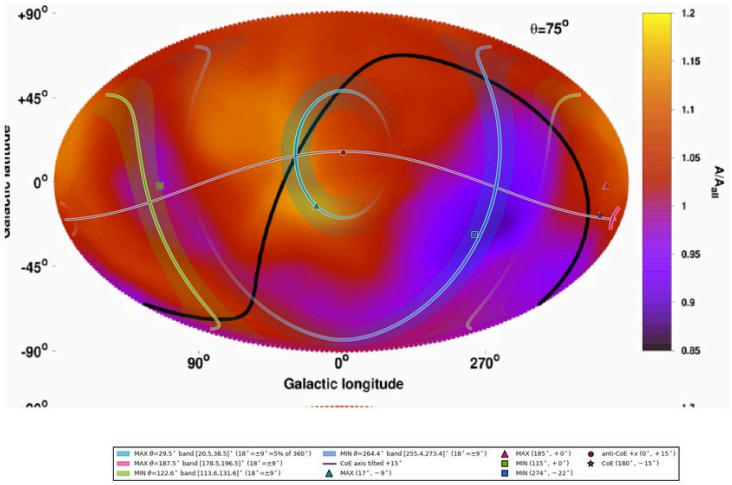


fig8_overlay_matched_grad_rot81.png: new (1975, 1515) vs ref (1975, 1515) | mean |diff| = 0.000/255, pixels differing >40: 0.000% -> identical
left: approved reference papers/spreadsheet_model/fig8_overlay_matched_grad_rot81.png | right: notebook re-run steps_v3/figs/fig8_overlay_matched_grad_rot81.png



Migkas Fig. 8 θ = 75° A/A_all + four CoE highlight bands, each 18° end-to-end (±9° = 5% of 360°)
(two MAX + two MIN; cyan/green/magenta/blue; anti-CoE at (0°, +15°); CoE axis tilted +15°; ℓ increases LEFT; gradient: darkest at θ ray, transparent opposite; +80.8° CCW about CoE axis)
Migkas Fig. 8 — θ = 75° A/A_all (their numbered colorbar)



Step 19 — Loop-flow page (the four transformation steps, earlier 26 Sep version)

What it shows. The step-by-step loop-flow figure: the $A(\ell)$ graph with its extrema bands, then the loops untilted, tilted, and with the gradient and roll, ending on the Migkas Fig. 8 overlay. The explanatory text comes from `stack_text.py`. Run from the `loops/` folder, whose `loop_fit.json` holds the fitted tilt and roll.

Source script: `/workspace/cosmology/papers/spreadsheet_model/loops/loop_flow2.py` (last modified 2026-09-26 20:56 CT), run verbatim. *Output-path edit:* `../loop_` → `/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/loop_`. *Reference image:* `papers/spreadsheet_model/loop_flow_thinshell.png`.

```
%%runscript loop_flow2.py --cwd /workspace/cosmology/papers/spreadsheet_model/loops
import json, sys, os, numpy as np, matplotlib
matplotlib.use("Agg"); import matplotlib.pyplot as plt
from matplotlib.collections import PolyCollection, LineCollection
from matplotlib.patches import Ellipse, FancyArrowPatch, Patch
from matplotlib.lines import Line2D
from PIL import Image
sys.path.insert(0, "/workspace/cosmology/figures"); sys.path.insert(0, "../bubble")
from coe_half_loop_geometry import lb_to_n, n_to_lb, ring_basis, MIGKAS_MAXES, MIGKAS_MINS, lon_disp_from_ell, great_circle_through_poles
_here = os.getcwd(); os.chdir("../bubble")
```

```
import analysis_curve as AC
from bubble_core import TH1, ML, MA, MLO, MHI, AFfour
os.chdir(_here)
F = json.load(open("loop_fit.json")); d = json.load(open("../bubble/thin_b_free_hug.json"))
A = AC.curve("ptshell", d); l0 = F["l0"]
FIT = F["fits"]["b0_only"]; UNT = F["fits"]["untilted"]
COLS = {"MAX1": "#0097b2", "MAX2": "#e0107a", "MIN1": "#6fa800", "MIN2": "#2166d4"}
pairs = F["pairs"]
keys = []
cm = {"MAX": 0, "MIN": 0}
for p in pairs: cm[p["kind"]] += 1; keys.append("%s%d" % (p["kind"], cm[p["kind"]]))
BH = 9.0
def moll(l, b):
    lon = np.radians(lon_disp_from_ell(np.asarray(l, float))); lat = np.radians(np.asarray(b, float))
    th = lat.copy()
    for _ in range(40):
        f = 2 * th + np.sin(2 * th) - np.pi * np.sin(lat); th = th - f / (2 + 2 * np.cos(2 * th) + 1e-12)
    return 2 * np.sqrt(2) / np.pi * lon * np.cos(th), np.sqrt(2) * np.sin(th)
def pts_lb(P):
    b = np.degrees(np.arcsin(np.clip(P[:, 2], -1, 1))); l = np.degrees(np.arctan2(P[:, 1], P[:, 0])) % 360; return l, b
def ring(D, Y, Z, t, phi):
    t = np.radians(t); return np.cos(t) * D[None] + np.sin(t) * (np.outer(np.sin(phi), Y) + np.outer(np.cos(phi), Z))
def sky_axes(ax):
    tt = np.linspace(0, 2 * np.pi, 400); ax.plot(2 * np.sqrt(2) * np.cos(tt), np.sqrt(2) * np.sin(tt), color="k", lw=0.9)
    for bb in (-60, -30, 0, 30, 60):
        x, y = moll(np.linspace(0.01, 359.99, 300), np.full(300, bb)); o = np.argsort(x); ax.plot(x[o], y[o], color="0.85", lw=0.5, zorder=0)
    for ll in range(0, 360, 30):
        x, y = moll(np.full(200, ll if ll else 0.001), np.linspace(-89.9, 89.9, 200)); ax.plot(x, y, color="0.85", lw=0.5, zorder=0)
    for ll in (90, 0, 270):
        x, y = moll(ll, 0); ax.text(x, y - 0.1, "%d°" % ll, fontsize=7, ha="center", va="top", color="0.35")
    for bb in (-60, -30, 30, 60):
        x, y = moll(180.01, bb); ax.text(x - 0.08, y, "%+d°" % bb, fontsize=6.5, ha="right", va="center", color="0.35")
    ax.set_xlim(-2.95, 2.95); ax.set_ylim(-1.5, 1.5); ax.set_aspect("equal"); ax.axis("off")
def draw_loops(ax, lat_b, lon_a, roll, gradient, to_xy=None, lw=1.6, under=None):
    D = lb_to_n(lon_a, lat_b); Y, Z = ring_basis(D, roll)
    to_xy = to_xy or (lambda l, b: moll(l, b))
    phi0 = np.pi / 2; phi = phi0 + np.linspace(-np.pi, np.pi, 721)
    alpha = 1 - 0.95 * np.abs(np.angle(np.exp(1j * (phi - phi0)))) / np.pi if gradient else np.ones_like(phi)
    for k, p in zip(keys, pairs):
        c = COLS[k]; t = p["theta"]
        lo, hi = ring(D, Y, Z, t - BH, phi), ring(D, Y, Z, t + BH, phi); mid = ring(D, Y, Z, t, phi)
        x1, y1 = to_xy(*pts_lb(lo)); x2, y2 = to_xy(*pts_lb(hi)); xm, ym = to_xy(*pts_lb(mid))
        quads, qa, segs, sa = [], [], [], []
        span = 0.8 if to_xy is None or True else 0.8
        scale = max(np.ptp(xm), 1e-9)
        for i in range(len(phi) - 1):
            q = np.array([[x1[i], y1[i]], [x1[i + 1], y1[i + 1]], [x2[i + 1], y2[i + 1]], [x2[i], y2[i]]])
            if np.max(np.ptp(q, axis=0)) > JUMP: continue
            quads.append(q); qa.append(0.5 * (alpha[i] + alpha[i + 1]))
            s = np.array([[xm[i], ym[i]], [xm[i + 1], ym[i + 1]]])
            if np.max(np.ptp(s, axis=0)) < JUMP: segs.append(s); sa.append(0.5 * (alpha[i] + alpha[i + 1]))
        from matplotlib.colors import to_rgba
        r, g, b_, _ = to_rgba(c)
        ax.add_collection(PolyCollection(quads, facecolors=[(r, g, b_, 0.34 * a) for a in qa], edgecolors="none", zorder=2))
```

```
        if under: ax.add_collection(LineCollection(segs, colors=[(1, 1, 1, 0.9 * a) for a in sa], linewidths=lw + 1.8, zorder=3, capstyle="butt"))
        ax.add_collection(LineCollection(segs, colors=[(r, g, b_, a) for a in sa], linewidths=lw, zorder=4, capstyle="butt"))
    return D, Y
def markers(ax, D, to_xy=None, ms=7.5, anti=True):
    to_xy = to_xy or (lambda l, b: moll(l, b))
    for k, p in zip(keys, pairs):
        x, y = to_xy(*p["migkas"]); ax.plot(x, y, "^" if p["kind"] == "MAX" else "s", ms=ms, color=COLS[k], mec="k", mew=0.7, zorder=8)
    if anti:
        la, ba = n_to_lb(D); x, y = to_xy(la, ba); ax.plot(x, y, "o", ms=6, color="#8e1b1b", mec="k", zorder=8)
        lc, bc = n_to_lb(-D); x, y = to_xy(lc, bc); ax.plot(x, y, "*", ms=9, color="#6b3fa0", mec="k", zorder=8)
def axis_line(ax, D, Y, to_xy=None, color="#5b2c6f"):
    to_xy = to_xy or (lambda l, b: moll(l, b))
    t = np.linspace(0, 2 * np.pi, 721); P = np.outer(np.cos(t), D) + np.outer(np.sin(t), Y)
    x, y = to_xy(*pts_lb(P)); segs = [np.array([[x[i], y[i]], [x[i + 1], y[i + 1]]]) for i in range(720) if abs(x[i + 1] - x[i]) < JUMP and abs(y[i +
1] - y[i]) < JUMP]
    ax.add_collection(LineCollection(segs, colors=color, linewidths=1.1, linestyles="--", zorder=5))
JUMP = 0.8
FW, FH = 17.0, 33.4
fig = plt.figure(figsize=(FW, FH))
def R(x_in, top_in, w_in, h_in): return [x_in / FW, 1 - (top_in + h_in) / FH, w_in / FW, h_in / FH]
MW, MH = 9.2, 4.6
XL = 0.55          # left column (panels)
XT, WT = 10.75, 5.95   # right column (explanations)
TOP = {1: 2.05, 2: 8.05, 3: 14.45, 4: 20.65, 5: 26.85}     # axes tops (in)
SPLIT_IN = 13.6    # page break (after step 2) for the 2-page PDF
rects = {1: R(XL + 0.5, TOP[1], 8.9, 4.1),
         2: R(XL, TOP[2], MW, MH), 3: R(XL, TOP[3], MW, MH), 4: R(XL, TOP[4], MW, MH), 5: R(XL, TOP[5], MW, MH)}
geo = {k: (r[0], r[1], r[2]) for k, r in rects.items()}
axs = {k: fig.add_axes(r, projection=("mollweide" if k in (2, 3, 4) else None)) for k, r in rects.items()}
# 1. A(l)
a = axs[1]; th2 = np.append(TH1, 360); A2 = np.append(A, A[0])
a.errorbar(ML, MA, yerr=[MLO, MHI], fmt="o", ms=2.8, color="0.5", ecolor="0.82", lw=0.7, label="Migkas Fig. 6")
a.plot(th2, AFfour(th2), color="0.25", lw=1.0, ls=":", label="$A_F$")
a.plot(th2, A2, color="k", lw=1.8, label="your model A, thin shell ($R^2$=%.3f)" % F["R2"])
for k, p in zip(keys, pairs):
    L = p["l_model"]; a.axvspan(L - BH, L + BH, color=COLS[k], alpha=0.22, lw=0)
    a.axvline(L, color=COLS[k], lw=1.2)
    a.text(L, 1.215, "%s\n$\\ell$=%.1f°" % (p["kind"], L), color=COLS[k], ha="center", va="bottom", fontsize=8, fontweight="bold")
a.set_xlim(0, 360); a.set_xticks(range(0, 361, 45)); a.set_ylim(0.76, 1.21)
a.set_xlabel("Galactic longitude $\\ell$ (deg)", fontsize=9); a.set_ylabel("$A/A_{\\rm all}$ (higher = faster)", fontsize=9)
a.legend(fontsize=7.5, loc="lower left", frameon=False)
a.set_title("1. $A(\\ell)$, latest fit: $\\ell_0$=%.2f°, Amp=%.3f, $\\varphi$=%.3f rad" % (l0, F["amp"], F["phi"]), fontsize=11, pad=34)
# ======== steps 2-5 in Chris's original format (make_coe_rings_gradient_loops.py panel B, make_migkas_fig8_gradient_loops_overlay.py) ========
import coe_gradient_loop_geometry as CG
import make_coe_rings_gradient_loops as RB
import make_migkas_fig8_gradient_loops_overlay as OV
from coe_half_loop_geometry import celestial_equator_lb, great_circle_through_poles, mu_pair_for_band
from coe_gradient_loop_geometry import small_circle_full_gradient, segment_scale, quantized_runs, rgba_array
MIN_OFF = 2.0   # display-only offset (deg) applied to BOTH MIN loops so the near-coincident pair is seen as two lines
# loop list in Chris's order MAX-1, MAX-2, MIN-1, MIN-2 (pairs are already in this order)
LOOPS = []
for k, p in zip(keys, pairs):
    t = p["theta"]; td = t + (MIN_OFF if p["kind"] == "MIN" else 0.0)
    lo, hi = td - BH, td + BH
```

```
    if hi > 180 and lo < 180 and abs(td - 180) < BH: hi = 179.999   # small ring round the CoE: band is a disc of radius |180-t|+9
    LOOPS.append(dict(key=k, kind=p["kind"], theta=t, tdisp=td, lo=lo, hi=hi, l=p["l_model"], mig=p["migkas"]))
CB = dict(COLS)   # loop colours = step-1 A(l) band colours (MAX1 cyan, MAX2 magenta, MIN1 green, MIN2 blue)
MKF = "#2b2b2b"   # Migkas marker fill (neutral); edge = paired loop colour
CO = dict(COLS)
LWB = {"MAX1": 2.4, "MAX2": 3.2, "MIN1": 1.9, "MIN2": 1.9}
LWO = {"MAX1": 3.0, "MAX2": 3.4, "MIN1": 2.6, "MIN2": 2.6}
OV.FILL_ALPHA = 0.42

def ribbon_B(ax, D, L, roll, lw, underlay=False, n_pts=721):
    """= RB.fill_mollweide_gradient_ribbon with a per-loop line width (+ optional white underlay)."""
    mu_a, mu_b = mu_pair_for_band(L["lo"], L["hi"]); mu_mid = float(np.cos(np.radians(L["tdisp"]))); color = CB[L["key"]]
    lon_a, lat_a, sc = RB._full_ring_lonlat_rad(D, mu_a, L["tdisp"], roll, n_pts=n_pts)
    lon_b, lat_b, _ = RB._full_ring_lonlat_rad(D, mu_b, L["tdisp"], roll, n_pts=n_pts)
    gap = 0.9 * np.pi; ql = np.stack([lon_a[:-1], lon_a[1:], lon_b[1:], lon_b[:-1]])
    runs = quantized_runs(segment_scale(sc), (ql.max(0) - ql.min(0)) <= gap)
    if runs: ax.add_collection(PolyCollection(RB._strip_polys_2d(lon_a, lat_a, lon_b, lat_b, runs), facecolors=rgba_array(color, [RB.FILL_ALPHA_B *
r[2] for r in runs]), edgecolors="none", zorder=2, linewidths=0))
    lon_m, lat_m, sc_m = RB._full_ring_lonlat_rad(D, mu_mid, L["tdisp"], roll, n_pts=1441)
    runs = quantized_runs(segment_scale(sc_m), np.abs(np.diff(lon_m)) <= gap)
    if runs:
        pl = RB._polylines_2d(lon_m, lat_m, runs)
        if underlay: ax.add_collection(LineCollection(pl, colors=rgba_array("white", [0.95 * r[2] for r in runs]), linewidths=lw + 1.6, zorder=3.9,
capstyle="butt", joinstyle="round"))
        ax.add_collection(LineCollection(pl, colors=rgba_array(color, [RB.LINE_ALPHA * r[2] for r in runs]), linewidths=lw, zorder=4, capstyle="butt",
joinstyle="round"))

def panel_B(ax, b0, roll, gradient, title):
    CG.ALPHA_FLOOR = 0.12 if gradient else 1.0
    D = lb_to_n(l0, b0)
    for b in [-60, -30, 0, 30, 60]:
        ell = np.linspace(0, 360, 361); RB.plot_lb_curve_mollweide(ax, ell, np.full_like(ell, b), color="#bdc3c7", lw=0.5, zorder=0)
    for e0 in range(0, 360, 30):
        bb = np.linspace(-90, 90, 181); RB.plot_lb_curve_mollweide(ax, np.full_like(bb, e0), bb, color="#bdc3c7", lw=0.5, zorder=0)
    ls, bs = great_circle_through_poles(D, n=721); RB.plot_lb_curve_mollweide(ax, ls, bs, color=RB.C_AXIS, lw=2.0, zorder=2)
    for L in LOOPS: ribbon_B(ax, D, L, roll, LWB[L["key"]], underlay=(L["key"] == "MAX2"))
    le, be = celestial_equator_lb(721); RB.plot_lb_curve_mollweide(ax, le, be, color=RB.C_EQ, lw=1.8, ls="--", zorder=2)
    la, ba = n_to_lb(D); lc, bc = n_to_lb(-D)
    ax.plot(np.radians(lon_disp_from_ell(la)), np.radians(ba), "o", ms=8, color=RB.C_ANTI, mec="k", mew=0.6, zorder=6)
    ax.plot(np.radians(lon_disp_from_ell(lc)), np.radians(bc), "*", ms=14, color=RB.C_COE, mec="k", mew=0.6, zorder=6)
    for L in LOOPS:
        ax.plot(np.radians(lon_disp_from_ell(L["mig"][0])), np.radians(L["mig"][1]), "^" if L["kind"] == "MAX" else "s", ms=10, color=MKF,
mec=CB[L["key"]], mew=2.2, zorder=7)
    te = np.array([135, 90, 45, 0, 315, 270, 225]); ax.set_xticks(np.radians(lon_disp_from_ell(te))); ax.set_xticklabels([r"$%d^\circ$" % e for e in
te], fontsize=7)
    ax.set_yticks(np.radians([-60, -30, 0, 30, 60])); ax.set_yticklabels([r"$-60^\circ$", r"$-30^\circ$", r"$0^\circ$", r"$+30^\circ$",
r"$+60^\circ$"], fontsize=7.5)
    ax.grid(False); ax.set_xlabel(r"Galactic $\ell$ (increases LEFT)", fontsize=8); ax.set_ylabel(r"Galactic $b$", fontsize=8)
    ax.set_title(title, fontsize=10, pad=6)
    CG.ALPHA_FLOOR = 0.12
    return D

def coe_inset(rect, D, roll, gradient, cols, lws, bg="white", ring_under=True):
    """Zoom on the small MAX-2 ring round the CoE (tangent-plane view, ell increases LEFT)."""
```

```
    CG.ALPHA_FLOOR = 0.12 if gradient else 1.0
    ai = fig.add_axes(rect); ai.set_facecolor(bg); c = -np.asarray(D, float); lc, bc = n_to_lb(c)
    lr, br = np.radians(lc), np.radians(bc); el = np.array([-np.sin(lr), np.cos(lr), 0]); eb = np.array([-np.sin(br) * np.cos(lr), -np.sin(br) *
np.sin(lr), np.cos(br)])
    proj = lambda P: (-np.degrees(P @ el / (P @ c)), np.degrees(P @ eb / (P @ c)))
    L = [q for q in LOOPS if q["key"] == "MAX2"][0]; col = cols["MAX2"]
    mu_a, mu_b = mu_pair_for_band(L["lo"], L["hi"])
    pa, _, sc = small_circle_full_gradient(D, mu_a, L["tdisp"], roll_deg=roll, n=361); pb, _, _ = small_circle_full_gradient(D, mu_b, L["tdisp"],
roll_deg=roll, n=361)
    xa, ya = proj(pa); xb, yb = proj(pb); runs = quantized_runs(segment_scale(sc))
    ai.add_collection(PolyCollection(RB._strip_polys_2d(xa, ya, xb, yb, runs), facecolors=rgba_array(col, [0.30 * r[2] for r in runs]),
edgecolors="none", zorder=2))
    pm, _, scm = small_circle_full_gradient(D, float(np.cos(np.radians(L["tdisp"]))), L["tdisp"], roll_deg=roll, n=721)
    xm, ym = proj(pm); runs = quantized_runs(segment_scale(scm)); pl = RB._polylines_2d(xm, ym, runs)
    if ring_under: ai.add_collection(LineCollection(pl, colors=rgba_array("white", [0.95 * r[2] for r in runs]), linewidths=lws + 1.5, zorder=3,
capstyle="butt"))
    ai.add_collection(LineCollection(pl, colors=rgba_array(col, [r[2] for r in runs]), linewidths=lws, zorder=4, capstyle="butt"))
    Y, _ = ring_basis(D, roll); tp = np.cos(np.radians(L["theta"])) * D + np.sin(np.radians(L["theta"])) * Y   # theta-ray point
    t = np.linspace(-0.5, 0.5, 50); gc = np.outer(np.cos(t), c) + np.outer(np.sin(t), np.cross(np.cross(c, [0, 0, 1]), c) / np.linalg.norm(np.cross(c,
[0, 0, 1])))
    gx, gy = proj(gc); ai.plot(gx, gy, color=RB.C_AXIS, lw=1.2, zorder=1)
    ai.plot(0, 0, "*", ms=10, color=RB.C_COE, mec="k", mew=0.5, zorder=6)
    mx, my = proj(lb_to_n(*L["mig"])[None])[0][0], proj(lb_to_n(*L["mig"])[None])[1][0]
    ai.plot(mx, my, "^", ms=7, color=MKF, mec=col, mew=1.8, zorder=7)
    for s in (-1, 1):
        pass
    R = 17; ai.set_xlim(R, -R); ai.set_ylim(-R, R); ai.set_aspect("equal"); ai.set_xticks([]); ai.set_yticks([])
    for sp in ai.spines.values(): sp.set_edgecolor("0.3"); sp.set_linewidth(0.8)
    ai.text(0.5, 0.97, "zoom: CoE ±%d°" % R, transform=ai.transAxes, fontsize=6.3, ha="center", va="top", color=("0.2" if bg == "white" else "white"))
    ai.text(0.5, 0.03, "MAX-2 ring r=%.1f°" % (180 - L["theta"]), transform=ai.transAxes, fontsize=6.3, ha="center", va="bottom", color=("0.2" if bg
== "white" else "white"))
    CG.ALPHA_FLOOR = 0.12
    return ai

tt = lambda F_: ", ".join("%+.1f" % v for v in F_["ring"])
# 2. untilted
D = panel_B(axs[2], 0.0, UNT["roll"], False, "2. Four full loops, untilted (anti-CoE at $(%.1f^\\circ,0^\\circ)$)\nloop radius $\\theta=\\ell_{\\rm
ext}-\\ell_0$; ring rms miss %.1f°" % (l0, UNT["rms_ring"]))
# 3. tilt fit
D = panel_B(axs[3], FIT["b"], FIT["roll"], False, "3. Best-fit tilt $b_0$=%+.1f°: anti-CoE $(%.1f^\\circ,%+.1f^\\circ)$\nring misses %s°; all inside
the ±9° bands" % (FIT["b"], l0, FIT["b"], tt(FIT)))
# 4. gradient
D = panel_B(axs[4], FIT["b"], FIT["roll"], True, "4. Full loops, opacity gradient darkest at $\\theta$ ray, 12%% opposite\n(roll %.1f° about CoE; MIN-
1 dark on one side, MIN-2 on the other)" % FIT["roll"])
# 5. overlay (Chris's overlay functions, colours, white/purple CoE-axis stroke; Migkas' own black celestial equator)
rgb = np.asarray(Image.open("/workspace/cosmology/map_compare/FULL_migkas_theta75.png").convert("RGB"))
seed = OV.detect_oval_bbox(rgb); reg = dict(OV.GOOD_REG)
cx, cy, Rx, Ry = reg["cx"], reg["cy"], reg["Rx"], reg["Ry"]
a = axs[5]; a.set_facecolor("white"); a.imshow(rgb, origin="upper", extent=(0, rgb.shape[1], rgb.shape[0], 0), interpolation="nearest")
x0, x1 = cx - Rx - 150, cx + Rx + 520; y0, y1 = cy - Ry - 110, cy + Ry + 130
a.set_xlim(x0, x1); a.set_ylim(y1, y0); a.set_aspect("equal"); a.axis("off")
clip_oval = Ellipse((cx, cy), width=2 * Rx, height=2 * Ry, facecolor="none", edgecolor="none", transform=a.transData); a.add_patch(clip_oval)
color_oval = (float(seed["cx"]), float(seed["cy"]), float(seed["Rx"]), float(seed["Ry"]), 0.978)
D = lb_to_n(l0, FIT["b"]); CG.ALPHA_FLOOR = 0.12
```

```
gl, gb = great_circle_through_poles(D, n=721)
OV.plot_lb_curve(a, gl, gb, cx, cy, Rx, Ry, clip_path=clip_oval, color_oval=color_oval, color="white", lw=1.15 + 1.6, alpha=0.95, zorder=3)
OV.plot_lb_curve(a, gl, gb, cx, cy, Rx, Ry, clip_path=clip_oval, color_oval=color_oval, color=OV.C_AXIS, lw=1.15, zorder=4)
_orig_ring = OV.plot_full_ring_gradient
for L in LOOPS:
    OV.plot_full_ring_gradient = (lambda *aa, lw=2.8, _w=LWO[L["key"]], **kk: _orig_ring(*aa, lw=_w, **kk))
    OV.fill_full_gradient_band(a, D, L["lo"], L["hi"], L["tdisp"], FIT["roll"], cx, cy, Rx, Ry, CO[L["key"]], inset=0.97, clip_path=clip_oval,
color_oval=color_oval)
OV.plot_full_ring_gradient = _orig_ring
def mk(l, b, m, c, ms, edge="k", mew=0.55):
    px, py = OV.lb_to_px(l, b, cx, cy, Rx, Ry); px, py = cx + (px - cx) * 0.985, cy + (py - cy) * 0.985
    a.plot(px, py, m, ms=ms + 1.8, color="white", mec="white", zorder=6); a.plot(px, py, m, ms=ms, color=c, mec=edge, mew=mew, zorder=7)
la, ba = n_to_lb(D); lc, bc = n_to_lb(-D)
mk(la, ba, "o", OV.C_ANTI, 5.5); mk(lc, bc, "*", OV.C_COE, 8)
for L in LOOPS: mk(L["mig"][0], L["mig"][1], "^" if L["kind"] == "MAX" else "s", MKF, 6.5, edge=CO[L["key"]], mew=1.8)
a.set_title("5. Overlay on Migkas Fig. 8, $\\theta=75^\\circ$ ($A\\propto H_0^2$): four full loops,\nopacity gradient darkest at $\\theta$ ray (CoE
axis white; celestial equator black)", fontsize=10, pad=4)
# insets on the small MAX-2 ring (steps 2-5)
IW, IH = 1.2 / FW, 1.2 / FH
for k, gr, cols, bb in ((2, False, CB, UNT["b"]), (3, False, CB, FIT["b"]), (4, True, CB, FIT["b"])):
    x, y, w = geo[k]; coe_inset([x + w - IW + 0.02, y - 0.001, IW, IH], lb_to_n(l0, bb), (UNT if k == 2 else FIT)["roll"], gr, cols, 2.6)
x, y, w = geo[5]; coe_inset([x + w - IW + 0.02, y + 0.002, IW, IH], lb_to_n(l0, FIT["b"]), FIT["roll"], True, CO, 2.6)
# arrows between panels: single vertical stack 0 -> 1 -> 2 -> 3 -> 4 -> 5
def arrow(p0, p1, rad=0.0):
    fig.add_artist(FancyArrowPatch((p0[0] / FW, 1 - p0[1] / FH), (p1[0] / FW, 1 - p1[1] / FH), transform=fig.transFigure,
                   arrowstyle="simple,head_width=13,head_length=14,tail_width=5", color="0.35", connectionstyle="arc3,rad=%s" % rad))
XA = XL + MW / 2
for y0_, y1_ in ((6.65, 7.2), (13.05, 13.5), (19.45, 19.9), (25.65, 26.1)): arrow((XA, y0_), (XA, y1_))
# legend (panel-B style) listing all four loops, plus the overlay colour key
def lab(L): return "%s-%s loop $\\theta$=%.1f° ($\\ell$=%.1f°), band ±9°%s" % (L["kind"], L["key"][-1], L["theta"], L["l"], (" (drawn +%.0f°)" %
MIN_OFF) if L["kind"] == "MIN" else "")
h = [Line2D([0], [0], color=RB.C_AXIS, lw=2.5, label="CoE$\\leftrightarrow$anti-CoE axis (great circle)")]
h += [Patch(facecolor=CB[L["key"]], alpha=0.55, edgecolor=CB[L["key"]], label=lab(L)) for L in LOOPS]
h += [Line2D([0], [0], color=RB.C_EQ, lw=1.8, ls="--", label="celestial equator (Dec=0)")]
h += [Line2D([0], [0], marker="^" if L["kind"] == "MAX" else "s", color="w", markerfacecolor=MKF, markeredgecolor=CB[L["key"]], markeredgewidth=2.0,
ms=8, ls="None",
            label="Migkas %s (%.0f°, %+.0f°)" % (L["kind"], L["mig"][0], L["mig"][1])) for L in LOOPS]
h += [Line2D([0], [0], marker="o", color="w", markerfacecolor=RB.C_ANTI, markeredgecolor="k", ms=8, ls="None", label="anti-CoE"),
      Line2D([0], [0], marker="*", color="w", markerfacecolor=RB.C_COE, markeredgecolor="k", ms=11, ls="None", label="CoE")]
fig.legend(handles=h, loc="lower center", ncol=3, fontsize=8.6, frameon=True, fancybox=True, bbox_to_anchor=(0.5, 0.012))
fig.text(0.5, 0.003, "Display offset: the two MIN loops (true cone radii %.1f° and %.1f° about anti-CoE) are each drawn %.0f° further along $\\theta$
so both lines are seen; all numbers use the true positions. "
         "Insets zoom on the %.1f°-radius MAX-2 ring round the CoE." % (LOOPS[2]["theta"], 360 - LOOPS[3]["theta"], MIN_OFF, 180 - LOOPS[1]["theta"]),
ha="center", va="bottom", fontsize=8.5, style="italic", color="0.25", wrap=True)
fig.suptitle("Stacked transformations: from your model A($\\ell$) to loops on the sky and the Migkas Fig. 8 overlay\n"
            "(thin-shell best fit; Galactic Mollweide, $\\ell$ increases LEFT)", fontsize=14, y=1 - 0.2 / FH, va="top")
exec(open("stack_text.py").read())
fig.savefig("/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/loop_flow_thinshell.png", dpi=150)
big = Image.open("/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/loop_flow_thinshell.png").convert("RGB")
im = big.copy(); im.thumbnail((1700, 1700)); im.save("/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/loop_flow_thinshell_web.jpg",
quality=88)
cut = int(round(SPLIT_IN * 150)); pages = [big.crop((0, 0, big.width, cut)), big.crop((0, cut, big.width, big.height))]
pages[0].save("/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/loop_flow_thinshell_stack.pdf", "PDF", resolution=150.0, save_all=True,
append_images=pages[1:])
```

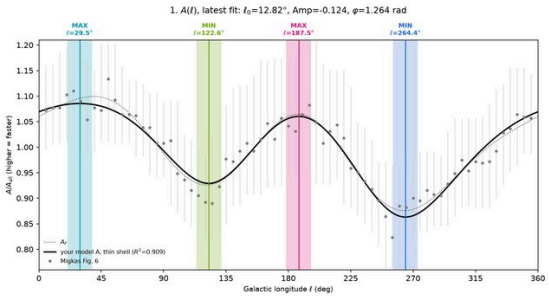
```
for i, p in enumerate(pages, 1):
    q = p.copy(); q.thumbnail((1700, 1700)); q.save("/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/loop_flow_thinshell_stack_p%d_web.jpg"
% i, quality=88); print("page", i, p.size, q.size)
print("ok", im.size)
```

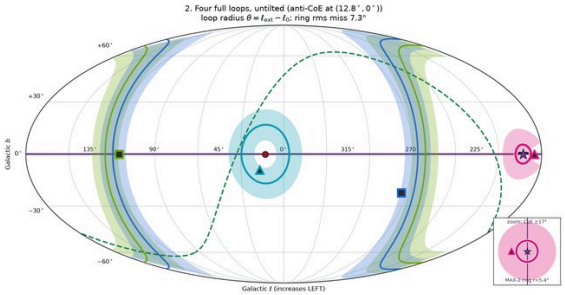
```
page 1 (2550, 2040) (1700, 1360)
page 2 (2550, 2970) (1460, 1700)
ok (865, 1700)
```

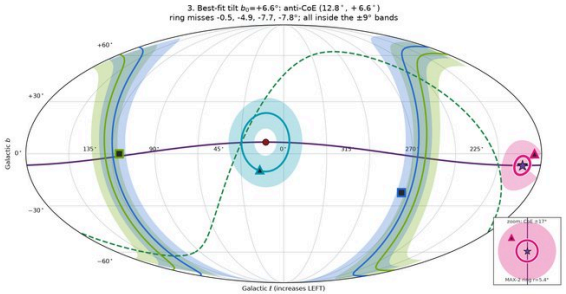
```
compare(FIGS / 'loop_flow_thinshell.png', '/workspace/cosmology/papers/spreadsheet_model/loop_flow_thinshell.png')
```

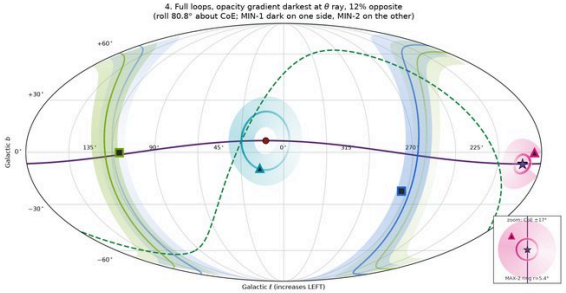
```
loop_flow_thinshell.png: new (2550, 5010) vs ref (2550, 5010) | mean |diff| = 0.000/255, pixels differing >40: 0.000% -> identical
left: approved reference papers/spreadsheet_model/loop_flow_thinshell.png   |   right: notebook re-run steps_v3/figs/loop_flow_thinshell.png
```

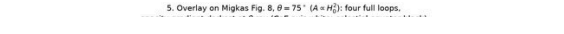
Stacked transformations: from your model A(ℓ) to loops on the sky and the Migkas Fig. 8 overlay
(thin-shell best fit; Galactic Mollweide, ℓ increases LEFT)











Step 1. Gravity breaks homology: your model A

A pure homologous blast (V = H R) would look isotropic from anywhere, so the CoE would drop out. Gravity decelerates matter at different positions relative to the CoE by different amounts, so the expansion rate seen from Earth depends on direction. Your model A (thin-shell best fit) turns this into $A(\ell) \propto H0^2$ along the Galactic plane, fitted to Migkas Fig. 6 with the parameters in Table 1a. Its two maxima and two minima are read off the curve (Table 1b); each coloured band is ±9° about an extremum, and the same colours are used for the loops on every map below.

Table 1a. Model A fit to Migkas Fig. 6

Parameter	Value
ℓ_0 (anti-CoE longitude)	12.82°
Amp	-0.124
φ	1.264 rad
R^2	0.909

Table 1b. Extrema of model A

Loop	ℓ_{ext} (°)	A/A_all
MAX-1	29.5	1.085
MIN-1	122.6	0.929
MAX-2	187.5	1.060
MIN-2	264.4	0.863

Step 2. Each extremum becomes a loop (untilted)

Model A is axisymmetric about the CoE axis, so a value read at longitude ℓ_{ext} in the plane holds on the whole cone of half-angle $\theta = \ell_{ext} - \ell_0$ about the anti-CoE direction. Drawing each cone on the sky turns the four extrema into four full loops, here with the axis kept in the Galactic plane (b0 = 0). The ring miss is the signed angular distance from each Migkas extremum to its loop (Table 2). MAX-2 is a small ring 5.4° round the CoE (see inset), and the two MIN loops have almost equal radii, so they are drawn 2° apart for display.

Table 2. Loop radii and ring misses, untilted

Loop	θ (°)	cone radius (°)	Migkas (ℓ, b)	miss (°)
MAX-1	16.7	16.7	(17°, -9°)	-6.8
MIN-1	109.8	109.8	(115°, +0°)	-7.6
MAX-2	174.6	5.4 (CoE)	(185°, +0°)	-2.5
MIN-2	251.6	108.4	(274°, -22°)	-10.2
RMS				7.3

Step 3. Tilt the CoE axis out of the plane

The anti-CoE longitude is fixed by ℓ_0, but the axis may point out of the Galactic plane. Tilting it to latitude b0 moves all four loops rigidly on the sphere; b0 is fitted by least squares on the ring misses, giving b0 = +6.6° and anti-CoE (12.8°, +6.6°). All four Migkas extrema then fall inside the ±9° bands (Table 3). Freeing ℓ_0 as well only reaches 5.8° RMS, so the remaining ~6° comes from the offset between the model and Migkas extremum longitudes, not from the orientation.

Table 3. Ring misses before and after the tilt (°)

Loop	b0 = 0	b0 = +6.6°
MAX-1	-6.8	-0.5
MIN-1	-7.6	-7.7
MAX-2	-2.5	-4.9
MIN-2	-10.2	-7.8
RMS	7.3	6.0

Step 4. Roll about the CoE axis: the gradient

The tilt fixes where the loops lie but not their orientation about the axis. A roll about the CoE axis sets each loop's θ-ray point, the point where the loop meets the great circle through the axis that plays the role of step 1's slice; there the loop value is the one actually read from A(ℓ). Each loop is drawn darkest at its θ-ray point and fades to 12% opacity opposite, so the dark end marks the measured direction and the faded part is the axisymmetric extrapolation. Because sin θ has opposite signs for MIN-1 and MIN-2, their dark ends lie on opposite sides. The roll, 80.8°, minimises the distance from the θ-ray points to the Migkas extrema (Table 4).

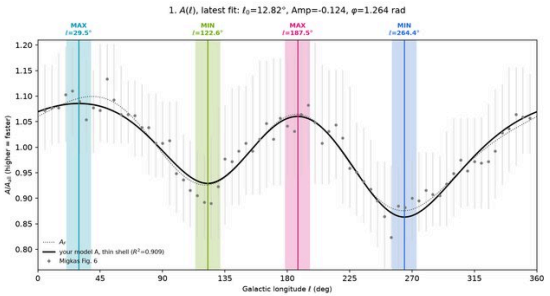
Table 4. θ-ray miss after roll 80.8° (°)

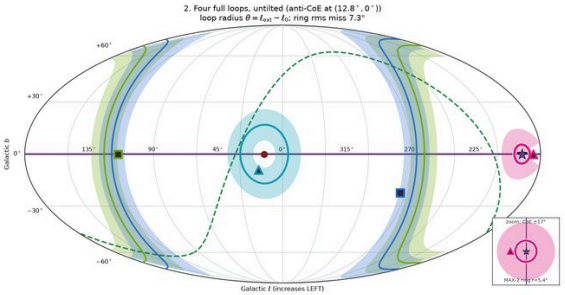
Loop	θ-ray miss
MAX-1	21.9
MIN-1	10.7
MAX-2	6.3
MIN-2	13.9
RMS	14.4

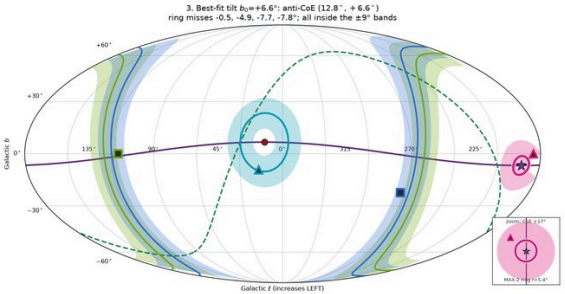
Step 5. Compare with Migkas Fig. 8

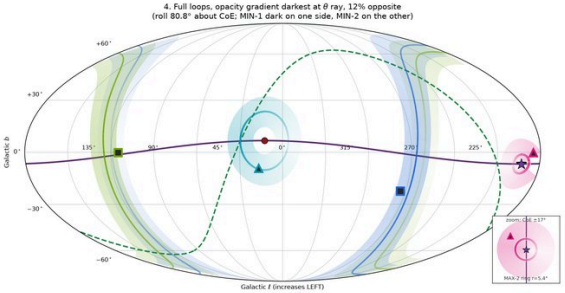
Finally the tilted and rolled loops are laid on Migkas et al.'s

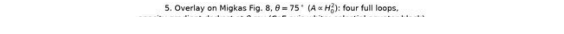
Stacked transformations: from your model A(ℓ) to loops on the sky and the Migkas Fig. 8 overlay
(thin-shell best fit; Galactic Mollweide, ℓ increases LEFT)











Step 1. Gravity breaks homology: your model A

A pure homologous blast (V = H R) would look isotropic from anywhere, so the CoE would drop out. Gravity decelerates matter at different positions relative to the CoE by different amounts, so the expansion rate seen from Earth depends on direction. Your model A (thin-shell best fit) turns this into $A(\ell) \propto H0^2$ along the Galactic plane, fitted to Migkas Fig. 6 with the parameters in Table 1a. Its two maxima and two minima are read off the curve (Table 1b); each coloured band is ±9° about an extremum, and the same colours are used for the loops on every map below.

Table 1a. Model A fit to Migkas Fig. 6

Parameter	Value
ℓ_0 (anti-CoE longitude)	12.82°
Amp	-0.124
φ	1.264 rad
R^2	0.909

Table 1b. Extrema of model A

Loop	ℓ_{ext} (°)	A/A_all
MAX-1	29.5	1.085
MIN-1	122.6	0.929
MAX-2	187.5	1.060
MIN-2	264.4	0.863

Step 2. Each extremum becomes a loop (untilted)

Model A is axisymmetric about the CoE axis, so a value read at longitude ℓ_{ext} in the plane holds on the whole cone of half-angle $\theta = \ell_{ext} - \ell_0$ about the anti-CoE direction. Drawing each cone on the sky turns the four extrema into four full loops, here with the axis kept in the Galactic plane (b0 = 0). The ring miss is the signed angular distance from each Migkas extremum to its loop (Table 2). MAX-2 is a small ring 5.4° round the CoE (see inset), and the two MIN loops have almost equal radii, so they are drawn 2° apart for display.

Table 2. Loop radii and ring misses, untilted

Loop	θ (°)	cone radius (°)	Migkas (ℓ, b)	miss (°)
MAX-1	16.7	16.7	(17°, -9°)	-6.8
MIN-1	109.8	109.8	(115°, +0°)	-7.6
MAX-2	174.6	5.4 (CoE)	(185°, +0°)	-2.5
MIN-2	251.6	108.4	(274°, -22°)	-10.2
RMS				7.3

Step 3. Tilt the CoE axis out of the plane

The anti-CoE longitude is fixed by ℓ_0, but the axis may point out of the Galactic plane. Tilting it to latitude b0 moves all four loops rigidly on the sphere; b0 is fitted by least squares on the ring misses, giving b0 = +6.6° and anti-CoE (12.8°, +6.6°). All four Migkas extrema then fall inside the ±9° bands (Table 3). Freeing ℓ_0 as well only reaches 5.8° RMS, so the remaining ~6° comes from the offset between the model and Migkas extremum longitudes, not from the orientation.

Table 3. Ring misses before and after the tilt (°)

Loop	b0 = 0	b0 = +6.6°
MAX-1	-6.8	-0.5
MIN-1	-7.6	-7.7
MAX-2	-2.5	-4.9
MIN-2	-10.2	-7.8
RMS	7.3	6.0

Step 4. Roll about the CoE axis: the gradient

The tilt fixes where the loops lie but not their orientation about the axis. A roll about the CoE axis sets each loop's θ-ray point, the point where the loop meets the great circle through the axis that plays the role of step 1's slice; there the loop value is the one actually read from A(ℓ). Each loop is drawn darkest at its θ-ray point and fades to 12% opacity opposite, so the dark end marks the measured direction and the faded part is the axisymmetric extrapolation. Because sin θ has opposite signs for MIN-1 and MIN-2, their dark ends lie on opposite sides. The roll, 80.8°, minimises the distance from the θ-ray points to the Migkas extrema (Table 4).

Table 4. θ-ray miss after roll 80.8° (°)

Loop	θ-ray miss
MAX-1	21.9
MIN-1	10.7
MAX-2	6.3
MIN-2	13.9
RMS	14.4

Step 5. Compare with Migkas Fig. 8

Finally the tilted and rolled loops are laid on Migkas et al.'s

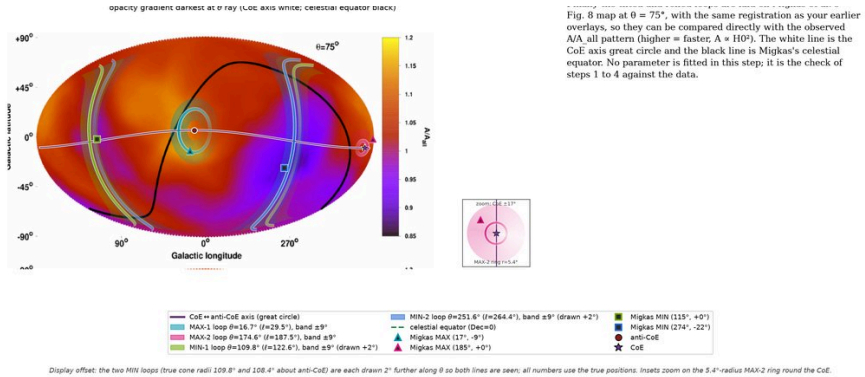


Fig. 8 map at θ = 75°, with the same registration as your earlier overlays, so they can be compared directly with the observed A/A_all pattern (higher = faster, A ∝ H0²). The white line is the CoE axis great circle and the black line is Migkas's celestial equator. No parameter is fitted in this step; it is the check of steps 1 to 4 against the data.

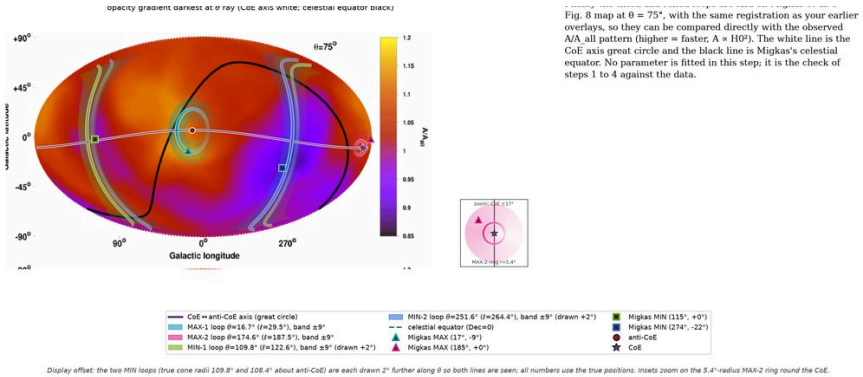


Fig. 8 map at θ = 75°, with the same registration as your earlier overlays, so they can be compared directly with the observed A/A_all pattern (higher = faster, A ∝ H0²). The white line is the CoE axis great circle and the black line is Migkas's celestial equator. No parameter is fitted in this step; it is the check of steps 1 to 4 against the data.

Step 20 — Journal page of the loop transformations (26 Sep)

What it shows. The journal-style page: the A graph (row 0), then Transformation 1 (each extremum becomes a full loop of radius $\theta = \ell_{\rm ext} - \ell_0$ about anti-CoE, untilted), Transformation 2 (tilt), Transformation 3 (roll, with the gradient darkest at the θ ray), and the Fig. 8 overlay. Each caption gives the RMS miss.

Source script: `/workspace/cosmology/papers/spreadsheet_model/loops/loop_journal.py` (last modified 2026-09-26 21:21 CT), run verbatim. *Output-path edit:* `../loop_` → `/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/loop_`; `Path("journal_rows")` → `Path("/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/journ`. *Reference image:* `papers/spreadsheet_model/loop_journal_page.png`.

```
%%runscript loop_journal.py --cwd /workspace/cosmology/papers/spreadsheet_model/loops
"""Journal page: A(l) + Chris's own 3-panel figure (make_coe_rings_gradient_loops.main) for three transformations
+ his Fig. 8 overlay (make_migkas_fig8_gradient_loops_overlay.main). Only colours, theta angles and axis orientation
(and the numbers in titles/legends) are changed; figures, layout and styling are his, unchanged."""
import json, sys, os, re, numpy as np, matplotlib
matplotlib.use("Agg"); import matplotlib.pyplot as plt
from matplotlib.figure import Figure
from pathlib import Path
from PIL import Image
sys.path.insert(0, "/workspace/cosmology/figures"); sys.path.insert(0, "../bubble")
import coe_gradient_loop_geometry as CG
import make_coe_rings_gradient_loops as RB
import make_migkas_fig8_gradient_loops_overlay as OV
from coe_half_loop_geometry import lb_to_n, n_to_lb, MIGKAS_MAXES, MIGKAS_MINS, band_edges, mu_pair_for_band, BAND_HALF
_here = os.getcwd(); os.chdir("../bubble")
import analysis_curve as AC
from bubble_core import TH1, ML, MA, MLO, MHI
os.chdir(_here)
OUT = Path("/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/journal_rows").resolve()
F = json.load(open("loop_fit.json")); d = json.load(open("../bubble/thin_b_free_hug.json"))
A = AC.curve("ptshell", d); l0 = F["l0"]; FIT = F["fits"]["b0_only"]; UNT = F["fits"]["untilted"]
```

```
P = {(p["kind"], round(p["migkas"][0])): p for p in F["pairs"]}
pMAX1, pMAX2, pMIN1, pMIN2 = P[("MAX", 17)], P[("MAX", 185)], P[("MIN", 115)], P[("MIN", 274)]
C = {"MAX1": "#0097b2", "MIN1": "#6fa800", "MAX2": "#e0107a", "MIN2": "#2166d4"}
THMAX = [pMAX1["theta"], pMAX2["theta"]]; THMIN = [pMIN1["theta"], pMIN2["theta"]]
# ---- colours and angles (only changes) ----
for M in (RB, OV):
    M.C_MAX_30, M.C_MAX_185, M.C_MIN_115, M.C_MIN_265 = C["MAX1"], C["MAX2"], C["MIN1"], C["MIN2"]
    M.COLORS_MAX = [C["MAX1"], C["MAX2"]]; M.COLORS_MIN = [C["MIN1"], C["MIN2"]]
OV.THETA_MAX, OV.THETA_MIN = list(THMAX), list(THMIN); OV.MAX_BANDS, OV.MIN_BANDS = band_edges(THMAX), band_edges(THMIN)
RB.STRIP_THETA_MAX, RB.STRIP_THETA_MIN = list(THMAX), list(THMIN)
RB.OUT_DIR = OUT; OV.OUT_DIR = OUT
def rays(ax, geo):   # his draw_panelA_theta_rays with the emphasised ray/arc at the MAX-1 angle instead of the fixed 30 deg
    th1 = geo["theta_max_from_plus_x_deg"][0]
    for mid, color in list(zip(geo["theta_max_from_plus_x_deg"], RB.COLORS_MAX)) + list(zip(geo["theta_min_from_plus_x_deg"], RB.COLORS_MIN)):
        t = np.radians(mid); x1, y1 = np.cos(t), np.sin(t); main = abs(mid - th1) < 0.5
        ax.plot([0.0, x1], [0.0, y1], color=color, lw=3.2 if main else 1.8, alpha=0.70 if main else 0.45, zorder=3, solid_capstyle="round")
        ax.plot([x1], [y1], "o", color=color, ms=4.5 if main else 3.5, alpha=0.9, zorder=5, markeredgecolor="none")
    phi = np.linspace(0.0, np.radians(th1), 60); ax.plot(0.38 * np.cos(phi), 0.38 * np.sin(phi), color=RB.C_MAX_30, lw=1.6, alpha=0.85, zorder=3)
    ax.text(0.38 * np.cos(np.radians(th1 / 2)) + 0.06, 0.38 * np.sin(np.radians(th1 / 2)) + 0.04, r"$\theta$", color=RB.C_MAX_30, fontsize=10,
fontweight="bold", ha="left", va="bottom", zorder=6)
RB.draw_panelA_theta_rays = rays
# ---- axis orientation per transformation ----
STATE = {}
def pose_for(b0):
    D = lb_to_n(l0, b0); return {"anti_lb": n_to_lb(D), "coe_lb": n_to_lb(-D), "n_anti": D, "n_coe": -D, "D": D, "tilt_b_deg": float(b0)}
def geometry(tilt_b, roll_deg):
    pose = pose_for(tilt_b); mb, nb = band_edges(THMAX), band_edges(THMIN)
    n_max = [lb_to_n(*lb) for lb in MIGKAS_MAXES]; n_min = [lb_to_n(*lb) for lb in MIGKAS_MINS]
    return {**pose, "n_maxes": n_max, "n_mins": n_min, "theta_max_from_plus_x_deg": list(THMAX), "theta_min_from_plus_x_deg": list(THMIN),
            "max_bands_deg": mb, "min_bands_deg": nb, "mu_max_bands": [mu_pair_for_band(*b) for b in mb], "mu_min_bands": [mu_pair_for_band(*b) for b
in nb],
            "sep_deg": float(np.degrees(np.arccos(np.clip(np.dot(n_max[0], n_min[1]), -1, 1)))), "band_half_width_deg": BAND_HALF,
            "roll_about_coe_deg": float(roll_deg), "full_gradient_loops": True, "gradient_rule": CG.GRADIENT_RULE}
RB.compute_geometry = geometry; OV.axis_pose = pose_for
# ---- update the numbers in his titles/legends to the latest parameters (text only) ----
exact = {int(round(t)): t for t in THMAX + THMIN}
def fix(s):
    b0 = STATE["b0"]; roll = STATE["roll"]; an, cn = n_to_lb(lb_to_n(l0, b0)), n_to_lb(-lb_to_n(l0, b0))
    s = re.sub(r"(?<!anti-)CoE (\$?)\(\$?\s*-?\d+\^\\circ,\s*[+−-]?\s*\d+\^\\circ\s*\$?\)(\$?)", lambda m: "CoE %s(%.1f^\\circ,%+.1f^\\circ)%s" %
(m.group(1), cn[0], cn[1], m.group(2)), s)
    _r = "CoE $(%.1f^\\circ,%+.1f^\\circ)$" % cn; s = re.sub(r"(?<!anti-)CoE \$\((\d+)\^\\circ,\s*[+−-]\d+\^\\circ\)\$", lambda m: _r, s)
    s = re.sub(r"anti-CoE( at| \$\+x\$| \+x)? \$\(\s*-?\d+\^\\circ,\s*[+-]?\d+\^\\circ\)\$", lambda m: "anti-CoE%s $(%.1f^\\circ,%+.1f^\\circ)$" %
(m.group(1) or "", an[0], an[1]), s)
    _r2 = "tilted $%+.1f^\\circ$" % b0; s = re.sub(r"tilted \$\+\d+\^\\circ\$", lambda m: _r2, s)
    _r3 = "$+%.1f^\\circ$ CCW" % roll; s = re.sub(r"\$\+\d+\^\\circ\$ CCW", lambda m: _r3, s)
    def th(m): return re.sub(r"(\d+)\^\\circ", lambda n: ("%.1f^\\circ" % exact[int(n.group(1))]) if int(n.group(1)) in exact else n.group(0),
m.group(0))
    s = re.sub(r"\\theta\\!=\\![^$]*", th, s)
    if not STATE["grad"]:
        s = s.replace(r"opacity gradient darkest at $\theta$ ray, fading to 12% opacity opposite", "uniform opacity (no gradient)")
        s = s.replace(r"$\alpha\propto1-0.88|\psi|/\pi$, $\psi$ = ring azimuth from $\theta$-ray point  ·  ", "")
        s = s.replace(r" ($\alpha\propto1-0.88|\psi|/\pi$)", "")
        s = s.replace(r"darkest at $\theta$ ray, fading to 12% opacity opposite; ", "").replace("full loops, opacity gradient", "full loops, uniform
opacity").replace("full loop, gradient", "full loop, uniform").replace("full loops, gradient", "full loops, uniform")
```

```
    s = s.replace("-0.0^\\circ", "0.0^\\circ").replace("+0.0^\\circ", "0.0^\\circ")
    return s
def fix_fig(fig):
    objs = list(fig.texts) + ([fig._suptitle] if fig._suptitle else [])
    for ax in fig.axes:
        objs += list(ax.texts) + [ax.title, ax._left_title, ax._right_title]
        if ax.get_legend(): objs += list(ax.get_legend().get_texts())
    for lg in fig.legends: objs += list(lg.get_texts())
    for t in objs:
        s = t.get_text()
        if s: t.set_text(fix(s))
_sf = Figure.savefig
from mpl_toolkits.mplot3d import proj3d
SIZES = []
def match_sphere(fig):
    """only size change: zoom panel C so the drawn sphere diameter equals panel A's circle diameter"""
    ax3 = [a for a in fig.axes if a.name == "3d"]; ax2 = [a for a in fig.axes if a.name == "rectilinear"]
    if not ax3 or not ax2: return
    aC, aA = ax3[0], ax2[0]; z = 1.0
    u, v = np.meshgrid(np.linspace(0, 2 * np.pi, 90), np.linspace(-np.pi / 2, np.pi / 2, 45))
    X, Y, Z = (np.cos(u) * np.cos(v)).ravel(), (np.sin(u) * np.cos(v)).ravel(), np.sin(v).ravel()
    for _ in range(6):
        aC.set_box_aspect((1, 1, 1), zoom=z); fig.canvas.draw()
        dA = np.ptp(aA.transData.transform([(-1, 0), (1, 0)])[:, 0])
        x2, y2, _ = proj3d.proj_transform(X, Y, Z, aC.get_proj()); P_ = aC.transData.transform(np.c_[x2, y2])
        dC = max(np.ptp(P_[:, 0]), np.ptp(P_[:, 1]))
        if abs(dC / dA - 1) < 0.01: break
        z *= dA / dC
    SIZES.append((round(float(dA), 1), round(float(dC), 1), round(z, 3)))
def savefig_fixed(self, *a, **k):
    fix_fig(self); match_sphere(self); return _sf(self, *a, **k)
Figure.savefig = savefig_fixed
# ---- rows 1-3: his figure, unchanged apart from the above ----
rows = []
for name, b0, roll, grad in (("row1_untilted", 0.0, 0.0, False), ("row2_tilt", FIT["b"], 0.0, False), ("row3_gradient", FIT["b"], FIT["roll"], True)):
    STATE.update(b0=b0, roll=roll, grad=grad); CG.ALPHA_FLOOR = 0.12 if grad else 1.0
    png, pdf, js = RB.main(tilt_b=b0, roll_deg=roll)
    tgt = OUT / (name + ".png"); os.replace(png, tgt); rows.append(tgt)
    for f in (pdf, js): os.remove(f)
CG.ALPHA_FLOOR = 0.12
# ---- row 4: his overlay ----
STATE.update(b0=FIT["b"], roll=FIT["roll"], grad=True)
import io, contextlib
with contextlib.redirect_stdout(io.StringIO()): OV.main(tilt_b=FIT["b"], roll_deg=FIT["roll"])
ov = sorted(OUT.glob("migkas_fig8_gradient_loops_*.png"), key=os.path.getmtime)[-1]; tgt = OUT / "row4_overlay.png"; os.replace(ov, tgt)
for f in OUT.glob("migkas_fig8_gradient_loops_*.json"): os.remove(f)
Figure.savefig = _sf
# ---- row 0: A vs angle ----
W0 = Image.open(rows[0]).width
fig, a = plt.subplots(figsize=(W0 / 160, 3.3), dpi=160)
th2 = np.append(TH1, 360); A2 = np.append(A, A[0])
a.errorbar(ML, MA, yerr=[MLO, MHI], fmt="o", ms=3, color="0.45", ecolor="0.8", lw=0.8, label="Migkas Fig. 6")
a.plot(th2, A2, color="k", lw=1.8, label="your model A, thin shell")
for k, p in (("MAX1", pMAX1), ("MIN1", pMIN1), ("MAX2", pMAX2), ("MIN2", pMIN2)):
```

```
    L = p["l_model"]; a.axvspan(L - 9, L + 9, color=C[k], alpha=0.22, lw=0); a.axvline(L, color=C[k], lw=1.3)
    a.text(L, 1.215, "%s  $\\ell$=%.1f°" % (p["kind"], L), color=C[k], ha="center", va="bottom", fontsize=10, fontweight="bold")
a.set_xlim(0, 360); a.set_xticks(range(0, 361, 45)); a.set_ylim(0.76, 1.21)
a.set_xlabel("Galactic longitude $\\ell$ (deg)"); a.set_ylabel("$A/A_{\\rm all}$ (higher = faster)")
a.legend(fontsize=9, loc="lower left", frameon=False)
a.set_title("Your model A($\\ell$), thin-shell best fit: $\\ell_0$ = %.2f°, Amp = %.3f, $\\varphi$ = %.3f rad, $R^2$ = %.3f (bands ±9°)" % (l0,
F["amp"], F["phi"], F["R2"]), fontsize=12, fontweight="bold", pad=26)
fig.subplots_adjust(left=0.06, right=0.985, top=0.80, bottom=0.17); fig.savefig(OUT / "row0_A.png", dpi=160, facecolor="white"); plt.close(fig)
# ---- caption strips (label + one line) with a small down arrow ----
def strip(label, cap, W):
    fig = plt.figure(figsize=(W / 160, 0.62), dpi=160)
    fig.patches.append(matplotlib.patches.FancyArrow(0.5, 0.97, 0, -0.30, width=0.0025, head_width=0.012, head_length=0.18, color="0.45",
transform=fig.transFigure, length_includes_head=True))
    fig.text(0.012, 0.22, label, fontsize=12, fontweight="bold", va="center"); fig.text(0.09, 0.22, cap, fontsize=11.5, va="center", style="italic")
    b = io.BytesIO(); fig.savefig(b, dpi=160, facecolor="white"); plt.close(fig); b.seek(0); return Image.open(b).convert("RGB")
caps = [("Transf. 1", "Transformation 1: each extremum becomes a full loop of radius $\\theta=\\ell_{\\rm ext}-\\ell_0$ about anti-CoE (%.1f°, 0°),
untilted; ring RMS miss %.1f°" % (l0, UNT["rms_ring"])),
        ("Transf. 2", "Transformation 2: tilt CoE axis $b_0$ = %+.1f° (anti-CoE (%.1f°, %+.1f°)); ring RMS miss %.1f° to %.1f°" % (FIT["b"], l0,
FIT["b"], UNT["rms_ring"], FIT["rms_ring"])),
        ("Transf. 3", "Transformation 3: roll %.1f° about the CoE axis; loop gradient darkest at the $\\theta$ ray, fading to 12%% opposite;
$\\theta$-ray RMS miss %.1f°" % (FIT["roll"], FIT["rms_ray"])),
        ("Transf. 4", "Transformation 4: overlay the tilted, rolled gradient loops on Migkas Fig. 8, $\\theta=75^\\circ$ ($A\\propto H_0^2$)")]
parts = [Image.open(OUT / "row0_A.png").convert("RGB")]
for (lab, cap), r in zip(caps, rows + [tgt]):
    im = Image.open(r).convert("RGB")
    if im.width != W0: im = im.resize((W0, int(im.height * W0 / im.width)), Image.LANCZOS)
    parts += [strip(lab, cap, W0), im]
Hs = sum(p.height for p in parts); page = Image.new("RGB", (W0, Hs), "white"); y = 0
for p in parts: page.paste(p, (0, y)); y += p.height
page.save("/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/loop_journal_page.png", dpi=(160, 160))
w = page.copy(); w.thumbnail((1700, 1700)); w.save("/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/loop_journal_page_web.jpg", quality=88)
cuts = [0, sum(p.height for p in parts[:5]), sum(p.height for p in parts[:7]), page.height]   # legible web parts: rows 0-2 | row 3 | overlay
for i in range(3):
    q = page.crop((0, cuts[i], page.width, cuts[i + 1])); q.thumbnail((1700, 1700));
q.save("/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/loop_journal_page_web_part%d.jpg" % (i + 1), quality=88)
print("A vs C diameters (px at 160 dpi), zoom:", SIZES); print("page", page.size, "web", w.size, [p.size for p in parts])
```

```
{
  "png": "/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/journal_rows/coe_rings_gradient_loops_tilt0.png",
  "pdf": "/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/journal_rows/coe_rings_gradient_loops_tilt0.pdf",
  "script": "/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/journal_rows/make_coe_rings_gradient_loops.py",
  "geometry_module": "/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/journal_rows/coe_half_loop_geometry.py",
  "gradient_module": "/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/journal_rows/coe_gradient_loop_geometry.py",
  "FULL_GRADIENT_LOOPS": true,
  "gradient_rule": "full loops; alpha *= (1+cos psi)/2, psi = ring azimuth from the \u03b8-ray point (ring point most along
sign(sin\u03b8_mid)\u00b7\u0176, \u0176 rolled for B/C; A unrolled); equivalently alpha_scale = (1 + sign(sin\u03b8_mid)\u00b7sin(phi))/2",
  "legacy_half_loop_rule_equiv_alpha_scale_ge_half": "\u03b8-ray side: keep (n\u00b7\u0176)*sin(\u03b8_mid)>=0 (Panel A: ey*sin(\u03b8_mid)>=0; TOP if
sin\u03b8>0, BOTTOM if sin\u03b8<0)",
  "panel_A_roll_deg": 0.0,
  "roll_applies_to_panels": [
    "B",
    "C"
  ],
  "base_alphas": {
    "fill_A": 0.28,
    "fill_B": 0.26,
    "fill_C": 0.25,
    "mid_line": 0.95
  },
  "axis_tilt_b_deg": 0.0,
  "roll_about_coe_deg": 0.0,
  "migkas_max_lb_deg": [
    [
      17.0,
      -9.0
    ],
    [
      185.0,
      0.0
    ]
  ],
  "migkas_min_lb_deg": [
    [
      115.0,
      0.0
    ],
    [
      274.0,
      -22.0
    ]
  ],
  "strip_theta_max_deg": [
    16.702461873818333,
    174.63115386586279
  ],
  "strip_theta_min_deg": [
    109.78243150454531,
    251.59236170210514
  ],
  "band_half_width_deg": 9.0,
  "band_full_width_deg": 18.0,
  "max_bands_deg": [
    [
      7.702461873818333,
```

```
      25.702461873818333
    ],
    [
      165.63115386586279,
      183.63115386586279
    ]
  ],
  "min_bands_deg": [
    [
      100.78243150454531,
      118.78243150454531
    ],
    [
      242.59236170210514,
      260.5923617021051
    ]
  ],
  "CoE_lb_deg": [
    192.8215,
    -0.0
  ],
  "anti_CoE_lb_deg": [
    12.8215,
    0.0
  ],
  "D_hat_galactic_xyz": [
    0.9750662140684075,
    0.2219141234403582,
    0.0
  ],
  "angular_sep_migkas_max0_min1_deg": 98.476384,
  "orientation": "anti-CoE at (0\u00b0, +0\u00b0), CoE at (180\u00b0, \u22120\u00b0); full loops, opacity gradient darkest at \u03b8 ray, fading to 12%
opacity opposite; roll about CoE = 0\u00b0 CCW (panels B/C only; panel A unrolled)",
  "plot_convention": "Migkas Galactic Mollweide: lon_disp=-ell_wrapped (ell increases LEFT)",
  "colors": {
    "MAX_30": "#0097b2",
    "MAX_185": "#e0107a",
    "MIN_115": "#6fa800",
    "MIN_265": "#2166d4"
  }
}

Wrote /workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/journal_rows/coe_rings_gradient_loops_tilt0.png
Wrote /workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/journal_rows/coe_rings_gradient_loops_tilt0.pdf
Wrote /workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/journal_rows/coe_rings_gradient_loops_tilt0.json
```

```
{
  "png": "/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/journal_rows/coe_rings_gradient_loops_tilt7.png",
  "pdf": "/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/journal_rows/coe_rings_gradient_loops_tilt7.pdf",
  "script": "/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/journal_rows/make_coe_rings_gradient_loops.py",
  "geometry_module": "/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/journal_rows/coe_half_loop_geometry.py",
  "gradient_module": "/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/journal_rows/coe_gradient_loop_geometry.py",
  "FULL_GRADIENT_LOOPS": true,
  "gradient_rule": "full loops; alpha *= (1+cos psi)/2, psi = ring azimuth from the \u03b8-ray point (ring point most along
sign(sin\u03b8_mid)\u00b7\u0176, \u0176 rolled for B/C; A unrolled); equivalently alpha_scale = (1 + sign(sin\u03b8_mid)\u00b7sin(phi))/2",
  "legacy_half_loop_rule_equiv_alpha_scale_ge_half": "\u03b8-ray side: keep (n\u00b7\u0176)*sin(\u03b8_mid)>=0 (Panel A: ey*sin(\u03b8_mid)>=0; TOP if
sin\u03b8>0, BOTTOM if sin\u03b8<0)",
  "panel_A_roll_deg": 0.0,
  "roll_applies_to_panels": [
    "B",
    "C"
  ],
  "base_alphas": {
    "fill_A": 0.28,
    "fill_B": 0.26,
    "fill_C": 0.25,
    "mid_line": 0.95
  },
  "axis_tilt_b_deg": 6.628740667916665,
  "roll_about_coe_deg": 0.0,
  "migkas_max_lb_deg": [
    [
      17.0,
      -9.0
    ],
    [
      185.0,
      0.0
    ]
  ],
  "migkas_min_lb_deg": [
    [
      115.0,
      0.0
    ],
    [
      274.0,
      -22.0
    ]
  ],
  "strip_theta_max_deg": [
    16.702461873818333,
    174.63115386586279
  ],
  "strip_theta_min_deg": [
    109.78243150454531,
    251.59236170210514
  ],
  "band_half_width_deg": 9.0,
  "band_full_width_deg": 18.0,
  "max_bands_deg": [
    [
      7.702461873818333,
```

```
      25.702461873818333
    ],
    [
      165.63115386586279,
      183.63115386586279
    ]
  ],
  "min_bands_deg": [
    [
      100.78243150454531,
      118.78243150454531
    ],
    [
      242.59236170210514,
      260.5923617021051
    ]
  ],
  "CoE_lb_deg": [
    192.8215,
    -6.6287
  ],
  "anti_CoE_lb_deg": [
    12.8215,
    6.6287
  ],
  "D_hat_galactic_xyz": [
    0.968547882712532,
    0.2204306244038185,
    0.1154354309474087
  ],
  "angular_sep_migkas_max0_min1_deg": 98.476384,
  "orientation": "anti-CoE at (0\u00b0, +7\u00b0), CoE at (180\u00b0, \u22127\u00b0); full loops, opacity gradient darkest at \u03b8 ray, fading to 12%
opacity opposite; roll about CoE = 0\u00b0 CCW (panels B/C only; panel A unrolled)",
  "plot_convention": "Migkas Galactic Mollweide: lon_disp=-ell_wrapped (ell increases LEFT)",
  "colors": {
    "MAX_30": "#0097b2",
    "MAX_185": "#e0107a",
    "MIN_115": "#6fa800",
    "MIN_265": "#2166d4"
  }
}

Wrote /workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/journal_rows/coe_rings_gradient_loops_tilt7.png
Wrote /workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/journal_rows/coe_rings_gradient_loops_tilt7.pdf
Wrote /workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/journal_rows/coe_rings_gradient_loops_tilt7.json
```

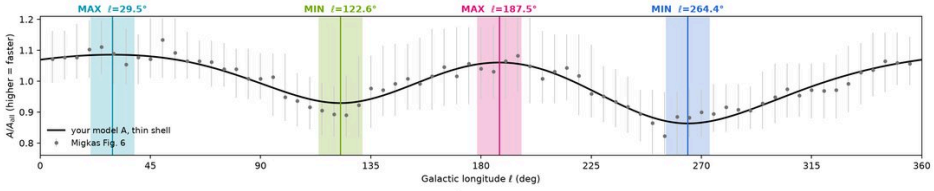
```
{
  "png": "/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/journal_rows/coe_rings_gradient_loops_tilt7_rot81ccw.png",
  "pdf": "/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/journal_rows/coe_rings_gradient_loops_tilt7_rot81ccw.pdf",
  "script": "/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/journal_rows/make_coe_rings_gradient_loops.py",
  "geometry_module": "/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/journal_rows/coe_half_loop_geometry.py",
  "gradient_module": "/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/journal_rows/coe_gradient_loop_geometry.py",
  "FULL_GRADIENT_LOOPS": true,
  "gradient_rule": "full loops; alpha *= (1+cos psi)/2, psi = ring azimuth from the \u03b8-ray point (ring point most along
sign(sin\u03b8_mid)\u00b7\u0176, \u0176 rolled for B/C; A unrolled); equivalently alpha_scale = (1 + sign(sin\u03b8_mid)\u00b7sin(phi))/2",
  "legacy_half_loop_rule_equiv_alpha_scale_ge_half": "\u03b8-ray side: keep (n\u00b7\u0176)*sin(\u03b8_mid)>=0 (Panel A: ey*sin(\u03b8_mid)>=0; TOP if
sin\u03b8>0, BOTTOM if sin\u03b8<0)",
  "panel_A_roll_deg": 0.0,
  "roll_applies_to_panels": [
    "B",
    "C"
  ],
  "base_alphas": {
    "fill_A": 0.28,
    "fill_B": 0.26,
    "fill_C": 0.25,
    "mid_line": 0.95
  },
  "axis_tilt_b_deg": 6.628740667916665,
  "roll_about_coe_deg": 80.75,
  "migkas_max_lb_deg": [
    [
      17.0,
      -9.0
    ],
    [
      185.0,
      0.0
    ]
  ],
  "migkas_min_lb_deg": [
    [
      115.0,
      0.0
    ],
    [
      274.0,
      -22.0
    ]
  ],
  "strip_theta_max_deg": [
    16.702461873818333,
    174.63115386586279
  ],
  "strip_theta_min_deg": [
    109.78243150454531,
    251.59236170210514
  ],
  "band_half_width_deg": 9.0,
  "band_full_width_deg": 18.0,
  "max_bands_deg": [
    [
      7.702461873818333,
```

```
      25.702461873818333
    ],
    [
      165.63115386586279,
      183.63115386586279
    ]
  ],
  "min_bands_deg": [
    [
      100.78243150454531,
      118.78243150454531
    ],
    [
      242.59236170210514,
      260.5923617021051
    ]
  ],
  "CoE_lb_deg": [
    192.8215,
    -6.6287
  ],
  "anti_CoE_lb_deg": [
    12.8215,
    6.6287
  ],
  "D_hat_galactic_xyz": [
    0.968547882712532,
    0.2204306244038185,
    0.1154354309474087
  ],
  "angular_sep_migkas_max0_min1_deg": 98.476384,
  "orientation": "anti-CoE at (0\u00b0, +7\u00b0), CoE at (180\u00b0, \u22127\u00b0); full loops, opacity gradient darkest at \u03b8 ray, fading to 12%
opacity opposite; roll about CoE = 81\u00b0 CCW (panels B/C only; panel A unrolled)",
  "plot_convention": "Migkas Galactic Mollweide: lon_disp=-ell_wrapped (ell increases LEFT)",
  "colors": {
    "MAX_30": "#0097b2",
    "MAX_185": "#e0107a",
    "MIN_115": "#6fa800",
    "MIN_265": "#2166d4"
  }
}

Wrote /workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/journal_rows/coe_rings_gradient_loops_tilt7_rot81ccw.png
Wrote /workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/journal_rows/coe_rings_gradient_loops_tilt7_rot81ccw.pdf
Wrote /workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/journal_rows/coe_rings_gradient_loops_tilt7_rot81ccw.json
A vs C diameters (px at 160 dpi), zoom: [(418.1, 418.5, np.float64(1.4)), (418.1, 418.5, np.float64(1.4)), (418.1, 418.5, np.float64(1.4)), (418.1,
418.5, np.float64(1.4)), (418.1, 418.5, np.float64(1.4)), (418.1, 418.5, np.float64(1.4))]
page (2278, 5713) web (678, 1700) [(2278, 528), (2278, 99), (2278, 1027), (2278, 99), (2278, 1027), (2278, 99), (2278, 988), (2278, 99), (2278, 1747)]
```

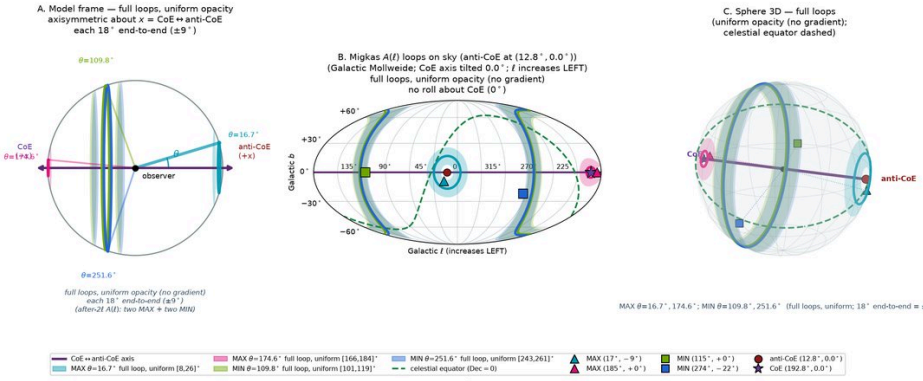

```
compare(FIGS / 'loop_journal_page.png', '/workspace/cosmology/papers/spreadsheet_model/loop_journal_page.png')
```

```
loop_journal_page.png: new (2278, 5713) vs ref (2278, 5713) | mean |diff| = 0.000/255, pixels differing >40: 0.000% -> identical
left: approved reference papers/spreadsheet_model/loop_journal_page.png   |   right: notebook re-run steps_v3/figs/loop_journal_page.png
```



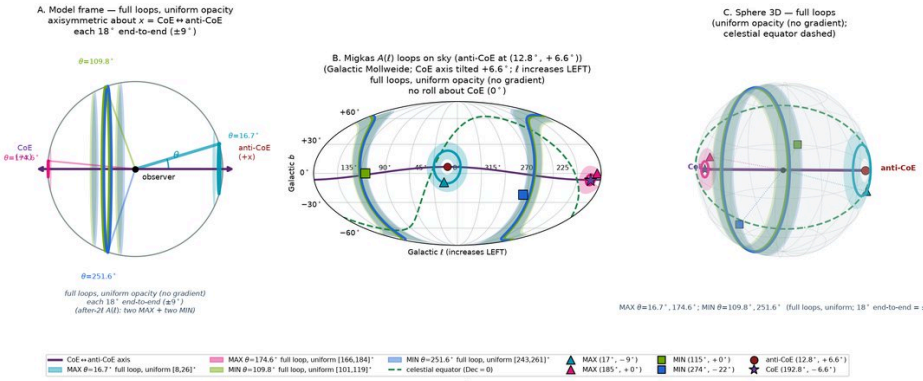
Transf. 1 *Transformation 1: each extremum becomes a full loop of radius $\theta = \ell_{ext} - \ell_0$ about anti-CoE (12.8°, 0°), untilted; ring RMS miss 7.3°*

Pure CoE axisymmetry: full loops, uniform opacity (no gradient)
(each 18° end-to-end = ±9°; MAX θ=16.7°/174.6°; MIN θ=109.8°/251.6°; anti-CoE (12.8°, 0.0°), CoE axis tilted 0.0°)



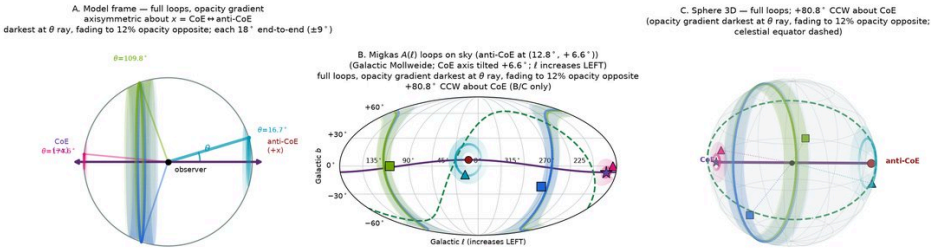
Transf. 2 *Transformation 2: tilt CoE axis b_0 = +6.6° (anti-CoE (12.8°, +6.6°)); ring RMS miss 7.3° to 6.0°*

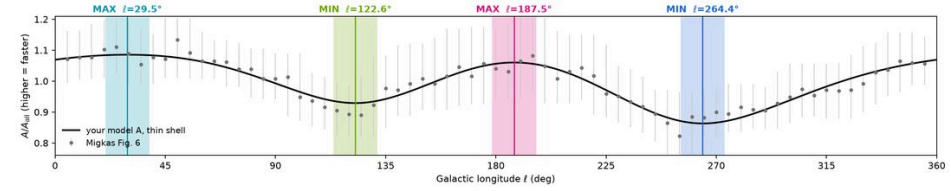
Pure CoE axisymmetry: full loops, uniform opacity (no gradient)
(each 18° end-to-end = ±9°; MAX θ=16.7°/174.6°; MIN θ=109.8°/251.6°; anti-CoE (12.8°, +6.6°), CoE axis tilted +6.6°)



Transf. 3 *Transformation 3: roll 80.8° about the CoE axis; loop gradient darkest at the θ ray, fading to 12% opposite; θ-ray RMS miss 14.4°*

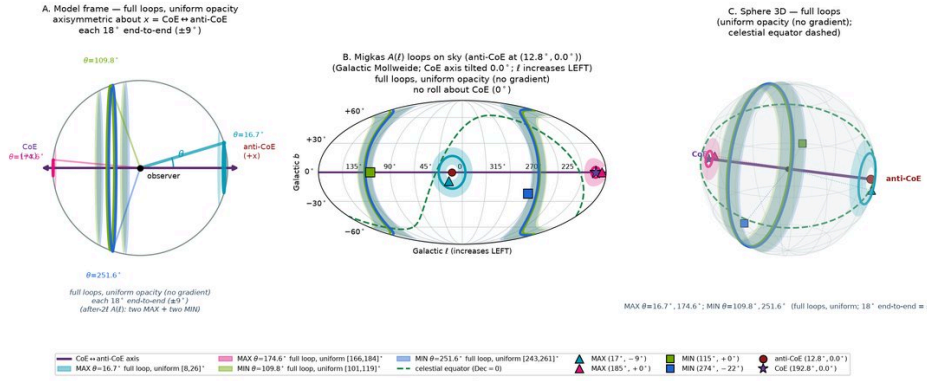
Pure CoE axisymmetry: full loops, opacity gradient darkest at θ ray, fading to 12% opacity opposite ($\alpha \propto 1 - 0.88|\psi|/\pi$)
(each 18° end-to-end = ±9°; MAX θ=16.7°/174.6°; MIN θ=109.8°/251.6°; anti-CoE (12.8°, +6.6°), CoE axis tilted +6.6°; +80.8° CCW about CoE axis)





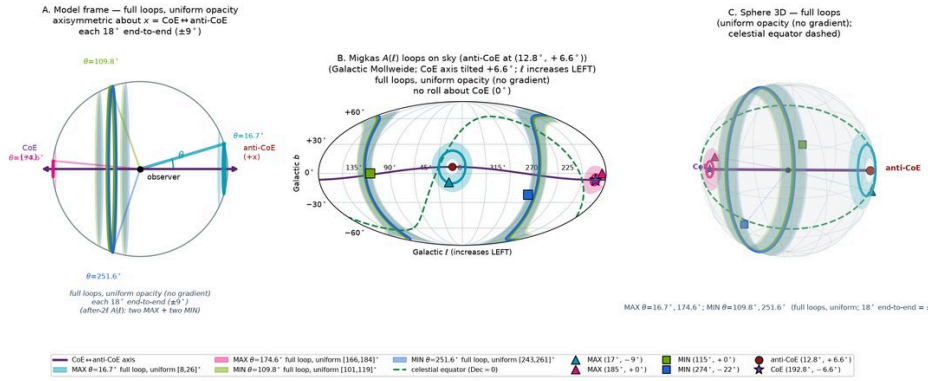
Transf. 1 *Transformation 1: each extremum becomes a full loop of radius $\theta = \ell_{ext} - \ell_0$ about anti-CoE (12.8°, 0°), untilted; ring RMS miss 7.3°*

Pure CoE axisymmetry: full loops, uniform opacity (no gradient)
(each 18° end-to-end = ±9°; MAX θ=16.7°/174.6°; MIN θ=109.8°/251.6°; anti-CoE (12.8°, 0.0°), CoE axis tilted 0.0°)



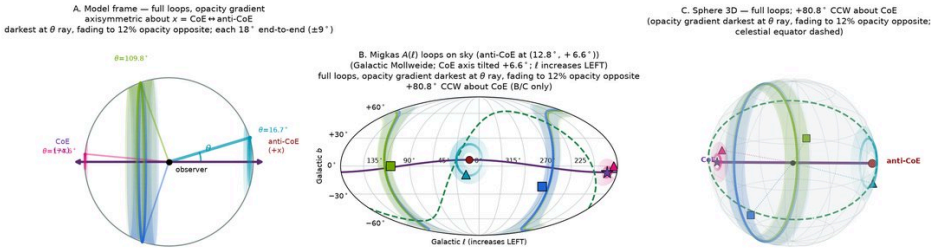
Transf. 2 *Transformation 2: tilt CoE axis b_0 = +6.6° (anti-CoE (12.8°, +6.6°)); ring RMS miss 7.3° to 6.0°*

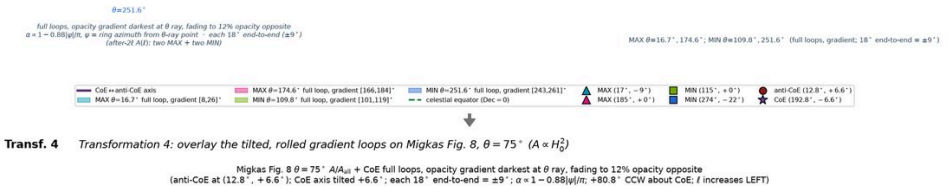
Pure CoE axisymmetry: full loops, uniform opacity (no gradient)
(each 18° end-to-end = ±9°; MAX θ=16.7°/174.6°; MIN θ=109.8°/251.6°; anti-CoE (12.8°, +6.6°), CoE axis tilted +6.6°)



Transf. 3 *Transformation 3: roll 80.8° about the CoE axis; loop gradient darkest at the θ ray, fading to 12% opposite; θ-ray RMS miss 14.4°*

Pure CoE axisymmetry: full loops, opacity gradient darkest at θ ray, fading to 12% opacity opposite ($\alpha \propto 1 - 0.88|\psi|/\pi$)
(each 18° end-to-end = ±9°; MAX θ=16.7°/174.6°; MIN θ=109.8°/251.6°; anti-CoE (12.8°, +6.6°), CoE axis tilted +6.6°; +80.8° CCW about CoE axis)

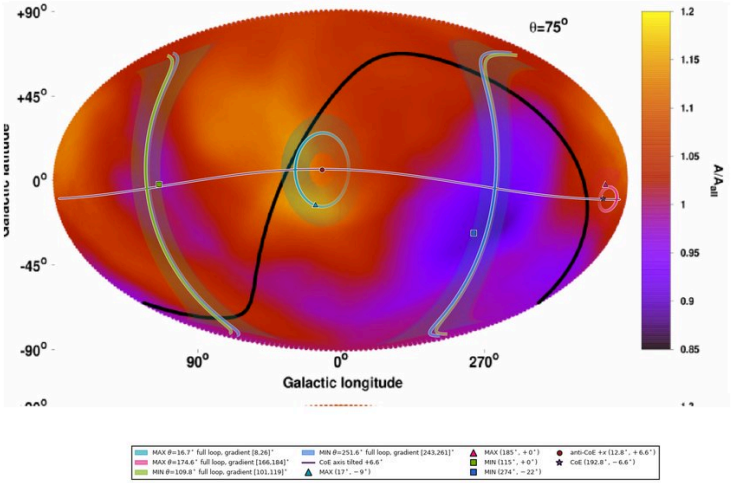




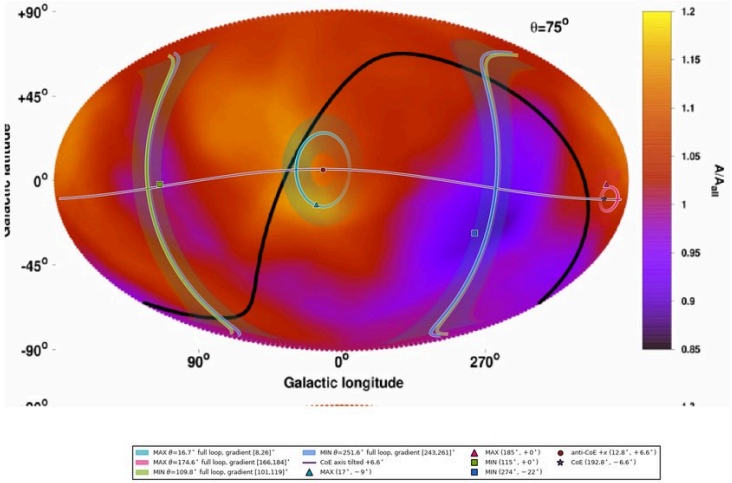
Transf. 4 *Transformation 4: overlay the tilted, rolled gradient loops on Migkas Fig. 8, $\theta = 75^\circ$ ($A \propto H_0^2$)*

Migkas Fig. 8 $\theta = 75^\circ$ $A/A_{\rm all}$ + CoE full loops, opacity gradient darkest at θ ray, fading to 12% opacity opposite (anti-CoE at (12.8°, +6.6°); CoE axis tilted +6.6°; each 18° end-to-end = ±9°; $\alpha \propto 1 - 0.88|\psi|/\pi$; +80.8° CCW about CoE; ℓ increases LEFT)

Migkas Fig. 8 — $\theta = 75^\circ$ $A/A_{\rm all}$ (their numbered colorbar)



Migkas Fig. 8 — $\theta = 75^\circ$ $A/A_{\rm all}$ (their numbered colorbar)



Step 21 — Loop stacks for the 45° and 80.8° rolls (26 Sep)

What it shows. For each roll (45° and the fitted 80.8°): the A graph, the three-panel loops in the Galactic-ℓ frame, and the Fig. 8 overlay, stacked on one page. This script loads the gradient module version it was written with (`coe_gradient_loop_geometry.cos_bak.py`).

Source script: `/workspace/cosmology/papers/spreadsheet_model/loops/loop_stack.py` (last modified 2026-09-26 22:28 CT), run verbatim. *Output-path edit:* `../loop_` → `/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/loop_` ; `../%s.png` → `/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/%s.png` ; `../%s_web.jpg` → `/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/%s_web.jpg` ; `Path("stack_rows")` → `Path("/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/stack` . *Reference image:* `papers/spreadsheet_model/loop_stack_roll45.png` , `papers/spreadsheet_model/loop_stack_roll80p8.png` .

```
%%runscript loop_stack.py --cwd /workspace/cosmology/papers/spreadsheet_model/loops
"""Journal page: A(l) + Chris's own 3-panel figure (make_coe_rings_gradient_loops.main) for three transformations
+ his Fig. 8 overlay (make_migkas_fig8_gradient_loops_overlay.main). Only colours, theta angles and axis orientation
(and the numbers in titles/legends) are changed; figures, layout and styling are his, unchanged."""
import json, sys, os, re, numpy as np, matplotlib
matplotlib.use("Agg"); import matplotlib.pyplot as plt
from matplotlib.figure import Figure
from pathlib import Path
from PIL import Image
sys.path.insert(0, "/workspace/cosmology/figures"); sys.path.insert(0, "../bubble")
import importlib.util
_sp = importlib.util.spec_from_file_location("coe_gradient_loop_geometry", "/workspace/cosmology/figures/coe_gradient_loop_geometry.cos_bak.py")
CG = importlib.util.module_from_spec(_sp); sys.modules["coe_gradient_loop_geometry"] = CG; _sp.loader.exec_module(CG)
import make_coe_rings_gradient_loops as RB
import make_migkas_fig8_gradient_loops_overlay as OV
from coe_half_loop_geometry import lb_to_n, n_to_lb, MIGKAS_MAXES, MIGKAS_MINS, band_edges, mu_pair_for_band, BAND_HALF
_here = os.getcwd(); os.chdir("../bubble")
import analysis_curve as AC
from bubble_core import TH1, ML, MA, MLO, MHI, AFfour
os.chdir(_here)
OUT = Path("/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/stack_rows").resolve(); OUT.mkdir(exist_ok=True)
F = json.load(open("loop_fit.json")); d = json.load(open("../bubble/thin_b_free_hug.json"))
A = AC.curve("ptshell", d); l0 = F["l0"]; FIT = F["fits"]["b0_only"]; UNT = F["fits"]["untilted"]
P = {(p["kind"], round(p["migkas"][0])): p for p in F["pairs"]}
pMAX1, pMAX2, pMIN1, pMIN2 = P[("MAX", 17)], P[("MAX", 185)], P[("MIN", 115)], P[("MIN", 274)]
C = {"MAX1": "#0097b2", "MIN1": "#6fa800", "MAX2": "#e0107a", "MIN2": "#2166d4"}
THMAX = [pMAX1["l_model"], pMAX2["l_model"]]; THMIN = [pMIN1["l_model"], pMIN2["l_model"]]   # model-A extremum angles ARE the angles from the anti-
CoE axis
# ---- colours and angles (only changes) ----
for M in (RB, OV):
    M.C_MAX_30, M.C_MAX_185, M.C_MIN_115, M.C_MIN_265 = C["MAX1"], C["MAX2"], C["MIN1"], C["MIN2"]
    M.COLORS_MAX = [C["MAX1"], C["MAX2"]]; M.COLORS_MIN = [C["MIN1"], C["MIN2"]]
OV.THETA_MAX, OV.THETA_MIN = list(THMAX), list(THMIN); OV.MAX_BANDS, OV.MIN_BANDS = band_edges(THMAX), band_edges(THMIN)
RB.STRIP_THETA_MAX, RB.STRIP_THETA_MIN = list(THMAX), list(THMIN)
RB.OUT_DIR = OUT; OV.OUT_DIR = OUT
def rays(ax, geo):   # his draw_panelA_theta_rays with the emphasised ray/arc at the MAX-1 angle instead of the fixed 30 deg
    th1 = geo["theta_max_from_plus_x_deg"][0]
    for mid, color in list(zip(geo["theta_max_from_plus_x_deg"], RB.COLORS_MAX)) + list(zip(geo["theta_min_from_plus_x_deg"], RB.COLORS_MIN)):
        t = np.radians(mid); x1, y1 = np.cos(t), np.sin(t); main = abs(mid - th1) < 0.5
        ax.plot([0.0, x1], [0.0, y1], color=color, lw=3.2 if main else 1.8, alpha=0.70 if main else 0.45, zorder=3, solid_capstyle="round")
        ax.plot([x1], [y1], "o", color=color, ms=4.5 if main else 3.5, alpha=0.9, zorder=5, markeredgecolor="none")
    phi = np.linspace(0.0, np.radians(th1), 60); ax.plot(0.38 * np.cos(phi), 0.38 * np.sin(phi), color=RB.C_MAX_30, lw=1.6, alpha=0.85, zorder=3)
    ax.text(0.38 * np.cos(np.radians(th1 / 2)) + 0.06, 0.38 * np.sin(np.radians(th1 / 2)) + 0.04, r"$\theta$", color=RB.C_MAX_30, fontsize=10,
fontweight="bold", ha="left", va="bottom", zorder=6)
RB.draw_panelA_theta_rays = rays
# ---- axis orientation per transformation ----
STATE = {}
def pose_for(b0):
    D = lb_to_n(l0, b0); return {"anti_lb": n_to_lb(D), "coe_lb": n_to_lb(-D), "n_anti": D, "n_coe": -D, "D": D, "tilt_b_deg": float(b0)}
def geometry(tilt_b, roll_deg):
    pose = pose_for(tilt_b); mb, nb = band_edges(THMAX), band_edges(THMIN)
    n_max = [lb_to_n(*lb) for lb in MIGKAS_MAXES]; n_min = [lb_to_n(*lb) for lb in MIGKAS_MINS]
    return {**pose, "n_maxes": n_max, "n_mins": n_min, "theta_max_from_plus_x_deg": list(THMAX), "theta_min_from_plus_x_deg": list(THMIN),
```

```
            "max_bands_deg": mb, "min_bands_deg": nb, "mu_max_bands": [mu_pair_for_band(*b) for b in mb], "mu_min_bands": [mu_pair_for_band(*b) for b
in nb],
            "sep_deg": float(np.degrees(np.arccos(np.clip(np.dot(n_max[0], n_min[1]), -1, 1)))), "band_half_width_deg": BAND_HALF,
            "roll_about_coe_deg": float(roll_deg), "full_gradient_loops": True, "gradient_rule": CG.GRADIENT_RULE}
RB.compute_geometry = geometry; OV.axis_pose = pose_for
# ---- update the numbers in his titles/legends to the latest parameters (text only) ----
exact = {int(round(t)): t for t in THMAX + THMIN}
def fix(s):
    b0 = STATE["b0"]; roll = STATE["roll"]; an, cn = n_to_lb(lb_to_n(l0, b0)), n_to_lb(-lb_to_n(l0, b0))
    s = re.sub(r"(?<!anti-)CoE (\$?)\(\$?\s*-?\d+\^\\circ,\s*[+−-]?\s*\d+\^\\circ\s*\$?\)(\$?)", lambda m: "CoE %s(%.1f^\\circ,%+.1f^\\circ)%s" %
(m.group(1), cn[0], cn[1], m.group(2)), s)
    _r = "CoE $(%.1f^\\circ,%+.1f^\\circ)$" % cn; s = re.sub(r"(?<!anti-)CoE \$\((\d+)\^\\circ,\s*[+−-]\d+\^\\circ\)\$", lambda m: _r, s)
    s = re.sub(r"anti-CoE( at| \$\+x\$| \+x)? \$\(\s*-?\d+\^\\circ,\s*[+-]?\d+\^\\circ\)\$", lambda m: "anti-CoE%s $(%.1f^\\circ,%+.1f^\\circ)$" %
(m.group(1) or "", an[0], an[1]), s)
    _r2 = "tilted $%+.1f^\\circ$" % b0; s = re.sub(r"tilted \$\+\d+\^\\circ\$", lambda m: _r2, s)
    _r3 = "$+%.1f^\\circ$ CCW" % roll; s = re.sub(r"\$\+\d+\^\\circ\$ CCW", lambda m: _r3, s)
    def th(m): return re.sub(r"(\d+)\^\\circ", lambda n: ("%.1f^\\circ" % exact[int(n.group(1))]) if int(n.group(1)) in exact else n.group(0),
m.group(0))
    s = re.sub(r"\\theta\\!=\\![^$]*", th, s)
    s = s.replace("fading to 12% opacity opposite", "transparent opposite").replace(r"$\alpha\propto1-0.88|\psi|/\pi$",
r"$\alpha\propto(1+\cos\psi)/2$").replace(r"$\alpha = 1 - 0.88|\psi|/\pi$", r"$\alpha\propto(1+\cos\psi)/2$")
    s = s.replace("A. Model frame", r"A. Galactic $\ell$ frame").replace(r"axisymmetric about $x$ = CoE$\leftrightarrow$anti-CoE", r"$\ell$ CCW from
$+x$; CoE axis at $\ell_0$=%.1f°" % l0)
    if not STATE["grad"]:
        s = s.replace(r"opacity gradient darkest at $\theta$ ray, fading to 12% opacity opposite", "uniform opacity (no gradient)")
        s = s.replace(r"$\alpha\propto1-0.88|\psi|/\pi$, $\psi$ = ring azimuth from $\theta$-ray point  ·  ", "")
        s = s.replace(r" ($\alpha\propto1-0.88|\psi|/\pi$)", "")
        s = s.replace(r"darkest at $\theta$ ray, fading to 12% opacity opposite; ", "").replace("full loops, opacity gradient", "full loops, uniform
opacity").replace("full loop, gradient", "full loop, uniform").replace("full loops, gradient", "full loops, uniform")
    s = s.replace("-0.0^\\circ", "0.0^\\circ").replace("+0.0^\\circ", "0.0^\\circ")
    return s
def fix_fig(fig):
    objs = list(fig.texts) + ([fig._suptitle] if fig._suptitle else [])
    for ax in fig.axes:
        objs += list(ax.texts) + [ax.title, ax._left_title, ax._right_title]
        if ax.get_legend(): objs += list(ax.get_legend().get_texts())
    for lg in fig.legends: objs += list(lg.get_texts())
    for t in objs:
        s = t.get_text()
        if s: t.set_text(fix(s))
_sf = Figure.savefig
from mpl_toolkits.mplot3d import proj3d
SIZES = []
def match_sphere(fig):
    """only size change: zoom panel C so the drawn sphere diameter equals panel A's circle diameter"""
    ax3 = [a for a in fig.axes if a.name == "3d"]; ax2 = [a for a in fig.axes if a.name == "rectilinear"]
    if not ax3 or not ax2: return
    aC, aA = ax3[0], ax2[0]; z = 1.0
    u, v = np.meshgrid(np.linspace(0, 2 * np.pi, 90), np.linspace(-np.pi / 2, np.pi / 2, 45))
    X, Y, Z = (np.cos(u) * np.cos(v)).ravel(), (np.sin(u) * np.cos(v)).ravel(), np.sin(v).ravel()
    for _ in range(6):
        aC.set_box_aspect((1, 1, 1), zoom=z); fig.canvas.draw()
        dA = np.ptp(aA.transData.transform([(-1, 0), (1, 0)])[:, 0])
        x2, y2, _ = proj3d.proj_transform(X, Y, Z, aC.get_proj()); P_ = aC.transData.transform(np.c_[x2, y2])
```

```
        dC = max(np.ptp(P_[:, 0]), np.ptp(P_[:, 1]))
        if abs(dC / dA - 1) < 0.01: break
        z *= dA / dC
    SIZES.append((round(float(dA), 1), round(float(dC), 1), round(z, 3)))
def savefig_fixed(self, *a, **k):
    fix_fig(self); match_sphere(self); return _sf(self, *a, **k)
Figure.savefig = savefig_fixed


# ---- panel A in Galactic l: rotate the model-frame drawing by l0 (l=0 at +x, CCW); loops land at their own l ----
import matplotlib.transforms as mtransforms
_panelA = RB.panel_A_model_frame
LTXT = []
def panelA_gal(ax, geo, roll_deg):
    _panelA(ax, geo, roll_deg)
    rot = mtransforms.Affine2D().rotate_deg(l0); c, s_ = np.cos(np.radians(l0)), np.sin(np.radians(l0))
    R2 = lambda x, y: (c * x - s_ * y, s_ * x + c * y)
    for ln in ax.lines: ln.set_transform(rot + ax.transData)
    for co in ax.collections: co.set_transform(rot + ax.transData)
    for t in list(ax.texts):
        txt = t.get_text()
        if isinstance(t, matplotlib.text.Annotation):
            t.xy = R2(*t.xy); t.set_position(R2(*t.get_position())); continue
        if "CoE" in txt or "theta\\!=" in txt: t.remove(); continue
        x, y = t.get_position()
        if y > -1.3: t.set_position(R2(x, y))      # keep the caption block under the circle where it was
    # thin l reference diameters + l ticks/labels
    pu = np.array([-s_, c]); ax.plot([-pu[0], pu[0]], [-pu[1], pu[1]], color="0.45", lw=0.7, ls=(0, (1.5, 2)), zorder=1.5)   # perpendicular to the
CoE axis
    for a_, ha, va, off in ((0, "left", "top", (0.02, -0.03)), (90, "center", "bottom", (0, 0.02)), (180, "right", "bottom", (-0.02, 0.03)), (270,
"center", "top", (0, -0.02))):
        u = np.array([np.cos(np.radians(a_)), np.sin(np.radians(a_))]); ax.plot(*zip(u * 1.0, u * 1.05), color="0.35", lw=0.9, zorder=2)
        ax.text(*(u * 1.06 + off), r"$\ell$=%d°" % a_, color="0.35", fontsize=7, ha=ha, va=va, zorder=6)
    ea = np.array([c, s_])
    ax.text(*(ea * 1.20), "anti-CoE\n$\\ell$=%.1f°" % l0, color=RB.C_ANTI, fontsize=8, ha="left", va="center")
    ax.text(*(-ea * 1.20), "CoE\n$\\ell$=%.1f°" % ((l0 + 180) % 360), color=RB.C_COE, fontsize=8, ha="right", va="center")
    for mid, col in list(zip(geo["theta_max_from_plus_x_deg"], RB.COLORS_MAX)) + list(zip(geo["theta_min_from_plus_x_deg"], RB.COLORS_MIN)):
        L = (mid + l0) % 360; u = np.array([np.cos(np.radians(L)), np.sin(np.radians(L))]); pos = u * 1.30; va = "center"
        LTXT.append(ax.text(*pos, r"$\theta\!=\!%.1f^\circ$" % mid, color=col, fontsize=7.5, ha="center", va=va, fontweight="bold"))
    ax.figure.canvas.draw(); rr_ = ax.figure.canvas.get_renderer()
    bbs = [(t.get_text()[:16], t.get_window_extent(rr_)) for t in ax.texts if t.get_text()]
    from itertools import combinations
    for (n1, b1), (n2, b2) in combinations(bbs, 2):
        if b1.overlaps(b2): print("A-OVERLAP", repr(n1), repr(n2))
RB.panel_A_model_frame = panelA_gal
import io, contextlib
RUNS = []
for ROLL, TAG in ((45.0, "roll45"), (FIT["roll"], "roll80p8")):
    STATE.update(b0=FIT["b"], roll=ROLL, grad=True)
    png, pdf, js = RB.main(tilt_b=FIT["b"], roll_deg=ROLL)
    three = OUT / ("threepanel_%s.png" % TAG); os.replace(png, three)
    for f in (pdf, js): os.remove(f)
    with contextlib.redirect_stdout(io.StringIO()): OV.main(tilt_b=FIT["b"], roll_deg=ROLL)
    ov = sorted(OUT.glob("migkas_fig8_gradient_loops_*.png"), key=os.path.getmtime)[-1]; ovp = OUT / ("overlay_%s.png" % TAG); os.replace(ov, ovp)
```

```
    for f in OUT.glob("migkas_fig8_gradient_loops_*.json"): os.remove(f)
    RUNS.append((TAG, three, ovp))
Figure.savefig = _sf
# ---- A vs angle, Figure-2 style ----
plt.rcParams.update({"font.size": 9, "font.family": "serif"})
U = json.load(open("../bubble/universe_thin.json"))["free"]
th2 = np.append(TH1, 360); Af2 = AFfour(th2); A2 = np.append(A, A[0])
fig, (a, b) = plt.subplots(2, 1, figsize=(6.6, 5.25), sharex=True, gridspec_kw={"height_ratios": [2.0, 1], "hspace": 0.06})
for k, p in (("MAX1", pMAX1), ("MIN1", pMIN1), ("MAX2", pMAX2), ("MIN2", pMIN2)):
    L = p["l_model"]
    for ax_ in (a, b): ax_.axvspan(L - 9, L + 9, color=C[k], alpha=0.22, lw=0, zorder=0)
a.errorbar(ML, MA, yerr=[MLO, MHI], fmt="o", ms=3, color="0.45", ecolor="0.75", lw=0.8, label="Migkas Fig. 6")
a.plot(th2, Af2, color="C0", lw=1.5, label="$A_F$")
a.plot(th2, A2, color="C3", lw=1.6, label="thin shell + core, $\\Omega_s$=%.2f, $\\ell_0$=%.1f° ($R^2$=%.3f; vs $A_F$ %.3f)" % (U["Os"], U["l0"],
U["R2"], U["R2F"]))
b.plot(th2, 100 * (A2 - Af2) / Af2, color="C3", lw=1.4, label="model")
a.set_ylabel("$A/A_{\\rm all}$"); a.set_ylim(0.62, 1.22); a.legend(fontsize=7.2, loc="lower left", frameon=False)
b.axhline(0, color="k", lw=0.6); b.plot(ML, 100 * (MA - AFfour(ML)) / AFfour(ML), "o", ms=2.5, color="0.45", label="data")
b.set_ylabel("$100\\,(\\cdot A_F)/A_F$ [%]", fontsize=8); b.set_xlabel("Galactic longitude $\\ell$ (deg)")
b.set_xlim(0, 360); b.set_xticks(range(0, 361, 45)); b.legend(fontsize=7, frameon=False, loc="upper right", ncol=2)
agr = OUT / "Agraph.png"; fig.savefig(agr, dpi=200, bbox_inches="tight", pad_inches=0.04); plt.close(fig)
# ---- stack at natural sizes ----
def trim(im, pad=12):
    g = np.asarray(im.convert("L")) < 250; ys, xs = np.nonzero(g)
    return im.crop((max(xs.min() - pad, 0), max(ys.min() - pad, 0), min(xs.max() + pad + 1, im.width), min(ys.max() + pad + 1, im.height)))
for TAG, three, ovp in RUNS:
    parts = [trim(Image.open(p).convert("RGB")) for p in (agr, three, ovp)]
    W = max(p.width for p in parts); H = sum(p.height for p in parts)
    page = Image.new("RGB", (W, H), "white"); y = 0
    for p in parts: page.paste(p, ((W - p.width) // 2, y)); y += p.height
    page.save("/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/loop_stack_%s.png" % TAG); w = page.copy(); w.thumbnail((1700, 1700));
w.save("/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/loop_stack_%s_web.jpg" % TAG, quality=88)
    for n, p in (("loop_threepanel_" + TAG, parts[1]), ("loop_overlay_" + TAG, parts[2])):
        p.save("/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/%s.png" % n); q = p.copy(); q.thumbnail((1700, 1700));
q.save("/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/%s_web.jpg" % n, quality=88)
    print(TAG, [p.size for p in parts], page.size)
q = parts[0].copy(); parts[0].save("/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/loop_Agraph.png"); q.thumbnail((1700, 1700));
q.save("/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/loop_Agraph_web.jpg", quality=88)
print("sizes", SIZES)
```

```
{
  "png": "/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/stack_rows/coe_rings_gradient_loops_tilt7_rot45ccw.png",
  "pdf": "/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/stack_rows/coe_rings_gradient_loops_tilt7_rot45ccw.pdf",
  "script": "/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/stack_rows/make_coe_rings_gradient_loops.py",
  "geometry_module": "/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/stack_rows/coe_half_loop_geometry.py",
  "gradient_module": "/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/stack_rows/coe_gradient_loop_geometry.py",
  "FULL_GRADIENT_LOOPS": true,
  "gradient_rule": "full loops; alpha *= (1+cos psi)/2, psi = ring azimuth from the \u03b8-ray point (ring point most along
sign(sin\u03b8_mid)\u00b7\u0176, \u0176 rolled for B/C; A unrolled); equivalently alpha_scale = (1 + sign(sin\u03b8_mid)\u00b7sin(phi))/2",
  "legacy_half_loop_rule_equiv_alpha_scale_ge_half": "\u03b8-ray side: keep (n\u00b7\u0176)*sin(\u03b8_mid)>=0 (Panel A: ey*sin(\u03b8_mid)>=0; TOP if
sin\u03b8>0, BOTTOM if sin\u03b8<0)",
  "panel_A_roll_deg": 0.0,
  "roll_applies_to_panels": [
    "B",
    "C"
  ],
  "base_alphas": {
    "fill_A": 0.28,
    "fill_B": 0.26,
    "fill_C": 0.25,
    "mid_line": 0.95
  },
  "axis_tilt_b_deg": 6.628740667916665,
  "roll_about_coe_deg": 45.0,
  "migkas_max_lb_deg": [
    [
      17.0,
      -9.0
    ],
    [
      185.0,
      0.0
    ]
  ],
  "migkas_min_lb_deg": [
    [
      115.0,
      0.0
    ],
    [
      274.0,
      -22.0
    ]
  ],
  "strip_theta_max_deg": [
    29.523945464158125,
    187.45263745620258
  ],
  "strip_theta_min_deg": [
    122.60391509488511,
    264.41384529244493
  ],
  "band_half_width_deg": 9.0,
  "band_full_width_deg": 18.0,
  "max_bands_deg": [
    [
      20.523945464158125,
```

```
      38.523945464158125
    ],
    [
      178.45263745620258,
      196.45263745620258
    ]
  ],
  "min_bands_deg": [
    [
      113.60391509488511,
      131.6039150948851
    ],
    [
      255.41384529244493,
      273.41384529244493
    ]
  ],
  "CoE_lb_deg": [
    192.8215,
    -6.6287
  ],
  "anti_CoE_lb_deg": [
    12.8215,
    6.6287
  ],
  "D_hat_galactic_xyz": [
    0.968547882712532,
    0.2204306244038185,
    0.1154354309474087
  ],
  "angular_sep_migkas_max0_min1_deg": 98.476384,
  "orientation": "anti-CoE at (0\u00b0, +7\u00b0), CoE at (180\u00b0, \u22127\u00b0); full loops, opacity gradient darkest at \u03b8 ray, fading to 12%
opacity opposite; roll about CoE = 45\u00b0 CCW (panels B/C only; panel A unrolled)",
  "plot_convention": "Migkas Galactic Mollweide: lon_disp=-ell_wrapped (ell increases LEFT)",
  "colors": {
    "MAX_30": "#0097b2",
    "MAX_185": "#e0107a",
    "MIN_115": "#6fa800",
    "MIN_265": "#2166d4"
  }
}

Wrote /workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/stack_rows/coe_rings_gradient_loops_tilt7_rot45ccw.png
Wrote /workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/stack_rows/coe_rings_gradient_loops_tilt7_rot45ccw.pdf
Wrote /workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/stack_rows/coe_rings_gradient_loops_tilt7_rot45ccw.json
```

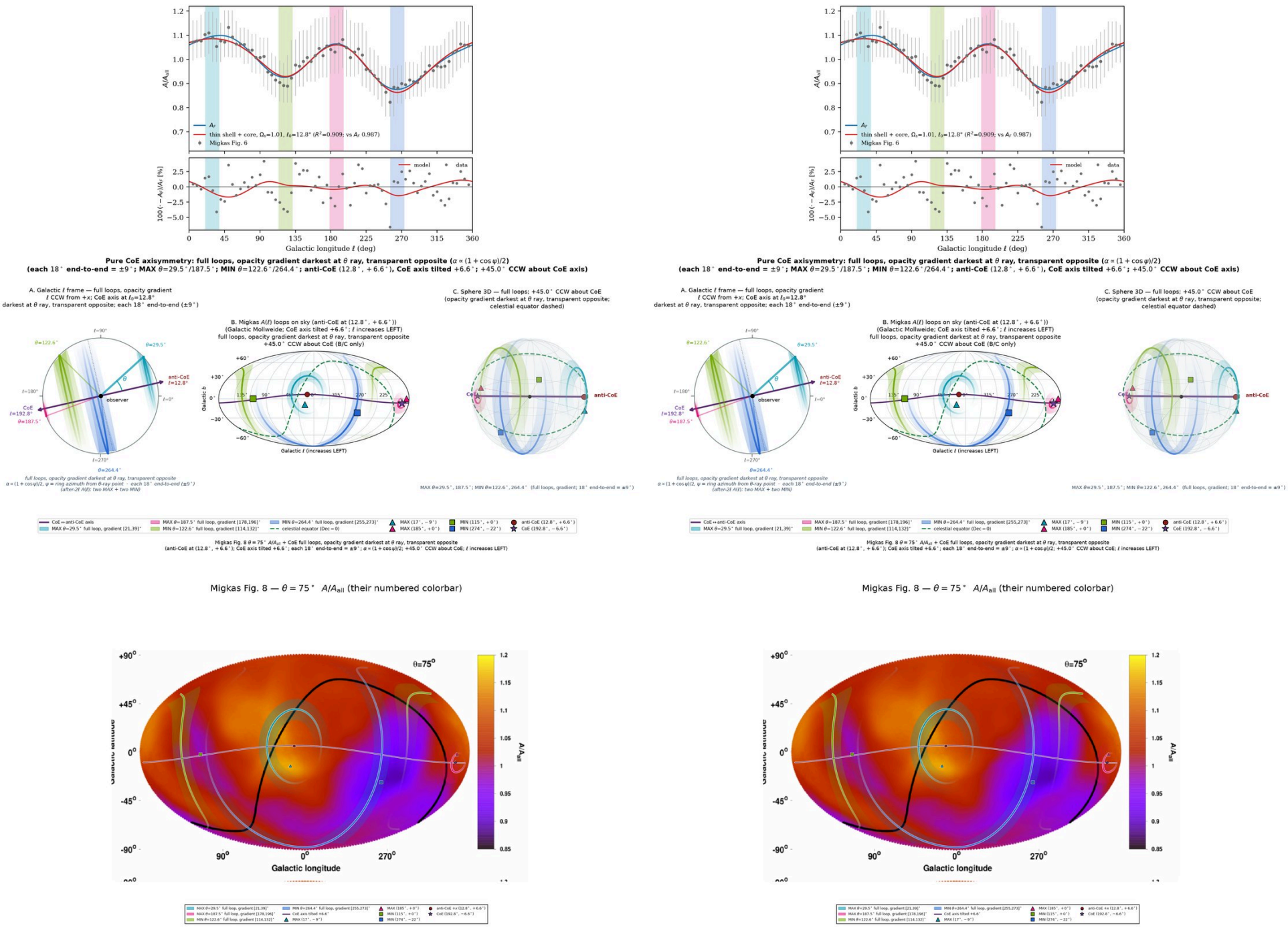
```
{
  "png": "/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/stack_rows/coe_rings_gradient_loops_tilt7_rot81ccw.png",
  "pdf": "/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/stack_rows/coe_rings_gradient_loops_tilt7_rot81ccw.pdf",
  "script": "/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/stack_rows/make_coe_rings_gradient_loops.py",
  "geometry_module": "/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/stack_rows/coe_half_loop_geometry.py",
  "gradient_module": "/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/stack_rows/coe_gradient_loop_geometry.py",
  "FULL_GRADIENT_LOOPS": true,
  "gradient_rule": "full loops; alpha *= (1+cos psi)/2, psi = ring azimuth from the \u03b8-ray point (ring point most along
sign(sin\u03b8_mid)\u00b7\u0176, \u0176 rolled for B/C; A unrolled); equivalently alpha_scale = (1 + sign(sin\u03b8_mid)\u00b7sin(phi))/2",
  "legacy_half_loop_rule_equiv_alpha_scale_ge_half": "\u03b8-ray side: keep (n\u00b7\u0176)*sin(\u03b8_mid)>=0 (Panel A: ey*sin(\u03b8_mid)>=0; TOP if
sin\u03b8>0, BOTTOM if sin\u03b8<0)",
  "panel_A_roll_deg": 0.0,
  "roll_applies_to_panels": [
    "B",
    "C"
  ],
  "base_alphas": {
    "fill_A": 0.28,
    "fill_B": 0.26,
    "fill_C": 0.25,
    "mid_line": 0.95
  },
  "axis_tilt_b_deg": 6.628740667916665,
  "roll_about_coe_deg": 80.75,
  "migkas_max_lb_deg": [
    [
      17.0,
      -9.0
    ],
    [
      185.0,
      0.0
    ]
  ],
  "migkas_min_lb_deg": [
    [
      115.0,
      0.0
    ],
    [
      274.0,
      -22.0
    ]
  ],
  "strip_theta_max_deg": [
    29.523945464158125,
    187.45263745620258
  ],
  "strip_theta_min_deg": [
    122.60391509488511,
    264.41384529244493
  ],
  "band_half_width_deg": 9.0,
  "band_full_width_deg": 18.0,
  "max_bands_deg": [
    [
      20.523945464158125,
```

```
      38.523945464158125
    ],
    [
      178.45263745620258,
      196.45263745620258
    ]
  ],
  "min_bands_deg": [
    [
      113.60391509488511,
      131.6039150948851
    ],
    [
      255.41384529244493,
      273.41384529244493
    ]
  ],
  "CoE_lb_deg": [
    192.8215,
    -6.6287
  ],
  "anti_CoE_lb_deg": [
    12.8215,
    6.6287
  ],
  "D_hat_galactic_xyz": [
    0.968547882712532,
    0.2204306244038185,
    0.1154354309474087
  ],
  "angular_sep_migkas_max0_min1_deg": 98.476384,
  "orientation": "anti-CoE at (0\u00b0, +7\u00b0), CoE at (180\u00b0, \u22127\u00b0); full loops, opacity gradient darkest at \u03b8 ray, fading to 12%
opacity opposite; roll about CoE = 81\u00b0 CCW (panels B/C only; panel A unrolled)",
  "plot_convention": "Migkas Galactic Mollweide: lon_disp=-ell_wrapped (ell increases LEFT)",
  "colors": {
    "MAX_30": "#0097b2",
    "MAX_185": "#e0107a",
    "MIN_115": "#6fa800",
    "MIN_265": "#2166d4"
  }
}

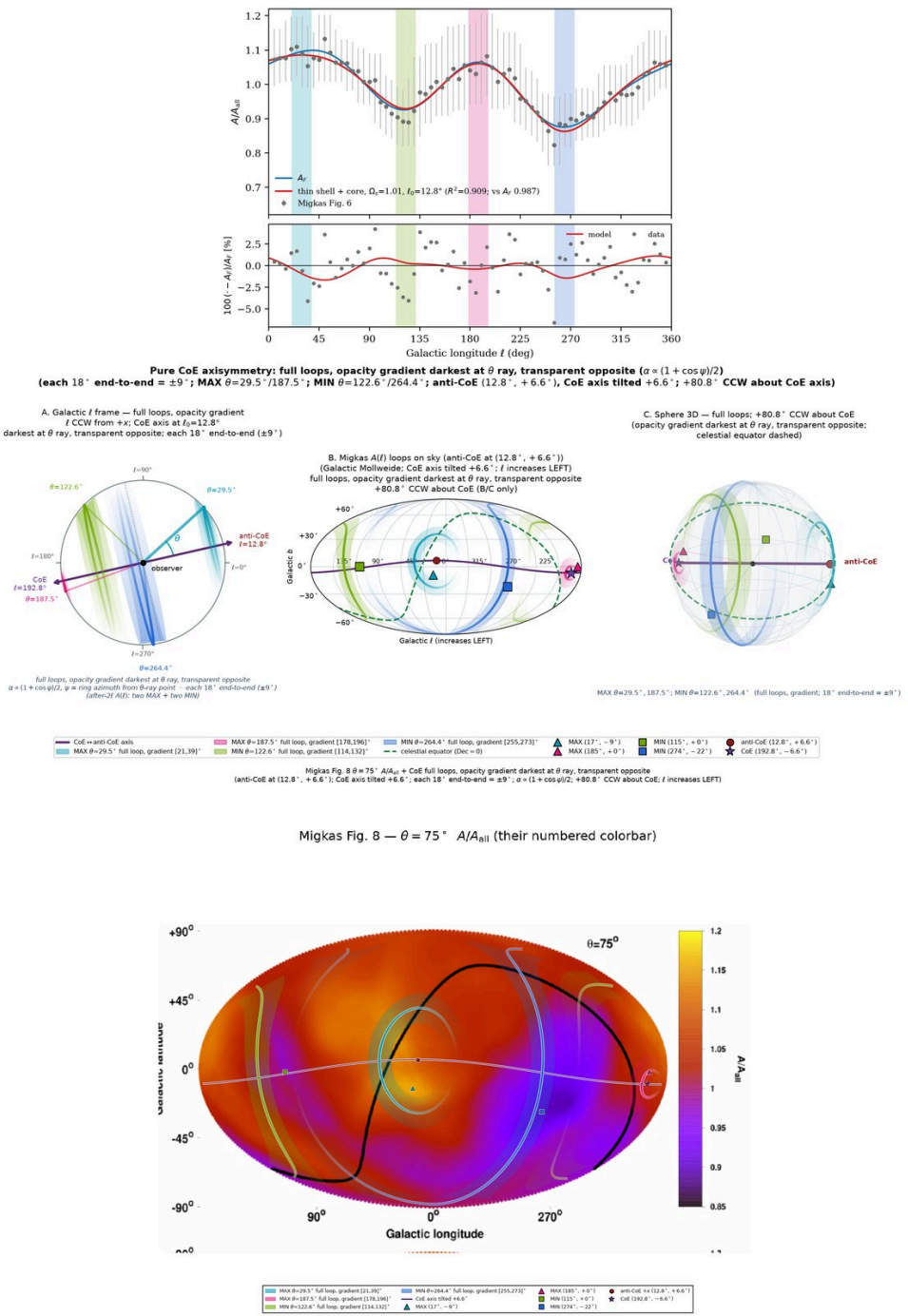
Wrote /workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/stack_rows/coe_rings_gradient_loops_tilt7_rot81ccw.png
Wrote /workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/stack_rows/coe_rings_gradient_loops_tilt7_rot81ccw.pdf
Wrote /workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/stack_rows/coe_rings_gradient_loops_tilt7_rot81ccw.json
roll45 [(1176, 905), (2302, 1018), (1516, 1410)] (2302, 3333)
roll80p8 [(1176, 905), (2302, 1018), (1516, 1410)] (2302, 3333)
sizes [(418.1, 418.5, np.float64(1.4)), (418.1, 418.5, np.float64(1.4)), (418.1, 418.5, np.float64(1.4)), (418.1, 418.5, np.float64(1.4))]
```


```
compare(FIGS / 'loop_stack_roll45.png', '/workspace/cosmology/papers/spreadsheet_model/loop_stack_roll45.png')
compare(FIGS / 'loop_stack_roll80p8.png', '/workspace/cosmology/papers/spreadsheet_model/loop_stack_roll80p8.png')
```

```
loop_stack_roll45.png: new (2302, 3333) vs ref (2302, 3333) | mean |diff| = 0.000/255, pixels differing >40: 0.000% -> identical
left: approved reference papers/spreadsheet_model/loop_stack_roll45.png   |   right: notebook re-run steps_v3/figs/loop_stack_roll45.png
```

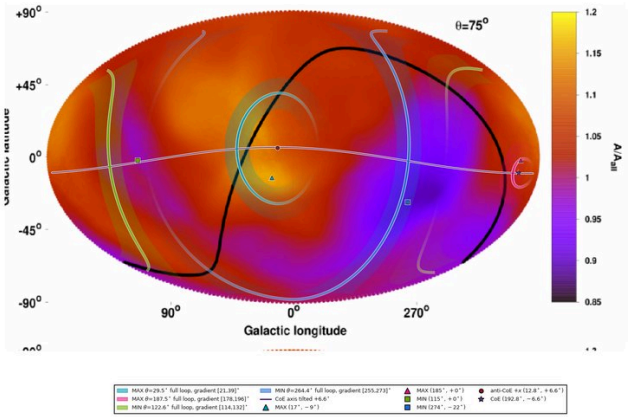


loop_stack_roll80p8.png: new (2302, 3333) vs ref (2302, 3333) | mean |diff| = 0.000/255, pixels differing >40: 0.000% -> identical
left: approved reference papers/spreadsheet_model/loop_stack_roll80p8.png | right: notebook re-run steps_v3/figs/loop_stack_roll80p8.png



Migkas Fig. 8 — $\theta = 75^\circ$ $A/A_{\rm all}$ (their numbered colorbar)

Migkas Fig. 8 — $\theta = 75^\circ$ $A/A_{\rm all}$ (their numbered colorbar)



Step 22 — Stacked figures: 3-column and 4-column (stack_4col) versions

What it shows. The summary stack: the model $A(\ell)$ graph (Newtonian $\Omega_s = 0.049$ plus the relativistic curve) at the top, then four loop rows (untilted/no roll; $+15^\circ$ tilt; $+15^\circ$ tilt with gradient; $+15^\circ$ tilt with gradient and $+80.8^\circ$ roll), plus the Migkas Fig. 8 overlays (matched, gradient, 80.8° roll). The 3-column version puts the overlays in a row at the bottom; the 4-column version puts each overlay beside its loop row at the original scale. This script composes existing images: the approved row and overlay images on disk, including three row images and two overlay images kept in the chat attachments folder.

Source script: `/workspace/cosmology/figures/make_stack_clean.py` (last modified 2026-09-27 12:28 CT), run verbatim. *Output-path edit:* `P = "/workspace/cosmology/papers/spreadsheet_model/` → `P = "/workspace/cosmology/papers/spreadsheet_model/"` `OUTP = "/workspac`; `im.save(P + name + ".jpg` → `im.save(OUTP + name + ".jpg`; `.save(P + name + "_web.jpg` → `.save(OUTP + name + "_web.jpg`. *Reference image:* `papers/spreadsheet_model/stack_3col_clean.jpg`, `papers/spreadsheet_model/stack_4col_clean.jpg`.

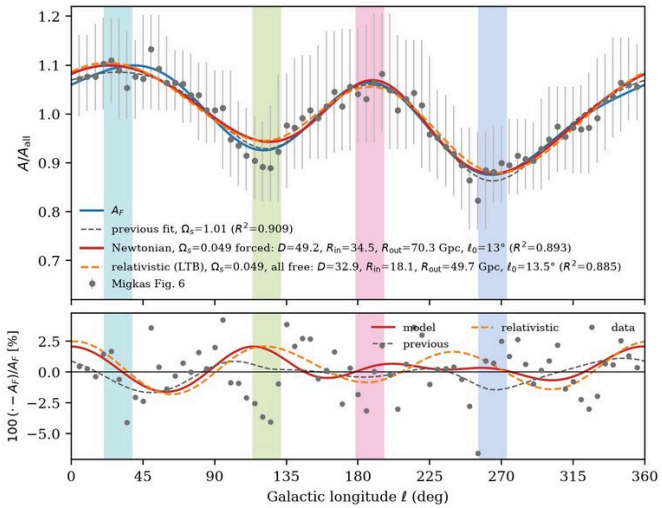
```
%%runscript make_stack_clean.py
from PIL import Image, ImageDraw, ImageFont
import numpy as np
A = "/home/box/agent-data/agents/196ab413-0708-4f81-99fc-58a32c5366c5/attachments/"
P = "/workspace/cosmology/papers/spreadsheet_model/"
OUTP = "/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/"   # notebook: outputs redirected
graph = Image.open(P + "bubble/newt_omb_rel_Agraph.png").convert("RGB")   # was attachment f480c67c... (newt_omb_Agraph); + relativistic curve
rows_src = [Image.open(f).convert("RGB") for f in (P + "loop_threepanel_notilt_web.jpg",
            A + "e2fab4ef4115016f7bc638a6ac1075749b6acbab6e4c2ca6b2f20e29f8735139.jpg",
            A + "fe905a57d5aaf4dede004a4cbdc50883bbf92c984372f9f260251bbfc057b757.jpg",
            A + "d6982109d332a918353df473def180356c133fb81196c90b6e0dea313238ef05.jpg")]
ovs = [Image.open(f).convert("RGB") for f in (A + "3b64499f8c49ab6537b8a73b1fd80965e8b5f8ae82d93d6c99b61dc4f5b8344d.jpg",
       A + "83d1ae71406713257f0623d2b732446ef1ee220de6167592d4a45c5cac55637c.jpg", P + "fig8_overlay_matched_grad_rot81_web.jpg")]
LAB = ["Untilted, no roll", "+15° tilt", "+15° tilt, gradient", "+15° tilt, gradient, +80.8° roll"]
OLAB = ["+15° tilt", "gradient", "+80.8° roll"]
FB = ImageFont.truetype("/usr/share/fonts/truetype/dejavu/DejaVuSans-Bold.ttf", 22)
FH = ImageFont.truetype("/usr/share/fonts/truetype/dejavu/DejaVuSans-Bold.ttf", 18)
FT = ImageFont.truetype("/usr/share/fonts/truetype/dejavu/DejaVuSans.ttf", 17)
W = 1700; LH = 40
def row_content(i):
    im = rows_src[i]
    if i == 0:
        return im.crop((0, 110, W, 628))            # keep A./B./C. headings + footnotes (first row only)
    c = im.crop((0, 222, W, 572)).copy()          # drops suptitle, A/C headings, footnotes, legend
    ImageDraw.Draw(c).rectangle((385, 234 - 222, 1212, 279 - 222), fill="white")   # B heading
    return c
legend = rows_src[3].crop((0, 712, W, 768))        # one legend strip (last row's)
def ov_crop(k, cbar):
    return ovs[k].crop((140, 355, 1445 if cbar else 1276, 1060))
def label(dr, x, y, t, f=FB, anchor="mm"):
    dr.text((x, y), t, font=f, fill="black", anchor=anchor)
def save(im, name):
    im.save(OUTP + name + ".jpg", quality=92)
    w = 1400; im.resize((w, round(im.height * w / im.width)), Image.LANCZOS).save(OUTP + name + "_web.jpg", quality=88)
```

```
    print(name, im.size)
# ---------------- 3-column version ----------------
parts = [("g", graph)]
blocks = []
H = graph.height + 20
for i in range(4):
    rc = row_content(i); blocks.append((i, rc)); H += LH + rc.height + 10
s3 = (W - 20 - 2 * 20) / (ov_crop(0, 0).width * 2 + ov_crop(2, 1).width)
oc = [ov_crop(0, 0), ov_crop(1, 0), ov_crop(2, 1)]
oc = [o.resize((round(o.width * s3), round(o.height * s3)), Image.LANCZOS) for o in oc]
OT = 26
H += LH + OT + max(o.height for o in oc) + 10 + legend.height + 10
im = Image.new("RGB", (W, H), "white"); dr = ImageDraw.Draw(im)
y = 0; im.paste(graph, ((W - graph.width) // 2, y)); y += graph.height + 20
for i, rc in blocks:
    label(dr, W // 2, y + LH // 2, LAB[i]); y += LH; im.paste(rc, (0, y)); y += rc.height + 10
label(dr, W // 2, y + LH // 2, "Migkas Fig. 8, θ = 75°, A/Aall"); y += LH
x = 10
for k, o in enumerate(oc):
    label(dr, x + round((1276 - 140) * s3) // 2, y + OT // 2, OLAB[k], FT)
    im.paste(o, (x, y + OT)); x += o.width + 20
y += OT + max(o.height for o in oc) + 10
im.paste(legend, (0, y)); y += legend.height + 10
save(im.crop((0, 0, W, y)), "stack_3col_clean")
# ---------------- 4-column version ----------------
s4 = 772 / 1304   # same overlay scale as stack_4col.jpg (no shrinking)
o4 = [ov_crop(0, 1), ov_crop(1, 0), ov_crop(2, 0)]
o4 = [o.resize((round(o.width * s4), round(o.height * s4)), Image.LANCZOS) for o in o4]
CW = max(o.width for o in o4) + 20; W4 = W + CW; HT = 30
Hs = [LH + blocks[0][1].height + 10]
for j in range(1, 4):
    rc = blocks[j][1]; Hs.append(max(LH + rc.height, (HT if j == 1 else 0) + o4[j - 1].height) + 12)
H4 = graph.height + 20 + sum(Hs) + legend.height + 10
im = Image.new("RGB", (W4, H4), "white"); dr = ImageDraw.Draw(im)
y = 0; im.paste(graph, ((W4 - graph.width) // 2, y)); y += graph.height + 20
for j in range(4):
    rc = blocks[j][1]
    label(dr, W // 2, y + LH // 2, LAB[j]); im.paste(rc, (0, y + LH))
    if j >= 1:
        oy = y
        if j == 1:
            label(dr, W + 10 + round((1276 - 140) * s4) // 2, y + HT // 2, "Migkas Fig. 8, θ = 75°, A/Aall", FH); oy += HT
        im.paste(o4[j - 1], (W + 10, oy))
    y += Hs[j]
im.paste(legend, (0, y)); y += legend.height + 10
save(im.crop((0, 0, W4, y)), "stack_4col_clean")
```

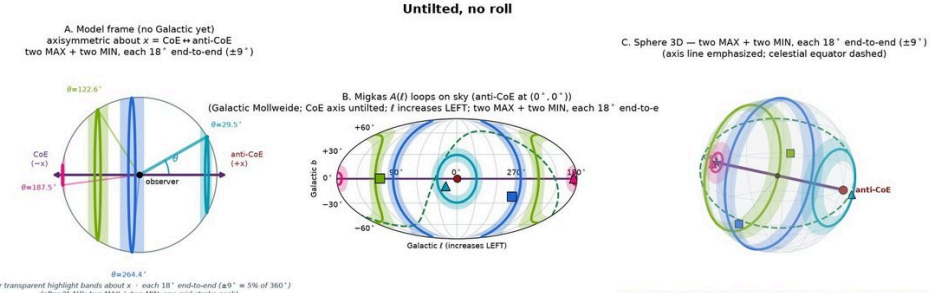
```
stack_3col_clean (1700, 3158)
stack_4col_clean (2493, 2876)
```

```
compare(FIGS / 'stack_3col_clean.jpg', '/workspace/cosmology/papers/spreadsheet_model/stack_3col_clean.jpg')
compare(FIGS / 'stack_4col_clean.jpg', '/workspace/cosmology/papers/spreadsheet_model/stack_4col_clean.jpg')
```

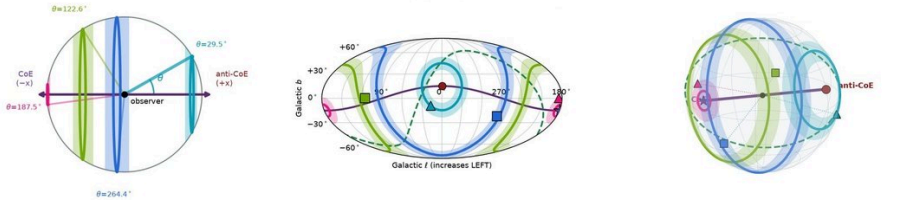
```
stack_3col_clean.jpg: new (1700, 3158) vs ref (1700, 3158) | mean |diff| = 0.000/255, pixels differing >40: 0.000% -> identical
left: approved reference papers/spreadsheet_model/stack_3col_clean.jpg   |   right: notebook re-run steps_v3/figs/stack_3col_clean.jpg
```



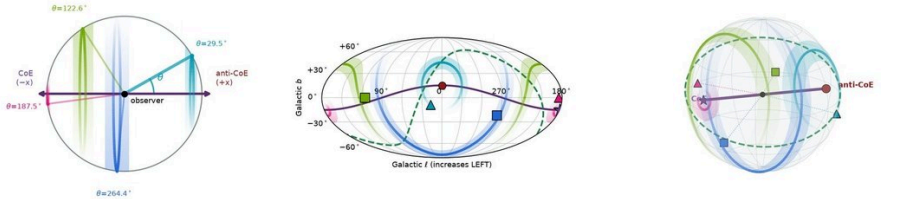
Untilted, no roll



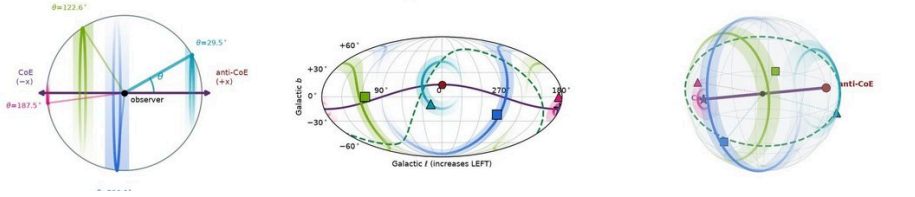
+15° tilt

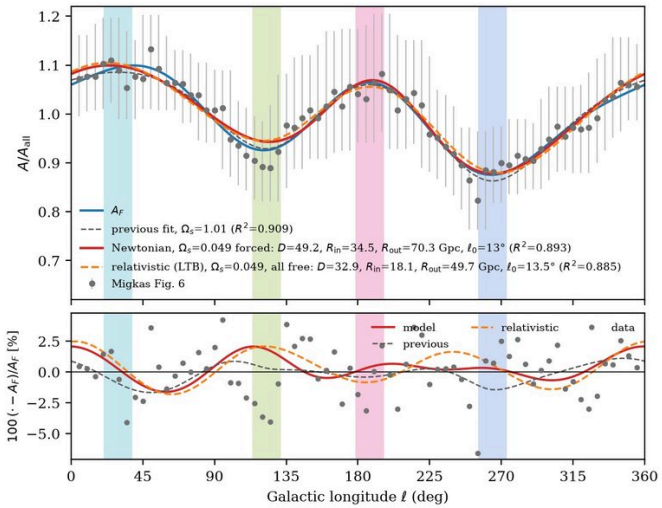


+15° tilt, gradient

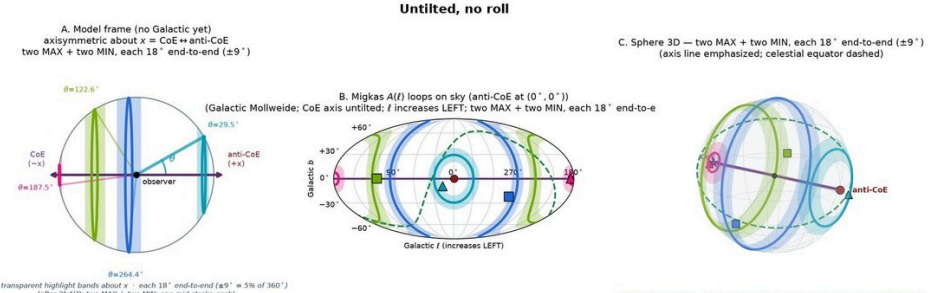


+15° tilt, gradient, +80.8° roll

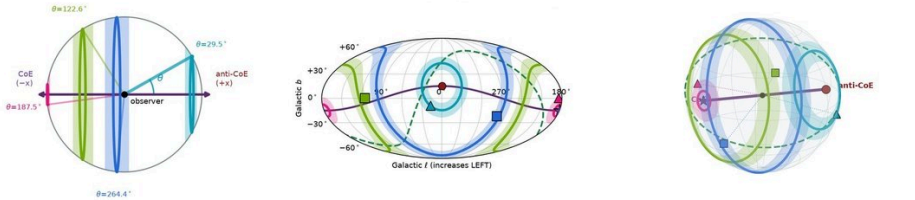




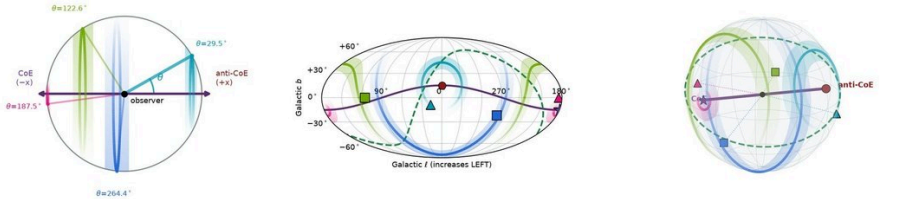
Untilted, no roll



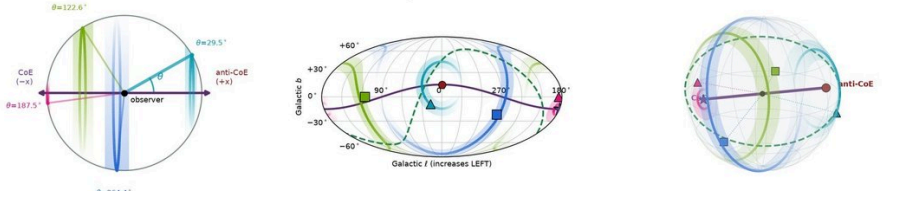
+15° tilt

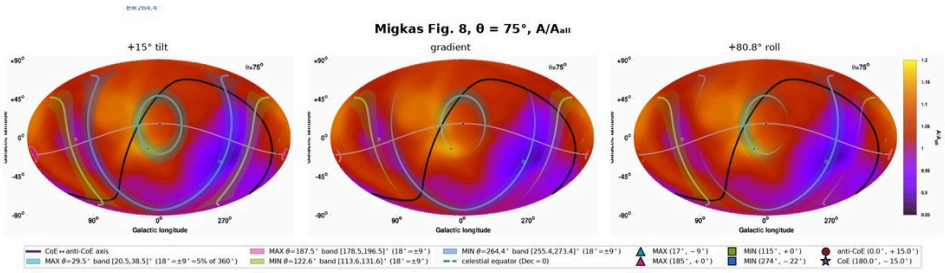


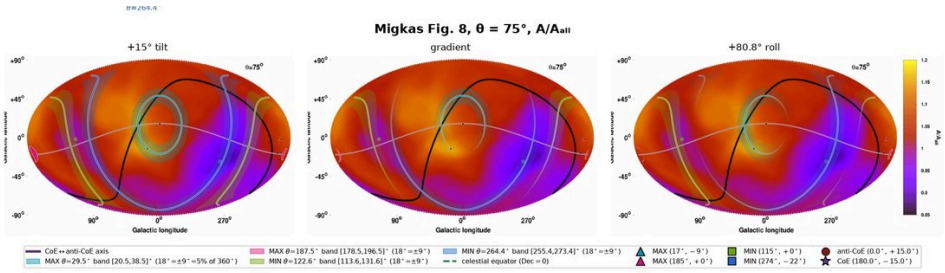
+15° tilt, gradient



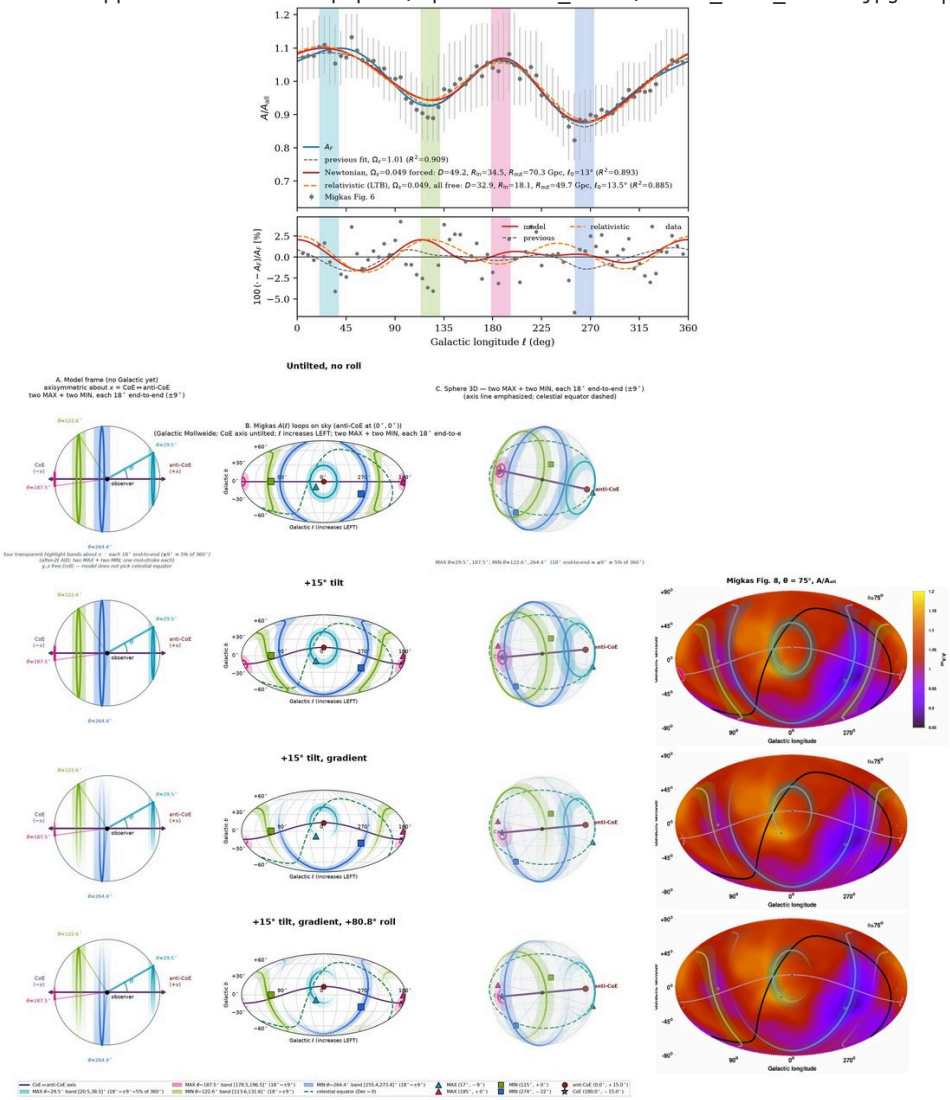
+15° tilt, gradient, +80.8° roll

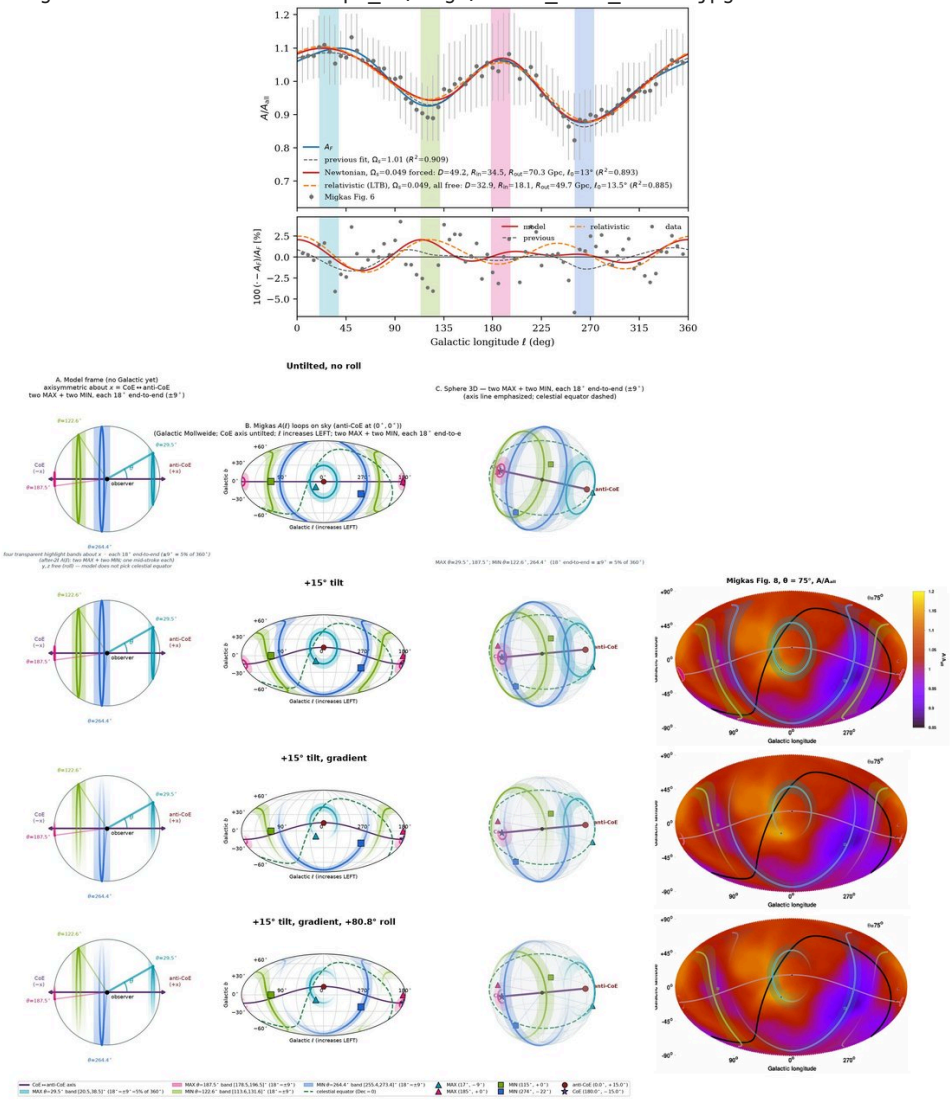






stack_4col_clean.jpg: new (2493, 2876) vs ref (2493, 2876) | mean |diff| = 0.000/255, pixels differing >40: 0.000% -> identical
left: approved reference papers/spreadsheet_model/stack_4col_clean.jpg | right: notebook re-run steps_v3/figs/stack_4col_clean.jpg





Step 23 — Thin-shell article, Figure 1 (model panels) and Figure 2 (A(angle) fits incl. relativistic 0.885)

What it shows. The two figures of `universe_bestfit_thinshell.pdf`, from its `.tex` source (`\includegraphics{universe_thin_drawing.png}` and `{universe_thin_Achart.png}`). *Figure 1*: three scales at the relativistic best fit. (a) Hubble velocity vectors: in a Sedov–Taylor blast wave (Taylor 1950; Sedov 1959) $\vec{V} = k\vec{r}$, so $\vec{V}_P - \vec{V}_O = k\vec{r}$ is Hubble's law and the CoE drops out. (b) The Hubble bubble to scale in Gpc: ordinary-matter shell ($\Omega_s = 0.049$) from $R_{\rm in} = 18.1$ to $R_{\rm out} = 49.7$ Gpc around an empty interior and the CoE core, the $D = 32.9$ Gpc circle through Earth, and the $R = 14.3$ Gpc vision sphere. (c) The cosmic shell with the core's Schwarzschild radius $r_s = 2GM_c/c^2$. *Figure 2*: Migkas points with error bars, A_F and the three model curves, normalised to the sky mean: Newtonian thin shell ($R^2 = 0.909$), Newtonian $\Omega_s = 0.049$ (0.893), and relativistic LTB (0.885). The lower panel shows percent differences from A_F. The script also rewrites the table file `universe_thin.json`; that copy is redirected to `steps_v3/figs/` too, so the data file on disk is untouched. Run from the `bubble/` folder, which holds its fit inputs.

Source script: `/workspace/cosmology/papers/spreadsheet_model/bubble/universe_thin.py` (last modified 2026-09-27 12:03 CT), run verbatim. *Output-path edit:* `open("universe_thin.json", "w")` → `open("/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/unive`; `../universe_thin_` → `/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/universe_th`; `open("../" + f + ".png")` → `open("/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/" + f`; `im.save("../" + f + "_web.jpg` → `im.save("/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/"`. *Reference image:* `papers/spreadsheet_model/universe_thin_drawing.png`, `papers/spreadsheet_model/universe_thin_Achart.png`.

```
%%runscript universe_thin.py --cwd /workspace/cosmology/papers/spreadsheet_model/bubble
import json, os, numpy as np, matplotlib
matplotlib.use("Agg"); import matplotlib.pyplot as plt
from matplotlib.patches import Circle, Wedge
from scipy.optimize import brentq
from PIL import Image
from bubble_core import AFfour, ML, MA, MLO, MHI, TH1, H0, GSI, RKM, GPC, IMIG, r2
import analysis_curve as AC
plt.rcParams.update({"font.size": 9, "font.family": "serif"})
c = 2.99792458e8; Msun = 1.989e30; Gm = GPC * 1e3
def phys(k):
    d = json.load(open("thin_%s.json" % k.replace("|", "_")))
    Dm = d["D_Gpc"] * Gm; rhoc = 3 * H0**2 / (8 * np.pi * GSI); rhos = d["Os"] * rhoc; Mcrit = 4 * np.pi / 3 * rhoc * Dm**3
    Mc = d["mcc"] * Mcrit; rin = d["rin"] * Dm; rout = d["rout"] * Dm
    Msh = lambda r: 4 * np.pi / 3 * rhos * (np.clip(r, rin, rout)**3 - rin**3)
    def Phi(r):
        if r < rin: ps = -GSI * 2 * np.pi * rhos * (rout**2 - rin**2)
        elif r < rout: ps = -GSI * (4 * np.pi / 3 * rhos * (r**3 - rin**3) / r + 2 * np.pi * rhos * (rout**2 - r**2))
        else: ps = -GSI * Msh(rout) / r
```

```
        return -GSI * Mc / r + ps
    v2 = d["vesc2"] * 2 * abs(Phi(Dm)); E = v2 / 2 + Phi(Dm)
    rt = brentq(lambda r: Phi(r) - E, Dm, 1e6 * Dm) if E < 0 else np.inf
    rs = lambda M: 2 * GSI * M / c**2 / Gm
    Mt = Mc + Msh(min(rt, 1e300))
    return dict(d=d, D=d["D_Gpc"], R=RKM / GPC, DR=d["DR"], Os=d["Os"], rhos=rhos, Mc=Mc, Mcrit=Mcrit, mcc=d["mcc"], Msh=Msh(Dm), Mtot=Mc + Msh(Dm),
                Mhole=4 * np.pi / 3 * rhos * rin**3, Rin=rin / Gm, Rout=rout / Gm, rs_c=rs(Mc), rs_D=rs(Mc + Msh(Dm)), rt=rt / Gm, rs_t=rs(Mt),
                vE=np.sqrt(v2) / c, vesc2=d["vesc2"], age=d["age"], t_turn=d["t_turn"], amp=d["amp"], phi=d["phi"], l0=d["l0"], R2=d["R2_mig"],
R2F=d["R2_four"],
                Lc=d["Lc"] * d["D_Gpc"], Dkm=d["DR"] * RKM, Lckm=d["Lc"] * d["DR"] * RKM)
P1, P2 = phys("b|free|hug"), phys("b|0.05|hug")
json.dump({k: {kk: vv for kk, vv in p.items() if kk != "d"} for k, p in (("free", P1), ("baryons", P2))},
open("/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/universe_thin.json", "w"), indent=1, default=float)
for p in (P1, P2): print({k: (round(v, 4) if isinstance(v, float) and abs(v) < 1e6 else v) for k, v in p.items() if k != "d"})
# ---- relativistic (exact LTB) thick ordinary-matter shell, z = 0.3 case (gr_ltb/refit_mass) ----
def rel():
    o = json.load(open("universe_rel_mcfree.json"))   # all free: D, Mc, R_in, R_out, l0, Amp, phi (old Mc-constrained fit: best_rows_thick.json)
    D = o["D"]; Dm = D * Gm; rhoc = 3 * H0**2 / (8 * np.pi * GSI); rhos = o["Os"] * rhoc
    rs = lambda M: 2 * GSI * M / c**2 / Gm; Rv = RKM / GPC; Rin, Rout = o["R_in"], o["R_out"]
    cur = np.array(o["A"])
    return dict(D=D, R=Rv, DR=D / Rv, Os=o["Os"], rhos=rhos, Mc=o["Mc_kg"], Mcrit=4 * np.pi / 3 * rhoc * Dm**3, mcc=o["Mc_kg"] / (4 * np.pi / 3 * rhoc
* Dm**3),
                Msh=o["M_D_kg"] - o["Mc_kg"], Mtot=o["M_D_kg"], Mshtot=4 * np.pi / 3 * rhos * ((Rout * Gm)**3 - (Rin * Gm)**3), Mhole=4 * np.pi / 3 *
rhos * (Rin * Gm)**3,
                Mvis=4 * np.pi / 3 * rhos * (Rv * Gm)**3, Rin=Rin, Rout=Rout, rs_c=rs(o["Mc_kg"]), rs_D=rs(o["M_D_kg"]), vE=H0 * Dm / c,
twoE=o["twoE"],
                age=o["age_Gyr"], amp=o["amp"], phi=o["phi"], l0=o["l0_s"], R2=o["R2_s"], R2ns=o["R2_ns"], R2F=r2(AFfour(TH1), cur), H_r=o["H_r"],
H_perp=o["H_perp"],
                z=o["z"], Lc=0.005 * D, Dkm=D * GPC, Lckm=0.005 * D * GPC, cur=cur)
P3 = rel()
def newt_omb():   # Newtonian, Omega_s = 0.049 forced (newt_omb.py)
    o = json.load(open("universe_newt_omb.json"))["Rs14.3|sine"]; D = o["D"]; Dm = D * Gm; rhoc = 3 * H0**2 / (8 * np.pi * GSI); rhos = o["Os"] * rhoc
    rs = lambda M: 2 * GSI * M / c**2 / Gm; Rv = RKM / GPC; cur = np.array(o["A"])
    return dict(D=D, R=Rv, DR=D / Rv, Os=o["Os"], rhos=rhos, Mc=o["Mc_kg"], Mcrit=o["Mc_kg"] / o["mcc"], mcc=o["mcc"], Msh=o["shell_inside_D_kg"],
Mtot=o["M_inside_D_kg"],
                Mshtot=o["shell_total_kg"], Mhole=4 * np.pi / 3 * rhos * (o["R_in"] * Gm)**3, Mvis=4 * np.pi / 3 * rhos * (Rv * Gm)**3, Rin=o["R_in"],
Rout=o["R_out"],
                rs_c=rs(o["Mc_kg"]), rs_D=rs(o["M_inside_D_kg"]), vE=o["VO_over_c"], vesc2=o["vesc2_full_potential"], age=o["age_Gyr"], amp=o["amp"],
phi=o["phi"], l0=o["l0"],
                R2=o["R2"], R2geo=json.load(open("universe_newt_omb.json"))["Rs14.3|geom"]["R2"], R2F=r2(AFfour(TH1), cur), Lc=0.005 * D, Dkm=D * GPC,
Lckm=0.005 * D * GPC, cur=cur)
P4 = newt_omb()
for q in (P1, P2): q["Mshtot"] = 4 * np.pi / 3 * q["rhos"] * ((q["Rout"] * Gm)**3 - (q["Rin"] * Gm)**3); q["Mvis"] = 4 * np.pi / 3 * q["rhos"] *
(q["R"] * Gm)**3
json.dump({k: {kk: vv for kk, vv in q.items() if kk not in ("d", "cur")} for k, q in (("free", P1), ("baryons", P2), ("rel", P3), ("newt_omb", P4))},
open("/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/universe_thin.json", "w"), indent=1, default=float)
print("REL", {k: (round(v, 4) if isinstance(v, float) and abs(v) < 1e6 else v) for k, v in P3.items() if k != "cur"})
p = P3; Dg, Rg, l0 = p["D"], p["R"], p["l0"]
lpk = (90 - np.degrees(p["phi"]) + l0) % 360
Ev = np.array([Dg, 0.0])
def sky(l, r): t = np.radians(l - l0); return Ev + r * np.array([np.cos(t), np.sin(t)])
# ---------------- Figure 1: one figure, three axes, unified style ----------------
from matplotlib.gridspec import GridSpec
from itertools import combinations
```

```
FS_A, FS_L, FS_T = 7.5, 8.5, 9            # annotations, axis labels/ticks, titles
plt.rcParams.update({"mathtext.fontset": "cm", "axes.linewidth": 0.8, "xtick.labelsize": 8, "ytick.labelsize": 8, "axes.labelsize": FS_L,
"axes.titlesize": FS_T,
                     "xtick.major.width": 0.8, "ytick.major.width": 0.8})
CRED, CBLU, CDC, CGRN, CPUR, CSH = "#d62728", "#0055d4", "#1f77b4", "#2ca02c", "#9467bd", "0.82"
VEL = dict(arrowstyle="-|>", color=CBLU, lw=1.3, mutation_scale=9, shrinkA=0, shrinkB=0)
POS = lambda ls="-": dict(arrowstyle="-|>", color="0.2", lw=0.9, ls=ls, mutation_scale=8, shrinkA=0, shrinkB=0)
exA = (-0.6, 5.6, -2.6, 3.4); exB = (-7, 82.7, -34.5, 34.5); E2 = 230
ka = (exA[1] - exA[0]) / (exA[3] - exA[2]); kb = (exB[1] - exB[0]) / (exB[3] - exB[2])
FW = 7.4; L0, G1, G2, R0 = 0.04, 0.58, 0.54, 0.10
H = (FW - L0 - G1 - G2 - R0) / (ka + kb + 1); BOT, TOP = 0.42, 0.24; FH = H + BOT + TOP
fig = plt.figure(figsize=(FW, FH))
gs = GridSpec(1, 5, figure=fig, width_ratios=[ka, G1 / H, kb, G2 / H, 1], wspace=0, left=L0 / FW, right=1 - R0 / FW, bottom=BOT / FH, top=1 - TOP /
FH)
vx, ax, bx = fig.add_subplot(gs[0]), fig.add_subplot(gs[2]), fig.add_subplot(gs[4])
def T(axx, *a, **k): k.setdefault("fontsize", FS_A); return axx.text(*a, **k)
# (a) Hubble velocity vectors (schematic)
C0_, O_, P_ = np.array([0, 0.9]), np.array([2.6, 0.9]), np.array([3.6, 2.3]); h_ = 0.4
VO, VP = h_ * (O_ - C0_), h_ * (P_ - C0_); VH = VP - VO
vx.annotate("", xy=O_, xytext=C0_, arrowprops=POS()); vx.annotate("", xy=P_, xytext=C0_, arrowprops=POS("--")); vx.annotate("", xy=P_, xytext=O_,
arrowprops=POS(":"))
for base, vec in ((O_, VO), (P_, VP), (P_, VH)): vx.annotate("", xy=base + vec, xytext=base, arrowprops=VEL)
vx.plot(*C0_, "o", color=CRED, ms=5, zorder=6); vx.plot(*O_, "o", color=CBLU, mec="k", mew=0.6, ms=5, zorder=6); vx.plot(*P_, "o", color="0.3",
ms=4.5, zorder=6)
T(vx, *(C0_ + [0, -0.18]), "CoE", color=CRED, ha="center", va="top")
T(vx, *(O_ + [0.02, -0.18]), "us ($O$)", ha="left", va="top")
T(vx, *(P_ + [0.1, -0.1]), "galaxy", ha="left", va="top")
T(vx, *((C0_ + O_) / 2 + [0, -0.12]), "$\\vec D$", ha="center", va="top")
T(vx, *((C0_ + P_) / 2 + [-0.12, 0.12]), "$\\vec D+\\vec r$", ha="right", va="bottom")
T(vx, *((O_ + P_) / 2 + [0.1, -0.05]), "$\\vec r$", ha="left", va="center")
T(vx, *(O_ + VO + [0.08, 0]), "$\\vec V_O$", color=CBLU, ha="left", va="bottom")
T(vx, *(P_ + VP + [0.04, -0.06]), "$\\vec V_P$", color=CBLU, ha="left", va="top")
T(vx, *(P_ + VH + [0.07, -0.02]), "$\\vec V_H$", color=CBLU, ha="left", va="bottom")
A_ = np.array([1.1, -1.45]); s_ = 1.6
for base, vec in ((A_, s_ * VO), (A_, s_ * VP), (A_ + s_ * VO, s_ * VH)): vx.annotate("", xy=base + vec, xytext=base, arrowprops=VEL)
T(vx, *(A_ + s_ * VO / 2 + [0, -0.1]), "$\\vec V_O$", color=CBLU, ha="center", va="top")
T(vx, *(A_ + s_ * VP / 2 + [-0.1, 0.1]), "$\\vec V_P$", color=CBLU, ha="right", va="bottom")
T(vx, *(A_ + s_ * VO + s_ * VH / 2 + [0.1, 0]), "$\\vec V_H$", color=CBLU, ha="left", va="center")
T(vx, 2.5, -2.55, "$\\vec V_H=\\vec V_P-\\vec V_O=H\\vec r$", ha="center", va="bottom")
vx.set_xlim(exA[0], exA[1]); vx.set_ylim(exA[2], exA[3]); vx.set_aspect("equal"); vx.set_xticks([]); vx.set_yticks([])
vx.set_title("(a) Hubble velocity vectors")
# (b) Hubble bubble scale
ax.add_patch(Circle((0, 0), p["Rout"], fc=CSH, ec="0.35", lw=0.8, zorder=0)); ax.add_patch(Circle((0, 0), p["Rin"], fc="white", ec="0.35", lw=0.8,
zorder=0.5))
ax.add_patch(Circle((0, 0), Dg, fill=False, color=CDC, lw=1.0, ls="--", zorder=2))
ax.add_patch(Circle(Ev, Rg, fc=CGRN, alpha=0.18, ec=CGRN, lw=1.2, zorder=2))
ax.add_patch(Circle((0, 0), p["rs_c"], fill=False, color=CRED, lw=1.4, zorder=3))
ax.plot(0, 0, "o", color=CRED, ms=5, zorder=6); T(ax, -5.5, 2.4, "CoE core", color=CRED, ha="left", va="bottom")
ax.plot(*Ev, "o", color=CBLU, mec="k", mew=0.6, ms=5, zorder=7); T(ax, p["Rin"] - 1.4, -1.0, "Earth", ha="right", va="top", zorder=7)
T(ax, 5.5, 11.7, "empty\ninterior", color="0.4", ha="center", va="center", linespacing=1.1)
T(ax, 0.5 * (Dg + p["Rout"]) * np.cos(np.radians(42)), 0.5 * (Dg + p["Rout"]) * np.sin(np.radians(42)), "shell", color="0.25", ha="center",
va="center", rotation=-48, rotation_mode="anchor")
tq = np.radians(125); q = Ev + Rg * np.array([np.cos(tq), np.sin(tq)])
ax.annotate("", xy=q, xytext=Ev, arrowprops=dict(arrowstyle="-|>", color=CGRN, lw=1.0, mutation_scale=8, shrinkA=0, shrinkB=0)); T(ax, (Ev[0] + q[0])
```

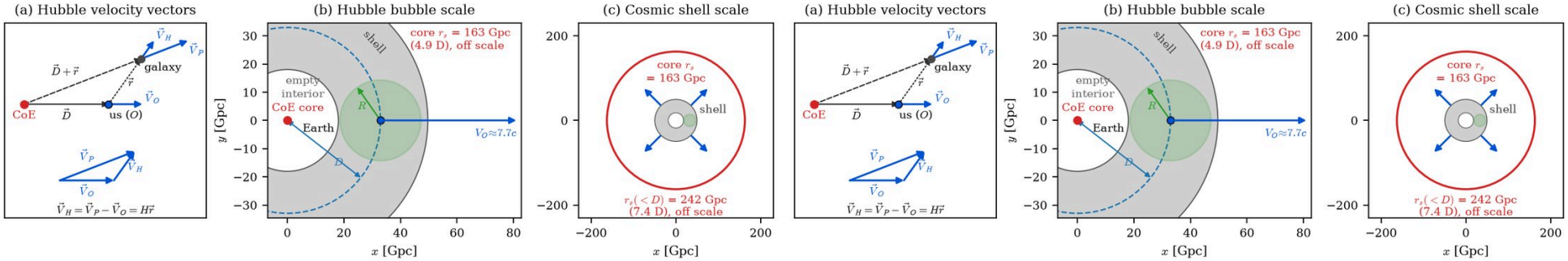
```
/ 2 - 1.0, (Ev[1] + q[1]) / 2 - 1.0, "$R$", color=CGRN, ha="right", va="center")
td = np.radians(-38); qd = Dg * np.array([np.cos(td), np.sin(td)])
ax.annotate("", xy=qd, xytext=(0, 0), arrowprops=dict(arrowstyle="<|-|>", color=CDC, lw=0.9, mutation_scale=7, shrinkA=0, shrinkB=0)); T(ax, *(qd *
0.58 + [1.4, -1.4]), "$D$", color=CDC, ha="left", va="top")
T(ax, 43.5, 33, "core $r_s$ = %.0f Gpc\n(%.1f D), off scale" % (p["rs_c"], p["rs_c"] / Dg), color=CRED, ha="left", va="top", linespacing=1.1)
VX1 = 81; ax.annotate("", xy=(VX1, 0), xytext=Ev, arrowprops=VEL, zorder=8)
vlab = T(ax, VX1, -2.8, "$V_O\\!\\approx\\!%.1fc$" % p["vE"], color=CBLU, ha="right", va="top")
ax.set_xlim(exB[0], exB[1]); ax.set_ylim(exB[2], exB[3]); ax.set_aspect("equal")
ax.set_xlabel("$x$ [Gpc]"); ax.set_ylabel("$y$ [Gpc]"); ax.set_title("(b) Hubble bubble scale")
# (c) Cosmic shell scale
bx.add_patch(Circle((0, 0), p["Rout"], fc=CSH, ec="0.35", lw=0.6, zorder=1)); bx.add_patch(Circle((0, 0), p["Rin"], fc="white", ec="0.35", lw=0.5,
zorder=1.5))
bx.add_patch(Circle((0, 0), p["rs_c"], fill=False, color=CRED, lw=1.4, zorder=3))
bx.add_patch(Circle(Ev, Rg, fc=CGRN, alpha=0.18, ec=CGRN, lw=1.2, zorder=2))   # vision sphere R, to scale (as in b)
T(bx, 0, 0.87 * p["rs_c"], "core $r_s$\n= %.0f Gpc" % p["rs_c"], color=CRED, ha="center", va="top", linespacing=1.1); T(bx, p["Rout"] + 6, 25,
"shell", color="0.25", ha="left", va="center")
T(bx, 0, -0.996 * E2, "$r_s(<D)$ = %s Gpc\n(%.1f D), off scale" % ("{:,.0f}".format(p["rs_D"]), p["rs_D"] / Dg), color=CRED, ha="center", va="bottom",
linespacing=1.0)
for ang in (45, 135, 225, 315):
    u = np.array([np.cos(np.radians(ang)), np.sin(np.radians(ang))]); bx.annotate("", xy=u * 0.87 * p["rs_c"] * 0.75, xytext=u * p["Rout"] * 1.05,
arrowprops=VEL, zorder=4)
bx.set_xlim(-E2, E2); bx.set_ylim(-E2, E2); bx.set_aspect("equal"); bx.set_xticks([-200, 0, 200]); bx.set_yticks([-200, 0, 200])
bx.set_xlabel("$x$ [Gpc]"); bx.set_title("(c) Cosmic shell scale")
# checks
fig.canvas.draw(); rr = fig.canvas.get_renderer()
for A_x in (vx, ax, bx):
    bbs = [(t.get_text()[:22], t.get_window_extent(rr)) for t in A_x.texts if t.get_text()]
    for (n1, b1), (n2, b2) in combinations(bbs, 2):
        if b1.overlaps(b2): print("OVERLAP", repr(n1), "|", repr(n2))
    W_ = A_x.get_window_extent(rr)
    for n1, b1 in bbs:
        if b1.x0 < W_.x0 or b1.x1 > W_.x1 or b1.y0 < W_.y0 or b1.y1 > W_.y1: print("OUTSIDE", repr(n1))
for A_x, circ, pad in ((ax, (p["Rin"], p["Rout"], p["rs_c"], Dg), 1.2), (bx, (p["Rin"], p["Rout"], p["rs_c"]), 17)):
    inv_ = A_x.transData.inverted()
    for t in A_x.texts:
        (x0, y0_), (x1, y1) = inv_.transform(t.get_window_extent(rr)); xs, ys = np.meshgrid(np.linspace(x0, x1, 60), np.linspace(y0_, y1, 20)); rr_ =
np.hypot(xs, ys)
        for R_ in circ:
            if rr_.min() - pad < R_ < rr_.max() + pad: print("EDGE", repr(t.get_text()[:20]), round(R_, 1), np.round([x0, y0_, x1, y1], 1))
print("axes heights (in)", [round(a_.get_window_extent(rr).height / fig.dpi, 3) for a_ in (vx, ax, bx)], "H", round(H, 3), "tight W",
round(fig.get_tightbbox(rr).width, 3))
fig.savefig("/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/universe_thin_drawing.png", dpi=200, bbox_inches="tight", pad_inches=0.02);
plt.close(fig)
plt.rcParams.update({"mathtext.fontset": "dejavusans", "axes.linewidth": 0.8, "xtick.labelsize": 9, "ytick.labelsize": 9, "axes.labelsize": 9,
"axes.titlesize": 12})
AH = float(os.environ.get("AH", "7.58"))
Af = AFfour(TH1); th2 = np.append(TH1, 360); Af2 = AFfour(th2)
fig, (a, b) = plt.subplots(2, 1, figsize=(6.6, AH), sharex=True, gridspec_kw={"height_ratios": [2.0, 1], "hspace": 0.06})
a.errorbar(ML, MA, yerr=[MLO, MHI], fmt="o", ms=3, color="0.45", ecolor="0.75", lw=0.8, label="Migkas Fig. 6")
a.plot(th2, Af2, color="C0", lw=1.5, label="$A_F$")
for pp, col, ls, lab in [(P1, "C3", "-", "Newtonian thin shell + core, $\\Omega_s$=%.2f"), (P4, "C2", ":", "Newtonian shell + core, $\\Omega_s$=%.3f
forced"), (P3, "C1", "--", "relativistic (LTB) shell + core, $\\Omega_s$=%.3f, $z$=0.3")]:
    A = AC.curve("ptshell", pp["d"]) if "d" in pp else pp["cur"]; assert abs(r2(MA, A[IMIG]) - pp["R2"]) < 2e-3, (r2(MA, A[IMIG]), pp["R2"])
    A2 = np.append(A, A[0])
```

```
    a.plot(th2, A2, color=col, ls=ls, lw=1.6 if ls != "--" else 1.3, label=(lab % pp["Os"]) + ", $\\ell_0$=%.1f° (fit $R^2$=%.3f; vs $A_F$ %.3f)" %
(pp["l0"], pp["R2"], pp["R2F"]))
    b.plot(th2, 100 * (A2 - Af2) / Af2, color=col, ls=ls, lw=1.4 if ls == "-" else 1.2, label={"-": "Newtonian", ":": "Newt. $\\Omega_s$=0.049", "--":
"relativistic"}[ls])
a.set_ylabel("$A/A_{\\rm all}$"); a.set_ylim(0.62, 1.22); a.legend(fontsize=7.2, loc="lower left", frameon=False)
b.axhline(0, color="k", lw=0.6); b.plot(ML, 100 * (MA - AFfour(ML)) / AFfour(ML), "o", ms=2.5, color="0.45", label="data")
b.set_ylabel("$100\\,(\\cdot-A_F)/A_F$ [%]", fontsize=8); b.set_xlabel("Galactic longitude $\\ell$ (deg)")
b.set_xlim(0, 360); b.set_xticks(range(0, 361, 45)); b.legend(fontsize=7, frameon=False, loc="upper right", ncol=4)
fig.savefig("/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/universe_thin_Achart.png", dpi=200, bbox_inches="tight")
for f in ["universe_thin_drawing", "universe_thin_Achart"]:
    im = Image.open("/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/" + f + ".png").convert("RGB"); im.thumbnail((1700, 1700));
im.save("/workspace/cosmology/papers/A_equals_fourier/steps_v3/figs/" + f + "_web.jpg", quality=88)
```

```
{'D': 93.7201, 'R': 14.3005, 'DR': 6.5536, 'Os': 1.0124, 'rhos': 0.0, 'Mc': 2.472258139581755e+54, 'Mcrit': 9.324647886726921e+56, 'mcc': 0.0027,
'Msh': np.float64(3.695681787866017e+56), 'Mtot': np.float64(3.7204043692618345e+56), 'Mhole': 5.744951904153747e+56, 'Rin': 79.4197, 'Rout': 108.0206,
'rs_c': 118.9912, 'rs_D': np.float64(17906.5219), 'rt': 990.7581, 'rs_t': np.float64(42041.6443), 'vE': np.float64(19.685), 'vesc2': 0.9013, 'age':
13.8309, 't_turn': 54.5691, 'amp': -0.124, 'phi': 1.2644, 'l0': 12.8215, 'R2': 0.9088, 'R2F': 0.9867, 'Lc': 0.4686, 'Dkm': 2.8919225210400707e+24,
'Lckm': 1.4459612605200357e+22}
{'D': 49.685, 'R': 14.3005, 'DR': 3.4744, 'Os': 0.05, 'rhos': 0.0, 'Mc': 2.383318897221138e+53, 'Mcrit': 1.3893452409706232e+56, 'mcc': 0.0017, 'Msh':
np.float64(4.437473459712923e+54), 'Mtot': np.float64(4.675805349435037e+54), 'Mhole': 2.509252745140193e+54, 'Rin': 35.3845, 'Rout': 63.9855, 'rs_c':
11.4711, 'rs_D': np.float64(225.0492), 'rt': inf, 'rs_t': np.float64(604.8166), 'vE': np.float64(10.8627), 'vesc2': 10.5579, 'age': 13.8872, 't_turn':
inf, 'amp': -0.0622, 'phi': 0.9331, 'l0': 13.2482, 'R2': 0.8931, 'R2F': 0.9769, 'Lc': 0.2484, 'Dkm': 1.5331308837249658e+24, 'Lckm':
7.665654418624828e+21}
REL {'D': 32.9401, 'R': 14.3005, 'DR': 2.3034, 'Os': 0.049, 'rhos': 0.0, 'Mc': 3.3795154244053916e+54, 'Mcrit': 4.0486197275839253e+55, 'mcc': 0.0835,
'Msh': 1.65666926404884e+54, 'Mtot': 5.0361846884542316e+54, 'Mshtot': 6.480355778284555e+54, 'Mhole': 3.2704699040605176e+53, 'Mvis':
1.6232363858405436e+53, 'Rin': 18.0618, 'Rout': 49.6841, 'rs_c': 162.658, 'rs_D': 242.3945, 'vE': 7.6913, 'twoE': 45.1086, 'age': 12.9769, 'amp':
-0.0558, 'phi': 0.8475, 'l0': 13.5, 'R2': 0.8848, 'R2ns': 0.5696, 'R2F': np.float64(0.9632), 'H_r': 78.153, 'H_perp': 65.9235, 'z': 0.3, 'Lc': 0.1647,
'Dkm': 1.0164316238833838e+24, 'Lckm': 5.082158119416919e+21}
EDGE 'Earth' 18.1 [ 6.2 -4.9 16.7 -1. ]
EDGE 'empty\ninterior' 18.1 [-1.6  7.4 12.6 16. ]
EDGE '' 32.9 [25.1  0.  32.9 11.2]
EDGE '' 18.1 [  0.  -20.   25.5   0. ]
EDGE '' 32.9 [  0.  -20.   25.5   0. ]
EDGE 'core $r_s$ = 163 Gpc' 49.7 [43.5 24.4 79.1 33. ]
EDGE '' 49.7 [32.9 -0.9 80.2  0.9]
EDGE '' 32.9 [32.9 -0.9 80.2  0.9]
EDGE 'core $r_s$\n= 163 Gpc' 162.7 [-64.9  84.3  64.9 141.5]
EDGE 'shell' 49.7 [ 55.7  11.9 113.1  38.1]
EDGE '$r_s(<D)$ = 242 Gpc\n' 162.7 [-122.4 -229.1  122.4 -177.1]
EDGE '' 49.7 [36.9 36.9 71.5 71.5]
EDGE '' 49.7 [-71.5  36.9 -36.9  71.5]
EDGE '' 49.7 [-71.5 -71.5 -36.9 -36.9]
EDGE '' 49.7 [ 36.9 -71.5  71.5 -36.9]
axes heights (in) [np.float64(1.842), np.float64(1.842), np.float64(1.842)] H 1.842 tight W 7.26
```

```
compare(FIGS / 'universe_thin_drawing.png', '/workspace/cosmology/papers/spreadsheet_model/universe_thin_drawing.png')
compare(FIGS / 'universe_thin_Achart.png', '/workspace/cosmology/papers/spreadsheet_model/universe_thin_Achart.png')
```

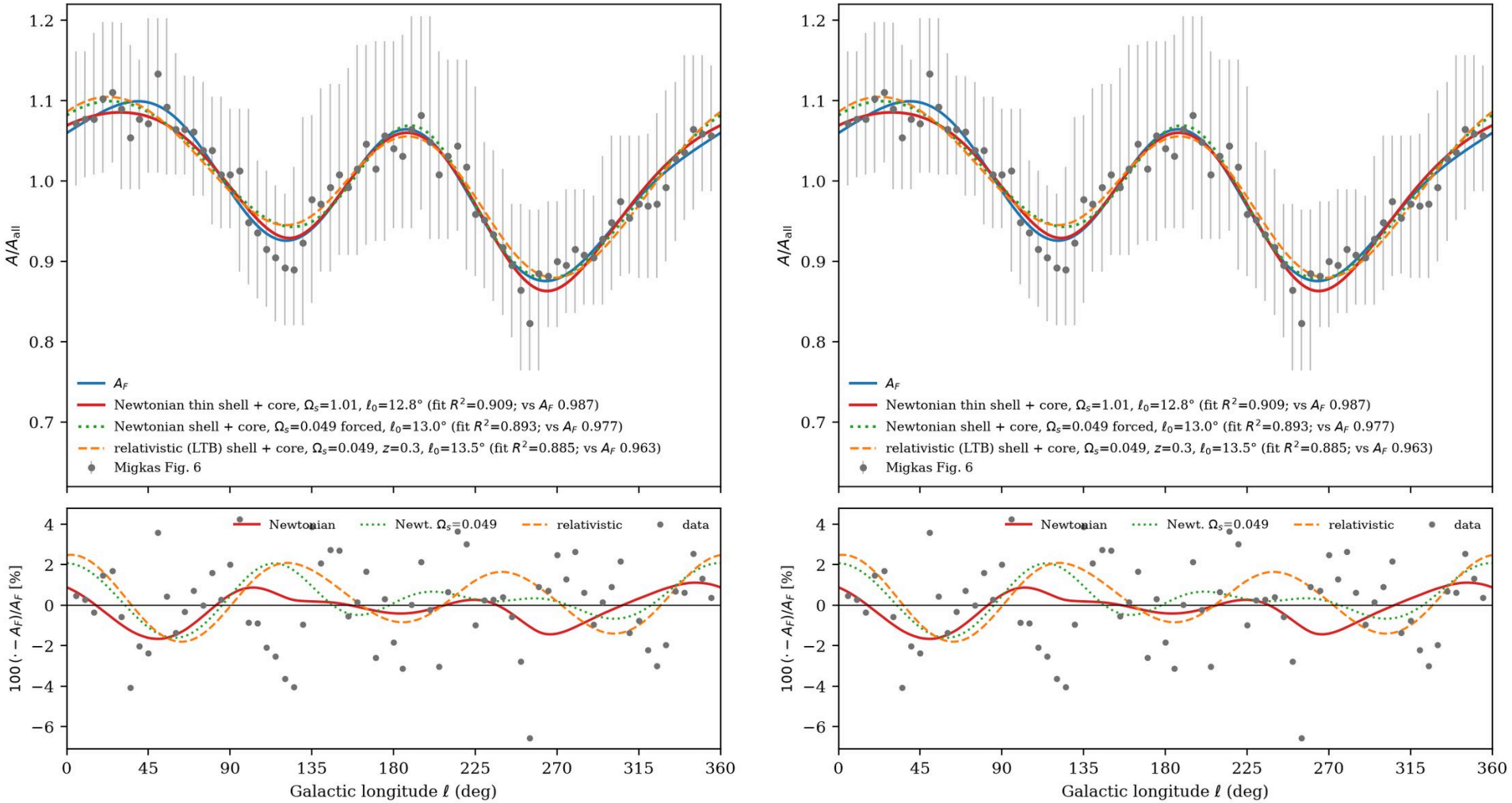
```
universe_thin_drawing.png: new (1460, 488) vs ref (1460, 488) | mean |diff| = 0.000/255, pixels differing >40: 0.000% -> identical
left: approved reference papers/spreadsheet_model/universe_thin_drawing.png   |   right: notebook re-run steps_v3/figs/universe_thin_drawing.png
```



(a) Hubble velocity vectors
(b) Hubble bubble scale
(c) Cosmic shell scale
galaxy
CoE
us (O)
shell
empty interior
CoE core
Earth
core r_s = 163 Gpc
(4.9 D), off scale
$r_s(<D)$ = 242 Gpc
(7.4 D), off scale
$V_O \approx 7.7c$
$\vec{V}_H = \vec{V}_P - \vec{V}_O = H\vec{r}$
x [Gpc]
y [Gpc]

universe_thin_Achart.png: new (1182, 1287) vs ref (1182, 1287) | mean |diff| = 0.000/255, pixels differing >40: 0.000% -> identical

left: approved reference papers/spreadsheet_model/universe_thin_Achart.png | right: notebook re-run steps_v3/figs/universe_thin_Achart.png



A/A_{all}
A_F
Newtonian thin shell + core, Ω_s=1.01, ℓ_0=12.8° (fit R^2=0.909; vs A_F 0.987)
Newtonian shell + core, Ω_s=0.049 forced, ℓ_0=13.0° (fit R^2=0.893; vs A_F 0.977)
relativistic (LTB) shell + core, Ω_s=0.049, z=0.3, ℓ_0=13.5° (fit R^2=0.885; vs A_F 0.963)
Migkas Fig. 6
Newtonian
Newt. Ω_s=0.049
relativistic
data
$100\,(\cdot - A_F)/A_F$ [%]
Galactic longitude ℓ (deg)

```
# Part B summary - match check for every figure
from IPython.display import Markdown
t = "| figure | reference | size ref | size new | mean abs diff (/255) | pixels differing >40 | verdict |\n|---|---|---|---|---|---|---|\n"
t += "\n".join(f"| {m['figure']} | {m['reference'].replace('/workspace/cosmology/', '')} | {m['size_ref'][0]}x{m['size_ref'][1]} | {m['size_new']
[0]}x{m['size_new'][1]} | {m['mean_abs_diff']:.3f} | {100*m['frac_px_diff']:.3f}% | {m['verdict']} |" for m in MATCH)
display(Markdown(t))
```

figure	reference	size ref	size new	mean abs diff (/255)	pixels differing >40	verdict
migkas_A_extrema_table2_mollweide.png	figures/migkas_A_extrema_table2_mollweide.png	2079x2612	2079x2612	0.000	0.000%	identical
migkas_fig8_theta75_A_extrema_only.png	figures/migkas_fig8_theta75_A_extrema_only.png	1975x1486	1975x1486	0.000	0.000%	identical
coe_rings_migkas_pin_axis.png	figures/coe_rings_migkas_pin_axis.png	2283x1027	2283x1027	0.000	0.000%	identical
migkas_fig8_theta75_with_coe_rings.png	figures/migkas_fig8_theta75_with_coe_rings.png	1975x1515	1975x1515	0.000	0.000%	identical
coe_rings_half_loops_tilt15.png	figures/coe_rings_half_loops_tilt15.png	2272x1027	2272x1027	0.000	0.000%	identical
coe_rings_half_loops_tilt15_rot45ccw.png	figures/coe_rings_half_loops_tilt15_rot45ccw.png	2272x1027	2272x1027	0.000	0.000%	identical
migkas_fig8_half_loops_tilt15.png	figures/migkas_fig8_half_loops_tilt15.png	1975x1515	1975x1515	0.000	0.000%	identical
migkas_fig8_half_loops_tilt15_rot45ccw.png	figures/migkas_fig8_half_loops_tilt15_rot45ccw.png	1975x1515	1975x1515	0.000	0.000%	identical
coe_rings_gradient_loops_tilt15.png	figures/coe_rings_gradient_loops_tilt15.png	2334x1027	2334x1027	0.000	0.000%	identical
coe_rings_gradient_loops_tilt15_rot45ccw.png	figures/coe_rings_gradient_loops_tilt15_rot45ccw.png	2334x1027	2334x1027	0.000	0.000%	identical
migkas_fig8_gradient_loops_tilt15.png	figures/migkas_fig8_gradient_loops_tilt15.png	1975x1515	1975x1515	0.000	0.000%	identical
migkas_fig8_gradient_loops_tilt15_rot45ccw.png	figures/migkas_fig8_gradient_loops_tilt15_rot45ccw.png	1975x1515	1975x1515	0.000	0.000%	identical
loop_threepanel_matched.png	papers/spreadsheet_model/loop_threepanel_matched.png	2262x1027	2262x1027	0.000	0.000%	identical
loop_threepanel_matched_grad.png	papers/spreadsheet_model/loop_threepanel_matched_grad.png	2262x1027	2262x1027	0.000	0.000%	identical
loop_threepanel_matched_grad_rot45.png	papers/spreadsheet_model/loop_threepanel_matched_grad_rot45.png	2262x1027	2262x1027	0.000	0.000%	identical
loop_threepanel_matched_grad_rotfit_rot81.png	papers/spreadsheet_model/loop_threepanel_matched_grad_rotfit_rot81.png	2262x1027	2262x1027	0.000	0.000%	identical
loop_threepanel_notilt.png	papers/spreadsheet_model/loop_threepanel_notilt.png	2262x1027	2262x1027	0.000	0.000%	identical
fig8_overlay_matched.png	papers/spreadsheet_model/fig8_overlay_matched.png	1975x1515	1975x1515	0.000	0.000%	identical
fig8_overlay_matched_grad.png	papers/spreadsheet_model/fig8_overlay_matched_grad.png	1975x1515	1975x1515	0.000	0.000%	identical
fig8_overlay_matched_grad_rot45.png	papers/spreadsheet_model/fig8_overlay_matched_grad_rot45.png	1975x1515	1975x1515	0.000	0.000%	identical
fig8_overlay_matched_grad_rot81.png	papers/spreadsheet_model/fig8_overlay_matched_grad_rot81.png	1975x1515	1975x1515	0.000	0.000%	identical
loop_flow_thinshell.png	papers/spreadsheet_model/loop_flow_thinshell.png	2550x5010	2550x5010	0.000	0.000%	identical
loop_journal_page.png	papers/spreadsheet_model/loop_journal_page.png	2278x5713	2278x5713	0.000	0.000%	identical
loop_stack_roll45.png	papers/spreadsheet_model/loop_stack_roll45.png	2302x3333	2302x3333	0.000	0.000%	identical
loop_stack_roll80p8.png	papers/spreadsheet_model/loop_stack_roll80p8.png	2302x3333	2302x3333	0.000	0.000%	identical
stack_3col_clean.jpg	papers/spreadsheet_model/stack_3col_clean.jpg	1700x3158	1700x3158	0.000	0.000%	identical

figure	reference	size ref	size new	mean abs diff (/255)	pixels differing >40	verdict
stack_4col_clean.jpg	papers/spreadsheet_model/stack_4col_clean.jpg	2493x2876	2493x2876	0.000	0.000%	identical
universe_thin_drawing.png	papers/spreadsheet_model/universe_thin_drawing.png	1460x488	1460x488	0.000	0.000%	identical
universe_thin_Achart.png	papers/spreadsheet_model/universe_thin_Achart.png	1182x1287	1182x1287	0.000	0.000%	identical